
Katamarina Cozystack

Документация v1.6

Полная offline-версия документации · cozystack.ru/docs/v1.6/

Оглавление

Введение

Cozystack v1.6	7
Введение	9

Начало работы

Начало работы	12
Требования	14
1. Установка Talos	18
2. Установка Kubernetes	21
3. Установка Cozystack	22
4. Создание tenant'a	38
5. Развертывание приложений	43

Изучение Cozystack

Изучение Cozystack	52
Ключевые концепции	53
Стек платформы	63
Система tenant'ов	71
Управление ресурсами	76
Talos Linux	81
Сценарии использования	83
Public Cloud	84
Private Cloud	85
Build Your Own Platform	86

Развертывание Cozystack

Развертывание Cozystack	88
Требования к оборудованию	91
Планирование ресурсов	96
1. Установка Talos	98
boot-to-talos	100
PXE	105
ISO	108
2. Установка Kubernetes	109
Talm	111
talos-bootstrap	125
talosctl	130
Изолированная среда	137
Troubleshooting	146
Generic Kubernetes	149
Ubuntu + Secure Boot	161
3. Установка Cozystack	166
Как платформа	168
Собственная платформа	184
Руководства для провайдеров	189

Oracle Cloud (OCI)	190
Hetzner	199
Servers.com	209
Ansible	214
Практические руководства	221
Установка на один диск	222
Развертывание в публичных сетях	224
Включение KubeSpan	225
Настройка bonding (LACP)	226
Включение Hugepages	232

Эксплуатация

Эксплуатация	235
Обновление до v1.0	237
Конфигурация	245
Пакет платформы	246
Варианты	267
Компоненты	272
Лицензии	274
Вебхук Lineage Controller	283
Брендирование	286
Телеметрия	293
VPC и подсети	296
Обслуживание кластера	297
Обновление Cozystack	298
Масштабирование кластера	300
Ротация CA	303
Сервисы кластера	305
Объектное хранилище	307
Пулы хранения	309
Бакеты	317
Backup классы	322
BootBox	331
EtcD	332
Ingress	334
SeaweedFS	337
Мониторинг	344
Alerting	348
Custom Metrics	357
Dashboards	361
Logs	366
Parameters	372
Setup	380
OpenID Connect	391
Сервер OIDC	392
Пользователи и роли	396
Самоподписанные сертификаты	398
Провайдеры идентификации	403
GitLab	404
Google	406

Несколько ЦОД	408
Конфигурация etcd	409
Конфигурация kube-scheduler	411
Топологические метки	413
LINSTOR DRBD	415
Сеть хранилища	417
SeaweedFS в нескольких ЦОД	420
Несколько площадок	424
Autoscaling	425
Azure	426
Hetzner площадок	438
Local-CCM	449
Network Mesh	450
FAQ и инструкции	453
ServiceAccount и API	455
Генерация kubeconfig	457
Устранение неполадок	458
etcd	463
Flux CD	464
Мониторинг	465
Kube-OVN	470
LINSTOR: контроллер	475
LINSTOR: база данных	477
Piraeus: custom resources	487
Классы планирования	489
GPU container workloads	495

Managed Kubernetes

Managed Kubernetes	501
GPU Sharing	529
Relocate etcd replicas	534

Managed Applications

Managed Applications	537
Backup and Recovery	539
External Apps	545
ClickHouse	552
FoundationDB	560
Harbor Container Registry	567
Kafka	571
MariaDB	576
MongoDB	582
NATS	589
OpenBAO	592
PostgreSQL	594
Qdrant	612
RabbitMQ	615
Redis	618
Tenant	621

Виртуализация

Виртуализация	627
Виртуальная машина	631
Диск виртуальной машины	641
Golden Images	643
Клонлируемые ВМ	649
Проброс GPU	653
Резервное копирование	661
Windows ВМ	669
MikroTik RouterOS	674
Миграция из Proxmox	677
Справочник по ресурсам	682

Хранилище

Хранилище	689
Подготовка дисков	690
Выделенная сеть	696
Настройка DRBD	705
Использование NFS	706
LINSTOR GUI	708
Шифрованное хранилище	710

Сеть

Сеть	713
Архитектура	715
VPC	727
VPN	729
Gateway API	732
HTTP-кеш	759
PROXY-protocol и hairpin	764
Публикация K8s API в DNS	773
Балансировщик TCP	779
Виртуальные маршрутизаторы	782

Cozystack API

Cozystack API	785
Go Types	789
REST API	795
ApplicationDefinition	796

Прочее

Руководство по разработке	802
Рoadmap	812
Контрибуторы	813

Введение

Документация Cozystack v1.6

<https://cozystack.ru/docs/v1.6/>

Свободная платформа и фреймворк для построения облачных сред

Cozystack это свободная PaaS-платформа для построения облачных инфраструктур.

Cozystack позволяет легко трансформировать ваш парк серверов в интеллектуальную систему с помощью простого REST API для развертывания Kubernetes кластеров, виртуализации и контейнеризации, сервисов баз данных (DBaaS), виртуальных машин, балансировщиков нагрузки, HTTP-кэширования и других сервисов.

Используйте Cozystack для создания собственного облака как для продуктивной нагрузки, так и для предоставления недорогих сред разработки.

•

Введение в Cozystack

Что такое Cozystack и его возможности

Начало работы с Cozystack: развёртывание частного облака с нуля

Сделайте первые шаги, запустите домашнюю лабораторию и соберите POC на Cozystack.

Изучение Cozystack

Узнайте, как использовать Cozystack в роли администратора кластера и владельца tenant'a.

Руководство по развёртыванию Cozystack: от инфраструктуры до готового кластера

Узнайте, как развернуть кластер Cozystack с помощью Talos Linux и Kubernetes. В этом руководстве описаны установка, настройка и лучшие практики надежного и безопасного развёртывания Cozystack.

Руководство по настройке и управлению кластером

Настройка, мониторинг, защита и обновление кластера Cozystack.

Управляемый (tenant) Kubernetes

Как развернуть и использовать изолированные управляемые Kubernetes-кластеры в Cozystack.

Managed Applications: Guides and Reference

Learn how to deploy, configure, access, and backup managed applications in Cozystack.

[Возможности виртуализации в Cozystack](#)

Всё о развёртывании, настройке и использовании виртуальных машин в Cozystack.

[Руководства по подсистеме хранения](#)

Практические руководства по подсистеме хранения

[Сетевые возможности](#)

Настройка сети, виртуальные маршрутизаторы, балансировщики нагрузки и другие сетевые возможности Cozystack.

[Cozystack API](#)

Cozystack API для управления сервисами и ресурсами

[Cozystack Руководство по разработке](#)

Cozystack Руководство по разработке

[Cozystack Рoadmap](#)

Cozystack Рoadmap

[Контрибуторы](#)

Люди, которые помогают развивать документацию и проект Cozystack.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Введение в Cozystack

<https://cozystack.ru/docs/v1.6/introduction/>

Что такое Cozystack и его возможности

Что такое Cozystack

Cozystack — это фреймворк на базе Kubernetes для построения среды частного облака. Он может использоваться отдельной компанией для запуска собственного **частного облака** или поставщиком услуг для предоставления **платформы как услуги** нескольким клиентам.

Cozystack закрывает наиболее важные потребности команды разработчиков:

- **Кластеры Kubernetes** для запуска приложений в среде разработки и в продакшене
- Стандартные **управляемые приложения** : базы данных, менеджеры очередей, кэши и многое другое
- **Виртуальные машины**
- Надёжное распределённое хранилище

Платформенный стек Cozystack включает надёжные компоненты, которые обычно устанавливаются в кластеры Kubernetes по отдельности. Здесь они собраны вместе и протестированы для бесперебойной совместной работы. Платформа виртуализации также встроена и не требует дополнительного оборудования. Вместо этого виртуальные машины запускаются непосредственно внутри Kubernetes.

Ещё одна мощная функция — **система тенантов**. Она позволяет изолировать отдельных разработчиков, команды или даже целые компании в их собственных полнофункциональных пространствах — на одном и том же оборудовании.

Ключевые возможности

Многопользовательский режим, мультитенантность и встроенный SSO

Cozystack разработан для использования несколькими командами, отделами или даже компаниями. Традиционный подход с выделением каждой команде отдельного пространства имён может быть слишком ограниченным. Командам может потребоваться несколько окружений с одинаковыми именами пространств имён, или им может не хватать прав root для управления собственными моделями доступа.

Система тенантов Cozystack решает эти проблемы, позволяя пользователям разворачивать среду Kubernetes-в-Kubernetes с помощью одного приложения. Пользователи вложенных кластеров Kubernetes имеют полный доступ и контроль. Система квот обеспечивает оптимальное использование оборудования, изолируя ресурсы во избежание проблемы «шумного соседа».

Пользователи платформы также могут формировать подробные отчёты об использовании ресурсов для каждого тенанта.

Система единого входа в Cozystack работает на базе Keycloak. Kubernetes API — как в корневом тенанте, так и во вложенных — поддерживает SSO из коробки.

Реплицированная система хранения

Не все компании могут позволить себе выделенное аппаратное обеспечение SAN или NAS. Cozystack включает надёжную распределённую систему хранения, позволяющую создавать отказоустойчивые реплицированные тома. Возможна даже репликация томов между несколькими центрами обработки данных.

Система виртуализации

Обычно приходится выбирать между виртуализацией и контейнеризацией. Cozystack объединяет оба подхода в единой платформе. Нет необходимости поддерживать отдельную инфраструктуру виртуализации. В Cozystack **виртуальные машины** запускаются непосредственно в Kubernetes и потребляют CPU, память, GPU и хранилище из общего пула ресурсов Kubernetes.

Управляемые базы данных без накладных расходов

Несмотря на высокую эффективность виртуализации Linux-на-Linux, она всё же вносит определённые накладные расходы. Cozystack избегает этого, запуская **управляемые базы данных** непосредственно в контейнерах на хостовом оборудовании. Можно развернуть несколько высокодоступных баз данных с выделенными IP-адресами на ограниченном оборудовании — при этом каждая имеет прямой доступ к CPU и хранилищу.

Экосистема Kubernetes

Нам неизвестен ни один другой дистрибутив Kubernetes с таким количеством встроенных инфраструктурных компонентов. (Серьёзно — пришлите нам ссылку, если найдёте!) Вместо того чтобы устанавливать компоненты и контроллеры вручную, вы просто выбираете **вариант Cozystack**, подходящий для ваших нужд. Все компоненты предварительно настроены, проверены на совместимость и обновляются вместе с фреймворком Cozystack.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Начало работы

Начало работы с Cozystack: развёртывание частного облака с нуля

<https://cozystack.ru/docs/v1.6/getting-started/>

Сделайте первые шаги, запустите домашнюю лабораторию и соберите POC на Cozystack.

Это руководство поможет вам выполнить первое развёртывание кластера Cozystack. По ходу работы вы познакомитесь с ключевыми концепциями, научитесь пользоваться Cozystack через дашборд и CLI и получите рабочий proof-of-concept.

Руководство разделено на несколько шагов. Завершайте каждый шаг перед переходом к следующему:

Шаг	Описание
Требования: подготовьте инфраструктуру и инструменты	Подготовьте инфраструктуру и установите необходимые CLI-инструменты на свою машину перед началом этого руководства.
1\.. Установите Talos Linux	Установите дистрибутив Talos Linux, подготовленный для Cozystack, с помощью [boot-to-talos](https://github.com/cozystack/boot-to-talos) — это, вероятно, самый простой способ установки.
2\.. Установите и инициализируйте кластер Kubernetes	Выполните bootstrap кластера Kubernetes с помощью Talm — инструмента управления конфигурацией Talos, созданного для Cozystack.
3\.. Установите и настройте Cozystack	Установите Cozystack, получите административный доступ, выполните базовую настройку и откройте дашборд Cozystack.
4\.. Создайте tenant для пользователей и команд	Создайте пользовательский tenant — основу RBAC в Cozystack — и получите доступ к нему через дашборд и Cozystack API.
5\.. Разверните управляемые приложения	Начните работать с Cozystack: разверните виртуальную машину, управляемое приложение и Kubernetes-кластер tenant'a.

•

Требования и набор инструментов

Подготовьте инфраструктуру и установите необходимые инструменты.

1\ Установка Talos Linux

Установка Talos Linux на любую машину с помощью cozystack/boot-to-talos.

2\ Установка и bootstrap кластера Kubernetes

Используйте Talm CLI, чтобы выполнить bootstrap кластера Kubernetes, готового к установке Cozystack.

3\ Установка и настройка Cozystack

Установите Cozystack, получите административный доступ, выполните базовую настройку и включите UI-дашборд.

4\ Создание пользовательского tenant'a и настройка доступа

Создайте пользовательский tenant — основу RBAC в Cozystack — и получите доступ к нему через дашборд и Cozystack API.

5\ Развёртывание управляемых приложений, VM и Kubernetes-кластера tenant'a

Начните использовать Cozystack: разверните виртуальную машину, управляемое приложение и Kubernetes-кластер tenant'a.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Требования и набор инструментов

<https://cozystack.ru/docs/v1.6/getting-started/requirements/>

Подготовьте инфраструктуру и установите необходимые инструменты.

Набор инструментов

На вашей рабочей станции должны быть установлены следующие инструменты:

- **talosctl**, клиент командной строки для Talos Linux (используйте серию v1.13.x, соответствующую Cozystack 1.6.0).
- **kubectl**, клиент командной строки для Kubernetes.
- **Talm**, собственный менеджер конфигурации Talos Linux от Cozystack:

```
curl -sSL https://github.com/cozystack/talm/raw/refs/heads/main/hack/install.sh
```

```
sh -s
```

Требования к оборудованию

Для прохождения этого руководства вам потребуется следующая конфигурация:

Узлы кластера: три bare-metal сервера или виртуальные машины. Требования к оборудованию зависят от вашего сценария использования:

- Минимальная
- Рекомендуемая
- Оптимальная

Ниже приведены базовые требования для запуска небольшой инсталляции. Минимальная рекомендуемая конфигурация для каждого узла:

Компонент	Требование
Хосты	3x физических хоста (или VM с host CPU passthrough)
Architecture	x86_64
CPU	8 ядер
RAM	24 ГБ
Основной диск	50 ГБ SSD (или RAW для VM)

Дополнительный диск	256 ГБ SSD (raw)
---------------------	------------------

Подходит для:

- Dev/Test-сред
- Небольших демонстрационных конфигураций
- 1-2 tenants
- До 3 кластеров Kubernetes
- Небольшого числа VM или баз данных

Для небольших production-сред рекомендуемая конфигурация каждого узла выглядит так:

Компонент	Требование
Хосты	3х физических хоста
Architecture	x86_64
CPU	16-32 ядра
RAM	64 ГБ
Основной диск	100 ГБ SSD или NVMe
Дополнительный диск	1-2 ТБ SSD или NVMe

Подходит для:

- Небольших и средних production-сред
- 5-10 tenants
- 5+ кластеров Kubernetes
- Десятков виртуальных машин или баз данных
- S3-совместимого хранилища

Для средних и крупных production-сред оптимальная конфигурация каждого узла выглядит так:

Компонент	Требование
Хосты	6х+ физических хостов
Architecture	x86_64
CPU	32-64 ядра
RAM	128-256 ГБ
Основной диск	200 ГБ SSD или NVMe
Дополнительный диск	4-10 ТБ NVMe

Подходит для:

-
- Крупных production-сред
 - 20+ tenants
 - Десятков кластеров Kubernetes
 - Сотен виртуальных машин и баз данных
 - S3-совместимого хранилища

Хранилище:

- Основной диск : используется для Talos Linux, хранилища etcd и загруженных образов. Требуется низкая задержка.
- Дополнительный диск : используется для данных пользовательских приложений (ZFS pool).

ОС:

- Любой дистрибутив Linux, например Ubuntu.
- Есть и [другие способы установки](#), для которых на старте требуется либо любой Linux, либо вообще не требуется установленная ОС.

BIOS/UEFI Settings:

- Secure Boot.

Talos Linux ships pre-signed kernel modules and works with Secure Boot enabled. On non-Talos Ubuntu hosts, the default piraeus-operator flow compiles DRBD in-cluster; the resulting unsigned modules are rejected by kernel lockdown when Secure Boot is enforced. The simplest path is to disable Secure Boot in BIOS/UEFI; alternatively, follow [Ubuntu + Secure Boot](#) to pre-install dkms-signed DRBD on the host.

Сеть:

- Маршрутизируемый домен с FQDN.

Если его нет, можно использовать [nip.io](#) с dash-нотацией.

- Узлы должны находиться в одном L2-сегменте сети.
- Anti-spoofing должен быть отключён.

Это необходимо для MetalLB — балансировщика нагрузки, используемого в Cozystack.

Виртуальные машины:

- В настройках гипервизора должен быть включён CPU passthrough, а модель CPU должна быть установлена в `host`.
- Должна быть включена nested virtualization.

Это требуется для виртуальных машин и Kubernetes-кластеров tenant'ов.

Более подробное описание требований к оборудованию для разных сценариев см. в разделе [Требования к оборудованию](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

1. Установка Talos Linux

<https://cozystack.ru/docs/v1.6/getting-started/install-talos/>

Установка Talos Linux на любую машину с помощью `cozystack/boot-to-talos`.

Перед началом

Убедитесь, что у вас есть узлы (bare-metal серверы или виртуальные машины), соответствующие [требованиям к оборудованию](#).

Цели

На этом шаге вы установите Talos Linux на bare-metal серверы или виртуальные машины, на которых сейчас работает другой дистрибутив Linux.

В этом руководстве используется `boot-to-talos` — простой CLI-инструмент, созданный командой Cozystack для пользователей и команд, внедряющих Cozystack. Существуют и другие способы [установки Talos Linux для Cozystack](#), но они здесь не рассматриваются и описаны в отдельных руководствах.

Установка

1\ Установите `boot-to-talos`

Установите `boot-to-talos` с помощью установочного скрипта:

<code>curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh</code>	<code>sh -s</code>
---	--------------------

2\ Запустите установку Talos

Запустите `boot-to-talos` и укажите значения конфигурации. Убедитесь, что используете сборку Talos от Cozystack, опубликованную в ghcr.io/cozystack/cozystack/talos. Для Cozystack v1.6.0 закреплённая версия Talos — v1.13.6. Переопределите значение по умолчанию в installer prompt:

```
$ boot-to-talos
```

```
Target disk [/dev/sda]:
```

```
Talos installer image [ghcr.io/cozystack/cozystack/talos:v1.11.6]:  
ghcr.io/cozystack/cozystack/talos:v1.13.6
```

```
Add networking configuration? [yes]:
```

Interface [eth0]:

IP address [10.0.2.15]:

Netmask [255.255.255.0]:

Gateway (or 'none') [10.0.2.2]:

Configure serial console? (or 'no') [ttyS0]:

Summary:

Image: ghcr.io/cozystack/cozystack/talos:v1.13.6

Disk: /dev/sda

Extra kernel args: ip=10.0.2.15::10.0.2.2:255.255.255.0::eth0::: console=ttyS0

WARNING: ALL DATA ON /dev/sda WILL BE ERASED!

Continue? [yes]:

2025/08/03 00:11:03 created temporary directory /tmp/installer-3221603450

2025/08/03 00:11:03 pulling image ghcr.io/cozystack/cozystack/talos:v1.13.6

2025/08/03 00:11:03 extracting image layers

2025/08/03 00:11:07 creating raw disk /tmp/installer-3221603450/image.raw (2 GiB)

2025/08/03 00:11:07 attached /tmp/installer-3221603450/image.raw to /dev/loop0

2025/08/03 00:11:07 starting Talos installer

2025/08/03 00:11:07 running Talos installer v1.13.6

2025/08/03 00:11:07 WARNING: config validation:

2025/08/03 00:11:07 use "worker" instead of "" for machine type

2025/08/03 00:11:07 created EFI (C12A7328-F81F-11D2-BA4B-00A0C93EC93B) size 104857600 bytes

2025/08/03 00:11:07 created BIOS (21686148-6449-6E6F-744E-656564454649) size 1048576 bytes

2025/08/03 00:11:07 created BOOT (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576000 bytes

2025/08/03 00:11:07 created META (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576 bytes

2025/08/03 00:11:07 formatting the partition "/dev/loop0p1" as "vfat" with label "EFI"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p2" as "zeroes" with label "BIOS"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p3" as "xfs" with label "BOOT"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p4" as "zeroes" with label "META"

2025/08/03 00:11:07 copying from io reader to /boot/A/vmlinuz

2025/08/03 00:11:07 copying from io reader to /boot/A/initramfs.xz

2025/08/03 00:11:08 writing /boot/grub/grub.cfg to disk

2025/08/03 00:11:08 executing: grub-install --boot-directory=/boot --removable --efi-directory=/boot/EFI /dev/loop0

2025/08/03 00:11:08 installation of v1.13.6 complete

```
2025/08/03 00:11:08 Talos installer finished successfully
2025/08/03 00:11:08 remounting all filesystems read-only
2025/08/03 00:11:08 copy /tmp/installer-3221603450/image.raw -> /dev/sda
2025/08/03 00:11:19 installation image copied to /dev/sda
2025/08/03 00:11:19 rebooting system
```

Следующий шаг

Продолжите руководство по Cozystack и [установите Kubernetes-кластер с помощью Talm](#).

Дополнительно:

- Прочитайте [обзор Talos Linux](#), чтобы понять, почему Talos Linux — оптимальный выбор ОС для Cozystack и что он даёт платформе.
- Узнайте больше о [\[boot-to-talos\]](https://cozystack.ru/docs/v1.6/install/talos/boot-to-talos/#about-the-application)(<https://cozystack.ru/docs/v1.6/install/talos/boot-to-talos/#about-the-application>).
- Загляните в github.com/cozystack/boot-to-talos и поставьте проекту звезду.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

2. Установка и bootstrap кластера Kubernetes

<https://cozystack.ru/docs/v1.6/getting-started/install-kubernetes/>

Используйте Talm CLI, чтобы выполнить bootstrap кластера Kubernetes, готового к установке Cozystack.

Цели

К началу этого шага у нас уже есть [три узла с установленным Talos Linux](#).

В результате этого шага у вас будет установленный и настроенный Kubernetes-кластер, готовый к установке Cozystack. Также вы получите `kubeconfig` для этого кластера и выполните базовые проверки его состояния.

Установка Kubernetes

Установите и выполните bootstrap кластера Kubernetes с помощью [Talm](#) — декларативного CLI-инструмента управления конфигурацией с готовыми пресетами для Cozystack.

Этот фрагмент руководства сейчас перерабатывается. Здесь появятся упрощённые инструкции по установке Talm без дополнительных опций и редких пограничных случаев, которые описаны в основном руководстве по Talm.

Следующий шаг

Продолжите руководство по Cozystack и [установите Cozystack](#).

Дополнительно:

- Загляните в github.com/cozystack/talm и поставьте проекту звезду.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

3. Установка и настройка Cozystack

<https://cozystack.ru/docs/v1.6/getting-started/install-cozystack/>

Установите Cozystack, получите административный доступ, выполните базовую настройку и включите UI-дашборд.

Цели

Это руководство описывает установку Cozystack как готовой к использованию платформы. Если вы хотите собрать собственную платформу, устанавливая только нужные компоненты, см. [руководство BYOP \(Build Your Own Platform\)](#).

На этом шаге мы установим Cozystack поверх [Kubernetes-кластера, подготовленного на предыдущем шаге](#).

Руководство проведёт вас через следующие этапы:

1. Установка оператора Cozystack
2. Подготовка файла конфигурации Cozystack и его применение
3. Настройка хранилища
4. Настройка сети
5. Развёртывание etcd, ingress и стека мониторинга в корневом tenant'e
6. Завершение развёртывания и вход в дашборд Cozystack

1\. Установка оператора Cozystack

Установите оператор Cozystack с помощью Helm chart из OCI-реестра. Оператор управляет всеми компонентами Cozystack и отвечает за жизненный цикл Platform Package.

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace
```

Замените `X.Y.Z` на нужную версию Cozystack. Доступные версии перечислены на [странице релизов Cozystack](#).

****Если установка прерывается из-за того, что `cozy-system` уже существует.**** Helm отказывается забирать себе namespace, который создал не он, и выводит ошибку `invalid ownership metadata`

(или namespaces "cozy-system" already exists в зависимости от версии Helm), если cozy-system остался после ранее прерванной установки или был создан вручную для этой цели.

Если namespace не управляется другим инструментом (Terraform, Argo CD, другим Helm release и т. д.), повторите команду с --take-ownership (требуется Helm 3.17+), чтобы Helm смог принять его под управление:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--take-ownership
```

Не используйте --take-ownership, если cozy-system уже принадлежит другой системе — Helm незаметно станет новым владельцем, а последующие обновления или удаление релиза Cozystack могут изменить или удалить namespace (и всё, что было принято под управление этим флагом) вопреки ожиданиям этой системы.

2\. Подготовка и применение Platform Package

2.1. Подготовьте файл конфигурации

Теперь, когда оператор запущен, подготовим для него файл конфигурации. Возьмите пример ниже и сохраните его в файл cozystack-platform.yaml :

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full
components:
platform:
values:
publishing:
host: "example.org"
apiServerEndpoint: "https://api.example.org:443"
exposedServices:
  - dashboard
  - api
```

networking:

podCIDR: "10.244.0.0/16"

podGateway: "10.244.0.1"

serviceCIDR: "10.96.0.0/16"

joinCIDR: "100.64.0.0/16"

Что нужно сделать:

1. Замените `example.org` в `publishing.host` и `publishing.apiServerEndpoint` на маршрутизируемое полное доменное имя (FQDN), которым вы управляете. Если у вас есть только публичный IP, но нет FQDN, используйте nip.io с dash-нотацией.

2. Используйте для `networking.*` те же значения, что и на предыдущем шаге, где вы выполняли bootstrap Kubernetes-кластера через Talm или `talosctl`. Значения из примера — это разумные значения по умолчанию, которые подходят для большинства случаев.

В этой конфигурации есть и другие значения, которые в рамках руководства менять не требуется. Тем не менее, кратко разберём каждое из них:

- `metadata.name` должен быть равен `cozystack.cozystack-platform`, чтобы совпасть с PackageSource, созданным установщиком.
- `publishing.host` используется как основной домен для всех сервисов, создаваемых в Cozystack, например дашборда, Grafana, Keycloak и т. д.
- `publishing.apiServerEndpoint` — это endpoint Cluster API. Он используется для генерации kubeconfig-файлов для пользователей. Рекомендуется использовать маршрутизируемые IP-адреса, а не локальные.
- `spec.variant: "isp-full"` означает, что используется наиболее полный набор компонентов Cozystack. Подробнее о вариантах см. в [справочнике по вариантам Cozystack](#).
- `publishing.exposedServices` перечисляет сервисы, которые должны быть доступны пользователям — здесь это дашборд (UI) и API.
- `networking.*` — это внутренняя сетевая конфигурация базового Kubernetes-кластера:
- `networking.podCIDR` — диапазон CIDR, из которого Kube-OVN выделяет IP-адреса pod'ам. Он не должен пересекаться ни с одной сетью, которую уже маршрутизируют ваши узлы.
- `networking.podGateway` — адрес шлюза, который Kube-OVN назначает подсети pod'ов по умолчанию. Используйте адрес .1 сети podCIDR (например, 10.244.0.1 для 10.244.0.0/16).
- `networking.serviceCIDR` — диапазон CIDR для сервисов ClusterIP. Он обязательно должен совпадать со значением `cluster.network.serviceSubnets`, которое вы использовали при bootstrap Kubernetes-кластера: это значение вшивается в kube-apiserver во время bootstrap и не может быть изменено без пересборки кластера, поэтому несоответствие здесь незаметно ломает DNS и маршрутизацию сервисов.
- `networking.joinCIDR` — диапазон CIDR для _join_ подсети Kube-OVN, внутренней сети, которая переносит трафик между узлами кластера и pod'ами. Значение по умолчанию 100.64.0.0/16 входит в общее адресное пространство RFC 6598 (100.64.0.0/10), зарезервированное для такого внутреннего использования. Меняйте его только если оно пересекается с сетью, которую уже маршрутизируют ваши узлы; подробности см. в

[справочнике по Kube-OVN join subnet](#).

Подробнее об этом файле конфигурации см. в [справочнике Platform Package](#).

По умолчанию Cozystack собирает анонимную статистику использования. Подробнее о том, какие данные собираются и как отказаться от этой функции, см. в [документации по телеметрии](#).

2.2. Примените Platform Package

Примените файл конфигурации:

```
kubectl apply -f cozystack-platform.yaml
```

Во время установки можно следить за логами оператора:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

2.3. Проверьте статус установки

Подождите немного, затем проверьте статус установки:

```
kubectl get hr -A
```

Повторяйте проверку, пока во всех строках не появится `True`, как в этом примере:

```
NAMESPACE NAME AGE READY STATUS
cozy-cert-manager cert-manager 4m1s True Release reconciliation succeeded
cozy-cert-manager cert-manager-issuers 4m1s True Release reconciliation succeeded
cozy-cilium cilium 4m1s True Release reconciliation succeeded
cozy-cluster-api capi-operator 4m1s True Release reconciliation succeeded
cozy-cluster-api capi-providers 4m1s True Release reconciliation succeeded
cozy-dashboard dashboard 4m1s True Release reconciliation succeeded
cozy-grafana-operator grafana-operator 4m1s True Release reconciliation succeeded
cozy-kamaji kamaji 4m1s True Release reconciliation succeeded
cozy-kubeovn kubeovn 4m1s True Release reconciliation succeeded
cozy-kubevirt-cdi kubevirt-cdi 4m1s True Release reconciliation succeeded
cozy-kubevirt-cdi kubevirt-cdi-operator 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt-operator 4m1s True Release reconciliation succeeded
cozy-linstor linstor 4m1s True Release reconciliation succeeded
cozy-linstor piraheus-operator 4m1s True Release reconciliation succeeded
```

```
cozy-mariadb-operator mariadb-operator 4m1s True Release reconciliation succeeded
cozy-metallb metallb 4m1s True Release reconciliation succeeded
cozy-monitoring monitoring 4m1s True Release reconciliation succeeded
cozy-postgres-operator postgres-operator 4m1s True Release reconciliation succeeded
cozy-rabbitmq-operator rabbitmq-operator 4m1s True Release reconciliation succeeded
cozy-redis-operator redis-operator 4m1s True Release reconciliation succeeded
cozy-telepresence telepresence 4m1s True Release reconciliation succeeded
cozy-victoria-metrics-operator victoria-metrics-operator 4m1s True Release reconciliation succeeded
tenant-root tenant-root 4m1s True Release reconciliation succeeded
```

Список компонентов в вашей установке может отличаться от приведённого выше, так как он зависит от вашей конфигурации и версии Cozystack.

Когда у всех компонентов появится `READY: True`, можно переходить к настройке подсистем.

3\ Настройка хранилища

Kubernetes нужен слой хранилища для предоставления persistent volumes приложениям, но собственного такого механизма у него нет. В качестве подсистемы хранения Cozystack использует [LINSTOR](#).

Далее мы получим доступ к интерфейсу LINSTOR, создадим storage pool'ы и определим storage class'ы.

3.1. Проверьте устройства хранения

1. Настройте alias для доступа к LINSTOR:

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

2. Выведите список узлов и проверьте их готовность:

```
linstor node list
```

В примере ниже показаны имена узлов и их состояние:

```
+-----+
| Node | NodeType | Addresses | State |
|=====|
| srv1 | SATELLITE | 192.168.100.11:3367 (SSL) | Online |
| srv2 | SATELLITE | 192.168.100.12:3367 (SSL) | Online |
| srv3 | SATELLITE | 192.168.100.13:3367 (SSL) | Online |
```

```
+-----+
```

3. Выведите список доступных пустых устройств:

```
linstor physical-storage list
```

В примере ниже отображаются те же имена узлов:

```
+-----+
| Size | Rotational | Nodes |
|=====|
| 107374182400 | True | srv3[/dev/sdb] |
| | | srv1[/dev/sdb] |
| | | srv2[/dev/sdb] |
+-----+
```

3.2. Создайте storage pool'ы

1. Создайте storage pool'ы на основе ZFS:

```
linstor ps cdp zfs srv1 /dev/sdb --pool-name data --storage-pool data
```

```
linstor ps cdp zfs srv2 /dev/sdb --pool-name data --storage-pool data
```

```
linstor ps cdp zfs srv3 /dev/sdb --pool-name data --storage-pool data
```

[Рекомендуется](#) установить для ZFS storage pool'ов `failmode=continue`, чтобы обработкой отказов диска занимался DRBD, а не ZFS.

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv1 -- zpool set failmode=continue data
```

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv2 -- zpool set failmode=continue data
```

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv3 -- zpool set failmode=continue data
```

2. Проверьте результат, выведя список storage pool'ов:

```
linstor sp l
```

Пример вывода:

```
+-----+
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | CanSnapshots | State |
| SharedName |
|=====|
| DfltDisklessStorPool | srv1 | DISKLESS | | | False | Ok | srv1;DfltDisklessStorPool |
+-----+
```

```
| DfltDisklessStorPool | srv2 | DISKLESS | | | False | Ok | srv2;DfltDisklessStorPool | |
| DfltDisklessStorPool | srv3 | DISKLESS | | | False | Ok | srv3;DfltDisklessStorPool |
| data | srv1 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv1;data |
| data | srv2 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv2;data |
| data | srv3 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv3;data |
+-----+-----+
```

3.3. Создайте storage class'ы

Наконец, можно создать несколько storage class'ов, один из которых будет классом по умолчанию.

1. Создайте файл с описанием storage class'ов. Ниже приведён разумный пример по умолчанию с двумя классами: `local` (по умолчанию) и `replicated`.

storageclasses.yaml:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: local
annotations:
storageclass.kubernetes.io/is-default-class: "true"
provisioner: linstor.csi.linbit.com
parameters:
linstor.csi.linbit.com/storagePool: "data"
linstor.csi.linbit.com/layerList: "storage"
linstor.csi.linbit.com/allowRemoteVolumeAccess: "false"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: replicated
provisioner: linstor.csi.linbit.com
parameters:
linstor.csi.linbit.com/storagePool: "data"
linstor.csi.linbit.com/autoPlace: "3"
```

```
linstor.csi.linbit.com/layerList: "drbd storage"
```

```
linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
```

```
property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io
```

```
property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io
```

```
property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
```

```
property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect
```

```
volumeBindingMode: Immediate
```

```
allowVolumeExpansion: true
```

2. Примените конфигурацию storage class'ов

```
kubectl apply -f storageclasses.yaml
```

3. Убедитесь, что storage class'ы были успешно созданы:

```
kubectl get storageclasses
```

Пример вывода:

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
```

```
local (default) linstor.csi.linbit.com Delete WaitForFirstConsumer true 11m
```

```
replicated linstor.csi.linbit.com Delete Immediate true 11m
```

4\. Настройка сети

Далее мы настроим способ доступа к кластеру Cozystack. На этом шаге есть два варианта в зависимости от доступной инфраструктуры:

- Для собственного bare metal или self-hosted VM выбирайте вариант с MetalLB. MetalLB — это балансировщик нагрузки по умолчанию в Cozystack.

- Для VM и выделенных серверов у облачных провайдеров выбирайте настройку через публичные IP. [Большинство облачных провайдеров не поддерживают MetalLB.](#)

Загляните в раздел [provider-specific installation](#). Возможно, там уже есть инструкции для вашего провайдера, подходящие для развёртывания production-ready кластера.

4.a Настройка MetalLB

В Cozystack используются три типа IP-адресов:

- IP-адреса узлов: постоянные и действуют только внутри кластера.

-
- Виртуальный floating IP: используется для доступа к одному из узлов кластера и также действует только внутри кластера.
 - IP-адреса внешнего доступа: используются LoadBalancer'ами для публикации сервисов за пределами кластера.

Сервисы с внешними IP могут публиковаться в двух режимах: L2 и BGP. Режим L2 проще, но требует, чтобы узлы находились в одном L2-домене, и хуже подходит для балансировки нагрузки. Режим BGP настраивается сложнее: нужны BGP peer'ы, готовые принимать анонсы, зато он позволяет организовать корректную балансировку и даёт больше гибкости при выборе диапазонов IP-адресов.

Выберите диапазон неиспользуемых IP-адресов для сервисов; в примере используется диапазон `192.168.100.200-192.168.100.250`. Если вы используете режим L2, эти IP должны либо принадлежать той же сети, что и узлы, либо до них должны существовать все необходимые маршруты.

Для режима BGP также потребуются IP-адреса BGP peer'ов и локальные и удалённые номера AS. В примере используются `192.168.20.254` как IP peer'а и номера AS 65000 и 65001 как локальный и удалённый соответственно.

Создайте и примените файл с описанием пула адресов.

```
metallb-ip-address-pool.yml
```

```
apiVersion: metallb.io/v1beta1
```

```
kind: IPAddressPool
```

```
metadata:
```

```
name: cozystack
```

```
namespace: cozy-metallb
```

```
spec:
```

```
addresses:
```

```
# используется для публикации сервисов за пределами кластера
```

```
  - 192.168.100.200-192.168.100.250
```

```
autoAssign: true
```

```
avoidBuggyIPs: false
```

```
kubectl apply -f metallb-ip-address-pool.yml
```

Создайте и примените ресурсы, необходимые для L2- или BGP-анонса.

- L2 mode
- BGP mode

L2Advertisement использует имя ресурса IPAddressPool, который мы создали на предыдущем шаге.

```
metallb-l2-advertisement.yml
```

```
apiVersion: metallb.io/v1beta1
```

```
kind: L2Advertisement
```

```
metadata:
```

```
name: cozystack
```

```
namespace: cozy-metallb
```

```
spec:
```

```
ipAddressPools:
```

```
  - cozystack
```

Примените изменения.

```
kubectl apply -f metallb-l2-advertisement.yml
```

Сначала создайте отдельный ресурс BGPPeer для каждого peer'a.

```
metallb-bgp-peer.yml
```

```
apiVersion: metallb.io/v1beta2
```

```
kind: BGPPeer
```

```
metadata:
```

```
name: peer1
```

```
namespace: cozy-metallb
```

```
spec:
```

```
myASN: 65000
```

```
peerASN: 65001
```

```
peerAddress: 192.168.20.254
```

Затем создайте один ресурс BGPAdvertisement.

```
metallb-bgp-advertisement.yml
```

```
apiVersion: metallb.io/v1beta1
```

```
kind: BGPAdvertisement
```

```
metadata:
```

```
name: cozystack
```

```
namespace: cozy-metallb
```

```
spec:
```

ipAddressPools:

- cozystack

Примените изменения.

```
kubectl apply -f metallb-bgp-peer.yml
```

```
kubectl apply -f metallb-bgp-advertisement.yml
```

Теперь, когда MetalLB настроен, включите `ingress` в `tenant-root`:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '
```

```
{ "spec": {
```

```
  "ingress": true
```

```
}}
```

Чтобы убедиться, что всё настроено корректно, проверьте HelmRelease'ы `ingress` и `ingress-nginx-system`:

```
kubectl -n tenant-root get hr ingress ingress-nginx-system
```

Пример корректного вывода:

```
NAME AGE READY STATUS
```

```
ingress 47m True Helm upgrade succeeded for release tenant-root/ingress.v3 with chart ingress@1.8.0
```

```
ingress-nginx-system 47m True Helm upgrade succeeded for release  
tenant-root/ingress-nginx-system.v2 with chart cozy-ingress-nginx@0.35.1
```

Затем проверьте состояние сервиса `root-ingress-controller`:

```
kubectl -n tenant-root get svc root-ingress-controller
```

Сервис должен быть развёрнут как `TYPE: LoadBalancer` и иметь корректный внешний IP:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
```

```
root-ingress-controller LoadBalancer 10.96.91.83 192.168.100.200 80/TCP,443/TCP 48m
```

4.b. Настройка публичных IP узлов

Если ваш облачный провайдер не поддерживает MetalLB, вы можете опубликовать `ingress controller` через внешние IP-адреса ваших узлов.

Если публичные IP привязаны непосредственно к узлам, укажите их. Если публичные IP предоставляются через 1:1 NAT, как у некоторых облачных провайдеров, используйте IP-адреса внешних сетевых интерфейсов.

В примере используются 192.168.100.11, 192.168.100.12 и 192.168.100.13.

Сначала добавьте внешние IP в Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
"spec": {
"components": {
"platform": {
"values": {
"publishing": {
"externalIPs": [
"192.168.100.11",
"192.168.100.12",
"192.168.100.13"
]
}
}
}
}
}
}'
```

Затем включите `ingress` для корневого tenant'a:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{
"spec":{
"ingress": true
}
}'
```

Наконец, добавьте внешние IP-адреса в список `externalIPs` в конфигурации Ingress:

```
kubectl patch -n tenant-root ingresses.apps.cozystack.io ingress --type=merge -p '{
"spec":{
"externalIPs": [
```

```
"192.168.100.11",
"192.168.100.12",
"192.168.100.13"
]
}
}'
```

После этого ваш Ingress будет доступен по указанным IP-адресам. Проверьте это так:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развёрнут как `TYPE: ClusterIP` и содержать полный список внешних IP:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
root-ingress-controller ClusterIP 10.96.91.83 192.168.100.11,192.168.100.12,192.168.100.13
80/TCP,443/TCP 48m
```

5\ Завершение установки

5.1. Настройте сервисы корневого tenant'a

Включите `etcd` и `monitoring` для корневого tenant'a:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '
{"spec":{
  "monitoring": true,
  "etcd": true
}}'
```

5.2. Проверьте состояние и состав кластера

Проверьте созданные persistent volume'ы:

```
kubectl get pvc -n tenant-root
```

Пример вывода:

```
NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE
data-etcd-0 Bound pvc-4cbd29cc-a29f-453d-b412-451647cd04bf 10Gi RWO local <unset> 2m10s
data-etcd-1 Bound pvc-1579f95a-a69d-4a26-bcc2-b15ccdbede0d 10Gi RWO local <unset> 115s
```

```
data-etcd-2 Bound pvc-907009e5-88bf-4d18-91e7-b56b0dbfb97e 10Gi RWO local <unset> 91s
grafana-db-1 Bound pvc-7b3f4e23-228a-46fd-b820-d033ef4679af 10Gi RWO local <unset> 2m41s
grafana-db-2 Bound pvc-ac9b72a4-f40e-47e8-ad24-f50d843b55e4 10Gi RWO local <unset> 113s
vmselect-cachedir-vmselect-longterm-0 Bound pvc-622fa398-2104-459f-8744-565eee0a13f1 2Gi RWO
local <unset> 2m21s
vmselect-cachedir-vmselect-longterm-1 Bound pvc-fc9349f5-02b2-4e25-8bef-6cbc5cc6d690 2Gi RWO
local <unset> 2m21s
vmselect-cachedir-vmselect-shortterm-0 Bound pvc-7acc7ff6-6b9b-4676-bd1f-6867ea7165e2 2Gi RWO
local <unset> 2m41s
vmselect-cachedir-vmselect-shortterm-1 Bound pvc-e514f12b-f1f6-40ff-9838-a6bda3580eb7 2Gi RWO
local <unset> 2m40s
vmstorage-db-vmstorage-longterm-0 Bound pvc-e8ac7fc3-df0d-4692-aebf-9f66f72f9fef 10Gi RWO local
<unset> 2m21s
vmstorage-db-vmstorage-longterm-1 Bound pvc-68b5ceaf-3ed1-4e5a-9568-6b95911c7c3a 10Gi RWO
local <unset> 2m21s
vmstorage-db-vmstorage-shortterm-0 Bound pvc-cee3a2a4-5680-4880-bc2a-85c14dba9380 10Gi RWO
local <unset> 2m41s
vmstorage-db-vmstorage-shortterm-1 Bound pvc-d55c235d-cada-4c4a-8299-e5fc3f161789 10Gi RWO
local <unset> 2m41s
```

Убедитесь, что все pod'ы запущены:

```
kubectrl get pod -n tenant-root
```

пример вывода:

```
NAME READY STATUS RESTARTS AGE
etcd-0 1/1 Running 0 2m1s
etcd-1 1/1 Running 0 106s
etcd-2 1/1 Running 0 82s
grafana-db-1 1/1 Running 0 119s
grafana-db-2 1/1 Running 0 13s
grafana-deployment-74b5656d6-5dcvn 1/1 Running 0 90s
grafana-deployment-74b5656d6-q5589 1/1 Running 1 (105s ago) 111s
root-ingress-controller-6ccf55bc6d-pg79l 2/2 Running 0 2m27s
root-ingress-controller-6ccf55bc6d-xbs6x 2/2 Running 0 2m29s
root-ingress-defaultbackend-686bcbbd6c-5zbvp 1/1 Running 0 2m29s
vmalert-vmalert-644986d5c-7hvwk 2/2 Running 0 2m30s
vmalertmanager-alertmanager-0 2/2 Running 0 2m32s
```

```
vmalertmanager-alertmanager-1 2/2 Running 0 2m31s
vminsert-longterm-75789465f-hc6cz 1/1 Running 0 2m10s
vminsert-longterm-75789465f-m2v4t 1/1 Running 0 2m12s
vminsert-shortterm-78456f8fd9-wlwww 1/1 Running 0 2m29s
vminsert-shortterm-78456f8fd9-xg7cw 1/1 Running 0 2m28s
vmselect-longterm-0 1/1 Running 0 2m12s
vmselect-longterm-1 1/1 Running 0 2m12s
vmselect-shortterm-0 1/1 Running 0 2m31s
vmselect-shortterm-1 1/1 Running 0 2m30s
vmstorage-longterm-0 1/1 Running 0 2m12s
vmstorage-longterm-1 1/1 Running 0 2m12s
vmstorage-shortterm-0 1/1 Running 0 2m32s
vmstorage-shortterm-1 1/1 Running 0 2m31s
```

Получите публичный IP ingress controller:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Пример вывода:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
root-ingress-controller LoadBalancer 10.96.16.141 192.168.100.200 80:31632/TCP,443:30113/TCP 3m33s
```

5.3 Доступ к дашборду Cozystack

Если вы включили `dashboard` в список `publishing.exposedServices` вашего Platform Package (как показано на шаге 2), дашборд Cozystack уже доступен.

Если в исходной конфигурации его не было, обновите Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "dashboard"}]'
```

Откройте `dashboard.example.org`, чтобы перейти в системный дашборд, где `example.org` — домен, указанный вами для `tenant-root`. Там вы увидите окно входа, ожидающее токен аутентификации.

Получите токен аутентификации для `tenant-root`:

<pre>kubectl get secret -n tenant-root tenant-root-token -o go-template='{{ printf "%s\n" (index .data "token" base64decode) }}'</pre>	
--	--

Войдите с этим токеном. Теперь вы можете пользоваться дашбордом как администратор.

Дальше вы сможете:

- Настроить OIDC и использовать его вместо токенов для аутентификации.
- Создавать пользовательские `tenant`'ы и выдавать пользователям доступ через токены или OIDC.

5.4 Доступ к метрикам в Grafana

Используйте `grafana.example.org` для доступа к системному мониторингу, где `example.org` — домен, указанный для `tenant-root`. В этом примере `grafana.example.org` доступен по адресу 192.168.100.200.

- логин: `admin`
- получите пароль:

```
kubectl get secret -n tenant-root  
grafana-admin-password -o go-template='{{  
printf "%s\n" (index .data "password"
```

```
base64decode) }}'
```

Следующий шаг

Продолжите руководство по Cozystack и [создайте пользовательский tenant](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

4. Создание пользовательского tenant'a и настройка доступа

<https://cozystack.ru/docs/v1.6/getting-started/create-tenant/>

Создайте пользовательский tenant — основу RBAC в Cozystack — и получите доступ к нему через дашборд и Cozystack API.

Цели

На этом шаге вы создадите пользовательский tenant — пространство, в котором пользователи смогут развёртывать приложения и виртуальные машины. Вы также получите учётные данные tenant'a и войдёте как пользователь с доступом к этому tenant'у.

Предварительные требования

Перед началом:

- Завершите предыдущие шаги руководства, чтобы получить работающий [кластер Cozystack](#), в котором уже настроены хранилище, сеть и управляющий дашборд.
- Убедитесь, что у вас есть доступ к дашборду, как описано на [предыдущем шаге руководства](#).
- Если вы используете OIDC, пользователи и роли уже должны быть настроены. Подробности о работе со встроенным OIDC-сервером см. в [руководстве по OIDC](#).

Во время [установки Kubernetes](#) для Cozystack вы должны были получить административный файл `kubeconfig` для нового кластера. Держите его под рукой — позже он может пригодиться для диагностики. Однако для повседневной работы лучше создать отдельные пользовательские учётные данные.

Введение

Tenant'ы — это механизм изоляции в Cozystack. Они используются для разделения клиентов, команд или окружений. У каждого tenant'a есть собственный набор приложений и один или несколько вложенных Kubernetes-кластеров. Пользователи tenant'a имеют полный доступ к своим кластерам. При необходимости вы можете настроить квоты для каждого tenant'a, чтобы ограничить потребление ресурсов и избежать перерасхода.

Чтобы узнать о tenant'ах больше, прочитайте руководство [Core Concepts](#).

Создание tenant'a

Tenant'ы создаются с помощью приложения Cozystack с именем `Tenant`. После установки в Cozystack уже существует встроенный tenant `tenant-root`. Этот корневой tenant зарезервирован для администраторов платформы и должен использоваться только для создания дочерних tenant'ов. Технически установить приложения в `tenant-root` можно, но для production-окружений это не рекомендуется.

- Через дашборд
- Через `kubectl`

1. Откройте дашборд от имени пользователя `tenant-root`.
2. Убедитесь, что текущий контекст установлен в `tenant-root`. При необходимости переключите контекст и перезагрузите страницу.
3. Перейдите на вкладку `Catalog`.
4. Найдите приложение `Tenant` и откройте его.
5. Ознакомьтесь с документацией, затем нажмите кнопку `Deploy`, чтобы перейти к странице параметров.
6. Заполните поле `name` tenant'a. Это единственный параметр, который нельзя изменить позже.
7. (Необязательно) Заполните доменное имя в `host`. Это доменное имя уже должно существовать. Убедитесь, что пользователь tenant'a имеет достаточный контроль над доменом для настройки DNS-записей. Если оставить поле пустым, по умолчанию будет использован домен `<name>.<cozystack-domain>`.
8. Отметьте флажки для установки системных приложений: `etcd`, `monitoring`, `ingress` и `seaweedfs`. Пользователи tenant'a не смогут устанавливать или удалять эти приложения — это могут делать только администраторы.

Параметр `etcd` требуется для вложенного Kubernetes. Выберите его до установки приложения Kubernetes в tenant'e. Отключайте его только если уверены, что tenant не будет использовать вложенный Kubernetes.

9. По умолчанию ресурсные квоты не задаются. Это означает отсутствие ограничений на использование ресурсов. При необходимости можно определить квоты, чтобы избежать их перерасхода.

10. Нажмите `Deploy`, чтобы установить приложение tenant'a в корневой tenant.

Создайте манифест `HelmRelease` для tenant'a. В качестве отправной точки можно взять манифест, созданный через дашборд:

`apiVersion: helm.toolkit.fluxcd.io/v2`

```
kind: HelmRelease
metadata:
name: tenant-team1
namespace: tenant-root
spec:
chart:
spec:
chart: tenant
reconcileStrategy: Revision
sourceRef:
kind: HelmRepository
name: cozystack-apps
namespace: cozy-public
version: 1.9.1
interval: 0s
values:
etcd: true
host: team1.example.org
ingress: true
monitoring: false
resourceQuotas: {}
seaweedfs: false
```

Примените манифест:

```
# Используйте kubecfg корневого tenant'a
export KUBECONFIG=./kubecfg-tenant-root
# Примените манифест
kubectl -n tenant-root apply -f hr-tenant-team1.yaml
```

Сетевые политики Cilium в Cozystack v1.0+ всегда изолируют соседние tenant'ы друг от друга — поля `isolated` больше нет ни в форме дашборда, ни в значениях `HelmRelease`. Pod'ы внутри namespace tenant'a также по умолчанию не могут обращаться к `kube-apiserver`, а при создании tenant'a с `etcd: true` ещё и к собственному `etcd`. Чтобы разрешить pod'у один из этих путей, добавьте ему метку `policy.cozystack.io/allow-to-apiserver: "true"` или `policy.cozystack.io/allow-to-etcd: "true"` соответственно. Полную таблицу и пример см. в примечаниях к обновлению:

[Tenant `isolated` flag removed](https://cozystack.ru/docs/v1.6/operations/upgrades/#tenant-isolated-flag-removed).

Вы можете помогать пользователям tenant'ов с установкой баз данных или вложенных Kubernetes-кластеров. Администратор может переключать контекст в дашборде и получать доступ к любому tenant'у. Пользователи tenant'а, в свою очередь, могут работать только со своим tenant'ом и его дочерними tenant'ами.

Получение kubernetesconfig tenant'а

Пользователям tenant'а нужен файл kubernetesconfig для доступа к своему Kubernetes-кластеру. Способ его получения зависит от того, включён ли OIDC в вашей установке Cozystack.

Если OIDC включён

Вы можете получить kubernetesconfig напрямую из дашборда, как описано в [руководстве по OIDC](#).

Если OIDC выключен

В этом случае администратору нужно получить токен service account из namespace tenant'а. Секрет с токеном имеет то же имя, что и tenant.

Чтобы получить токен для tenant'а с именем `team1`, выполните:

<code>kubectl -n tenant-team1 get secret tenant-team1 -o json</code>	<code>jq -r '.data.token'</code>	<code>@base64d'</code>
--	----------------------------------	------------------------

Затем подставьте этот токен в шаблон kubernetesconfig и сохраните файл под именем `kubernetesconfig-tenant-<name>.yaml`.

Обязательно задайте namespace по умолчанию равным имени tenant'а. Многие GUI-клиенты будут показывать ошибки доступа, если namespace не указан явно.

Тот же токен можно использовать и для входа пользователя tenant'а в дашборд Cozystack, если OIDC отключён.

Получение kubernetesconfig вложенного Kubernetes

Как правило, администраторам не нужно получать kubernetesconfig для вложенных Kubernetes-кластеров.

Такие кластеры устанавливаются самим пользователем tenant'а внутри его namespace. Пользователи tenant'а полностью контролируют свои вложенные Kubernetes-окружения.

Чтобы получить доступ к вложенному Kubernetes-кластеру, пользователь tenant'а может скачать kubernetesconfig прямо со страницы соответствующего приложения в дашборде.

5. Развёртывание управляемых приложений, ВМ и Kubernetes-кластера tenant'a

<https://cozystack.ru/docs/v1.6/getting-started/deploy-app/>

Начните использовать Cozystack: разверните виртуальную машину, управляемое приложение и Kubernetes-кластер tenant'a.

Цели

Это руководство поможет вам подготовить окружение для запуска типичного веб-приложения с распространёнными сервисными зависимостями — PostgreSQL и Redis — в Cozystack, PaaS-платформе на базе Kubernetes.

Вы научитесь:

- Развёртывать управляемые приложения в своём tenant'e: базу данных PostgreSQL и кэш Redis.
- Создавать управляемый Kubernetes-кластер, настраивать DNS и получать доступ к кластеру.
- Развёртывать контейнеризованное приложение в новом кластере.

Для прохождения этого руководства вам не нужны глубокие знания Kubernetes — большая часть шагов выполняется через веб-интерфейс Cozystack.

Это быстрый способ выполнить первое успешное развёртывание в Cozystack. После завершения у вас будет рабочее окружение для собственных приложений и надёжная база, на которую можно опираться и показывать команде.

Предварительные требования

Перед началом:

- Кластер Cozystack уже должен быть [установлен и работать](#). На уровне инфраструктуры ничего устанавливать или настраивать не потребуется — это руководство предполагает, что эта часть уже сделана вами или кем-то из вашей команды.
- Tenant и учётные данные: у вас должен быть доступ к своему tenant'у в Cozystack. Это может быть либо `kubeconfig`, либо вход в дашборд через OIDC. Если доступа нет, обратитесь к своей Ops-команде или воспользуйтесь [руководством по созданию tenant'a](#).
- DNS для dev/test: чтобы открыть развернутое приложение по HTTPS, нужно заранее настроить DNS-запись. Предпочтительнее использовать wildcard-запись, так как это удобнее.

CLI необязателен. Использовать `kubectl` или `helm` необязательно. Все основные шаги, включая создание Kubernetes-кластера и управляемых сервисов, можно полностью выполнить через

дашборд Cozystack. CLI понадобится только на этапе развёртывания приложения в Kubernetes-кластере.

1\ Вход в дашборд Cozystack

Откройте дашборд Cozystack в браузере. Обычно ссылка выглядит так:
`https://dashboard.<cozystack_domain>`.

В зависимости от того, как в вашем кластере Cozystack настроена аутентификация, вы увидите один из следующих вариантов:

- Экран входа через OIDC с кнопкой, которая перенаправляет в Keycloak.
- Экран входа по токену, где нужно вручную вставить токен из файла `kubeconfig`.

Выберите подходящий способ входа:

- OIDC
- `kubeconfig`

Нажмите кнопку `OIDC Login`. Вы будете перенаправлены на страницу входа Keycloak.

Введите свои учётные данные и нажмите `Login`. Если всё настроено корректно, вы войдёте в систему и вернётесь обратно в дашборд.

В этой форме нет поля `username` — только поле `token`. Нужный токен можно взять из `kubeconfig`.

1. Откройте `kubeconfig` и скопируйте значение токена (это длинная строка). Убедитесь, что копируете его без лишних пробелов и переносов строки.
2. Вставьте его в форму и нажмите `Submit`.

После входа дашборд автоматически откроется в контексте вашего `tenant'a`.

Вы можете увидеть уже работающие системные приложения, например `ingress` или `monitoring` — ими управляет администратор кластера. Как пользователь `tenant'a` вы не можете устанавливать или изменять их, но ваши собственные приложения будут работать рядом с ними в изолированном `tenant-окружении`.

2\ Создание управляемого PostgreSQL

Cozystack позволяет разворачивать управляемые базы данных прямо на уровне оборудования для максимальной производительности. Каждая база создаётся внутри `namespace tenant'a` и автоматически доступна из вашего вложенного Kubernetes-кластера.

Если вы знакомы с сервисами вроде AWS RDS или GCP Cloud SQL, опыт будет похожим — с той разницей, что здесь всё полностью интегрировано в Cozystack и изолировано внутри вашего `tenant'a`.

По ходу этого руководства вы сможете выбирать между дашбордом Cozystack (UI) и `kubectl`:

* Дашборд Cozystack — самый быстрый и простой вариант, особенно если вы впервые работаете с Cozystack.

*`kubectl`** даёт более глубокое понимание того, как управляемые сервисы разворачиваются под капотом.

Хотя ни один из этих подходов не отражает типичный production-процесс развёртывания сервисов, оба хорошо подходят для обучения и экспериментов, поэтому отлично подходят для этого руководства.

2.1 Развёртывание PostgreSQL

- Дашборд Cozystack
- `kubectl`

1. Откройте дашборд Cozystack и перейдите на вкладку Catalog.
2. Найдите карточку приложения Postgres и нажмите на неё, чтобы открыть встроенную документацию.
3. Нажмите кнопку Deploy , чтобы открыть страницу конфигурации развёртывания.
4. Укажите `instaphoto-postgres` в поле **name**. Имя приложения должно быть уникальным внутри tenant'a и не может быть изменено после развёртывания**.
5. Просмотрите остальные параметры. Они уже заполнены разумными значениями по умолчанию, поэтому их можно оставить без изменений.
 - Попробуйте и Visual editor , и YAML editor. Между ними можно переключаться в любой момент.
 - В YAML editor есть встроенные комментарии-подсказки.
 - Не переживайте, если не уверены в части настроек. Большинство из них можно изменить позже.
6. Снова нажмите Deploy. База данных будет установлена в namespace вашего tenant'a.

Создайте манифест `postgres.yaml` со следующим содержимым:

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: postgres-instaphoto-dev
  namespace: tenant-team1
spec:
  chart:
  spec:
```

```
chart: postgres
reconcileStrategy: Revision
sourceRef:
kind: HelmRepository
name: cozystack-apps
namespace: cozy-public
version: 0.10.0
interval: 0s
values:
databases:
myapp:
roles:
admin:
  - user1
external: true
replicas: 2
resourcesPreset: nano
size: 5Gi
users:
user1:
password: strongpassword
```

Примените манифест командой:

```
kubectl apply -f postgres.yaml
```

Совет: похожий манифест можно сначала сгенерировать через дашборд, развернув приложение Postgres. Затем экспортируйте конфигурацию и отредактируйте её по необходимости. Это удобно, если вы хотите воспроизвести или автоматизировать установку.

2.2 Получение параметров подключения

Перейдите на вкладку Applications , затем найдите и откройте приложение `instaphoto-postgres`. После установки и готовности приложения параметры подключения появятся в разделе Application Resources в дашборде.

- На вкладке Secrets находятся пароли базы данных для каждого созданного вами пользователя.
- На вкладке Services перечислены внутренние сервисные endpoints:
- Используйте `postgres-<name>-ro` для подключения к read-only replica.

- Используйте `postgres-<name>-rw` для подключения к primary (read-write) инстансу.

Эти имена сервисов резолвятся изнутри вложенного Kubernetes-кластера и могут использоваться в конфигурации вашего приложения.

Если нужно подключаться к базе данных извне кластера, можно включить внешнюю публикацию, установив параметр `external` в `true`. Это создаст сервис `postgres-<name>-external-write` с публичным IP-адресом.

[!] Включайте внешний доступ только при реальной необходимости. Публикация баз данных в интернет повышает риски безопасности и в большинстве случаев её следует избегать.

3\ Создание сервиса кэширования

Начиная с этого момента, для доступа к платформе используйте учётные данные `tenant'a`. Для `kubectl` используйте `kubeconfig tenant'a`, а для входа в дашборд — токен из него.

- Дашборд Cozystack
- `kubectl`

1. Откройте дашборд.
2. Повторите те же шаги, что и для PostgreSQL, но для приложения Redis.
3. В приложении Redis есть параметр `authEnabled`, который создаёт пользователя по умолчанию. Для нашего приложения этого достаточно.
4. После настройки параметров нажмите кнопку Deploy. Приложение будет установлено в ваш `tenant`.

Создайте файл манифеста `redis.yaml` со следующим содержимым:

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: redis-instaphoto
  namespace: tenant-team1
spec:
  chart:
    spec:
      chart: redis
      reconcileStrategy: Revision
    sourceRef:
      kind: HelmRepository
```

```
name: cozystack-apps
namespace: cozy-public
version: 0.6.0
interval: 0s
values:
authEnabled: true
external: false
replicas: 2
resources: {}
resourcesPreset: nano
size: 1Gi
```

Затем примените его:

```
kubectl apply -f redis.yaml
```

Через некоторое время приложение Redis будет установлено в tenant `team1`. Сгенерированный пароль можно посмотреть в дашборде.

- Дашборд Cozystack
- kubectl

1. Откройте дашборд от имени пользователя tenant - `team1`.
2. В левом меню нажмите вкладку Applications.
3. Найдите приложение `redis-instaphoto` и откройте его.
4. Пароль отображается в разделе Secrets , где есть кнопки для копирования и показа значения.

```
# Используйте kubeconfig tenant'a
export KUBECONFIG=./kubeconfig-tenant-team1

# Получите пароль
kubectl -n tenant-team1 get secret redis-instaphoto-auth
```

4\.. Развёртывание вложенного Kubernetes-кластера

Вложенный Kubernetes-кластер создаётся так же, как база данных и кэш. Однако здесь есть несколько важных дополнительных моментов:

- ****В tenant'е должен быть включён etcd****

Сервис `etcd` обязателен для запуска вложенного Kubernetes-кластера и может быть включён только администратором Cozystack.

-
- Проверьте свою квоту.

Убедитесь, что в `tenant` достаточно CPU, RAM и дисковых ресурсов для создания и работы кластера.

- Выберите подходящий пресет инстанса.

Избегайте слишком маленьких пресетов. Один узел Kubernetes потребляет примерно 2.5 ГБ RAM только на системные компоненты. Например, если выбрать пресет с 4 ГБ RAM, под реальные рабочие нагрузки останется около 1.5 ГБ. Для тестов 4 ГБ достаточно, но в целом лучше выделить меньше узлов с большим объёмом памяти, чем много узлов с минимальной RAM.

- ****При необходимости включите `ingress` и `cert-manager`****

Если вы планируете развёртывать веб-приложения, почти наверняка понадобятся `ingress` и управление сертификатами. Оба параметра можно включить флажками при настройке приложения вложенного Kubernetes в Cozystack.

Когда вложенный Kubernetes-кластер будет готов, его `kubeconfig`-файлы появятся на вкладке Secrets на странице приложения в дашборде. Будут доступны несколько вариантов:

- ****`admin.conf`**** — стандартный `kubeconfig` для доступа к новому кластеру. С его помощью можно создавать дополнительных пользователей Kubernetes.
- ****`admin.svc`**** — тот же токен, что и в `admin.conf`, но адрес API server указывает на внутреннее имя сервиса. Используйте его для приложений, которым нужен доступ к API изнутри кластера.
- ****`super-admin.conf`**** — аналог `admin.conf`, но с расширенными административными правами. Предназначен для диагностики и задач обслуживания кластера.
- ****`super-admin.svc`**** — то же, что и `super-admin.conf`, но с внутренним адресом API server.

5\ Обновление DNS и доступ к кластеру

После развёртывания вложенный Kubernetes-кластер автоматически получит один из floating IP-адресов основного кластера.

Узнать назначенное DNS-имя и IP-адрес можно двумя способами:

- Откройте страницу приложения кластера в дашборде.
- Проверьте статус `ingress` через `kubectl`.

После того как вы узнаете правильные DNS-имя и IP-адрес, обновите DNS-настройки так, чтобы ваш домен или поддомен указывал на этот IP.

После обновления и распространения DNS-записей вы сможете подключиться к вложенному Kubernetes-кластеру с помощью скачанного `kubeconfig`.

Пример настройки и использования:

1. Сохраните содержимое `admin.conf` в файл, например `~/.kube/kubeconfig-team1.example.org`:

```
$ cat ~/.kube/kubeconfig-team1.example.org
```

```
apiVersion: v1
```

```
clusters:
```

```
  - cluster:
```

```
certificate-authority-data: LS0tL
```

```
...
```

2. Укажите этот файл в переменной окружения `KUBECONFIG` и убедитесь, что узлы готовы:

```
$ export KUBECONFIG=~/.kube/kubeconfig-team1.example.org
```

```
$ kubectl get nodes
```

```
NAME STATUS ROLES AGE VERSION
```

```
kubernetes-dev-md0-vn8dh-jjbm9 Ready ingress-nginx 29m v1.60.11
```

```
kubernetes-dev-md0-vn8dh-xhsvl Ready ingress-nginx 25m v1.60.11
```

6\. Развёртывание приложения с помощью Helm

Начиная с этого момента, работа с вашим кластером ничем не отличается от работы с обычным Kubernetes-окружением.

Для развёртывания Kubernetes-native приложений можно использовать `kubectl`, `helm` или ваш CI/CD pipeline.

Чтобы развернуть приложение:

1. Обновите значения Helm chart, указав корректные учётные данные для базы данных и кэша.

2. Выполните обычную команду Helm для развёртывания, например:

```
helm upgrade --install <release-name> <chart-path> -f values.yaml
```

Имена сервисов, например базы данных и кэша, не требуют DNS-суффиксов. В пределах одного namespace к ним можно обращаться напрямую по имени сервиса.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Изучение Cozystack

Изучение Cozystack

<https://cozystack.ru/docs/v1.6/guides/>

Узнайте, как использовать Cozystack в роли администратора кластера и владельца tenant'a.

-

Ключевые концепции

Познакомьтесь с ключевыми концепциями Cozystack: management cluster, tenants, variants и lifecycle PackageSource/Package.

Архитектура Cozystack и стек платформы

Узнайте о ключевых компонентах, которые обеспечивают функциональность и гибкость Cozystack

Система tenant'ов

Узнайте о tenants и о том, как Cozystack помогает управлять ресурсами и повышать безопасность.

Управление ресурсами в Cozystack

Как CPU, memory и presets работают для VM, Kubernetes-кластеров и managed workloads в Cozystack; и как перенастраивать ресурсы через UI, CLI или API.

Talos Linux в Cozystack

Узнайте, почему Cozystack использует Talos Linux как основу Kubernetes-кластеров. Познакомьтесь с преимуществами Talos Linux: надежностью, масштабируемостью и оптимизацией под Kubernetes.

Сценарии использования

Сценарии использования Cozystack.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Ключевые концепции

<https://cozystack.ru/docs/v1.6/guides/concepts/>

Познакомьтесь с ключевыми концепциями Cozystack: management cluster, tenants, variants и lifecycle PackageSource/Package.

Cozystack — open-source, Kubernetes-native платформа, которая превращает bare-metal или виртуальную инфраструктуру в полноценное multi-tenant cloud. В основе платформы лежит несколько базовых building blocks:

- management cluster , в котором работает сама платформа;
- tenants , обеспечивающие строгую иерархическую изоляцию;
- tenant clusters , дающие пользователям собственные Kubernetes control planes;
- богатый каталог managed applications и виртуальных машин;
- variants , собирающие эти компоненты в готовый stack.

Понимание того, как эти концепции связаны друг с другом, поможет планировать, разворачивать и эксплуатировать Cozystack, независимо от того, строите ли вы внутреннюю developer platform или public cloud service.

Management Cluster

Cozystack — это система сервисов, работающих в Kubernetes-кластере, обычно развернутом поверх Talos Linux на bare metal или виртуальных машинах. Такой Kubernetes-кластер называется management cluster , чтобы подчеркнуть его роль и отличить от tenant Kubernetes clusters. Полный доступ к management cluster есть только у администраторов Cozystack.

Management cluster используется для развертывания заранее настроенных приложений: tenants, системных компонентов, managed apps, ВМ и tenant clusters. Пользователи Cozystack могут взаимодействовать с management cluster через dashboard и API, а также разворачивать managed applications. Однако у них нет административных прав, и они не могут разворачивать произвольные приложения в management cluster; для этого используются tenant clusters.

Tenant

Tenant в Cozystack — основная единица изоляции и безопасности, похожая на Kubernetes namespace, но с расширенной областью действия. Каждый tenant представляет изолированную среду со своими ресурсами, сетью и RBAC (role-based access control). Некоторые cloud providers используют для похожей сущности термин “projects”.

Когда Cozystack используется для построения private cloud и внутренней development platform, tenant обычно принадлежит команде или подкоманде. В hosting business, где Cozystack является основой public cloud, tenant может принадлежать клиенту.

Подробнее: [Tenant System](#).

Tenant Cluster

Пользователи могут разворачивать отдельные Kubernetes-кластеры в своих tenants. Это не namespaces management cluster, а полноценные Kubernetes-in-Kubernetes кластеры.

Tenant clusters — это то, что многие cloud providers называют “managed Kubernetes”. Они используются как development, testing и production environments.

Подробнее: [tenant Kubernetes clusters](#).

Managed Applications

Cozystack поставляется с каталогом managed applications (services), которые можно разворачивать на платформе с минимальными усилиями. В него входят relational databases (PostgreSQL, MySQL/MariaDB), NoSQL/queues (Redis, NATS, Kafka, RabbitMQ), HTTP cache, load balancer и другие сервисы.

Tenants, tenant Kubernetes clusters и VM также являются managed applications с точки зрения Cozystack. Они создаются тем же пользовательским workflow и управляются через Helm и Flux, как и другие приложения.

Подробнее: [managed applications](#).

Cozystack API

Вместо proprietary API или управления только через UI Cozystack предоставляет функциональность через [Kubernetes Custom Resources](#) и стандартный Kubernetes API, доступный через REST API, клиент `kubectl` и Cozystack dashboard.

Такой подход хорошо сочетается с role-based access control. Неадминистративные пользователи могут использовать `kubectl` для доступа к management cluster, но их kubeconfig разрешает им создавать custom resources только в своих tenants.

Подробнее: [Cozystack API](#).

Variants

Variants — это заранее определенные конфигурации Cozystack, которые задают, какие bundles и components включены. Каждый variant тестируется, версионизируется и гарантированно работает как единое целое. Они упрощают установку, снижают риск неправильной конфигурации и помогают выбрать подходящий набор функций для вашего развертывания.

Подробнее: [Variants](#).

PackageSource и Package

`PackageSource` и `Package` — две Custom Resource Definitions (CRDs), которые управляют всем lifecycle приложений в Cozystack.

- `PackageSource` (cluster-scoped) определяет, что доступно: он ссылается на Flux source (OCIRepository или GitRepository), который опрашивает внешний registry, перечисляет variants, объявляет dependencies и задает components, из которых состоит каждое приложение.
- `Package` (cluster-scoped) определяет, что развернуто: он выбирает variant из `PackageSource`, передает per-component value overrides и запускает создание HelmRelease, который управляет реальными Kubernetes resources.

Вместе они образуют declarative pipeline: external charts проходят через Flux sources и artifact generators в готовые к установке Helm charts, после чего Packages создают из них работающие workloads.

OCIRepositories: Platform и Packages

Cozystack использует два ресурса OCIRepository, чтобы управлять update flow и гарантировать выполнение migrations до обновления любых компонентов.

Начальный OCIRepository (cozystack-platform)

Создается cozystack-operator во время bootstrap. Operator получает platform source URL (например, `oci://ghcr.io/cozystack/cozystack/cozystack-packages`) и создает OCIRepository с именем `cozystack-platform`. Этот repository указывает на artifact platform chart, настраивается через значения installer (`platformSourceUrl`, `platformSourceRef`) и предоставляет platform chart, который создаст migrations и вторичный OCIRepository.

Вторичный OCIRepository (cozystack-packages)

Создается Helm chart платформы (`packages/core/platform/templates/repository.yaml`). Он копирует spec из `cozystack-platform` и создает новый OCIRepository с именем `cozystack-packages`. Этот repository используется всеми PackageSources (`networking`, `monitoring`, `postgres-operator` и т. д.), содержит все system и application charts и отделяет platform source от component PackageSources.

Порядок миграций

Схема с двумя repositories гарантирует, что system migrations выполняются до обновления любых компонентов:

flowchart TD

A["Installer Chart (helm install)"]

B["coystack-operator запускается"]

C["Начальный OCIRepository (coystack-platform)
создается operator'ом"]

D["Platform Chart из начального OCIRepository"]

E["Pre-upgrade hooks: последовательный запуск migrations
обновление ConfigMap coystack-version"]

F["Вторичный OCIRepository (coystack-packages)
создается platform chart"]

G["PackageSources ссылаются на coystack-packages"]

H["HelmReleases системных компонентов разворачиваются"]

A --> B --> C --> D --> E --> F --> G --> H

Когда выходит новая версия платформы и кластер обновляется:

1. Начальный OCIRepository (coystack-platform) предоставляет новый platform chart.
2. Во время `helm upgrade pre-upgrade hooks` из platform chart последовательно выполняют migrations (от текущей версии к целевой).
3. Каждый migration script выполняет необходимые преобразования и обновляет ConfigMap coystack-version.
4. После завершения migrations platform chart создает или обновляет OCIRepository coystack-packages.
5. PackageSources ссылаются на coystack-packages и запускают reconciliation системных компонентов.

Это гарантирует, что migrations выполняются до обновления компонентов, а migration scripts берутся из той же версии chart, которая разворачивается.

Reconciliation Flow

Полная цепочка reconciliation от внешнего registry до работающих Kubernetes resources:

flowchart TD

REG["Внешний Helm Registry
(OCI Registry или Git Repo)"]

SRC["Flux Source
(OCIRepository / GitRepository)
периодически опрашивает registry"]

PS["PackageSource
(cluster-scoped)
определяет variants, dependencies,
libraries и components"]

AG["ArtifactGenerator
(namespace cozy-system)
собирает ExternalArtifact
для каждого component"]

EA["ExternalArtifact
собранный Helm chart
готовый к установке"]

PKG["Package
(cluster-scoped)
выбирает variant + per-component values"]

HR["HelmRelease
(namespace-scoped)
ссылается на ExternalArtifact через chartRef"]

K8S["Kubernetes Resources
(Pods, Services, ConfigMaps, Secrets, ...)"]

REG --> SRC

PS -->|"references via sourceRef"| SRC

PS -->|"reconciler creates"| AG

AG -->|"reads charts from"| SRC

AG --> EA

PKG -->|"triggers creation of"| HR

HR -->|"references via chartRef"| EA

HR --> K8S

Имена ExternalArtifacts строятся по шаблону `<packagesource>-<variant>-<component>`, при этом точки заменяются дефисами, чтобы соответствовать правилам именования Kubernetes. Например, PackageSource `cozystack.keycloak` с variant `default` и component `keycloak` создает `cozystack-keycloak-default-keycloak`.

Package Dependencies

Variants в PackageSource могут объявлять `dependsOn`, чтобы отложить создание HelmRelease до готовности всех dependencies:

flowchart LR

A["Package A"]

B["Package B"]

CHECK{"Все dependencies
Ready?"}

C["Package C"]

HR["HelmRelease для C"]

A -->|"status: Ready"| CHECK

B -->|"status: Ready"| CHECK

CHECK -->|"Yes"| HR

CHECK -->|"No"| WAIT["Package C ожидает"]

Если все dependencies имеют статус `Ready`, dependent Package продолжает создание своего HelmRelease. Иначе Package остается в состоянии ожидания, пока условия не будут выполнены.

Dependencies в PackageSource используются на двух уровнях:

- Variant-level (`spec.variants.dependsOn[ ]`): ссылается на имена других Packages. PackageReconciler проверяет, что все dependencies готовы, прежде чем создавать HelmReleases. Это гарантирует, что infrastructure packages (например, CNI, storage) полностью запущены до установки dependent packages. Поле `spec.ignoreDependencies` в Package может отключить эту проверку для отдельных dependencies.

-
- Component-level (`spec.variants.components.install.dependsOn[]`): преобразуется в поле `spec.dependsOn[]` ресурса `HelmRelease`. Эти dependencies обеспечивают правильный порядок установки components внутри package.

Управление namespaces и values

Когда `PackageReconciler` создает `HelmReleases` для `Package`, он также:

- Создает namespaces, объявленные в полях component `Install.namespace`, и задает labels вроде `cozystack.io/system=true` и `pod-security.kubernetes.io/enforce=privileged`, где это необходимо.
- Передает cluster-wide configuration через Secret `cozystack-values`. `CozyValuesReplicator` следит за этим Secret в `cozy-system` и копирует его в каждый namespace с label `cozystack.io/system=true`. Каждый `HelmRelease` ссылается на этот Secret через `valuesFrom`, чтобы все компоненты получали согласованную конфигурацию платформы.

Update Flow

Когда новая версия chart публикуется в registry, обновления автоматически проходят через reconciliation chain:

flowchart TD

PUSH["Публикация новой версии chart
в OCI registry"]

FLUX["Flux обнаруживает изменение digest
(периодический polling)"]

REBUILD["ArtifactGenerator пересобирает
ExternalArtifact"]

UPGRADE["HelmRelease запускает
helm upgrade"]

DEPLOY["Новая версия развернута"]

PUSH --> FLUX

FLUX --> REBUILD

REBUILD --> UPGRADE

UPGRADE --> DEPLOY

Чтобы ускорить синхронизацию и не ждать следующего polling interval (Flux sources находятся в namespace `cozy-system`):

```
flux reconcile source oci <source-name> --namespace cozy-system
```

Чтобы изменить values приложения без смены версии chart, patch'ите `Package` CR напрямую. Values задаются отдельно для каждого component в `spec.components.<component-name>.values`:

```
kubectrl patch package <name> --type merge --patch '{"spec":{"components":{"<component-name>":{"values":{"key":"value"}}}}}'
```

Rollback Strategies

Есть три подхода к rollback Package, от наиболее до наименее рекомендуемого:

GitOps rollback (рекомендуется): опубликуйте предыдущую версию chart в OCI registry. Flux обнаружит изменение и запустит upgrade к “старой” версии через стандартный reconciliation flow.

flowchart LR

PUSH["Публикация предыдущего chart
в registry"]

FLUX["Flux обнаруживает изменение"]

UP["Helm upgrade
к предыдущей версии"]

OK["Rollback завершен"]

PUSH --> FLUX --> UP --> OK

Emergency rollback: выполните `helm rollback` напрямую и `suspend HelmRelease`, чтобы Flux не применил снова более новую версию. Это обходит GitOps и должно использоваться только в emergency-ситуациях.

flowchart LR

ROLLBACK["helm rollback
<release> <revision>"]

SUSPEND["flux suspend helmrelease
<name>"]

NOTE["Flux НЕ будет применять изменения
пока HelmRelease suspended"]

ROLLBACK --> SUSPEND --> NOTE

Controlled rollback: сначала `suspend HelmRelease`, затем выполните `helm rollback`, исправьте chart в registry и `resume HelmRelease`.

flowchart TD

S1["Suspend HelmRelease"]

S2["helm rollback"]

S3["Исправить chart в registry"]

S4["Resume HelmRelease"]

S5["Flux reconciles
с исправленным chart"]

S1 --> S2 --> S3 --> S4 --> S5

Автоматическое восстановление FluxPlunger

FluxPlunger — компонент автоматического восстановления, который обрабатывает распространенную ошибку HelmRelease “has no deployed releases”. Такая ошибка возникает, когда состояние release в Helm становится неконсистентным.

flowchart TD

ERR["HelmRelease переходит в error state
'has no deployed releases'"]

DETECT["FluxPlunger обнаруживает ошибку"]

FIND["Находит последний release Secret"]

SUSPEND["Suspends HelmRelease"]

DEL["Удаляет устаревший Secret"]

ANNOTATE["Записывает обработанную версию
в annotation (crash recovery)"]

RESUME["Resumes HelmRelease"]

REINSTALL["Flux выполняет чистую переустановку"]

ERR --> DETECT

DETECT --> FIND

FIND --> SUSPEND

SUSPEND --> DEL

DEL --> ANNOTATE

ANNOTATE --> RESUME

RESUME --> REINSTALL

Если FluxPlunger аварийно завершится в середине процесса, annotation

flux-plunger.cozystack.io/last-processed-version позволит ему корректно продолжить работу при следующей reconciliation.

Summary lifecycle operations

Действие	Что сделать	Кто обрабатывает
Обновить версию chart	Опубликовать новый chart в OCI registry	Flux + ArtifactGenerator
Обновить values	Patch Package CR	Package controller + HelmRelease
Ускорить синхронизацию	<code>flux reconcile source oci <name></code>	Ручной trigger
GitOps rollback	Опубликовать предыдущую версию chart в registry	Flux (standard flow)
Emergency rollback	<code>helm rollback \+ suspend</code> HelmRelease	Ручное вмешательство
Восстановиться после ошибки	Автоматически через FluxPlunger	FluxPlunger controller

cozyrpg CLI

`cozyrpg` — command-line tool для интерактивного управления ресурсами Package и PackageSource. Он выполняет dependency resolution, выбор variant и безопасное удаление с cascade analysis, чтобы вам не приходилось вручную писать YAML manifests.

Установка

Готовые binaries для Linux, macOS и Windows (amd64 и arm64) доступны в составе каждого релиза Cozystack.

Команды

`cozyrpg add` — установка packages

Устанавливает один или несколько packages с автоматическим dependency resolution:

```
cozyrpg add cozystack.keycloak cozystack.monitoring
```

```
cozyrpg add --file packages.yaml
```

Для каждого package команда `cozyrpg add`:

1. Находит соответствующий PackageSource в кластере.
2. Предлагает выбрать variant, если доступно несколько вариантов.
3. Разрешает все transitive dependencies (topological sort).
4. Создает Package resources в порядке dependencies-first, пропуская уже установленные packages.

`cozyrpg list` — список packages

```
cozyrpg list # доступные PackageSources
```

```
cozyrpg list --installed # установленные Packages
```

```
cozyrpg list --installed --components # установленные Packages с деталями components
```

Пример вывода:

```
NAME VARIANT READY STATUS
```

```
cozystack.networking cilium True reconciliation succeeded, generated 2 helmrelease(s)
```

```
cozystack.keycloak default False DependenciesNotReady
```

`cozyrpg del` — удаление packages

Безопасно удаляет packages с анализом reverse dependencies:

```
cozypkg del coystack.keycloak
```

Перед удалением `cozypkg del` показывает, какие другие установленные packages зависят от целевого, и запрашивает подтверждение. Packages удаляются в reverse topological order (сначала dependents).

`cozypkg dot` — визуализация dependencies

Генерирует dependency graph в формате GraphViz DOT:

```
cozypkg dot | dot -Tpng > dependencies.png
```

```
cozypkg dot --installed --components # component-level graph установленных packages
```

Отсутствующие dependencies подсвечиваются красным, что помогает быстро находить неполные установки.

Как cozypkg вписывается в lifecycle

flowchart LR

USER["Пользователь"]

CLI["cozypkg add"]

PKG["Package CR"]

CTRL["Package Controller"]

HR["HelmRelease"]

USER -->|"selects variant"| CLI

CLI -->|"creates"| PKG

PKG -->|"reconciled by"| CTRL

CTRL -->|"creates"| HR

`cozypkg` работает исключительно с custom resources `Package` и `PackageSource`. Он не взаимодействует с `HelmReleases`, `ArtifactGenerators` или `Flux sources` напрямую — ими управляют controllers, описанные выше.

Вы всегда можете управлять Package resources через `kubectl` вместо `cozypkg`. CLI лишь автоматизирует выбор variant, порядок dependencies и cascade analysis.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Архитектура Cozystack и стек платформы

<https://cozystack.ru/docs/v1.6/guides/platform-stack/>

Узнайте о ключевых компонентах, которые обеспечивают функциональность и гибкость Cozystack

Эта статья объясняет состав Cozystack через четыре слоя и показывает роль и ценность каждого компонента в platform stack.

Обзор

Чтобы понять состав Cozystack, удобно рассматривать его как набор подсистем, расположенных слоями от hardware до пользовательских сервисов:

Слой 1: ОС и hardware

Это базовый слой, который обеспечивает работу кластера на bare metal. Он состоит из Talos Linux и Kubernetes-кластера, установленного на Talos.

Talos Linux

Talos Linux — Linux-дистрибутив, созданный и оптимизированный для одной цели: запускать Kubernetes. Он обеспечивает основу надежности и безопасности в кластере Cozystack. Его использование позволяет Cozystack ограничить technology stack, повышая стабильность и безопасность.

Подробнее см. в разделе [Talos Linux](#).

Kubernetes

Kubernetes уже стал своего рода de facto standard для управления server workloads.

Одна из ключевых особенностей Kubernetes — удобный и единый API, понятный всем (все описывается YAML). Кроме того, Kubernetes использует лучшие software design patterns, обеспечивающие постоянное восстановление в любых ситуациях (reconciliation method) и эффективное масштабирование на большое количество серверов.

Это полностью решает проблему интеграции, так как существующие virtualization platforms обычно имеют устаревшие и довольно сложные APIs, которые нельзя расширять без изменения исходного кода. В результате часто приходится создавать собственные custom solutions, что требует дополнительных усилий.

Слой 2: Infrastructure Services

Второй слой содержит ключевые компоненты, отвечающие за storage, networking и virtualization. Добавление этих компонентов к базовому Kubernetes-кластеру делает его значительно функциональнее.

Flux CD

FluxCD предоставляет простой и единообразный интерфейс как для установки всех компонентов платформы, так и для управления их lifecycle. Разработчики Cozystack выбрали FluxCD как core element платформы, считая его новым industry standard для platform engineering.

KubeVirt

KubeVirt добавляет в Cozystack возможности virtualization. Он позволяет создавать virtual machines и worker nodes для tenant Kubernetes clusters.

KubeVirt — проект, начатый глобальными industry leaders с общим видением объединить Kubernetes и мир virtualization. Он расширяет возможности Kubernetes, предоставляя удобные abstractions для запуска и управления virtual machines, а также связанными сущностями: snapshots, presets, virtual volumes и другими.

Сейчас проект KubeVirt совместно развивают такие известные компании, как RedHat, NVIDIA и ARM.

DRBD и LINSTOR

DRBD и LINSTOR — основа replicated storage в Cozystack.

DRBD — самый быстрый replication block storage, работающий прямо в Linux kernel. DRBD отвечает только за репликацию данных, а для надежного хранения используются проверенные временем технологии, такие как LVM или ZFS. Kernel module DRBD включен в mainline Linux kernel и более десяти лет применяется для построения fault-tolerant systems.

DRBD управляется через LINSTOR — систему, интегрированную с Kubernetes. LINSTOR является management layer для создания virtual volumes на базе DRBD. Он позволяет управлять сотнями или тысячами virtual volumes в кластере Cozystack.

Kube-OVN

Сетевая функциональность Cozystack основана на Kube-OVN и Cilium.

OVN — свободная реализация virtual network fabric для Kubernetes и OpenStack на базе технологии Open vSwitch. С Kube-OVN вы получаете надежную и функциональную virtual network, которая обеспечивает изоляцию между tenants и предоставляет floating addresses для virtual machines.

В будущем это позволит бесшовно интегрироваться с другими clusters и customer network services.

Cilium

Использование Cilium вместе с OVN обеспечивает максимально эффективные и гибкие network policies, а также производительную services network в Kubernetes за счет offloaded Linux network stack на базе современной технологии eBPF.

Cilium — очень перспективный проект, широко используемый и поддерживаемый множеством cloud providers по всему миру.

Слой 3: Platform Services

Это компоненты, которые предоставляют пользовательскую функциональность Cozystack и его managed applications.

OpenAPI UI

OpenAPI UI предоставляет основной web interface для развертывания и управления приложениями в Cozystack. Он служит главным dashboard, через который пользователи взаимодействуют с Cozystack API в удобном интерфейсе.

Интерфейс построен поверх OpenAPI specifications Cozystack и автоматически генерирует forms и документацию для всех доступных managed applications. Пользователи могут разворачивать databases, Kubernetes clusters, virtual machines и другие services прямо через dashboard без необходимости вручную писать YAML manifests.

Dashboard также интегрируется с OIDC authentication через Keycloak, обеспечивая безопасный single sign-on доступ к платформе.

Kamaji

Cozystack использует Kamaji Control Plane для развертывания tenant Kubernetes clusters. Kamaji предоставляет простой и удобный способ запускать все необходимые Kubernetes control-plane components в containers. Worker nodes затем подключаются к этим control planes и выполняют пользовательские workloads.

Подход, разработанный проектом Kamaji, повторяет дизайн современных clouds и обеспечивает security by design: у конечных пользователей нет control plane nodes для их clusters.

Grafana

Grafana вместе с Grafana Loki и расширением OnCall предоставляет единый интерфейс для Observability. Он позволяет удобно просматривать charts, logs и управлять alerts для инфраструктуры и приложений.

Victoria Metrics

Victoria Metrics позволяет максимально эффективно собирать, хранить и обрабатывать metrics в формате Open Metrics, делая это эффективнее Prometheus в той же конфигурации.

MetalLB

MetalLB — load balancer по умолчанию для Kubernetes. С его помощью ваши services могут получать public addresses, доступные не только изнутри, но и снаружи сети вашего кластера.

HAProxy

HAProxy — продвинутый и широко известный TCP balancer. Он непрерывно проверяет доступность services и аккуратно балансирует production traffic между ними в real time.

См. справочник приложения: [TCP Balancer](#)

SeaweedFS

SeaweedFS — простая и хорошо масштабируемая distributed file system, созданная для двух основных целей: хранить миллиарды файлов и отдавать их быстрее. Она обеспечивает доступ O(1), обычно за одну операцию чтения с диска.

Kubernetes Operators

Cozystack включает набор Kubernetes operators, используемых для управления system services и managed applications.

Слой 4: Пользовательские сервисы

Cozystack поставляется с набором пользовательских приложений, заранее настроенных на надежность и resource efficiency, с включенными monitoring и observability:

- [Tenant Kubernetes clusters](#) — полнофункциональные managed Kubernetes clusters для development и production workloads.
- [Managed applications](#), например databases и queues.
- [Virtual machines](#) с поддержкой Linux и Windows OS.
- [Networking appliances](#), включая VPN, HTTP cache, TCP load balancer и virtual routers.

Managed Kubernetes

Cozystack разворачивает и управляет tenant Kubernetes clusters как самостоятельными приложениями внутри изолированной среды каждого tenant. Эти clusters полностью отделены от root management cluster и предназначены для развертывания tenant-specific или customer-developed applications.

Развертывание включает следующие компоненты:

- Kamaji Control Plane : [Kamaji](#) — open-source проект, который упрощает развертывание Kubernetes control planes как pods внутри root cluster. Каждый control plane pod включает

основные компоненты: `kube-apiserver`, `controller-manager` и `scheduler`, что обеспечивает эффективную multi-tenancy и использование ресурсов.

- Etcd Cluster : dedicated etcd cluster разворачивается с помощью [aenix-io/etcd-operator](#) от Aenix. Он предоставляет надежное и масштабируемое key-value storage для Kubernetes control plane.

- Worker Nodes : virtual machines создаются как worker nodes. Эти узлы настраиваются для подключения к tenant Kubernetes cluster, позволяя разворачивать и управлять workloads.

Такая архитектура обеспечивает изолированные, масштабируемые и эффективные Kubernetes environments для каждого tenant.

- Поддерживаемая версия: Kubernetes v1.32.4
- Operator: [aenix-io/etcd-operator](#) v0.4.2
- Справочник managed application: [Kubernetes](#)

Virtual Machines

В Cozystack virtualization features работают на базе [KubeVirt](#). Cozystack предоставляет несколько applications для virtualization functionality:

- [Virtual machine instance](#) с более расширенной конфигурацией.
- [Virtual machine disk](#) с выбором image sources.
- [VM image \(Golden Disk\)](#), который делает OS images локально доступными, ускоряя создание VM и экономя network traffic.

ClickHouse

ClickHouse — open source высокопроизводительная column-oriented SQL database management system (DBMS). Он используется для online analytical processing (OLAP). В платформе Cozystack для предоставления ClickHouse используется Altinity operator.

- Поддерживаемая версия: 24.9.2.42
- Kubernetes operator: [Altinity/clickhouse-operator](#) v0.25.0
- Website: [clickhouse.com](#)
- Справочник managed application: [ClickHouse](#)

Kafka

Apache Kafka — open-source distributed event streaming platform. Она предоставляет unified, high-throughput, low-latency platform для обработки real-time data feeds. Cozystack использует [Strimzi](#) для запуска Apache Kafka cluster в Kubernetes в разных deployment configurations.

- Поддерживаемая версия: Apache Kafka 3.9.0
- Kubernetes operator: [strimzi/strimzi-kafka-operator](#) v0.45.0

-
- Website: kafka.apache.org
 - Справочник managed application: [Kafka](#)

MariaDB (MySQL fork)

MySQL — широко используемая и хорошо известная relational database. Реализация в платформе позволяет создавать replicated MariaDB cluster. Этим cluster управляет набирающий популярность mariadb-operator.

Для каждой database доступен интерфейс настройки users, их permissions, а также schedules для создания backups с помощью [Restic](#) — одного из наиболее эффективных доступных инструментов.

- Поддерживаемая версия: MariaDB 11.4.3
- Kubernetes operator: [mariadb-operator/mariadb-operator](#) v0.18.0
- Website: mariadb.com
- Справочник managed application: [MySQL](#)

NATS Messaging

NATS — open-source, простая, безопасная и высокопроизводительная messaging system. Она предоставляет data layer для cloud native applications, IoT messaging и microservices architectures.

- Поддерживаемая версия: NATS 2.10.17
- Website: nats.io
- Справочник managed application: [NATS](#)

PostgreSQL

Сегодня PostgreSQL — самая популярная relational database. Ее platform-side реализация включает self-healing replicated cluster. Управление выполняется с помощью популярного в сообществе CloudNativePG operator.

- Поддерживаемая версия: PostgreSQL 17
- Kubernetes operator: [cloudnative-pg/cloudnative-pg](#) v1.24.0
- Website: cloudnative-pg.io
- Справочник managed application: [PostgreSQL](#)

RabbitMQ

RabbitMQ — широко известный message broker. Platform-side реализация позволяет создавать failover clusters под управлением официального RabbitMQ operator.

- Поддерживаемая версия: RabbitMQ 4.1.0+ (latest stable version)
- Kubernetes operator: [rabbitmq/cluster-operator](#) v1.10.0
- Website: rabbitmq.com

-
- Справочник managed application: [RabbitMQ](#)

Redis

Redis — наиболее часто используемое key-value in-memory data store. Чаще всего он применяется как cache, storage для user sessions или message broker. Platform-side реализация включает replicated failover Redis cluster с Sentinel. Им управляет [spotahome/redis-operator](#).

- Поддерживаемая версия: Redis 6.2.6+ (based on [alpine](#))
- Kubernetes operator: [spotahome/redis-operator](#) v1.3.0-rc1
- Website: [redis.io](#)
- Справочник managed application: [Redis](#)

VPN Service

VPN Service работает на базе Outline Server — продвинутого и удобного VPN solution. Внутри он известен как “Shadowbox”, что упрощает настройку и предоставление Shadowsocks servers. Он запускает Shadowsocks instances по запросу.

Протокол Shadowsocks использует symmetric encryption algorithms. Это обеспечивает быстрый доступ в интернет и затрудняет анализ и блокировку трафика через DPI (Deep Packet Inspection).

- Поддерживаемая версия: Outline Server, v1.12.3+ (stable)
- Website: [getoutline.org](#)
- Справочник managed application: [VPN](#)

HTTP Cache

HTTP caching service на базе Nginx помогает защитить приложение от перегрузки с помощью мощного Nginx. Nginx традиционно используется для построения CDNs и caching servers.

Platform-side реализация обеспечивает эффективное caching без использования clustered file system. Она также поддерживает horizontal scaling без дублирования данных на нескольких servers.

- Включенные версии: Nginx 1.25.3, HAProxy latest stable.
- Website: [nginx.org](#)
- Справочник managed application: [HTTP Cache](#)

TCP Balancer

Managed TCP Load Balancer service обеспечивает развертывание и управление load balancers. Он эффективно распределяет входящий TCP traffic между несколькими backend servers, обеспечивая high availability и optimal resource utilization.

TCP Load Balancer service работает на базе [HAProxy](#) — зрелого и надежного TCP load balancer.

-
- Справочник managed application: [TCP balancer](#)
 - Docs: [HAProxy Documentation](#)

Tenants

Tenants в Cozystack реализованы как managed applications. Подробнее о tenants см. в разделе [Tenant System](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Система tenant'ов

<https://cozystack.ru/docs/v1.6/guides/tenants/>

Узнайте о tenants и о том, как Cozystack помогает управлять ресурсами и повышать безопасность.

Введение

Tenant в Cozystack — основная единица изоляции и безопасности, похожая на Kubernetes namespace, но с расширенной областью действия. Каждый tenant представляет изолированную среду со своими ресурсами, сетью и RBAC (role-based access control). Некоторые cloud providers используют для похожей сущности термин “projects”.

Администраторы и пользователи Cozystack создают tenants с помощью приложения Tenant из application catalog. Tenants можно создавать через Cozystack dashboard (UI), `kubect1` или напрямую через Cozystack API.

Вложенность tenants

Все пользовательские tenants принадлежат базовому tenant'у `root`. Tenant `root` используется только для развертывания пользовательских tenants и системных компонентов. Все пользовательские приложения разворачиваются в соответствующих tenants.

Tenants могут быть вложенными: администратор tenant'а может создавать sub-tenants как приложения из каталога Cozystack. Parent tenants могут делиться ресурсами с children и наблюдать за их приложениями. В свою очередь, children могут использовать сервисы parent tenant.

Совместное использование cluster services

В tenants могут быть развернуты `cluster services`. Cluster services — это middleware services, предоставляющие базовую функциональность tenants и user-facing applications.

В tenant `root` по умолчанию есть набор сервисов вроде `etcd`, `ingress` и `monitoring`. Tenants нижнего уровня могут запускать собственные cluster services или использовать сервисы parent tenant.

Например, пользователь Cozystack создает следующие tenants и services:

- Tenant `foo` внутри tenant `root`, со своими экземплярами `etcd` и `monitoring`.
- Tenant `bar` внутри tenant `foo`, со своим экземпляром `etcd`.
- Tenant `Kubernetes cluster` и `Postgres database` в tenant `bar`.

Всем приложениям нужны сервисы вроде `ingress` и `monitoring`. Так как у tenant `bar` этих сервисов нет, приложения будут использовать сервисы parent tenant.

Эта конфигурация будет разрешена так:

-
- Tenant Kubernetes cluster будет хранить данные в собственном etcd сервиса tenant bar.
 - Все metrics будут собираться в monitoring stack parent tenant foo.
 - Доступ к приложениям будет идти через общий ingress, развернутый в tenant root.

Сетевая изоляция между tenants

Каждый namespace tenant'a изолирован от sibling tenants с помощью Cilium network policies, автоматически устанавливаемых chart'ом tenant. Отключить это для отдельного tenant нельзя: предыдущее поле `isolated` было удалено в Cozystack v1.0. Pod'ы внутри namespace tenant'a также по умолчанию не могут обращаться к kube-apiserver, а также к собственному etcd tenant'a, если tenant создан с `etcd: true`. Для доступа им нужно явно включить одну из двух labels pod'a:

- `policy.cozystack.io/allow-to-apiserver: "true"` — доступ к in-cluster Kubernetes API (для operators, dashboards и т. д.).
- `policy.cozystack.io/allow-to-etcd: "true"` — доступ к собственному etcd tenant'a (применимо только если tenant создан с `etcd: true`).

См. [Tenant `isolated` flag removed](<https://cozystack.ru/docs/v1.6/operations/upgrades/#tenant-isolated-flag-removed>) в upgrade notes, где приведен полный пример.

Настройка tenant services

Флаги tenant `etcd`, `monitoring`, `ingress` и `seaweedfs` устанавливают `_default_` конфигурацию каждого сервиса. После запуска сервиса вы можете изменить его spec: добавить storage pools, настроить resource quotas, переключить topology SeaweedFS на `MultiZone` и т. д., редактируя underlying application CR. Такие ручные изменения не перезаписываются при reconciliation parent Tenant.

Workflow состоит из двух шагов:

1. Включите flag в tenant (checkbox в Dashboard или `etcd: true / seaweedfs: true / ...` в `spec.values` manifest'a Tenant `HelmRelease`, который вы применяете через `kubectl`). Cozystack создаст соответствующий application CR со значениями по умолчанию.
2. Отредактируйте application CR на месте. Например, чтобы добавить pool в экземпляр SeaweedFS tenant-root:

```
kubectl edit -n tenant-root seaweedfses.apps.cozystack.io seaweedfs
```

Или patch'ите его non-interactively:

```
kubectl patch -n tenant-root seaweedfses.apps.cozystack.io seaweedfs \
--type=merge -p '{"spec":{"volume":{"pools":{"ssd":{"diskType":"ssd","size":"50Gi"}}}}}'
```

Та же схема применяется к каждому tenant-level application CR: `etcd`, `monitoring`, `ingress`, `seaweedfs`. См. [SeaweedFS storage pools](#) как подробный пример полного flow: включение SeaweedFS в tenant и последующая настройка созданного CR.

Не пытайтесь предварительно настроить tenant-level service, применяя его CR manifest _до_ создания tenant — вы получите ошибку “namespace not found”. Редактирование самого ресурса Tenant с попыткой вложить service-specific поля (например, SeaweedFS pools) в spec Tenant тоже не сработает: tenant-level flags являются booleans, а per-service spec находится в отдельном resource. Сначала включите flag, затем редактируйте downstream CR.

Уникальные доменные имена

У каждого tenant есть собственный домен. По умолчанию, если не указано иное, он наследует домен parent tenant с префиксом из своего имени. Например, если у tenant root домен example.org, то tenant foo по умолчанию получает домен foo.example.org. Однако его можно переопределить и указать другой домен, например example.com.

Kubernetes-кластеры, созданные в namespace этого tenant, получают домены вида kubernetes-cluster.foo.example.org.

Ограничения имен tenants

Имена tenants должны быть alphanumeric. Использовать дефисы (-) в именах tenants нельзя, в отличие от других сервисов. Это ограничение нужно, чтобы сохранить согласованное именование tenants, nested tenants и сервисов, развернутых внутри них.

Например:

- Root tenant называется root, но внутри он представлен как tenant-root.
- Пользовательский tenant называется foo, что дает tenant-foo.
- Однако tenant нельзя назвать foo-bar, потому что разбор имен вроде tenant-foo-bar может быть неоднозначным.

Namespace layout tenant’ов

Каждый tenant соответствует Kubernetes workload namespace. Tenant root — особый случай: его namespace жестко задан как tenant-root. Для каждого nested tenant namespace выводится из workload namespace его parent и собственного имени по двум правилам:

- Tenant, созданный напрямую внутри tenant-root, получает namespace tenant-<name>. Prefix parent tenant-root- не включается.
- Tenant, созданный на любой более глубокой вложенности, получает namespace <parent-workload-namespace>-<name>, где имя child добавляется к полному namespace parent.

Например, начиная с tenant-root:

Tenant path	Workload namespace
root	tenant-root
root/alpha	tenant-alpha

root/alpha/beta	tenant-alpha-beta
root/alpha/beta/gamma	tenant-alpha-beta-gamma

И Helm chart `tenant`, и aggregated API реализуют эти правила при создании нового tenant:

- helper Helm chart в `[packages/apps/tenant/templates/_helpers.tpl]` (https://github.com/cozystack/cozystack/blob/main/packages/apps/tenant/templates/_helpers.tpl) вычисляет namespace для устанавливаемого child release;
- функция `computeTenantNamespace` в `[pkg/registry/apps/application/rest.go]` (<https://github.com/cozystack/cozystack/blob/main/pkg/registry/apps/application/rest.go>) публикует то же значение как `status.namespace` в Tenant CR.

Так как сами имена tenants ограничены alphanumeric-символами (см. `_Ограничения имен tenants_` выше), фрагменты namespace никогда не содержат tenant-internal дефисов.

Имена Kubernetes namespaces являются RFC 1123 labels и не могут превышать 63 символа. Так как глубокие tenants накапливают всю цепочку ancestors в имени workload namespace (`tenant-alpha-beta-gamma`), длинные имена tenants в сочетании с глубокой вложенностью могут упереться в этот лимит. Планируйте иерархию заранее: короткие имена tenants на глубоких уровнях или более плоские trees, если длинные имена неизбежны. Kubernetes отклонит создание namespace, если вычисленное имя превысит 63 символа, а содержащий tenant Helm release покажет это отклонение как reconcile failure.

Определение parent и child relationships

Downstream integrations — custom dashboards, audit tooling, cost-allocation jobs, policy engines — иногда должны обходить tenant tree, чтобы показывать breadcrumbs, вычислять inherited settings или ограничивать scope запросов. Может показаться удобным выводить parent namespace, разбивая workload namespace по - и собирая его обратно без последнего segment. Сегодня это работает только потому, что имена tenants ограничены alphanumeric-символами, поэтому символ - однозначно разделяет ancestor segments; кроме того, такой способ предполагает, что текущие правила генерации namespace никогда не изменятся. Оба предположения являются implementation details, а не стабильным контрактом.

Стабильный контракт — сам custom resource `Tenant`. Cozystack хранит каждый Tenant CR в workload namespace его parent, поэтому:

- `**metadata.namespace` Tenant CR равен workload namespace parent**. Это надежный указатель на parent без string parsing.
- `**status.namespace` Tenant CR равен собственному** workload namespace tenant (там живут приложения tenant, nested tenants и `HelmRelease`).
- Чтобы получить direct children tenant с workload namespace `N`, перечислите Tenant CRs, у которых `metadata.namespace == N`. Через `kubectl` это одна команда к workload namespace parent:

```
kubectl get tenants --namespace <parent-workload-namespace>
```

Ресурс `tenants` обслуживается Cozystack aggregated API (`apps.cozystack.io/v1alpha1`), поэтому `cozystack-api` должен быть запущен и доступен клиенту. Выполните `kubectl api-resources --api-group apps.cozystack.io`, чтобы убедиться, что ресурс виден в вашем kubeconfig context.

Этот подход стабилен независимо от того, является `tenant` direct child `tenant-root` или более глубоким descendant, и переживет будущие изменения namespace layout, потому что вообще от него не зависит.

Справочник

См. справочник по приложению, реализующему управление tenants:
[tenant](https://cozystack.ru/docs/v1.6/applications/tenant/#parameters)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управление ресурсами в Cozystack

<https://cozystack.ru/docs/v1.6/guides/resource-management/>

Как CPU, memory и presets работают для VM, Kubernetes-кластеров и managed workloads в Cozystack; и как перенастраивать ресурсы через UI, CLI или API.

Введение

Cozystack запускает все, включая системные компоненты и пользовательские приложения, как сервисы в Kubernetes-кластере с конечным пулом CPU и memory.

В этом руководстве описано, как пользователи могут настраивать ресурсы, доступные приложению, и как Cozystack обрабатывает такую конфигурацию.

Настройка ресурсов сервиса

Ресурсы, доступные каждому сервису (managed application, VM или tenant cluster), задаются в его configuration file. В Cozystack есть два способа указать CPU time и memory, доступные сервису:

- С помощью resource presets.
- С помощью явной resource configuration.

Использование resource presets

Cozystack предоставляет набор именованных resource presets. У каждого пользовательского сервиса, включая managed applications, tenant Kubernetes clusters и virtual machines, есть preset по умолчанию.

При развертывании сервиса preset задается в переменной конфигурации `resourcesPreset`, например:

```
## @param resourcesPreset Default sizing preset used when resources is omitted.
```

```
## Allowed values: none, nano, micro, small, medium, large, xlarge, 2xlarge.
```

```
resourcesPreset: "small"
```

Название preset	CPU	memory
nano	100m	128Mi
micro	250m	256Mi
small	500m	512Mi
medium	500m	1Gi

large	1	2Gi
xlarge	2	4Gi
2xlarge	4	8Gi

Для CPU единица `m` означает 1/1000 полной CPU time.

Presets Cozystack определены во внутренней библиотеке `[cozy-lib]` (<https://github.com/cozystack/cozystack/tree/main/packages/library/cozy-lib>).

Явное определение ресурсов

Конфигурация сервиса может явно задавать доступные CPU и memory через переменную `resources`. В Cozystack используется простой формат resource configuration для `cpu` и `memory`:

@param resources Explicit CPU and memory configuration for each ClickHouse replica.

When left empty, the preset defined in `resourcesPreset` is applied.

resources:

cpu: 1

memory: 2Gi

Если одновременно заданы `resources` и `resourcesPreset`, используется `resources`, а `resourcesPreset` игнорируется.

Resource requests и limits

В Cozystack все запускается как Kubernetes services, а Kubernetes использует два важных механизма управления ресурсами: `requests` и `limits`. Сначала разберем, что это такое.

- Resource request определяет объем ресурса, который будет зарезервирован для сервиса и всегда предоставлен. Если ресурса недостаточно для выполнения request, сервис вообще не запустится.
- Resource limit определяет, сколько сервис может использовать из свободного пула ресурсов.

Подробное объяснение того, как `requests` и `limits` работают в Kubernetes, см. в разделе [Resource Management for Pods and Containers](#).

CPU time легко делится между несколькими сервисами с неравномерной CPU load. Поэтому распространенная практика — задавать низкие CPU requests и значительно более высокие limits. Для CPU-intensive сервисов оптимальным может быть соотношение 1:2 или 1:4. Для менее CPU-intensive сервисов даже 1:10 может дать хорошую resource efficiency и при этом быть достаточным.

Memory, наоборот, это ресурс, который после выдачи сервису обычно нельзя забрать обратно без OOM-kill сервиса. Поэтому memory requests обычно лучше задавать на уровне, который гарантирует работу сервиса.

CPU Allocation Ratio

В Cozystack есть единая configuration variable `cpuAllocationRatio`, являющаяся single source of truth. Она определяет соотношение между CPU requests и limits для всех сервисов.

CPU allocation ratio задается в Platform Package:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full
components:
platform:
values:
# ...
resources:
cpuAllocationRatio: 4
```

По умолчанию `cpuAllocationRatio` равен 10. Это значит, что CPU requests будут составлять 1/10 от CPU limits. Cozystack заимствует это значение по умолчанию из [KubeVirt](#).

Как Cozystack выводит CPU requests и limits

```
## @param resources Explicit CPU and memory configuration for each ClickHouse replica.
## When left empty, the preset defined in resourcesPreset is applied.
resources:
cpu: 1
## actual cpu limit: 1
## actual cpu request: (cpu / cpu-allocation-ratio)
memory: 2Gi
```

Пример 1, настройка по умолчанию: `cpu-allocation-ratio: 10`

Название preset	resources.cpu	фактический CPU request	фактический CPU limit
nano	100m	10m	100m
micro	250m	25m	250m
small	500m	50m	500m
medium	500m	50m	500m
large	1	100m	1
xlarge	2	200m	2
2xlarge	4	400m	4

Пример 2: cpu-allocation-ratio: 4

Название preset	resources.cpu	фактический CPU request	фактический CPU limit
nano	100m	25m	100m
micro	250m	63m	250m
small	500m	125m	500m
medium	500m	125m	500m
large	1	250m	1
xlarge	2	500m	2
2xlarge	4	1	4

Формат конфигурации до v0.31.0

До Cozystack v0.31.0 конфигурация сервисов позволяла пользователям явно задавать requests и limits. После обновления Cozystack с более ранних версий до v0.31.0 или новее такие сервисы не требуют немедленных действий.

При обновлении таких приложений пользователям нужно изменить конфигурацию на новый формат.

resources:

requests:

cpu: 250m

memory: 512Mi

limits:

cpu: 1

memory: 2Gi

У этого изменения было несколько причин.

Managed applications предполагают, что пользователю не нужно глубоко знать Kubernetes. Однако явная настройка requests/limits была “leaky abstraction”: она путала пользователей и приводила к misconfigurations.

Для hosting companies, запускающих public clouds на Cozystack, единое соотношение по всему cloud критически важно. Такой подход помогает обеспечить стабильный уровень сервиса и упрощает billing.

Пользователи, которые разворачивают собственные приложения в tenant Kubernetes clusters, по-прежнему могут задавать точные resource requests и limits.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Talos Linux в Cozystack

<https://cozystack.ru/docs/v1.6/guides/talos/>

Узнайте, почему Cozystack использует Talos Linux как основу Kubernetes-кластеров. Познакомьтесь с преимуществами Talos Linux: надежностью, масштабируемостью и оптимизацией под Kubernetes.

Почему Cozystack использует Talos Linux

Talos Linux — это Linux-дистрибутив, созданный и оптимизированный для одной задачи: запуска Kubernetes. Он является основой надежности и безопасности кластера Cozystack. Такой выбор позволяет Cozystack строго ограничить технологический стек и сделать систему максимально стабильной.

Посмотрим, почему разработчики Cozystack выбрали Talos как основу Kubernetes-кластера и что он дает платформе.

Надежный и простой

Talos Linux — immutable ОС, управляемая через API. В ней нет лишних движущихся частей: традиционного пакетного менеджера, привычной файловой структуры и возможности запускать что-либо, кроме Kubernetes-контейнеров.

Базовый слой платформы включает актуальную версию ядра, все необходимые kernel modules, container runtime и Kubernetes-like API для взаимодействия с системой. Обновление системы выполняется полной перезаписью образа Talos на диск “как есть”.

Масштабируемый и воспроизводимый

Talos Linux реализует принцип infrastructure-as-code. Talos настраивается через внешний декларативный manifest, который можно хранить в Git и переиспользовать для всех операций: повторного развертывания того же кластера, добавления узлов и других задач.

Когда вы находите оптимальную конфигурацию или решаете эксплуатационную проблему, вы один раз применяете изменение в manifest и сразу распространяете его на любое количество узлов, что упрощает масштабирование. Все узлы автоматически сходятся к одной и той же конфигурации, что устраняет configuration drift и делает troubleshooting предсказуемым.

Заточен под Kubernetes

Talos содержит встроенную логику для bootstrap и сопровождения Kubernetes-кластера, снижая сложность первой установки. Он обеспечивает полный lifecycle management как операционной системы, так и Kubernetes через единый набор команд `talosctl`, включая обновления, замену узлов и disaster recovery.

Настроен для Cozystack

Cozystack предоставляет подготовленную сборку Talos, в которую уже включены extensions и kernel modules, необходимые для storage, networking и observability stack. Благодаря этому кластеры готовы к production-сценариям сразу после запуска.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Сценарии использования

<https://cozystack.ru/docs/v1.6/guides/use-cases/>

Сценарии использования Cozystack.

-

[Использование Cozystack для построения public cloud](#)

Как использовать Cozystack для построения public cloud

[Использование Cozystack для построения private cloud](#)

Как использовать Cozystack для построения private cloud

[Build Your Own Platform \(BYOP\)](#)

Как собрать собственную платформу с Cozystack, устанавливая только нужные компоненты

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование Cozystack для построения public cloud

<https://cozystack.ru/docs/v1.6/guides/use-cases/public-cloud/>

Как использовать Cozystack для построения public cloud

Cozystack можно использовать как backend для public cloud.

Обзор

Cozystack позиционируется как framework для построения public cloud. Ключевое слово здесь — framework. В этом сценарии важно понимать, что Cozystack создан для cloud providers, а не для конечных пользователей напрямую.

Несмотря на наличие графического интерфейса, текущая модель безопасности не предполагает публичного пользовательского доступа к вашему management cluster.

Вместо этого конечные пользователи получают доступ к собственным Kubernetes-кластерам, могут заказывать LoadBalancers и дополнительные сервисы из них, но не имеют доступа к management cluster на базе Cozystack и ничего не знают о нем.

Таким образом, для интеграции с billing system достаточно научить вашу систему обращаться в management Kubernetes и размещать YAML-файл, описывающий нужный сервис. Остальную работу Cozystack выполнит за вас.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование Cozystack для построения private cloud

<https://cozystack.ru/docs/v1.6/guides/use-cases/private-cloud/>

Как использовать Cozystack для построения private cloud

Cozystack можно использовать как платформу для построения private cloud на основе Infrastructure-as-Code.

Обзор

Один из сценариев использования — self-service портал для пользователей внутри компании, где они могут заказывать нужный сервис или managed database.

Вы можете применять лучшие GitOps-практики: пользователи запускают собственные Kubernetes-кластеры и базы данных под свои задачи простым commit'ом конфигурации в ваш infrastructure Git repository.

Благодаря стандартизации подхода к разворачиванию приложений вы можете расширять возможности платформы с помощью обычных Helm charts.

Пример repository, показывающий настройку сервисов Cozystack через GitOps:

- <<https://github.com/aenix-io/cozystack-gitops-example>>

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Build Your Own Platform (BYOP)

<https://cozystack.ru/docs/v1.6/guides/use-cases/kubernetes-distribution/>

Как собрать собственную платформу с Cozystack, устанавливая только нужные компоненты

Cozystack можно использовать в режиме BYOP (Build Your Own Platform): устанавливать только нужные вам компоненты из package repository Cozystack, а не разворачивать всю платформу целиком.

Обзор

Cozystack предоставляет систему управления packages, вдохновленную пакетными менеджерами Linux-дистрибутивов. Cozystack Operator управляет ресурсами PackageSource и Package, а CLI-инструмент `cozyrpk` предоставляет интерактивный интерфейс для просмотра доступных packages, разрешения зависимостей и выборочной установки.

Такой подход полезен, если:

- У вас уже есть Kubernetes-кластер, и вам нужны только отдельные компоненты.
- В кластере уже настроены сеть и хранилище.
- Вы хотите полностью контролировать, какие компоненты устанавливаются.

Variant `default` у `cozystack-platform` не устанавливает компоненты — он только регистрирует PackageSources. После этого вы используете `cozyrpk`, чтобы устанавливать отдельные packages: `networking`, `storage`, `ingress`, `database operators` и другие.

Пошаговое руководство см. в разделе [установка BYOP](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Развертывание Cozystack

Руководство по развертыванию Cozystack: от инфраструктуры до готового кластера

<https://cozystack.ru/docs/v1.6/install/>

Узнайте, как развернуть кластер Cozystack с помощью Talos Linux и Kubernetes. В этом руководстве описаны установка, настройка и лучшие практики надежного и безопасного развертывания Cozystack.

Руководство по Cozystack

Если вы устанавливаете Cozystack впервые, рекомендуем [пройти руководство по Cozystack](#). В нем показан самый короткий путь к созданию proof-of-concept кластера Cozystack.

Общий путь установки

Установка Cozystack на bare-metal серверы или виртуальные машины состоит из трех последовательных шагов. Для каждого шага есть несколько вариантов. Мы указываем рекомендуемый вариант, но также приводим альтернативы, чтобы процесс установки оставался гибким:

1. [Установите Talos Linux](#) на bare-metal серверы или виртуальные машины с Linux либо без операционной системы.
2. [Установите и инициализируйте кластер Kubernetes](#) поверх Talos Linux.
3. [Установите и настройте Cozystack](#) в кластере Kubernetes.

Изолированная среда

Cozystack можно установить в изолированной среде без прямого доступа к Интернету. Главное отличие такой установки — использование проху registry для образов:

1. [Установите Talos Linux](#) на bare-metal серверы или виртуальные машины с Linux либо без операционной системы.
2. [Настройте узлы Talos для изолированной среды и инициализируйте кластер Kubernetes](#).
3. [Установите и настройте Cozystack](#) в кластере Kubernetes.

Автоматизированная установка с Ansible

Для типовых развертываний Linux (Ubuntu, Debian, RHEL, Rocky, openSUSE) [коллекция Ansible](#) автоматизирует весь процесс: подготовку ОС, инициализацию кластера k3s и установку Cozystack.

Установка у конкретных провайдеров

Для облачных провайдеров есть отдельные руководства, которые охватывают все шаги: от подготовки инфраструктуры до установки и настройки Cozystack. Если это ваш случай, рекомендуем использовать руководства ниже:

- [Hetzner](#)
- [Oracle Cloud Infrastructure \(OCI\)](#)
- [Servers.com](#)

Обновление и настройка после развертывания

После развертывания кластера перейдите в раздел [Администрирование кластера](#), чтобы выполнить следующие действия:

- [Настройка OIDC](#)
- [Развертывание Cozystack в конфигурации с несколькими дата-центрами](#)
- [Обновление Cozystack](#)

•

[Требования к оборудованию](#)

Определите требования к оборудованию для вашего сценария использования Cozystack.

[Рекомендации по планированию системных ресурсов](#)

Сколько системных ресурсов выделять на узел в зависимости от масштаба кластера.

[Установка Talos Linux на bare metal или виртуальные машины](#)

Шаг 1: установка Talos Linux на виртуальные машины или bare metal для последующей инициализации кластера Cozystack.

[Установка и настройка кластера Kubernetes](#)

Шаг 2: установка и настройка кластера Kubernetes, готового к установке Cozystack.

[Установка и настройка Cozystack](#)

Шаг 3: установка Cozystack в Kubernetes-кластер — как готовой к использованию платформы или в режиме BYOP (Build Your Own Platform).

[Развертывание кластера Cozystack в облаках и у хостинг-провайдеров](#)

Руководства по развертыванию кластеров Cozystack у конкретных облачных и хостинг-провайдеров.

[Автоматизированная установка с Ansible](#)

Развертывание Cozystack на generic Kubernetes с помощью Ansible collection `cozystack.installer`

[Руководства для особых случаев развертывания Cozystack](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Требования к оборудованию

<https://cozystack.ru/docs/v1.6/install/hardware-requirements/>

Определите требования к оборудованию для вашего сценария использования Cozystack.

Cozystack использует **Talos Linux** — минималистичный дистрибутив Linux, предназначенный исключительно для запуска Kubernetes. Обычно это означает, что сервер нельзя совместно использовать с сервисами, которые не запущены Cozystack. Хорошая новость в том, что какой бы сервис вам ни понадобился, Cozystack сможет запустить его корректно: безопасно, эффективно и в полностью контейнеризованной или виртуализованной среде.

Требования к оборудованию зависят от вашего сценария использования. Ниже приведены несколько распространенных вариантов развертывания; изучите их, чтобы определить, какая конфигурация лучше всего подходит под ваши задачи.

- Минимальная
- Рекомендуемая
- Оптимальная

Ниже приведены базовые требования для запуска небольшой инсталляции. Минимальная рекомендуемая конфигурация для каждого узла:

Компонент	Требование
Хосты	3х физических хоста (или VM с host CPU passthrough)
Architecture	x86_64
CPU	8 ядер
RAM	24 ГБ
Основной диск	50 ГБ SSD (или RAW для VM)
Дополнительный диск	256 ГБ SSD (raw)

Подходит для:

- Dev/Test-сред
- Небольших демонстрационных конфигураций
- 1-2 tenants
- До 3 кластеров Kubernetes
- Небольшого числа VM или баз данных

Для небольших production-сред рекомендуемая конфигурация каждого узла выглядит так:

Компонент	Требование
-----------	------------

Хосты	3х физических хоста
Architecture	x86_64
CPU	16-32 ядра
RAM	64 ГБ
Основной диск	100 ГБ SSD или NVMe
Дополнительный диск	1-2 ТБ SSD или NVMe

Подходит для:

- Небольших и средних production-сред
- 5-10 tenants
- 5+ кластеров Kubernetes
- Десятков виртуальных машин или баз данных
- S3-совместимого хранилища

Для средних и крупных production-сред оптимальная конфигурация каждого узла выглядит так:

Компонент	Требование
Хосты	6х+ физических хостов
Architecture	x86_64
CPU	32-64 ядра
RAM	128-256 ГБ
Основной диск	200 ГБ SSD или NVMe
Дополнительный диск	4-10 ТБ NVMe

Подходит для:

- Крупных production-сред
- 20+ tenants
- Десятков кластеров Kubernetes
- Сотен виртуальных машин и баз данных
- S3-совместимого хранилища

Вычислительные ресурсы:

- Три или более физических либо виртуальных сервера с архитектурой amd64/x86_64 и характеристиками, указанными в таблице выше.
- Для виртуализированных серверов должна быть включена вложенная виртуализация, а модель CPU должна быть установлена в `host` (без эмуляции).

-
- Для установки через PXE требуется дополнительный управляющий инстанс, подключенный к той же сети, с любой Linux-системой, способной запускать Docker-контейнер. Он также должен иметь архитектуру x86-64-v2; в случае виртуальной машины этого, скорее всего, можно добиться, установив модель CPU в `host`.

Хранилище:

Хранилище в кластере Cozystack используется как системой, так и пользовательскими workload'ами. Есть два варианта: выделить отдельный диск для каждой роли или выделить место под пользовательское хранилище на системном диске. Для хранилища узлов control plane критически важна низкая задержка, поэтому рекомендуются локальные SSD.

Использование двух дисков

Разделение дисков по ролям — основной и более надежный вариант.

- Основной диск : на этом диске находятся операционная система Talos Linux, необходимые системные модули ядра, базовые системные поды Cozystack, логи и базовые контейнерные образы. Также поверх него будет работать кластер etcd, поэтому следует использовать том с низкой задержкой, предпочтительно локальный SSD.

Минимальные размеры зависят от конфигурации (см. таблицу выше). Установка Talos ожидает `/dev/sda` в качестве системного диска (диски virtio обычно отображаются как `/dev/vda`).

- Дополнительный диск : предназначен для данных workload'ов, его размер можно увеличивать в зависимости от требований workload'ов. Используется для выделения томов через PersistentVolumeClaims (PVC).

Минимальные размеры зависят от конфигурации (см. таблицу выше). Путь к диску (обычно `/dev/sdb`) будет указан в конфигурации хранилища. Он не влияет на установку Talos.

Подробнее о настройке Linstor StorageClass см. в [руководстве по разворачиванию Cozystack](#)

Использование одного диска

Можно использовать один диск с выделенным пространством для пользовательского хранилища. См. [Как установить Talos на машину с одним диском](#) Рекомендуется использовать локальный SSD-диск.

Сеть:

- Машинам должно быть разрешено использовать дополнительные IP-адреса, либо должен быть доступен внешний балансировщик нагрузки. Использование дополнительных IP-адресов по умолчанию отключено и в большинстве публичных облаков должно включаться явно.
- Могут потребоваться дополнительные публичные IP-адреса для ingress и виртуальных машин. Проверьте, поддерживает ли ваш облачный провайдер floating IP.
- Маршрутизируемый FQDN-домен (или используйте [nip.io](#) с записью через дефис)
- Размещение в одном L2-сегменте сети

-
- Отключенный anti-spoofing (требуется для MetalLB)
 - Минимум 1 Гбит/с (для production рекомендуется 10 Гбит/с)
 - Низкая задержка между узлами кластера

Production-кластер

Для production-среды учитывайте следующее:

Вычислительные ресурсы:

- Для запуска высокодоступных приложений обязательно иметь как минимум три worker-узла. Если один из трех узлов станет недоступен из-за аппаратного сбоя или обслуживания, кластер будет работать в деградированном состоянии. Кластеры баз данных и реплицированное хранилище продолжат работать, но запуск новых экземпляров баз данных или создание реплицированных томов будет невозможным.
- Настоятельно рекомендуется, хотя и не обязательно, использовать отдельные серверы для master-узлов Kubernetes. Чистый worker-узел гораздо проще выводить из эксплуатации для обслуживания или обновления, чем узел, который одновременно является управляющим.

Сеть:

- В конфигурации с несколькими дата-центрами оптимально иметь прямые выделенные оптические каналы между ними.
- Серверы должны поддерживать out-of-band management (IPMI, iLO, iDRAC и т. д.) для удаленного мониторинга, восстановления и управления.

Распределенный кластер

С помощью Cozystack можно построить [распределенный кластер](#).

Сеть:

- Для распределенного кластера требуется быстрая и надежная сеть, и у нее обязательно должен быть низкий RTT (Round Trip Time), так как Kubernetes не рассчитан на эффективную работу в сетях с высокой задержкой.

Дата-центры в пределах одного города обычно имеют задержку менее 1 мс, что является идеальным вариантом. _Максимально допустимый_ RTT — 10 мс. Запуск Kubernetes или реплицированного хранилища в сети с RTT выше 20 мс настоятельно не рекомендуется. Для измерения фактического RTT можно использовать команду `ping`.

- Также рекомендуется иметь как минимум 2–3 узла на дата-центр в распределенном кластере. Это гарантирует, что кластер сможет пережить потерю одного дата-центра без серьезных нарушений работы.

-
- Если сложно поддерживать единое адресное пространство между дата-центрами, вместо внешнего VPN можно включить KubeSpan — функцию Talos Linux, которая создает full-mesh VPN между узлами на базе WireGuard.

Высокодоступные приложения

Обеспечение высокой доступности добавляет требования к базовой production-среде.

Сеть:

- Рекомендуется иметь несколько сетевых карт 10 Гбит/с или быстрее. Трафик хранилища и приложений можно разделить, назначив их разным сетевым интерфейсам.
- Ожидайте значительный объем горизонтального межузлового трафика внутри кластеров. Обычно он возникает из-за обмена данными между несколькими репликами сервисов и баз данных, развернутыми на разных узлах. Кроме того, виртуальные машины с live migration требуют реплицированных томов, что дополнительно увеличивает объем трафика.

Планирование системных ресурсов

Подробные рекомендации по выделению системных ресурсов (CPU и памяти) на узел с учетом масштаба кластера и количества tenants см. в разделе [Рекомендации по планированию системных ресурсов](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Рекомендации по планированию системных ресурсов

<https://cozystack.ru/docs/v1.6/install/resource-planning/>

Сколько системных ресурсов выделять на узел в зависимости от масштаба кластера.

Это руководство помогает спланировать выделение системных ресурсов на узел с учетом размера кластера и количества tenants. Рекомендации основаны на production-развертываниях и дают достаточно точные оценки для планирования.

Важно: указанные значения относятся только к системным компонентам. К ним нужно добавить требования tenant workload'ов (приложений, баз данных, кластеров Kubernetes, виртуальных машин и т. д.).

Быстрый старт : выделите как минимум 2 ядра CPU и 6 ГБ RAM на узел для системных компонентов. Для точного расчета требований с учетом размера кластера и количества tenants используйте таблицу или калькулятор ниже.

Примечание о выделении ресурсов : эти значения отражают ожидаемое потребление при нормальной работе, а не жесткое резервирование ресурсов. Kubernetes динамически планирует workload'ы, а системные компоненты будут потреблять примерно эти объемы, при этом оставшаяся емкость останется доступной для tenant workload'ов.

Требования к ресурсам

Требования зависят как от размера кластера (количества узлов), так и от количества tenants. Если у каждого tenant много активных сервисов (5+), ориентируйтесь на значения из следующей категории tenants.

Размер кластера	Узлы	До 5 tenants	6-14 tenants	15-30 tenants	31+ tenants
Малый	3-5	CPU: 2 ядра			
RAM: 6 ГБ	CPU: 2 ядра				
RAM: 6 ГБ	CPU: 3 ядра				
RAM: 10 ГБ	CPU: 3 ядра				

RAM: 15 ГБ

Средний | 6-10 | CPU: 3 ядра

RAM: 7 ГБ | CPU: 3 ядра

RAM: 7 ГБ | CPU: 3 ядра

RAM: 12 ГБ | CPU: 4 ядра

RAM: 18 ГБ

Крупный | 11+ | CPU: 3 ядра

RAM: 9 ГБ | CPU: 3 ядра

RAM: 10 ГБ | CPU: 4 ядра

RAM: 15 ГБ | CPU: 4 ядра

RAM: 22 ГБ

Советы по планированию:

- Отслеживайте фактическое потребление ресурсов и корректируйте значения при необходимости
- Закладывайте запас на рост 20-30%
- При высокой активности tenants рассмотрите увеличение CPU на 50-100%, а памяти — на 100-300%

Рассчитайте свои требования

Используйте калькулятор ниже, чтобы определить требования для вашей конкретной конфигурации:

Nodes: 5

Tenants: 5

CPU per Node

2 cores

RAM per Node

6 GB

Почему требования к ресурсам масштабируются

Потребление системных ресурсов растёт вместе с размером кластера и количеством tenants, потому что системным компонентам нужно обрабатывать больше объектов Kubernetes для мониторинга, применять больше сетевых политик и собирать и обрабатывать больше логов.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Talos Linux на bare metal или виртуальные машины

<https://cozystack.ru/docs/v1.6/install/talos/>

Шаг 1: установка Talos Linux на виртуальные машины или bare metal для последующей инициализации кластера Cozystack.

Первый шаг развертывания кластера Cozystack — установка Talos Linux на bare-metal серверы или виртуальные машины. Перед началом убедитесь, что виртуальные машины или bare-metal серверы уже подготовлены. Для планирования установки см. [требования к оборудованию](#).

Если вы устанавливаете Cozystack впервые, рекомендуем [начать с руководства по Cozystack](#).

Варианты установки

Есть несколько способов установить Talos на bare-metal сервер или виртуальную машину. У них разные ограничения и оптимальные сценарии использования:

- Рекомендуется: [загрузка в Talos Linux из другой Linux ОС с помощью [boot-to-talos](https://cozystack.ru/docs/v1.6/install/talos/boot-to-talos/)](<https://cozystack.ru/docs/v1.6/install/talos/boot-to-talos/>) — простой способ установки, который полностью работает из userspace и не требует внешних зависимостей, кроме образа Talos.

Выберите этот вариант, если вы впервые работаете с Talos или если у вас есть VM с предустановленной ОС от облачного провайдера.

- [Установка с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах](#) — требует дополнительной управляющей машины, но позволяет устанавливать систему сразу на несколько хостов.
- [Установка с помощью ISO-образа](#) — оптимальна для систем, которые умеют автоматизировать установку с ISO.

Следующие шаги

- После установки Talos Linux у вас будет набор узлов, готовых к следующему шагу: [установке и инициализации кластера Kubernetes](#).
- Прочитайте [обзор Talos Linux](#), чтобы узнать, почему Talos Linux является оптимальным выбором ОС для Cozystack и что он дает платформе.

•

[Установка Talos Linux с помощью boot-to-talos](#)

Установите Talos Linux с помощью `boot-to-talos` — удобного CLI-приложения, которому нужен только образ Talos.

[Установка Talos Linux с помощью PXE](#)

Как установить Talos Linux с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах.

[Установка Talos Linux с помощью ISO](#)

Как установить Talos Linux с помощью ISO

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Talos Linux с помощью boot-to-talos

<https://cozystack.ru/docs/v1.6/install/talos/boot-to-talos/>

Установите Talos Linux с помощью `boot-to-talos` — удобного CLI-приложения, которому нужен только образ Talos.

В этом руководстве описано, как установить Talos Linux на хост с любым другим дистрибутивом Linux с помощью `boot-to-talos`.

`boot-to-talos` создан командой Cozystack, чтобы помочь пользователям и командам, внедряющим Cozystack, установить Talos — самый сложный шаг процесса. Он полностью работает из `userspace` и не имеет внешних зависимостей, кроме установочного образа Talos.

Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Совместимость версий

При установке Cozystack на Talos должны совпасть три версии:

Компонент	Откуда берется	Что должно совпадать
Talos на узле	флаг <code>-image</code> , переданный в <code>boot-to-talos</code>	версия Talos, поставляемая с релизом Cozystack, который вы устанавливаете
<code>talosctl</code> на вашей рабочей станции	скачивается отдельно из релизов siderolabs/talos	major.minor версии Talos, записанной на узел
Cozystack	флаг <code>--version</code> , переданный в <code>helm upgrade --install cozy-installer</code>	— (основа; все остальное следует за ней)

Для Cozystack 1.6.0 закреплённая версия Talos — v1.13.6 (`[packages/core/talos/images/talos/profiles/installer.yaml]`(<https://github.com/cozystack/cozystack/blob/v1.6.0/packages/core/talos/images/talos/profiles/installer.yaml>)). Используйте `ghcr.io/cozystack/cozystack/talos:v1.13.6` как образ для `boot-to-talos` и скачайте `talosctl v1.13.x`.

`boot-to-talos v0.7.x` содержит собственный жестко заданный образ по умолчанию (`ghcr.io/cozystack/cozystack/talos:v1.11.6` в v0.7.1, см.

`[cmd/boot-to-talos/main.go]`(<https://github.com/cozystack/boot-to-talos/blob/v0.7.1/cmd/boot-to-talos/main.go>)). Если в интерактивном prompt оставить это значение по умолчанию для кластера, на котором вы планируете запускать Cozystack 1.6.0, в итоге вы получите узел Talos v1.11, тогда как `installer` Cozystack и шаблоны `Talm` рассчитаны на Talos v1.13 — вы столкнетесь с несовпадением

на этапе bootstrap. Всегда вводите образ, соответствующий целевому релизу Cozystack (или передавайте `-image` в командной строке).

Режимы

`boot-to-talos` поддерживает два режима установки:

1. `boot` – извлекает `kernel` и `initrd` из Talos installer и загружает их напрямую с помощью механизма `kexec`.
2. `install` – подготавливает окружение, запускает Talos installer, а затем перезаписывает системный диск установленным образом.

Примечание: Если один режим не работает в вашей системе, попробуйте другой. Разные методы могут лучше работать на разных операционных системах.

Установка

1\ Установка `boot-to-talos`

- Используйте установочный скрипт:

<code>curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh</code>	<code>sh -s</code>
---	--------------------

- Скачайте бинарный файл со [страницы релизов GitHub](#):

```
wget https://github.com/cozystack/boot-to-talos/releases/latest/download/boot-to-talos-linux-amd64.tar.gz
```

2\ Запуск установки Talos

Запустите `boot-to-talos` и укажите значения конфигурации. Обязательно используйте собственную сборку Talos от Cozystack, доступную по адресу ghcr.io/cozystack/cozystack/talos.

Mode:

1. `boot` – extract the kernel and `initrd` from the Talos installer and boot them directly using the `kexec` mechanism.
2. `install` – prepare the environment, run the Talos installer, and then overwrite the system disk with the installed image.

Mode [1]: 2

Target disk [/dev/sda]:

Talos installer image [ghcr.io/cozystack/cozystack/talos:v1.11.6]:

ghcr.io/cozystack/cozystack/talos:v1.13.6

Add networking configuration? [yes]:

Interface [eth0]:

IP address [10.0.2.15]:

Netmask [255.255.255.0]:

Gateway (or 'none') [10.0.2.2]:

Configure serial console? (or 'no') [ttyS0]:

Summary:

Image: ghcr.io/cozystack/cozystack/talos:v1.13.6

Disk: /dev/sda

Extra kernel args: ip=10.0.2.15::10.0.2.2:255.255.255.0::eth0:::: console=ttyS0

WARNING: ALL DATA ON /dev/sda WILL BE ERASED!

Continue? [yes]:

2025/08/03 00:11:03 created temporary directory /tmp/installer-3221603450

2025/08/03 00:11:03 pulling image ghcr.io/cozystack/cozystack/talos:v1.13.6

2025/08/03 00:11:03 extracting image layers

2025/08/03 00:11:07 creating raw disk /tmp/installer-3221603450/image.raw (2 GiB)

2025/08/03 00:11:07 attached /tmp/installer-3221603450/image.raw to /dev/loop0

2025/08/03 00:11:07 starting Talos installer

2025/08/03 00:11:07 running Talos installer v1.13.6

2025/08/03 00:11:07 WARNING: config validation:

2025/08/03 00:11:07 use "worker" instead of "" for machine type

2025/08/03 00:11:07 created EFI (C12A7328-F81F-11D2-BA4B-00A0C93EC93B) size 104857600 bytes

2025/08/03 00:11:07 created BIOS (21686148-6449-6E6F-744E-656564454649) size 1048576 bytes

2025/08/03 00:11:07 created BOOT (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576000 bytes

2025/08/03 00:11:07 created META (0FC63DAF-8483-4772-8E79-3D69D8477DE4) size 1048576 bytes

2025/08/03 00:11:07 formatting the partition "/dev/loop0p1" as "vfat" with label "EFI"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p2" as "zeroes" with label "BIOS"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p3" as "xfs" with label "BOOT"

2025/08/03 00:11:07 formatting the partition "/dev/loop0p4" as "zeroes" with label "META"

2025/08/03 00:11:07 copying from io reader to /boot/A/vmlinuz

2025/08/03 00:11:07 copying from io reader to /boot/A/initramfs.xz

2025/08/03 00:11:08 writing /boot/grub/grub.cfg to disk

```
2025/08/03 00:11:08 executing: grub-install --boot-directory=/boot --removable --efi-directory=/boot/EFI
/dev/loop0
2025/08/03 00:11:08 installation of v1.13.6 complete
2025/08/03 00:11:08 Talos installer finished successfully
2025/08/03 00:11:08 remounting all filesystems read-only
2025/08/03 00:11:08 copy /tmp/installer-3221603450/image.raw -> /dev/sda
2025/08/03 00:11:19 installation image copied to /dev/sda
2025/08/03 00:11:19 rebooting system
```

О приложении

`boot-to-talos` — это open source проект, размещенный на github.com/cozystack/boot-to-talos. Он включает CLI, написанный на Go, и установочный скрипт на Bash. Доступны сборки для нескольких архитектур:

- `linux-amd64`
- `linux-arm64`
- `linux-i386`

Как это работает

Понимать эти шаги для установки Talos Linux необязательно.

Рабочий процесс зависит от выбранного режима:

Режим `boot`

При использовании режима `boot` `boot-to-talos` выполняет следующие шаги:

1. Распаковывает Talos installer в RAM

Извлекает слои из контейнера Talos installer во временный `tmpfs`. Обратите внимание, что Docker на этом шаге не нужен.

2. Извлекает kernel и `initrd`

Извлекает kernel (`vmlinux`) и initial ramdisk (`initramfs.xz`) из образа Talos installer.

3. Загружает kernel через `kexec`

Использует системный вызов `kexec`, чтобы загрузить kernel Talos и `initrd` в память с переданными параметрами командной строки kernel.

4. Перезагружает в Talos

Выполняет `kexec - -exec`, чтобы переключиться на kernel Talos без физической перезагрузки. После загрузки можно применить конфигурацию Talos и завершить установку.

Режим install

При использовании режима `install boot-to-talos` выполняет следующие шаги:

1. Распаковывает Talos installer в RAM

Извлекает слои из контейнера Talos installer во временный `tmpfs`. Обратите внимание, что Docker на этом шаге не нужен.

2. Собирает системный образ

Создает sparse-файл `image.raw`, подключенный через loop device, и выполняет Talos `_installer_` внутри chroot. Затем installer размечает диск, форматирует разделы и размещает GRUB и системные файлы.

3. Записывает поток на диск

Копирует `image.raw` на выбранное блочное устройство блоками по 4 MiB и выполняет `fsync` после каждой записи, чтобы данные были полностью сохранены до перезагрузки.

4. Перезагружает систему

Команда `echo b > /proc/sysrq-trigger` выполняет немедленную перезагрузку в только что установленный Talos Linux.

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Talos Linux с помощью PXE

<https://cozystack.ru/docs/v1.6/install/talos/pxe/>

Как установить Talos Linux с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах.

В этом руководстве описано, как установить Talos Linux на bare-metal серверы или виртуальные машины с помощью временных DHCP- и PXE-серверов, запущенных в Docker-контейнерах. Этот метод требует дополнительной управляющей машины, но позволяет устанавливать систему сразу на несколько хостов.

Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Зависимости

Для установки Talos этим способом на управляющем хосте потребуются следующие зависимости:

- docker
- kubectl

Обзор инфраструктуры

Установка

Запустите matchbox с готовым образом Talos для Cozystack:

```
sudo docker run --name=matchbox -d --net=host ghcr.io/cozystack/cozystack/matchbox:v0.30.0 \
-address=:8080 \
-log-level=debug
```

Запустите DHCP-сервер:

```
sudo docker run --name=dnsmaq -d --cap-add=NET_ADMIN --net=host
quay.io/poseidon/dnsmaq:v0.5.0-32-g4327d60-amd64 \
-d -q -p0 \
--dhcp-range=192.168.100.3,192.168.100.199 \
--dhcp-option=option:router,192.168.100.1 \
--enable-tftp \
```

```
--tftp-root=/var/lib/tftpboot \  
--dhcp-match=set:bios,option:client-arch,0 \  
--dhcp-boot=tag:bios,undionly.kpxe \  
--dhcp-match=set:efi32,option:client-arch,6 \  
--dhcp-boot=tag:efi32,ipxe.efi \  
--dhcp-match=set:efibc,option:client-arch,7 \  
--dhcp-boot=tag:efibc,ipxe.efi \  
--dhcp-match=set:efi64,option:client-arch,9 \  
--dhcp-boot=tag:efi64,ipxe.efi \  
--dhcp-userclass=set:ipxe,iPXE \  
--dhcp-boot=tag:ipxe,http://192.168.100.254:8080/boot.ipxe \  
--log-queries \  
--log-dhcp
```

Для установки в изолированной среде добавьте NTP- и DNS-серверы:

```
--dhcp-option=option:ntp-server,10.100.1.1 \  
--dhcp-option=option:dns-server,10.100.25.253,10.100.25.254 \  

```

Где:

- 192.168.100.3, 192.168.100.199 — диапазон для выделения IP-адресов
- 192.168.100.1 — ваш gateway
- 192.168.100.254 — адрес вашего управляющего сервера

Проверьте состояние контейнеров:

```
docker ps
```

пример вывода:

```
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

```
06115f09e689 quay.io/poseidon/dnsmasq:v0.5.0-32-g4327d60-amd64 "/usr/sbin/dnsmasq -..." 47  
seconds ago Up 46 seconds dnsmasq
```

```
6bf638f0808e ghcr.io/cozystack/cozystack/matchbox:v0.30.0 "/matchbox -address=..." 3 minutes ago Up  
3 minutes matchbox
```

Запустите серверы. Теперь они должны автоматически загрузиться с вашего PXE-сервера.

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Talos Linux с помощью ISO

<https://cozystack.ru/docs/v1.6/install/talos/iso/>

Как установить Talos Linux с помощью ISO

В этом руководстве описано, как установить Talos Linux на bare-metal серверы или виртуальные машины. Обратите внимание, что Cozystack предоставляет собственные сборки Talos, протестированные и оптимизированные для запуска кластера Cozystack.

Установка

1. Скачайте ISO Talos Linux для Cozystack v1.6.0 со [страницы релиза](#).

```
wget https://github.com/cozystack/cozystack/releases/download/v1.6.0/metal-amd64.iso
```

2. Загрузите машину с подключенным ISO.

3. Нажмите **и заполните сетевые настройки:

Следующие шаги

После установки Talos перейдите к [установке и инициализации кластера Kubernetes](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка и настройка кластера Kubernetes

<https://cozystack.ru/docs/v1.6/install/kubernetes/>

Шаг 2: установка и настройка кластера Kubernetes, готового к установке Cozystack.

Второй шаг развертывания кластера Cozystack — установка и настройка кластера Kubernetes. В результате вы получите установленный и настроенный кластер Kubernetes, готовый к установке Cozystack.

Если вы устанавливаете Cozystack впервые, [начните с руководства по Cozystack](#).

Варианты установки

Talos Linux (рекомендуется)

Для production-развертываний Cozystack рекомендует использовать [Talos Linux](#) в качестве базовой операционной системы. Предварительное условие для этих методов — [установленный Talos Linux](#).

Есть несколько способов настроить узлы Talos и инициализировать кластер Kubernetes:

- Рекомендуется : [использование Talm](#) — декларативного CLI-инструмента с готовыми пресетами для Cozystack, который использует возможности Talos API.
- [\[Использование talos-bootstrap\]\(https://cozystack.ru/docs/v1.6/install/kubernetes/talos-bootstrap/\)](#) — интерактивного скрипта для инициализации кластеров Kubernetes на Talos OS.
- [Использование talosctl](#) — специализированного CLI-инструмента для управления Talos.
- [Установка в изолированной среде](#) возможна с Talm или talosctl.

Generic Kubernetes

Cozystack также можно развернуть на других дистрибутивах Kubernetes:

- [Generic Kubernetes](#) — развертывание Cozystack на k3s, kubeadm, RKE2 или других дистрибутивах.

Если при установке возникнут проблемы, см. [раздел Troubleshooting](#).

Следующие шаги

- После установки и настройки кластера Kubernetes можно [установить и настроить Cozystack](#).

.

[Использование Talm для инициализации кластера Cozystack](#)

`talm` — декларативный CLI-инструмент, созданный разработчиками Cozystack и оптимизированный для развертывания Cozystack.

Рекомендуется для `infrastructure-as-code` и `GitOps`.

[Использование скрипта talos-bootstrap для инициализации кластера Cozystack](#)

`talos-bootstrap` — CLI для пошаговой инициализации кластера, созданный разработчиками Cozystack.

Рекомендуется для первых развертываний.

[Использование talosctl для инициализации кластера Cozystack](#)

`talosctl` — стандартный CLI Talos Linux: он требует больше boilerplate-кода, но дает полную гибкость настройки.

[Инициализация кластера в изолированной среде](#)

Инициализация кластера Cozystack в изолированной (`air-gapped`) среде с `mirror`’ами `container registry`.

[Устранение неполадок при установке Kubernetes](#)

Инструкции по решению типовых проблем, которые могут возникнуть при установке Kubernetes с помощью `talm`, `talos-bootstrap` или `talosctl`.

[Развертывание Cozystack на Generic Kubernetes](#)

Как развернуть Cozystack на `k3s`, `kubeadm`, `RKE2` или других дистрибутивах Kubernetes без Talos Linux

[Ubuntu Secure Boot: DRBD prerequisites](#)

Prepare Ubuntu LTS hosts with UEFI Secure Boot enabled for Cozystack non-Talos installation by pre-installing `drbd-dkms` from the LINBIT PPA

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование Talm для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.6/install/kubernetes/talm/>

`talm` — декларативный CLI-инструмент, созданный разработчиками Cozystack и оптимизированный для развертывания Cozystack.

Рекомендуется для `infrastructure-as-code` и `GitOps`.

В этом руководстве описано, как установить и настроить Kubernetes в кластере Talos Linux с помощью Talm. После выполнения этого руководства у вас будет кластер Kubernetes, готовый к установке Cozystack.

Talm — Helm-подобная утилита для декларативного управления конфигурацией Talos Linux. Talm был создан Ænix, чтобы сделать конфигурации управления кластером более декларативными и настраиваемыми. Talm поставляется с готовыми пресетами для Cozystack.

Предварительные требования

К началу работы с этим руководством **Talos Linux должен быть установлен** на нескольких узлах, но еще не инициализирован (`bootstrapped`). Эти узлы должны находиться в одной подсети или иметь публичные IP-адреса.

В руководстве используется пример, где узлы кластера находятся в подсети `192.168.123.0/24` и имеют следующие IP-адреса:

- `node1`: private `192.168.123.11` or public `12.34.56.101`.
- `node2`: private `192.168.123.12` or public `12.34.56.102`.
- `node3`: private `192.168.123.13` or public `12.34.56.103`.

Публичные IP-адреса необязательны. Для установки с Talm нужен только доступ к узлам: напрямую, через VPN, bastion host или другим способом. В примерах этого руководства по умолчанию используются private IP, а public IP используются в инструкциях и примерах, относящихся к конфигурации с public IP.

Если вы используете DHCP, вы можете не знать IP-адреса, назначенные узлам в private subnet. Узлы с Talos Linux [открывают Talos API на порту `50000`](<https://www.talos.dev/v1.13/learn-more/talos-network-connectivity/>). Чтобы найти их, можно использовать `nmap`, указав маску сети (`192.168.123.0/24` в примере):

```
nmap -Pn -n -p 50000 192.168.123.0/24 -vv | grep 'Discovered'
```

Пример вывода:

Discovered open port 50000/tcp on 192.168.123.11

Discovered open port 50000/tcp on 192.168.123.12

Discovered open port 50000/tcp on 192.168.123.13

1\ Установка зависимостей

Для этого руководства нужно установить несколько инструментов:

- Talm. Чтобы установить последнюю сборку для вашей платформы, скачайте и запустите установочный скрипт:

<code>curl -sSL https://github.com/cozystack/talm/raw/refs/heads/main/hack/install.sh</code>	<code>sh -s</code>
--	--------------------

Для Talm доступны бинарные файлы для Linux, macOS и Windows, как для AMD, так и для ARM. Также можно [скачать бинарный файл с GitHub](#) или [собрать Talm из исходного кода](#).

- talosctl распространяется как brew package:

```
brew install siderolabs/tap/talosctl
```

Другие варианты установки см. в [руководстве по установке talosctl](https://www.talos.dev/v1.13/talos-guides/install/talosctl/)

2\ Инициализация конфигурации кластера

Первый шаг — инициализировать шаблоны конфигурации и указать значения для шаблонизации.

2.1 Инициализация конфигурации

Начните с инициализации конфигурации для нового кластера, используя preset `cozystack`:

```
mkdir -p cozystack-cluster
```

```
cd cozystack-cluster
```

```
talm init --preset cozystack --name mycluster
```

Структура проекта в основном повторяет обычный Helm chart:

- `charts` \- a directory that includes a common library chart with functions used for querying information from Talos Linux.
- `Chart.yaml` \- a file containing the common information about your project; the name of the chart is used as the name for the newly created cluster.
- `templates` \- a directory used to describe templates for the configuration generation.

-
- `secrets.yaml` \- a file containing secrets for your cluster.
 - `secrets.encrypted.yaml`, `talosconfig.encrypted` \- encrypted counterparts produced from `talm.key` (commit these to git instead of the plaintext files).
 - `talm.key` \- the project-local age key used for encrypt / decrypt. Back this up; without it the encrypted files cannot be reopened.
 - `values.yaml` \- a common values file used to provide parameters for the templating.
 - `.talm-preset.lock` \- a machine-managed file recording the preset name and its content hash at init time; used to detect preset drift after a `talm` binary upgrade. Commit it to git so the baseline is shared across the team.
 - `nodes` \- an optional directory used to describe and store generated configuration for nodes.

Доступные пресеты

`talm` поставляется с двумя встроенными пресетами:

- `cozystack` — production-пресет, используемый в этом руководстве.
- `talm` — минимальный `library chart` для продвинутых пользователей, желающих построить собственный пресет поверх него.

Имя пресета передаётся через `-p / --preset`.

Справочник флагов `talm init`

Каноничный список см. по `talm init -h`. Сгруппировано по режимам:

Создание нового проекта (режим по умолчанию):

- `-p, --preset <name>` — пресет для генерации файлов.
- `-N, --name <cluster-name>` — имя кластера.
- `--endpoints <list>` — Talos API endpoints (через запятую), встраиваемые в `talosconfig.contexts.<name>.endpoints` для клиента `talosctl`. См. «Флаги endpoint: клиент `talosctl` против control plane Kubernetes» ниже.
- `--cluster-endpoint <url>` — URL control-plane Kubernetes, записываемый в `values.yaml::endpoint` (например, `https://<vip>:6443`). При `init` проверяется на наличие `scheme + host + port`.
- `--image <ref>` — переопределить образ установщика Talos, записываемый в `values.yaml` пресета (например, `factory.talos.dev/installer/<sha256>:<version>`).
- `--talos-version <ver>` — желаемая contract-версия Talos для шаблонизации с обратной совместимостью (например, `v1.12`).
- `--force` — перезаписывать существующие файлы без запроса.

Флаги endpoint: клиент `talosctl` против control plane Kubernetes

В проектах `talm` слово «endpoint» обозначает две разные сущности:

- `**talosconfig.contexts.<name>.endpoints**` — список записей `host[:port]`, которые клиент `talosctl` использует для доступа к Talos API. Заполняется флагом `--endpoints` (множественное

число, список через запятую).

- ****values.yaml::endpoint**** — один URL со scheme + host + port, который чарт подставляет в `cluster.controlPlane.endpoint` в MachineConfig каждого узла. Именно к нему обращаются kubelet и kube-proxy. Заполняется флагом `--cluster-endpoint` (единственное число, полный URL).

Когда `--endpoints` задан ровно с одним значением, `init` автоматически выводит `values.yaml::endpoint` как `https://<это>:6443`, потому что случай с единственной целью однозначен. При нескольких `endpoints` автоматический вывод не выполняется (выбор одного узла молча привязал бы доступность кластера к нему) — задайте `--cluster-endpoint` явно или заполните `values.yaml::endpoint` позже вручную. В конце `init` выводит подсказку, если поле осталось пустым.

Обновление существующего проекта до последнего встроенного library chart:

- `-u`, `--update` — заново извлечь `charts/talm/` и другие файлы пресета из бинарника `talm`.
- `--preset` обязателен; `--name` — нет.
- `--force` — автоматически принимать все diff шаблонов пресета (пропустить интерактивный запрос; безопасно использовать в CI).

`--update` перезаписывает только файлы, поставляемые пресетом; ваши изменения в `values.yaml`, `secrets.yaml`, `templates/` и `nodes/` сохраняются.

Управление зашифрованными secrets на месте:

- `-e`, `--encrypt` — зашифровать `secrets.yaml` / `talosconfig` / `kubeconfig` в их `.encrypted`-копии. Требуется `talm.key`.
- `-d`, `--decrypt` — обратная операция. Не требует `--preset` или `--name`.

Обновление до новой версии Talm

Когда новая версия `talm` поставляется с более новым встроенным library chart, обновите проект на месте:

```
cd cozystack-cluster
```

```
talm init --update --preset cozystack # интерактивно: запрашивает по каждому diff шаблона пресета
```

```
talm init --update --preset cozystack --force # неинтерактивно: автоматически принять все diff
```

`--update` re-syncs the vendored `charts/talm/` exactly — files that the new library no longer ships (or strays like `.DS_Store`) are pruned — and advances the preset baseline in `.talm-preset.lock`.

Chart Drift Detection (Talm v0.32+)

Render commands read the project's local `charts/talm/` copy, never the binary's built-in charts, so upgrading the `talm` binary does not touch your project — the vendored chart silently goes stale. Release builds of `talm` detect this and print a non-fatal **WARN:** line on `stderr` for two independent signals:

- Library drift : the vendored `charts/talm/` differs by content from the copy built into the binary. A pure version stamp difference stays silent; a real difference is reported with a sample of the differing paths (`modified: / extra: / missing:`).
- Preset drift : the binary ships a newer preset than the baseline pinned in `.talm-preset.lock` at init time. Your `templates/` edits are never reported as drift — the comparison is binary-vs-baseline, not binary-vs-project.

Both warnings point at the remediation above. To escalate the warning into a hard error (exit 1) — for example, in CI — set `strictCharts: true` in `Chart.yaml` so the whole team inherits it, or pass `--strict-charts` for a single run. Under strict mode, a baseline that cannot be verified (a corrupted or deleted `.talm-preset.lock`, an unreadable `charts/talm/`) also blocks, so deleting the baseline is not a bypass; without strict mode, such failures degrade to a warning, and projects created before baseline pinning stay silent.

Цикл шифрования / расшифровки

В git вы коммитите зашифрованные копии, а `talm` читает копии в открытом виде. Для перехода между ними используйте:

```
talm init --encrypt # secrets.yaml -> secrets.encrypted.yaml; talosconfig -> talosconfig.encrypted
talm init --decrypt # обратная операция — не требует --preset или --name
```

Если потерять файл `talm.key`, зашифрованные копии станут нечитаемыми, поэтому храните резервную копию ключа отдельно. Когда `talm init --decrypt` запускается в проекте без `talm.key`, `talm` показывает оба пути восстановления в подсказке ошибки: восстановить сохранённый ключ или заново выполнить `talm init` для регенерации (с явным предупреждением, что регенерация записывает новые secrets, из-за чего старый `secrets.encrypted.yaml` нельзя будет расшифровать без исходного ключа).

2.2. Редактирование значений конфигурации и шаблонов

Сила Talm — в шаблонизации. Есть несколько файлов с исходными значениями и шаблонами, которые можно редактировать: `Chart.yaml`, `values.yaml` и `templates/`. Talm использует эти значения и шаблоны для генерации конфигурации Talos для всех узлов кластера: и control plane, и workers.

Все часто изменяемые значения конфигурации находятся в `values.yaml`:

```
## Используется для доступа к control plane кластера
endpoint: "https://192.168.100.10:6443"

## Домен кластера Cozystack API — используется сервисами и K8s-кластерами tenant'ов для
доступа к управляющему кластеру
clusterDomain: cozy.local

## Floating IP — должен быть неиспользуемым IP-адресом в той же подсети, что и узлы
floatingIP: 192.168.100.10
```

```
## Исходный образ Talos: используйте последнюю доступную версию
## https://github.com/cozystack/cozystack/pkgs/container/cozystack%2Ftalos
image: "ghcr.io/cozystack/cozystack/talos:v1.13.6"

## Подсеть Pod'ов — используется для назначения IP-адресов pod'ам
podSubnets:
    - 10.244.0.0/16

## Подсеть сервисов — используется для назначения IP-адресов сервисам
serviceSubnets:
    - 10.96.0.0/16

## Подсеть с IP-адресами узлов
advertisedSubnets:
    - 192.168.100.0/24

## Добавьте URL OIDC issuer, чтобы включить OIDC — см. комментарии ниже.
oidcIssuerUrl: ""

certSANs: []
```

На этом шаге не нужно указывать IP-адреса узлов. Вы укажете их позже, при генерации конфигураций узлов.

Расширение формируемой конфигурации Talos (Talm v0.30+)

Пресет `cozystack` поставляется с подобранными значениями по умолчанию для `machine.kernel.modules`, `machine.sysctls`, `machine.kubelet.extraConfig` и `machine.files`. Операторам, желающим дополнить любой из них без форка чарта, доступны четыре ключа values `extra*`:

Ключ	Тип	Семантика в пресете <code>cozystack</code>
<code>extraKernelModules</code>	<code>list</code>	Добавляется к встроенным модулям (<code>openvswitch</code> , <code>drbd</code> , <code>zfs</code> , <code>spl</code> , <code>vfiopci</code> , <code>vfiommu_type1</code>). Каждый элемент — spec kernel-модуля Talos.

<code>extraKubeletExtraArgs</code>	<code>map</code>	Объединяется с <code>kubelet.extraConfig</code> после заданных пресетом <code>cpuManagerPolicy: static</code> , <code>maxPods: 512</code> . Ключи оператора НЕ должны конфликтовать со встроенными — <code>yaml.v3</code> отвергает дублирующиеся ключи <code>map</code> при декодировании, поэтому конфликт валит <code>render</code> с точной подсказкой на проблемный ключ. Если нужно другое значение по умолчанию — форкните пресет.
<code>extraSysctls</code>	<code>map</code>	Объединяется с <code>machine.sysctls</code> после встроенных записей пресета: размеры ARP-кэша <code>gc_thresh1/2/3</code> , всегда включённый тюнинг DRBD/LINSTOR (<code>tcp_orphan_retries</code> , <code>tcp_fin_timeout</code> , <code>netdev_max_backlog</code> , <code>netdev_budget</code> , <code>netdev_budget_usecs</code>), <code>vm.nr_hugepages</code> (когда задан) и тройка <code>tcp_keepalive_*</code> , пока включён <code>tcpKeepaliveTuning</code> . Все они принадлежат пресету — действует тот же контракт «конфликт валит <code>render</code> », что и для <code>extraKubeletExtraArgs</code> . Значения должны быть строками YAML (Talos ожидает строки даже для числовых <code>sysctls</code>).
<code>extraMachineFiles</code>	<code>list</code>	Добавляется к записям пресета для настройки CRI и <code>lvm.conf</code> . Talos отвергает дублирующиеся <code>path</code> : при применении.

Пример дополнения `values.yaml`:

`extraKernelModules`:

- name: `nf_conntrack`

```
extraKubeletExtraArgs:
feature-gates: "NodeSwap=true"
extraSysctls:
net.core.somaxconn: "65535"
extraMachineFiles:
  - path: /etc/example.conf
op: create
content: "hello = world"
```

Пресет `generic` не предоставляет значений по умолчанию ни в одной из этих секций — каждый блок формируется только когда соответствующий ключ `extra*` непуст.

Помимо точек расширения `extra*`, пресет `cozystack` предоставляет два преднастроенных параметра, которые можно менять без форка чарта:

Ключ	По умолчанию	Эффект
<code>tcpKeepaliveTuning</code>	<code>false</code>	Когда <code>true</code> , добавляет <code>net.ipv4.tcp_keepalive_time=600 / intvl=10 / probes=6</code> в <code>machine.sysctls</code> , освобождая «мёртвый» простаивающий сокет примерно за 660 с вместо стандартных для ядра ~2 ч. Эти <code>sysctls</code> действуют на всё ядро — они меняют обнаружение сбоев для каждого долгоживущего простаивающего TCP-соединения на узле, а не только для DRBD, — поэтому включаются по желанию. DRBD и так обнаруживает мёртвых пиров за секунды через собственный <code>ring</code> на уровне протокола, поэтому оставляйте выключенным, если вам специально не нужно более быстрое обнаружение мёртвых сокетов на всём узле.

<code>etcd.quotaBackendBytes</code>	<code>"8589934592"</code> (8 ГиБ)	Потолок размера backend-БД <code>etcd</code> , выставаемый как <code>cluster.etcd.extraArgs.quota-backend-bytes</code> только на узлах <code>controlplane</code> . Поднимает собственное значение <code>etcd</code> по умолчанию в 2 ГиБ, чтобы <code>control plane</code> с большим количеством <code>LINSTOR</code> , суммарно хранящий множество <code>CRD DRBD</code> -ресурсов, не срабатывал по <code>alarm NOSPACE</code> . Это потолок, а не резервирование: небольшая БД остаётся небольшой и не требует дополнительных <code>RAM/диска</code> . Установите <code>" "</code> , чтобы вернуться к встроенному значению <code>etcd</code> по умолчанию. Управляет общим размером БД, а не размером отдельного объекта — записи по объектам ограничены фиксированным лимитом тела запроса <code>kube-apiserver</code> в 3 МиБ, у которого нет параметра конфигурации.
-------------------------------------	-----------------------------------	---

Пять всегда включённых `sysctls DRBD/LINSTOR`, перечисленных в строке `extraSysctls` выше, поставляются безусловно в пресете `cozystack` — они устраняют исчерпание `TCP`-портов, наблюдаемое при штормах переподключений `DRBD`, и не имеют аналога в пресете `generic`.

2.3 Добавление конфигурации Keycloak

По умолчанию кластер будет доступен только при аутентификации с помощью токена. Однако можно настроить `OIDC`-провайдера, чтобы использовать аутентификацию на основе учётных записей. Эта настройка начинается на данном шаге и продолжается позже, после установки `Cozystack`.

Чтобы настроить `Keycloak` как `OIDC`-провайдера, внесите следующие изменения в шаблоны:

- Для `Talm v0.6.6` или новее: в `./templates/_helpers.tpl` замените `keycloak.example.com` на `keycloak.<your-domain.tld>`.
- Для `Talm` старше `v0.6.6` обновите `./templates/_helpers.tpl` следующим образом:

`cluster:`

```
apiServer:
extraArgs:
oidc-issuer-url: "https://keycloak.example.com/realms/cozy"
oidc-client-id: "kubernetes"
oidc-username-claim: "preferred_username"
oidc-groups-claim: "groups"
```

2.4 Encrypted user values and secret redaction (Talm v0.32+)

Beyond `secrets.yaml` (the Talos bootstrap secrets), templates often inject operator-supplied secrets into the config — a registry password, an OIDC client secret, a static-pod env value. Talm lets you keep those encrypted in git the same way as `secrets.yaml`, decrypt them in memory at render time, and keep them out of committed node files, terminal output, and CI logs.

****Step 1 — put the secret values in `values-secret.yaml`:**

```
registryPassword: "s3cr3t-high-entropy-value"
```

Step 2 — encrypt it with the project's `talm.key`. `talm init --encrypt` produces `values-secret.encrypted.yaml`. Commit the encrypted file; the plaintext `values-secret.yaml` is git-ignored.

Step 3 — reference the encrypted file from `Chart.yaml` by adding it to `templateOptions.valueFiles`, so both `talm template` and `talm apply` read it:

```
templateOptions:
valueFiles:
  - values-secret.encrypted.yaml
```

Referencing it only via the CLI `--values` flag is a foot-gun: the modeline in a node file does not persist value files, so a later `talm apply` would re-render WITHOUT the secret and silently drop the field. Talm surfaces a warning when an encrypted file is passed via `--values` but is not in `templateOptions.valueFiles`.

Step 4 — use the values in templates like any other: `{{ .Values.registryPassword | quote }}`.

How secrets are handled across commands:

Command	Behavior
<code>talm template (stdout)</code>	secret values render as <code>***</code> ; <code>--show-secrets</code> prints them verbatim.

<code>talm template -I (node file)</code>	secret values are omitted entirely from the committed node file — the real value is re-rendered in memory only at apply, so no plaintext (or ciphertext) ever lands in <code>nodes/*.yaml</code> .
<code>talm apply --dry-run</code>	both diffs redact secrets: talm's structured drift preview AND the server-returned <code>Config diff</code> block. <code>--show-secrets-in-drift</code> reveals them.

The `--show-secrets-in-drift` flag governs every secret-bearing surface of the apply dry-run, covering both these user values and the Talos bootstrap material (`cluster.ca.key`, `machine.token`, encryption secrets, Wireguard keys, etc.). By default, a dry-run never prints a CA private key or a user secret in cleartext.

`talm apply` honors the full set of value sources, matching `talm template: --values, --set, --set-string, --set-file, --set-json, --set-literal`, merged on top of the `templateOptions.*` defaults from `Chart.yaml`. This keeps `template` and `apply` rendering identically.

Sharp edge — value-based matching. Redaction matches by exact value across the whole rendered config, so a secret whose plaintext coincides with an ordinary structural string (a password literally set to `controlplane`, or a bare port like `6443`) will also redact that unrelated field. Prefer high-entropy values; do not encrypt low-entropy strings that collide with non-secret config.

3\. Генерация конфигурационных файлов узлов

Следующий шаг — создать конфигурационные файлы узлов из шаблонов. Создайте каталог `nodes` и соберите информацию с каждого узла в отдельный файл для этого узла:

```
mkdir nodes
```

```
talm template -e 192.168.123.11 --nodes 192.168.123.11 -t templates/controlplane.yaml -i > nodes/node1.yaml
```

```
talm template -e 192.168.123.12 --nodes 192.168.123.12 -t templates/controlplane.yaml -i > nodes/node2.yaml
```

```
talm template -e 192.168.123.13 --nodes 192.168.123.13 -t templates/controlplane.yaml -i > nodes/node3.yaml
```

Параметр `--insecure (-i)` нужен потому, что Talm должен получить конфигурационные данные с узлов Talos, которые еще не инициализированы, находятся в `maintenance mode` и поэтому не могут принять аутентифицированное соединение. Узлы будут инициализированы только на следующем шаге с помощью `talm apply`.

Сгенерированные файлы содержат блок комментариев с обнаруженными сетевыми интерфейсами и дисками. Эти файлы можно отредактировать перед применением, чтобы настроить сетевую конфигурацию. Например, если нужно настроить `network bonding (LACP)`, см. [Настройка bonding \(LACP\)](#).

4\. Применение конфигурации и инициализация кластера

На этом этапе конфигурационные файлы в `node/*.yaml` готовы к применению на узлах.

4.1 Применение конфигурационных файлов

Используйте `talm apply`, чтобы применить конфигурационные файлы к соответствующим узлам:

```
talm apply -f nodes/node1.yaml -i
```

```
talm apply -f nodes/node2.yaml -i
```

```
talm apply -f nodes/node3.yaml -i
```

Эта команда инициализирует узлы и настраивает аутентифицированное соединение, поэтому дальше `-i (--insecure)` не потребуется. Если команда выполнена успешно, она вернет IP узла:

```
$ talm apply -f nodes/node1.yaml -i
- talm: file=nodes/node1.yaml, nodes=[192.168.123.11], endpoints=[192.168.123.11]
```

Позже с `talm apply` также можно использовать следующие опции:

- `--dry-run` \- dry run mode покажет diff с существующей конфигурацией без внесения изменений.
- `-m try` \- try mode откатит конфигурацию через 1 минуту.

4.2 Ожидание перезагрузки

Дождитесь, пока все узлы перезагрузятся. Если использовался установочный носитель, например USB-накопитель, извлеките его, чтобы узлы загрузились с внутреннего диска.

Когда узлы будут готовы, они откроют порт `50000`; это признак того, что узел завершил настройку Talos и перезагрузился. Если нужно автоматизировать проверку готовности узлов, используйте такой пример:

```
timeout 60 sh -c 'until \
nc -nzv 192.168.123.11 50000 && \
nc -nzv 192.168.123.12 50000 && \
nc -nzv 192.168.123.13 50000; \
do sleep 1; done'
```

4.3. Инициализация Kubernetes

Инициализируйте кластер Kubernetes, выполнив `talm bootstrap` для одного из узлов control plane:

```
talm bootstrap -f nodes/node1.yaml
```

5\.. Доступ к кластеру Kubernetes

На этом этапе кластер Kubernetes готов к установке Cozystack.

До этого шага взаимодействие с кластером выполнялось через Talos API и `talosctl`. Для следующих шагов нужны Kubernetes API и `kubectl`, которым требуется `kubeconfig`.

5.1. Получение kubeconfig

Используйте Talm, чтобы сгенерировать административный `kubeconfig`:

```
talm kubeconfig -f nodes/node1.yaml
```

Эта команда создаст файл `kubeconfig` в текущем каталоге.

5.2. Изменение URL Cluster API

Теперь в `kubeconfig` URL Cluster API указывает на floating IP (VIP) в private subnet.

Если вместо floatingIP используется public IP, обновите endpoint соответствующим образом. Отредактируйте `kubeconfig` — замените URL кластера на public IP одного из узлов:

```
apiVersion: v1
clusters:
  - cluster:
certificate-authority-data: ...
  - server: https://10.0.1.101:6443
+ server: https://12.34.56.101:6443
```

5.3. Активация kubeconfig

Затем настройте переменную `KUBECONFIG` или используйте другие инструменты, чтобы сделать этот `kubeconfig` доступным вашему клиенту `kubectl`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Чтобы сделать этот `kubeconfig` постоянно доступным, можно сделать его конфигурацией по умолчанию (`~/.kube/config`), использовать `kubectl config use-context` или применить другие методы. См. [документацию Kubernetes о доступе к кластеру](#).

5.4. Проверка доступности кластера

Проверьте, что кластер доступен:

```
kubectl get ns
```

Пример вывода:

```
NAME STATUS AGE
default Active 7m56s
kube-node-lease Active 7m56s
kube-public Active 7m56s
kube-system Active 7m56s
```

5.5. Проверка состояния узлов

Проверьте состояние узлов кластера:

```
kubectl get nodes
```

Вывод показывает состояние узлов и версию Kubernetes:

```
NAME STATUS ROLES AGE VERSION
node1 NotReady control-plane 7m56s v1.33.1
node2 NotReady control-plane 7m56s v1.33.1
node3 NotReady control-plane 7m56s v1.33.1
```

Обратите внимание, что все узлы показывают STATUS: `NotReady`, и на этом этапе это нормально. Так происходит потому, что стандартный [CNI-плагин Kubernetes](#) был отключен в конфигурации Talos, чтобы Cozystack мог установить собственный CNI-плагин.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes, готовый к установке Cozystack. Чтобы завершить установку, следуйте руководству по развертыванию, начиная с раздела [Установка Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование скрипта talos-bootstrap для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.6/install/kubernetes/talos-bootstrap/>

`talos-bootstrap` — CLI для пошаговой инициализации кластера, созданный разработчиками Cozystack.

Рекомендуется для первых развертываний.

`talos-bootstrap` — интерактивный скрипт для инициализации кластеров Kubernetes на Talos OS.

Он был создан разработчиками Cozystack, чтобы упростить установку Talos Linux на bare-metal узлы и сделать ее удобнее.

1\ Установка зависимостей

Установите следующие зависимости:

```
- talosctl
- dialog
- nmap
```

Скачайте последнюю версию `talos-bootstrap` со [страницы релизов](#) или напрямую из trunk:

```
curl -fsSL -o /usr/local/bin/talos-bootstrap \
https://github.com/cozystack/talos-bootstrap/raw/master/talos-bootstrap
chmod +x /usr/local/bin/talos-bootstrap
talos-bootstrap --help
```

2\ Подготовка конфигурационных файлов

1. Начните с создания каталога конфигурации для нового кластера:

```
mkdir -p cluster1
cd cluster1
```

2. Создайте файл конфигурационного `patch patch.yaml` с общими настройками узлов, используя следующий пример:

```
machine:
kubenet:
```

nodeIP:

validSubnets:

- 192.168.100.0/24

extraConfig:

maxPods: 512

sysctls:

net.ipv4.neigh.default.gc_thresh1: "4096"

net.ipv4.neigh.default.gc_thresh2: "8192"

net.ipv4.neigh.default.gc_thresh3: "16384"

kernel:

modules:

- name: openvswitch
- name: drbd

parameters:

- usermode_helper=disabled
- name: zfs
- name: spl
- name: vfio_pci
- name: vfio_iommu_type1

install:

image: ghcr.io/cozystack/cozystack/talos:v1.13.6

registries:

mirrors:

docker.io:

endpoints:

- https://mirror.gcr.io

files:

- content: |

[plugins]

[plugins."io.containerd.grpc.v1.cri"]

device_ownership_from_security_context = true

[plugins."io.containerd.cri.v1.runtime"]

device_ownership_from_security_context = true

path: /etc/cri/conf.d/20-customization.part

op: create

- op: overwrite

path: /etc/lvm/lvm.conf

```
permissions: 0o644
content: |
backup {
  backup = 0
  archive = 0
}
devices {
  global_filter = [ "r|^/dev/drbd.|", "r|^/dev/dm-.|", "r|^/dev/zd.|", "r|^/dev/loop.|" ]
}

cluster:
network:
cni:
name: none
dnsDomain: cozy.local
podSubnets:
  - 10.244.0.0/16
serviceSubnets:
  - 10.96.0.0/16
```

Do not change op: on these entries

Talos rejects `op: create` for any file outside `/var`, returning the error `create operation not allowed outside of /var` — the only exception is the special-cased `/etc/cri/conf.d/20-customization.part`. Because `/etc/lvm/lvm.conf` already exists on the node, it must use `op: overwrite`. Changing the `op` (or pointing `create` at another `/etc` path) fails the `WriteUserFiles` boot step: the node pauses and enters a reboot loop, and `talosctl bootstrap` reports only `bootstrap is not available yet` with no obvious cause.

3. Создайте еще один файл конфигурационного `patch patch-controlplane.yaml` с настройками, которые относятся только к узлам control plane:

```
machine:
nodeLabels:
node.kubernetes.io/exclude-from-external-load-balancers:
$patch: delete
cluster:
allowSchedulingOnControlPlanes: true
controllerManager:
extraArgs:
```

bind-address: 0.0.0.0

scheduler:

extraArgs:

bind-address: 0.0.0.0

apiServer:

certSANs:

- 127.0.0.1

proxy:

disabled: true

discovery:

enabled: false

etcd:

advertisedSubnets:

- 192.168.100.0/24

4. Чтобы настроить Keycloak как OIDC-провайдера, добавьте следующий раздел в `patch-controlplane.yaml`, заменив `example.com` на свой домен:

cluster:

apiServer:

extraArgs:

oidc-issuer-url: "https://keycloak.example.com/realms/cozy"

oidc-client-id: "kubernetes"

oidc-username-claim: "preferred_username"

oidc-groups-claim: "groups"

3\1. Инициализация и доступ к кластеру

Когда конфигурационные файлы будут готовы, запустите `talos-bootstrap` на каждом узле кластера:

в каталоге конфигурации кластера

`talos-bootstrap install`

[!] Если узлы работают во внешней сети, каждый узел нужно явно указать в аргументе:

`talos-bootstrap install -n 1.2.3.4`

Где `1.2.3.4` — IP-адрес удаленного узла.

`talos-bootstrap` включит `bootstrap` на первом настроенном узле кластера. Если нужно повторно выполнить `bootstrap` кластера `etcd`, удалите строку `BOOTSTRAP_ETCD=false` из файла `cluster.conf`.

Повторите этот шаг для остальных узлов кластера.

После завершения команды `install talos-bootstrap` сохранит конфигурацию кластера как `./kubeconfig`.

Настройте `kubectl` на использование новой конфигурации, экспортировав переменную `KUBECONFIG`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Чтобы сделать этот `kubeconfig` постоянно доступным, можно сделать его конфигурацией по умолчанию (`~/.kube/config`), использовать `kubectl config use-context` или применить другие методы. См. [документацию Kubernetes о доступе к кластеру](#).

Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get ns
```

Пример вывода:

```
NAME STATUS AGE
default Active 7m56s
kube-node-lease Active 7m56s
kube-public Active 7m56s
kube-system Active 7m56s
```

[!] Все узлы будут отображаться как `READY: False`, и на этом этапе это нормально. Так происходит потому, что на предыдущем шаге стандартный CNI-плагин был отключен, чтобы Cozystack мог установить собственный CNI-плагин.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes, готовый к установке Cozystack. Чтобы завершить установку, следуйте руководству по развертыванию, начиная с раздела [Установка Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование talosctl для инициализации кластера Cozystack

<https://cozystack.ru/docs/v1.6/install/kubernetes/talosctl/>

`talosctl` — стандартный CLI Talos Linux: он требует больше boilerplate-кода, но дает полную гибкость настройки.

В этом руководстве описано, как подготовить кластер Talos Linux к разворачиванию Cozystack с помощью `talosctl` — специализированного CLI-инструмента для управления Talos.

Предварительные требования

К началу работы с этим руководством Talos OS должна быть загружена с ISO на нескольких узлах, но еще не инициализирована (bootstrapped). Эти узлы должны находиться в одной подсети или иметь публичные IP-адреса.

В руководстве используется пример, где узлы кластера находятся в подсети `192.168.123.0/24` и имеют следующие IP-адреса:

- `192.168.123.11`
- `192.168.123.12`
- `192.168.123.13`

IP `192.168.123.10` — внутренний адрес, который не принадлежит ни одному из этих узлов, но создается Talos. Он используется как VIP.

Если вы используете DHCP, вы можете не знать IP-адреса, назначенные узлам. Чтобы найти их, можно использовать `nmap`, указав маску сети (`192.168.123.0/24` в примере):

<code>nmap -Pn -n -p 50000 192.168.123.0/24 -vv</code>	<code>grep 'Discovered'</code>
--	--------------------------------

Пример вывода:

```
Discovered open port 50000/tcp on 192.168.123.11
Discovered open port 50000/tcp on 192.168.123.12
Discovered open port 50000/tcp on 192.168.123.13
```

1\ Подготовка конфигурационных файлов

1. Начните с создания каталога конфигурации для нового кластера:

```
mkdir -p cluster1
```

```
cd cluster1
```

2. Сгенерируйте файл `secrets`. Эти `secrets` позже будут внедрены в конфигурацию и использованы для установки аутентифицированных соединений с узлами Talos:

```
talosctl gen secrets
```

3. Создайте файл конфигурационного `patch` `patch.yaml`:

```
machine:
```

```
kubelet:
```

```
nodeIP:
```

```
validSubnets:
```

```
  - 192.168.123.0/24
```

```
extraConfig:
```

```
maxPods: 512
```

```
sysctls:
```

```
net.ipv4.neigh.default.gc_thresh1: "4096"
```

```
net.ipv4.neigh.default.gc_thresh2: "8192"
```

```
net.ipv4.neigh.default.gc_thresh3: "16384"
```

```
kernel:
```

```
modules:
```

```
  - name: openvswitch
```

```
  - name: drbd
```

```
parameters:
```

```
  - usermode_helper=disabled
```

```
  - name: zfs
```

```
  - name: spl
```

```
  - name: vfio_pci
```

```
  - name: vfio_iommu_type1
```

```
install:
```

```
image: ghcr.io/cozystack/cozystack/talos:v1.13.6
```

```
registries:
```

```
mirrors:
```

```
docker.io:
```

```
endpoints:
```

```
  - https://mirror.gcr.io
```

```
files:
```

```
  - content: |
```

```
[plugins]
[plugins."io.containerd.cri.v1.runtime"]
device_ownership_from_security_context = true
path: /etc/cri/conf.d/20-customization.part
op: create
    - op: overwrite
path: /etc/lvm/lvm.conf
permissions: 0o644
content: |
backup {
backup = 0
archive = 0
}
devices {
global_filter = [ "r|^/dev/drbd.|", "r|^/dev/dm-.|", "r|^/dev/zd.|", "r|^/dev/loop.|" ]
}

cluster:
apiServer:
extraArgs:
oidc-issuer-url: "https://keycloak.example.org/realms/cozy"
oidc-client-id: "kubernetes"
oidc-username-claim: "preferred_username"
oidc-groups-claim: "groups"
network:
cni:
name: none
dnsDomain: cozy.local
podSubnets:
    - 10.244.0.0/16
serviceSubnets:
    - 10.96.0.0/16
```

Do not change op: on these entries

Talos rejects `op: create` for any file outside `/var`, returning the error `create operation not allowed outside of /var` — the only exception is the special-cased `/etc/cri/conf.d/20-customization.part`. Because `/etc/lvm/lvm.conf` already exists on the node, it must use `op: overwrite`. Changing the `op` (or

pointing `create` at another `/etc` path) fails the `WriteUserFiles` boot step: the node pauses and enters a reboot loop, and `talosctl bootstrap` reports only `bootstrap is not available yet` with no obvious cause.

4. Создайте еще один файл конфигурационного `patch patch-controlplane.yaml` с настройками только для узлов control plane:

Обратите внимание, что VIP-адрес используется в `machine.network.interfaces[0].vip.ip`:

`machine:`

`nodeLabels:`

`node.kubernetes.io/exclude-from-external-load-balancers:`

`$patch: delete`

`network:`

`interfaces:`

`- interface: eth0`

`vip:`

`ip: 192.168.123.10`

`cluster:`

`allowSchedulingOnControlPlanes: true`

`controllerManager:`

`extraArgs:`

`bind-address: 0.0.0.0`

`scheduler:`

`extraArgs:`

`bind-address: 0.0.0.0`

`apiServer:`

`certSANs:`

`- 127.0.0.1`

`proxy:`

`disabled: true`

`discovery:`

`enabled: false`

`etcd:`

`advertisedSubnets:`

`- 192.168.123.0/24`

2\ Генерация конфигурационных файлов узлов

Когда patch-файлы будут готовы, сгенерируйте конфигурационные файлы для каждого узла. Обратите внимание, что используются три файла, созданные на предыдущем шаге: `secrets.yaml`, `patch.yaml` и `patch-controlplane.yaml`.

URL `192.168.123.10:6443` использует тот же VIP, который упоминался выше, а порт `6443` — стандартный порт Kubernetes API.

```
talosctl gen config \
cozystack https://192.168.123.10:6443 \
--with-secrets secrets.yaml \
--config-patch=@patch.yaml \
--config-patch-control-plane @patch-controlplane.yaml
export TALOSCONFIG=$PWD/talosconfig
```

`192.168.123.11`, `192.168.123.12` и `192.168.123.13` — это узлы. В этой конфигурации все узлы являются управляющими.

3\ Применение конфигурации узлов

Примените конфигурацию ко всем узлам, а не только к управляющим.

```
talosctl apply -f controlplane.yaml -n 192.168.123.11 -e 192.168.123.11 -i
talosctl apply -f controlplane.yaml -n 192.168.123.12 -e 192.168.123.12 -i
talosctl apply -f controlplane.yaml -n 192.168.123.13 -e 192.168.123.13 -i
```

Также можно использовать следующие опции:

- `--dry-run` \- dry run mode покажет diff с существующей конфигурацией.
- `-m try` \- try mode откатит конфигурацию через 1 минуту.

3.1. Ожидание перезагрузки узлов

Дождитесь, пока все узлы перезагрузятся. Извлеките установочный носитель (например, USB-накопитель), чтобы узлы загрузились с внутреннего диска.

Готовые узлы будут открывать порт `50000`; это признак того, что узел завершил настройку Talos и перезагрузился.

Если нужно дождаться готовности узлов в скрипте, используйте такой пример:

```
timeout 60 sh -c 'until nc -nzv 192.168.123.11 50000 && \
```

```
nc -nzv 192.168.123.12 50000 && \  
nc -nzv 192.168.123.13 50000; \  
do sleep 1; done'
```

4\.. Инициализация и доступ к кластеру

Запустите `talosctl bootstrap` на одном узле control plane — этого достаточно для инициализации всего кластера:

```
talosctl bootstrap -n 192.168.123.11 -e 192.168.123.11
```

Для доступа к кластеру сгенерируйте административный `kubeconfig`:

```
talosctl kubeconfig -n 192.168.123.11 -e 192.168.123.11 kubeconfig
```

Настройте `kubectl` на использование новой конфигурации, экспортировав переменную `KUBECONFIG`:

```
export KUBECONFIG=$PWD/kubeconfig
```

Чтобы сделать этот `kubeconfig` постоянно доступным, можно сделать его конфигурацией по умолчанию (`~/.kube/config`), использовать `kubectl config use-context` или применить другие методы. См. [документацию Kubernetes о доступе к кластеру](#).

Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get ns
```

Пример вывода:

```
NAME STATUS AGE  
default Active 7m56s  
kube-node-lease Active 7m56s  
kube-public Active 7m56s  
kube-system Active 7m56s
```

[!] Все узлы будут отображаться как `READY: False`, и на этом этапе это нормально. Так происходит потому, что на предыдущем шаге стандартный CNI-плагин был отключен, чтобы Cozystack мог установить собственный CNI-плагин.

Следующие шаги

Теперь у вас есть инициализированный кластер Kubernetes, готовый к установке Cozystack. Чтобы завершить установку, следуйте руководству по развертыванию, начиная с раздела [Установка Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Инициализация кластера в изолированной среде

<https://cozystack.ru/docs/v1.6/install/kubernetes/air-gapped/>

Инициализация кластера Cozystack в изолированной (air-gapped) среде с mirror'ами container registry.

Введение

В этом руководстве описаны шаги для инициализации кластера Cozystack в изолированной среде.

Установка в air-gapped среде означает, что кластер не имеет прямого доступа к Интернету. Все необходимые ресурсы, такие как образы и metadata, должны быть доступны в частной сети.

Настройка узлов Talos

Для установки с Talm достаточно один раз внести все указанные изменения в `/templates/_helpers.tpl`, а затем собрать фактические конфигурационные файлы узлов с помощью `talm template`.

Для установки с `talosctl` изменения нужно внести в `patch.yaml` и `patch-controlplane.yaml`.

1\ Настройка NTP-серверов

Точная синхронизация времени критически важна для кластера. В конфигурации Talos machine укажите локальные NTP-серверы, доступные внутри вашей частной сети:

machine:

time:

servers:

example values

- 192.168.0.4

- 10.10.0.5

Убедитесь, что эти NTP-серверы доступны с первого узла Talos.

2\ Настройка mirror'ов Container Registry

Поскольку кластер не может обращаться к публичным container registries, он должен использовать их локальные mirror'ы. Создание таких mirror'ов выходит за рамки этого руководства.

Обновите конфигурацию machine следующим образом, указав IP-адреса и порты локальных mirror'ов для каждого registry:

machine:

registries:

mirrors:

docker.io:

endpoints:

- http://10.0.0.1:8082

ghcr.io:

endpoints:

- http://10.0.0.1:8083

gcr.io:

endpoints:

- http://10.0.0.1:8084

registry.k8s.io:

endpoints:

- http://10.0.0.1:8085

quay.io:

endpoints:

- http://10.0.0.1:8086

cr.fluentbit.io:

endpoints:

- http://10.0.0.1:8087

docker-registry3.mariadb.com:

endpoints:

- http://10.0.0.1:8088

config:

"10.0.0.1:8082":

tls:

insecureSkipVerify: true

auth:

username: myuser

password: mypass

Значения `config.[0].auth.*` приведены как пример; используйте реальные учетные данные. Убедитесь, что ваши локальные registry proxies mirror'ят все необходимые образы для компонентов Talos и Kubernetes.

3\.. Добавление CA-сертификата

Чтобы использовать частный Certificate Authority, добавьте его сертификат на узлы.

```
# tal: nodes=["10.10.10.10"], endpoints=["10.10.10.10"], templates=["templates/controlplane.yaml"]
```

```
# THIS FILE IS AUTOGENERATED. PREFER TEMPLATE EDITS OVER MANUAL ONES.
```

```
machine:
```

```
# ...
```

```
# ...
```

```
discovery:
```

```
enabled: false
```

```
etcd:
```

```
advertisedSubnets:
```

```
    - 10.4.100.10/24
```

```
allowSchedulingOnControlPlanes: true
```

```
apiVersion: v1alpha1
```

```
kind: TrustedRootsConfig
```

```
name: my-enterprise-ca
```

```
certificates: |
```

```
-----BEGIN CERTIFICATE-----
```

```
...
```

```
-----END CERTIFICATE-----
```

4\.. Применение изменений

После внесения описанных выше изменений можно применить конфигурацию и инициализировать кластер:

Использование Talm

Пересоберите конфигурационные файлы узлов и примените их к каждому узлу:

```
talm template -e <ip> -n <ip> -t templates/controlplane.yaml -i > nodes/node1.yaml
```

```
talm apply -f nodes/node1.yaml
```

```
# повторите для каждого узла
```

Затем инициализируйте кластер обычным способом:

```
talm bootstrap -f nodes/node1.yaml
```

Подробнее см. в [руководстве по настройке Talm](#).

Использование talosctl

Примените конфигурацию к каждому узлу:

```
talosctl apply -f controlplane.yaml -n <ip> -e <ip> -i
```

Затем инициализируйте кластер с помощью одного из узлов:

```
talosctl bootstrap -n <ip> -e <ip>
```

Подробнее см. в [руководстве по настройке talosctl](https://cozystack.ru/docs/v1.6/install/kubernetes/talosctl/).

5\.. Настройка mirror'ов Container Registry для Tenant Kubernetes

Tenant-кластеры Kubernetes в Cozystack используют [Kamaji](#) для control plane. Компоненты control plane работают как pods на узлах management-кластера, поэтому они автоматически используют registry mirrors, настроенные для Talos на шаге 2.

Однако tenant worker nodes работают как отдельные виртуальные машины со своим экземпляром containerd. Для этих worker-узлов нужна отдельная конфигурация registry mirror.

Чтобы выполнить эту настройку, сначала нужно развернуть кластер Cozystack версии v0.32.0 или новее (или обновить существующий кластер до этой версии). Проверьте текущую версию кластера командой:

<pre>kubectl get deploy -n cozy-system cozystack -oyaml</pre>	<pre>grep installer</pre>
---	---------------------------

Вариант А: настройка через platform package

Platform package может автоматически сгенерировать secret patch-containerd из раздела [registries](#) в platform values.

Добавьте раздел [registries](#) в cozystack-platform.yaml :

apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full
components:
platform:
values:
... существующие publishing, networking и т. д. ...
registries:
mirrors:
docker.io:
endpoints:
 - http://10.0.0.1:8082
ghcr.io:
endpoints:
 - http://10.0.0.1:8083
gcr.io:
endpoints:
 - http://10.0.0.1:8084
registry.k8s.io:
endpoints:
 - http://10.0.0.1:8085
quay.io:
endpoints:
 - http://10.0.0.1:8086
cr.fluentbit.io:
endpoints:
 - http://10.0.0.1:8087
docker-registry3.mariadb.com:
endpoints:
 - http://10.0.0.1:8088
config:
"10.0.0.1:8082":
tls:

```
insecureSkipVerify: true
```

```
auth:
```

```
username: myuser
```

```
password: mypass
```

Затем примените его:

```
kubectl apply -f cozystack-platform.yaml
```

Это создаст secret `patch-containerd` в namespace `cozy-system`, который автоматически копируется в каждый tenant-кластер Kubernetes.

Альтернативно: применить patch к существующему platform package

Если platform package уже развернут, можно добавить registry mirrors с помощью patch:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
```

```
"spec": {
```

```
"components": {
```

```
"platform": {
```

```
"values": {
```

```
"registries": {
```

```
"mirrors": {
```

```
"docker.io": {
```

```
"endpoints": ["http://10.0.0.1:8082"]
```

```
},
```

```
"ghcr.io": {
```

```
"endpoints": ["http://10.0.0.1:8083"]
```

```
},
```

```
"gcr.io": {
```

```
"endpoints": ["http://10.0.0.1:8084"]
```

```
},
```

```
"registry.k8s.io": {
```

```
"endpoints": ["http://10.0.0.1:8085"]
```

```
},
```

```
"quay.io": {
```

```
"endpoints": ["http://10.0.0.1:8086"]
```

```
},
```

```
"cr.fluentbit.io": {
```

```
"endpoints": ["http://10.0.0.1:8087"]
},
"docker-registry3.mariadb.com": {
"endpoints": ["http://10.0.0.1:8088"]
}
},
"config": {
"10.0.0.1:8082": {
"tls": {
"insecureSkipVerify": true
},
"auth": {
"username": "myuser",
"password": "mypass"
}
}
}
}
}
}
}
}
}'
```

Вариант В: создание secret вручную

Также можно напрямую создать [Kubernetes Secret](#) с именем `patch-containerd`:

```
apiVersion: v1
kind: Secret
metadata:
name: patch-containerd
namespace: cozy-system
type: Opaque
stringData:
docker.io.toml: |
```

```
server = "https://registry-1.docker.io"
[host."http://10.0.0.1:8082"]
capabilities = ["pull", "resolve"]
skip_verify = true

ghcr.io.toml: |
server = "https://ghcr.io"
[host."http://10.0.0.1:8083"]
capabilities = ["pull", "resolve"]
skip_verify = true

gcr.io.toml: |
server = "https://gcr.io"
[host."http://10.0.0.1:8084"]
capabilities = ["pull", "resolve"]
skip_verify = true

registry.k8s.io.toml: |
server = "https://registry.k8s.io"
[host."http://10.0.0.1:8085"]
capabilities = ["pull", "resolve"]
skip_verify = true

quay.io.toml: |
server = "https://quay.io"
[host."http://10.0.0.1:8086"]
capabilities = ["pull", "resolve"]
skip_verify = true

cr.fluentbit.io.toml: |
server = "https://cr.fluentbit.io"
[host."http://10.0.0.1:8087"]
capabilities = ["pull", "resolve"]
skip_verify = true

docker-registry3.mariadb.com.toml: |
server = "https://docker-registry3.mariadb.com"
[host."http://10.0.0.1:8088"]
capabilities = ["pull", "resolve"]
skip_verify = true
```

Если registry mirrors требуют аутентификации, добавьте пользовательский header `Authorization` с учетными данными в Base64:

```
server = "https://registry-1.docker.io"
[host."http://10.0.0.1:8082"]
capabilities = ["pull", "resolve"]
skip_verify = true
[host."http://10.0.0.1:8082".header]
Authorization = "Basic bXl1c2VyOm15cGFzcw=="
```

Чтобы сгенерировать значение в Base64, выполните:

<code>echo -n 'myuser:mypass'</code>	<code>base64</code>
--------------------------------------	---------------------

Для динамической или token-based аутентификации (например, Docker Hub) используйте [Kubernetes image pull secrets](#) вместо учетных данных в открытом виде.

Как это работает

Secret `patch-containerd` из namespace `cozy-system` автоматически копируется в namespace каждого tenant-кластера Kubernetes во время развертывания. Данные secret монтируются в worker node VM как конфигурационные файлы registry для containerd в `/etc/containerd/certs.d/<registry>/hosts.toml`.

Конфигурация для отдельного кластера

Можно настроить registry mirrors для конкретного tenant-кластера Kubernetes вместо использования глобального secret `patch-containerd`:

- Tenant-кластер должен быть развернут с Kubernetes package версии 0.23.1 или новее, которая доступна начиная с Cozystack 0.32.1.
- Перед развертыванием tenant-кластера создайте Kubernetes Secret с именем `kubernetes-<cluster-name>-patch-containerd` в namespace tenant-кластера, используя тот же формат, что и в примерах выше.

Важно: если существуют и глобальный secret `patch-containerd`, и secret для отдельного кластера, глобальный secret имеет приоритет, а per-cluster secret игнорируется. Чтобы использовать конфигурацию для отдельного кластера, убедитесь, что глобального secret `patch-containerd` в namespace `cozy-system` нет.

Подробнее о значениях конфигурации registry см. в [руководстве по настройке CRI Plugin](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок при установке Kubernetes

<https://cozystack.ru/docs/v1.6/install/kubernetes/troubleshooting/>

Инструкции по решению типовых проблем, которые могут возникнуть при установке Kubernetes с помощью `tal`, `talos-bootstrap` или `talosctl`.

На этой странице приведены инструкции по решению типовых проблем, которые могут возникнуть при установке Kubernetes с помощью `tal`, `talos-bootstrap` или `talosctl`.

No Talos nodes in maintenance mode found!

Если скрипт `talos-bootstrap` не обнаруживает узлы, выполните следующие шаги для диагностики и устранения проблемы:

1. Проверьте сетевой сегмент

Убедитесь, что скрипт запускается в том же сетевом сегменте, что и узлы. Это критически важно, чтобы скрипт мог взаимодействовать с узлами.

2. Используйте Nmap для обнаружения узлов

Проверьте, может ли `nmap` обнаружить узел, выполнив следующую команду:

```
nmap -Pn -n -p 50000 192.168.0.0/24
```

Эта команда сканирует сеть на наличие узлов, которые слушают порт `50000`. В выводе должны быть перечислены все узлы в сетевом сегменте, слушающие этот порт; это означает, что они доступны.

3. Проверьте подключение talosctl

Затем убедитесь, что `talosctl` может подключиться к конкретному узлу, особенно если узел находится в `maintenance mode`:

```
talosctl -e "${node}" -n "${node}" get machinestatus -i
```

Ошибка вида ниже обычно означает, что локальный бинарный файл `talosctl` устарел:

```
rpc error: code = Unimplemented desc = unknown service resource.ResourceService
```

Обновление `talosctl` до последней версии должно решить проблему.

4. Запустите talos-bootstrap в debug mode

Если предыдущие шаги не помогли, запустите `talos-bootstrap` в `debug mode`, чтобы получить больше диагностической информации.

Выполните скрипт с опцией `-x`, чтобы включить `debug mode`:

```
bash -x talos-bootstrap
```

Обратите внимание на последнюю команду, показанную перед ошибкой; часто она указывает на команду, которая завершилась неудачно, и помогает в дальнейшей диагностике.

Ожидание `runtime service` в Talos может привести к тому, что узел останется в состоянии `booting` в консоли Talos. Если вы хотите использовать `lldpd`, можно применить `patch` к узлам. Продолжайте, если у вас есть подключение через `talosctl`.

```
cat > lldpd.patch.yaml <<EOF
apiVersion: v1alpha1
kind: ExtensionServiceConfig
name: lldpd
configFiles:
  - content: |
    configure lldp status disabled
  mountPath: /usr/local/etc/lldp/lldpd.conf
EOF
```

Чтобы применить `patch` к конкретному узлу, выполните:

```
talosctl patch mc -p @lldpd.patch.yaml -n <node> -e <node>
```

Проверьте, на каких узлах установлен `lldpd`:

```
node_net='192.168.100.0/24'
```

<code>nmap -Pn -n -T4 -p50000 --open -oG - \$node_net</code>	<code>awk '/50000\/open/ { system("talosctl get extensions -n \"\$2\" -e \"\$2\"</code>	<code>grep lldpd") }'</code>
--	---	------------------------------

Если хотите применить `patch` ко всем узлам:

<code>nmap -Pn -n -T4 -p50000 --open -oG - \$node_net</code>	<code>awk '/50000\/open/ {print "talosctl patch mc -p @lldpd.patch.yaml -n \"\$2\" -e \"\$2\" "'</code>
--	---

Проверьте состояние в консоли Talos:

<code>talosctl dashboard -n \$(nmap -Pn -n -T4 -p50000 --open -oG - \$node_net</code>

<code>awk '/50000\open/ {print \$2}'</code>

<code>paste -sd,)</code>

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Развертывание Cozystack на Generic Kubernetes

<https://cozystack.ru/docs/v1.6/install/kubernetes/generic/>

Как развернуть Cozystack на k3s, kubeadm, RKE2 или других дистрибутивах Kubernetes без Talos Linux

В этом руководстве описано, как развернуть Cozystack на generic-дистрибутивах Kubernetes, таких как k3s, kubeadm или RKE2. Хотя Talos Linux остается рекомендуемой платформой для production-развертываний, Cozystack поддерживает развертывание на других дистрибутивах Kubernetes с помощью bundle `isp-full-generic`.

Когда использовать Generic Kubernetes

Рассмотрите использование generic Kubernetes вместо Talos Linux, если:

- У вас уже есть кластер Kubernetes, который вы хотите дополнить Cozystack
- Ваша инфраструктура не поддерживает Talos Linux (некоторые облачные провайдеры, embedded-системы)
- Вам нужны специфические функции или пакеты Linux, недоступные в Talos

Для новых production-развертываний рекомендуется [Talos Linux](#) благодаря его преимуществам в безопасности и эксплуатации.

Предварительные требования

Ubuntu hosts with UEFI Secure Boot enabled require pre-installing `drbd-dkms` before deploying Cozystack. The default `piraeus-operator` flow compiles DRBD in-cluster and `insmod`s the unsigned module, which kernel lockdown rejects with `Key was rejected by service`. See [Ubuntu + Secure Boot](#) for the workaround.

Supported Distributions

Cozystack протестирован на:

- k3s v1.32+ (рекомендуется для single-node и edge-развертываний)
- kubeadm v1.28+
- RKE2 v1.28+

Требования к хостам

- Операционная система : Ubuntu 22.04+ или Debian 12+ (kernel 5.x+ с systemd)
- Архитектура : amd64 или arm64

- Оборудование : см. [требования к оборудованию](#)

Обязательные пакеты

Установите следующие пакеты на всех узлах:

```
apt-get update
```

```
apt-get install -y nfs-common open-iscsi multipath-tools
```

Обязательные модули ядра

Загрузите модуль `br_netfilter` (требуется для sysctl-настроек bridge netfilter):

```
modprobe br_netfilter
```

```
echo "br_netfilter" > /etc/modules-load.d/br_netfilter.conf
```

Обязательные сервисы

Включите и запустите обязательные сервисы:

```
systemctl enable --now iscsid
```

```
systemctl enable --now multipathd
```

Настройка sysctl

[!] Критично : приведенные ниже sysctl-настройки обязательны для корректной работы Cozystack. Без этих настроек компоненты Kubernetes будут завершаться с ошибками из-за недостаточного количества inotify watches.

Создайте `/etc/sysctl.d/99-cozystack.conf` со следующим содержимым:

```
# Inotify limits (critical for Cozystack)
```

```
fs.inotify.max_user_watches = 524288
```

```
fs.inotify.max_user_instances = 8192
```

```
fs.inotify.max_queued_events = 65536
```

```
# Filesystem limits
```

```
fs.file-max = 2097152
```

```
fs.aio-max-nr = 1048576
```

```
# Network forwarding (required for Kubernetes)
```

```
net.ipv4.ip_forward = 1
```

```
net.ipv4.conf.all.forwarding = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

```
# VM tuning
vm.swappiness = 1
```

Примените настройки:

```
sysctl --system
```

Настройка Kubernetes

Cozystack самостоятельно управляет сетью (Cilium/KubeOVN), хранилищем (LINSTOR) и ingress (NGINX). Ваш дистрибутив Kubernetes должен быть настроен так, чтобы не устанавливать эти компоненты.

Обязательная конфигурация

Компонент	Требование
CNI	Отключен — Cozystack разворачивает Cilium или KubeOVN
Ingress Controller	Отключен — Cozystack разворачивает NGINX
Storage Provisioner	Отключен — Cozystack разворачивает LINSTOR
kube-proxy	Отключен — его заменяет Cilium
Cluster Domain	Должен быть <code>cozy.local</code>

При установке k3s используйте следующие флаги:

```
curl -sL https://get.k3s.io | INSTALL_K3S_EXEC="server \
--disable=traefik \
--disable=service-lb \
--disable=local-storage \
--disable=metrics-server \
--disable-network-policy \
--disable-kube-proxy \
--flannel-backend=none \
```

```
--cluster-domain=cozy.local \  
--tls-san=<YOUR_NODE_IP> \  
--kubelet-arg=max-pods=220" sh -
```

Замените <YOUR_NODE_IP> на IP-адрес вашего узла.

Создайте конфигурационный файл kubeadm:

```
apiVersion: kubeadm.k8s.io/v1beta3  
kind: ClusterConfiguration  
networking:  
podSubnet: "10.244.0.0/16"  
serviceSubnet: "10.96.0.0/16"  
dnsDomain: "cozy.local"
```

```
apiVersion: kubeproxy.config.k8s.io/v1alpha1  
kind: KubeProxyConfiguration  
mode: "none" # Cilium заменит kube-proxy
```

Инициализируйте кластер без стандартного CNI:

```
kubeadm init --config kubeadm-config.yaml --skip-phases=addon/kube-proxy
```

Не устанавливайте CNI-плагин после `kubeadm init` — Cozystack автоматически развернет Kube-OVN и Cilium.

Создайте `/etc/rancher/rke2/config.yaml`:

```
cni: none  
disable:  
  - rke2-ingress-nginx  
  - rke2-metrics-server  
cluster-domain: cozy.local  
disable-kube-proxy: true
```

Установка Cozystack

1\. Применение CRD

Скачайте и примените Custom Resource Definitions:

```
kubectl apply -f https://github.com/cozystack/cozystack/releases/download/v1.6.0/cozystack-crds.yaml
```

2\ Развертывание Cozystack Operator

Скачайте generic-манифест operator, замените placeholder адреса API server и примените его:

```
curl -fsSL https://github.com/cozystack/cozystack/releases/download/v1.6.0/cozystack-operator-generic.yaml \
| sed 's/REPLACE_ME/<YOUR_NODE_IP>/' \
```

```
kubectl apply -f -
```

Замените `<YOUR_NODE_IP>` на IP-адрес вашего Kubernetes API server (только IP, без протокола и порта).

Манифест включает deployment operator, ConfigMap `cozystack-operator-config` с адресом API server и ресурс `PackageSource`.

3\ Создание Platform Package

После запуска operator и reconciliation ресурса `PackageSource` создайте ресурс `Package`, чтобы запустить установку платформы.

[!] Важно : значения `podCIDR` и `serviceCIDR` должны совпадать с конфигурацией вашего кластера Kubernetes. Разные дистрибутивы используют разные значения по умолчанию:

- k3s : 10.42.0.0/16 (pods), 10.43.0.0/16 (services)
- kubeadm : 10.244.0.0/16 (pods), 10.96.0.0/16 (services)
- RKE2 : 10.42.0.0/16 (pods), 10.43.0.0/16 (services)

Пример для k3s (для других дистрибутивов скорректируйте CIDR):

```
apiVersion: cozystack.io/v1alpha1
```

```
kind: Package
```

```
metadata:
```

```
name: cozystack.cozystack-platform
```

```
# Package имеет cluster scope — namespace не нужен
```

```
spec:
```

```
variant: isp-full-generic
```

```
components:
```

```
platform:
```

```
values:
```

```
publishing:
```

```
host: "example.com"
apiServerEndpoint: "https://<YOUR_NODE_IP>:6443"
networking:
podCIDR: "10.42.0.0/16"
podGateway: "10.42.0.1"
serviceCIDR: "10.43.0.0/16"
joinCIDR: "100.64.0.0/16"
```

Скорректируйте значения:

Поле	Описание
<code>publishing.host</code>	Ваш домен для сервисов Cozystack
<code>publishing.apiServerEndpoint</code>	URL Kubernetes API endpoint
<code>networking.podCIDR</code>	CIDR pod-сети (должен совпадать с вашей k8s-конфигурацией)
<code>networking.podGateway</code>	Первый IP в pod CIDR (например, <code>10.42.0.1</code> для <code>10.42.0.0/16</code>)
<code>networking.serviceCIDR</code>	CIDR service-сети (должен совпадать с вашей k8s-конфигурацией)
<code>networking.joinCIDR</code>	Сеть для связи с nested-кластерами

Примените его:

```
kubectl apply -f cozystack-platform-package.yaml
```

Имя Package должно совпадать с именем PackageSource (`cozystack.cozystack-platform`). Проверить доступные PackageSources можно командой `kubectl get packagesource`.

4\ Мониторинг установки

Следите за ходом установки:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

Проверьте состояние HelmRelease:

```
kubectl get hr -A
```

Во время начального развертывания HelmRelease в первые несколько минут может показывать ошибки вроде `ExternalArtifact not found` или `dependency is not ready`, пока Cilium и другие

базовые компоненты проходят reconciliation. Это ожидаемо — подождите несколько минут и проверьте снова.

Можно убедиться, что Cilium развернут и сеть между узлами настроена, дождавшись, пока узлы перейдут в состояние Ready:

```
kubectrl wait --for=condition=Ready nodes --all --timeout=300s
```

Пример: Ansible Playbook

Ниже приведен минимальный Ansible playbook для подготовки узлов и развертывания Cozystack.

Сначала установите необходимые Ansible collections:

```
ansible-galaxy collection install ansible.posix community.general kubernetes.core ansible.utils
```

Playbook подготовки узлов

```
- name: Prepare nodes for Cozystack
hosts: all
become: true
tasks:
  - name: Load br_netfilter module
    community.general.modprobe:
      name: br_netfilter
      persistent: present

  - name: Install required packages
    ansible.builtin.apt:
      name:
        - nfs-common
        - open-iscsi
        - multipath-tools
      state: present
      update_cache: true

  - name: Configure sysctl for Cozystack
    ansible.posix.sysctl:
      name: "{{ item.name }}"
      value: "{{ item.value }}"
    sysctl_set: true
```

state: present

reload: true

loop:

- { name: fs.inotify.max_user_watches, value: "524288" }
- { name: fs.inotify.max_user_instances, value: "8192" }
- { name: fs.inotify.max_queued_events, value: "65536" }
- { name: fs.file-max, value: "2097152" }
- { name: fs.aio-max-nr, value: "1048576" }
- { name: net.ipv4.ip_forward, value: "1" }
- { name: net.ipv4.conf.all.forwarding, value: "1" }
- { name: net.bridge.bridge-nf-call-iptables, value: "1" }
- { name: net.bridge.bridge-nf-call-ip6tables, value: "1" }
- { name: vm.swappiness, value: "1" }
- name: Enable iscsid service

ansible.builtin.systemd:

name: iscsid

enabled: true

state: started

- name: Enable multipathd service

ansible.builtin.systemd:

name: multipathd

enabled: true

state: started

Playbook развёртывания Cozystack

В этом примере используются CIDR по умолчанию для k3s. Скорректируйте их для kubeadm (10.244.0.0/16, 10.96.0.0/16) или своей пользовательской конфигурации.

-
- name: Deploy Cozystack

hosts: localhost

connection: local

vars:

cozystack_root_host: "example.com"

cozystack_api_host: "10.0.0.1"

cozystack_api_port: "6443"

k3s defaults - adjust for kubeadm (10.244.0.0/16, 10.96.0.0/16)

cozystack_pod_cidr: "10.42.0.0/16"

cozystack_svc_cidr: "10.43.0.0/16"

tasks:

- name: Apply Cozystack CRDs

ansible.builtin.command:

cmd: kubectl apply -f https://github.com/cozystack/cozystack/releases/download/v1.6.0/cozystack-crds.yaml

changed_when: true

- name: Download and apply Cozystack operator manifest

ansible.builtin.shell:

cmd: >

curl -fsSL https://github.com/cozystack/cozystack/releases/download/v1.6.0/cozystack-operator-generic.yaml

| sed 's/REPLACE_ME/{{ cozystack_api_host }}/'

| kubectl apply -f -

changed_when: true

- name: Wait for PackageSource to be ready

kubernetes.core.k8s_info:

api_version: cozystack.io/v1alpha1

kind: PackageSource

name: cozystack.cozystack-platform

register: pkg_source

until: >

pkg_source.resources | length > 0 and

(

pkg_source.resources[0].status.conditions

| selectattr('type', 'equalto', 'Ready')

| map(attribute='status')

| first

| default('False')

) == "True"

retries: 30

delay: 10

- name: Create Platform Package

kubernetes.core.k8s:

```
state: present
definition:
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full-generic
components:
platform:
values:
publishing:
host: "{{ cozystack_root_host }}"
apiServerEndpoint: "https://{{ cozystack_api_host }}:{{ cozystack_api_port }}"
networking:
podCIDR: "{{ cozystack_pod_cidr }}"
podGateway: "{{ cozystack_pod_cidr | ansible.utils.ipaddr('1') | ansible.utils.ipaddr('address') }}"
serviceCIDR: "{{ cozystack_svc_cidr }}"
joinCIDR: "100.64.0.0/16"
```

Устранение неполадок

Некорректный image tag linstor-scheduler

Симптом : ошибка `InvalidImageName` для pod `linstor-scheduler`.

Причина : формат версии k3s (например, `v1.35.0+k3s1`) содержит `+`, который недопустим в Docker image tags.

Решение : это исправлено в Cozystack v1.0.0+. Убедитесь, что используете последний релиз.

KubeOVN не планируется

Симптом : pods `ovn-central` остаются в состоянии `Pending`.

Причина : KubeOVN использует Helm `lookup` для поиска узлов control plane, что может не сработать на свежих кластерах.

Решение : убедитесь, что ваш Platform Package содержит явную конфигурацию `MASTER_NODES`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full-generic
components:
platform:
values:
networking:
kubeovn:
MASTER_NODES: "<YOUR_CONTROL_PLANE_IP>"
```

Ключ — `kubeovn` (без дефиса), он соответствует полю в `packages/core/platform/values.yaml` — см. также `[networking.kubeovn.MASTER_NODES]`(<https://cozystack.ru/docs/v1.6/operations/configuration/platform-package/>) в справочнике Platform Package.

Cilium не может подключиться к API server

Симптом : pods Cilium находятся в CrashLoopBackOff с ошибками подключения к API.

Причина : single-node кластеры или нестандартные API endpoints требуют явной конфигурации.

Решение : проверьте, что ваш Platform Package содержит корректные настройки API server:

```
spec:
components:
networking:
values:
cilium:
k8sServiceHost: "<YOUR_API_HOST>"
k8sServicePort: "6443"
```

Ошибки лимитов inotify

Симптом : pods завершаются с ошибками "too many open files" или ошибками inotify.

Причина : стандартные лимиты inotify в Linux слишком низкие для Kubernetes.

Решение : примените sysctl-настройки из раздела Настройка sysctl и перезагрузите узел.

Следующие шаги

После завершения установки Cozystack:

1. Настройте хранилище с LINSTOR
2. Настройте root tenant
3. Разверните первое приложение

Ссылки

- [PR #1939: Non-Talos Kubernetes Support](#)
- [Issue #1950: Complete non-Talos Support](#)
- [k3s Documentation](#)
- [kubeadm Documentation](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Ubuntu Secure Boot: DRBD prerequisites

<https://cozystack.ru/docs/v1.6/install/kubernetes/ubuntu-secure-boot/>

Prepare Ubuntu LTS hosts with UEFI Secure Boot enabled for Cozystack non-Talos installation by pre-installing drbd-dkms from the LINBIT PPA

This page covers a single prerequisite for installing Cozystack on Ubuntu hosts where UEFI Secure Boot is enabled. If your nodes are Talos Linux, or Ubuntu with Secure Boot disabled, you can skip this page — the default piraeus-operator flow handles those cases.

Why this is needed

Cozystack uses [LINSTOR](#) for replicated block storage, which depends on the DRBD 9.x kernel module. The piraeus-operator that ships with Cozystack defaults to compiling DRBD from source on each node at runtime and loading the module via `insmod`.

On hosts where UEFI Secure Boot is enabled, the kernel's lockdown subsystem rejects unsigned modules at load time:

```
insmod: ERROR: could not insert module ./drbd.ko: Key was rejected by service
```

(A preceding `chcon: can't apply partial context to unlabeled file './drbd.ko'` line may also appear when the loader image sets `LB_SELINUX_AS` — that warning is benign and unrelated to the load failure.)

The compiled `.ko` files are not signed against any key the host trusts, so `init_module()` fails. The linstor satellite Pod never reaches `Ready` and Cozystack storage stays broken.

This is common on bare-metal Ubuntu installs and on cloud SKUs that ship with Secure Boot enabled. Stock cloud-VM images on AWS, GCP, and Azure typically ship without Secure Boot enforcement and the default piraeus-operator flow works as-is.

This guide covers Ubuntu LTS only. Debian 12 has the same DRBD-unsigned-module problem under Secure Boot but LINBIT does not publish a Debian PPA — those operators must use LINBIT's customer-portal apt mirror or build and sign drbd-dkms manually. RHEL/SUSE flows (LINBIT-managed RPM repos with pre-signed kmods) are out of scope here.

Recommended fix: pre-install drbd-dkms on the host

Install the LINBIT-published `drbd-dkms` package on each node before deploying Cozystack, then add a `usermode_helper=disabled` `modprobe.d` drop-in and mask the host-side `drbd.service`. Minimal Ubuntu cloud images do not ship `add-apt-repository`; install `software-properties-common` first if it is missing. The full flow on each node:

1. Add LINBIT PPA, install `drbd-dkms`. The package goes through Ubuntu's standard `shim-signed` \+ `dkms` pipeline:

- On first `dkms` install with no existing MOK present, Ubuntu's `shim-signed` `postinst` generates a per-host signing keypair at `/var/lib/shim-signed/mok/MOK.priv` and `/var/lib/shim-signed/mok/MOK.der` (referenced from `/etc/dkms/framework.conf` as `mok_signing_key` / `mok_certificate`).
- `dkms` compiles the module against the running kernel and signs the freshly built `.ko` against that key.
- During `apt-get install`, `debconf` prompts the operator for an enrollment password; the matching public key is queued for MOK enrollment via `mokutil --import`. If the install runs non-interactively (`DEBIAN_FRONTEND=noninteractive`), the prompt is bypassed and the operator must run `mokutil --import /var/lib/shim-signed/mok/MOK.der` manually after the install.

2. Write `/etc/modprobe.d/cozystack-drbd.conf` with options `drbd usermode_helper=disabled`. `piraeus-operator`'s `daemonset` sets `LB_FAIL_IF_USERMODE_HELPER_NOT_DISABLED=yes` on the loader, so the loader fails on a host-loaded module without that param (see `LINBIT/drbd` [entry.sh line 332](https://github.com/LINBIT/drbd/blob/8c279459a32823b495a2649fc6dafc9fdfac1c7f/docker/entry.sh#L332)) and the env wiring in `piraeus-operator`'s `satellite daemonset`).

3. `systemctl mask drbd.service`. `drbd-utils` lands on the host transitively as a hard `apt` dependency of `drbd-dkms` and ships a `drbd.service` unit that would race `piraeus-operator`'s `satellite` container if accidentally enabled.

4. Write `/etc/modules-load.d/cozystack-drbd.conf` containing `drbd` so `systemd-modules-load.service` loads the module on every boot. Without this file the host depends on whatever (if any) `modules-load.d` entry the `drbd-utils` package ships, which has varied across releases — making it explicit removes the ambiguity and matches the path the companion Ansible playbook writes.

5. Reboot each node and confirm MOK enrollment at the shim console (Enroll MOK -> View key -> Continue -> enter the password set during step 1's `debconf` prompt or via `mokutil --import`). One reboot per node is required for the operator to walk through shim's MOK Manager. This step cannot be automated — it is the design of UEFI Secure Boot.

6. After enrollment, the kernel trusts the signing key and the `dkms`-built DRBD module loads with the right param.

Once the host has DRBD 9.x loaded with `usermode_helper=disabled`, `piraeus-operator`'s loader detects the host-loaded module on Pod startup and exits cleanly without attempting its own compile and `insmod`. No operator-side configuration is required.

MOK enrollment is interactive and per-node. The operator must access the console (physical, IPMI, or KVM) of each node on the next reboot and walk through shim's MOK Manager. There is no Kubernetes-level workaround — kernel module signing under Secure Boot is a host-firmware concern.

For the manual procedure on each node, see below. Automation is being added to [cozystack/ansible-cozystack (PR #39)](https://github.com/cozystack/ansible-cozystack/pull/39); v1.6 ships only the manual path because the automation has not yet landed in a tagged collection release.

Manual path

For operators who do not use Ansible, the equivalent steps on each Ubuntu LTS node are:

1. Install add-apt-repository (not present on minimal cloud images).

```
sudo apt-get update
```

```
sudo apt-get install --yes software-properties-common
```

2. Add LINBIT PPA and install drbd-dkms. During the install, debconf

prompts for a one-time MOK enrollment password — choose any

password you can re-enter at the shim console after reboot.

```
sudo add-apt-repository --yes ppa:linbit/linbit-drbd9-stack
```

```
sudo apt-get install --yes drbd-dkms
```

3. Configure the required module parameter.

```
sudo tee /etc/modprobe.d/cozystack-drbd.conf <<'EOF'
```

```
options drbd usermode_helper=disabled
```

```
EOF
```

4. Mask the host drbd.service so it cannot race piraeus-operator.

```
sudo systemctl mask drbd.service
```

5. Persist the module load on boot. The drbd-utils package's own

/lib/modules-load.d/ entry has varied across releases; an explicit

/etc/ entry overrides anything in /lib/ and removes the ambiguity.

```
echo drbd
```

```
sudo tee /etc/modules-load.d/cozystack-drbd.conf
```

6. Reboot. At the shim MOK Manager menu, choose:

Enroll MOK -> View key -> Continue -> Yes -> enter the password

you set at the debconf prompt during step 2.

```
sudo reboot
```

After the reboot, verify the module loaded with the right param:

```
cat /sys/module/drbd/parameters/usermode_helper
```

expected: disabled

Then deploy Cozystack as usual. The piraeus-operator loader will detect the host-loaded DRBD and exit cleanly.

What happens at deploy time

When `piraeus-operator`'s satellite Pod starts on a node, its `drbd-module-loader` initContainer runs `[LINBIT/drbd's entry.sh](https://github.com/LINBIT/drbd/blob/master/docker/entry.sh)`. Lines 328–339 short-circuit the compile + insmod path when DRBD is already loaded on the host:

```
if grep -q '^drbd ' /proc/modules; then
echo "DRBD module is already loaded"
[[ $LB_FAIL_IF_USERMODE_HELPER_NOT_DISABLED == yes ]] \
&& ! grep -qw disabled /sys/module/drbd/parameters/usermode_helper \
&& die "...
drbd_matches_min_version "$LB_DRBD_MIN_LOADED_VERSION" || die "...
exit 0
fi
```

Two preconditions are checked:

1. The module must be loaded with `usermode_helper=disabled`. The `modprobe.d` drop-in above takes care of this.
2. The loaded version must satisfy `LB_DRBD_MIN_LOADED_VERSION`. `piraeus-operator`'s daemonset sets it to 9. LINBIT's `drbd-dkms` ships DRBD 9.x, so this is automatic.

Both conditions met -> `exit 0` -> satellite Pod proceeds. No operator-side change to `piraeus-operator` or `LinstorSatelliteConfiguration` is needed.

Alternatives

- Talos Linux ([guide](#)) ships pre-signed DRBD modules in its system extensions and has none of these problems. Recommended for new deployments where you do not need a custom Linux distro.
- Disable Secure Boot in the host's UEFI firmware. The default in-cluster compile path then works without modification. Operationally undesirable for fleets where Secure Boot is part of the security baseline, but a valid escape hatch.
- Build and sign `drbd-dkms` manually against your own enterprise CA. This is what production environments with their own MOK / shim signing infrastructure already do; it is out of scope for this guide.

Troubleshooting

****modprobe drbd returns Key was rejected by service after dkms install**** — the dkms-generated MOK key is queued but not yet enrolled. Reboot the node and confirm enrollment at the shim console (Enroll MOK -> View key -> Continue -> dkms password). Re-run `modprobe`.

****cat /sys/module/drbd/parameters/usermode_helper is not disabled**** — the modprobe.d drop-in is missing or the module was loaded before it was written. `sudo rmmod drbd && sudo modprobe drbd` after the drop-in is in place, or reboot.

****piraeus-operator loader Pod logsCould not load DRBD kernel modules**** even after host install — check `LB_DRBD_MIN_LOADED_VERSION` via `kubectl --namespace cozy-linstor describe pod --selector app.kubernetes.io/component=linstor-satellite` and `cat /proc/drbd` on the host. The host module must be at least the loader's required minimum (9 per piraeus-operator's daemonset). LINBIT PPA `drbd-dkms` is 9.x so this is rarely the issue.

Ubuntu 26.04+ or interim releases (Oracular 24.10, Plucky 25.04) — LINBIT's PPA publishes `drbd-dkms` only for the LTS series LINBIT keeps current. Check [the PPA detail page](#) for the current series list. On unsupported releases the PPA add fails on a 404 Release file. Build and sign `drbd-dkms` manually, downgrade to a supported LTS, or wait for LINBIT to publish for your release.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка и настройка Cozystack

<https://cozystack.ru/docs/v1.6/install/cozystack/>

Шаг 3: установка Cozystack в Kubernetes-кластер — как готовой к использованию платформы или в режиме BYOP (Build Your Own Platform).

Третий шаг развертывания кластера Cozystack — установка Cozystack в ранее установленный и настроенный кластер Kubernetes. Предварительное условие для этого шага — [установленный кластер Kubernetes](#).

Cozystack можно установить в двух режимах, в зависимости от того, насколько детально вы хотите управлять устанавливаемыми компонентами:

Как платформа

Установите Cozystack как готовую к использованию платформу, где все компоненты управляются автоматически. Вы выбираете [variant](#) (например, `isp-full`), а Cozystack устанавливает и настраивает все необходимые компоненты — сеть, хранилище, мониторинг, dashboard, operators и managed applications.

Это рекомендуемый подход для большинства пользователей, которым нужна полностью функциональная платформа из коробки.

[Установить Cozystack как платформу](#)

Build Your Own Platform (BYOP)

Используйте Cozystack, чтобы собрать собственную платформу, устанавливая только нужные компоненты. Вы устанавливаете operator с `variant default`, который предоставляет только package registry (PackageSources). Затем с помощью CLI-инструмента `cozyupkg` выборочно устанавливаете отдельные packages — `networking`, `storage`, `ingress`, `operators` и любые другие компоненты, доступные в репозитории Cozystack.

Этот подход подходит, если у вас уже есть существующий кластер Kubernetes с частью инфраструктуры или если вам нужны только отдельные компоненты из экосистемы Cozystack.

[Собрать собственную платформу с Cozystack](#)

•

[Установка Cozystack как платформы](#)

Установите Cozystack как готовую к использованию платформу, где все компоненты управляются автоматически.

Собственная платформа (BYOP)

Соберите собственную платформу с Cozystack, устанавливая только нужные компоненты с помощью CLI-инструмента `cozурkg`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Cozystack как платформы

<https://cozystack.ru/docs/v1.6/install/cozystack/platform/>

Установите Cozystack как готовую к использованию платформу, где все компоненты управляются автоматически.

Третий шаг развертывания кластера Cozystack — установка Cozystack в кластер Kubernetes, ранее установленный и настроенный на узлах Talos Linux. Предварительное условие для этого шага — [установленный кластер Kubernetes](#).

Если вы устанавливаете Cozystack впервые, рекомендуем начать с [руководства по Cozystack](#).

Чтобы спланировать production-ready установку, следуйте руководству ниже. По структуре оно повторяет tutorial, но содержит гораздо больше деталей и объясняет разные варианты установки.

1\. Установка Cozystack Operator

Установите Cozystack operator с помощью Helm из OCI registry:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace
```

Замените X.Y.Z на нужную версию Cozystack. Доступные версии можно найти на [странице релизов Cozystack](#).

Эта команда устанавливает operator, CRD и создает ресурс PackageSource.

Установка на ОС без Talos

По умолчанию Cozystack operator настроен на использование функции [KubePrism](#) из Talos Linux, которая позволяет обращаться к Kubernetes API через локальный адрес на узле.

Если вы устанавливаете Cozystack не на Talos Linux, укажите variant operator во время установки:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--set cozystackOperator.variant=generic \
--set cozystack.apiServerHost=<YOUR_API_SERVER_IP> \
```

```
--set cozystack.apiServerPort=6443
```

Замените <YOUR_API_SERVER_IP> на внутренний IP-адрес вашего Kubernetes API server (только IP, без протокола и порта).

Полное руководство по разворачиванию Cozystack на generic-дистрибутивах Kubernetes см. в разделе [Развертывание Cozystack на Generic Kubernetes](#).

2\. Описание и применение Platform Package

После запуска operator следующий шаг — описать Platform Package и применить его. Platform Package — это ресурс `Package`, который определяет [variant Cozystack](#), [настройки компонентов](#), ключевые сетевые настройки, опубликованные сервисы и другие параметры.

Конфигурацию Cozystack можно обновлять после установки. Однако некоторые значения, показанные ниже, обязательны для установки.

Ниже минимальный пример `cozystack-platform.yaml` :

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
  platform:
  values:
  publishing:
  host: "example.org"
  apiServerEndpoint: "https://api.example.org:443"
  exposedServices:
    - dashboard
    - api
  networking:
  podCIDR: "10.244.0.0/16"
  podGateway: "10.244.0.1"
  serviceCIDR: "10.96.0.0/16"
  joinCIDR: "100.64.0.0/16"
```

Имя Package должно быть `cozystack.cozystack-platform`, чтобы совпадать с PackageSource, созданным installer. Проверить доступные PackageSources можно командой `kubectl get packagesource`.

2.1. Выбор variant

Состав Cozystack определяется variant. Variant `isp-full` — самый полный: он охватывает все уровни от оборудования до managed applications. Выбирайте его, если разворачиваете Cozystack на bare metal или VM и хотите использовать все возможности платформы.

Если вы разворачиваете Cozystack в предоставленном Kubernetes-кластере или хотите развернуть только Kubernetes-кластер без сервисов, см. [обзор и сравнение variants](#).

2.2. Тонкая настройка компонентов

Можно добавить дополнительные компоненты или убрать компоненты, включенные по умолчанию. См. [справочник components](#).

Если вы разворачиваете платформу на VM или выделенных серверах облачного провайдера, скорее всего, нужно отключить MetalLB и включить специфичный для провайдера load balancer либо использовать другую сетевую конфигурацию. См. раздел [установка у конкретных провайдеров](#). В нем может быть полное руководство для вашего провайдера, которое можно использовать для разворачивания production-ready кластера.

2.3. Описание сетевой конфигурации

Замените `example.org` в `publishing.host` и `publishing.apiServerEndpoint` на маршрутизируемое fully-qualified domain name (FQDN), которым вы управляете. Если у вас есть только public IP, но нет маршрутизируемого FQDN, используйте [nip.io](#) с записью через дефис.

В следующем разделе приведены разумные значения по умолчанию. Проверьте, что они совпадают с настройками узлов Talos, которые вы использовали на предыдущих шагах. Если вы использовали Talm для установки Kubernetes, значения должны совпадать.

networking:

podCIDR: "10.244.0.0/16"

podGateway: "10.244.0.1"

serviceCIDR: "10.96.0.0/16"

joinCIDR: "100.64.0.0/16"

Cozystack по умолчанию собирает анонимную статистику использования. Подробнее о том, какие данные собираются и как отказаться от сбора, см. в [документации по Telemetry](#).

2.4. Применение Platform Package

Когда конфигурационный файл будет готов, примените его:

```
kubectl apply -f cozystack-platform.yaml
```

Во время установки можно отслеживать логи operator:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

Подождите некоторое время, затем проверьте состояние установки:

```
kubectl get hr -A
```

Дождитесь, пока все releases перейдут в состояние `Ready`:

```
NAMESPACE NAME AGE READY STATUS
```

```
cozy-cert-manager cert-manager 4m1s True Release reconciliation succeeded
cozy-cert-manager cert-manager-issuers 4m1s True Release reconciliation succeeded
cozy-cilium cilium 4m1s True Release reconciliation succeeded
cozy-cluster-api capi-operator 4m1s True Release reconciliation succeeded
cozy-cluster-api capi-providers 4m1s True Release reconciliation succeeded
cozy-dashboard dashboard 4m1s True Release reconciliation succeeded
cozy-grafana-operator grafana-operator 4m1s True Release reconciliation succeeded
cozy-kamaji kamaji 4m1s True Release reconciliation succeeded
cozy-kubeovn kubeovn 4m1s True Release reconciliation succeeded
cozy-kubevirt-cdi kubevirt-cdi 4m1s True Release reconciliation succeeded
cozy-kubevirt-cdi kubevirt-cdi-operator 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt-operator 4m1s True Release reconciliation succeeded
cozy-linstor linstor 4m1s True Release reconciliation succeeded
cozy-linstor piraeus-operator 4m1s True Release reconciliation succeeded
cozy-mariadb-operator mariadb-operator 4m1s True Release reconciliation succeeded
cozy-metallb metallb 4m1s True Release reconciliation succeeded
cozy-monitoring monitoring 4m1s True Release reconciliation succeeded
cozy-postgres-operator postgres-operator 4m1s True Release reconciliation succeeded
cozy-rabbitmq-operator rabbitmq-operator 4m1s True Release reconciliation succeeded
cozy-redis-operator redis-operator 4m1s True Release reconciliation succeeded
cozy-telepresence telepresence 4m1s True Release reconciliation succeeded
cozy-victoria-metrics-operator victoria-metrics-operator 4m1s True Release reconciliation succeeded
tenant-root tenant-root 4m1s True Release reconciliation succeeded
```

Разделение узлов Control Plane и Worker

Обычно Cozystack требует как минимум три worker-узла для запуска workload'ов в HA mode. В компонентах Cozystack нет tolerations, которые позволили бы им запускаться на узлах control plane.

Однако для тестирования часто используется всего три узла. Либо у вас могут быть только крупные аппаратные узлы, и вы хотите использовать их одновременно для control-plane и worker workload'ов. В этом случае нужно удалить control-plane taint с узлов.

Пример удаления control-plane taint с узлов:

```
kubecttl taint nodes --all node-role.kubernetes.io/control-plane-
```

3\.. Настройка хранилища

Kubernetes нужна подсистема хранения, чтобы предоставлять persistent volumes приложениям, но собственной такой подсистемы в Kubernetes нет. Cozystack предоставляет [LINSTOR](#) как подсистему хранения.

На следующих шагах мы получим доступ к интерфейсу LINSTOR, создадим storage pools и определим storage classes.

3.1. Проверка устройств хранения

1. Настройте alias для доступа к LINSTOR:

```
alias linstor='kubecttl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

2. Выведите список узлов и проверьте их готовность:

```
linstor node list
```

Пример вывода показывает имена узлов и состояние:

```
+-----+
| Node | NodeType | Addresses | State |
|=====|
| srv1 | SATELLITE | 192.168.100.11:3367 (SSL) | Online |
| srv2 | SATELLITE | 192.168.100.12:3367 (SSL) | Online |
| srv3 | SATELLITE | 192.168.100.13:3367 (SSL) | Online |
+-----+
```

3. Выведите список доступных пустых устройств:

```
linstor physical-storage list
```

Пример вывода показывает те же имена узлов:

```
+-----+
| Size | Rotational | Nodes |
|=====|
| 107374182400 | True | srv3[/dev/sdb] |
| | | srv1[/dev/sdb] |
| | | srv2[/dev/sdb] |
+-----+
```

3.2. Создание Storage Pools

1. Создайте storage pools с помощью ZFS или LVM.

Также можно восстановить ранее созданные storage pools после reset узла.

```
- linstor ps cdp zfs srv1 /dev/sdb --pool-name data --storage-pool data
linstor ps cdp zfs srv2 /dev/sdb --pool-name data --storage-pool data
linstor ps cdp zfs srv3 /dev/sdb --pool-name data --storage-pool data
```

Рекомендуется выставить `failmode=continue` для ZFS storage pools чтобы позволить DRBD обрабатывать ошибки диска вместо ZFS.

```
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv1 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv2 -- zpool set failmode=continue data
kubectl exec -ti -n cozy-linstor ds/linstor-satellite.srv3 -- zpool set failmode=continue data
```

```
linstor ps cdp lvm srv1 /dev/sdb --pool-name data --storage-pool data
linstor ps cdp lvm srv2 /dev/sdb --pool-name data --storage-pool data
linstor ps cdp lvm srv3 /dev/sdb --pool-name data --storage-pool data
```

```
for node in $(kubectl get nodes --no-headers -o custom-columns=":metadata.name"); do
echo "linstor storage-pool create zfs $node data data"
done
# linstor storage-pool create zfs <node> data data
```

2. Проверьте результат, выведя список storage pools:

```
linstor sp l
```

Пример вывода:

```
+-----+
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | CanSnapshots | State |
| SharedName |
|=====|
|=====|
| DfltDisklessStorPool | srv1 | DISKLESS | | | False | Ok | srv1;DfltDisklessStorPool | |
| DfltDisklessStorPool | srv2 | DISKLESS | | | False | Ok | srv2;DfltDisklessStorPool |
| DfltDisklessStorPool | srv3 | DISKLESS | | | False | Ok | srv3;DfltDisklessStorPool |
| data | srv1 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv1;data |
| data | srv2 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv2;data |
| data | srv3 | ZFS | data | 96.41 GiB | 99.50 GiB | True | Ok | srv3;data |
+-----+
```

3.3. Создание Storage Classes

Создайте storage classes, один из которых должен быть default class.

1. Создайте файл с определениями storage classes. Ниже приведен разумный пример по умолчанию с двумя классами: `local` (default) и `replicated`.

storageclasses.yaml:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: local
annotations:
storageclass.kubernetes.io/is-default-class: "true"
provisioner: linstor.csi.linbit.com
parameters:
linstor.csi.linbit.com/storagePool: "data"
linstor.csi.linbit.com/layerList: "storage"
linstor.csi.linbit.com/allowRemoteVolumeAccess: "false"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true

```

```
apiVersion: storage.k8s.io/v1
```

```
kind: StorageClass
metadata:
name: replicated
provisioner: linstor.csi.linbit.com
parameters:
linstor.csi.linbit.com/storagePool: "data"
linstor.csi.linbit.com/autoPlace: "3"
linstor.csi.linbit.com/layerList: "drbd storage"
linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"
property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io
property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io
property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary
property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect
volumeBindingMode: Immediate
allowVolumeExpansion: true
```

2. Примените конфигурацию storage class

```
kubectl apply -f storageclasses.yaml
```

3. Проверьте, что storage classes успешно созданы:

```
kubectl get storageclasses
```

Пример вывода:

```
NAME PROVISIONER RECLAIMPOLICY VOLUMEBINDINGMODE ALLOWVOLUMEEXPANSION AGE
local (default) linstor.csi.linbit.com Delete WaitForFirstConsumer true 11m
replicated linstor.csi.linbit.com Delete Immediate true 11m
```

4\ Настройка сети

Далее настроим доступ к кластеру Cozystack. На этом шаге есть два варианта в зависимости от доступной инфраструктуры:

- Для собственного bare metal или self-hosted VM выберите вариант MetalLB. MetalLB — load balancer Cozystack по умолчанию.
- Для VM и выделенных серверов у облачных провайдеров выберите настройку public IP. **Большинство облачных провайдеров не поддерживают MetalLB.**

См. раздел [установка у конкретных провайдеров](#). Там могут быть инструкции для вашего провайдера, которые можно использовать для развертывания production-ready кластера.

4.a Настройка MetalLB

Cozystack использует три типа IP-адресов:

- Node IPs: постоянные адреса, действительные только внутри кластера.
- Virtual floating IP: используется для доступа к одному из узлов кластера и действителен только внутри кластера.
- External access IPs: используются LoadBalancers для публикации сервисов за пределами кластера.

Сервисы с external IP можно публиковать в двух режимах: L2 и BGP. Режим L2 прост, но требует, чтобы узлы находились в одном L2-домене, и плохо распределяет нагрузку. BGP сложнее в настройке: нужны BGP peers, готовые принимать announcements, но он дает корректное распределение нагрузки и больше возможностей выбора диапазонов IP-адресов.

Выберите диапазон неиспользуемых IP для сервисов; здесь используется диапазон `192.168.100.200-192.168.100.250`. Если используется режим L2, эти IP должны быть либо из той же сети, что и узлы, либо к ним должны быть настроены все необходимые маршруты.

Для режима BGP также потребуются IP-адреса BGP peers и локальный и удаленный AS numbers. Здесь мы используем `192.168.20.254` как peer IP, а AS numbers 65000 и 65001 — как local и remote.

Создайте и примените файл с описанием address pool.

```
metallb-ip-address-pool.yml
```

```
apiVersion: metallb.io/v1beta1
```

```
kind: IPAddressPool
```

```
metadata:
```

```
name: cozystack
```

```
namespace: cozy-metallb
```

```
spec:
```

```
addresses:
```

```
# используется для публикации сервисов за пределами кластера
```

```
  - 192.168.100.200-192.168.100.250
```

```
autoAssign: true
```

```
avoidBuggyIPs: false
```

```
kubectl apply -f metallb-ip-address-pool.yml
```

Создайте и примените ресурсы, необходимые для L2 или BGP advertisement.

- *

L2Advertisement использует имя ресурса IPAddressPool, который мы создали на предыдущем шаге.

metallb-l2-advertisement.yml

```
apiVersion: metallb.io/v1beta1
```

```
kind: L2Advertisement
```

```
metadata:
```

```
  name: cozystack
```

```
  namespace: cozy-metallb
```

```
spec:
```

```
  ipAddressPools:
```

```
    - cozystack
```

Примените изменения.

```
kubectl apply -f metallb-l2-advertisement.yml
```

Сначала создайте отдельный ресурс BGPPeer для каждого peer.

metallb-bgp-peer.yml

```
apiVersion: metallb.io/v1beta2
```

```
kind: BGPPeer
```

```
metadata:
```

```
  name: peer1
```

```
  namespace: cozy-metallb
```

```
spec:
```

```
  myASN: 65000
```

```
  peerASN: 65001
```

```
  peerAddress: 192.168.20.254
```

Затем создайте один ресурс BGPAdvertisement.

metallb-bgp-advertisement.yml

```
apiVersion: metallb.io/v1beta1
```

```
kind: BGPAdvertisement
```

```
metadata:
name: cozystack
namespace: cozy-metallb
spec:
ipAddressPools:
  - cozystack
```

Примените изменения.

```
kubectl apply -f metallb-bgp-peer.yml
kubectl apply -f metallb-bgp-advertisement.yml
```

После настройки MetalLB включите `ingress` в `tenant-root`:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '
{"spec":{
  "ingress": true
}}'
```

Чтобы подтвердить успешную настройку, проверьте `HelmReleases ingress` и `ingress-nginx-system`:

```
kubectl -n tenant-root get hr ingress ingress-nginx-system
```

Пример корректного вывода:

```
NAME AGE READY STATUS
ingress 47m True Helm upgrade succeeded for release tenant-root/ingress.v3 with chart ingress@1.8.0
ingress-nginx-system 47m True Helm upgrade succeeded for release
tenant-root/ingress-nginx-system.v2 with chart cozy-ingress-nginx@0.35.1
```

Затем проверьте состояние сервиса `root-ingress-controller`:

```
kubectl -n tenant-root get svc root-ingress-controller
```

Сервис должен быть развернут как `TYPE: LoadBalancer` и иметь корректный `external IP`:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
root-ingress-controller LoadBalancer 10.96.91.83 192.168.100.200 80/TCP,443/TCP 48m
```

4.b. Настройка Node Public IP

Если ваш облачный провайдер не поддерживает MetalLB, ingress controller можно опубликовать через external IP на узлах.

Если public IP подключены напрямую к узлам, укажите их. Если public IP предоставляются через 1:1 NAT, как в некоторых облаках, используйте IP-адреса внешних сетевых интерфейсов.

Здесь используются 192.168.100.11, 192.168.100.12 и 192.168.100.13.

Сначала примените patch к Platform Package, указав IP для публикации:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
"spec": {
"components": {
"platform": {
"values": {
"publishing": {
"externalIPs": [
"192.168.100.11",
"192.168.100.12",
"192.168.100.13"
]
}
}
}
}
}
}'
```

Затем включите ingress для root tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{
"spec":{
"ingress": true
}
}'
```

После этого Ingress будет доступен на указанных IP. Проверьте это следующим образом:

```
kubectl get svc -n tenant-root root-ingress-controller
```

Сервис должен быть развернут как TYPE: ClusterIP и иметь полный диапазон external IP:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
------	------	------------	-------------	---------	-----

root-ingress-controller	ClusterIP	10.96.91.83	192.168.100.11,192.168.100.12,192.168.100.13	80/TCP,443/TCP	48m
-------------------------	-----------	-------------	--	----------------	-----

5\.. Завершение установки

5.1. Настройка сервисов Root Tenant

Включите `etcd` и `monitoring` для root tenant:

```
kubectrl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{
  "spec":{
    "ingress": true,
    "monitoring": true,
    "etcd": true
  }
}'
```

5.2. Проверка состояния и состава кластера

Проверьте созданные persistent volumes:

```
kubectrl get pvc -n tenant-root
```

пример вывода:

NAME	STATUS	VOLUME	CAPACITY	ACCESS	MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
data-etcd-0	Bound	pvc-4cbd29cc-a29f-453d-b412-451647cd04bf	10Gi	RWO	local	<unset>		2m10s
data-etcd-1	Bound	pvc-1579f95a-a69d-4a26-bcc2-b15ccdbede0d	10Gi	RWO	local	<unset>		115s
data-etcd-2	Bound	pvc-907009e5-88bf-4d18-91e7-b56b0dbfb97e	10Gi	RWO	local	<unset>		91s
grafana-db-1	Bound	pvc-7b3f4e23-228a-46fd-b820-d033ef4679af	10Gi	RWO	local	<unset>		2m41s
grafana-db-2	Bound	pvc-ac9b72a4-f40e-47e8-ad24-f50d843b55e4	10Gi	RWO	local	<unset>		113s
vmselect-cachedir-vmselect-longterm-0	Bound	pvc-622fa398-2104-459f-8744-565eee0a13f1	2Gi	RWO	local	<unset>		2m21s
vmselect-cachedir-vmselect-longterm-1	Bound	pvc-fc9349f5-02b2-4e25-8bef-6cbc5cc6d690	2Gi	RWO	local	<unset>		2m21s
vmselect-cachedir-vmselect-shortterm-0	Bound	pvc-7acc7ff6-6b9b-4676-bd1f-6867ea7165e2	2Gi	RWO	local	<unset>		2m41s
vmselect-cachedir-vmselect-shortterm-1	Bound	pvc-e514f12b-f1f6-40ff-9838-a6bda3580eb7	2Gi	RWO	local	<unset>		2m40s

vmstorage-db-vmstorage-longterm-0 Bound pvc-e8ac7fc3-df0d-4692-aebf-9f66f72f9fef 10Gi RWO local <unset> 2m21s

vmstorage-db-vmstorage-longterm-1 Bound pvc-68b5ceaf-3ed1-4e5a-9568-6b95911c7c3a 10Gi RWO local <unset> 2m21s

vmstorage-db-vmstorage-shortterm-0 Bound pvc-cee3a2a4-5680-4880-bc2a-85c14dba9380 10Gi RWO local <unset> 2m41s

vmstorage-db-vmstorage-shortterm-1 Bound pvc-d55c235d-cada-4c4a-8299-e5fc3f161789 10Gi RWO local <unset> 2m41s

Проверьте, что все pods запущены:

kubectl get pod -n tenant-root

Пример вывода:

NAME READY STATUS RESTARTS AGE

etcd-0 1/1 Running 0 2m1s

etcd-1 1/1 Running 0 106s

etcd-2 1/1 Running 0 82s

grafana-db-1 1/1 Running 0 119s

grafana-db-2 1/1 Running 0 13s

grafana-deployment-74b5656d6-5dcvn 1/1 Running 0 90s

grafana-deployment-74b5656d6-q5589 1/1 Running 1 (105s ago) 111s

root-ingress-controller-6ccf55bc6d-pg79l 2/2 Running 0 2m27s

root-ingress-controller-6ccf55bc6d-xbs6x 2/2 Running 0 2m29s

root-ingress-defaultbackend-686bcbbd6c-5zbvp 1/1 Running 0 2m29s

vmalert-vmalert-644986d5c-7hvwk 2/2 Running 0 2m30s

vmalertmanager-alertmanager-0 2/2 Running 0 2m32s

vmalertmanager-alertmanager-1 2/2 Running 0 2m31s

vminsert-longterm-75789465f-hc6cz 1/1 Running 0 2m10s

vminsert-longterm-75789465f-m2v4t 1/1 Running 0 2m12s

vminsert-shortterm-78456f8fd9-wlwww 1/1 Running 0 2m29s

vminsert-shortterm-78456f8fd9-xg7cw 1/1 Running 0 2m28s

vmselect-longterm-0 1/1 Running 0 2m12s

vmselect-longterm-1 1/1 Running 0 2m12s

vmselect-shortterm-0 1/1 Running 0 2m31s

vmselect-shortterm-1 1/1 Running 0 2m30s

vmstorage-longterm-0 1/1 Running 0 2m12s

```
vmstorage-longterm-1 1/1 Running 0 2m12s
vmstorage-shortterm-0 1/1 Running 0 2m32s
vmstorage-shortterm-1 1/1 Running 0 2m31s
```

Теперь можно получить public IP ingress controller:

```
kubectl get svc -n tenant-root root-ingress-controller
```

пример вывода:

```
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
root-ingress-controller LoadBalancer 10.96.16.141 192.168.100.200 80:31632/TCP,443:30113/TCP 3m33s
```

5.3 Доступ к Cozystack Dashboard

Если вы включили `dashboard` в `publishing.exposedServices` вашего Platform Package, Cozystack Dashboard уже должен быть доступен.

Если исходный Package не включал его, примените patch к Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p ' [{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "
dashboard"} ]'
```

Откройте `dashboard.example.org`, чтобы получить доступ к системному dashboard, где `example.org` — ваш домен, указанный для `tenant-root`. Там вы увидите окно входа, которое ожидает authentication token.

Получите authentication token для `tenant-root`:

<pre>kubectl get secret -n tenant-root tenant-root-token -o go-template='{{ printf "%s\n" (index .data "token" base64decode) }}'</pre>	
--	--

Войдите с помощью token. Теперь dashboard доступен вам как администратору.

Далее вы сможете:

- Настроить OIDC для аутентификации вместо tokens.
- Создавать user tenants и предоставлять пользователям доступ к ним через tokens или OIDC.

5.4 Доступ к метрикам в Grafana

Используйте `grafana.example.org` для доступа к системному мониторингу, где `example.org` — ваш домен, указанный для `tenant-root`. В этом примере `grafana.example.org` находится по адресу `192.168.100.200`.

- login: `admin`

- запросите password:

```
kubectl get secret -n tenant-root  
grafana-admin-password -o go-template='{{  
printf "%s\n" (index .data "password"
```

```
base64decode) }}'
```

Следующие шаги

- Настройте OIDC.
- Создайте `user tenant.nant" %}}`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Собственная платформа (BYOP)

<https://cozystack.ru/docs/v1.6/install/cozystack/kubernetes-distribution/>

Соберите собственную платформу с Cozystack, устанавливая только нужные компоненты с помощью CLI-инструмента `cozupkg`.

Обзор

Cozystack можно использовать в режиме BYOP (Build Your Own Platform) — примерно так же, как дистрибутивы Linux позволяют устанавливать только нужные пакеты. Вместо развертывания полной платформы со всеми компонентами вы выборочно устанавливаете из package repository Cozystack только то, что вам нужно.

Такой подход полезен, когда:

- У вас уже есть кластер Kubernetes, и вам нужны только отдельные компоненты (например, Postgres operator или monitoring).
- В вашем кластере уже настроены networking (CNI) и storage, и вы не хотите, чтобы Cozystack управлял ими.
- Вам нужен полный контроль над тем, какие компоненты установлены и как они настроены.

Рабочий процесс опирается на два ресурса Kubernetes, управляемых Cozystack Operator:

- PackageSource — описывает package repository и доступные variants для каждого package.
- Package — объявляет, что конкретный package должен быть установлен в выбранном variant, опционально с пользовательскими values.

CLI-инструмент `cozupkg` предоставляет удобный интерфейс для работы с этими ресурсами: просмотра доступных packages, разрешения зависимостей и интерактивной установки packages.

1\. Установка Cozystack Operator

Установите Cozystack operator с помощью Helm из OCI registry:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace
```

Замените `X.Y.Z` на нужную версию Cozystack. Доступные версии можно найти на [странице релизов Cozystack](#).

Если установка выполняется на Kubernetes-дистрибутив без Talos (k3s, kubeadm, RKE2 и т. д.), укажите `variant operator`:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system \
--create-namespace \
--set cozystackOperator.variant=generic \
--set cozystack.apiServerHost=<YOUR_API_SERVER_IP> \
--set cozystack.apiServerPort=6443
```

Operator устанавливает FluxCD (в all-in-one mode, работающем без CNI) и создает начальный `PackageSource cozystack.cozystack-platform`.

На этом этапе существует только один `PackageSource`:

```
kubectrl get packagesource
```

```
NAME VARIANTS READY STATUS
```

```
cozystack.cozystack-platform default,isp-full,isp-full... True ...
```

2\. Установка `cozypkg`

Установите CLI-инструмент `cozypkg` с помощью Homebrew:

```
brew tap cozystack/tap
brew install cozypkg
```

Готовые бинарные файлы для других платформ доступны на [странице релизов GitHub](#).

3\. Установка `Platform Package`

Первый шаг — установить package `cozystack-platform` с variant `default`. Этот variant не устанавливает компоненты — он только регистрирует `PackageSources` для всех packages, доступных в репозитории Cozystack.

```
cozypkg add cozystack.cozystack-platform
```

Инструмент предложит выбрать variant. Выберите `default`:

```
PackageSource: cozystack.cozystack-platform
```

Available variants:

1. default
2. isp-full
3. isp-full-generic
4. isp-hosted

Select variant (1-4): 1

После установки platform package станут доступны все остальные PackageSources:

```
cozypkg list
```

```
NAME VARIANTS READY STATUS
```

```
cozystack.cert-manager default True ...
```

```
cozystack.cozystack-platform default,isp-full,isp-full... True ...
```

```
cozystack.ingress-nginx default True ...
```

```
cozystack.linstor default True ...
```

```
cozystack.metallb default True ...
```

```
cozystack.monitoring default True ...
```

```
cozystack.networking noop,cilium,cilium-kilo,... True ...
```

```
cozystack.postgres-operator default True ...
```

```
...
```

4\ Установка packages

Используйте `cozypkg add`, чтобы установить любой доступный package. Инструмент автоматически разрешает зависимости и предлагает выбрать variant для каждого package, который нужно установить.

```
cozypkg add <package-name>
```

Например, при установке package, зависящего от `networking`, `cozypkg` обнаружит зависимость, покажет уже установленные packages и предложит выбрать variant для каждой отсутствующей зависимости.

Сетевые variants

Package `cozystack.networking` имеет несколько variants для разных окружений:

Variant	Описание
---------	----------

<code>noop</code>	Ничего не устанавливает. Используйте, если <code>networking</code> уже настроен в кластере (например, существующие CNI и kube-proxy).
<code>cilium</code>	Cilium CNI для кластеров Talos Linux.
<code>cilium-generic</code>	Cilium CNI для generic-дистрибутивов Kubernetes (k3s, kubeadm, RKE2).
<code>kubeovn-cilium</code>	Cilium + KubeOVN для Talos Linux. Требуется для полноценной виртуализации (live migration).
<code>kubeovn-cilium-generic</code>	Cilium + KubeOVN для generic-дистрибутивов Kubernetes.
<code>cilium-kilo</code>	Cilium + Kilo для cluster mesh на базе WireGuard.

Если в вашем кластере уже настроен CNI-плагин, выберите `noop`. Поскольку `networking` является зависимостью большинства других `packages`, `variant noop` удовлетворяет зависимость, ничего не устанавливая.

Просмотр установленных packages

Чтобы увидеть, какие `packages` сейчас установлены и какие у них `variants`:

```
cozypkg list --installed
```

```
NAME VARIANT READY STATUS
```

```
cozystack.cozystack-platform default True ...
```

```
cozystack.networking noop True ...
```

```
cozystack.cert-manager default True ...
```

5\.. Переопределение values компонентов

Каждый `package` состоит из одного или нескольких компонентов (Helm charts). Values для конкретных компонентов можно переопределить, напрямую отредактировав ресурс `Package`.

Спец `Package` поддерживает `map components`, где можно указать `values` для каждого компонента:

```
apiVersion: cozystack.io/v1alpha1
```

```
kind: Package
```

```
metadata:
```

```
name: cozystack.metallb
```

```
spec:
```

variant: default

components:

metallb:

values:

metallb:

frrk8s:

enabled: true

Примените ресурс:

```
kubectl apply -f metallb-package.yaml
```

Доступные values для компонента смотрите в соответствующем values.yaml в [репозитории Cozystack](#).

Также можно включать или отключать отдельные компоненты внутри package:

спес:

components:

some-component:

enabled: false

6\.. Удаление packages

Чтобы удалить установленный package:

```
cozypkg del <package-name>
```

Инструмент проверяет обратные зависимости: если другие установленные packages зависят от удаляемого, он перечислит их и запросит подтверждение перед удалением всех затронутых packages.

Следующие шаги

- Узнайте о [variants Cozystack](#) и о том, как они определяют состав packages.
- Подробности о переопределении параметров компонентов см. в [справочнике Components](#).
- Для установки полной платформы см. [руководство по установке Platform](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Развертывание кластера Cozystack в облаках и у хостинг-провайдеров

<https://cozystack.ru/docs/v1.6/install/providers/>

Руководства по развертыванию кластеров Cozystack у конкретных облачных и хостинг-провайдеров.

В этом разделе собраны руководства по развертыванию кластеров Cozystack у конкретных облачных и хостинг-провайдеров. Они объясняют все шаги и детали процесса развертывания, включая:

- Особенности инфраструктуры и сети провайдера
- Установку Talos Linux способом, подходящим для провайдера
- Настройку кластера Kubernetes, включая специфичные для провайдера параметры сети, хранилища и других ресурсов
- Установку Cozystack в Kubernetes-кластер с компонентами и конфигурациями, специфичными для провайдера

•

[Как установить Cozystack в Oracle Cloud Infrastructure](#)

Как установить Cozystack в Oracle Cloud Infrastructure

[Как установить Cozystack в Hetzner](#)

Как установить Cozystack в Hetzner

[Установка Cozystack в Servers.com](#)

Установка Cozystack в инфраструктуре Servers.com.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как установить Cozystack в Oracle Cloud Infrastructure

<https://cozystack.ru/docs/v1.6/install/providers/oracle-cloud/>

Как установить Cozystack в Oracle Cloud Infrastructure

Введение

В этом руководстве описано, как установить Talos в Oracle Cloud Infrastructure и развернуть кластер Kubernetes, готовый для Cozystack. После выполнения руководства можно перейти к [установке самого Cozystack](#).

Это руководство создано для поддержки развертывания development-кластеров командой Cozystack. Если при прохождении руководства возникнут проблемы, создайте issue в [cozystack/website](#) или поделитесь опытом в [сообществе Cozystack](#).

1\ Загрузка образа Talos в Oracle Cloud

Первый шаг — сделать установочный образ Talos Linux доступным в Oracle Cloud как custom image.

1. Скачайте архив образа Talos Linux для Cozystack v1.6.0 со [страницы релиза](#) и распакуйте его:

```
wget https://github.com/cozystack/cozystack/releases/download/v1.6.0/metal-amd64.raw.xz
xz -d metal-amd64.raw.xz
```

В результате вы получите файл `metal-amd64.raw`, который затем можно загрузить в OCI.

2. Следуйте документации OCI, чтобы [загрузить образ в bucket в OCI Object Storage](#).

3. Затем следуйте документации, чтобы [импортировать этот образ как custom image](#). Используйте следующие настройки:

- Image type : QCOW2
- Launch mode : Paravirtualized mode

1. Затем получите [OCID](#) образа и сохраните его для следующих шагов.

2\ Создание инфраструктуры

Цель этого шага — подготовить инфраструктуру согласно [требованиям к кластеру Cozystack](#).

Это можно сделать вручную через Oracle Cloud dashboard или с помощью Terraform.

2.1 Подготовка конфигурации Terraform

Если вы используете Terraform, первый шаг — подготовить конфигурацию.

См. [полный пример конфигурации Terraform](#) для развертывания нескольких узлов Talos в Oracle Cloud Infrastructure.

Ниже приведен сокращенный пример конфигурации Terraform, создающей три виртуальные машины со следующими private IP:

- 192.168.1.11
- 192.168.1.12
- 192.168.1.13

Эти VM также будут иметь VLAN-интерфейс с подсетью 192.168.100.0/24, используемой для внутреннего взаимодействия кластера.

Обратите внимание на часть, которая ссылается на OCID образа Talos с предыдущего шага:

```
source_details {  
  source_type = "image"  
  source_id = var.talos_image_id  
}
```

Полный пример конфигурации:

```
terraform {  
  backend "local" {}  
  required_providers {  
    oci = { source = "oracle/oci", version = "~> 6.35" }  
  }  
}  
  
resource "oci_core_vcn" "cozy_dev1" {  
  display_name = "cozy-dev1"  
  cidr_blocks = ["192.168.0.0/16"]  
  compartment_id = var.compartment_id  
}  
  
resource "oci_core_network_security_group" "cozy_dev1_allow_all" {  
  display_name = "allow-all"
```

```
compartment_id = var.compartment_id
vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_subnet" "test_subnet" {
display_name = "cozy-dev1"
cidr_block = "192.168.1.0/24"
compartment_id = var.compartment_id
vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_network_security_group_security_rule" "cozy_dev1_ingress" {
network_security_group_id = oci_core_network_security_group.cozy_dev1_allow_all.id
direction = "INGRESS"
protocol = "all"
source = "0.0.0.0/0"
source_type = "CIDR_BLOCK"
}

resource "oci_core_network_security_group_security_rule" "cozy_dev1_egress" {
network_security_group_id = oci_core_network_security_group.cozy_dev1_allow_all.id
direction = "EGRESS"
protocol = "all"
destination = "0.0.0.0/0"
destination_type = "CIDR_BLOCK"
}

resource "oci_core_internet_gateway" "cozy_dev1" {
display_name = "cozy-dev1"
compartment_id = var.compartment_id
vcn_id = oci_core_vcn.cozy_dev1.id
}

resource "oci_core_default_route_table" "cozy_dev1_default_rt" {
manage_default_resource_id = oci_core_vcn.cozy_dev1.default_route_table_id

compartment_id = var.compartment_id
display_name = "cozy?dev1?default"

route_rules {
```

```
destination = "0.0.0.0/0"
destination_type = "CIDR_BLOCK"
network_entity_id = oci_core_internet_gateway.cozy_dev1.id
}
}

resource "oci_core_vlan" "cozy_dev1_vlan" {
display_name = "cozy-dev1-vlan"
compartment_id = var.compartment_id
vcn_id = oci_core_vcn.cozy_dev1.id

cidr_block = "192.168.100.0/24"
nsg_ids = [oci_core_network_security_group.cozy_dev1_allow_all.id]
}

variable "node_private_ips" {
type = list(string)
default = ["192.168.1.11", "192.168.1.12", "192.168.1.13"]
}

variable "compartment_id" {
description = "OCID of the OCI compartment"
type = string
}

variable "availability_domain" {
description = "Availability domain for the instances"
type = string
}

variable "talos_image_id" {
description = "OCID of the imported Talos Linux image"
type = string
}

resource "oci_core_instance" "cozy_dev1_nodes" {
count = length(var.node_private_ips)
display_name = "cozy-dev1-node-${count.index + 1}"
compartment_id = var.compartment_id
availability_domain = var.availability_domain
```

```
shape = "VM.Standard3.Flex"
preserve_boot_volume = false
preserve_data_volumes_created_at_launch = false

create_vnic_details {
  subnet_id = oci_core_subnet.test_subnet.id
  nsg_ids = [oci_core_network_security_group.cozy_dev1_allow_all.id]
  private_ip = var.node_private_ips[count.index]
}

source_details {
  source_type = "image"
  source_id = var.talos_image_id
}

launch_volume_attachments {
  display_name = "cozy-dev1-node${count.index + 1}-data"
  launch_create_volume_details {
    display_name = "cozy-dev1-node${count.index + 1}-data"
    compartment_id = var.compartment_id
    size_in_gbs = "512"
    volume_creation_type = "ATTRIBUTES"
    vpus_per_gb = "10"
  }
  type = "paravirtualized"
}

shape_config {
  memory_in_gbs = "32"
  ocpus = "4"
}

resource "oci_core_vnic_attachment" "cozy_dev1_vlan_vnic" {
  count = length(var.node_private_ips)
  instance_id = oci_core_instance.cozy_dev1_nodes[count.index].id

  create_vnic_details {
    vlan_id = oci_core_vlan.cozy_dev1_vlan.id
  }
}
```

```
}  
}
```

2.2 Применение конфигурации

Когда конфигурация будет готова, аутентифицируйтесь в OCI и примените ее с Terraform:

```
oci session authenticate --region us-ashburn-1 --profile-name=DEFAULT
```

```
terraform init
```

```
terraform apply
```

В результате этих команд виртуальные машины будут развернуты и настроены.

Сохраните public IP-адреса, назначенные ВМ, для следующего шага. В этом примере адреса такие:

```
- 1.2.3.4
```

```
- 1.2.3.5
```

```
- 1.2.3.6
```

3\ Настройка Talos и инициализация кластера Kubernetes

Следующий шаг — применить конфигурации и установить Talos Linux. Это можно сделать несколькими способами.

В этом руководстве используется [Talm](#) — CLI-инструмент для декларативного управления Talos Linux. В Talm есть шаблоны конфигурации, специализированные для развертывания Cozystack, поэтому мы используем его.

Если Talm не установлен, [скачайте последний binary](#) для вашей ОС и архитектуры. Сделайте его исполняемым и сохраните в `/usr/local/bin/talm`:

```
# выберите нужную архитектуру из release artifacts
```

```
wget -O talm https://github.com/cozystack/talm/releases/latest/download/talm-darwin-arm64
```

```
chmod +x talm
```

```
mv talm /usr/local/bin/talm
```

3.1 Подготовка конфигурации Talm

1. Создайте каталог для конфигурационных файлов нового кластера:

```
mkdir -p cozystack-cluster
```

```
cd cozystack-cluster
```

2. Инициализируйте конфигурацию Talm для Cozystack:

```
talm init --preset cozystack --name mycluster
```

3. Сгенерируйте шаблон конфигурации для каждого узла, указав IP-адрес узла:

Используйте public IP узла, назначенный OCI

```
talm template \  
--nodes 1.2.3.4 \  
--endpoints 1.2.3.4 \  
--template templates/controlplane.yaml \  
--insecure \  
> nodes/node0.yaml
```

Повторите то же для каждого узла, используя его public IP:

```
talm template ... > nodes/node1.yaml  
talm template ... > nodes/node2.yaml
```

Использование `templates/controlplane.yaml` означает, что узел будет работать одновременно как control plane и worker. Три совмещенных узла — предпочтительная конфигурация для небольшого PoC-кластера.

Параметр `--insecure (-i)` нужен потому, что Talm должен получить конфигурационные данные с узла, который еще не инициализирован и поэтому не может принять аутентифицированное соединение. Узел будет инициализирован несколькими шагами позже с помощью `talm apply`.

Public IP узла должен быть указан и в параметре `--nodes (-n)`, и в `--endpoints (-e)`. Подробнее о конфигурации узлов Talos и endpoints см. в [документации Talos](#)

4. При необходимости отредактируйте конфигурационный файл узла.

- Обновите `hostname` на нужное имя:

```
``yaml  
machine:  
network:  
hostname: node1  
``
```

- Добавьте конфигурацию private interface в раздел `machine.network.interfaces` и перенесите `vip` в эту конфигурацию. Эта часть конфигурации не генерируется автоматически, поэтому значения нужно заполнить вручную:

- `interface`: берется из “Discovered interfaces” путем сопоставления параметров `private interface`.

- `addresses`: используйте адрес, указанный для Layer 2 (L2).

Пример:

machine:

network:

interfaces:

- `interface`: `eth0`

addresses:

- `1.2.3.4/29`

routes:

- `network`: `0.0.0.0/0`

gateway: `1.2.3.1`

- `interface`: `eth1`

addresses:

- `192.168.100.11/24`

`vip`:

`ip`: `192.168.100.10`

После этих шагов конфигурационные файлы узлов готовы к применению.

3.2 Инициализация Talos и запуск кластера Kubernetes

Следующий этап — инициализировать узлы Talos и выполнить `bootstrap` кластера Kubernetes.

1. Выполните `talm apply` для всех узлов, чтобы применить конфигурации:

```
talm apply -f nodes/node0.yaml --insecure
```

```
talm apply -f nodes/node1.yaml --insecure
```

```
talm apply -f nodes/node2.yaml --insecure
```

Узлы перезагрузятся, и Talos будет установлен на диск. Параметр `--insecure (-i)` нужен при первом запуске `talm apply` на каждом узле.

2. Выполните `talm bootstrap` на первом узле кластера. Например:

```
talm bootstrap -f nodes/node0.yaml
```

3. Получите `kubeconfig` с любого узла control plane с помощью Talm. В этом примере все три узла являются control-plane узлами:

```
talm kubeconfig -f nodes/node0.yaml
```

4. Отредактируйте `kubeconfig`, указав IP-адрес `server` как один из узлов `control plane`, например:

```
server: https://1.2.3.4:6443
```

5. Экспортируйте переменную `KUBECONFIG`, чтобы использовать `kubeconfig`, и проверьте подключение к кластеру:

```
export KUBECONFIG=${PWD}/kubeconfig
```

```
kubectl get nodes
```

Вы должны увидеть, что узлы доступны и находятся в состоянии `NotReady`, что ожидаемо на этом этапе:

```
NAME STATUS ROLES AGE VERSION
```

```
node0 NotReady control-plane 2m21s v1.32.0
```

```
node1 NotReady control-plane 1m47s v1.32.0
```

```
node2 NotReady control-plane 1m43s v1.32.0
```

Теперь у вас есть кластер Kubernetes, подготовленный к установке Cozystack. Чтобы завершить установку, следуйте руководству по разворачиванию, начиная с раздела [Установка Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как установить Cozystack в Hetzner

<https://cozystack.ru/docs/v1.6/install/providers/hetzner/>

Как установить Cozystack в Hetzner

Это руководство поможет установить Cozystack на выделенные серверы [Hetzner](#). Процесс состоит из нескольких шагов: подготовки инфраструктуры, установки Talos Linux, настройки cloud-init и инициализации кластера.

Подготовка инфраструктуры и сети

Установка в Hetzner включает общие [требования к оборудованию](#) с несколькими дополнениями.

Варианты сети

Есть два варианта сетевой связности между узлами Cozystack в кластере:

- Создание подсети с помощью vSwitch. Этот вариант рекомендуется для production-сред.

Для этого варианта выделенные серверы должны быть развернуты через [Hetzner robot](#). Также Hetzner требует использовать собственный load balancer RobotLB вместо стандартного для Cozystack MetalLB. Cozystack включает RobotLB как optional component начиная с релиза v0.35.0.

- Использование только public IP выделенных серверов. Этот вариант подходит для proof-of-concept установки, но не рекомендуется для production.

Настройка подсети с vSwitch

Выполните следующие шаги, чтобы подготовить серверы к установке Cozystack:

1. Выполните сетевые настройки в Hetzner (только для варианта vSwitch subnet).

Выполните шаги из [раздела Prerequisites](#) в README RobotLB:

1. Создайте [vSwitch](#).
2. Используйте его, чтобы назначить IP вашим выделенным серверам в Hetzner.
3. Создайте подсеть, чтобы [подключить выделенные серверы](#).

Обратите внимание, что RobotLB не нужно разворачивать вручную. Вместо этого вы настроите Cozystack на установку RobotLB как optional component на шаге "Установка Cozystack" в этом руководстве.

Отключение Secure Boot

1. Убедитесь, что Secure Boot отключен.

The Talos installation procedure used in this guide (rescue-mode `installimage` writing a standard EFI/MBR layout) requires Secure Boot disabled. Talos does support UEFI Secure Boot via its UKI / SecureBoot installation flow, but that path is not covered here. If your server is configured to use Secure Boot, disable it in BIOS before continuing — otherwise it will block the server from booting after Talos installation.

Проверьте это следующей командой:

```
# mokutil --sb-state
```

```
SecureBoot disabled
```

```
Platform is in Setup Mode
```

В остальной части руководства будем считать, что используется следующая сетевая конфигурация:

- Облачная сеть Hetzner — `10.0.0.0/16`, имя `network-1`.
- vSwitch subnet с выделенными серверами — `10.0.1.0/24`
- VLAN ID vSwitch — `4000`
- Есть три выделенных сервера со следующими public и private IP:
 - `node1`, public IP `12.34.56.101`, IP в vSwitch subnet `10.0.1.101`
 - `node2`, public IP `12.34.56.102`, IP в vSwitch subnet `10.0.1.102`
 - `node3`, public IP `12.34.56.103`, IP в vSwitch subnet `10.0.1.103`

1\ Установка Talos Linux

Первый этап развертывания Cozystack — установка Talos Linux на выделенные серверы.

Talos — дистрибутив Linux, созданный для максимально безопасного и эффективного запуска Kubernetes. Чтобы узнать, почему Cozystack использует Talos как основу кластера, прочитайте [Talos Linux в Cozystack](#).

1.1 Установка boot-to-talos в Rescue Mode

Talos будет загружен из rescue system Hetzner с помощью утилиты `[boot-to-talos]` (<https://github.com/cozystack/boot-to-talos>). Позже, при применении конфигурации Talm, Talos будет установлен на диск. Выполните эти шаги на каждом выделенном сервере.

1. Переведите сервер в rescue mode и войдите на него по SSH.
2. Определите диск, который позже будет использоваться для Talos (например, `/dev/nvme0n1`).
3. Скачайте и установите `boot-to-talos`:


```
curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh
```

```
sh -s
```

После этого бинарный файл `boot-to-talos` должен быть доступен в `PATH`:

```
boot-to-talos -h
```

1.2. Установка Talos Linux с boot-to-talos

1. Запустите installer:

```
boot-to-talos
```

При запросе:

- Выберите режим `1. boot`.
- Подтвердите или измените образ Talos installer. Значение по умолчанию указывает на образ Talos от Cozystack (стандартный образ Cozystack подходит).
- Укажите сетевые настройки (имя интерфейса, IP-адрес, netmask, gateway), соответствующие подготовленной ранее конфигурации (vSwitch subnet или public IP).
- При необходимости настройте serial console, если используете ее для удаленного доступа.

Утилита скачает образ Talos installer, извлечет kernel и initramfs и загрузит узел в Talos Linux (с помощью механизма kexes), не изменяя диски.

1.3. Загрузка в Talos Linux

После завершения `boot-to-talos` сервер автоматически перезагрузится в Talos Linux в maintenance mode.

Повторите ту же процедуру для всех выделенных серверов в кластере. Когда все узлы загрузятся в Talos, переходите к следующему разделу и настройте их с помощью Talm.

2\ Установка кластера Kubernetes

Теперь, когда Talos загружен в maintenance mode, он должен получить конфигурацию и настроить кластер Kubernetes. Есть [несколько вариантов](#) создания и применения конфигурации Talos. В этом руководстве используется [Talm](#) — собственный инструмент Cozystack для управления конфигурацией Talos.

Эта часть руководства основана на общем [руководстве по Talm](#), но содержит инструкции и примеры, специфичные для Hetzner.

2.1. Подготовка конфигурации узлов с Talm

1. Если Talm еще не установлен, начните с установки последней версии для вашей ОС:

<pre>curl -sSL https://github.com/cozystack/talm/raw/refs/heads/main/hack/install.sh</pre>	<pre>sh -s</pre>
--	------------------

2. Создайте каталог для конфигурации кластера и инициализируйте в нем проект Talm.

Обратите внимание, что в Talm есть встроенный preset для Cozystack, который используется через `--preset cozystack`:

```
mkdir -p hetzner-cluster
```

```
cd hetzner-cluster
```

```
talm init --preset cozystack --name hetzner
```

Теперь в каталоге `hetzner-cluster` создан набор файлов. Подробнее о роли каждого файла см. в [руководстве по Talm](#).

3. Отредактируйте `values.yaml`, изменив следующие значения:

- В списке `advertisedSubnets` должна быть vSwitch subnet.
- `endpoint` и `floatingIP` должны использовать неназначенный IP из этой подсети. Этот IP будет использоваться для доступа к cluster API через `talosctl` и `kubectl`.
- `podSubnets` и `serviceSubnets` должны использовать другие подсети из облачной сети Hetzner, которые не пересекаются друг с другом и с vSwitch subnet.

```
endpoint: "https://10.0.1.100:6443"
```

```
clusterDomain: cozy.local
```

```
# floatingIP указывает на primary etcd node
```

```
floatingIP: 10.0.1.100
```

```
image: "ghcr.io/cozystack/cozystack/talos:v1.13.6"
```

```
podSubnets:
```

- 10.244.0.0/16

```
serviceSubnets:
```

- 10.96.0.0/16

```
advertisedSubnets:
```

```
# vSwitch subnet
```

- 10.0.1.0/24

```
oidcIssuerUrl: ""
```

```
certSANs: []
```

1. Создайте конфигурационные файлы узлов из шаблонов и values:

```
mkdir -p nodes
```

```
talm template -e 12.34.56.101 -n 12.34.56.101 -t templates/controlplane.yaml -i > nodes/node1.yaml
```

```
talm template -e 12.34.56.102 -n 12.34.56.102 -t templates/controlplane.yaml -i > nodes/node2.yaml
```

```
talm template -e 12.34.56.103 -n 12.34.56.103 -t templates/controlplane.yaml -i > nodes/node3.yaml
```

В этом руководстве предполагается, что у вас только три выделенных сервера, поэтому все они должны быть узлами control plane. Если серверов больше и вы хотите разделить control plane и worker-узлы, используйте `templates/worker.yaml` для создания worker configs:

```
talm template -e 12.34.56.104 -n 12.34.56.104 -t templates/worker.yaml -i > nodes/worker1.yaml
```

2. Отредактируйте конфигурационный файл каждого узла, добавив VLAN configuration.

Используйте следующий diff как пример и учитывайте, что для каждого узла должен использоваться его IP в subnet:

```
machine:
```

```
network:
```

```
interfaces:
```

```
    - deviceSelector:
```

```
    # ...
```

```
    - vip:
```

```
    - ip: 10.0.1.100
```

```
+ vlans:
```

```
+ - addresses:
```

```
+ # отличается для каждого узла
```

```
+ - 10.0.1.101/24
```

```
+ routes:
```

```
+ - network: 10.0.0.0/16
```

```
+ gateway: 10.0.1.1
```

```
+ vlanId: 4000
```

```
+ vip:
```

```
+ ip: 10.0.1.100
```

2.2. Применение конфигурации узлов

1. Когда конфигурационные файлы будут готовы, примените конфигурацию к каждому узлу:

```
talm apply -f nodes/node1.yaml -i
```

```
talm apply -f nodes/node2.yaml -i
```

```
talm apply -f nodes/node3.yaml -i
```

Эта команда инициализирует узлы и настраивает аутентифицированное соединение, поэтому дальше `-i` (`--insecure`) не потребуется. Если команда выполнена успешно, она вернет IP узла:

```
$ talm apply -f nodes/node1.yaml -i
```

```
- talm: file=nodes/node1.yaml, nodes=[12.34.56.101], endpoints=[12.34.56.101]
```

2. Дождитесь, пока все узлы перезагрузятся, и переходите к следующему шагу. Когда узлы будут готовы, они откроют порт `50000`; это признак того, что узел завершил настройку Talos и перезагрузился.

Если нужно автоматизировать проверку готовности узлов, используйте такой пример:

```
timeout 60 sh -c 'until \  
nc -nzv 12.34.56.101 50000 && \  
nc -nzv 12.34.56.102 50000 && \  
nc -nzv 12.34.56.103 50000; \  
do sleep 1; done'
```

3. Инициализируйте кластер Kubernetes с одного из узлов control plane:

```
talm bootstrap -f nodes/node1.yaml
```

4. Сгенерируйте административный `kubeconfig` для доступа к кластеру через тот же узел control plane:

```
talm kubeconfig -f nodes/node1.yaml
```

5. Измените URL server в `kubeconfig` на public IP

```
apiVersion: v1
```

```
clusters:
```

```
- cluster:
```

```
- server: https://10.0.1.101:6443
```

```
+ server: https://12.34.56.101:6443
```

6. Затем настройте переменную `KUBECONFIG` или другие инструменты, чтобы сделать эту конфигурацию доступной клиенту `kubectl`:

```
export KUBECONFIG=$PWD/kubeconfig
```

7. Проверьте, что кластер доступен с новым `kubeconfig`:

```
kubectl get ns
```

Пример вывода:

```
NAME STATUS AGE
default Active 7m56s
kube-node-lease Active 7m56s
kube-public Active 7m56s
kube-system Active 7m56s
```

На этом этапе у вас есть выделенные серверы с Talos Linux и развернутый на них кластер Kubernetes. Также у вас есть `kubeconfig`, который будет использоваться для доступа к кластеру через `kubectl` и установки Cozystack.

3\. Установка Cozystack

Финальный этап развертывания кластера Cozystack в Hetzner — установка Cozystack в подготовленный кластер Kubernetes.

3.1. Запуск Cozystack Installer

1. Установите Cozystack operator:

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version 1.6.0 \
--namespace cozy-system \
--create-namespace
```

В примере installer закреплен на Cozystack v1.6.0. Для более нового patch-релиза в той же minor-серии выберите нужный tag на [странице релизов](#).

2. Создайте файл Platform Package, `cozystack-platform.yaml`.

Обратите внимание, что этот файл повторно использует подсети для pods и services, которые использовались в `values.yaml` перед созданием конфигурации Talos с Talm. Также обратите внимание, как стандартный load balancer Cozystack MetalLB заменяется на RobotLB с помощью `disabledPackages` и `enabledPackages`.

Замените `example.org` на маршрутизируемое fully-qualified domain name (FQDN), которое вы будете использовать для платформы на базе Cozystack. Если такого домена нет, можно использовать [nip.io](#) с записью через дефис.

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
```

```
name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
  platform:
  values:
  bundles:
  disabledPackages:
    - cozystack.metallb
  enabledPackages:
    - cozystack.hetzner-robotlb
  publishing:
  host: "example.org"
  apiServerEndpoint: "https://api.example.org:443"
  exposedServices:
    - dashboard
    - api
  networking:
  ## podSubnets из конфигурации узлов
  podCIDR: "10.244.0.0/16"
  podGateway: "10.244.0.1"
  ## serviceSubnets из конфигурации узлов
  serviceCIDR: "10.96.0.0/16"
```

3. Примените Platform Package:

```
kubectl apply -f cozystack-platform.yaml
```

Operator запустит установку, которая займет некоторое время. При необходимости можно отслеживать логи operator:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

4. Проверьте состояние установки:

```
kubectl get hr -A
```

Когда установка завершится, все сервисы перейдут в состояние `READY: True`:

```
NAMESPACE NAME AGE READY STATUS
```

```
cozy-cert-manager cert-manager 4m1s True Release reconciliation succeeded
```

```
cozy-cert-manager cert-manager-issuers 4m1s True Release reconciliation succeeded
cozy-cilium cilium 4m1s True Release reconciliation succeeded
...
```

3.2 Создание Load Balancer с RobotLB

Hetzner требует использовать собственный RobotLB вместо стандартного MetalLB в Cozystack. RobotLB уже установлен как компонент Cozystack и работает как сервис внутри него. Теперь ему нужен token для создания ресурса load balancer в Hetzner.

1. Создайте Hetzner API token для RobotLB.

Перейдите в консоль Hetzner, откройте Security и создайте token с разрешениями `Read` и `Write`.

2. Передайте token в RobotLB, чтобы создать load balancer в Hetzner.

Используйте Hetzner API token, чтобы создать Kubernetes secret в Cozystack.

- Если используется private network (vSwitch), укажите имя сети:

```
```bash
export ROBOTLB_HCLOUD_TOKEN="<<token>"
export ROBOTLB_DEFAULT_NETWORK="<<network name>"

kubectl create secret generic hetzner-robotlb-credentials \
--namespace=cozy-hetzner-robotlb \
--from-literal=ROBOTLB_HCLOUD_TOKEN="$ROBOTLB_HCLOUD_TOKEN" \
--from-literal=ROBOTLB_DEFAULT_NETWORK="$ROBOTLB_DEFAULT_NETWORK"
```
```

- Если используются только public IP (без vSwitch), не указывайте `ROBOTLB_DEFAULT_NETWORK`:

```
```bash
export ROBOTLB_HCLOUD_TOKEN="<<token>"

kubectl create secret generic hetzner-robotlb-credentials \
--namespace=cozy-hetzner-robotlb \
--from-literal=ROBOTLB_HCLOUD_TOKEN="$ROBOTLB_HCLOUD_TOKEN"
```
```

В этом случае RobotLB будет использовать public IP узлов (ExternalIP) как targets load balancer.

Чтобы это работало, на узлах должны быть настроены адреса ExternalIP.

Самый простой способ добиться этого — установить [local-ccm](#),

который автоматически назначает public IP в поле `.status.addresses` узлов.

После получения token сервис RobotLB в Cozystack создаст load balancer в Hetzner.

3.3 Настройка хранилища с LINSTOR

Настройка LINSTOR в Hetzner не отличается от других инфраструктурных конфигураций. Следуйте [руководству по настройке хранилища](#) из tutorial по Cozystack.

3.4. Запуск сервисов в Root Tenant

Настройте базовые сервисы (`etcd`, `monitoring` и `ingress`) в root tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{
  "spec": {
    "ingress": true,
    "monitoring": true,
    "etcd": true
  }
}'
```

Примечания и troubleshooting

[!] Если возникают проблемы с загрузкой Talos Linux на узле, это может быть связано с параметрами serial console в конфигурации GRUB: `console=tty1 console=ttyS0`. Попробуйте перезагрузиться в rescue mode и удалить эти параметры из конфигурации GRUB на третьем разделе основного системного диска (`$DISK1`).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Установка Cozystack в Servers.com

<https://cozystack.ru/docs/v1.6/install/providers/servers-com/>

Установка Cozystack в инфраструктуре Servers.com.

Перед установкой

1\ Сеть

1. Настройте L2 Network

1. Перейдите в Networks > L2 Segment и нажмите Add Segment.

Сначала выберите Private , выберите регион, добавьте серверы, задайте имя и сохраните.

2. Установите тип Native. Сделайте то же самое для Public.

2\ Доступ

1. Создайте SSH-ключи для доступа к серверу.

2. Перейдите в Identity and Access > SSH and Keys.

3. Создайте новые ключи или добавьте свои.

Настройка ОС

1\. Операционная система и доступ

[!] В rescue mode доступна только публичная сеть; частная L2 network недоступна. Для установки Talos используйте обычную ОС (например, Ubuntu), а не rescue mode.

1. В панели управления Servers.com установите Ubuntu на сервер (например, через Dedicated Servers > Server Details > OS install) и убедитесь, что выбран ваш SSH-ключ.

2. После завершения установки подключитесь по SSH, используя внешний IP сервера (Details > Public IP).

2\. Установка Talos с boot-to-talos

Talos будет загружен из установленной Ubuntu с помощью утилиты `[boot-to-talos]`(<https://github.com/cozystack/boot-to-talos>). Позже, при применении конфигурации Talm, Talos будет установлен на диск. Выполните эти шаги на каждом сервере.

1. Проверьте информацию о блочных устройствах, чтобы найти диск, который позже будет использоваться для Talos (например, `/dev/sda`).

```
# lsblk
```

```
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
```

```
sda 259:4 0 476.9G 0 disk
```

```
sdb 259:0 0 476.9G 0 disk
```

2. Скачайте и установите `boot-to-talos`:

| | |
|---|-----------------------|
| <pre>curl -sSL https://github.com/cozystack/boot-to-talos/raw/refs/heads/main/hack/install.sh</pre> | <pre>sudo sh -s</pre> |
|---|-----------------------|

После установки проверьте, что бинарный файл доступен:

```
boot-to-talos -h
```

3. Запустите installer:

```
sudo boot-to-talos
```

При запросе:

- Выберите режим 1. `boot`.
- Подтвердите или измените образ Talos installer (стандартный образ Cozystack подходит).
- Укажите сетевые настройки, соответствующие публичному интерфейсу (`bond0`) и default gateway.

Утилита скачает образ Talos installer и загрузит узел в Talos Linux (с помощью механизма kexes), не изменяя диски.

Для полностью автоматизированной установки можно использовать non-interactive mode:

```
sudo boot-to-talos -yes
```

3\ Загрузка в Talos

После завершения `boot-to-talos` сервер автоматически перезагрузится в Talos Linux в maintenance mode. Повторите ту же процедуру для всех серверов, затем переходите к их настройке с Talm.

Конфигурация Talos

Используйте [Talm](#), чтобы применить config и установить Talos Linux на диск.

1. Скачайте последний [binary Talm](#) и сохраните его в `/usr/local/bin/talm`

2. Сделайте его исполняемым:

```
chmod +x /usr/local/bin/talm
```

Установка с Talm

1. Создайте каталог для нового кластера:

```
mkdir -p cozystack-cluster
```

```
cd cozystack-cluster
```

2. Выполните следующую команду, чтобы инициализировать Talm для Cozystack:

```
talm init --preset cozystack --name mycluster
```

После инициализации сгенерируйте шаблон конфигурации командой:

```
talm -n 1.2.3.4 -e 1.2.3.4 template -t templates/controlplane.yaml -i > nodes/nodeN.yaml
```

3. При необходимости отредактируйте конфигурационный файл узла:

1. Обновите `hostname` на нужное имя.

```
machine:
```

```
network:
```

```
hostname: node1
```

2. Обновите `nameservers`, указав публичные серверы, потому что внутренний DNS `servers.com` недоступен из частной сети.

```
machine:
```

```
network:
```

```
nameservers:
```

```
- 8.8.8.8
```

```
- 1.1.1.1
```

3. Добавьте конфигурацию `private interface` и перенесите `vip` в этот раздел. Этот раздел не генерируется автоматически:

-
- `interface` \- берется из “Discovered interfaces” путем сопоставления MAC-адреса `private interface`, указанного в письме провайдера. (Из двух интерфейсов выберите тот, у которого есть `uplink`).
 - `addresses` \- используйте адрес, указанный для Layer 2 (L2).

machine:

network:

interfaces:

- interface: `bond0`

addresses:

- `1.2.3.4/29`

routes:

- network: `0.0.0.0/0`

gateway: `1.2.3.1`

bond:

interfaces:

- `enp1s0f1`
- `enp3s0f1`

mode: `802.3ad`

xmitHashPolicy: `layer3+4`

lacpRate: `slow`

miimon: `100`

- interface: `bond1`

addresses:

- `192.168.102.11/23`

bond:

interfaces:

- `enp1s0f0`
- `enp3s0f0`

mode: `802.3ad`

xmitHashPolicy: `layer3+4`

lacpRate: `slow`

miimon: `100`

vip:

ip: `192.168.102.10`

Шаги выполнения:

1. Выполните `talm apply -f nodeN.yml` для всех узлов, чтобы применить конфигурации. Узлы будут перезагружены, и Talos будет установлен на диск.

2. Убедитесь, что Talos установлен на диск, выполнив `talm get systemdisk -f nodeN.yml` для каждого узла. Вывод должен быть похож на:

```
NODE NAMESPACE TYPE ID VERSION DISK
```

```
1.2.3.4 runtime SystemDisk system-disk 1 sda
```

Если вывод пустой, значит Talos все еще работает из RAM и еще не установлен на диск.

3. Выполните bootstrap-команду для первого узла кластера, например:

```
talm bootstrap -f nodes/node1.yml
```

4. Получите `kubeconfig` с первого узла, например:

```
talm kubeconfig -f nodes/node1.yml
```

5. Отредактируйте `kubeconfig`, указав IP-адрес одного из узлов control plane, например:

```
server: https://1.2.3.4:6443
```

6. Экспортируйте переменную для использования `kubeconfig` и проверьте подключение к Kubernetes:

```
export KUBECONFIG=${PWD}/kubeconfig
```

```
kubectl get nodes
```

Теперь для продолжения установки следуйте руководству Get Started , начиная с раздела [Установка Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Автоматизированная установка с Ansible

<https://cozystack.ru/docs/v1.6/install/ansible/>

Развертывание Cozystack на generic Kubernetes с помощью Ansible collection `cozystack.installer`

Ansible collection `[cozystack.installer]`(<https://github.com/cozystack/ansible-cozystack>) автоматизирует весь pipeline развертывания: подготовку ОС, bootstrap кластера k3s и установку Cozystack. Она подходит для развертывания Cozystack на bare-metal серверах или VM со стандартным дистрибутивом Linux.

Когда использовать Ansible

Рассмотрите этот подход, если:

- Вам нужно полностью автоматизированное, повторяемое развертывание от чистой ОС до работающего Cozystack
- Вы развертываете платформу на generic Linux (Ubuntu, Debian, RHEL, Rocky, openSUSE), а не на Talos Linux
- Вы хотите управлять несколькими узлами через один inventory-файл

Шаги ручной установки без Ansible см. в руководстве [Generic Kubernetes](#).

Предварительные требования

Управляющая машина

- Python ≥ 3.9
- Ansible ≥ 2.15

Целевые узлы

- Операционная система : Ubuntu/Debian, RHEL 8+/CentOS Stream 8+/Rocky/Alma или openSUSE/SLE
- Архитектура : amd64 или arm64
- SSH-доступ с passwordless sudo
- Требования к CPU, RAM и дискам см. в [требованиях к оборудованию](#)

Установка

1\ Установка Ansible collection

```
ansible-galaxy collection install git+https://github.com/cozystack/ansible-cozystack.git
```

Установите необходимые dependency collections. Файл `requirements.yml` не включен в packaged collection, поэтому скачайте его из репозитория:

```
curl --silent --location --output /tmp/requirements.yml \
https://raw.githubusercontent.com/cozystack/ansible-cozystack/main/requirements.yml
ansible-galaxy collection install --requirements-file /tmp/requirements.yml
```

Это установит следующие зависимости:

- `ansible.posix`, `community.general`, `kubernetes.core` — из Ansible Galaxy
- `[k3s.orchestration](https://github.com/k3s-io/k3s-ansible)` — collection для развертывания k3s, устанавливается из Git

2\ Создание inventory

Создайте файл `inventory.yml`. Внутренний (private) IP каждого узла должен использоваться как host key, потому что KubeOVN проверяет IP хостов через `NODE_IPS`. Public IP, если он отличается, указывается в `ansible_host`.

```
cluster:
```

```
children:
```

```
server:
```

```
hosts:
```

```
10.0.0.10:
```

```
ansible_host: 203.0.113.10
```

```
agent:
```

```
hosts:
```

```
10.0.0.11:
```

```
ansible_host: 203.0.113.11
```

```
10.0.0.12:
```

```
ansible_host: 203.0.113.12
```

```
vars:
```

```
ansible_port: 22
```

```
ansible_user: ubuntu
```

```
# настройки k3s — доступные версии см. на https://github.com/k3s-io/k3s/releases
```

```
k3s_version: v1.35.0+k3s3
token: "CHANGE_ME" # ЗАМЕНИТЕ на надежный случайный secret
api_endpoint: "10.0.0.10"
cluster_context: my-cluster

# настройки Cozystack
cozystack_api_server_host: "10.0.0.10"
cozystack_root_host: "cozy.example.com"
cozystack_platform_variant: "isp-full-generic"
# cozystack_k3s_extra_args: "--tls-san=203.0.113.10" # добавьте public IP, если узлы находятся за NAT

**Заменитетoken на надежный случайный secret.** Этот token используется для подключения
узлов k3s и дает полный доступ к кластеру. Сгенерируйте его командой openssl rand -hex 32.

**Всегда явно закрепляйтесozystack_chart_version.** Collection поставляется с версией по
умолчанию, которая может не совпадать с релизом, который вы хотите развернуть. Укажите ее в
inventory, чтобы избежать неожиданных обновлений:

cozystack_chart_version: "1.6.0"
```

Доступные версии см. в [релизах Cozystack](#).

3\ Создание playbook

Создайте файл `site.yml`, который последовательно запускает подготовку ОС, развертывание k3s и установку Cozystack.

Репозиторий collection содержит примеры prepare playbooks для каждого поддерживаемого семейства ОС в каталоге `[examples/]` (<https://github.com/cozystack/ansible-cozystack/tree/main/examples>). Скопируйте подходящий для вашей целевой ОС файл в каталог проекта, затем подключите его как локальный файл:

-

Скопируйте `prepare-ubuntu.yml` из `examples/ubuntu/`, затем создайте `site.yml`:

```
- name: Prepare nodes
ansible.builtin.import_playbook: prepare-ubuntu.yml

- name: Deploy k3s cluster
ansible.builtin.import_playbook: k3s.orchestration.site

- name: Install Cozystack
ansible.builtin.import_playbook: cozystack.installer.site
```

Скопируйте `prepare-rhel.yml` из `examples/rhel/`, затем создайте `site.yml`:

```
- name: Prepare nodes
ansible.builtin.import_playbook: prepare-rhel.yml

- name: Deploy k3s cluster
ansible.builtin.import_playbook: k3s.orchestration.site

- name: Install Cozystack
ansible.builtin.import_playbook: cozystack.installer.site
```

Скопируйте `prepare-suse.yml` из [examples/suse/](#), затем создайте `site.yml`:

```
- name: Prepare nodes
ansible.builtin.import_playbook: prepare-suse.yml

- name: Deploy k3s cluster
ansible.builtin.import_playbook: k3s.orchestration.site

- name: Install Cozystack
ansible.builtin.import_playbook: cozystack.installer.site
```

4\ Запуск playbook

```
ansible-playbook --inventory inventory.yml site.yml
```

Playbook автоматически выполняет следующие шаги:

1. Prepare nodes — устанавливает необходимые пакеты (`nfs-common`, `open-iscsi`, `multipath-tools`), настраивает `sysctl`, включает `storage services`
2. Deploy k3s — выполняет bootstrap кластера k3s с настройками, совместимыми с Cozystack (отключает встроенные Traefik, ServiceLB, kube-proxy, Flannel; задает `cluster-domain=cozy.local`)
3. Install Cozystack — устанавливает Helm и plugin `helm-diff` (используется для идиempотентных обновлений), разворачивает chart `cozy-installer`, ожидает `operator` и `CRD`, затем создает Platform Package

Справочник конфигурации

Основные переменные

| Переменная | По умолчанию | Описание |
|--|------------------------------|-----------------------------------|
| <code>cozystack_api_server_host</code> | <code>_(обязательно)_</code> | Внутренний IP узла control plane. |

| | | |
|----------------------------|------------------|---|
| cozystack_chart_version | 1.6.0 | Версия Helm chart Cozystack. Закрепляйте явно. |
| cozystack_platform_variant | isp-full-generic | Platform variant: default, isp-full, isp-hosted, isp-full-generic. |
| cozystack_root_host | "" | Домен для сервисов Cozystack. Оставьте пустым, чтобы пропустить publishing configuration. |

Сеть

| Переменная | По умолчанию | Описание |
|---------------------------|---------------|---|
| cozystack_pod_cidr | 10.42.0.0/16 | Диапазон Pod CIDR. |
| cozystack_pod_gateway | 10.42.0.1 | Gateway pod-сети. |
| cozystack_svc_cidr | 10.43.0.0/16 | Диапазон Service CIDR. |
| cozystack_join_cidr | 100.64.0.0/16 | Join CIDR для межузлового взаимодействия. |
| cozystack_api_server_port | 6443 | Порт Kubernetes API server. |

Расширенные настройки

| Переменная | По умолчанию | Описание |
|-----------------------------------|--|--|
| cozystack_chart_ref | oci://ghcr.io/cozystack/cozystack-cozy-installer | OCI reference для Helm chart. |
| cozystack_operator_variant | generic | Operator variant: generic, talos, hosted. |
| cozystack_namespace | cozy-system | Namespace для Cozystack operator и ресурсов. |
| cozystack_release_name | cozy-installer | Имя Helm release. |
| cozystack_release_namespace | kube-system | Namespace, где хранится secret Helm release (не namespace operator). |
| cozystack_kubeconfig | /etc/rancher/k3s/k3s.yaml | Путь к kubeconfig на целевом узле. |
| cozystack_create_platform_package | true | Создавать ли Platform Package после установки chart. |

| | | |
|--------------------------------|---------------------|---|
| coystack_helm_version | 3.17.3 | Версия Helm для установки на целевые узлы. |
| coystack_helm_binary | /usr/local/bin/helm | Путь к бинарному файлу Helm на целевых узлах. |
| coystack_helm_diff_version | 3.12.5 | Версия plugin helm-diff. |
| coystack_operator_wait_timeout | 300 | Timeout в секундах для готовности operator. |

Переменные prepare playbook

Примерные prepare playbooks (скопированные из каталога `examples/`) поддерживают дополнительные переменные:

| Переменная | По умолчанию | Описание |
|-------------------------|--------------|--|
| coystack_flush_iptables | false | Очистить chain INPUT в iptables перед установкой. Полезно у облачных провайдеров со строгими правилами по умолчанию. |
| coystack_k3s_extra_args | " " | Дополнительные аргументы для k3s server (например, <code>- -tls-san=<PUBLIC_IP></code> для узлов за NAT). |

Проверка

После завершения playbook проверьте развертывание с первого server-узла:

```
# Проверить operator
kubectl get deployment coystack-operator --namespace cozy-system

# Проверить Platform Package
kubectl get packages.coystack.io coystack.coystack-platform

# Проверить все pods
kubectl get pods --all-namespaces
```

Идемпотентность

Playbook идемпотентен: повторный запуск не будет повторно применять ресурсы, которые не изменились. Platform Package применяется только при обнаружении diff через `kubectl diff`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Руководства для особых случаев развертывания Cozystack

<https://cozystack.ru/docs/v1.6/install/how-to/>

•

Как установить Talos на машину с одним диском

Как установить Talos на машину с одним диском, выделив место на системном диске под пользовательское хранилище

Развертывание Kubernetes в публичной сети

Как включить KubeSpan

Как включить KubeSpan.

Как настроить network bonding (LACP)

Как настроить network bonding LACP (802.3ad) для агрегации каналов и резервирования

Как включить Hugepages

Как включить Hugepages

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как установить Talos на машину с одним диском

<https://cozystack.ru/docs/v1.6/install/how-to/single-disk/>

Как установить Talos на машину с одним диском, выделив место на системном диске под пользовательское хранилище

Стандартная конфигурация Talos предполагает, что у каждого узла есть основной и дополнительный диски, используемые соответственно для системного и пользовательского хранилища. Однако можно использовать один диск, выделив на нем место для пользовательского хранилища.

Эту конфигурацию нужно применить при первом `[talosctl apply](https://cozystack.ru/docs/v1.6/install/kubernetes/talosctl/#3-apply-node-configuration)` или `[talm apply](https://cozystack.ru/docs/v1.6/install/kubernetes/talm/#3-apply-node-configuration)` — том, который выполняется с флагом `-i (--insecure)`. Применение изменений после инициализации не даст эффекта.

Для `talosctl` добавьте следующие строки в `patch.yaml`:

```
apiVersion: v1alpha1
kind: VolumeConfig
name: EPHEMERAL
provisioning:
minSize: 70GiB
```

```
apiVersion: v1alpha1
kind: UserVolumeConfig
name: data-storage
provisioning:
diskSelector:
match: disk.transport == 'nvme'
minSize: 400GiB
```

Для `talm` добавьте те же строки в конец конфигурационного файла первого узла, например `nodes/node1.yaml`.

Подробнее см. в документации Talos: <<https://www.talos.dev/v1.13/talos-guides/configuration/disk-management/>>.

После применения конфигурации очистите раздел `data-storage`:

```
kubectl -n kube-system debug -it --profile sysadmin --image=alpine node/node1
```

```
apk add util-linux
```

```
umount /dev/nvme0n1p6 ### Раздел, выделенный под пользовательское хранилище
```

```
rm -rf /host/var/mnt/data-storage
```

```
wipefs -a /dev/nvme0n1p6
```

```
exit
```

После настройки хранилища добавьте новый раздел в LINSTOR:

```
linstor ps cdp zfs node1 nvme0n1p6 --pool-name data --storage-pool data1
```

Проверьте результат:

```
linstor sp l
```

Вывод будет похож на этот пример:

```
+-----+
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | CanSnapshots | State |
| SharedName |
|=====|
|=====|
| DfltDisklessStorPool | node1 | DISKLESS | | | False | Ok | node1;DfltDisklessStorPool |
| data | node1 | ZFS | data | 351.46 GiB | 476 GiB | True | Ok | node1;data |
```

| | | | | | | | | | | |
|--|-------|-------|-----|------|------------|---------|------|----|--------------|--|
| | data1 | node1 | ZFS | data | 378.93 GiB | 412 GiB | True | Ok | node1; data1 | |
|--|-------|-------|-----|------|------------|---------|------|----|--------------|--|

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Развертывание Kubernetes в публичной сети

<https://cozystack.ru/docs/v1.6/install/how-to/public-ip/>

Кластер Kubernetes для Cozystack можно развернуть, используя только публичные сети:

- И управляющие, и worker-узлы имеют публичные IP-адреса.
- Worker-узлы подключаются к управляющим узлам через публичный Интернет, без частной внутренней сети или VPN.

Такая конфигурация не рекомендуется для production, но может использоваться для исследований и тестирования, когда ограничения хостинга не позволяют создать частную сеть.

Чтобы включить такую конфигурацию при развертывании с `talosctl`, добавьте следующие данные в конфигурационные файлы узлов:

cluster:

controlPlane:

endpoint: https://<MANAGEMENT_NODE_IP>:6443

Для `tal` добавьте те же строки в конец конфигурационного файла первого узла, например `nodes/node1.yaml`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как включить KubeSpan

<https://cozystack.ru/docs/v1.6/install/how-to/kubespans/>

Как включить KubeSpan.

Talos Linux предоставляет full-mesh сеть WireGuard для вашего кластера.

Чтобы включить эту функциональность, настройте [KubeSpan](#) и [Cluster Discovery](#) в конфигурации Talos Linux:

```
machine:
network:
kubespans:
  enabled: true
cluster:
discovery:
  enabled: true
```

Поскольку KubeSpan инкапсулирует трафик в туннель WireGuard, для Kube-OVN также нужно настроить более низкое значение MTU.

Для этого добавьте следующее в компонент `networking` вашего Platform Package:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
# ...
components:
networking:
values:
kube-ovn:
mtu: 1222
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как настроить network bonding (LACP)

<https://cozystack.ru/docs/v1.6/install/how-to/bonding/>

Как настроить network bonding LACP (802.3ad) для агрегации каналов и резервирования

Network bonding позволяет объединить несколько физических сетевых интерфейсов в один логический интерфейс. Это увеличивает пропускную способность и обеспечивает резервирование каналов.

LACP (Link Aggregation Control Protocol, IEEE 802.3ad) — самый распространенный режим bonding, который динамически согласует агрегацию каналов с сетевым коммутатором.

LACP требует настройки и на сервере, и на сетевом коммутаторе. Убедитесь, что на коммутаторе настроен соответствующий LACP port-channel для портов сервера.

Определение сетевых интерфейсов

После выполнения `talm template` сгенерированный конфигурационный файл узла будет содержать блок комментариев с обнаруженными сетевыми интерфейсами:

```
machine:
network:
# -- Discovered interfaces:
# eno1:
# hardwareAddr: aa:bb:cc:dd:ee:f0
# busPath: 0000:02:00.0
# driver: tg3
# vendor: Broadcom Inc. and subsidiaries
# product: NetXtreme BCM5719 Gigabit Ethernet PCIe
# eno2:
# hardwareAddr: aa:bb:cc:dd:ee:f1
# busPath: 0000:02:00.1
# driver: tg3
# vendor: Broadcom Inc. and subsidiaries
# product: NetXtreme BCM5719 Gigabit Ethernet PCIe
# eth0:
# hardwareAddr: aa:bb:cc:dd:ee:f2
# busPath: 0000:04:00.0
```

```
# driver: bnx2x
# vendor: Broadcom Inc. and subsidiaries
# product: NetXtreme II BCM57810 10 Gigabit Ethernet
# eth1:
# hardwareAddr: aa:bb:cc:dd:ee:f3
# busPath: 0000:04:00.1
# driver: bnx2x
# vendor: Broadcom Inc. and subsidiaries
# product: NetXtreme II BCM57810 10 Gigabit Ethernet
```

Выберите интерфейсы, которые хотите объединить в `bond`. Обычно это порты одной скорости, подключенные к одному коммутатору или стеку коммутаторов. Запишите значения `busPath` — они понадобятся далее.

Настройка `bonding`

Отредактируйте сгенерированный конфигурационный файл узла (например, `nodes/node1.yaml`) и замените стандартный раздел `machine.network.interfaces` на конфигурацию `bond`:

```
machine:
network:
interfaces:
  - interface: bond0
dhcp: false
bond:
mode: 802.3ad
adSelect: bandwidth
miimon: 100
updelay: 200
downdelay: 200
minLinks: 1
xmitHashPolicy: encap3+4
deviceSelectors:
  - busPath: "0000:04:00.0"
  - busPath: "0000:04:00.1"
addresses:
  - 192.168.100.11/24
routes:
```

- network: 0.0.0.0/0
gateway: 192.168.100.1

Описание параметров bond

| Параметр | Значение | Описание |
|----------------|-----------|---|
| mode | 802.3ad | LACP — динамическая агрегация каналов с согласованием на коммутаторе |
| adSelect | bandwidth | Выбирает активный агрегатор по наибольшей суммарной пропускной способности |
| miimon | 100 | Интервал мониторинга канала в миллисекундах |
| updelay | 200 | Задержка (мс) перед переводом восстановленного канала в активное состояние |
| downdelay | 200 | Задержка (мс) перед объявлением отказавшего канала отключенным |
| minLinks | 1 | Минимальное количество активных каналов, при котором bond остается поднятым |
| xmitHashPolicy | encap3+4 | Хеширование по IP и TCP/UDP-порту для распределения нагрузки между каналами |

Выбор интерфейсов

Рекомендуемый способ выбора участников bond — по пути PCI-шины с помощью `deviceSelectors`. Это надежнее, чем имена интерфейсов, которые могут меняться между перезагрузками:

bond:

deviceSelectors:

- busPath: "0000:04:00.0"
- busPath: "0000:04:00.1"

Также можно выбирать по имени интерфейса:

bond:

interfaces:

- eth0
- eth1

Или по аппаратному адресу:

bond:

deviceSelectors:

- hardwareAddr: "aa:bb:cc:dd:ee:f2"
- hardwareAddr: "aa:bb:cc:dd:ee:f3"

VLAN поверх bond

Поверх bond можно создавать VLAN-интерфейсы. Это удобно для разделения трафика (например, management, storage, tenant-сетей):

machine:

network:

interfaces:

- interface: bond0

dhcp: false

bond:

mode: 802.3ad

adSelect: bandwidth

miimon: 100

updelay: 200

downdelay: 200

minLinks: 1

xmitHashPolicy: encap3+4

deviceSelectors:

- busPath: "0000:04:00.0"
- busPath: "0000:04:00.1"

addresses:

- 192.168.100.11/24

routes:

- network: 0.0.0.0/0

gateway: 192.168.100.1

vlan:

- vlanId: 100

addresses:

- 10.0.0.11/24

Floating IP (VIP) с bonding

Для узлов control plane разместите раздел `vip` на интерфейсе (или VLAN), который используется для API endpoint кластера:

machine:

network:

interfaces:

- interface: bond0

dhcp: false

bond:

mode: 802.3ad

adSelect: bandwidth

miimon: 100

updelay: 200

downdelay: 200

minLinks: 1

xmitHashPolicy: encap3+4

deviceSelectors:

- busPath: "0000:04:00.0"

- busPath: "0000:04:00.1"

addresses:

- 192.168.100.11/24

routes:

- network: 0.0.0.0/0

gateway: 192.168.100.1

vip:

ip: 192.168.100.10

Убедитесь, что floating IP совпадает с адресом, настроенным в `values.yaml`.

Применение конфигурации

После редактирования всех файлов узлов примените конфигурацию обычным способом:

```
talm apply -f nodes/node1.yaml -i
```

```
talm apply -f nodes/node2.yaml -i
```

```
talm apply -f nodes/node3.yaml -i
```

Флаг `-i` (`--insecure`) нужен только при первом применении, когда узлы находятся в `maintenance mode`. Для уже инициализированных узлов не указывайте этот флаг: `talm apply -f nodes/node1.yaml`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как включить Hugepages

<https://cozystack.ru/docs/v1.6/install/how-to/hugepages/>

Как включить Hugepages

Hugepages для Cozystack можно включить как во время первоначальной установки, так и в любой момент после нее. Применение этой конфигурации после установки потребует полной перезагрузки узла.

Подробнее см. в документации ядра Linux: [HugeTLB Pages](#).

Использование Talm

Требуется Talm v0.16.0 или новее.

1. Добавьте следующие строки в `values.yaml`:

...

`certSANs: []`

`nr_hugepages: 3000`

`vm.nr_hugepages` — это количество страниц на 2Mi.

2. Примените конфигурацию:

```
talm apply -f nodes/node0.yaml
```

3. Затем перезагрузите узлы:

```
talm -f nodes/node0.yaml reboot
```

Использование talosctl

1. Добавьте следующие строки в шаблон узла:

`machine:`

`sysctls:`

`vm.nr_hugepages: "3000"`

`vm.nr_hugepages` — это количество страниц на 2Mi.

2. Примените конфигурацию:

```
talosctl apply -f nodetemplate.yaml -n 192.168.123.11 -e 192.168.123.11
```

3. Перезагрузите узлы:

```
talosctl reboot -n 192.168.123.11 -e 192.168.123.11
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Эксплуатация

Руководство по настройке и управлению кластером

<https://cozystack.ru/docs/v1.6/operations/>

Настройка, мониторинг, защита и обновление кластера Cozystack.

Настройка, мониторинг, защита и обновление кластера Cozystack.

-

[Обновление с v0.41 до v1.0](#)

Пошаговое руководство по обновлению Cozystack с v0.41.x до v1.0

[Конфигурация кластера Cozystack](#)

Как настраивать кластер Cozystack, включая варианты, компоненты и другие ключевые параметры

[VPC и подсети](#)

Как использовать VPC и подсети

[Руководства по обслуживанию кластера](#)

Руководства по регулярным операциям с кластером: добавлению и удалению узлов, обновлению Talos и другим задачам.

[Справочник cluster services](#)

Системные middleware packages, которые разворачиваются в tenants и предоставляют основную функциональность пользовательским приложениям.

[Использование OpenID Connect с Cozystack](#)

OIDC в Cozystack

[Руководства для нескольких дата-центров](#)

Как запускать Cozystack в растянутом Kubernetes-кластере

[Multi-Location кластеры](#)

Расширение Cozystack management clusters на несколько locations с помощью Kilo WireGuard mesh, cloud autoscaling и local cloud controller manager.

[Частые вопросы и практические руководства](#)

База знаний с частыми вопросами и расширенными настройками

[Руководство по устранению неполадок Cozystack](#)

Начальные шаги для проверки состояния кластера и поиска проблем.

[Scheduling Classes](#)

Ограничение tenant workload конкретными узлами или failure domain с помощью ресурсов SchedulingClass и планировщика Cozystack.

[Running Containerized GPU Workloads](#)

Run CUDA pods and other containerized GPU workloads on Cozystack management nodes that ship the NVIDIA driver and container toolkit via the distro package manager.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Обновление с v0.41 до v1.0

<https://cozystack.ru/docs/v1.6/operations/upgrades/>

Пошаговое руководство по обновлению Cozystack с v0.41.x до v1.0

Обзор

Версия 1.0 вносит крупное изменение в control plane Cozystack: теперь он полностью модульный и состоит из независимых пакетов, которыми управляет новый `cozystack-operator`.

Ключевые изменения:

- Старый `installer deployment` заменен на `cozystack-operator`.
- Конфигурация больше не хранится в `ConfigMap`: теперь она задается `custom resource Package`.
- `Assets server` заменен одним `OCI image`.
- Добавлены новые CRD: `Package` и `PackageSource`.

Нижележащими сущностями по-прежнему остаются `Helm releases`, поэтому во время обновления `workload` не пересоздаются и не затрагиваются.

Критические изменения совместимости

В этом разделе перечислены все пользовательские `breaking changes`, появившиеся в v1.0. Большинство изменений обрабатывается автоматически миграциями платформы, которые запускаются во время обновления. Перед обновлением просмотрите этот список, чтобы понять влияние на ваши рабочие нагрузки.

Пользователи `FerretDB` : приложение `FerretDB` удалено без автоматической миграции. Нужно сделать резервную копию данных до обновления. См. `FerretDB` удален ниже.

MySQL переименован в MariaDB

Приложение `mysql` переименовано в `mariadb`, чтобы точнее отражать используемый движок базы данных.

Все `Kubernetes resources` переименовываются автоматически во время обновления:

| Ресурс | До | После |
|--------------------|--------|----------|
| Application Kind | MySQL | MariaDB |
| HelmRelease prefix | mysql- | mariadb- |

| | | |
|---------------|--------------------------|----------------------------|
| Имена Service | mysql-<name>-primary | mariadb-<name>-primary |
| Имена Secret | mysql-<name>-credentials | mariadb-<name>-credentials |
| Имена PVC | storage-mysql-<name>-* | storage-mariadb-<name>-* |

Если ваши приложения подключаются к MySQL services по Kubernetes DNS name, например `mysql-mysdb-primary.<namespace>.svc`, после миграции нужно обновить строку подключения и использовать новый префикс `mariadb-`.

FerretDB удален

Приложение FerretDB полностью удалено из платформы. Автоматической миграции нет.

Если у вас есть запущенные инстансы FerretDB, необходимо сделать резервную копию всех данных до обновления. После обновления FerretDB больше не будет доступен как managed application.

Virtual Machine разделен на VM Disk и VM Instance

Монолитное приложение `virtual-machine` заменено двумя отдельными приложениями:

- `vm-disk` управляет образами дисков виртуальных машин.
- `vm-instance` управляет инстансами виртуальных машин и ссылается на диски, созданные `vm-disk`.

Миграция выполняется автоматически и сохраняет:

- Данные дисков: `PersistentVolumes` сохраняются и перепривязываются.
- IP- и MAC-адреса `Kube-OVN`.
- `LoadBalancer` IP для VM, опубликованных наружу.

Кроме того, `boolean`-поле `running` заменено на `runStrategy`:

| Старое значение | Новое значение |
|-----------------------------|----------------------------------|
| <code>running: true</code> | <code>runStrategy: Always</code> |
| <code>running: false</code> | <code>runStrategy: Halted</code> |

Поле `runStrategy` также принимает значения `Manual`, `RerunOnFailure` и `Once`.

Monitoring переведен на новую схему deployment

Стек мониторинга был реструктурирован. `HelmRelease` с именем `monitoring` в каждом `tenant namespace` мигрируется в новый `release` с именем `monitoring-system`.

Миграция выполняется автоматически: все компоненты мониторинга (VictoriaMetrics, Grafana, Alerta, VLogs) получают новые labels и принимаются новым HelmRelease.

Формат VPC subnets изменен с map на array

Поле `subnets` в конфигурации VPC (VirtualPrivateCloud) изменено с map на array.

До:

```
subnets:  
my-subnet:  
  cidr: 10.0.0.0/24
```

После:

```
subnets:  
  - name: my-subnet  
    cidr: 10.0.0.0/24
```

Для существующих VPC resources миграция выполняется автоматически.

Конфигурация MongoDB users и databases унифицирована

Формат конфигурации пользователей MongoDB был реструктурирован. Users и databases теперь задаются в отдельных секциях.

До:

```
users:  
myuser:  
  db: mydb  
  roles:  
    - name: readWrite  
  db: mydb
```

После:

```
users:  
myuser: {}  
databases:  
mydb:  
roles:  
admin:
```

- myuser

Для существующих инстансов MongoDB миграция выполняется автоматически.

Флаг `tenant isolated` удален

Поле `isolated` удалено из конфигурации Tenant. Сетевая изоляция теперь всегда применяется для каждого tenant через Cilium network policies: per-tenant отключения нет. Если раньше вы полагались на `isolated: false`, чтобы разрешить неограниченный трафик между tenant, теперь это больше невозможно.

Workload внутри tenant namespace по-прежнему должны обращаться к нескольким control-plane targets: внутрикластерному Kubernetes API server, собственному etcd tenant и т. д. Tenant chart предоставляет набор Cilium network policies, которые открывают эти пути по принципу opt-in через pod labels. Если pod внутри tenant namespace не может обратиться к одному из этих targets, добавьте соответствующую метку в его pod template:

| Target | Метка на pod |
|--|---|
| Внутрикластерный Kubernetes API server | <code>policy.cozystack.io/allow-to-apiserver: "true"</code> |
| Services собственного etcd cluster tenant (применимо только если tenant был создан с <code>etcd: true</code>) | <code>policy.cozystack.io/allow-to-etcd: "true"</code> |

Политика `allow-to-apiserver`, устанавливаемая tenant chart, сопоставляет трафик со встроенной сущностью Cilium `kube-apiserver`, которую Cilium разрешает в реальные endpoints API server. Вам не нужно знать Service CIDR или адрес Service `kubernetes`: метки на pod достаточно.

Пример: разрешить pod из Deployment обращаться к `kube-apiserver`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-operator
spec:
  selector:
    matchLabels:
      app: my-operator
  template:
    metadata:
      labels:
        app: my-operator
        policy.cozystack.io/allow-to-apiserver: "true"
```

spec:

containers:

- name: my-operator

image: example.com/my-operator:v1.0.0

Без метки трафик к `kube-apiserver` блокируется политикой `allow-to-apiserver CiliumNetworkPolicy`, которую `tenant chart` устанавливает в каждый `tenant namespace`. Тот же шаблон применяется к `allow-to-etcd`.

Внутренние изменения архитектуры

Следующие внутренние изменения не влияют напрямую на рабочие нагрузки приложений, но важны для скриптов автоматизации или пользовательских инструментов, которые взаимодействуют с внутренними компонентами Cozystack:

- Flux AIO теперь устанавливается и управляется `cozystack-operator`, а не отдельным компонентом.
- CRD `CozystackResourceDefinition` переименован в `ApplicationDefinition`.
- Компоненты `legacy installer` (`cozystack Deployment` и `cozystack-assets StatefulSet`) удалены.
- Namespace и `HelmRelease tenant-root` теперь управляются Helm через `release cozystack-basics`.

Предварительные требования

1\ Установите необходимые инструменты

Для миграции нужны следующие инструменты:

- `kubectl` и `jq` \- стандартные инструменты администрирования кластера.
- `helm` \- нужен для установки нового `operator`.
- `cozyrpg` \- новый CLI для управления ресурсами `Package` и `PackageSource`. Скачайте его со [страницы релизов Cozystack](#).
- `cozyhr` \- необязательный инструмент для управления значениями `HelmRelease`. Скачайте его из [репозитория cozyhr](#).

2\ Проверьте `kubectl context`

Убедитесь, что текущий `kubectl context` указывает на кластер, который вы обновляете:

```
kubectl config current-context
```

3\ Обновитесь до последней версии `v0.41.x`

Перед миграцией на v1.0 убедитесь, что используете самый свежий patch release v0.41.

Проверьте текущую версию:

```
kubectl get configmap -n cozy-system cozystack -o jsonpath='{.metadata.labels.cozystack\io/version}'
```

Если используется более старая версия, сначала обновитесь до последней v0.41.x по [стандартной процедуре обновления](#).

4\ Проверьте состояние кластера

Перед обновлением убедитесь, что все HelmRelease находятся в healthy-состоянии:

```
kubectl get hr -A
```

```
grep -v "True"
```

Если какие-либо releases не находятся в состоянии Ready, устраните эти проблемы до продолжения.

Шаги обновления

Шаг 1. Защитите критичные ресурсы

Добавьте аннотации к namespace `cozy-system` и ConfigMap `cozystack-version`, чтобы Helm не удалил их при обновлении installer release:

```
kubectl annotate namespace cozy-system helm.sh/resource-policy=keep --overwrite
```

```
kubectl annotate configmap -n cozy-system cozystack-version helm.sh/resource-policy=keep --overwrite
```

Этот шаг обязателен. Без этих аннотаций обновление Helm installer release может удалить namespace `cozy-system` и все ресурсы внутри него.

Шаг 2. Установите Cozystack Operator

Установите новый operator с помощью Helm из OCI registry. Команда развернет `cozystack-operator`, установит две новые CRD (`Package` и `PackageSource`) и создаст ресурс `PackageSource` для платформы.

```
helm upgrade --install cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version <TARGET_VERSION> \
--namespace cozy-system \
--create-namespace \
--take-ownership
```

Замените `<TARGET_VERSION>` на нужную версию release, например `1.0.0`.

Проверьте, что operator запущен:

```
kubectl get pods -n cozy-system -l app=cozystack-operator
```

Шаг 3. Сгенерируйте Platform Package

Скрипт миграции читает существующие ConfigMap (`cozystack`, `cozystack-branding`, `cozystack-scheduling`) из namespace `cozy-system` и преобразует их в ресурс `Package` с новой структурой `values`.

Скачайте и запустите скрипт миграции из репозитория Cozystack:

| | |
|---|-----------------|
| <pre>curl -fsSL https://raw.githubusercontent.com/cozystack/cozystack/main/hack/migrate-to-version-1.0.sh</pre> | <pre>bash</pre> |
|---|-----------------|

Скрипт:

1. Прочитает конфигурацию из существующих ConfigMap.
2. Преобразует старые имена bundle (`paas-*`) в новые имена variant (`isp-*`).
3. Сгенерирует ресурс `Package` и покажет его для проверки.
4. Запросит подтверждение перед применением.

Также можно скачать скрипт и запустить его локально, чтобы просмотреть перед выполнением:

```
curl -fsSL -o migrate-to-version-1.0.sh \
https://raw.githubusercontent.com/cozystack/cozystack/main/hack/migrate-to-version-1.0.sh
chmod +x migrate-to-version-1.0.sh
./migrate-to-version-1.0.sh
```

Шаг 4. Наблюдайте за миграцией

Как только Platform Package применен, operator запускает процесс миграции. Миграции удаляют старый `installer deployment` и `assets server`, преобразуют существующие manifests в новый формат и выполняют `reconcile` всех компонентов под новым управлением на основе `Package`.

Следите за статусами `HelmRelease`:

```
kubectl get hr -A
```

Дождитесь, пока все releases покажут `READY: True`.

Шаг 5. Удалите старые ConfigMap

После проверки, что все компоненты healthy, удалите старые ConfigMap, которые больше не используются:

```
kubect! delete configmap -n cozy-system cozystack cozystack-branding cozystack-scheduling
```

Шаг 6. Проверьте миграцию

Проверьте, что Platform Package прошел reconcile:

```
kubect! get packages.cozystack.io cozystack.cozystack-platform
```

Запустите полную проверку состояния кластера:

```
kubect! get hr -A | grep -v "True"
```

```
kubect! get pods -n cozy-system
```

Если какие-либо HelmRelease не находятся в состоянии Ready, проверьте логи operator для подробностей.

Устранение неполадок

Operator не запускается

Если pod operator находится в CrashLoopBackOff, проверьте логи:

```
kubect! logs -n cozy-system deploy/cozystack-operator --previous
```

HelmRelease зависли после миграции

Во время миграции некоторые HelmRelease могут временно показывать ошибки, пока operator выполняет reconcile. Подождите несколько минут и проверьте снова. Если проблемы сохраняются, обратитесь к [чеклисту диагностики](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Конфигурация кластера Cozystack

<https://cozystack.ru/docs/v1.6/operations/configuration/>

Как настраивать кластер Cozystack, включая варианты, компоненты и другие ключевые параметры

В этом разделе документации описана конфигурация кластера Cozystack.

-

[Справочник Platform Package](#)

Справочник по Cozystack Platform Package, который задает ключевые параметры конфигурации для установки и эксплуатации Cozystack.

[Варианты Cozystack: обзор и сравнение](#)

Справочник по вариантам Cozystack: состав, конфигурация и сравнение.

[Справочник компонентов Cozystack](#)

Полный справочник по компонентам Cozystack.

[Лицензии](#)

Лицензии open-source компонентов, поставляемых с Cozystack.

[Lineage Controller Webhook](#)

Что делает lineage-controller-webhook, как он разворачивается и какой параметр важно знать.

[White Labeling](#)

Настройка брендинга в Cozystack Dashboard и на страницах аутентификации Keycloak, включая custom Keycloak themes

[Telemetry](#)

Телеметрия Cozystack

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник Platform Package

<https://cozystack.ru/docs/v1.6/operations/configuration/platform-package/>

Справочник по Cozystack Platform Package, который задает ключевые параметры конфигурации для установки и эксплуатации Cozystack.

На этой странице описана роль Cozystack Platform Package и приведен полный справочник по его values.

Основная конфигурация Cozystack задается custom resource **Package**. Этот Package включает **вариант Cozystack, настройки компонентов**, ключевые сетевые параметры, опубликованные сервисы и другие опции.

Пример

Ниже пример конфигурации для установки Cozystack с вариантом `isp-full`, root host `example.org`, а также опубликованными и доступными пользователям Cozystack Dashboard и API:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full
components:
platform:
values:
publishing:
host: "example.org"
apiServerEndpoint: "https://api.example.org:443"
exposedServices:
  - dashboard
  - api
networking:
podCIDR: "10.244.0.0/16"
podGateway: "10.244.0.1"
serviceCIDR: "10.96.0.0/16"
joinCIDR: "100.64.0.0/16"
```

Справочник

Поля уровня Package

| Поле | Описание |
|---------------------------|--|
| <code>spec.variant</code> | Вариант, используемый для установки, например <code>isp-full</code> , <code>isp-full-generic</code> , <code>isp-hosted</code> , <code>distro-full</code> . |

Platform values (`spec.components.platform.values.*`)

Публикация

| Значение | По умолчанию | Описание | |
|---|--|--|--|
| <code>publishing.host</code> | <code>"example.org"</code> | The main domain for all services created under Cozystack, such as the dashboard, Grafana, Keycloak, etc. | |
| <code>publishing.apiServerEndpoint</code> | <code>" "</code> | Used for generating kubeconfig files for your users. It is recommended to use a routable FQDN or IP address instead of local-only addresses. Example: <code>"https://api.example.org"</code> . | |
| <code>publishing.exposedServices</code> | <code>[api, dashboard, vm-exportproxy, cdi-uploadproxy]</code> | List of services to expose. Possible values: <code>api</code> , <code>dashboard</code> , <code>cdi-uploadproxy</code> , <code>vm-exportproxy</code> . | |
| <code>publishing.ingressName</code> | <code>"tenant-root"</code> | Ingress controller to use for exposing services. | |

| | | | |
|--|------------------|--|--|
| <code>publishing.externalIPs</code> | <code>[]</code> | List of external IPs used for the specified ingress controller. If not specified, a LoadBalancer service is used by default. | |
| <code>publishing.ingressNameAdmin</code> | <code>" "</code> | Name of a separate admin ingress that system components attach their admin routes to (currently only Keycloak) — for example one bound to a private IP and unreachable from the internet. On the Gateway path the value is a namespace (of a Gateway named <code>cozystack</code>); on the ingress-nginx path it is an <code>ingressClassName</code> . The two coincide because Cozystack names each <code>ingressClasses</code> after its tenant namespace. Empty (the default) keeps admin endpoints on <code>publishing.ingressName</code> . Two caveats: Cozystack does not create the admin Gateway — provision it out of band first, or the admin routes attach to nothing; and the admin routes render only when Keycloak's <code>ingress.adminHost</code> is set, so setting this key alone is a no-op. | |

| | | | |
|-------------------------------|-----|---|--|
| publishing.exposureClass.name | " " | <p>Logical name of the cluster-scoped <code>ExposureClass</code> to render. Empty (the default) leaves the <code>publishing.externalIPs</code> path untouched. When set, the host ingress — with <code>publishing.externalIPs</code> empty — publishes its Service as type: <code>LoadBalancer</code>, and <code>cozystack-controller</code> translates the class into the chosen backend's VIP pool and announcer.</p> <p>This is the migration path off <code>Service.spec.externalIPs</code>, deprecated in Kubernetes v1.36 (KEP-5707).</p> | |
|-------------------------------|-----|---|--|

| | | | |
|--|---|---|--|
| | * | * | <p>The removal is phased: the <code>AllowServiceExternalIPs</code> feature gate ships enabled in v1.36, then defaults to false (KEP target: v1.40), at which point kube-proxy stops programming rules for the field — the API still accepts it, so the Service looks healthy while no traffic arrives. Later phases lock the gate off and strip kube-proxy's support outright, and finally drop the gate and the <code>DenyServiceExternalIPs</code> admission controller. The KEP gives those later releases as approximate and the v1.36 announcement commits only to “a future minor release”, so treat the default flipping to false as the deadline and migrate onto a class before that upgrade rather than planning against a specific version.</p> |
|--|---|---|--|

| | | | |
|---|----------------------------|--|--|
| <code>publishing.exposureClass.backend</code> | <code>"externalIPs"</code> | <p>LoadBalancer mechanism behind the class. <code>externalIPs</code> pins node IPs with no pool or announcer (the historical behaviour); <code>metallb</code> renders an <code>IPAddressPool</code> \+ <code>L2Advertisement</code>; <code>cilium</code> renders a <code>CiliumLoadBalancerIPPool</code> \+ <code>CiliumL2AnnouncementPolicy</code>; <code>robotlb</code> uses a cloud (Hetzner) load balancer, where allocation and announcement happen off-cluster. Once <code>publishing.exposureClass.name</code> is set, a pool or cloud backend (<code>metallb</code>, <code>cilium</code>, <code>robotlb</code>) is mutually exclusive with a non-empty <code>publishing.externalIPs</code> — the platform render fails with an explicit error rather than shipping the dead end, because the host ingress Service would stay <code>ClusterIP</code> \+ <code>externalIPs</code>, never become type: <code>LoadBalancer</code>, and the allocated pool would go unused.</p> | |
| <code>publishing.exposureClass.addresses</code> | <code>[]</code> | VIP CIDRs or start-end ranges for the pool-backed backends (<code>metallb</code> , <code>cilium</code>). | |
| <code>publishing.exposureClass.l2</code> | <code>true</code> | L2 (ARP / NDP) announcement for the bare-metal backends. | |

| | | | |
|---|---------------------------------|--|--|
| <code>publishing.exposureClasses.isDefault</code> | <code>true</code> | Make this the class used by any <code>ServiceExposure</code> that names none. A <code>ServiceExposure</code> is the cluster-internal resource <code>cozystack-controller</code> reconciles to publish a Service through the class's backend; workloads do not create one directly. Because this defaults to <code>true</code> , hand-creating a second <code>ExposureClass</code> without clearing its default flag leaves two default classes, and the controller then fails every unnamed <code>ServiceExposure</code> with <code>AmbiguousDefaultClass</code> — keep exactly one default. | |
| <code>publishing.certificate.s.solver</code> | <code>"http01"</code> | ACME challenge solver type for default letsencrypt issuer. Possible values: <code>http01</code> , <code>dns01</code> . | |
| <code>publishing.certificate.s.issuerName</code> | <code>"letsencrypt-prod"</code> | <code>ClusterIssuer</code> name for TLS certificates used in system Helm releases. | |

| | | | |
|--|--------------------|--|--|
| <code>publishing.certificate
s.wildcard</code> | <code>false</code> | <p>Opt-in shared wildcard certificate on the default ingress-nginx path (<code>gateway.enabled=false</code>). When <code>true</code> with <code>solver=dns01</code> and no <code>publishing.certificate.s.wildcardSecretName</code>, the platform issues one <code>*.<root-host> \+ <root-host> Certificate</code> via the <code>DNS-01 ClusterIssuer</code> and serves it as the ingress controller's default SSL certificate, so system services stop minting a per-host ACME certificate each — avoiding Let's Encrypt rate limits at scale, at parity with the Gateway API path. Ignored on <code>http01</code> (cannot issue wildcards) and when <code>gateway.enabled=true</code> (the <code>TenantGateway</code> controller issues the wildcard there).</p> | |
|--|--------------------|--|--|

| | | | |
|--|---|---|--|
| | * | * | <p>Enabling it feeds the issued Secret name into the same cluster values channel as <code>publishing.certificates.wildcardSecretName</code>, so it carries the same child-tenant hazard and the same open bug (cozystack/cozystack#3296): every tenant's system ingresses drop their per-host ACME certificate cluster-wide, but the wildcard is served only by the publishing controller. A child tenant running its own ingress controller (<code>ingress: true</code>) is then left with no certificate and serves ingress-nginx's built-in self-signed one; an inheriting child on the shared controller instead gets a hostname mismatch, because a single-label <code>*.<root-host></code> wildcard does not cover a nested <code><service>.<tenant>.<root-host></code> host or a custom <code>ingress.host</code> on another domain. Only enable it when every exposed host is covered and no child tenant runs its own ingress controller. Off by default so a <code>dns01</code> cluster is never switched silently on upgrade.</p> |
|--|---|---|--|

| | | | |
|--|-----|--|--|
| publishing.certificate
s.wildcardSecretName | " " | Operator-provided wildcard TLS Secret. When set, platform services and the root tenant's ingress/Gateway serve this pre-existing Secret instead of minting per-host ACME certificates (only the NAME travels the values channel — never the key material), and takes precedence over publishing.certificate s.wildcard. The Secret must exist in the publishing namespace (tenant-root by default), hold valid PEM under tls.crt / tls.key (created as kubernetes.io/tls, though only the material is validated), and cover the served hosts. Scoped to the root tenant but not enforced — the name reaches every tenant, so a child running its own ingress controller can be left serving a self-signed certificate (cozystack/cozystack#3296). See Gateway API -> Certificates for the full behaviour and the three child-tenant cases. Leave empty to keep ACME issuance. | |
|--|-----|--|--|

| | | | |
|---|--|--|--|
| <code>publishing.certificate
s.dns01.provider</code> | <code>"cloudflare"</code> | DNS-01 provider when <code>solver=dns01</code> . Possible values: <code>cloudflare</code> , <code>route53</code> , <code>digitalocean</code> , <code>rfc2136</code> . Both the per-tenant Issuer (rendered by <code>cozystack-controller</code> from the <code>TenantGateway</code> CR) and the cluster-wide <code>letsencrypt-prod</code> / <code>letsencrypt-stage</code> <code>ClusterIssuers</code> used by the legacy ingress flow read this. | |
| <code>publishing.certificate
s.dns01.cloudflare.secretName</code> | <code>"cloudflare-api-token-secret"</code> | Secret name holding a Cloudflare API token with <code>Zone:Read</code> \+ <code>Zone:DNS>Edit</code> on the apex zone. | |
| <code>publishing.certificate
s.dns01.cloudflare.secretKey</code> | <code>"api-token"</code> | Key inside the Secret holding the API token. | |
| <code>publishing.certificate
s.dns01.route53.region</code> | <code>" "</code> | AWS region of the Route53 hosted zone. Required when <code>provider=route53</code> . | |
| <code>publishing.certificate
s.dns01.route53.accessKeyID</code> | <code>" "</code> | IAM access key ID. Optional when running with IRSA / instance profile. | |
| <code>publishing.certificate
s.dns01.route53.secretName</code> | <code>" "</code> | Secret name holding the IAM secret access key. Optional when running with IRSA / instance profile. | |
| <code>publishing.certificate
s.dns01.route53.secretKey</code> | <code>"secret-access-key"</code> | Key inside the Route53 Secret holding the secret access key. | |

| | | | |
|---|--|---|--|
| <code>publishing.certificate.s.dns01.digitalocean.secretName</code> | <code>"digitalocean-api-token-secret"</code> | Secret name holding a DigitalOcean API token with write access to the apex domain. | |
| <code>publishing.certificate.s.dns01.digitalocean.secretKey</code> | <code>"access-token"</code> | Key inside the Secret holding the DigitalOcean token. | |
| <code>publishing.certificate.s.dns01.rfc2136.nameserver</code> | <code>" "</code> | <code>host:port</code> of the authoritative nameserver accepting RFC 2136 dynamic updates. Required when <code>provider=rfc2136</code> . | |
| <code>publishing.certificate.s.dns01.rfc2136.tsigKeyName</code> | <code>" "</code> | TSIG key name authorising the dynamic updates. Required when <code>provider=rfc2136</code> . | |
| <code>publishing.certificate.s.dns01.rfc2136.tsigAlgorithm</code> | <code>"HMACSHA256"</code> | TSIG HMAC algorithm. | |
| <code>publishing.certificate.s.dns01.rfc2136.secretName</code> | <code>" "</code> | Secret name holding the TSIG key material. Required when <code>provider=rfc2136</code> . | |
| <code>publishing.certificate.s.dns01.rfc2136.secretKey</code> | <code>"tsig-secret-key"</code> | Key inside the Secret holding the TSIG key. | |
| <code>publishing.proxyProtocol</code> | <code>false</code> | Enables PROXY-protocol on the host ingress-nginx and auto-deploys ouroboros to fix the resulting hairpin-NAT problem. The upstream L4 LB in front of ingress-nginx must already be injecting PROXY-v1 headers before this flag flips on; see the linked page for verification recipes and the disable path. | |

| | | | |
|---|--------------------|---|--|
| <code>publishing.proxyProtocolAcknowledgeUnclean</code> | <code>false</code> | Acknowledgement gate for the <code>helm.sh/resource-policy: keep</code> asymmetry on the host disable path. Flipping <code>publishing.proxyProtocol</code> from <code>true</code> back to <code>false</code> stops emitting the <code>cozystack.ouroboros</code> Package CR but does not uninstall the existing one — the platform render fails until either the Package CR is deleted (which triggers the chart's pre-delete cleanup hook) or this flag is set to <code>true</code> to confirm the operator has handled the asymmetry. See hairpin-proxy-protocol-> Disable path for the full sequence. | |
|---|--------------------|---|--|

`publishing.exposure` is not listed above on purpose: the key existed in v1.4 only and was removed before v1.5 shipped. It still works on a v1.4 cluster, but has no effect from v1.5 onward —

`publishing.exposureClass` is its successor. Drop the key when upgrading from v1.4 — and check what the host ingress Service becomes. A v1.4 cluster running `publishing.exposure: loadBalancer` had a `LoadBalancer` Service with `externalTrafficPolicy: Local`; from v1.5 the key is ignored, so a cluster with `publishing.externalIPs` set silently falls back to a `ClusterIP` Service with `externalTrafficPolicy: Cluster` and stops preserving client source IPs.

Сеть

| Значение | По умолчанию | Описание |
|---------------------------------------|------------------------------|--|
| <code>networking.clusterDomain</code> | <code>"cozy.local"</code> | Внутреннее доменное имя кластера. |
| <code>networking.podCIDR</code> | <code>"10.244.0.0/16"</code> | Pod-подсеть, из которой Pods получают IP-адреса. |
| <code>networking.podGateway</code> | <code>"10.244.0.1"</code> | Адрес gateway для pod-подсети. |
| <code>networking.serviceCIDR</code> | <code>"10.96.0.0/16"</code> | Service-подсеть, из которой Services получают IP-адреса. |

| | | |
|--|------------------------------|--|
| <code>networking.joinCIDR</code> | <code>"100.64.0.0/16"</code> | Подсеть <code>join</code> для сетевого взаимодействия между Node и Pod. Подробнее см. в документации kube-ovn . |
| <code>networking.kubeovn.MASTER_NODES</code> | <code>" "</code> | Разделённый запятыми список IP-адресов master-узлов KubeOVN. По умолчанию KubeOVN использует <code>lookup</code> , чтобы найти control-plane-узлы по label <code>node-role.kubernetes.io/control-plane</code> . На новых кластерах <code>lookup</code> может вернуть пустой результат. Задайте это значение, чтобы переопределить поведение. |

Bundles

| Значение | По умолчанию | Описание |
|--------------------------------------|-------------------------|---|
| <code>bundles.system.enabled</code> | <code>false</code> | Включить system bundle. Управляется оператором на основе <code>spec.variant</code> . |
| <code>bundles.system.variant</code> | <code>"isp-full"</code> | Вариант system bundle. Варианты: <code>isp-full</code> , <code>isp-full-generic</code> , <code>isp-hosted</code> . Управляется оператором на основе <code>spec.variant</code> . |
| <code>bundles.iaas.enabled</code> | <code>false</code> | Включить IaaS bundle. Управляется оператором на основе <code>spec.variant</code> . |
| <code>bundles.paas.enabled</code> | <code>false</code> | Включить PaaS bundle. Управляется оператором на основе <code>spec.variant</code> . |
| <code>bundles.naas.enabled</code> | <code>false</code> | Включить NaaS bundle. Управляется оператором на основе <code>spec.variant</code> . |
| <code>bundles.enabledPackages</code> | <code>[]</code> | Список опциональных компонентов bundle, которые нужно включить в установку. Подробнее см. "Как включать и отключать компоненты bundle" . |

| | | |
|---------------------------------------|-----------------|---|
| <code>bundles.disabledPackages</code> | <code>[]</code> | Список компонентов bundle, которые нужно исключить из установки. Подробнее см. "Как включать и отключать компоненты bundle" . |
|---------------------------------------|-----------------|---|

Аутентификация

| Значение | По умолчанию | Описание |
|---|--------------------|--|
| <code>authentication.oidc.enabled</code> | <code>false</code> | Включить функцию OIDC в Cozystack. |
| <code>authentication.oidc.insecureSkipVerify</code> | <code>false</code> | Пропускать проверку TLS-сертификата OIDC provider. |
| <code>authentication.oidc.keycloakExtraRedirectUri</code> | <code>""</code> | Дополнительный redirect URI для Keycloak OIDC client. |
| <code>authentication.oidc.keycloakInternalUrl</code> | <code>""</code> | Внутренний URL для backend-to-backend-запросов к Keycloak. Когда он задан, oauth2-proxy dashboard пропускает OIDC discovery и направляет запросы token, JWKS, userinfo и logout через этот URL, сохраняя browser redirects на внешний URL.
Пример: <code>http://keycloak-http.cozy-keycloak.svc:8080/realms/cozy</code> . |

Gateway

Общеплатформенная интеграция Gateway API. Фактический per-tenant Gateway создаётся только для tenants, которые явно включают его через `tenant.spec.gateway: true` (обычно `tenant-root` плюс любой tenant, которому нужен собственный LB IP, кастомный арех или отдельный ACME-аккаунт). Все остальные tenants в дереве публикуются через Gateway ближайшего предка, который им владеет — так же, как наследование `_namespace.ingress`. Полную архитектуру и путь миграции см. в [руководстве по Gateway API](#).

| Значение | По умолчанию | Описание |
|----------|--------------|----------|
|----------|--------------|----------|

| | | |
|------------------------------|--------------------|---|
| <code>gateway.enabled</code> | <code>false</code> | Enable Gateway API support across the platform. When <code>true</code> , cert-manager <code>ClusterIssuers</code> use an <code>http01.gatewayHTTPRoute</code> solver attached to the publishing tenant's Gateway, and exposed services (dashboard, keycloak, grafana, alerta, harbor, bucket, cozystack-api, vm-exportproxy, cdi-uploadproxy) render <code>HTTPRoute</code> / <code>TLSRoute</code> instead of <code>Ingress</code> . Materialising the actual per-tenant Gateway still requires an owning tenant to set <code>tenant.spec.gateway: true</code> . |
|------------------------------|--------------------|---|

| | | |
|---------------|------|--|
| gateway.http2 | true | <p>Advertise HTTP/2 via TLS ALPN (h2, then <code>http/1.1</code>) on every Gateway API listener served by the bundled Cilium dataplane. Browsers negotiate HTTP/2 exclusively through ALPN, so with this off every client silently falls back to HTTP/1.1 — the pre-Gateway ingress-nginx path advertised h2 out of the box, hence on by default. Affects only the client<->gateway hop: gateway<->backend connections stay HTTP/1.1 unless a <code>Service</code> opts in per GEP-1911 by declaring <code>appProtocol: kubernetes.io/h2c</code> on its port (that backend-protocol support is switched on together with ALPN). Maps to Cilium's cluster-wide <code>enable-gateway-api-alpn</code> agent setting, so it covers the root and all tenant Gateways at once, with no per-Gateway granularity; only effective on bundles where Cozystack manages Cilium (<code>isp-full</code>, <code>isp-full-generic</code>). Flipping it re-rolls the <code>cilium</code> DaemonSet on the next platform upgrade, the same disruption profile as any other Cilium config change.</p> |
|---------------|------|--|

| | | |
|----------------------------|-------------|---|
| gateway.attachedNamespaces | (see below) | Namespaces whose HTTPRoute / TLSRoute resources should publish through the owning tenant Gateway. The controller patches namespace.cozystack.io/gateway = <owner> onto each listed namespace so its routes pass the HTTPS and TLS-passthrough listeners' allowedRoutes label selector. The port-80 HTTP listener uses a separate, narrower whitelist (<owner-tenant-ns> and cozy-cert-manager only) and does NOT admit routes from attachedNamespaces. The publishing tenant's own namespace and descendants are admitted via the same label written by the tenant chart. Tenant namespaces (tenant-*) may be listed too — they simply pick up the gateway-attach label alongside the cozy-* system namespaces. The default namespace is included by default because the Kubernetes API TLSRoute lives next to the kubernetes Service in default. |
|----------------------------|-------------|---|

gateway.attachedNamespaces по умолчанию:

gateway:

attachedNamespaces:

- cozy-cert-manager
- cozy-dashboard
- cozy-keycloak
- cozy-system
- cozy-harbor
- cozy-bucket
- cozy-kubevirt
- cozy-kubevirt-cdi
- cozy-monitoring
- cozy-linstor-gui

- default

Планирование

| Значение | По умолчанию | Описание |
|--|------------------|---|
| <code>scheduling.globalAppTopologySpreadConstraints</code> | <code>" "</code> | Глобальные pod topology spread constraints, применяемые ко всем managed applications. |

Backup storage

| Value | Default | Description |
|----------------------------|-----------------|---|
| <code>backupStorage</code> | <code>{}</code> | S3 coordinates for the platform-managed <code>cozy-default</code> BackupClass. The whole block is forwarded into the <code>backupstrategy-controller</code> component and deep-merged over its chart defaults; keys include <code>provisionBucket</code> , <code>bucketName</code> , <code>endpoint</code> , <code>region</code> , <code>forcePathStyle</code> , <code>systemSecretName</code> , and <code>systemNamespaces</code> . See Backup Classes for the knob-by-knob reference. |

Брендинг

| Значение | По умолчанию | Описание |
|-----------------------|-----------------|--|
| <code>branding</code> | <code>{}</code> | Объект конфигурации UI branding. Доступные поля и использование описаны в руководстве White Labeling . У отдельных полей, например <code>titleText</code> , <code>logoSvg</code> , есть собственные значения по умолчанию, если они не заданы. |

Registries

Конфигурация mirrors для container registry. Позволяет направлять image pulls через локальные mirrors.

| Значение | По умолчанию | Описание |
|---------------------------------|-----------------|---|
| <code>registries.mirrors</code> | <code>{}</code> | Карта имён registry hostnames к mirror endpoints. Каждая запись сопоставляет registry, например <code>docker.io</code> , со списком mirror endpoints. |
| <code>registries.config</code> | <code>{}</code> | Конфигурация для отдельных endpoints, например настройки TLS. |

Пример:

`registries:`

`mirrors:`

`docker.io:`

`endpoints:`

- `http://10.0.0.1:8082`

`ghcr.io:`

`endpoints:`

- `http://10.0.0.1:8083`

`config:`

`"10.0.0.1:8082":`

`tls:`

`insecureSkipVerify: true`

Ресурсы

| Значение | По умолчанию | Описание |
|--|--------------|---|
| <code>resources.cpuAllocationRatio</code> | 10 | CPU allocation ratio: на 1 vCPU запрашивается <code>1/cpuAllocationRatio</code> CPU. Подробное объяснение и примеры см. в Resource Management . |
| <code>resources.memoryAllocationRatio</code> | 1 | Memory allocation ratio: на единицу настроенной памяти запрашивается <code>1/memoryAllocationRatio</code> памяти. |

| | | |
|--|----|---|
| <code>resources.ephemeralStorageAllocationRatio</code> | 40 | Ephemeral storage allocation ratio: на единицу настроенного хранилища запрашивается <code>1/ephemeralStorageAllocationRatio</code> ephemeral storage. |
|--|----|---|

Внутренние поля

Этими полями автоматически управляет оператор Cozystack, их не следует изменять вручную.

| Значение | По умолчанию | Описание |
|---------------------------------------|----------------------|--|
| <code>sourceRef.kind</code> | "OCIRepository" | Kind source reference для platform package. |
| <code>sourceRef.name</code> | "cozystack-platform" | Имя source reference. |
| <code>sourceRef.namespace</code> | "cozy-system" | Namespace source reference. |
| <code>sourceRef.path</code> | "/" | Path source reference. |
| <code>migrations.enabled</code> | false | Включены ли platform migrations. |
| <code>migrations.image</code> | — | Container image, используемый для запуска platform migrations. |
| <code>migrations.targetVersion</code> | — | Номер целевой версии миграции. |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Варианты Cozystack: обзор и сравнение

<https://cozystack.ru/docs/v1.6/operations/configuration/variants/>

Справочник по вариантам Cozystack: состав, конфигурация и сравнение.

Введение

Variants \- это предопределенные конфигурации Cozystack, определяющие, какие bundles и компоненты включены. Каждый вариант тестируется, версионизируется и гарантированно работает как единое целое. Они упрощают установку, снижают риск неправильной конфигурации и помогают выбрать подходящий набор возможностей для вашего развертывания.

Это руководство предназначено для infrastructure engineers, DevOps-команд и platform architects, которые планируют разворачивать Cozystack в разных окружениях. В нем объясняется, как варианты Cozystack помогают адаптировать установку под конкретные потребности: от полнофункциональной platform-as-a-service до полного ручного контроля над установленными packages.

Обзор вариантов

| Компонент | default | isp-full | isp-full-generic | isp-hosted |
|--------------------------|---------|--------------------|--------------------|------------|
| Managed Kubernetes | | + | + | |
| Managed Applications | | + | + | + |
| Virtual Machines | | + | + | |
| Cozystack Dashboard (UI) | | + | + | + |
| Cozystack API | | + | + | + |
| Kubernetes Operators | | + | + | + |
| Monitoring subsystem | | + | + | + |
| Подсистема хранения | | LINSTOR | LINSTOR | |
| Сетевая подсистема | | Kube-OVN \+ Cilium | Kube-OVN \+ Cilium | |

| | | | | |
|----------------------------|--|-------------|----------|--|
| Подсистема виртуализации | | KubeVirt | KubeVirt | |
| Подсистема OS и Kubernetes | | Talos Linux | | |

Выбор подходящего варианта

Variants объединяют bundles из разных слоев под конкретные задачи. Одни рассчитаны на полноценные platform-сценарии, другие - на workloads в cloud-hosted окружениях или полностью ручное управление packages.

default

`default` \- минимальный вариант, который предоставляет только набор PackageSources (ссылки на package registry). Bundles и компоненты заранее не настроены: все packages управляются вручную через [cozypkg](#). Используйте этот вариант, когда нужен полный контроль над тем, какие packages установлены и настроены. Именно этот вариант используется в workflow [Build Your Own Platform \(BYOP\)](#).

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: default
```

isp-full

`isp-full` \- полнофункциональный вариант PaaS и IaaS, предназначенный для установки на Talos Linux. Он включает все bundles и предоставляет полный набор компонентов Cozystack, создавая полноценный PaaS-опыт. Некоторые компоненты верхних слоев опциональны и могут быть исключены при установке.

`isp-full` предназначен для установки на bare-metal servers или VM.

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
```

```
name: cozystack.cozystack-platform
spec:
  variant: isp-full
  components:
  platform:
  values:
  networking:
  podCIDR: "10.244.0.0/16"
  podGateway: "10.244.0.1"
  serviceCIDR: "10.96.0.0/16"
  joinCIDR: "100.64.0.0/16"
  publishing:
  host: "example.org"
  apiServerEndpoint: "https://192.168.100.10:6443"
  exposedServices:
    - api
    - dashboard
    - cdi-uploadproxy
    - vm-exportproxy
```

`isp-full-generic`

`isp-full-generic` предоставляет такой же полнофункциональный PaaS и IaaS, как `isp-full`, но предназначен для обычных дистрибутивов Kubernetes, таких как k3s, kubeadm или RKE2. Используйте этот вариант, если нужен полный набор возможностей Cozystack без обязательного Talos Linux.

Подробные инструкции по установке см. в [руководстве Generic Kubernetes](#).

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full-generic
  components:
  platform:
```

```
values:
networking:
podCIDR: "10.244.0.0/16"
podGateway: "10.244.0.1"
serviceCIDR: "10.96.0.0/16"
joinCIDR: "100.64.0.0/16"
publishing:
host: "example.org"
apiServerEndpoint: "https://192.168.100.10:6443"
exposedServices:
  - api
  - dashboard
  - cdi-uploadproxy
  - vm-exportproxy
```

`isp-hosted`

Cozystack можно установить как platform-as-a-service (PaaS) поверх существующего managed Kubernetes-кластера, обычно созданного cloud provider. Для этого сценария предназначен вариант `isp-hosted`. Его можно использовать с `kind` и любыми cloud-based Kubernetes-кластерами.

`isp-hosted` включает PaaS и NaaS bundles, предоставляя Cozystack API и UI, managed applications и tenant Kubernetes clusters. Он не включает CNI plugins, виртуализацию или хранилище.

Kubernetes-кластер, используемый для развертывания Cozystack, должен соответствовать следующим требованиям:

- Listening address некоторых компонентов Kubernetes должен быть изменен с `localhost` на маршрутизируемый адрес.
- Kubernetes API server должен быть доступен на `localhost`.

Пример конфигурации:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-hosted
components:
platform:
```

values:

publishing:

host: "example.org"

apiServerEndpoint: "https://192.168.100.10:6443"

exposedServices:

- api
- dashboard

Подробнее

Полный список параметров конфигурации для каждого варианта см. в [справочнике по конфигурации](#).

Полный список компонентов и инструкции по их включению и отключению см. в [справочнике компонентов](#).

Чтобы развернуть выбранный вариант, следуйте [руководству по установке Cozystack](#) или [руководствам для конкретных провайдеров](#). Если вы устанавливаете Cozystack впервые, лучше использовать вариант `isp-full` и пройти [tutorial Cozystack](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник компонентов Cozystack

<https://cozystack.ru/docs/v1.6/operations/configuration/components/>

Полный справочник по компонентам Cozystack.

Переопределение параметров компонентов

Иногда нужно переопределить отдельные параметры компонентов. Для этого измените соответствующий ресурс Package и укажите значения в секции `spec.components`. Структура values соответствует файлу [values.yaml](#) соответствующего системного chart в репозитории Cozystack.

Например, если нужно включить режим FRR-K8s для MetalLB, посмотрите его [values.yaml](#), чтобы понять доступные параметры, а затем измените Package `cozystack.metallb`:

```
apiVersion: cozystack.io/v1alpha1
```

```
kind: Package
```

```
metadata:
```

```
name: cozystack.metallb
```

```
namespace: cozy-system
```

```
spec:
```

```
variant: default
```

```
components:
```

```
metallb:
```

```
values:
```

```
metallb:
```

```
frrk8s:
```

```
enabled: true
```

Включение и отключение компонентов

В bundles есть опциональные компоненты, которые нужно явно включать в установку. Обычные компоненты bundle, наоборот, можно отключить и исключить из установки, если они не нужны.

Используйте `bundles.enabledPackages` и `bundles.disabledPackages` в values Platform Package. Каждая запись в этих списках - это полное имя Package, такое же, как в выводе `kubectl get package`. Все platform packages находятся под префиксом `cozystack.`, например `cozystack.metallb`, `cozystack.hetzner-robotlb`, `cozystack.nfs-driver`. Перед редактированием Platform Package выполните `kubectl get package`, чтобы увидеть точные имена, доступные в вашем кластере.

Например, при [установке Cozystack в Hetzner](#) нужно заменить load balancer по умолчанию, MetalLB, на RobotLB, специально сделанный для Hetzner:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
variant: isp-full
components:
platform:
values:
bundles:
disabledPackages:
  - cozystack.metallb
enabledPackages:
  - cozystack.hetzner-robotlb
# остальная конфигурация
```

Компоненты нужно отключать до установки Cozystack. Применение обновленной конфигурации с `disabledPackages` не удалит компоненты, которые уже установлены. Чтобы удалить уже установленные компоненты, вручную удалите Helm release следующей командой:

```
kubectl delete hr -n <namespace> <component>
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Лицензии

<https://cozystack.ru/docs/v1.6/operations/configuration/licenses/>

Лицензии open-source компонентов, поставляемых с Cozystack.

На этой странице перечислены open-source компоненты, поставляемые с Cozystack, сгруппированные по роли в платформе. Charts, CRDs, controllers и application APIs, поддерживаемые Cozystack, лицензированы под Apache-2.0 и не перечисляются ниже по отдельности. Для каждого upstream-компонента карточка содержит ссылку на upstream-файл лицензии.

Этот справочник вручную сверяется с текущим набором компонентов [next](#). Container images могут включать дополнительные пакеты операционной системы и библиотечные зависимости со своими лицензиями. Зафиксированные upstream-версии managed runtimes (PostgreSQL, MariaDB, Kafka и другие) могут меняться между minor-релизами Cozystack: проверяйте версию Cozystack, которую вы используете.

Операционная система и Kubernetes runtime

Talos Linux

Immutable Linux-дистрибутив, созданный для узлов Kubernetes.

MPL-2.0

Kubernetes

Ядро оркестрации контейнеров, используемое и для management-кластера, и для tenant-кластеров.

Apache-2.0

Kamaji

Hosted control planes для tenant Kubernetes-кластеров.

Apache-2.0

Cluster API

Декларативное создание tenant Kubernetes-кластеров: core, operator и Kamaji/KubeVirt providers.

Apache-2.0

KubeVirt

Виртуальные машины как Kubernetes-native workloads: core, CDI, CSI иinstancetypeypes.

Apache-2.0

Networking

Cilium

CNI на базе eBPF для pod networking и NetworkPolicy.

Apache-2.0

Kube-OVN

Виртуальная сеть на базе OVN, используемая для VPC и floating IP.

Apache-2.0

Multus CNI

Несколько сетевых интерфейсов на pod.

Apache-2.0

MetallB

Bare-metal load balancer для Kubernetes Services.

Apache-2.0

ingress-nginx

HTTP ingress controller.

Apache-2.0

Gateway API CRDs

Стандартные определения Kubernetes Gateway API.

Apache-2.0

CoreDNS

Cluster DNS server.

Apache-2.0

ExternalDNS

Синхронизация Kubernetes-ресурсов с внешними DNS providers.

Apache-2.0

Kilo

Mesh-сеть между географически распределенными узлами.

Apache-2.0

Hetzner RobotLB

Интеграция load balancer для dedicated hardware Hetzner.

MIT

Хранилище

LINSTOR / Piraeus

Реплицированное блочное хранилище на базе DRBD: LINSTOR server, CSI, scheduler extender, GUI, Piraeus operator.

GPL-3.0; Apache-2.0

SeaweedFS

Распределенное object storage, лежащее в основе managed Bucket service.

Apache-2.0

CSI Driver NFS

NFS CSI driver.

Apache-2.0

CSI Snapshot Controller

External snapshotter и VolumeSnapshot CRDs.

Apache-2.0

Velero

Резервное копирование кластера и persistent volumes.

Apache-2.0

Container Object Storage Interface

COSI controller для managed object storage.

Apache-2.0

S3

S3 Manager

Web UI для S3-compatible buckets.

Apache-2.0

Наблюдаемость

VictoriaMetrics Operator

Хранение, прием и Prometheus-compatible query layer для метрик.

Apache-2.0

Grafana Operator

Управляет экземплярами Grafana, dashboards и datasources.

Apache-2.0

Fluent Bit

Log forwarder, работающий на каждом узле.

Apache-2.0

kube-state-metrics

Экспортирует состояние объектов Kubernetes как метрики.

Apache-2.0

node-exporter

Системные и аппаратные метрики с каждого узла.

Apache-2.0

Prometheus Operator CRDs

CRDs для Prometheus-style monitoring resources, используемые VictoriaMetrics.

Apache-2.0

Metrics Server

Метрики Kubelet для HPA и `kubectl top`.

Apache-2.0

Goldpinger

Проверки pod-to-pod connectivity в кластере.

Apache-2.0

Autoscaling и управление ресурсами

Vertical Pod Autoscaler

Вертикальный подбор ресурсов для pods (chart: MIT).

Apache-2.0

Cluster Autoscaler

Горизонтальное масштабирование node pools.

Apache-2.0

Stakater Reloader

Перезапускает pods при изменении их ConfigMaps или Secrets.

Apache-2.0

GPU и ускорители

NVIDIA GPU Operator

Lifecycle драйвера, container runtime и device-plugin для NVIDIA GPUs.

Apache-2.0

HAMi

GPU sharing и fractional GPU scheduling.

Apache-2.0

GitOps и автоматизация платформы

Flux

GitOps engine. ControlPlane Flux Operator и instance chart лицензированы под AGPL-3.0; upstream Flux controllers - под Apache-2.0.

Apache-2.0; AGPL-3.0

Aenix etcd Operator

Управляет etcd-кластерами, используемыми tenant Kamaji control planes.

Apache-2.0

[cert-manager](#)

Автоматический выпуск и ротация TLS-сертификатов.

Apache-2.0

[External Secrets Operator](#)

Синхронизирует secrets из внешнего KMS в Kubernetes.

Apache-2.0

[SAP ClusterSecret Operator](#)

Реплицирует secrets между namespaces.

Apache-2.0

[Tinkerbelle Smee](#)

iPXE / DHCP boot server для bare-metal provisioning.

Apache-2.0

[Telepresence](#)

Локальная разработка против удаленного кластера (Traffic Manager).

Apache-2.0

Identity, registry и admin UI

[Keycloak](#)

OIDC provider для platform и tenant SSO; разворачивается через KubeRocketCI Keycloak Operator.

Apache-2.0

[Harbor](#)

OCI registry для container images и Helm charts.

Apache-2.0

Managed database runtimes

PostgreSQL

Управляется через CloudNativePG operator (Apache-2.0).

PostgreSQL License

MariaDB Server

Управляется через mariadb-operator (MIT).

GPL-2.0

MongoDB (Percona Server)

Управляется через Percona Operator for MongoDB (Apache-2.0).

SSPL-1.0

ClickHouse

Server и Keeper, управляются через Altinity ClickHouse Operator (Apache-2.0).

Apache-2.0

OpenSearch

Управляется через opensearch-k8s-operator (Apache-2.0).

Apache-2.0

Qdrant

Vector database, разворачивается через upstream Qdrant Helm chart.

Apache-2.0

FoundationDB

Управляется через FoundationDB Kubernetes Operator (Apache-2.0).

Apache-2.0

Redis

Управляется через Spotahome Redis Operator (Apache-2.0). Cozystack поддерживает Redis 7.4 и Redis 8.

RSALv2 or SSPLv1 (7.x) / AGPLv3 (8.x)

Managed messaging и caching runtimes

Apache Kafka

Управляется через Strimzi Kafka Operator (Apache-2.0).

Apache-2.0

NATS

Lightweight messaging server, разворачивается через upstream NATS Helm chart.

Apache-2.0

RabbitMQ

Управляется через RabbitMQ Cluster Operator (MPL-2.0).

MPL-2.0; Apache-2.0 for some files

OpenBao

Fork HashiCorp Vault для управления secrets, разворачивается через upstream OpenBao Helm chart.

MPL-2.0

Managed networking services

NGINX

Используется managed HTTP Cache service.

BSD-2-Clause

HAProxy

Используется managed TCP Balancer и HTTP Cache services.

GPL-2.0 with exceptions

IP

IP2Location modules

GeoIP modules, включенные в HTTP Cache (IP2Location и IP2Proxy).

MIT

Outline Server (Shadowsocks)

Основа managed VPN service.

Apache-2.0

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Lineage Controller Webhook

<https://cozystack.ru/docs/v1.6/operations/configuration/lineage-controller-webhook/>

Что делает lineage-controller-webhook, как он разворачивается и какой параметр важно знать.

lineage controller webhook \- это mutating admission webhook, поставляемый в составе Package cozystack.cozystack-engine. При каждом CREATE и UPDATE tenant-ресурсов Pod, Secret, Service, PersistentVolumeClaim, Ingress или WorkloadMonitor он поднимается по графу владения и записывает идентичность владеющего Cozystack Application в labels ресурса (apps.cozystack.io/application.{group,kind,name}). Dashboard Cozystack, aggregated API server и механизм SchedulingClass опираются на эти labels.

Webhook зарегистрирован с failurePolicy: Fail, поэтому kube-apiserver должен иметь доступ к рабочему pod webhook, чтобы tenant CREATE/UPDATE-запросы проходили успешно.

Схема разворачивания по умолчанию

Chart разворачивает один Deployment, построенный по образцу cozystack-api:

- 2 replicas (переопределяется через replicas).
- **Мягкий nodeAffinity** с предпочтением node-role.kubernetes.io/control-plane (Exists совпадает и с пустым значением Talos, и с "true" в k3s/kubeadm). Это именно мягкое предпочтение: pods попадают на control-plane-узел, когда он доступен, иначе на любой worker. Для managed Kubernetes (EKS / AKS / GKE), tenant-кластеров Cozy-in-Cozy и любых кластеров, где control-plane-узлы не видны, переопределение не нужно: webhook просто планируется в другом месте.
- **Разрешающие tolerations** ({operator: Exists}), чтобы control-plane-узел с taint NoSchedule принимал pod, когда мягкий affinity может быть выполнен.
- **Мягкий podAntiAffinity** по kubernetes.io/hostname, чтобы реплики по возможности распределялись по разным узлам.
- **PodDisruptionBudget** с maxUnavailable: 1. При replicas: 2+ он ограничивает disruption одним pod; при replicas: 1 это полезный по-оп.
- **Servicespec.trafficDistribution: PreferClose**, чтобы apiserver предпочитал endpoint webhook на своем узле, если он существует, и прозрачно переключался на удаленный endpoint иначе. Требуется Kubernetes >= 1.31; более старые кластеры незаметно возвращаются к стандартному распределению по всему кластеру, что тоже безопасно, но без предпочтения локальности.

Такая схема работает как есть во всех дистрибутивах Kubernetes, которые поддерживает Cozystack. В нормальной эксплуатации ничего переопределять не требуется.

Увеличение числа реплик

Если нужно больше двух реплик, например чтобы держать по одному webhook pod рядом с каждым apiserver на control plane из пяти узлов, переопределите значение `replicas` через `Package cozystack.cozystack-engine` так же, как для любого другого компонента (см. [Компоненты](#)):

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-engine
namespace: cozy-system
spec:
variant: default
components:
lineage-controller-webhook:
values:
lineageControllerWebhook:
replicas: 5
```

`localK8sAPIEndpoint.enabled` deprecated

Chart все еще предоставляет `localK8sAPIEndpoint.enabled`. При значении `true` он добавляет `KUBERNETES_SERVICE_HOST=status.hostIP` и `KUBERNETES_SERVICE_PORT=6443`, чтобы webhook обращался к apiserver на своем узле. Изначально это было добавлено, чтобы избежать задержки на пути `webhook-to-apiserver`. Сейчас значение по умолчанию - `false`, а сам параметр планируется удалить после того, как причина с задержкой будет устранена внутри webhook.

Важно

Не включайте `localK8sAPIEndpoint.enabled` со стандартными values chart. Добавленный `status.hostIP` корректен только когда pod работает на узле с kube-apiserver, а мягкий control-plane affinity в chart этого не гарантирует. Если флаг включен, а pod запланирован не на control-plane-узел, controller уйдет в crash loop, пытаясь подключиться к IP, где нет apiserver. В сочетании с `failurePolicy: Fail` это означает отказ tenant CREATE/UPDATE-операций.

Проверка разворачивания

```
kubectl -n cozy-system get deploy lineage-controller-webhook
```

```
kubectl -n cozy-system get pods -l app=lineage-controller-webhook
```

```
kubectl -n cozy-system get svc
lineage-controller-webhook -o yaml
```

```
grep trafficDistribution
```

Быстрая end-to-end-проверка, которая вызывает webhook через apiserver:

```
kubectl create ns lineage-webhook-test
```

```
kubectl -n lineage-webhook-test create service clusterip probe \
```

```
--clusterip=None --dry-run=server
```

```
kubectl delete ns lineage-webhook-test
```

Dry-run CREATE проходит через mutating admission webhook; если webhook недоступен, команда завершается ошибкой `failed calling webhook "lineage.cozystack.io"`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

White Labeling

<https://cozystack.ru/docs/v1.6/operations/configuration/white-labeling/>

Настройка брендинга в Cozystack Dashboard и на страницах аутентификации Keycloak, включая custom Keycloak themes

White labeling позволяет заменить стандартный брендинг Cozystack собственными логотипами и текстами в Dashboard UI и на страницах аутентификации Keycloak.

Обзор

Брендинг настраивается через поле `branding` в Platform Package (`spec.components.platform.values.branding`). Конфигурация автоматически распространяется на:

- Dashboard : логотип, заголовок страницы, текст footer, favicon и идентификатор tenant
- Keycloak : display name realm на страницах аутентификации

Конфигурация

Измените Platform Package, чтобы добавить или обновить секцию `branding`:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cozystack-platform
spec:
  variant: isp-full # используйте свой вариант
  components:
    platform:
      values:
        branding:
          # Брендинг Dashboard
          titleText: "My Company Dashboard"
          footerText: "My Company Platform"
          tenantText: "Production v1.0"
          logoText: ""
          logoSvg: "<base64-encoded SVG>"
```

iconSvg: "<base64-encoded SVG>"

Брендинг Keycloak

brandName: "My Company"

brandHtmlName: "<div style='font-weight:bold;'>My Company</div>"

Примените изменения:

kubectl apply --server-side --filename platform-package.yaml

Поля конфигурации

Поля Dashboard

| Поле | По умолчанию | Описание |
|------------|----------------------------|--|
| titleText | Cozystack Dashboard | Заголовок вкладки браузера и текст header в Dashboard. |
| footerText | Cozystack | Текст, отображаемый в footer Dashboard. |
| tenantText | Строка версии platform | Версия или идентификатор tenant, отображаемый в Dashboard. |
| logoText | "" (пусто) | Альтернативный текстовый логотип. Используется, если SVG-логотип не задан. |
| logoSvg | Логотип Cozystack (base64) | SVG-логотип в base64, отображаемый в header Dashboard. |
| iconSvg | Иконка Cozystack (base64) | SVG-иконка в base64, используемая как browser favicon. |

Поля Keycloak

| Поле | По умолчанию | Описание |
|-----------|--------------|--|
| brandName | Не задано | Plain text имя realm, отображаемое во вкладке браузера Keycloak. |

| | | |
|---------------|-----------|--|
| brandHtmlName | Не задано | HTML-форматированное имя realm, отображаемое на страницах входа Keycloak. Поддерживает inline HTML/CSS для стилизованного брендинга. |
|---------------|-----------|--|

Подготовка SVG-логотипов

SVG-переменные с учетом темы

Dashboard поддерживает template variables в SVG, которые адаптируются к светлой и темной темам:

- `{token.colorText}` \- во время выполнения заменяется на цвет текста текущей темы

Примечание: Синтаксис `{token.colorText}` не является валидным XML. Значение атрибута намеренно не заключено в кавычки, потому что Dashboard выполняет raw string substitution в исходном SVG перед render: заменяет `{token.colorText}` на фактическое значение цвета. Это означает, что SVG-файлы с такими placeholders нельзя напрямую открыть в браузере или проверить XML parser. Это ожидаемое поведение, соответствующее upstream-реализации Dashboard.

Пример SVG с theme-aware переменной:

```
<svg width="150" height="30" viewBox="0 0 150 30" fill="none"
xmlns="http://www.w3.org/2000/svg">
<path d="M10 5h30v20H10z" fill={token.colorText} />
<text x="50" y="20" fill={token.colorText}>My Company</text>
</svg>
```

Конвертация SVG в Base64

Закодируйте SVG-файлы в строки base64:

| | |
|-----------------------------|-------------------|
| base64 < logo.svg | tr -d '\n' |
|-----------------------------|-------------------|

Пример workflow

Закодировать логотипы

LOGO_B64=\$(base64 < logo.svg | tr -d '\n')

| | |
|---|--------------------|
| ICON_B64=\$(base64 < icon.svg | tr -d '\n') |
|---|--------------------|

Изменить Platform Package

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform \
--type merge --server-side \
--patch "{
  \"spec\": {
    \"components\": {
      \"platform\": {
        \"values\": {
          \"branding\": {
            \"logoSvg\": \"\$LOGO_B64\",
            \"iconSvg\": \"\$ICON_B64\"
          }
        }
      }
    }
  }
}"
```

Проверка

После применения изменений проверьте, что брендинг настроен корректно:

1. Проверьте Platform Package :

```
kubectl get packages.cozystack.io cozystack.cozystack-platform \
```

| | |
|--|-------------------|
| <code>--output jsonpath='{.spec.components.platform.values.branding}'</code> | <code>jq .</code> |
|--|-------------------|

2. Dashboard : откройте URL Dashboard и проверьте логотип, заголовок, footer и favicon.

3. Keycloak : откройте страницу входа Keycloak и проверьте display name realm.

Примечание: Чтобы увидеть обновленный брендинг, может потребоваться hard refresh (Ctrl+Shift+R / Cmd+Shift+R) или очистка кеша браузера.

Custom Keycloak themes

Для более глубокой визуальной настройки страниц аутентификации Keycloak (login, registration, account management) можно подключать custom themes, собранные как container images.

Контракт theme image

Theme image должен содержать файлы темы в директории `/themes/`. Структура директорий должна соответствовать стандартному [формату themes Keycloak](#):

```
/themes/  
my-brand/  
login/  
theme.properties  
resources/  
css/  
img/  
account/  
theme.properties
```

При запуске `pod init containers` копируют файлы из каждого theme image в директорию Keycloak `/opt/keycloak/themes/`. Встроенные themes Keycloak, упакованные в JAR-файлы, не затрагиваются.

Если несколько theme images содержат файлы по одному и тому же path, более поздние записи в списке имеют приоритет.

Конфигурация

Custom themes настраиваются на системном компоненте Keycloak. Измените Package `cozystack.keycloak`:

```
apiVersion: cozystack.io/v1alpha1  
kind: Package  
metadata:  
  name: cozystack.keycloak  
  namespace: cozy-system  
spec:  
  variant: default  
  components:  
    keycloak:  
      values:  
        themes:
```

```
- name: my-brand
image: registry.example.com/my-keycloak-theme:v1.0
```

Примените изменения:

```
kubectl apply --server-side --filename keycloak-package.yaml
```

Поля theme

| Поле | Обязательно | Описание |
|-------|-------------|---|
| name | Да | Идентификатор theme. Используется как имя init container после приведения к формату DNS-1123. |
| image | Да | Container image, содержащий файлы theme в <code>/themes/</code> . |

Private registries

Если theme images хранятся в private registry, добавьте `imagePullSecrets`:

keycloak:

values:

themes:

```
- name: my-brand
image: private-registry.example.com/my-keycloak-theme:v1.0
imagePullSecrets:
  - name: my-registry-secret
```

Указанный Secret должен существовать в namespace `cozy-keycloak`.

Активация custom theme

После развертывания theme image активируйте его в Keycloak:

1. Откройте admin console Keycloak.
2. Перейдите в Realm Settings > Themes.
3. Выберите custom theme в выпадающем списке для нужного типа темы: login, account, email или admin.
4. Сохраните изменения.

Миграция с v0

В Cozystack v0 брендинг настраивался через отдельный ConfigMap `cozystack-branding` в namespace `cozy-system`. В v1 этот ConfigMap больше не используется. [Скрипт миграции](#) автоматически преобразует старые значения ConfigMap в поле `branding` Platform Package.

Если раньше вы использовали подход с ConfigMap, ручная миграция не нужна: процесс обновления выполнит ее автоматически.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Telemetry

<https://cozystack.ru/docs/v1.6/operations/configuration/telemetry/>

Телеметрия Cozystack

В этом документе описана телеметрия в проекте Cozystack: зачем собираются данные, какие именно данные собираются, как они обрабатываются и как отказаться от участия.

Зачем мы собираем телеметрию

Cozystack как open source проект развивается благодаря обратной связи сообщества и пониманию реального использования. Данные телеметрии позволяют maintainers понимать, как Cozystack применяется в реальных сценариях. Эти данные помогают принимать решения о приоритете функций, стратегиях тестирования, исправлениях ошибок и общем развитии проекта. Без телеметрии решения приходилось бы принимать на основе предположений или ограниченной обратной связи, что могло бы замедлить улучшения или привести к функциям, которые не соответствуют потребностям пользователей. Телеметрия помогает направлять разработку реальными паттернами использования и требованиями сообщества, делая платформу более надежной и ориентированной на пользователей.

Что и как мы собираем

Cozystack стремится соблюдать [LF Telemetry Data Policy](#), обеспечивая ответственную практику сбора данных с уважением к приватности пользователей и прозрачности.

Мы собираем не персональные сведения о пользователях, а неперсональные метрики использования компонентов Cozystack. В частности, собирается информация об инфраструктуре кластера (узлы, хранилище, сеть), установленных packages и экземплярах приложений. Эти данные помогают понимать распространенные конфигурации и тенденции использования в разных установках.

Телеметрию собирают два компонента:

- cozystack-operator \- собирает метрики уровня кластера (узлы, хранилище, packages)
- cozystack-controller \- собирает метрики уровня приложений (развернутые экземпляры приложений)

Чтобы подробно увидеть, какие данные собираются, можно изучить реализацию телеметрии:

- [Telemetry Client](#)
- [Telemetry Server](#)

Пример telemetry payload

Ниже показан типичный telemetry payload в Cozystack.

От cozystack-operator (инфраструктура кластера):

```
cozy_cluster_info{cozystack_version="v1.0.0",kubernetes_version="v1.31.4"} 1
cozy_nodes_count{os="linux (Talos (v1.8.4))",kernel="6.6.64-talos"} 3
cozy_cluster_capacity{resource="cpu"} 168
cozy_cluster_capacity{resource="memory"} 811020009472
cozy_cluster_capacity{resource="nvidia.com/TU104GL_TESLA_T4"} 3
cozy_loadbalancers_count 1
cozy_pvs_count{driver="linstor.csi.linbit.com",size="5Gi"} 7
cozy_pvs_count{driver="linstor.csi.linbit.com",size="10Gi"} 6
cozy_package_info{name="cozystack.core",variant="default"} 1
cozy_package_info{name="cozystack.storage",variant="linstor"} 1
cozy_package_info{name="cozystack.monitoring",variant="default"} 1
```

От cozystack-controller (экземпляры приложений):

```
cozy_application_count{kind="Tenant"} 2
cozy_application_count{kind="Postgres"} 5
cozy_application_count{kind="Redis"} 3
cozy_application_count{kind="Kubernetes"} 2
cozy_application_count{kind="VirtualMachine"} 0
```

Данные собираются компонентами, работающими внутри Cozystack. Они периодически собирают и отправляют статистику использования в наш защищенный backend. Система телеметрии обеспечивает анонимизацию, агрегацию и безопасное хранение данных, а доступ к ним строго контролируется для защиты приватности пользователей.

Отказ от телеметрии

Мы уважаем вашу приватность и выбор в отношении телеметрии. Если вы не хотите участвовать в сборе telemetry data, Cozystack предоставляет простой способ отказаться.

Отключение:

Чтобы отключить отправку телеметрии, обновите Helm release оператора Cozystack с флагом `disableTelemetry`:

```
helm upgrade cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--namespace cozy-system \
```

```
--version X.Y.Z \
```

```
--set cozystackOperator.disableTelemetry=true
```

Замените `X.Y.Z` на установленную у вас версию Cozystack.

Do not use `--reuse-values` when upgrading the Cozystack operator. The Helm chart values contain hardcoded references to the platform OCI repository. Reusing old values would result in the operator pointing to old package versions.

If you have custom values (e.g., `disableTelemetry`), pass them explicitly with `--set`.

Эта команда обновляет оператор и отключает сбор телеметрии. Если позже нужно снова включить телеметрию, выполните ту же команду с `disableTelemetry=false`.

Заключение

Телеметрия в Cozystack нужна, чтобы развитие проекта опиралось на данные, отвечало потребностям сообщества и обеспечивало постоянное улучшение. Ваше участие или отказ от него помогает формировать будущее Cozystack и делать платформу более эффективной и удобной для всех.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

VPC и подсети

<https://cozystack.ru/docs/v1.6/operations/vpc/>

Как использовать VPC и подсети

Подключение workloads к подсетям

После создания VPC с набором подсетей к ним можно подключать рабочую нагрузку. Сейчас подключение к подсетям поддерживают только виртуальные машины.

1\ Получите ID подсетей

Сначала нужно определить ID подсетей. ID автоматически формируются из имен ресурсов, указанных при создании. Проверьте их на вкладке Details вашей VPC.

2\ Укажите ID подсетей при создании ресурса

При создании VM укажите ID подсетей в разделе Subnets настроек ресурса.

Каждая подсеть будет представлена как дополнительный сетевой интерфейс. Для некоторых гостевых операционных систем нужно добавить конфигурацию сетевого интерфейса в user-data виртуальной машины. Также можно поднять дополнительные интерфейсы вручную и получить IP-адреса через DHCP.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Руководства по обслуживанию кластера

<https://cozystack.ru/docs/v1.6/operations/cluster/>

Руководства по регулярным операциям с кластером: добавлению и удалению узлов, обновлению Talos и другим задачам.

Руководства по регулярным операциям с кластером: добавлению и удалению узлов, обновлению Talos и другим задачам.

-

Обновление Cozystack и проверки после обновления

Обновление системных компонентов Cozystack.

Масштабирование кластера: добавление и удаление узлов

Добавление и удаление узлов в кластере Cozystack.

Как ротировать Certificate Authority

Как ротировать Certificate Authority

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Обновление Cozystack и проверки после обновления

<https://cozystack.ru/docs/v1.6/operations/cluster/upgrade/>

Обновление системных компонентов Cozystack.

О версиях Cozystack

Cozystack использует поэтапный процесс выпуска релизов, чтобы сохранять стабильность и гибкость во время разработки.

Существует три типа релизов:

- Alpha, Beta и Release Candidate (RC) \- предварительные версии, например `v0.42.0-alpha.1` или `v0.42.0-rc.1`, которые используются для финального тестирования и проверки.
- Стабильные релизы \- обычные версии, например `v0.42.0`, с полным набором функций и тщательным тестированием. Такие версии обычно добавляют новые возможности, обновляют зависимости и могут содержать изменения API.
- Патч-релизы \- обновления только с исправлениями ошибок, например `v0.42.1`, которые выпускаются после стабильного релиза из отдельной release-ветки.

В production-средах настоятельно рекомендуется устанавливать только стабильные релизы и патч-релизы.

Полный список релизов доступен на странице [Releases](#) в GitHub.

Подробнее о процессе выпуска Cozystack см. в документе [Cozystack Release Workflow](#).

Обновление Cozystack

1\ Проверьте состояние кластера

Перед обновлением проверьте текущее состояние кластера Cozystack по шагам из раздела

- [Чек-лист диагностики](#)

Убедитесь, что Platform Package находится в рабочем состоянии и содержит ожидаемую конфигурацию:

```
kubectl get packages.cozystack.io cozystack.cozystack-platform -o yaml
```

2\ Защитите критически важные ресурсы

Перед обновлением добавьте аннотацию `helm.sh/resource-policy=keep` к namespace `cozy-system` и ConfigMap `cozystack-version`, чтобы Helm не удалил их во время обновления:

```
kubectl annotate namespace cozy-system helm.sh/resource-policy=keep --overwrite
```

```
kubectl annotate configmap -n cozy-system cozystack-version helm.sh/resource-policy=keep --overwrite
```

Этот шаг обязателен. Без этих аннотаций удаление или обновление Helm-релиза установщика может удалить namespace `cozy-system` и все ресурсы внутри него.

3\. Обновите оператор Cozystack

Обновите Helm-релиз оператора Cozystack до целевой версии:

Do not use `--reuse-values` when upgrading the Cozystack operator. The Helm chart values contain hardcoded references to the platform OCI repository. Reusing old values would result in the operator pointing to old package versions.

If you have custom values (e.g., `disableTelemetry`), pass them explicitly with `--set`.

```
helm upgrade cozystack oci://ghcr.io/cozystack/cozystack/cozy-installer \
--version X.Y.Z \
--namespace cozy-system
```

Логи оператора можно посмотреть так:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

4\. Проверьте состояние кластера после обновления

```
kubectl get pods -n cozy-system
```

```
kubectl get hr -A
```

```
grep -v "True"
```

Если статус pod указывает на сбой, проверьте логи:

```
kubectl logs -n cozy-system deploy/cozystack-operator --previous
```

Чтобы убедиться, что все работает как ожидается, повторите шаги из раздела

- [Чек-лист диагностики](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Масштабирование кластера: добавление и удаление узлов

<https://cozystack.ru/docs/v1.6/operations/cluster/scaling/>

Добавление и удаление узлов в кластере Cozystack.

Как добавить узел в кластер Cozystack

Узел добавляется почти так же, как при обычной установке Cozystack.

1. Установите Talos на узел, используя кастомный образ Talos для Cozystack.

2. Сгенерируйте конфигурацию для нового узла по руководству [Talm](#) или [talosctl](#).

Например, конфигурация узла control plane:

```
talm template -e 192.168.123.20 -n 192.168.123.20 -t templates/controlplane.yaml -i > nodes/nodeN.yaml
```

а для worker-узла:

```
talm template -e 192.168.123.20 -n 192.168.123.20 -t templates/worker.yaml -i > nodes/nodeN.yaml
```

3. Примените сгенерированную конфигурацию к узлу по руководству [Talm](#) или [talosctl](#). Например:

```
talm apply -f nodes/nodeN.yaml -i
```

4. Дождитесь, пока узел перезагрузится и самостоятельно подключится к кластеру. Запускать bootstrap вручную или устанавливать на него Cozystack не нужно: все будет выполнено автоматически.

Результат можно проверить командой `kubectl get nodes`.

Как удалить узел из кластера Cozystack

Когда узел кластера выходит из строя, Cozystack автоматически поддерживает отказоустойчивость, пересоздавая реплицированные PVC и workloads на других узлах. Однако иногда для устранения проблем узел нужно удалить:

- PV локального хранилища могут остаться привязанными к отказавшему узлу, что вызывает проблемы с новыми pod. Такие ресурсы нужно очистить вручную.
- Отказавший узел будет по-прежнему существовать в кластере, что может привести к несогласованности состояния кластера и повлиять на планирование pod.

Шаг 1: удалите узел из кластера

Выполните следующую команду, чтобы удалить отказавший узел. Замените `mynode` на фактическое имя узла:

```
kubectl delete node mynode
```

Если отказавший узел был узлом control plane, также удалите его etcd member из кластера etcd:

```
talm -f nodes/node1.yaml etcd member list
```

Пример вывода:

```

NODE ID HOSTNAME PEER URLS CLIENT URLS LEARNER
37.27.60.28 2ba6e48b8cf1a0c1 node1 https://192.168.100.11:2380 https://192.168.100.11:2379 false
37.27.60.28 b82e2194fb76ee42 node2 https://192.168.100.12:2380 https://192.168.100.12:2379 false
37.27.60.28 f24f4de3d01e5e88 node3 https://192.168.100.13:2380 https://192.168.100.13:2379 false
```

Затем удалите соответствующий member. Замените ID на ID отказавшего узла:

```
talm -f nodes/node1.yaml etcd remove-member f24f4de3d01e5e88
```

Шаг 2: удалите PVC и pod, привязанные к отказавшему узлу

Ниже несколько команд, которые помогут очистить отказавший узел:

- Удалите PVC, привязанные к отказавшему узлу:

(Замените `mynode` на имя отказавшего узла)

| | | | | |
|-------------------------------------|------------------------------|---|---|--------------------|
| <code>kubectl get pv -o json</code> | <code>jq -r '.items[]</code> | <code>select(.spec.nodeAffinity.required.nodeSelectorTerms[0].matchExpressions[0].values[0] == "mynode").spec.claimRef</code> | <code>"kubectl delete pvc -n \(.namespace) \(.name)"</code> | <code>sh -x</code> |
|-------------------------------------|------------------------------|---|---|--------------------|

- Удалите pod, зависшие в состоянии `Pending` во всех namespaces:

| | | |
|---------------------------------|---|--------------------|
| <code>kubectl get pod -A</code> | <code>awk '/Pending/ {print "kubectl delete pod -n " \$1 " " \$2}'</code> | <code>sh -x</code> |
|---------------------------------|---|--------------------|

Шаг 3: проверьте состояние ресурсов

После очистки проверьте проблемы с ресурсами с помощью `linstor advise`:

```
# linstor advise resource
```

```
+-----+
-----+

: Resource : Issue : Possible fix :

+=====+
=====+

: pvc-02b0c0a1-e0b6-4e98-9384-60ff24f3b3b6 : Resource expected to have 3 replicas, got only 2. : linstor
rd ap --place-count 3 pvc-02b0c0a1-e0b6-4e98-9384-60ff24f3b3b6 :

: pvc-06e3b406-23f0-4f10-8b03-84063c1b2a12 : Resource expected to have 3 replicas, got only 2. : linstor
rd ap --place-count 3 pvc-06e3b406-23f0-4f10-8b03-84063c1b2a12 :

: pvc-a0b8aeaf-076e-4bd9-93ed-c4db09c04d0b : Resource expected to have 3 replicas, got only 2. :
linstor rd ap --place-count 3 pvc-a0b8aeaf-076e-4bd9-93ed-c4db09c04d0b :

: pvc-a523eb3b-c3b6-468d-abe5-f6afbbf31081 : Resource expected to have 3 replicas, got only 2. : linstor
rd ap --place-count 3 pvc-a523eb3b-c3b6-468d-abe5-f6afbbf31081 :

: pvc-cf7e87b5-3e6d-4034-903d-4625830fb5b4 : Resource expected to have 1 replicas, got only 0. : linstor
rd ap --place-count 1 pvc-cf7e87b5-3e6d-4034-903d-4625830fb5b4 :

: pvc-d344bc83-97fd-4489-bbe7-5399eea57165 : Resource expected to have 3 replicas, got only 2. :
linstor rd ap --place-count 3 pvc-d344bc83-97fd-4489-bbe7-5399eea57165 :

: pvc-d39345a9-5446-4c64-a5ba-957ff7c7a31f : Resource expected to have 3 replicas, got only 2. : linstor
rd ap --place-count 3 pvc-d39345a9-5446-4c64-a5ba-957ff7c7a31f :

: pvc-db6d4236-93bd-4268-9dcc-0ed275b17067 : Resource expected to have 1 replicas, got only 0. :
linstor rd ap --place-count 1 pvc-db6d4236-93bd-4268-9dcc-0ed275b17067 :

: pvc-ebb412c3-083c-4eee-93dc-70917ea6d87e : Resource expected to have 1 replicas, got only 0. :
linstor rd ap --place-count 1 pvc-ebb412c3-083c-4eee-93dc-70917ea6d87e :

: pvc-f107aacb-78d7-4ac6-97f8-8ed529a9c292 : Resource expected to have 3 replicas, got only 2. : linstor
rd ap --place-count 3 pvc-f107aacb-78d7-4ac6-97f8-8ed529a9c292 :

: pvc-f347d71a-b646-45e5-a717-f0a745061beb : Resource expected to have 1 replicas, got only 0. : linstor
rd ap --place-count 1 pvc-f347d71a-b646-45e5-a717-f0a745061beb :

: pvc-f6e96c83-6144-4510-b0ab-61936db52391 : Resource expected to have 3 replicas, got only 2. :
linstor rd ap --place-count 3 pvc-f6e96c83-6144-4510-b0ab-61936db52391 :

+-----+
-----+
```

Run the `linstor rd ap` commands suggested in the “Possible fix” column to restore the desired replica count.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как ротировать Certificate Authority

<https://cozystack.ru/docs/v1.6/operations/cluster/rotate-ca/>

Как ротировать Certificate Authority

Talos создает корневые центры сертификации со сроком действия 10 лет, и все сертификаты Talos API и Kubernetes API выпускаются этими корневыми СА. В обычной эксплуатации почти никогда не требуется ротировать корневой СА-сертификат и ключ для Talos API и Kubernetes API.

Ротация корневого СА нужна только в следующих случаях:

- есть подозрение, что private key был скомпрометирован;
- нужно отозвать доступ к кластеру для утекшего `talosconfig` или `kubeconfig`;
- прошло 10 лет.

Ротация СА для Talos API

Чтобы ротировать Talos СА для management-кластера, используйте следующую команду.

Сначала запустите ее в dry-run-режиме, чтобы предварительно посмотреть изменения:

```
talm -f nodes/node.yaml rotate-ca --talos=true --kubernetes=false
```

Затем выполните реальную ротацию:

```
talm -f nodes/node.yaml rotate-ca --talos=true --kubernetes=false --dry-run=false
```

После завершения ротации скачайте новый `talosconfig` из secrets.

Ротация СА для management-кластера Kubernetes

Чтобы ротировать Kubernetes СА для management-кластера, используйте следующую команду.

Сначала запустите ее в dry-run-режиме, чтобы предварительно посмотреть изменения:

```
talm -f nodes/node.yaml rotate-ca --talos=false --kubernetes=true
```

Затем выполните реальную ротацию:

```
talm -f nodes/node.yaml rotate-ca --talos=false --kubernetes=true --dry-run=false
```

Ротация СА для tenant-кластера Kubernetes

См.: <<https://kamaji.clastix.io/guides/certs-lifecycle/>>

```
export NAME=k8s-cluster-name
export NAMESPACE=k8s-cluster-namespace

kubectl -n ${NAMESPACE} delete secret ${NAME}-ca
kubectl -n ${NAMESPACE} delete secret ${NAME}-sa-certificate

kubectl -n ${NAMESPACE} delete secret ${NAME}-api-server-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-api-server-kubelet-client-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-datastore-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-front-proxy-client-certificate
kubectl -n ${NAMESPACE} delete secret ${NAME}-konnectivity-certificate

kubectl -n ${NAMESPACE} delete secret ${NAME}-admin-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-controller-manager-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-konnectivity-kubeconfig
kubectl -n ${NAMESPACE} delete secret ${NAME}-scheduler-kubeconfig

kubectl delete po -l app.kubernetes.io/name=kamaji -n cozy-kamaji
kubectl delete po -l app=${NAME}-kcsi-driver
```

Дождитесь перезапуска `pod virt-launcher-kubernetes-*`. После этого скачайте новый сертификат Kubernetes.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник cluster services

<https://cozystack.ru/docs/v1.6/operations/services/>

Системные middleware packages, которые разворачиваются в tenants и предоставляют основную функциональность пользовательским приложениям.

Мониторинг

Система мониторинга в Cozystack предоставляет полноценную наблюдаемость для ресурсов системного уровня и уровня tenant. Она работает на двух основных уровнях: общекластерный мониторинг инфраструктурных компонентов и tenant-specific мониторинг пользовательских приложений и сервисов.

Обзор архитектуры

- Системный уровень : мониторит основные компоненты Cozystack, Kubernetes-кластеры и базовую инфраструктуру.
- Уровень tenant : предоставляет изолированные monitoring stacks для каждого tenant, позволяя им мониторить свои приложения без взаимного влияния.

Ключевые компоненты

- VMAgent : собирает метрики из разных источников и отправляет их в VictoriaMetrics.
- VMCluster : кластер VictoriaMetrics для хранения и запросов метрик.
- Grafana : инструмент визуализации и dashboards для метрик и логов.
- Alerta : система alerting для уведомлений на основе thresholds метрик.

Потоки данных

Метрики идут от exporters, например node-exporters и kube-state-metrics, в VMAgent, который затем записывает их в VMCluster. Grafana запрашивает данные из VMCluster для визуализации, а Alerta обрабатывает alerts из VMCluster или других источников.

Подробную конфигурацию см. в [справочнике Monitoring Hub](#).

Cozystack включает ряд cluster services. Они разворачиваются через настройки tenant, а не через каталог приложений.

Каждый tenant может иметь собственную копию cluster service или использовать сервисы родительского tenant. Подробнее о механизме sharing services см. в [Tenant System](#)

.

[Object Storage](#)

S3-compatible object storage в Cozystack на базе SeaweedFS и COSI

[Backup Classes](#)

The platform-managed cozy-default BackupClass: what it covers, how tenants reference it, and the admin override paths.

[Справочник сервиса BootBox](#)

[Справочник сервиса Etcd](#)

[Справочник Ingress-NGINX Controller](#)

[Справочник сервиса SeaweedFS](#)

[Справочник Monitoring Hub](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Object Storage

<https://cozystack.ru/docs/v1.6/operations/services/object-storage/>

S3-compatible object storage в Cozystack на базе SeaweedFS и COSI

Cozystack предоставляет S3-compatible object storage на базе [SeaweedFS](#) и [SeaweedFS COSI Driver](#).

Как это работает

Object storage в Cozystack состоит из нескольких уровней:

1. SeaweedFS работает как cluster service в namespace tenant и предоставляет S3 backend.
2. Storage pools (опционально) разделяют volume servers по типу диска (SSD, HDD, NVMe), каждый pool создает собственный набор COSI resources.
3. BucketClasses определяют, как создаются buckets. Каждый pool создает стандартный BucketClass и BucketClass с суффиксом `-lock` для object locking.
4. BucketAccessClasses определяют credential policies: read-write и read-only.
5. Buckets \- пользовательские ресурсы, которые запрашивают хранилище из BucketClass и создают credentials для каждого пользователя через BucketAccess.

SeaweedFS Cluster

```
+-- Storage Pool "ssd" (diskType: ssd)
| +- BucketClass: tenant-seaweedfs-ssd
| +- BucketClass: tenant-seaweedfs-ssd-lock
| +- BucketAccessClass: tenant-seaweedfs-ssd (readwrite)
| +- BucketAccessClass: tenant-seaweedfs-ssd-readonly (readonly)
+-- Storage Pool "hdd" (diskType: hdd)
+-- BucketClass: tenant-seaweedfs-hdd
+-- ...
```

Руководства

- [Storage Pools](#) – настройка SeaweedFS storage pools для tiered storage
- [Buckets](#) – создание buckets и управление user credentials

Справочник

-
- [SeaweedFS Service Reference](#) – autogenerated parameter table для SeaweedFS package
 - [SeaweedFS Multi-DC Configuration](#) – развертывание SeaweedFS в нескольких дата-центрах

•

[Storage Pools](#)

Настройка SeaweedFS storage пулов для tiered object storage

[Buckets и пользователи](#)

Создание S3 buckets и управление учетными данными

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Storage Pools

<https://cozystack.ru/docs/v1.6/operations/services/object-storage/storage-pools/>

Настройка SeaweedFS storage пулов для tiered object storage

Storage пулы позволяют разделять SeaweedFS volume servers по типу диска. Каждый пул создает отдельный Volume StatefulSet с tag SeaweedFS `diskType` и соответствующий набор COSI resources (BucketClasses и BucketAccessClasses), на которые могут ссылаться buckets.

Когда использовать пулы

Используйте storage пулы, когда в кластере доступны разные уровни хранения и требуется управлять тем, какой из них будет использоваться для бакета. Например, у вас могут быть быстрые NVMe-диски для часто запрашиваемых данных и большие HDD-диски для архивного хранения.

Если все volume servers используют одно и то же хранилище, пулы не нужны: достаточно BucketClass по умолчанию.

Включение SeaweedFS для tenant

Перед настройкой пулов включите SeaweedFS для tenant:

```
kubectl patch -n tenant-root tenants.apps.cozystack.io root --type=merge -p '{
  "spec":{
    "seaweedfs": true
  }
}'
```

Дождитесь готовности SeaweedFS HelmRelease:

```
kubectl -n tenant-root get hr seaweedfs
```

Ожидаемый вывод:

```
NAME AGE READY STATUS
```

```
seaweedfs 2m True Helm upgrade succeeded for release tenant-root/seaweedfs.v1 with chart seaweedfs@...
```

Конфигурация pool

Когда SeaweedFS запущен, пропатчите его HelmRelease, чтобы добавить storage пулы:

```
kubectl patch -n tenant-root helmreleases.helm.toolkit.fluxcd.io seaweedfs --type=merge -p '{
  "spec":{
    "values":{
      "volume":{
        "pools":{
          "ssd":{
            "diskType": "ssd",
            "size": "50Gi",
            "storageClass": "local-nvme"
          },
          "hdd":{
            "diskType": "hdd",
            "size": "500Gi",
            "storageClass": "local-hdd",
            "replicas": 3
          }
        }
      }
    }
  }
}'
```

Эквивалентный полный ресурс выглядит так:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
  name: seaweedfs
  namespace: tenant-example
spec:
  host: s3.example.com
  topology: Simple
  volume:
    replicas: 2
```

size: 100Gi
pools:
ssd:
diskType: ssd
size: 50Gi
storageClass: local-nvme
hdd:
diskType: hdd
size: 500Gi
storageClass: local-hdd
replicas: 3

Параметры pool

| Параметр | Обязательный | Описание |
|-----------------|--------------|--|
| diskType | Да | Tag типа диска SeaweedFS: lowercase alphanumeric, например <code>ssd</code> , <code>hdd</code> , <code>nvme</code> |
| replicas | Нет | Количество реплик volume server. По умолчанию <code>volume.replicas</code> |
| size | Нет | Размер PVC на реплику. По умолчанию <code>volume.size</code> |
| storageClass | Нет | Kubernetes StorageClass для PVC. По умолчанию <code>volume.storageClass</code> |
| resources | Нет | Явные CPU/memory limits. По умолчанию <code>volume.resources</code> |
| resourcesPreset | Нет | Sizing preset, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code> |

Правила именования

Имена пулов должны быть валидными DNS labels: строчные буквы, цифры и дефисы. Следующие суффиксы зарезервированы и не должны использоваться как имена пулов:

- Имена, заканчивающиеся на `-lock` (зарезервировано для object-lock BucketClasses)
- Имена, заканчивающиеся на `-readOnly` (зарезервировано для read-only BucketAccessClasses)

COSI resources, создаваемые для каждого pool

Каждый pool автоматически создает четыре COSI resources:

| Ресурс | Шаблон имени | Назначение |
|-------------------|-----------------------------|---|
| BucketClass | {namespace}-{pool} | Стандартное создание bucket |
| BucketClass | {namespace}-{pool}-lock | Создание bucket с включенным object locking |
| BucketAccessClass | {namespace}-{pool} | Read-write credentials |
| BucketAccessClass | {namespace}-{pool}-readonly | Read-only credentials |

Например, pool с именем `ssd` в namespace `tenant-example` создает:

- BucketClass `tenant-example-ssd`
- BucketClass `tenant-example-ssd-lock`
- BucketAccessClass `tenant-example-ssd`
- BucketAccessClass `tenant-example-ssd-readonly`

Примечание: Набор COSI resources по умолчанию без пула всегда создается с использованием только имени namespace, например `tenant-example`, `tenant-example-lock`. Он соответствует `volume.servers`, работающим с настройкой верхнего уровня `volume.diskType`.

MultiZone topology с пулами

В многозонной топологии пулы определяются отдельно для каждой зоны в `volume.zones[zone].pools`.

Параметры зоны

Каждая зона, помимо пула, принимает следующие параметры:

| Параметр | Обязательный | Описание |
|---------------------------|--------------|---|
| <code>replicas</code> | Нет | Количество реплик <code>volume server</code> в этой зоне. По умолчанию <code>volume.replicas</code> |
| <code>size</code> | Нет | Размер PVC на реплику. По умолчанию <code>volume.size</code> |
| <code>storageClass</code> | Нет | Kubernetes StorageClass для PVC. По умолчанию <code>volume.storageClass</code> |

| | | |
|---------------------------|-----|---|
| <code>dataCenter</code> | Нет | Имя data center в SeaweedFS. По умолчанию имя ключа zone, например zone <code>dc1</code> получает <code>dataCenter: dc1</code> |
| <code>nodeSelector</code> | Нет | YAML nodeSelector для планирования pods сервера volume. По умолчанию <code>topology.kubernetes.io/zone: <zoneName></code> |
| <code>pools</code> | Нет | сопоставление пулов хранения для этой зоны. Структура такая же, как у <code>volume.pools</code> |

Пример

Пропатчите SeaweedFS HelmRelease, чтобы добавить пулы для каждой зоны:

```
kubectl patch -n tenant-example helmreleases.helm.toolkit.fluxcd.io seaweedfs --type=merge -p '{
"spec":{
"values":{
"volume":{
"zones":{
"dc1":{
"pools":{
"ssd":{"diskType": "ssd", "size": "50Gi"},
"hdd":{"diskType": "hdd", "size": "500Gi"}
}
},
"dc2":{
"pools":{
"ssd":{"diskType": "ssd", "size": "50Gi"},
"hdd":{"diskType": "hdd", "size": "500Gi"}
}
}
}
}
}
```

}

В этом примере zone `dc1` автоматически получает `dataCenter: dc1` и `nodeSelector: {topology.kubernetes.io/zone: dc1}`.

Чтобы переопределить значения по умолчанию, задайте их явно:

```
volume:
zones:
us-east-1a:
dataCenter: us-east
nodeSelector:
topology.kubernetes.io/zone: us-east-1a
node.kubernetes.io/instance-type: storage-optimized
pools:
ssd:
diskType: ssd
size: 50Gi
```

Эквивалентный полный ресурс с явными настройками zones:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
name: seaweedfs
namespace: tenant-example
spec:
host: s3.example.com
topology: MultiZone
volume:
replicas: 2
size: 100Gi
zones:
dc1:
pools:
ssd:
diskType: ssd
size: 50Gi
```

```
hdd:
diskType: hdd
size: 500Gi

dc2:
pools:
ssd:
diskType: ssd
size: 50Gi

hdd:
diskType: hdd
size: 500Gi
```

Каждая комбинация зона+пул создает собственный Volume StatefulSet. В этом примере это четыре StatefulSets: `seaweedfs-volume-dc1-ssd`, `seaweedfs-volume-dc1-hdd`, `seaweedfs-volume-dc2-ssd` и `seaweedfs-volume-dc2-hdd`.

COSI resources дедуплицируются между зонами: если и `dc1`, и `dc2` определяют pool `ssd` с одинаковым `diskType`, создается только один набор ресурсов BucketClass/BucketAccessClass.

`volume.pools` верхнего уровня не разрешен в MultiZone topology. Вместо этого определяйте пулы внутри каждой зоны.

Проверка

После развертывания SeaweedFS с пулами проверьте ресурсы:

```
# Проверьте, что volume server StatefulSets созданы для каждого pool
kubectl get statefulset -n tenant-example -l app.kubernetes.io/name=seaweedfs

# Проверьте BucketClasses
kubectl get bucketclass

# Проверьте BucketAccessClasses
kubectl get bucketaccessclass
```

Вы должны увидеть ресурсы BucketClass и BucketAccessClass для каждого имени pool.

Связанная документация

- [Buckets](#) – создание buckets, использующих конкретный storage pool
- [SeaweedFS Service Reference](#) – полный справочник параметров

- [SeaweedFS Multi-DC Configuration](#) – руководство по multi-DC развертыванию

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Buckets и пользователи

<https://cozystack.ru/docs/v1.6/operations/services/object-storage/buckets/>

Создание S3 buckets и управление учетными данными

Приложение Bucket создает S3 bucket через COSI и подготавливает учетные данные для каждого пользователя в виде Kubernetes Secrets.

Создание Bucket

Минимальный bucket использует BucketClass по умолчанию и не создает пользователей:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
  name: my-bucket
  namespace: tenant-example
spec: {}
```

Это создает BucketClaim в BucketClass по умолчанию (`tenant-example`). Чтобы bucket был полезен, добавьте хотя бы одного пользователя (см. Пользователи ниже).

Выбор Storage Pool

Если экземпляр SeaweedFS определяет [storage pools](#), используйте поле `storagePool`, чтобы выбрать конкретный pool:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Bucket
metadata:
  name: my-bucket
  namespace: tenant-example
spec:
  storagePool: ssd
```

Это создает BucketClaim в BucketClass `tenant-example-ssd`.

Когда `storagePool` пустой (значение по умолчанию), bucket использует BucketClass по умолчанию.

Object Locking

Чтобы создать bucket с включенной блокировкой объектов, задайте `locking: true`:

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: Bucket
```

```
metadata:
```

```
name: my-bucket
```

```
namespace: tenant-example
```

```
spec:
```

```
storagePool: ssd
```

```
locking: true
```

Это создает `BucketClaim` в `BucketClass` с суффиксом `-lock`, например `tenant-example-ssd-lock`. `BucketClasses` с включенной блокировкой используют политику удаления `Retain` и настраивают блокировку объектов в режиме `COMPLIANCE` с периодом хранения по умолчанию.

Примечание: Object locking нельзя включить или отключить после создания bucket. Если нужно изменить эту настройку, создайте новый bucket.

Пользователи

Map `users` определяет именованных S3-пользователей для bucket. Каждая запись создает ресурс `COSI BucketAccess` и соответствующий `Kubernetes Secret` с S3 учетные данные.

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: Bucket
```

```
metadata:
```

```
name: my-bucket
```

```
namespace: tenant-example
```

```
spec:
```

```
storagePool: ssd
```

```
users:
```

```
admin: {}
```

```
reader:
```

```
readonly: true
```

Это создает двух пользователей:

| Пользователь | Доступ | Используемый BucketAccessClass | Имя Secret |
|--------------|------------|--------------------------------|------------------|
| admin | read-write | tenant-example-ssd | my-bucket-admin |
| reader | read-only | tenant-example-ssd-read-only | my-bucket-reader |

Параметры пользователя

| Параметр | Тип | По умолчанию | Описание |
|----------|------|--------------|--|
| readonly | bool | false | При true создает учетные данные из BucketAccessClass с суффиксом -readonly |

Доступ к учетным данным

Каждый пользователь получает Kubernetes Secret с именем `{bucket-name}-{username}` в том же namespace. Secret содержит S3 учетные данные, созданные COSI driver:

```
kubectrl get secret my-bucket-admin -n tenant-example -o yaml
```

Secret содержит поля, необходимые для настройки S3 client: endpoint, access key, secret key. Точный набор полей зависит от реализации COSI driver.

Ротация учетных данных

Учетные данные пользователя bucket (access key и secret key) генерируются один раз при первом создании пользователя и не могут быть обновлены на месте. Чтобы ротировать учетные данные пользователя, удалите пользователя из map `users` и примените изменения, затем добавьте пользователя обратно и примените изменения еще раз:

Шаг 1: удалите пользователя, чтобы удалить существующие учетные данные

список:

```
users: {}
```

Шаг 2: добавьте пользователя обратно, чтобы создать новый набор учетных данных

список:

```
users:
```

```
admin: {}
```

Предостережение: Все приложения, использующие старые учетные данные, потеряют доступ между шагом 1 и шагом 2. После завершения шага 2 обновите приложения, указав новые учетные данные из Secret.

Логика выбора BucketClass

Имя BucketClass составляется из трех частей:

```
{seaweedfs-namespace}{-storagePool}[-lock]
```

| storagePool | locking | Используемый BucketClass |
|-------------|---------|--------------------------|
| _empty_ | false | tenant-example |
| _empty_ | true | tenant-example-lock |
| ssd | false | tenant-example-ssd |
| ssd | true | tenant-example-ssd-lock |

Аналогично составляется BucketAccessClass:

```
{seaweedfs-namespace}{-storagePool}[-readonly]
```

Полный пример

Разверните bucket в pool `ssd` с одним admin-пользователем и одним read-only пользователем:

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: Bucket
```

```
metadata:
```

```
name: media-assets
```

```
namespace: tenant-example
```

```
spec:
```

```
storagePool: ssd
```

```
locking: false
```

```
users:
```

```
app:
```

```
readonly: false
```

```
backup-reader:
```

```
readonly: true
```

После создания bucket получите учетные данные:

Read-write учетные данные для пользователя "app"

```
kubectrl get secret media-assets-app -n tenant-example \
```

| | |
|------------------------------------|--|
| <code>-o jsonpath='{.data}'</code> | <code>jq 'map_values(@base64d)'</code> |
|------------------------------------|--|

Read-only учетные данные для пользователя "backup-reader"

```
kubectrl get secret media-assets-backup-reader -n tenant-example \
```

| | |
|------------------------------------|--|
| <code>-o jsonpath='{.data}'</code> | <code>jq 'map_values(@base64d)'</code> |
|------------------------------------|--|

Связанная документация

- [Storage Pools](#) – настройка tiered storage для выбора pool
- [SeaweedFS Service Reference](#) – полный справочник параметров

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Backup Classes

<https://cozystack.ru/docs/v1.6/operations/services/backup-classes/>

The platform-managed cozy-default BackupClass: what it covers, how tenants reference it, and the admin override paths.

Cozystack ships a single platform-managed BackupClass named `cozy-default`. It is provisioned automatically when the `backupstrategy-controller` package is installed and references the system-managed bucket provisioned through the `apps.cozystack.io/Bucket` CR `cozy-backups` in the `tenant-root` namespace (the real S3 bucket name is the COSI-assigned one from `BucketClaim.status.bucketName`).

Tenants reference `cozy-default` from `BackupJob`, `Plan`, and `RestoreJob` resources — they do not supply S3 credentials, endpoints, or paths. The platform projects the system-managed credentials Secret into the tenant namespace per `BackupJob` (or, for long-lived references like Velero's `BackupStorageLocation`, into a fixed list of system namespaces on a periodic tick), and the default strategy templates encode `<namespace>/<application>` into every S3 path so two tenants with the same application name never collide.

Supported applications

Bound by `cozy-default` (work out-of-the-box)

| Application Kind | Driver | Strategy CR |
|---|---|--|
| <code>apps.cozystack.io/Postgres</code> | CloudNativePG (barman) | <code>strategy.backups.cozystack.io/CNPG cozy-default-cnpg</code> |
| <code>apps.cozystack.io/MariaDB</code> | mariadb-operator dump | <code>strategy.backups.cozystack.io/MariaDB cozy-default-mariadb</code> |
| <code>apps.cozystack.io/ClickHouse</code> | Altinity <code>clickhouse-backup</code> sidecar | <code>strategy.backups.cozystack.io/Altinity cozy-default-altinity</code> |
| <code>apps.cozystack.io/Etcd</code> | etcd-operator snapshot | <code>strategy.backups.cozystack.io/Etcd cozy-default-etcd</code> |
| <code>apps.cozystack.io/VMInstance</code> | Velero + <code>kubevirt-velero-plugin</code> | <code>strategy.backups.cozystack.io/Velero cozy-default-velero-vminstance</code> |
| <code>apps.cozystack.io/VMDisk</code> | Velero | <code>strategy.backups.cozystack.io/Velero cozy-default-velero-vmdisk</code> |

Shipped but NOT bound (admin opt-in required)

| Application Kind | Driver | Strategy CR |
|--------------------------------|---------------------------------------|--|
| apps.cozystack.io/FoundationDB | FoundationDB operator
backup_agent | strategy.backups.cozystack.io/FoundationDB cozy-default-foundationdb |

The FoundationDB strategy CR is rendered by the chart so admins can reference it from a custom BackupClass once the operator-side plumbing (mounting `cozy-backups-creds` into the `cozy-foundationdb-operator` Deployment) is wired manually. See the FoundationDB caveat below.

Endpoint format per driver

Different operators expect different endpoint shapes; the strategy templates rendered by `backupstrategy-controller` resolve one S3 endpoint (via the `backupstrategy-controller.endpoint` helper) and adapt it to each consumer's contract. For a provisioned bucket (`provisionBucket: true`, the default) the endpoint is derived from the COSI bucket's system-credentials Secret (`backupStorage.systemSecretName`) and forced to `https://` — the external S3 ingress with an ACME cert, the only endpoint the backup operators can verify. SeaweedFS's in-cluster S3 serves TLS on `:8333` behind the self-signed "SeaweedFS CA", and the Etcd Strategy S3 schema has no `caCert` field, so the in-cluster endpoint cannot be targeted directly. The `backupStorage.endpoint` chart value (a full URL like `http://seaweedfs-s3.tenant-root.svc:8333`) is the fallback, used for external S3 (`provisionBucket: false`) and for offline `helm template/pre-reconcile` renders where the Secret lookup returns nothing. The resolved endpoint is adapted per consumer:

| Driver | Strategy template field | Form |
|--------------------|--|--|
| CNPG (Postgres) | <code>barmanObjectStore.endpointURL</code> | full URL (scheme preserved) |
| Etcd | <code>destination.s3.endpoint</code> | full URL (scheme preserved) |
| MariaDB | <code>storage.s3.endpoint</code> | bare host:port (scheme stripped); <code>tls.enabled</code> derived from the scheme |
| FoundationDB | <code>blobStoreConfiguration.accountName \+ urlParameters.secure_connection</code> | bare host:port + derived secure flag |
| Velero | <code>BackupStorageLocation.spec.config.s3Url</code> | full URL (scheme preserved) |
| ClickHouse sidecar | <code>S3_ENDPOINT</code> env | bare host:port (from projected Secret) |

The projected `cozy-backups-creds.endpoint` key is stripped of scheme so chart-emitted sidecars (ClickHouse) consume it directly. Drivers that need the full URL receive the resolved endpoint described above — derived from the COSI system Secret (forced `https://`) for a provisioned bucket, or the `backupStorage.endpoint` fallback for external S3.

VM-driven (Velero) backups land in the same `cozy-backups` bucket under the `velero/` prefix. A `BackupStorageLocation` named `cozy-default` is shipped by the `backupstrategy-controller` chart (`packages/system/backupstrategy-controller/templates/velero-bsl.yaml`) so `endpoint/bucket/region` come from the same `backupStorage` values block used by Strategy CRs and the projector.

FoundationDB caveat

The strategy CR `cozy-default-foundationdb` is shipped, but it is not bound by `cozy-default` yet. Restore runs `fdbrestore` from inside the `cozy-foundationdb-operator` Deployment, which does not yet mount `cozy-backups-creds`. Until the operator deployment is updated to mount the projected Secret, FDB platform-default restore silently fails — admins who need it today should keep using a per-app `Bucket` plus a custom `BackupClass`, or wire the credentials file into the operator deployment themselves.

Cleanup gotcha (zombie backup_agent). Unlike CNPG/MariaDB/Altinity (one-shot operator-side Backup CRs), the FoundationDB driver creates an `apps.foundationdb.org/FoundationDBBackup` CR that drives a long-lived `backup_agent` Deployment streaming continuously to S3. Deleting a Cozystack `Backup` (e.g. via retention sweeping) does NOT stop that Deployment — the agent keeps writing until the next `BackupJob`'s `stopOtherFoundationDBBackups` call swaps it out, until an admin invokes `examples/backups/foundationdb/cleanup.sh`, or until the operator-side CR is deleted by hand. If a tenant deletes their last Cozystack `Backup` and never submits another `BackupJob`, the agent pods will continue running indefinitely and accumulate S3 PUTs. This is intentional today (the driver has no RBAC verb to stop the operator-side CR on Cozystack-Backup deletion) but admins should be aware of it.

ClickHouse: opt-in to the system bucket

The `clickhouse-backup` sidecar runs inside the ClickHouse Pod itself, so the Helm chart is what wires its S3 credentials. Existing tenants on the legacy `backup.s3*` values continue to work unchanged. To switch a release onto the platform bucket, set:

```
backup:
enabled: true
useSystemBucket: true
```

When `useSystemBucket: true`:

- The chart-emitted `<release>-backup-s3` Secret is no longer rendered.
- The sidecar consumes `cozy-backups-creds` (projected by the platform).
- `S3_PATH` is set to `<namespace>/<release>` so two tenants with the same ClickHouse release name never share a prefix.

`s3Region`, `s3Bucket`, `endpoint`, `s3AccessKey`, `s3SecretKey`, and `s3CredentialsSecret` are ignored in this mode.

Inspecting the defaults

```
kubectl get backupclasses
kubectl get backupclass cozy-default -o yaml
kubectl -n tenant-root get bucket cozy-backups
kubectl -n tenant-root get secret bucket-cozy-backups-system-credentials
kubectl -n cozy-velero get backupstoragelocation cozy-default
```

The bucket lives in `tenant-root` and is provisioned through the `apps.cozystack.io/Bucket` CR. The system-managed credentials Secret never leaves that namespace. The `backupstrategy-controller` projects a copy under the name `cozy-backups-creds` into a tenant namespace right before each BackupJob or RestoreJob runs, and refreshes the same Secret in `cozy-velero` (and any other namespace listed in `backupStorage.systemNamespaces`) on a 1-minute tick. The projected Secret carries multiple key formats so each driver finds what it needs in one place:

| Key | Consumer |
|--|--------------------------------------|
| <code>AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY</code> | CNPG, MariaDB, Etcd |
| <code>accessKey / secretKey</code> (plus <code>bucketName</code> , <code>endpoint</code> , <code>region</code>) | ClickHouse sidecar |
| <code>cloud</code> | Velero (AWS credentials file format) |
| <code>blob_credentials.json</code> | FoundationDB backup_agent |

Bootstrap window

On a fresh-cluster install, the Velero `BackupStorageLocation cozy-default` is rendered before the credentials projector has had a chance to copy `cozy-backups-creds` into `cozy-velero`. The BSL reports `Unavailable` until the projector's first synchronous round completes (which runs as soon as the `backupstrategy-controller` acquires leadership — in practice moments after the Pod becomes Ready, typically tens of seconds after `helm install` returns, not minutes). Velero rejects new Backup AND Restore requests against `storageLocation: cozy-default` during that window. Plan VM backup automation accordingly, or wait for the BSL to become ready before submitting backups: `kubectl -n cozy-velero wait backupstoragelocation cozy-default --for=jsonpath='{.status.phase}'=Available --timeout=5m`.

Note on controller restarts. The BSL flickers `Unavailable` on every `backupstrategy-controller` pod restart while the projector replays its first synchronous round. The window is short (single-digit seconds) but operators who alert on BSL availability should suppress alerts during the controller's `kube_pod_container_status_restarts_total{container=backupstrategy-controller}` events or use a longer evaluation window than the projector tick (60s).

Cozy-default Bucket bootstrap

`cozy-default` ships an `apps.cozystack.io/Bucket cozy-backups` CR in `tenant-root`, which the bucket-application chart turns into a `BucketClaim`; the COSI driver then assigns the real S3 bucket name and writes it to the `BucketClaim's .status.bucketName`. The strategy templates and the Velero BSL all read

that real bucket name (Helm lookup against the BucketClaim). On a fresh install the BucketClaim takes a short reconcile cycle to populate its status — until it does, the strategy templates render empty and only the Bucket CR + BackupClass are present in the cluster. The HelmRelease re-reconciles on its interval (5 minutes by default — set by the cozystack operator's `helmrelease-interval` flag, not a Flux default), at which point the populated BucketClaim status causes the missing strategy templates to materialise.

If you need the BackupClass functional immediately (e.g. an e2e), trigger a Flux reconcile (`flux reconcile helmrelease backupstrategy-controller -n cozy-backup-controller`) once you see `kubectl get bucketclaim -n tenant-root bucket-cozy-backups -o jsonpath='{.status.bucketName}'` non-empty.

Observability

The credentials projector emits two Prometheus counters labelled by namespace (and reason for failures):

- `cozystack_backup_credentials_projection_successes_total`
- `cozystack_backup_credentials_projection_failures_total`

Alert on `rate(cozystack_backup_credentials_projection_failures_total[5m]) > 0` or `absent_over_time(cozystack_backup_credentials_projection_successes_total[10m])` to catch a stale BSL credential or a malformed source Secret without log scraping.

Admin overrides for `cozy-default`

`cozy-default` is rendered by the `backupstrategy-controller` chart and owned by Flux's `helm-controller`. ****Direct**`kubectl edit backupclass cozy-default` is overwritten on the next `helm reconcile` ****** — the same applies to its companion `strategy.backups.cozystack.io/*` CRs (`cozy-default-cnpg`, `cozy-default-etcd`, `cozy-default-mariadb`, `cozy-default-altinity`, `cozy-default-foundationdb`, the two `cozy-default-velero-*`). The supported override path is the `backupStorage` block on the ****platform component** of the `cozystack.cozystack-platform` Package CR:

`apiVersion: cozystack.io/v1alpha1`

`kind: Package`

`metadata:`

`name: cozystack.cozystack-platform`

`spec:`

`components:`

`platform:`

`values:`

`backupStorage:`

`provisionBucket: true` # default; set false for external S3

`bucketName: cozy-backups` # `apps.cozystack.io/Bucket` release name

`endpoint: http://seaweedfs-s3.tenant-root.svc.cozy.local:8333`

region: us-east-1

forcePathStyle: true

systemSecretName: bucket-cozy-backups-system-credentials

systemNamespaces:

- cozy-velero

The platform chart forwards this block into the child Package `cozystack.backupstrategy-controller` as component values, from where the `cozystack` operator merges it into the `backupstrategy-controller` HelmRelease over the chart defaults. Two paths that look plausible do not work:

`spec.components.backupstrategy-controller` on the `cozystack.cozystack-platform` Package is silently ignored (the only component under that PackageSource is `platform`), and patching the child Package `cozystack.backupstrategy-controller` directly is reverted whenever the platform helm-reconcile re-renders it.

| Knob | Effect |
|---------------------------------|--|
| <code>provisionBucket</code> | Toggle creation of the in-cluster <code>apps.cozystack.io/Bucket</code> CR. Set <code>false</code> for external S3 (see Disabling the platform-managed bucket). |
| <code>bucketName</code> | Two modes. With <code>provisionBucket: true</code> (default): K8s name of the Bucket CR + lookup key for the COSI BucketClaim — the actual S3 bucket name is the COSI-assigned UUID, surfaced through <code>BucketClaim.status.bucketName</code> . With <code>provisionBucket: false</code> : taken verbatim as the real S3 bucket name and baked into every strategy CR + the Velero BSL. |
| <code>namespace</code> | Namespace the Bucket CR (and its system-credentials Secret) lives in — <code>tenant-root</code> by default. Must be a tenant namespace (<code>tenant-*</code>): the Bucket chart's RBAC helper fails the Helm render for any other prefix. |
| <code>bucketNameOverride</code> | Escape hatch for offline <code>helm template</code> renders — bypasses the live-cluster BucketClaim lookup. Leave empty in production. |

| | |
|-------------------------------|---|
| <code>endpoint</code> | Fallback S3 endpoint. For a provisioned bucket the strategy CRs + Velero BSL derive the endpoint from the COSI system Secret (external ACME ingress, forced <code>https://</code>) instead; this value is used only for external S3 (<code>provisionBucket: false</code>) and offline renders. For external S3, switching it to <code>https://</code> enables TLS in the MariaDB/FoundationDB strategies — ensure the CA bundle is reachable to the relevant operator/driver Pods first. |
| <code>region</code> | Re-projected into <code>cozy-backups-creds</code> on the next reconcile. Pod-restart required for chart-emitted clients consuming the region via env (ClickHouse sidecar today). |
| <code>forcePathStyle</code> | Path-style addressing; SeaweedFS S3 requires it, AWS S3 typically doesn't. |
| <code>systemSecretName</code> | Name of the human-friendly Secret produced by the Bucket app (or pre-created manually for external S3). The projector also accepts the raw COSI Secret format. |
| <code>systemNamespaces</code> | Namespaces where the controller eagerly projects <code>cozy-backups-creds</code> (Velero BSL, FDB operator). Tenants are projected lazily during BackupJob reconcile. |

When the override needs to go beyond storage coordinates — different retention, different driver->Kind binding, multi-region split — create a sibling BackupClass with a unique name (anything but `cozy-default`). Sibling BackupClasses live outside the chart, are admin-owned, and Flux will not touch them. Tenants opt in by setting `backupClassName: <your-class>` on their BackupJobs.

Tuning via a custom BackupClass

The defaults aim at a reasonable middle (30-day retention, gzip compression where applicable). To override for a specific tenant or workload, create your own BackupClass pointing at the same strategy CRs but with tweaked `parameters`, or a fresh strategy CR. Common knobs:

- CNPG strategy : `barmanObjectStore.retentionPolicy`, `data.compression`, `wal.compression`.
- MariaDB strategy : `compression`, `maxRetention`, `databases[]`.
- Altinity strategy : tune the `clickhouse-backup` sidecar via `backup.*` values on the ClickHouse release; the strategy Pod is a thin HTTP client.
- FoundationDB strategy : `snapshotPeriodSeconds`, `agentCount`, `urlParameters[]`.
- Velero strategy (VMInstance / VMDisk) : `ttl`, `includedResources[]`, `excludedResources[]`.
- Etcd strategy : today the strategy is path-only; combine with `Plan.spec.retentionPolicy` for trim cadence.

The system-managed credentials Secret is the only way for in-cluster strategies to reach `cozy-backups`. Do not embed access keys in `BackupClass.parameters` — the security model relies on Secret references, and `parameters` end up in `Backup.status.underlyingResources`, which tenants can read.

Disabling the platform-managed bucket

If a deployment runs against an external S3 (no SeaweedFS), set `backupStorage.provisionBucket: false` through the same platform Package path as above (`spec.components.platform.values.backupStorage`) and create the source credentials Secret in `tenant-root` manually (flat-key format: `accessKey / secretKey / endpoint / bucketName`; or the raw COSI BucketInfo JSON). In the same `backupStorage` block, update `endpoint`, `region`, `**andbucketName**`: with `provisionBucket: false` the strategies and the Velero BSL take `bucketName` verbatim as the real S3 bucket name (no COSI lookup), so it must name the actual bucket on the external S3 — the `bucketName` key inside the Secret alone is not enough. The Velero `BackupStorageLocation` picks the same values up automatically (the chart renders it from the same `backupStorage` block), so no separate BSL configuration is needed. Note that disabling the cluster-default BSL itself (the chart's `velero.bslEnabled` value) is not carried by the `backupStorage` override path — the platform Package forwards only the `backupStorage` block.

Upgrade notes from chart-managed backups

****Postgresbackup.enabled: true with placeholder credentials no longer renders barmanObjectStore on upgrade.****

The pre-v1.5 defaults for `backup.s3AccessKey / backup.s3SecretKey` in `packages/apps/postgres/values.yaml` were the literal `"<your-access-key>" / "<your-secret-key>"` placeholders, so the Postgres chart still rendered `spec.backup.barmanObjectStore` on the `cnpg.io/Cluster` (with junk credentials, `archive_command` failing at runtime). Starting with v1.5 those defaults are empty strings and the chart NO LONGER renders the backup block at all when the placeholders are unmodified. Tenants on the legacy chart-managed flow who relied on those placeholders see their `barmanObjectStore` disappear from the live Cluster on `helm upgrade`. Action — pick one:

* Move to the platform flow (recommended). Set `backup.useSystemBucket: true`; the chart leaves `barmanObjectStore` unset and the CNPG backup driver SSA-patches it onto the live Cluster at first BackupJob time. No tenant-side keys required.

* Stay on the legacy chart-managed flow. Supply real `backup.s3AccessKey / backup.s3SecretKey` (or a pre-existing `backup.s3CredentialsSecret.name`); the chart renders `barmanObjectStore` exactly as before.

The same `useSystemBucket` opt-in applies to ClickHouse — see ClickHouse: opt-in to the system bucket. When `useSystemBucket: true` is set on ClickHouse, the legacy `<release>-backup` CronJob, credential Secret, and backup script are no longer rendered (they are mutually exclusive with the platform flow); migrate scheduled backups to a `backups.cozystack.io/Plan` against `cozy-default`.

Tenant workflow

Tenants only ever see the BackupClass name. Typical apply:

apiVersion: backups.cozystack.io/v1alpha1

kind: BackupJob

metadata:

name: ad-hoc

namespace: tenant-acme

spec:

backupClassName: cozy-default

applicationRef:

apiGroup: apps.cozystack.io

kind: Postgres

name: orders-db

See also

- [Application Backup and Recovery](#) — the tenant guide for database backups (BackupJob, Plan, RestoreJob).
- [Backup and Recovery \(VMs\)](#) — the tenant guide for VMInstance / VMDisk backups.
- [Platform Package Reference](#) — where the backupStorage override lives among the other platform values.

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0 (96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

Справочник сервиса BootBox

<https://cozystack.ru/docs/v1.6/operations/services/bootbox/>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------|--|----------|----------|
| whitelistHTTP | Защищает HTTP, включая whitelist клиентских сетей. | bool | true |
| whitelist | Список клиентских сетей. | []string | [] |
| machines | Конфигурация экземпляров физических машин. | []object | [] |
| machines[i].hostname | Hostname. | string | "" |
| machines[i].arch | Архитектура. | string | "" |
| machines[i].ip | Конфигурация IP-адреса. | object | {} |
| machines[i].ip.address | IP-адрес. | string | "" |
| machines[i].ip.gateway | IP gateway. | string | "" |
| machines[i].ip.netmask | Netmask. | string | "" |
| machines[i].leaseTime | Lease time. | int | 0 |
| machines[i].mac | MAC-адреса. | []string | [] |
| machines[i].nameServers | Name servers. | []string | [] |
| machines[i].timeServers | Time servers. | []string | [] |
| machines[i].uefi | UEFI. | bool | false |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник сервиса Etcd

<https://cozystack.ru/docs/v1.6/operations/services/etcd/>

This chart renders an `EtcdCluster` (`etcd-operator.cozystack.io/v1alpha2`) managed by the `cozystack etcd-operator`, plus the `cert-manager` CA chain and a `Kamaji DataStore`. TLS uses `secretRef` mode: this chart's own `cert-manager Certificate` resources issue the server, operator-client and peer certs from the `etcd-issuer / etcd-peer-issuer` CA Issuers, and the `EtcdCluster` only `_references_` the resulting `Secrets` via `serverSecretRef / operatorClientSecretRef / peer.secretRef` (the operator itself mints nothing). This is deliberate: `spec.tls` is immutable in `v1alpha2` and `etcd-migrate` adopts legacy clusters into exactly this `secretRef` shape, so the chart must match it or every post-adoption reconcile is rejected. `etcd` runs `cert-only` (`--client-cert-auth`, no password auth); consumers authenticate by presenting a `commonName=root` client cert (`etcd-client-tls`) signed by the same CA.

Backups

Backups are driven by the `Cozystack BackupClass` flow: a `BackupJob / RestoreJob` against the `Etcd` strategy (`strategy.backups.cozystack.io/v1alpha1`). See `examples/backups/etcd/` for the end-to-end demo and `internal/backupcontroller/etcdstrategy_controller.go` for the driver. The driver materialises one `EtcdSnapshot` (`etcd-operator.cozystack.io/v1alpha2`) per `BackupJob`; restore is in-place (the driver suspends this chart's `HelmRelease`, deletes the live `EtcdCluster`, and re-creates it with `spec.bootstrap.restore.source.s3` populated from the `Backup` artefact's coordinates).

The legacy `backup.*` values block and the `EtcdBackupSchedule` CRD no longer exist under the `v1alpha2` operator — scheduled backups are expressed as a `BackupClass / Plan` instead.

Parameters

Common parameters

| Name | Description | Type | Value |
|--------------|--|----------|--------|
| version | etcd version (X.Y.Z) to run, set on the <code>EtcdCluster's spec.version</code> ; the operator pulls etcd from its own configured image. | string | 3.6.11 |
| size | Persistent Volume size. | quantity | 4Gi |
| storageClass | StorageClass used to store the data. | string | "" |
| replicas | Number of etcd replicas. | int | 3 |

| | | | |
|------------------|----------------------------------|----------|-------|
| resources | Resource configuration for etcd. | object | {} |
| resources.cpu | Number of CPU cores allocated. | quantity | 1000m |
| resources.memory | Amount of memory allocated. | quantity | 512Mi |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник Ingress-NGINX Controller

<https://cozystack.ru/docs/v1.6/operations/services/ingress/>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-----------------|---|----------|----------|
| replicas | Количество реплик ingress-nginx. | int | 2 |
| whitelist | Список клиентских сетей. | []string | [] |
| cloudflareProxy | Восстановление исходных IP посетителей, когда включён Cloudflare proxied. | bool | false |

| | | | |
|------------------|---|----------|-------|
| proxyProtocol | Включить PROXY-protocol (use-proxy-protocol) на tenant Ingress. Upstream L4 load balancer перед ingress-nginx ОБЯЗАТЕЛЬНО должен уже добавлять заголовки PROXY-protocol v1, иначе весь трафик к этому ingress ломается — включая внутрикластерный и прямой (hairpin) доступ к его публичным hostnames через Service. Взаимоисключаяюще с cloudflareProxy. Для host (внешнего) ingress вместо этого используйте платформенный переключатель publishing.proxyProtocol; per-app значение там отклоняется, так как оно обошло бы обработку hairpin в ouroboros и защиту от опасного отключения. | bool | false |
| resources | Явная конфигурация CPU и памяти для каждой реплики ingress-nginx. Если не задано, применяется пресет, указанный в resourcesPreset . | object | {} |
| resources.cpu | CPU, доступный каждой реплике. | quantity | "" |
| resources.memory | Память (RAM), доступная каждой реплике. | quantity | "" |

| | | | |
|-----------------|--|--------|----------|
| resourcesPreset | Пресет размера по умолчанию, используемый, когда resources не задан. | string | t1.micro |
|-----------------|--|--------|----------|

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник сервиса SeaweedFS

<https://cozystack.ru/docs/v1.6/operations/services/seaweedfs/>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------|--|--------|----------|
| host | Hostname для внешнего доступа к SeaweedFS (по умолчанию subdomain s3 для host tenant). | string | "" |
| topology | Топология кластера SeaweedFS. | string | Simple |
| replicationFactor | Replication factor: количество реплик для каждого volume в кластере SeaweedFS. | int | 2 |

Конфигурация компонентов SeaweedFS

| Имя | Описание | Тип | Значение |
|-----------------|--|----------|----------|
| db | Конфигурация database. | object | {} |
| db.replicas | Количество реплик database. | int | 2 |
| db.size | Размер Persistent Volume. | quantity | 10Gi |
| db.storageClass | StorageClass, используемый для хранения данных. | string | "" |
| db.resources | Явная конфигурация CPU и memory. Если не задано, применяется preset из resourcesPres et. | object | {} |

| | | | |
|--------------------------------------|---|-----------------------|-----------------------|
| <code>db.resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>db.resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>db.resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |
| <code>master</code> | Конфигурация master service. | <code>object</code> | <code>{}</code> |
| <code>master.replicas</code> | Количество реплик master. | <code>int</code> | <code>3</code> |
| <code>master.resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>master.resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>master.resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>master.resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |
| <code>filer</code> | Конфигурация filer service. | <code>object</code> | <code>{}</code> |
| <code>filer.replicas</code> | Количество реплик filer. | <code>int</code> | <code>2</code> |
| <code>filer.resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>filer.resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>filer.resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |

| | | | |
|--------------------------------------|--|-----------------------|-----------------------|
| <code>filer.resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |
| <code>filer.grpcHost</code> | Hostname для публикации или внешнего доступа к <code>filer service</code> . | <code>string</code> | <code>""</code> |
| <code>filer.grpcPort</code> | Port для внешнего доступа к <code>filer service</code> . | <code>int</code> | <code>443</code> |
| <code>filer.whitelist</code> | Список IP-адресов или CIDR ranges, которым разрешен доступ к <code>filer service</code> . | <code>[]string</code> | <code>[]</code> |
| <code>volume</code> | Конфигурация <code>volume service</code> . | <code>object</code> | <code>{}</code> |
| <code>volume.replicas</code> | Количество <code>volume replicas</code> . | <code>int</code> | <code>2</code> |
| <code>volume.size</code> | Размер Persistent Volume. | <code>quantity</code> | <code>10Gi</code> |
| <code>volume.storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>""</code> |
| <code>volume.diskType</code> | SeaweedFS disk type tag для <code>volume servers</code> по умолчанию, например <code>"hdd"</code> , <code>"ssd"</code> . | <code>string</code> | <code>""</code> |
| <code>volume.resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>volume.resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>volume.resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>volume.resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |

| | | | |
|--|---|--------------------------------|-----------------|
| <code>volume.zones</code> | Map zones для MultiZone topology. У каждой zone может быть собственное количество replicas и size. | <code>map[string]object</code> | <code>{}</code> |
| <code>volume.zones[name].replicas</code> | Количество replicas в zone. | <code>int</code> | <code>0</code> |
| <code>volume.zones[name].size</code> | Размер хранилища zone. | <code>quantity</code> | <code>""</code> |
| <code>volume.zones[name].dataCenter</code> | Имя data center SeaweedFS для этой zone. По умолчанию имя zone. | <code>string</code> | <code>""</code> |
| <code>volume.zones[name].nodeSelector</code> | YAML nodeSelector для этой zone (по умолчанию: <code>topology.kubernetes.io/zone:</code>). | <code>string</code> | <code>""</code> |
| <code>volume.zones[name].storageClass</code> | StorageClass, используемый для хранения данных zone. По умолчанию <code>volume.storageClass</code> . | <code>string</code> | <code>""</code> |
| <code>volume.zones[name].pools</code> | Map storage pools для этой zone. Каждый pool создает отдельный Volume StatefulSet на zone. | <code>map[string]object</code> | <code>{}</code> |
| <code>volume.zones[name].pools[name].diskType</code> | SeaweedFS disk type tag, например <code>"ssd"</code> , <code>"hdd"</code> , <code>"nvme"</code> . | <code>string</code> | <code>""</code> |
| <code>volume.zones[name].pools[name].replicas</code> | Количество volume replicas. По умолчанию <code>volume.replicas (Simple)</code> или <code>zone.replicas/volume.replicas (MultiZone)</code> . | <code>int</code> | <code>0</code> |

| | | | |
|--|---|--------------------------------|-----------------|
| <code>volume.zones[name].pools[name].size</code> | Размер Persistent Volume. По умолчанию <code>volume.size (Simple)</code> или <code>zone.size/volume.size (MultiZone)</code> . | <code>quantity</code> | <code>""</code> |
| <code>volume.zones[name].pools[name].storageClass</code> | Kubernetes StorageClass для pool. По умолчанию <code>volume.storageClass</code> . | <code>string</code> | <code>""</code> |
| <code>volume.zones[name].pools[name].resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>volume.zones[name].pools[name].resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>volume.zones[name].pools[name].resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>volume.zones[name].pools[name].resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code> . | <code>string</code> | <code>{}</code> |
| <code>volume.pools</code> | Map storage pools. Каждый pool создает отдельный Volume StatefulSet со своим disk type. | <code>map[string]object</code> | <code>{}</code> |
| <code>volume.pools[name].diskType</code> | SeaweedFS disk type tag, например <code>"ssd"</code> , <code>"hdd"</code> , <code>"nvme"</code> . | <code>string</code> | <code>""</code> |
| <code>volume.pools[name].replicas</code> | Количество volume replicas. По умолчанию <code>volume.replicas (Simple)</code> или <code>zone.replicas/volume.replicas (MultiZone)</code> . | <code>int</code> | <code>0</code> |

| | | | |
|--|---|-----------------------|-----------------------|
| <code>volume.pools[name].size</code> | Размер Persistent Volume. По умолчанию <code>volume.size (Simple)</code> или <code>zone.size/volume.size (MultiZone)</code> . | <code>quantity</code> | <code>""</code> |
| <code>volume.pools[name].storageClass</code> | Kubernetes StorageClass для pool. По умолчанию <code>volume.storageClass</code> . | <code>string</code> | <code>""</code> |
| <code>volume.pools[name].resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>volume.pools[name].resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>volume.pools[name].resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>volume.pools[name].resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. По умолчанию <code>volume.resourcesPreset</code> . | <code>string</code> | <code>{}</code> |
| <code>s3</code> | Конфигурация S3 service. | <code>object</code> | <code>{}</code> |
| <code>s3.replicas</code> | Количество реплик S3. | <code>int</code> | <code>2</code> |
| <code>s3.resources</code> | Явная конфигурация CPU и memory. Если не задано, применяется preset из <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>s3.resources.cpu</code> | Количество выделенных CPU cores. | <code>quantity</code> | <code>""</code> |
| <code>s3.resources.memory</code> | Объем выделенной memory. | <code>quantity</code> | <code>""</code> |
| <code>s3.resourcesPreset</code> | Sizing preset по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник Monitoring Hub

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/>

Архитектура потоков данных

flowchart TD

A[VMAgent] --> B[VMCluster]

B --> C[Grafana]

B --> D[Alerta]

E[Fluent Bit] --> F[VLogs]

Описание компонентов

- VMAgent : легкий agent, который собирает metrics из разных источников и отправляет их в VictoriaMetrics.
- VMCluster : кластер VictoriaMetrics, который хранит и обрабатывает time-series data для эффективных queries.
- Grafana : open-source платформа для monitoring и observability с настраиваемыми dashboards.
- Alerta : alerting system, которая обрабатывает и управляет alerts из monitoring systems.
- Fluent Bit : быстрый и легкий log processor и forwarder.
- VLogs : VictoriaLogs, высокопроизводительная система управления logs для хранения и запросов.

Архитектура визуализации

graph TD

A[VictoriaMetrics
VMCluster] --> B[Grafana]

C[VLogs] --> B

D[External Prometheus] --> B

E[Custom Application Metrics] --> B

B --> F[Pre-built Dashboards
Cluster Overview, Node Metrics, etc.]

B --> G[Custom Dashboards
Time Series, Stat, Table, etc.]

F --> H[Visualization
CPU Usage, Memory, Network]

G --> H

Описание компонентов визуализации

- VictoriaMetrics VMCluster : основное хранилище metrics и query engine, предоставляющий данные Grafana.
- VLogs : система VictoriaLogs для интеграции log data в visualizations.
- External Prometheus : дополнительные sources metrics, которые можно интегрировать.
- Custom Application Metrics : пользовательские metrics из приложений.
- Grafana : платформа визуализации, которая отображает dashboards.
- Pre-built Dashboards : стандартные dashboards для типовых monitoring views.
- Custom Dashboards : созданные пользователями dashboards с разными типами panels.
- Visualization : итоговый вывод с metrics вроде CPU, memory и network usage.

Архитектура мониторинга

graph TD

A[Kubernetes Cluster] --> B[VMAgent]

C[Applications] --> B

D[Infrastructure] --> B

B --> E[VictoriaMetrics VMCluster]

E --> F[Grafana]

E --> G[Alerta]

F --> H[Dashboards & Visualizations]

G --> I[Alerts & Notifications]

Описание компонентов архитектуры мониторинга

- Kubernetes Cluster : основная платформа, где работают workloads и доступны metrics endpoints.
- Applications : пользовательские приложения, предоставляющие custom metrics.
- Infrastructure : базовое оборудование и системные metrics.
- VMAgent : собирает metrics из разных sources и отправляет их в VictoriaMetrics.
- VictoriaMetrics VMCluster : хранит и обрабатывает time-series metrics data.
- Grafana : предоставляет visualization и dashboarding.
- Alerta : управляет alerting и notifications.
- Dashboards & Visualizations: user interfaces для monitoring data.
- Alerts & Notifications: система уведомления operators о проблемах.

Поток alerting

flowchart TD

A[Metrics Collection
VMAgent] --> B[VictoriaMetrics
VMCluster]

B --> C[Alert Rules
Evaluation]

C --> D{Condition Met?}

D --> |Yes| E[Generate Alert]

D --> |No| A

E --> F[Alerta
Alert Manager]

F --> G[Grouping &
Deduplication]

G --> H[Routing &
Notification]

H --> I[Email/SMS/Slack/etc.]

Описание компонентов потока alerting

- Metrics Collection : сбор metrics из sources через VMAgent.
- VictoriaMetrics VMCluster : хранение и querying metrics data.
- Alert Rules Evaluation : проверка metrics по заранее заданным thresholds.
- Generate Alert : создание alert instances при выполнении условий.
- Alerta Alert Manager : обработка и управление alerts.
- Grouping & Deduplication: организация alerts для устранения duplicates.
- Routing & Notification: направление alerts в нужные channels.
- Email/SMS/Slack/etc. : конечные способы доставки notifications.

Архитектура логирования

graph TD

A[Application Logs] --> B[Fluent Bit]

C[Kubernetes
Container Logs] --> B

D[System Logs] --> B

B --> E[VictoriaLogs
VLogs]

E --> F[Grafana
Log Panels]

F --> G[Log Visualization
& Search]

E --> H[Log Queries
& Analysis]

Описание компонентов архитектуры логирования

- Application Logs : logs, генерируемые пользовательскими приложениями.
- Kubernetes Container Logs : logs контейнеров, работающих в Kubernetes.
- System Logs : infrastructure и system-level logs.

-
- Fluent Bit : легкий log processor, который собирает и пересылает logs.
 - VictoriaLogs VLogs : высокопроизводительная система хранения и querying logs.
 - Grafana Log Panels : интеграция для визуализации logs в Grafana dashboards.
 - Log Visualization & Search: interfaces для просмотра и поиска log data.
 - Log Queries & Analysis: tools для querying и анализа log information.

•

[Параметры мониторинга](#)

Настройка и управление параметрами мониторинга в Cozystack.

[Настройка мониторинга](#)

Руководство по установке и настройке мониторинга в Cozystack

[Monitoring Dashboards](#)

Как визуализировать метрики и создавать custom dashboards в Grafana для мониторинга кластеров и приложений Cozystack.

[Логи мониторинга](#)

Как собирать, хранить, искать и анализировать логи в Cozystack с помощью Fluent Bit и VictoriaLogs для полноценной наблюдаемости.

[Сбор custom metrics](#)

Подключение собственных Prometheus exporters к tenant monitoring stack Cozystack с помощью VMServiceScrape и VMPodScrape.

[Alerting в мониторинге](#)

Настройка и управление alerts в системе мониторинга Cozystack с помощью Alerta и Alertmanager.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Alerting в мониторинге

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/alerting/>

Настройка и управление alerts в системе мониторинга Cozystack с помощью Alerta и Alertmanager.

Обзор

Система оповещения в Cozystack объединяет Prometheus, Alertmanager и Alerta, обеспечивая полноценный мониторинг и уведомления. Alerts создаются на основе метрик, собранных VMAgent и сохраненных в VMCluster, затем Alertmanager группирует их и устраняет дубликаты, после чего Alerta управляет отправкой уведомлений по различным каналам, например Telegram и Slack.

Поток alerting

sequenceDiagram

participant P as Prometheus

participant AM as Alertmanager

participant A as Alerta

participant T as Telegram

participant S as Slack

P->>AM: Send Alert

AM->>A: Forward Alert

A->>T: Send Notification

A->>S: Send Notification

Настройка alerts в Alerta

Alerta - это система оповещения, интегрированная в стек мониторинга Cozystack. Она обрабатывает алерты из разных источников и отправляет оповещения через несколько каналов.

Alert rules

Алерты создаются на основе Prometheus правил, определенных в конфигурации мониторинга. Пользовательские правила оповещения можно настроить, изменив ресурсы PrometheusRule в namespace вашего tenant.

Чтобы создать свое правило оповещения, создайте манифесты PrometheusRule с выражениями, которые срабатывают при выполнении заданных условий. Каждое правило включает:

- expr : PromQL выражение для вычисления.

-
- for : время, в течение которого условие должно непрерывно выполняться перед срабатыванием алерта.
 - labels : metadata, например severity.
 - annotations : дополнительная информация, включаемая в уведомления.

Пример custom alert rule:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: custom-alerts
  namespace: tenant-name
spec:
  groups:
    - name: custom.rules
  rules:
    - alert: HighCPUUsage
      expr: (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))) * 100 > 80
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Обнаружена высокая загрузка CPU"
        description: "Загрузка CPU выше 80% более 5 минут"
```

Уровни severity

Alerta поддерживает следующие уровни критичности:

- informational : низкоприоритетная информация
- warning : потенциальные проблемы, требующие внимания
- critical : срочные проблемы, требующие немедленных действий
- major : значимые проблемы, влияющие на эксплуатацию
- minor : небольшие проблемы

В конфигурации Alerta можно задать, какие уровни критичности вызывают оповещение.

Интеграции

Интеграция Telegram

Чтобы включить Telegram оповещения, задайте следующее в настройках monitoring:

alerta:

alerts:

telegram:

token: "your-telegram-bot-token"

chatID: "chat-id-1,chat-id-2"

disabledSeverity:

- informational

Интеграция Slack

Для Slack оповещений:

alerta:

alerts:

slack:

url: "https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK"

disabledSeverity:

- informational

- warning

Интеграция Email

Чтобы включить email оповещения:

alerta:

alerts:

email:

smtpHost: "smtp.example.com"

smtpPort: 587

smtpUser: "alerts@example.com"

smtpPassword: "your-password"

fromAddress: "alerts@example.com"

toAddress: "team@example.com"

disabledSeverity:

- informational

Интеграция PagerDuty

Для PagerDuty оповещений:

alerta:

alerts:

pagerduty:

serviceKey: "YOUR_PAGERDUTY_INTEGRATION_KEY"

disabledSeverity:

- informational
- warning

Подробные параметры конфигурации см. в справочнике [Monitoring Hub](#).

Примеры alerts

Ниже приведены распространённые примеры алертов для мониторинга системы:

Alert по CPU usage

- alert: HighCPUUsage

expr: 100 - (avg by(instance) (irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80

for: 5m

labels:

severity: warning

annotations:

summary: "Высокая загрузка CPU на {{ \$labels.instance }}"

description: "Загрузка CPU составляет {{ \$value }}% более 5 минут"

Alert по memory usage

- alert: HighMemoryUsage

expr: (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100 > 90

for: 5m

labels:

severity: critical

annotations:

summary: "Высокое использование памяти на {{ \$labels.instance }}"

description: "Использование памяти составляет {{ \$value }}% более 5 минут"

Alert no disk space

- alert: LowDiskSpace
expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 10
for: 5m
labels:
severity: critical
annotations:
summary: "Мало свободного места на диске на {{ \$labels.instance }}"
description: "Доступное место на диске составляет {{ \$value }}% более 5 минут"

WorkloadNotOperational Alert

- alert: WorkloadNotOperational
expr: up{job="workload-monitor"} == 0
for: 1m
labels:
severity: critical
annotations:
summary: "Workload {{ \$labels.workload }} не работает"
description: "Workload monitor сообщает, что workload недоступен"

Alert по недоступному network interface

- alert: NetworkInterfaceDown
expr: node_network_up{device!~"lo"} == 0
for: 2m
labels:
severity: critical
annotations:
summary: "Network interface {{ \$labels.device }} недоступен на {{ \$labels.instance }}"
description: "Network interface недоступен более 2 минут"

Alert no crash Kubernetes pod

- alert: KubernetesPodCrashLooping
expr: rate(kube_pod_container_status_restarts_total[10m]) > 0.5
for: 5m

labels:

severity: warning

annotations:

summary: "Pod {{ \$labels.pod }} находится в crash loop"

description: "Pod перезапускается чаще одного раза за 2 минуты"

Alert по высокой network latency

- alert: HighNetworkLatency

expr: node_network_receive_bytes_total / node_network_receive_packets_total > 1500

for: 5m

labels:

severity: warning

annotations:

summary: "Высокая network latency на {{ \$labels.instance }}"

description: "Средний размер пакета превышает 1500 bytes, что может указывать на проблемы latency"

Управление alerts

Escalation

Alerts можно эскалировать на основе длительности и важности. Настройте политики эскалирования в Alerta, чтобы автоматически повышать критичность или уведомлять по дополнительным каналам, если алерт остается нерешенным.

Эскалация помогает гарантировать своевременное устранение критических проблем. Правила эскалации можно определить на основе:

- Time thresholds, например эскалация через 15 минут
- Severity levels
- Alert attributes, например конкретные сервисы или окружения

Пример настройки эскалации:

- Warning alerts переходят в critical через 30 минут
- Critical alerts немедленно уведомляют дежурных специалистов
- Major alerts уведомляют руководство через 1 час

Чтобы настроить эскалацию в Alerta, используйте web interface или API для настройки политик эскалации для разных alert types.

Suppression

Alerts можно временно приостанавливать или отключать с помощью функции silencing в Alerta. Это полезно во время окон технического обслуживания, плановых простоев или расследования известных проблем, когда отправка уведомлений не требуется.

Silences можно создавать для конкретных алертов или на основе фильтров вроде окружения, ресурса или типа события. Silenced alerts остаются видимыми в дашборде Alerta, но не создают оповещений.

Чтобы создать silence:

1. Откройте web interface Alerta
2. Перейдите в раздел Alerts
3. Выберите alert для silence или используйте фильтры, чтобы silence применился к нескольким алертам
4. Выберите "Silence" и задайте длительность и причину

Либо используйте API:

```
curl -X POST https://alerta.example.com/api/v2/silences \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "environment": "production",
  "resource": "server-01",
  "event": "HighCPUUsage",
  "startTime": "2023-12-01T00:00:00Z",
  "duration": 3600,
  "comment": "Scheduled maintenance"
}'
```

Silences также можно управлять через Alertmanager для более продвинутой приостановки на основе маршрутизации.

Конфигурация Alertmanager

Alertmanager выполняет маршрутизацию, группировку и дедупликацию алертов перед отправкой уведомлений. Он выступает посредником между Prometheus и системами оповещения, такими как Alerta.

Grouping

Алерты можно группировать по меткам, чтобы снизить уровень шума и избежать перегрузки из-за большого количества оповещений. Настройте группировку в конфигурации Alertmanager:

```
route:
group_by: ['alertname', 'cluster', 'namespace']
group_wait: 10s
group_interval: 10s
repeat_interval: 1h
receiver: 'default'
```

- group_by : labels, по которым группируются alerts
- group_wait : время ожидания перед отправкой первого notification
- group_interval : interval между notifications для одной группы
- repeat_interval : минимальное время между notifications

Routing

Маршрутизируйте алерты различным получателям на основе меток, чтобы отправлять адресные уведомления:

```
route:
receiver: 'default'
routes:
  - match:
severity: critical
receiver: 'critical-alerts'
  - match:
team: devops
receiver: 'devops-team'
  - match_re:
namespace: 'kube-.*'
receiver: 'kubernetes-alerts'
receivers:
  - name: 'default'
slack_configs:
  - api_url: 'https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK'
channel: '#alerts'
```

```
- name: 'critical-alerts'
pagerduty_configs:
  - service_key: 'YOUR_PAGERDUTY_KEY'
  - name: 'devops-team'
email_configs:
  - to: 'devops@example.com'
from: 'alertmanager@example.com'
smarthost: 'smtp.example.com:587'
auth_username: 'alertmanager@example.com'
auth_password: 'password'
  - name: 'kubernetes-alerts'
webhook_configs:
  - url: 'http://alerta.example.com/api/webhooks/prometheus'
send_resolved: true
```

Inhibition

Используйте правила подавления, чтобы не отправлять одни алерты, когда уже сработали другие связанные с ними алерты:

```
inhibit_rules:
  - source_match:
    alertname: 'NodeDown'
  target_match:
    alertname: 'PodCrashLooping'
  equal: ['node']
```

Подробнее о конфигурации Alertmanager см. в [официальной документации](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Сбор custom metrics

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/custom-metrics/>

Подключение собственных Prometheus exporters к tenant monitoring stack Cozystack с помощью VMServiceScrape и VMPodScrape.

Обзор

Tenant monitoring в Cozystack поддерживает сбор пользовательских метрик из ваших приложений и экспортеров. Tenant VMAgent обнаруживает таргеты для сбора метрик по меткам namespace Kubernetes, что позволяет подключать к мониторингу любые приложения., которое предоставляет Prometheus-compatible endpoint `/metrics`.

В этом руководстве показано, как создать ресурсы VMServiceScrape и VMPodScrape, чтобы tenant VMAgent собирал ваши пользовательские метрики и делал их доступными в Grafana.

Предварительные требования

- Monitoring включен для вашего tenant (см. [Monitoring Setup](#))
- Ваше приложение или экспортеры развернуты и предоставляют совместимый с Prometheus эндпоинт `/metrics`
- У вас есть доступ к кластеру через `kubectl`

Использование VMServiceScrape

VMServiceScrape указывает tenant VMAgent собирать метрики с эндпоинта за Kubernetes Service.

Пример

Предположим, у вас есть Service `my-app` в namespace `my-app-ns`, который предоставляет метрики на port `metrics` по пути `/metrics`:

```
apiVersion: operator.victoriametrics.com/v1beta1
```

```
kind: VMServiceScrape
```

```
metadata:
```

```
name: my-app-metrics
```

```
namespace: my-app-ns
```

```
spec:
```

```
selector:
```

matchLabels:

app: my-app

endpoints:

- port: metrics

path: /metrics

interval: 30s

Примените ресурс:

kubectl apply --filename vmservicescrape.yaml --namespace my-app-ns

Ключевые поля

| Поле | Описание |
|---------------------------|---|
| spec.selector.matchLabels | Label selector для поиска целевого Service |
| spec.endpoints[].port | Именованный порт в объекте Service, с которого выполняется сбор метрик. |
| spec.endpoints[].path | HTTP path для метрик (по умолчанию /metrics) |
| spec.endpoints[].interval | интервал сбора (по умолчанию наследуется от VMAgent, обычно 30s) |

Использование VMPodScrape

VMPodScrape собирает метрики напрямую с Pods без необходимости в создании сервиса. Это полезно для экспортёров, работающих в sidecar-контейнерах, или приложений, для которых не создан соответствующий объект Service.

Пример

Предположим, у вас есть Pods с label app: my-worker, которые предоставляют метрики на port 9090 по path /metrics:

apiVersion: operator.victoriametrics.com/v1beta1

kind: VMPodScrape

metadata:

name: my-worker-metrics

namespace: my-app-ns

spec:

selector:

```
matchLabels:
app: my-worker
podMetricsEndpoints:
  - port: "9090"
path: /metrics
```

Примените ресурс:

```
kubectl apply --filename vmpodscrape.yaml --namespace my-app-ns
```

Ключевые поля

| Поле | Описание |
|--|--|
| <code>spec.selector.matchLabels</code> | Label selector для поиска целевых Pods |
| <code>spec.podMetricsEndpoints[].port</code> | Имя или номер порта на Pod для сбора метрик |
| <code>spec.podMetricsEndpoints[].path</code> | HTTP путь для метрик (по умолчанию <code>/metrics</code>) |

Проверка сбора метрик

После создания `VMServiceScrape` или `VMPodScrape` проверьте, что tenant `VMAgent` собирает данные с ваших таргетов.

Проверьте targets VMAgent

Выведите pods `VMAgent` в namespace `tenant` и откройте страницу `targets`:

```
kubectl get pods --namespace <tenant-namespace> --selector app.kubernetes.io/name=vmagent
```

Сделайте `port-forward` к UI `VMAgent`, чтобы посмотреть активные таргеты:

```
kubectl port-forward --namespace <tenant-namespace> service/vmagent-vmagent 8429:8429
```

Затем откройте <http://localhost:8429/targets> в браузере. Новый target должен появиться в списке со статусом `UP`.

Запросите метрики в Grafana

1. Откройте Grafana по адресу <https://grafana.<tenant-host>>
2. Перейдите в Explore
3. Выберите datasource `VictoriaMetrics`

4. Выполните PromQL query для одной из custom metrics, например:

```
up{job="my-app-ns/my-app-metrics"}
```

Результат 1 подтверждает, что target успешно обрабатывается.

Устранение неполадок

- Target не появляется в VMAgent : проверьте, что namespace имеет label `namespace.cozystack.io/monitoring: <tenant-namespace>` и что `VMServiceScrape/VMPodScrape` создан в этом namespace
- Target показывает status DOWN : проверьте, что приложение запущено и metrics эндпоинт доступен на настроенном port и path
- Метрик нет в Grafana : убедитесь, что VMAgent пишет в правильный VMCluster, проверив логи VMAgent:

```
kubectll logs --namespace <tenant-namespace> --selector app.kubernetes.io/name=vmagent
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Monitoring Dashboards

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/dashboards/>

Как визуализировать метрики и создавать custom dashboards в Grafana для мониторинга кластеров и приложений Cozystack.

Обзор

Cozystack интегрирует Grafana как основной инструмент визуализации метрик, собираемых VictoriaMetrics (VM). В этом разделе описан доступ к предварительно настроенным дашбордам, создание собственных визуализаций и подключение внешних источников данных для полноценной наблюдаемости кластеров и приложений Cozystack.

Доступ к Grafana

Чтобы открыть Grafana и посмотреть дашборды:

1. Перейдите по URL Grafana: <https://grafana.<tenant-domain>>, где `<tenant-domain>` \- домен вашего tenant.
2. Войдите в систему, используя учётные данные tenant (OIDC или token-based authentication).
3. После входа предварительно настроенные дашборды доступны в разделе "Dashboards".

Первичную настройку и детали конфигурации см. в [Monitoring Setup](#).

Pre-built dashboards

Cozystack предоставляет набор предварительно настроенные дашбордов в Grafana, которые автоматически разворачиваются и обновляются через monitoring stack. Эти дашборды определены в файле `packages/extra/monitoring/dashboards.list` и сразу дают представление о производительности системы и приложений.

Dashboards инфраструктуры кластера

- Kubernetes Cluster Overview : дает верхнеуровневый обзор всего Kubernetes-кластера, включая статус узлов, health pods, использование CPU/memory/disk по кластеру и производительность API server. Полезен для быстрой проверки состояния кластера и выявления узких мест, связанных с ресурсами.
- Node Metrics : подробные метрики по узлам: CPU usage, memory consumption, disk I/O, network traffic и system load. Включает панели для отдельных узлов и агрегированные представления. Подходит для диагностики проблем конкретного узла.

-
- ETCD Metrics : мониторит состояние кластера ETCD, включая latency, storage usage, leader elections и database operations. Важен для надежности данных Kubernetes control plane.
 - Storage Metrics : показывает компоненты хранения, такие как Linstor и SeaweedFS: volume usage, I/O operations, replication status и performance metrics. Помогает управлять ресурсами хранения данных и диагностировать связанные с ними проблемы.

Dashboards приложений и сервисов

- Tenant Applications : Настраиваемые дашборды для пользовательских приложений, отображающие частоту запросов, частоту ошибок, время отклика и пропускную способность. Подходят для веб-сервисов, API и микросервисов, работающих в tenant namespaces.
- Service Mesh : метрики сетевых компонентов, включая ingress controllers (например, NGINX, Traefik), load balancers и service mesh proxies. Отображают характеристики трафика, задержки, частоту ошибок и состояние сетевой связности.
- Database Metrics : специализированные дашборды для поддерживаемых баз данных, таких как PostgreSQL, MySQL, Redis и других. Включают показатели производительности запросов, количество подключений, долю попаданий в кеш и метрики хранилища. Например, дашборд PostgreSQL отображает активные подключения, медленные запросы и состояние репликации.

Эти дашборды регулярно обновляются в новых релизах. Скриншоты и визуальные примеры см. в release notes с предпросмотром дашбордов в [блоге Cozystack](#).

Создание custom dashboards

Если предварительно настроенные дашборды не покрывают ваши потребности, можно создавать пользовательские дашборды в Grafana, чтобы визуализировать конкретные метрики или объединять данные из нескольких источников.

Шаги создания custom dashboard

1. Откройте Grafana : войдите в Grafana с tenant credentials.
2. Создайте новый dashboard : нажмите значок "+" в боковой панели и выберите "Dashboard".
3. Добавьте панели : нажмите "Add new panel", чтобы создать блок визуализации. Выберите тип панели и настройте источники данных.
4. Настройте queries : используйте MetricsQL (язык запросов VictoriaMetrics) для получения и преобразования данных.
5. Настройте layout : расположите панели, задайте временные диапазоны и добавьте аннотации или переменные для интерактивного взаимодействия с дашбордом.
6. Сохраните и поделитесь : сохраните дашборд, настройте права доступа и при необходимости экспортируйте его для повторного использования.

Примеры queries

Ниже несколько распространенных MetricsQL запросов для пользовательских панелей:

- Использование CPU pod :

```
rate(container_cpu_usage_seconds_total{pod=~"$pod"}[5m])
```

Показывает CPU usage rate для выбранных pods во времени.

- Процент использования memory :

```
(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
```

Показывает memory utilization на узлах в процентах.

- Network traffic :

```
rate(node_network_receive_bytes_total[5m]) + rate(node_network_transmit_bytes_total[5m])
```

Мониторит входящий и исходящий network traffic.

- Response time приложения :

```
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket{job="my-app"}[5m]))
```

Вычисляет 95-й percentile response time для приложения.

Типы panels и best practices

- Time Series (Graph) : Подходит для отображения изменений показателей во времени, например загрузки CPU или частоты запросов. Используйте этот тип визуализации для анализа исторических данных.
- Stat : показывает одиночные значения, например текущий процент CPU или общее число запросов. Удобен для быстрых метрик.
- Table : показывает табличные данные, например топ процессов или сводные данные по оповещениям. Полезен для подробных списков.
- Heatmap : отображает плотность распределения значений, например частоту ошибок по временным интервалам. Эффективен для выявления закономерностей.
- Gauge : представляет значения на шкале, например процент использования диска.

При создании панелей учитывайте:

- Используйте подходящие временные промежутки и интервалы обновлений.
- Добавляйте пороговые значения и правила оповещения непосредственно на панели для проактивного мониторинга.
- Используйте переменные для динамической фильтрации, например по namespace или имени pod.

Расширенные queries и functions см. в [документации VictoriaMetrics MetricsQL](#).

Интеграция external data sources

Cozystack позволяет интегрироваться с внешними системами мониторинга, чтобы централизовать данные наблюдаемости.

Добавление External Prometheus

Чтобы интегрировать внешний экземпляр Prometheus:

1. В Grafana перейдите в "Configuration" > "Data Sources" > "Add data source".
2. Выберите тип "Prometheus".
3. Укажите URL внешнего Prometheus, данные аутентификации (если требуются) и интервал сбора метрик.
4. Проверьте соединение и сохраните.
5. Используйте PromQL в dashboards, чтобы запрашивать внешние данные.

Custom application metrics

Для приложений, предоставляющих пользовательские метрики:

- Убедитесь, что приложение предоставляет метрики в формате Prometheus, например через эндпоинт `/metrics`.
- Настройте VMAgent в Cozystack на scraping этих endpoints, обновив monitoring configuration.
- Метрики будут загружены в VM и доступны для запросов в Grafana.

Для совместимости соблюдайте [metric naming conventions](#) Prometheus. Примеры конфигурации см. в [Monitoring Hub Reference](#).

Интеграция Grafana data sources

Эта диаграмма показывает, как внешние источники данных интегрируются в Grafana для централизованного мониторинга.

graph TD

A[VictoriaMetrics
VM Data Source
MetricsQL Queries] --> B[Grafana]

C[VLogs
Log Data Source
Log Queries] --> B

D[External Prometheus
PromQL Queries] --> B

E[Custom Application Metrics
Prometheus Format] --> B

B --> F[Dashboards
Pre-built and Custom]

F --> G[Visualization
Metrics + Logs Correlation]

Конфигурация data sources

Grafana в Cozystack заранее настроена с оптимизированными источниками данных для бесшовной интеграции.

VictoriaMetrics (VM) Data Source

- Type : Prometheus совместимые (MetricsQL).
- URL : внутренний эндпоинт VM cluster в namespace tenant.
- Authentication : автоматическая через service account tokens.
- Usage : основной источник метрик временных рядов. Поддерживает высокопроизводительное выполнение запросов и агрегацию данных.

VLogs Data Source

- Type : пользовательский плагин для запросов к логам.
- Purpose : включает визуализацию логов и корреляцию с метриками.
- Configuration : автоматически настраивается для tenant-specific потоков логов.
- Usage : добавляйте панели логгирования в дашборды, чтобы объединять метрики и логи, например для поиска проблем приложений.

Чтобы изменить настройки источников данных, откройте Grafana admin panel (нужны admin privileges) или обновите monitoring configuration через Cozystack API. Подробные параметры см. в [Monitoring Hub Reference](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Логи мониторинга

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/logs/>

Как собирать, хранить, искать и анализировать логи в Cozystack с помощью Fluent Bit и VictoriaLogs для полноценной наблюдаемости.

Сбор и хранение логов

Cozystack использует Fluent Bit для сбора логов и VictoriaLogs для хранения и запросов. Логи собираются из разных источников внутри кластера и хранятся в выделенных log storages, настроенных для каждого tenant.

Настройка logs storages

Log storages настраиваются через параметры monitoring hub. У каждого tenant может быть несколько экземпляров log storage с настраиваемыми периодами хранения и размерами хранилища.

| Параметр | Описание | Тип | По умолчанию |
|----------------------------------|---|----------|--------------|
| logsStorages | Массив конфигураций log storage | []object | [] |
| logsStorages[i].name | Имя экземпляра storage | string | " " |
| logsStorages[i].retentionPeriod | Период хранения для логов, например "30d" | string | "1" |
| logsStorages[i].storage | Размер persistent volume | string | "10Gi" |
| logsStorages[i].storageClassName | StorageClass для хранения данных | string | "replicated" |

Подробные параметры конфигурации см. в [Monitoring Hub Reference](#).

Fluent Bit inputs и outputs

Fluent Bit настроен на сбор логов из:

- Kubernetes Pods : container logs из всех namespaces
- System Logs : логи уровня узла и системных сервисов
- Application Logs : пользовательские источники логов, подключаемые через sidecar-контейнеры.

Пример конфигурации Fluent Bit input

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: fluent-bit-config
data:
  fluent-bit.conf: |
    [INPUT]
    Name tail
    Path /var/log/containers/*.log
    Parser docker
    Tag kube.*
    Refresh_Interval 5

    [OUTPUT]
    Name vlogs
    Match kube.*
    Host vlogs-cluster
    Port 9428
```

Логи отправляются в VictoriaLogs для хранения и индексирования. Output plugin дополняет логи metadata, например именами pod, namespaces и временными метками.

Архитектура логирования

Следующая диаграмма показывает архитектуру логирования в Cozystack: как логи проходят от разных источников к хранилищу и инструментам визуализации.

```
graph TD
  A[Kubernetes Pods] --> B[Fluent Bit]
  C[System Logs] --> B
  D[Application Logs] --> B
  B --> E[VictoriaLogs]
  E --> F[Grafana]
  F --> G[Log Analysis and Dashboards]
```

Поиск и анализ логов

VictoriaLogs (VLogs) предоставляет мощные возможности запросов к сохраненным логам. Для расширенного анализа логов используйте VLogs через Grafana или напрямую через API.

Использование VictoriaLogs

- Query Language : используйте синтаксис запросов VLogs, чтобы фильтровать логи по fields, time ranges и patterns.
- Integration with Grafana : визуализируйте логи вместе с метриками в dashboards.

Пример VLogs query

Чтобы найти записи об ошибках из конкретного пода:

```
_level:ERROR AND kubernetes_pod_name: "my-app-pod"
```

Filters и metadata

Логи в Cozystack содержат подробные метаданные для эффективной фильтрации:

- Pod Metadata : `kubernetes_pod_name`, `kubernetes_namespace_name`, `kubernetes_container_name`
- Tenant : `tenant` \- определяет, к какому tenant относятся логи
- Log Levels : `_level` (INFO, WARN, ERROR и т. д.)
- Timestamps : автоматическое распознавание и обработка временных меток.
- Custom Labels : labels, относящиеся к конкретному приложению и добавляемые при сборе логов.

Расширенная фильтрация

Используйте сложные запросы для корреляции логов:

```
kubernetes_namespace_name: "kube-system" AND _level: "WARN" AND _msg: timeout
```

Подробнее о запросах VLogs см. в [документации VictoriaLogs](#).

Просмотр логов tenant Kubernetes cluster

Когда рабочие нагрузки выполняются в [tenant Kubernetes cluster](#), их логи собираются и отправляются в экземпляр VictoriaLogs родительского tenant. Затем эти логи можно запрашивать в Grafana с помощью специальных фильтров по label.

Предварительные требования

Включите addon `monitoringAgents` в tenant Kubernetes cluster. Он разворачивает внутри кластера агентов, которые собирают логи и отправляют их в VictoriaLogs.

Через Cozystack дашборд задайте `addons.monitoringAgents.enabled: true` в параметрах Kubernetes application или примените это программно:

addons:

monitoringAgents:

enabled: true

Подробности см. в [параметрах Managed Kubernetes](#).

Labels логов

Логи из tenant Kubernetes clusters обогащаются следующими labels:

| Label | Описание | Пример |
|---------------------------|---|-------------------------|
| tenant | Идентификатор tenant
(формат: tenant-<name>) | tenant-workload |
| kubernetes_namespace_name | Namespace внутри tenant
Kubernetes cluster | default |
| kubernetes_pod_name | Имя pod | my-app-6b7b8c9b89-ccqgf |
| kubernetes_container_name | Имя container внутри pod | my-app |

Запрос логов в Grafana

1. Откройте Grafana по адресу `https://grafana.<tenant-host>`
2. Перейдите в Explore
3. Выберите datasource VictoriaLogs
4. Используйте query builder или напишите query напрямую

Фильтр всех логов tenant

tenant: "tenant-workload"

Фильтр по tenant и namespace

tenant: "tenant-workload" AND kubernetes_namespace_name: "default"

Фильтр логов конкретного pod

tenant: "tenant-workload" AND kubernetes_namespace_name: "default" AND kubernetes_pod_name: "my-app-6b7b8c9b89-ccqgf"

Интеграция с приложениями

Чтобы повысить наблюдаемость логов, добавьте в приложения structured logging и enrichment labels.

Structured logs

Используйте форматы structured logging, например JSON, для лучшего parsing и querying:

```
{
  "timestamp": "2023-10-01T12:00:00Z",
  "level": "INFO",
  "message": "User login successful",
  "user_id": "12345",
  "action": "login"
}
```

Обогащение логов labels

Добавляйте пользовательские labels в логи для расширенной фильтрации:

- Tenant Labels : автоматически добавляются для multi-tenancy
- Application Labels : custom labels, например `app_version`, `environment`
- Business Labels : domain-specific metadata

Пример конфигурации приложения

Настройте логгирование в приложении так, чтобы он включал структурированные данные:

```
import logging
import json

logger = logging.getLogger(__name__)

def log_event(level, message, **kwargs):
    log_entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "level": level,
        "message": message,
        **kwargs
    }
    logger.info(json.dumps(log_entry))
```

Убедитесь, что парсеры Fluent Bit настроены на обработку вашего формата логов. Подробности настройки см. в [Monitoring Setup](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Параметры мониторинга

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/parameters/>

Настройка и управление параметрами мониторинга в Cozystack.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|------|---|--------|----------|
| host | Hostname для внешнего доступа к Grafana (по умолчанию subdomain grafana для host tenant). | string | " " |

Конфигурация metrics storage

| Имя | Описание | Тип | Значение |
|--|---|----------|----------|
| metricsStorages | Конфигурация экземпляров metrics storage. | []object | [...] |
| metricsStorages[i].name | Имя экземпляра storage. | string | " " |
| metricsStorages[i].retentionPeriod | Retention period для metrics. | string | " " |
| metricsStorages[i].deduplicationInterval | Deduplication interval для metrics. | string | " " |
| metricsStorages[i].storage | Размер persistent volume. | string | 10Gi |
| metricsStorages[i].storageClassName | StorageClass, используемый для данных. | string | " " |
| metricsStorages[i].vminsert | Конфигурация vminsert. | object | {} |

| | | | |
|---|--|-----------------------|------------------|
| <code>metricsStorages[i].vminsert.minAllowed</code> | Минимальные гарантированные resources. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vminsert.minAllowed.cpu</code> | CPU request. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vminsert.minAllowed.memory</code> | Memory request. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vminsert.maxAllowed</code> | Максимально разрешенные resources. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vminsert.maxAllowed.cpu</code> | CPU limit. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vminsert.maxAllowed.memory</code> | Memory limit. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vmsselect</code> | Конфигурация vmselect. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vmsselect.minAllowed</code> | Минимальные гарантированные resources. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vmsselect.minAllowed.cpu</code> | CPU request. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vmsselect.minAllowed.memory</code> | Memory request. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vmsselect.maxAllowed</code> | Максимально разрешенные resources. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vmsselect.maxAllowed.cpu</code> | CPU limit. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vmsselect.maxAllowed.memory</code> | Memory limit. | <code>quantity</code> | <code>" "</code> |
| <code>metricsStorages[i].vmsstorage</code> | Конфигурация vmstorage. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vmsstorage.minAllowed</code> | Минимальные гарантированные resources. | <code>object</code> | <code>{}</code> |

| | | | |
|--|--|-----------------------|-----------------|
| <code>metricsStorages[i].vms
storage.minAllowed.cpu</code> | CPU request. | <code>quantity</code> | <code>""</code> |
| <code>metricsStorages[i].vms
storage.minAllowed.memo
ry</code> | Memory request. | <code>quantity</code> | <code>""</code> |
| <code>metricsStorages[i].vms
storage.maxAllowed</code> | Максимально
разрешенные
resources. | <code>object</code> | <code>{}</code> |
| <code>metricsStorages[i].vms
storage.maxAllowed.cpu</code> | CPU limit. | <code>quantity</code> | <code>""</code> |
| <code>metricsStorages[i].vms
storage.maxAllowed.memo
ry</code> | Memory limit. | <code>quantity</code> | <code>""</code> |

Конфигурация logs storage

| Имя | Описание | Тип | Значение |
|--|---|-----------------------|-------------------------|
| <code>logsStorages</code> | Конфигурация
экземпляров logs
storage. | <code>[]object</code> | <code>[...]</code> |
| <code>logsStorages[i].name</code> | Имя экземпляра
storage. | <code>string</code> | <code>""</code> |
| <code>logsStorages[i].retent
ionPeriod</code> | Retention period для
logs. | <code>string</code> | <code>1</code> |
| <code>logsStorages[i].storag
e</code> | Размер persistent
volume. | <code>string</code> | <code>10Gi</code> |
| <code>logsStorages[i].storag
eClassName</code> | StorageClass,
используемый для
хранения данных. | <code>string</code> | <code>replicated</code> |

Конфигурация Alerta

| Имя | Описание | Тип | Значение |
|-----------------------------|---|---------------------|-------------------|
| <code>alerta</code> | Конфигурация сервиса
Alerta. | <code>object</code> | <code>{}</code> |
| <code>alerta.storage</code> | Размер persistent
volume для database. | <code>string</code> | <code>10Gi</code> |

| | | | |
|--|--|----------|-------|
| <code>alerta.storageClassName</code> | StorageClass, используемый для database. | string | "" |
| <code>alerta.resources</code> | Конфигурация resources. | object | {} |
| <code>alerta.resources.requests</code> | Resource requests. | object | {} |
| <code>alerta.resources.requests.cpu</code> | CPU request. | quantity | 100m |
| <code>alerta.resources.requests.memory</code> | Memory request. | quantity | 256Mi |
| <code>alerta.resources.limits</code> | Resource limits. | object | {} |
| <code>alerta.resources.limits.cpu</code> | CPU limit. | quantity | 1 |
| <code>alerta.resources.limits.memory</code> | Memory limit. | quantity | 1Gi |
| <code>alerta.alerts</code> | Конфигурация alert routing. | object | {} |
| <code>alerta.alerts.telegram</code> | Конфигурация Telegram alerts. | object | {} |
| <code>alerta.alerts.telegram.token</code> | Telegram bot token. | string | "" |
| <code>alerta.alerts.telegram.chatID</code> | Telegram chat ID(s), разделенные запятыми. | string | "" |
| <code>alerta.alerts.telegram.disabledSeverity</code> | Список severities без alerts, например ["informational", "warning"]. | []string | [] |
| <code>alerta.alerts.slack</code> | Конфигурация Slack alerts. | object | {} |
| <code>alerta.alerts.slack.url</code> | Configuration URI для Slack alerts. | string | "" |
| <code>alerta.alerts.slack.disabledSeverity</code> | Список severities без alerts, например ["informational", "warning"]. | []string | [] |

Конфигурация Grafana

| Имя | Описание | Тип | Значение |
|-----------------------------------|--|----------|----------|
| grafana | Конфигурация Grafana. | object | {} |
| grafana.db | Конфигурация database. | object | {} |
| grafana.db.size | Размер persistent volume для database. | string | 10Gi |
| grafana.resources | Конфигурация resources. | object | {} |
| grafana.resources.requests | Resource requests. | object | {} |
| grafana.resources.requests.cpu | CPU request. | quantity | 100m |
| grafana.resources.requests.memory | Memory request. | quantity | 256Mi |
| grafana.resources.limits | Resource limits. | object | {} |
| grafana.resources.limits.cpu | CPU limit. | quantity | 1 |
| grafana.resources.limits.memory | Memory limit. | quantity | 1Gi |

Конфигурация Vmagent

| Имя | Описание | Тип | Значение |
|------------------------|---|--------|----------|
| vmagent | Конфигурация VictoriaMetrics Agent. | object | {} |
| vmagent.externalLabels | External labels, применяемые ко всем metrics. | object | {} |
| vmagent.remoteWrite | Конфигурация remote write. | object | {} |

OIDC authentication

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

| | | | |
|--|--|---------------------|-------------------|
| <code>oidc</code> | OIDC authentication for the Grafana instance. See docs/oidc-grafana.md for the operator guide. | <code>object</code> | <code>{}</code> |
| <code>oidc.mode</code> | Identity mode. <code>None</code> : no OIDC, only the <code>grafana-admin-password</code> Secret works. <code>System</code> : trust the platform <code>cozy</code> realm via a per-instance confidential client and audience binding; zero-config default. <code>CustomConfig</code> : trust a tenant-supplied issuer directly (BYO); <code>cozy</code> is not in the path. | <code>string</code> | <code>None</code> |
| <code>oidc.customConfig</code> | Tenant-supplied <code>[auth.generic_oauth]</code> payload; consumed only when <code>mode: CustomConfig</code> . | <code>object</code> | <code>{}</code> |
| <code>oidc.customConfig.config</code> | Inline map of <code>[auth.generic_oauth]</code> keys. Written verbatim into the Grafana CR <code>spec.config</code> . Mutually exclusive with <code>secretRef.name</code> . | <code>object</code> | <code>{}</code> |
| <code>oidc.customConfig.secretRef</code> | Reference to a Secret holding an <code>auth.ini</code> fragment; mounted under <code>/etc/grafana/oidc</code> and loaded via <code>GF_PATHS_CUSTOM_INI</code> . | <code>object</code> | <code>{}</code> |

| | | | |
|---|---|----------|----|
| <code>oidc.customConfig.secretRef.name</code> | Name of an existing Secret in the release namespace whose <code>auth.ini</code> key holds a ready-made <code>[auth.generic_oauth]</code> block. Mutually exclusive with <code>customConfig.config</code> . | string | "" |
| <code>oidc.users</code> | Users granted access to this Grafana instance's org. Each entry is reconciled into Grafana org membership + role by a chart-owned post-install/post-upgrade Job. Works for both <code>System</code> and <code>CustomConfig</code> modes; ignored when <code>mode: None</code> . Users not listed here get nothing (no default org role from group mapping). | []object | [] |

| | | | |
|----------------------------------|---|---------------------|-----------------|
| <code>oidc.users[i].email</code> | <p>Email address matched against the <code>email</code> claim from the issuer. The chart pre-provisions a Grafana account with this email so the <code>users-Job</code> can set the <code>org</code> role before the operator's first login (<code>oauth_allow_insecure_email_lookup = true</code> glues the two together). Must match a conservative pattern: local part is <code>[A-Za-z0-9._%+ -]</code>, domain is <code>[A-Za-z0-9.-]+ \. [A-Za-z]{2,}</code>. Whitespace, quoted literals, and unusual URL-reserved characters are rejected on schema level so the <code>reconcile-Job</code>'s shell handling never has to defend against malformed input.</p> | <code>string</code> | <code>""</code> |
| <code>oidc.users[i].role</code> | <p>Grafana org role to bind: <code>Admin</code>, <code>Editor</code>, or <code>Viewer</code>. Applied to the release's Grafana org (<code>Main Org.</code>); server-level <code>GrafanaAdmin</code> promotion is out of scope for Phase 1. Required — the <code>users-Job</code> POSTs <code>{loginOrEmail, role}</code> to <code>/api/orgs/1/users</code> and PATCHes <code>{role}</code> to converge across re-runs; omitting the field would render <code>role: "null"</code> into the body and fail with HTTP 400.</p> | <code>string</code> | <code>{}</code> |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Настройка мониторинга

<https://cozystack.ru/docs/v1.6/operations/services/monitoring/setup/>

Руководство по установке и настройке мониторинга в Cozystack

Обзор

Cozystack предоставляет комплексную систему monitoring и alerting для Kubernetes-кластеров и приложений. Система основана на Victoria Metrics для хранения метрик и логов, Grafana для визуализации, Alerta для оповещений и WorkloadMonitor для мониторинга состояния приложений.

Архитектура разделена на два уровня:

- Системный уровень : мониторинг инфраструктуры кластера
- Уровень tenant : изолированный мониторинг для каждого tenant

Установка мониторинга

Системный monitoring

System monitoring устанавливается автоматически при разворачивании платформы через HelmRelease `monitoring-agents` в `packages/apps/kubernetes/templates/helmreleases/monitoring-agents.yaml`.

Для ручной установки через локальный chart:

```
helm upgrade --install monitoring-agents ./packages/system/monitoring-agents -n cozy-monitoring --create-namespace
```

Мониторинг tenant

Мониторинг для конкретного tenant активируется patch ресурса Tenant (CRD `apps.cozystack.io/v1alpha1/Tenant`).

Через Dashboard UI

1. Откройте Cozystack dashboard
2. Перейдите в управление tenant
3. Выберите tenant и задайте `monitoring: true` в values

Программно

```
kubectl patch tenant <tenant-name> --type merge -p '{"spec":{"values":{"monitoring": true}}}'
```

После активации FluxCD автоматически разворачивает HelmRelease `monitoring` в namespace `tenant` из шаблона `packages/apps/tenant/templates/monitoring.yaml`.

Архитектура компонентов

Системный уровень

VMAgent

- Роль : agent для сбора cluster метрик
- Конфигурация :
 - Scrape interval: 30 секунд
 - External labels: `cluster: cozystack, tenant: tenant-root`
 - Remote write: в VMCluster tenant-root (shortterm и longterm)
 - Источники метрик :
 - Kubernetes API (Pods, Services, Deployments)
 - kube-state-metrics
 - Node Exporter
 - cAdvisor (via kubelet)

VMRule

- Recording Rules : агрегированные метрики
 - `container_memory:kmem` \- kernel memory контейнеров
 - `kube_persistentvolume_is_local` \- Local PVs
 - `kube_controller_pod` \- связи Pod с контроллерами
- Alerting Rules : стандартные алерты Prometheus
 - TargetDown \- недоступные таргеты
 - Watchdog \- тестовый алерт
 - Kubernetes-specific alerts (apiserver, etcd, nodes, pods)

Fluent Bit

- Роль : сбор и агрегация логов
- Inputs :
 - Container logs: `/var/log/containers/*.log`
 - Kubernetes events
 - Audit logs: `/var/log/audit/kube/*.log`
- Outputs : Victoria Logs (`vlogs-generic.tenant-root.svc:9428`)
- Parsers : docker, cri, kubernetes metadata

Alertmanager

- Роль : routing alerts
- Конфигурация :
- Grouping по `alertname`, `namespace`, `cluster`
- Routes: Alerta webhook, blackhole для `severity="none"`
- Интеграция : webhook в Alerta для нотификаций

Уровень tenant

Victoria Metrics Cluster

- Компоненты :
- `vminsert`: прием метрик (2 replicas)
- `vmselect`: запросы метрик (2 replicas, 2Gi cache)
- `vmstorage`: хранение метрик (2 replicas, 10Gi PVC)
- Storage :
- Shortterm: хранение 3 дня, дедупликация 15s
- Longterm: хранение 14 дней, дедупликация 5m
- Replication: factor 2

Victoria Logs

- Роль : хранение структурированных логов
- Конфигурация : хранение 1 год, 10Gi PVC

Grafana

- Роль : визуализация метрик и логов
- Database : PostgreSQL (10Gi PVC)
- Datasources :
- Victoria Metrics (метрики)
- Victoria Logs (логи)
- Ingress : `grafana.{tenant-host}`
- Resources : 2 replicas, limits 1 CPU/1Gi RAM
- Dashboards : готовые дашборды для всех компонентов Cozystack

Alerta

- Роль : централизованная система оповещения
- Database : PostgreSQL (10Gi PVC)
- Notifications : Telegram, Slack
- API : Protected by API key

- Ingress : alerta.{tenant-host}

VMAlert (tenant)

- Роль : вычисление правил алертинга для tenant
- Datasource : запросы к краткосрочным данным
- Remote write : запись краткосрочных данных
- Evaluation interval : 15 seconds

VMAgent (tenant)

- Роль : сбор метрик из namespace tenant
- Selector : namespace.cozystack.io/monitoring
- External labels : cluster: cozystack, tenant: {namespace}
- Remote write : запись краткосрочных и долгосрочных данных

Мониторинг приложений

WorkloadMonitor CRD

WorkloadMonitor (cozystack.io/v1alpha1/WorkloadMonitor) в первую очередь используется для биллинга, отслеживания состояний workloads и сбора метрик ресурсов. Alerting является дополнительной функцией.

Спецификация

apiVersion: cozystack.io/v1alpha1

kind: WorkloadMonitor

metadata:

name: mysql-instance

spec:

replicas: 1

minReplicas: 1

kind: mysql

type: mysql

selector:

app.kubernetes.io/instance: mysql-instance

version: "10.11"

Status

status:
operational: true # true, если AvailableReplicas >= MinReplicas
availableReplicas: 1
observedReplicas: 1

WorkloadMonitor Controller

- Отслеживает Pods, PVCs, Services по selector
- Создает CRD Workload с агрегированными метриками для биллинга
- Экспортирует metrics через kube-state-metrics

Alerting Integration

Хотя WorkloadMonitor предназначен для биллинга, он также интегрируется с alerting. Alert срабатывает при `cozy_workload_status_operational{operational="false"} == 1`:

1. VMAAlert вычисляет rule
2. Отправляет alert в Alertmanager
3. Alertmanager направляет его в Alerta
4. Alerta отправляет notifications (Telegram/Slack)

Примеры для приложений

MySQL

```
apiVersion: cozystack.io/v1alpha1
kind: WorkloadMonitor
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicas }}
  minReplicas: 1
  kind: mysql
  type: mysql
  selector:
    app.kubernetes.io/instance: {{ .Release.Name }}
  version: {{ .Chart.Version }}
```

Kafka

Для Kafka:

```
apiVersion: cozystack.io/v1alpha1
kind: WorkloadMonitor
metadata:
  name: {{ .Release.Name }}-kafka
spec:
  replicas: {{ .Values.kafka.replicas }}
  minReplicas: 1
  kind: kafka
  type: kafka
  selector:
    app.kubernetes.io/name: kafka
    app.kubernetes.io/instance: {{ .Release.Name }}
  version: {{ .Chart.Version }}
```

Для Zookeeper:

```
apiVersion: cozystack.io/v1alpha1
kind: WorkloadMonitor
metadata:
  name: {{ .Release.Name }}-zookeeper
spec:
  replicas: {{ .Values.zookeeper.replicas }}
  minReplicas: 1
  kind: zookeeper
  type: zookeeper
  selector:
    app.kubernetes.io/name: zookeeper
    app.kubernetes.io/instance: {{ .Release.Name }}
  version: {{ .Chart.Version }}
```

Аналогичные конфигурации применяются для PostgreSQL, Redis, ClickHouse, RabbitMQ, NATS и других приложений.

Поток данных

```

graph TD
    subgraph "System Level (tenant-root)"
        SYS_VMA[VMASgent System]
        SYS_FB[Fluent Bit]
        SYS_VM[VMCluster tenant-root]
        SYS_VL[VLogs tenant-root]
        SYS_AM[Alertmanager]
    end

    subgraph "Tenant Level"
        T_VMA[VMASgent Tenant]
        T_VM[VMCluster Tenant<br/>shortterm + longterm]
        T_VL[VLogs Tenant]
        T_VA[VMAlert Tenant]
        T_AM[Alertmanager Tenant]
        T_AL[Alerta Tenant]
        T_G[Grafana Tenant]
        T_WM[WorkloadMonitor]
    end

    SYS_VMA --> |metrics| SYS_VM
    SYS_FB --> |logs| SYS_VL

```

| | | |
|----------------------|---------------|-------------|
| SYS_AM --> | alerts | T_AL |
|----------------------|---------------|-------------|

```

T_VMA --> |metrics| T_VM
T_VA --> |read| T_VM
T_VA --> |write| T_VM
T_VA --> |alerts| T_AM
T_AM --> |webhook| T_AL
T_AL --> |Telegram/Slack| Users
T_G --> |datasources| T_VM
T_G --> |datasources| T_VL

```

| | | |
|--------------------|----------------|--------------|
| T_WM --> | metrics | T_VMA |
|--------------------|----------------|--------------|

```

Apps[Applications<br/>MySQL, Kafka, etc.] --> T_WM
Infra[Infrastructure<br/>Pods, Nodes, etc.] --> SYS_VMA
Infra --> SYS_FB

```

Конфигурация и настройка

Параметры values.yaml

Системный monitoring (packages/system/monitoring-agents/values.yaml)

```
# VMAgent
vmagent:
  shardCount: 1
  externalLabels:
  cluster: cozystack
  tenant: tenant-root
  remoteWrite:
    - url: "http://vminsert-shortterm.tenant-root.svc:8480/insert/0/prometheus"
    - url: "http://vminsert-longterm.tenant-root.svc:8480/insert/0/prometheus"

# Fluent Bit
fluent-bit:
  inputs:
    - tail:
  path: /var/log/containers/*.log
    - kubernetes_events:
  kube_url: https://kubernetes.default.svc:443
  outputs:
    - vlogs:
  url: http://vlogs-generic.tenant-root.svc:9428
```

Monitoring tenant (packages/extra/monitoring/values.yaml)

```
# VMCluster
metricsStorages:
  shortterm:
  retention: "3d"
  deduplication: "15s"
  longterm:
  retention: "14d"
  deduplication: "5m"
```

Grafana

grafana:

host: grafana.example.com

resources:

limits:

cpu: 1

memory: 1Gi

Alerta

alerta:

host: alerta.example.com

telegram:

token: "your-telegram-token"

slack:

webhook: "your-slack-webhook"

Dashboard Grafana

Cozystack включает готовые дашборды для:

- Kubernetes : nodes, pods, control-plane
- Victoria Metrics : cluster, agent, alert
- Applications : nginx, postgres, redis, kafka, mysql, etc.
- Flux : control-plane, stats
- Storage : linstor, seaweedfs

Dashboards определены в `packages/extra/monitoring/dashboards.list`.

Чтобы открыть Grafana:

1. Перейдите на `https://grafana.{tenant-host}`
2. Войдите с дефолтными учетными данными (admin/admin) или настроенными
3. Посмотрите готовые дашборды в разделе Dashboards

Безопасность и масштабируемость

Безопасность

- RBAC : минимальные права для service accounts
- Network Policies : ограничение трафика между компонентами

-
- TLS : Ingress с TLS сертификатами
 - API Keys : аутентификация для Alerta

Масштабируемость

- Horizontal Pod Autoscaling : для компонентов VM
- Resource Limits : настраиваемые CPU/Memory
- Storage : PVC с настраиваемым storageClass
- Sharding : VMAgent поддерживает распределение нагрузки

Интеграции

- Alertmanager : Webhook, email, PagerDuty, Slack
- Grafana : plugins для Victoria Metrics, Loki
- External Storage : remote write во внешние VM instances
- GitOps : управление через FluxCD

Диагностика и troubleshooting

Проверка состояния

Чтобы проверить настройку мониторинга:

Проверить status VMCluster

```
kubectl get vmcluster -n <tenant-namespace>
```

Проверить pods VMAgent

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=vmagent
```

Проверить pods Grafana

```
kubectl get pods -n <tenant-namespace> -l app.kubernetes.io/name=grafana
```

Проверить pods Alerta

```
kubectl get pods -n <tenant-namespace> -l app.kubernetes.io/name=alerta
```

Проверить status WorkloadMonitor

```
kubectl get workloadmonitor -n <tenant-namespace>
```

Просмотр логов

Логи VMAgent

```
kubectl logs -n cozy-monitoring deployment/vmagent
```

```
# Логи VMAgent
```

```
kubectl logs -n <tenant-namespace> deployment/vmalert
```

```
# Логи Fluent Bit
```

```
kubectl logs -n cozy-monitoring daemonset/fluent-bit
```

```
# Логи Grafana
```

```
kubectl logs -n <tenant-namespace> deployment/grafana
```

Типичные проблемы

- Metrics не собираются : проверьте selectors в ресурсах VMServiceScrape
- Alerts не срабатывают : проверьте rules в ресурсах VMRule и конфигурацию Alertmanager
- Logs не поступают : проверьте конфигурацию Fluent Bit и outputs
- WorkloadMonitor не работает : проверьте selector labels и status CRD
- Grafana недоступна : проверьте ingress configuration и TLS certificates
- Alerta notifications не отправляются : проверьте API keys и webhook URLs

Дополнительные troubleshooting-команды

```
# Проверить health VMCluster
```

```
kubectl describe vmcluster -n <tenant-namespace>
```

```
# Вывести active alerts
```

```
kubectl get prometheusrules -n <tenant-namespace>
```

```
# Проверить network policies
```

```
kubectl get networkpolicies -n <tenant-namespace>
```

```
# Проверить RBAC permissions
```

```
kubectl auth can-i get pods --as=system:serviceaccount:<namespace>:<serviceaccount>
```

Эта документация покрывает все аспекты monitoring и alerting в Cozystack на основе анализа реального кода проекта.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование OpenID Connect с Cozystack

<https://cozystack.ru/docs/v1.6/operations/oidc/>

OIDC в Cozystack

-

Включение OIDC Server

Как включить OIDC Server

Создание пользователей и назначение ролей

Как создавать пользователей и назначать им роли

Self-Signed Certificates

Как настроить OIDC с self-signed certificates

Identity providers

Управление identity providers.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Включение OIDC Server

https://cozystack.ru/docs/v1.6/operations/oidc/enable_oidc/

Как включить OIDC Server

Предварительные требования

1. Конфигурация OIDC API server должен быть настроен на использование OIDC. Если используется Talos Linux, machine configuration должна включать следующие параметры:

cluster:

apiServer:

extraArgs:

oidc-issuer-url: "https://keycloak.example.org/realms/cozy"

oidc-client-id: "kubernetes"

oidc-username-claim: "preferred_username"

oidc-groups-claim: "groups"

Для Taln Добавьте в `values.yaml` в репозитории taln:

oidcIssuerUrl: "https://keycloak.<YOUR_ROOT_DOMAIN>/realms/cozy"

2. Доступность домена Убедитесь, что домен `keycloak.example.org` доступен из кластера и резолвится в root ingress controller.

3. Конфигурация storage Storage должен быть корректно настроен.

Конфигурация

Если все предварительные требования выполнены, можно переходить к настройке.

Шаг 1: включите OIDC в Cozystack

Пропатчите Platform Package, чтобы включить OIDC. Это также автоматически опубликует сервис Keycloak:

```
kubectrl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{
```

```
"spec": {
```

```
"components": {
```

```
"platform": {  
  "values": {  
    "authentication": {  
      "oidc": {  
        "enabled": true  
      }  
    }  
  }  
}
```

Если нужно добавить дополнительные redirect URLs для dashboard client (например, при доступе к dashboard через port-forwarding), пропатчите Platform Package. Несколько redirect URLs разделяются запятыми.

```
kubectrl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{  
  "spec": {  
    "components": {  
      "platform": {  
        "values": {  
          "authentication": {  
            "oidc": {  
              "keycloakExtraRedirectUri": "http://127.0.0.1:8080/oauth2/callback/,http://localhost:8080/oauth2/callback/  
/"  
            }  
          }  
        }  
      }  
    }  
  }  
}'
```

Опционально : если нужно, чтобы dashboard обращался к Keycloak через внутреннюю сеть кластера вместо external ingress, задайте `keycloakInternalUrl`. Это полезно в окружениях с self-signed certificates или ограниченным внешним доступом. Подробности см. в [Self-Signed Certificates](#).

В течение минуты CozyStack выполнит reconcile и создаст три новых ресурса HelmRelease:

```
# kubectl get hr -n cozy-keycloak
cozy-keycloak keycloak 26s Unknown Running 'install' action with a timeout of 5m0s
cozy-keycloak keycloak-configure 26s False dependency 'cozy-keycloak/keycloak-operator' is not ready
cozy-keycloak keycloak-operator 26s False dependency 'cozy-keycloak/keycloak' is not ready
```

Шаг 2: дождитесь завершения установки

Дождитесь, пока все ресурсы успешно установятся и перейдут в состояние Ready:

```
NAME AGE READY STATUS
keycloak 2m19s True Release reconciliation succeeded
keycloak-configure 2m19s True Release reconciliation succeeded
keycloak-operator 2m19s True Release reconciliation succeeded
```

Выполните reconcile tenants:

```
kubectl annotate -n tenant-root hr/tenant-root reconcile.fluxcd.io/forceAt=$(date
+"%Y-%m-%dT%H:%M:%SZ") --overwrite
```

Шаг 3: откройте Keycloak

Теперь Keycloak доступен по адресу <https://keycloak.example.org> (замените `example.org` на домен вашей инфраструктуры).

Чтобы получить credentials Keycloak для пользователя `admin` по умолчанию, выполните:

| | |
|---|--|
| <pre>kubectl get secret -o yaml -n cozy-keycloak keycloak-credentials -o go-template='{{ printf " %s\n" (index .data "password" base64decode) }}'</pre> | |
|---|--|

1. Переключите realm на `cozy`.

2. Создайте пользователя в realm `cozy`.

Создание пользователя в realm `cozy` описано в [документации Keycloak](#).

3. После создания пользователя перейдите к его details в Keycloak admin console и включите toggle "Verified email". Это нужно для корректной работы OIDC authentication.

4. Добавьте пользователя в группу `cozystack-cluster-admin`.

5. Теперь вы сможете войти в dashboard с OIDC credentials.

Если dashboard все еще запрашивает token вместо login/password, вручную выполните reconcile:

```
kubectl annotate -n cozy-dashboard hr/dashboard reconcile.fluxcd.io/forceAt=$(date
+"%Y-%m-%dT%H:%M:%SZ") --overwrite
```

Шаг 4: получите kubeconfig

Чтобы получить доступ к кластеру через Dashboard, скачайте kubeconfig: выберите развернутый tenant и скопируйте secret из resource map.

Этот kubeconfig будет автоматически настроен на использование OIDC authentication и namespace, выделенного tenant.

Установите [kubelogin](#), необходимый для использования kubeconfig с OIDC.

Homebrew (macOS and Linux)

```
brew install int128/kubelogin/kubelogin
```

Krew (macOS, Linux, Windows and ARM)

```
kubectl krew install oidc-login
```

Chocolatey (Windows)

```
choco install kubelogin
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Создание пользователей и назначение ролей

https://cozystack.ru/docs/v1.6/operations/oidc/users_and_roles/

Как создавать пользователей и назначать им роли

Создание пользователей и назначение им ролей.

Обзор

При создании tenant в Cozy (начиная с версии 1.6.0) roles, RoleBindings и группы Keycloak автоматически создаются в Kubernetes cluster.

Создание пользователя описано в документации: [Keycloak Admin Console Documentation](#)

Назначение роли пользователю для tenant

1. Откройте Keycloak : Чтобы получить login credentials, проверьте secret следующей командой:

```
kubectrl get secret keycloak-credentials -n cozy-keycloak -o yaml
```

Адрес Keycloak : Адрес Keycloak будет соответствовать значению `publishing.host`, указанному в Platform Package. Например, если Package содержит:

спес:

components:

platform:

values:

publishing:

host: "infra.example.org"

Тогда Keycloak будет доступен по адресу: `keycloak.infra.example.org`

Если вы планируете интеграцию с внешними сервисами как clients или как IdPs, адрес Keycloak должен быть публично доступен и достижим для этих сервисов.

Настройка ролей для каждого tenant в Cozy

На уровне кластера

```
- **cozystack-cluster-admin**
```

-
- Разрешено все.
 - **coystack-cluster-admin**
 - Разрешено все в api group ""
 - Разрешено все для helmreleases в helm.toolkit.fluxcd.io и apps.coystack.io

На уровне tenant

- **tenant-abc-view**
- Read-only доступ к ресурсам из нашего API.
- Возможность просматривать logs.
- **tenant-abc-use**
- Все предыдущие permissions.
- VNC access для virtual machines.
- **tenant-abc-admin**
- Все предыдущие permissions.
- Возможность удалять pods, а также все permissions из tenant-abc-use.
- Возможность создавать, обновлять и удалять ресурсы из нашего API, кроме tenant, monitoring, etcd, ingress.
- **tenant-abc-super-admin**
- Все предыдущие permissions.
- Возможность создавать, обновлять и удалять tenant, monitoring, etcd и ingress.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Self-Signed Certificates

<https://cozystack.ru/docs/v1.6/operations/oidc/self-signed-certificates/>

Как настроить OIDC с self-signed certificates

В этом руководстве описано, как настроить Kubernetes API server для OIDC authentication с Keycloak при использовании self-signed certificates. По умолчанию Cozystack выпускает certificates через LetsEncrypt, но в некоторых окружениях, например air-gapped или private enterprise networks, может использоваться custom CA.

Предварительные требования

- Cozystack cluster с включенным OIDC (см. [Enable OIDC Server](#))
- Control plane nodes на Talos Linux
- `talosctl`, настроенный для вашего кластера
- Установленный `kubelogin`

Шаг 1: получите сертификат Keycloak

Получите certificate из ingress controller:

```
echo | openssl s_client -connect <KEYCLOAK_INGRESS_IP>:443 \
```

```
-servername keycloak.example.org  
2>/dev/null
```

```
openssl x509
```

Замените `<KEYCLOAK_INGRESS_IP>` на IP-адрес ingress controller, а `keycloak.example.org` на фактический домен Keycloak.

Сохраните вывод (certificate между `-----BEGIN CERTIFICATE-----` и `-----END CERTIFICATE-----`) для следующего шага.

Шаг 2: настройте Talos control plane nodes

Для каждого control plane node добавьте следующее в machine configuration:

machine:

network:

extraHostEntries:

- ip: `<KEYCLOAK_INGRESS_IP>`

aliases:

- keycloak.example.org

files:

- content: |

-----BEGIN CERTIFICATE-----

<YOUR_CERTIFICATE_CONTENT>

-----END CERTIFICATE-----

permissions: 0o644

path: /var/oidc-ca.crt

op: create

cluster:

apiServer:

extraArgs:

oidc-issuer-url: https://keycloak.example.org/realms/cozy

oidc-client-id: kubernetes

oidc-username-claim: preferred_username

oidc-groups-claim: groups

oidc-ca-file: /etc/kubernetes/oidc/ca.crt

extraVolumes:

- hostPath: /var/oidc-ca.crt

mountPath: /etc/kubernetes/oidc/ca.crt

Примените конфигурацию к каждому control plane node:

talosctl apply-config -n <NODE_IP> -f nodes/<node>.yaml

Конфигурация `extraHostEntries` гарантирует, что домен Keycloak корректно резолвится внутри кластера, что важно при использовании `internal ingress IPs`.

Опционально: настройте internal Keycloak URL для Dashboard

По умолчанию `oauth2-proxy` Cozystack Dashboard подключается к Keycloak через `external ingress URL`. В окружениях с `self-signed certificates` или ограниченным внешним доступом можно настроить dashboard на использование внутреннего `cluster service` Keycloak для `backend requests` (token exchange, JWKS validation, userinfo, logout), сохранив `browser redirects` на внешний URL.

Пропатчите Platform Package:

kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=merge -p '{

```
"spec": {
  "components": {
    "platform": {
      "values": {
        "authentication": {
          "oidc": {
            "keycloakInternalUrl": "http://keycloak-http.cozy-keycloak.svc:8080/realms/cozy"
          }
        }
      }
    }
  }
}
```

Это влияет только на oauth2-proxy dashboard (pod-to-pod communication). Kubernetes API server все равно требует `extraHostEntries` для доступа к Keycloak, потому что `kube-apiserver` использует host-level DNS и не может резолвить cluster service names.

Шаг 3: настройте kubelogin

Установите kubelogin, если он еще не установлен:

```
# Homebrew (macOS and Linux)
```

```
brew install int128/kubelogin/kubelogin
```

```
# Krew (macOS, Linux, Windows and ARM)
```

```
kubectrl krew install oidc-login
```

```
# Chocolatey (Windows)
```

```
choco install kubelogin
```

Save the CA certificate from Step 1 to a file on your local machine:

```
# Save the certificate to a file (e.g., ~/.kube/oidc-ca.pem)
```

```
cat > ~/.kube/oidc-ca.pem <<EOF
```

```
-----BEGIN CERTIFICATE-----
```

```
<YOUR_CERTIFICATE_CONTENT>
```

```
-----END CERTIFICATE-----
```

EOF

Set up OIDC login (this will open a browser for authentication):

```
kubectrl oidc-login setup \  
--oidc-issuer-url=https://keycloak.example.org/realms/cozy \  
--oidc-client-id=kubernetes \  
--certificate-authority=~/.kube/oidc-ca.pem
```

Configure kubectrl credentials:

```
kubectrl config set-credentials oidc \  
--exec-api-version=client.authentication.k8s.io/v1 \  
--exec-interactive-mode=IfAvailable \  
--exec-command=kubectrl \  
--exec-arg=oidc-login \  
--exec-arg=get-token \  
--exec-arg="--oidc-issuer-url=https://keycloak.example.org/realms/cozy" \  
--exec-arg="--oidc-client-id=kubernetes" \  
--exec-arg="--certificate-authority=~/.kube/oidc-ca.pem"
```

Переключитесь на OIDC user и проверьте:

```
kubectrl config set-context --current --user=oidc  
kubectrl get nodes
```

Если CA вашей организации уже установлен в system trust store (часто встречается в enterprise environments), флаг `--certificate-authority` можно полностью опустить: kubelogin автоматически использует system CA bundle.

Избегайте `--insecure-skip-tls-verify`. Если вы не можете установить CA certificate на свою машину или передать его через `--certificate-authority`, можно временно использовать `--insecure-skip-tls-verify`, но это отключает TLS verification и не рекомендуется для production.

Устранение неполадок

Проверьте OIDC logs API Server

```
kubectrl logs -n kube-system -l  
component=kube-apiserver --tail=50
```

```
grep oidc
```

Проверьте, что OIDC flags применены

kubect! get pods -n kube-system -l component=kube-apiserver \

| | | |
|--|-------------|-----------|
| -o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep oidc |
|--|-------------|-----------|

Типичные проблемы

- Certificate not found : убедитесь, что path certificate file в `extraVolumes` совпадает с path, указанным в `oidc-ca-file`.
- Domain resolution fails : проверьте, что `extraHostEntries` корректно настроен на всех control plane nodes.
- Authentication fails : проверьте, что пользователь существует в Keycloak и состоит в нужных groups (см. [Users and Roles](#)).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Identity providers

https://cozystack.ru/docs/v1.6/operations/oidc/identity_providers/

Управление identity providers.

-

[Как настроить GitLab как Identity Provider](#)

Как настроить GitLab как Identity Provider

[Как настроить Google как Identity Provider](#)

Как настроить Google как Identity Provider

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как настроить GitLab как Identity Provider

https://cozystack.ru/docs/v1.6/operations/oidc/identity_providers/gitlab/

Как настроить GitLab как Identity Provider

GitLab можно использовать как identity provider для Keycloak.

Обзор

Создайте Application в GitLab

- Откройте https://gitlab.com/groups/<YOUR_GROUP>/-/settings/applications
- Нажмите Add new application
- Name: cozy, Redirect URI: <https://keycloak.<root-host>/realms/cozy/broker/gitlab/endpoint>
- Включите Confidential, api, read_api, read_user, openid, profile, email
- Скопируйте и сохраните Secret

Настройте Keycloak Identity Provider

Создайте ресурс KeycloakRealmIdentityProvider со следующей конфигурацией:

```
apiVersion: v1.edp.epam.com/v1
kind: KeycloakRealmIdentityProvider
metadata:
  name: gitlab
spec:
  realmRef:
    name: keycloakrealm-cozy
    kind: ClusterKeycloakRealm
  alias: gitlab
  authenticateByDefault: false
  enabled: true
  providerId: "gitlab"
  config:
    clientId: "YOUR GITLAB APP ID"
    clientSecret: "YOUR GITLAB APP SECRET"
    syncMode: "IMPORT"
```

mappers:

- name: "username"

identityProviderMapper: "oidc-username-idp-mapper"

identityProviderAlias: "gitlab"

config:

target: "LOCAL"

syncMode: "INHERIT"

template: "\${ALIAS}---\${CLAIM.preferred_username}"

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как настроить Google как Identity Provider

https://cozystack.ru/docs/v1.6/operations/oidc/identity_providers/google/

Как настроить Google как Identity Provider

Настройте Google

- Перейдите в [Google Console](#), войдите в консоль с Google account, после чего откроется Google Developer Console. После входа используйте выпадающее меню вверху слева, чтобы создать новый project.
- Нажмите "New Project", чтобы продолжить.
- Введите имя project и выберите Organisation, если у вас несколько organisations. Затем нажмите "Create".
- После создания project появится pop-up с предложением настроить consent screen. Если он не появился, перейдите в Dashboard и откройте "Explore and enable APIs". Затем нажмите "Credentials" > "Configure Consent Screen" и переходите к следующему шагу.
- Нажмите "External", так как нужно разрешить вход в приложение любому Google account, затем нажмите "Create".
- После этого вы будете перенаправлены на страницы, где нужно настроить несколько параметров:
 - Application type: Public
 - Application name: Your application name
 - Authorised domains: Your application top-level domain name
 - Application Homepage link: Your application homepage
 - Application Privacy Policy link: Your application privacy policy link
- Теперь перейдите к Create Credentials в navbar и нажмите "OAuth Client ID".
- Выберите Application type "Web application" и задайте имя приложения. Затем добавьте ссылку из вкладки Keycloak в "Authorized Redirect URIs" и нажмите "Create". Ссылка должна выглядеть примерно так:

`https://YOUR_KEYCLOAK_DOMAIN/auth/realms/cozy/broker/google/endpoint`

- После завершения появится pop-up с информацией, нужной на следующем шаге. Вам понадобятся "Client ID" и "Client secret", поэтому сохраните их в безопасном месте.

Настройте Keycloak Identity Provider

Создайте ресурс `KeycloakRealmIdentityProvider` со следующей конфигурацией:

```
apiVersion: v1.edp.epam.com/v1
kind: KeycloakRealmIdentityProvider
metadata:
name: google
spec:
realmRef:
name: keycloakrealm-cozy
kind: ClusterKeycloakRealm
alias: google
authenticateByDefault: false
enabled: true
providerId: "google"
config:
clientId: "YOUR GOOGLE APP ID"
clientSecret: "YOUR GOOGLE APP SECRET"
syncMode: "IMPORT"
mappers:
  - name: "username"
identityProviderMapper: "oidc-username-idp-mapper"
identityProviderAlias: "google"
config:
target: "LOCAL"
syncMode: "INHERIT"
template: "${ALIAS}---${CLAIM.email}"
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Руководства для нескольких дата-центров

<https://cozystack.ru/docs/v1.6/operations/stretched/>

Как запускать Cozystack в растянутом Kubernetes-кластере

Эти руководства показывают, как настроить кластер для запуска Cozystack на узлах, расположенных в разных дата-центрах.

-

Настройка таймаутов etcd

Параметры, необходимые для работы etcd в растянутом кластере

Конфигурация kube-scheduler

Конфигурация kube-scheduler

Метки узлов

Добавьте метки на узлы, чтобы workloads планировались корректно

Конфигурация LINSTOR DRBD

Параметры, необходимые для работы LINSTOR в растянутом кластере

Настройка выделенной сети для распределенного хранилища с LINSTOR

Настройте LINSTOR так, чтобы он предпочитал локальные каналы для хранилища и переходил на междатацентровые соединения при необходимости

Конфигурация SeaweedFS Multi-DC

Как развернуть SeaweedFS в нескольких дата-центрах

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Настройка таймаутов etcd

<https://cozystack.ru/docs/v1.6/operations/stretched/etcd/>

Параметры, необходимые для работы etcd в растянутом кластере

Возможные проблемы

etcd \- надежное key-value хранилище для критически важных данных, где Kubernetes API server хранит свое состояние. Значения не считаются записанными, пока кворум узлов не подтвердит запись данных. Кроме того, etcd постоянно проверяет, что в кластере есть лидер и каждый узел знает, какой именно. Обычно кластер etcd запускается на узлах в одном дата-центре, где задержка мала, а таймауты по умолчанию настроены на максимально быстрое сообщение об ошибках. В растянутом кластере, где узлы находятся в разных дата-центрах, RTT значительно выше, и стандартные таймауты могут вызывать ложные срабатывания, как при разделении сети. В этом случае данные остаются согласованными, но etcd перейдет в readonly-режим, чтобы предотвратить split-brain. Если это происходит часто, многие другие компоненты Kubernetes начинают падать с неинформативными сообщениями об ошибках.

Конфигурация

Чтобы избежать частых перевыборов лидера, настройте в конфигурации etcd следующие параметры:

- `heartbeat-interval` \- интервал между heartbeat
- `election-timeout` \- время ожидания heartbeat перед запуском выборов

Пример аргументов командной строки для etcd:

```
--election-timeout=10000 --heartbeat-interval=1000
```

Пример для Talos

Если вы управляете узлами Talos через `tal`, добавьте параметры в секцию `cluster.etcd` шаблона:

cluster:

etcd:

extraArgs:

heartbeat-interval: "1000"

election-timeout: "10000"

Конфигурация kube-scheduler

<https://cozystack.ru/docs/v1.6/operations/stretched/kubeschedulerconfiguration/>

Конфигурация kube-scheduler

Добавьте метки на узлы

kubectl label node <nodename> topology.kubernetes.io/zone=A

Создайте глобальные topology spread constraints для приложений

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cozystack-scheduling
  namespace: cozy-system
data:
  globalAppTopologySpreadConstraints: |
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
```

Настройте PodTopologySpread

См.: <<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/#cluster-level-default-constraints>>

Для установки через Talm добавьте в `templates/_helpers.tpl`:

```
cluster:
...
scheduler:
  config:
    apiVersion: kubescheduler.config.k8s.io/v1beta3
```

kind: KubeSchedulerConfiguration

profiles:

- schedulerName: default-scheduler

pluginConfig:

- name: PodTopologySpread

args:

defaultConstraints:

- maxSkew: 1

topologyKey: topology.kubernetes.io/zone

whenUnsatisfiable: ScheduleAnyway

defaultingType: List

Примените изменения:

talm template -f nodes/node1.yaml -I

talm apply -f nodes/node1.yaml

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Метки узлов

<https://cozystack.ru/docs/v1.6/operations/stretched/labels/>

Добавьте метки на узлы, чтобы workloads планировались корректно

Как работают топологические метки

При запуске Kubernetes-кластера в нескольких дата-центрах важно понимать, какие workloads нужно размещать рядом друг с другом, например базу данных и backend-приложение, а какие - распределять по разным площадкам, например несколько реплик frontend, реплики базы данных или реплики томов. Первый шаг к этому - добавить метки на узлы Kubernetes. В публичных облаках обычно используются термины `zone` и `region`. В Kubernetes наиболее распространенный способ обозначить географическое расположение - использовать метки `topology.kubernetes.io/zone` и `topology.kubernetes.io/region` или только метку `zone`.

Пример для Talos

В других дистрибутивах Kubernetes метки обычно задаются вручную, но в Talos топологию можно описать как код.

Пример конфигурации `talm`:

```
values.yaml:

nodes:
node1:
labels:
topology.kubernetes.io/zone: "us-west-1"
topology.kubernetes.io/region: "us-west"
node2:
labels:
topology.kubernetes.io/zone: "us-east-1"
topology.kubernetes.io/region: "us-east"

_helpers.tpl:

{{- $nodeLabels := .Values.nodes | dig (include "talm.discovered.hostname" .) "labels" nil }}
machine:
{{- with $nodeLabels }}
```

nodeLabels:

```
{{- toYaml . | nindent 4 }}
```

```
{{- end }}
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Конфигурация LINSTOR DRBD

<https://cozystack.ru/docs/v1.6/operations/stretched/drbd-tuning/>

Параметры, необходимые для работы LINSTOR в растянутом кластере

Введение

В этом руководстве описана конфигурация, необходимая для использования хранилища LINSTOR в растянутом (распределенном) кластере Cozystack.

DRBD (Distributed Replicated Block Device) - система репликации блочных устройств на уровне ядра, работающая по сети. Сервер LINSTOR управляет томами DRBD, включая их создание, удаление и оркестрацию между узлами.

Сложности при использовании DRBD

DRBD считает данные записанными только после того, как они достигли кворума узлов. Но поскольку для конечного пользователя DRBD выглядит как блочное устройство, при нехватке узлов для кворума он должен вернуть ошибку в пределах заданного таймаута.

Проблема в том, что таймауты по умолчанию рассчитаны на локальные сети с высокой пропускной способностью и низкой задержкой. При обмене между дата-центрами подтверждение от удаленного узла может приходить долго из-за сетевой перегрузки. Это похоже на поведение `etcd` в растянутых условиях, где стандартные таймауты могут приводить к ложным отказам кворума.

Если хотя бы одно устройство DRBD сообщает о потере кворума, Piraeus HA controller выполнит fencing узла, чтобы предотвратить сбой других workloads. Это может привести к workloads, которые невозможно запланировать, и даже к шторму ребалансировки.

Конфигурация

Самый эффективный подход - задать глобальные параметры соединения для кластера LINSTOR командой `linstor controller drbd-options`. Она сразу применяет настройки ко всем существующим ресурсам DRBD без индивидуальной настройки или перезапуска:

Также применяется к существующим ресурсам DRBD

```
linstor controller drbd-options --connect-int 15 --ping-int 15 --ping-timeout 20 --drbd-timeout 120
```

Эти значения рассчитаны на междатацентровые окружения, где задержка выше, чем в обычной локальной сети.

| Параметр | Значение | Значение по умолчанию | Рекомендуемое значение |
|-----------------------------|---|-----------------------|------------------------|
| <code>--connect-int</code> | Интервал между попытками TCP-соединения, в секундах. | 10 | 15 |
| <code>--ping-int</code> | Интервал между keeralive ping, в секундах. | 10 | 15 |
| <code>--ping-timeout</code> | Время ожидания ответа на ping перед тем, как реер считается недоступным, в десятых долях секунды. | 5 | 20 |
| <code>--drbd-timeout</code> | Максимальное время ожидания сетевого ответа до срабатывания таймаута, в десятых долях секунды. | 60 | 120 |

Корректировка этих параметров помогает избежать лишнего fencing и прерывания workloads в растянутых кластерах.

Также обратите внимание на руководство по [общей настройке DRBD](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Настройка выделенной сети для распределенного хранилища с LINSTOR

<https://cozystack.ru/docs/v1.6/operations/stretched/linstor-dedicated-network/>

Настройте LINSTOR так, чтобы он предпочитал локальные каналы для хранилища и переходил на междатацентровые соединения при необходимости

Введение

В этом руководстве описано, как повысить надежность и производительность хранилища в распределенных кластерах Cozystack.

В гиперконвергентных кластерах для трафика хранилища обычно выделяют отдельную сеть. Однако не всегда возможно выделить отдельные storage-каналы между дата-центрами.

Если для хранилища нет выделенных междатацентровых каналов, есть два варианта:

- изолировать storage-узлы в каждом дата-центре;
- передавать storage-трафик через существующие uplink вместе с другими workloads.

В этом руководстве показано, как настроить LINSTOR на использование выделенной сети для storage-трафика внутри каждого дата-центра и при необходимости переключаться на общие каналы между дата-центрами.

Предварительные требования

Это руководство опирается на руководство [Выделенная сеть для LINSTOR](#) и добавляет методы и шаблоны конфигурации, характерные для окружений с несколькими дата-центрами. Чтобы применять описанные здесь шаблоны, важно понимать, как работают интерфейсы узлов и пути соединений. Сначала ознакомьтесь с предыдущим руководством: в нем эти концепции описаны подробно.

Чтобы применять разные настройки соединений в зависимости от расположения узла, добавьте на узлы соответствующие метки. Инструкции см. в руководстве [Топологические метки узлов](#). В этом руководстве для различения дата-центров используется метка `topology.kubernetes.io/zone`.

Конфигурация соединений

В этом примере есть три дата-центра: `dc1`, `dc2` и `dc3`. Дата-центры соединены прямыми оптическими линиями, а интерфейс называется `region10g`. Узлы в дата-центрах `dc1` и `dc2` имеют отдельный сетевой коммутатор и сетевые интерфейсы `san` только для storage-трафика. В дата-центре `dc3` нет выделенного сетевого коммутатора для хранилища, и весь трафик между

узлами в `dc3` маршрутизируется через сеть по умолчанию. Для подключения к другим дата-центрам используется VPN-сервер, подключенный к оптическим линиям. На узлах в `dc3` есть VLAN-интерфейс с IP из подсети `region10g`.

Рассмотрим сценарий с тремя дата-центрами: `dc1`, `dc2` и `dc3`.

- Дата-центры соединены прямыми оптическими линиями, которые доступны узлам как интерфейс `region10g`.
- Узлы в `dc1` и `dc2` подключены к выделенному сетевому коммутатору для хранилища и используют отдельный интерфейс `san` только для storage-трафика.
- В дата-центре `dc3` нет выделенной storage-сети. Весь трафик внутри `dc3` идет через сеть по умолчанию.

Для такой схемы оптимальная маршрутизация storage-трафика выглядит так:

- Узлы в `dc1` и `dc2` используют интерфейс `san` для локальной репликации хранилища.
- Узлы в `dc3` используют сетевой интерфейс по умолчанию для локальной репликации хранилища, избегая лишних переходов через VPN-сервер.
- Междатацентровый storage-трафик идет через интерфейс `region10g`.

Ниже custom resource definition (CRD) LINSTOR, который реализует эту логику:

```
apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
name: intra-datacenter
spec:
selector:
  - matchLabels:
  - key: topology.kubernetes.io/zone
op: Same
  - key: topology.kubernetes.io/zone
op: In
values:
# Только в этих дата-центрах есть интерфейс san
  - dc1
  - dc2
paths:
  - name: intra-datacenter
interface: san

# Узлы в dc3 используют сеть по умолчанию.
```

Для репликации внутри этого дата-центра отдельное соединение не требуется.

```
apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
name: cross-datacenter
spec:
selector:
  - matchLabels:
  - key: topology.kubernetes.io/zone
op: NotSame
paths:
  - name: cross-datacenter
interface: region10g
```

После применения этой конфигурации можно проверить получившиеся пути соединений:

```
LINSTOR ==> node-connection list
+-----+
: Node A : Node B : Properties :
+=====+
: node01 : node02 : last-applied=["Paths/intra-dat... :
: node01 : node03 : last-applied=["Paths/cross-dat... :
: node01 : node04 : last-applied=["Paths/cross-dat... :
....
```

Пары узлов внутри дата-центра `dc3` не будут иметь настроенного custom node connection и будут использовать интерфейс по умолчанию.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Конфигурация SeaweedFS Multi-DC

<https://cozystack.ru/docs/v1.6/operations/stretched/seaweedfs-multidc/>

Как развернуть SeaweedFS в нескольких дата-центрах

В этом руководстве описано, как развернуть SeaweedFS в нескольких дата-центрах ("multi-DC"). Multi-zone конфигурация SeaweedFS доступна начиная с Cozystack v0.34.0.

Конфигурация SeaweedFS Multi-DC

Чтобы растянуть SeaweedFS на несколько DC, создайте новый кластер в режиме multi-DC.

По умолчанию SeaweedFS работает в одном дата-центре (DC), и уже запущенное single-DC-развертывание нельзя переключить в multi-DC-режим. Если нужно изменить топологию, удалите текущий экземпляр SeaweedFS и создайте новый с нужным режимом.

Удобный порядок действий:

1. Разверните tenant с `seaweedfs: false`.
2. Создайте новый экземпляр SeaweedFS в namespace tenant, используя нужную топологию.
3. Обновите tenant, установив `seaweedfs: true`.

1\ Создать tenant без SeaweedFS

Поле `isolated`, которое в ранних релизах Cozystack было доступно в объекте Tenant, удалено в v1.0. Текущая модель сетевой изоляции описана в upgrade notes: [флаг `isolated` у Tenant удален](<https://cozystack.ru/docs/v1.6/operations/upgrades/#tenant-isolated-flag-removed>).

apiVersion: apps.cozystack.io/v1alpha1

kind: Tenant

metadata:

name: dev

namespace: tenant-root

spec:

etcd: false

host: ""

ingress: false

monitoring: false

seaweedfs: false

2\ Разверните SeaweedFS в режиме Multi-Zone

apiVersion: apps.cozystack.io/v1alpha1

kind: SeaweedFS

metadata:

name: seaweedfs

namespace: tenant-dev

spec:

host: dev.s3.example.org

replicas: 4

size: 100Gi

topology: MultiZone

zones:

dc1: {}

dc2: {}

dc3: {}

В этом примере SeaweedFS запускает 12 volume server: 4 реплики на 3 зоны.

Параметр `zones` перечисляет каждый дата-центр. Любая настройка, пропущенная внутри зоны, наследует значения верхнего уровня: `replicas`, `size` и другие.

3\ Включите SeaweedFS в tenant

Когда экземпляр SeaweedFS будет готов, включите его в tenant. Примените следующее обновленное описание ресурса:

apiVersion: apps.cozystack.io/v1alpha1

kind: Tenant

metadata:

name: dev

namespace: tenant-root

spec:

etcd: false

host: ""

ingress: false

monitoring: false

seaweedfs: true

Теперь создание Bucket доступно для всего дерева этого tenant.

Использование удаленного экземпляра SeaweedFS

Можно опубликовать одно развертывание SeaweedFS для других кластеров Cozystack и разрешить им подключаться в режиме Client.

1\ Экспортируйте SeaweedFS

Задайте gRPC endpoint filer и список сетей, которым разрешено подключение:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
  name: seaweedfs
  namespace: tenant-dev
spec:
  host: dev.s3.example.org
  replicas: 4
  size: 100Gi
  topology: MultiZone
  zones:
    dc1: {}
    dc2: {}
    dc3: {}
  filer:
    grpcHost: filer-dev.s3.example.org
  whitelist:
    - 0.0.0.0/0 # открыть для всех сетей (настройте для production)
```

2\ Настройте удаленный SeaweedFS

Подключите удаленный экземпляр SeaweedFS из другого кластера:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: SeaweedFS
metadata:
  name: seaweedfs
```

spec:
host: dev.s3.example.org
topology: Client
filer:
grpcHost: filer-dev.s3.example.org

3\ Включите доступ к удаленному SeaweedFS

Чтобы локальный кластер мог аутентифицироваться в удаленном SeaweedFS filer, экспортируйте `secret seaweedfs-client-cert` из удаленного кластера:

```
kubectl get secret seaweedfs-client-cert -n tenant-dev -o yaml \
> seaweedfs-client-cert.yaml
```

Откройте `seaweedfs-client-cert.yaml` и удалите поля `namespace`, `labels` и `annotations`. Они относятся к удаленному кластеру и не должны применяться локально. После очистки manifest должен выглядеть примерно так:

```
apiVersion: v1
kind: Secret
type: kubernetes.io/tls
metadata:
name: seaweedfs-client-cert
data:
tls.crt: ...
tls.key: ...
```

Примените secret в целевой namespace локального кластера:

```
kubectl apply -f seaweedfs-client-cert.yaml -n tenant-root
```

Примечание: В режиме Client кластер не создает volume server, а просто переиспользует удаленный экземпляр SeaweedFS для всех операций с bucket.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Multi-Location кластеры

<https://cozystack.ru/docs/v1.6/operations/multi-location/>

Расширение Cozystack management clusters на несколько locations с помощью Kilo WireGuard mesh, cloud autoscaling и local cloud controller manager.

В этом разделе описано расширение Cozystack management cluster на несколько физических locations (on-premises + cloud, multi-cloud и т. д.) с помощью WireGuard mesh networking.

Настройка состоит из трех компонентов:

- [Networking Mesh](#) – Kilo WireGuard mesh с Cilium IPIP encapsulation
- [Local CCM](#) – cloud controller manager для определения IP узлов и lifecycle
- [Cluster Autoscaling](#) – автоматическое создание узлов в cloud providers

•

[Networking Mesh](#)

Настройка Kilo WireGuard mesh с Cilium для связности multi-location cluster.

[Local Cloud Controller Manager](#)

Определение IP узлов и lifecycle management для multi-location clusters.

[Cluster Autoscaling](#)

Автоматическое масштабирование узлов Cozystack management clusters с помощью Kubernetes Cluster Autoscaler.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Cluster Autoscaling

<https://cozystack.ru/docs/v1.6/operations/multi-location/autoscaling/>

Автоматическое масштабирование узлов Cozystack management clusters с помощью Kubernetes Cluster Autoscaler.

System package `cluster-autoscaler` включает автоматическое масштабирование узлов для Cozystack management clusters. Он мониторит pending pods и автоматически создает или удаляет cloud nodes в зависимости от нагрузки.

Перед настройкой autoscaling завершите настройку [Networking Mesh](#) и [Local CCM](#).

Cozystack предоставляет pre-configured variants для разных cloud providers:

- [Hetzner Cloud](#) – масштабирование с помощью Hetzner Cloud servers
- [Azure](#) – масштабирование с помощью Azure Virtual Machine Scale Sets

Каждый variant разворачивается как отдельный Cozystack Package с provider-specific configuration.

•

[Cluster Autoscaler для Hetzner Cloud](#)

Настройка автоматического масштабирования узлов в Hetzner Cloud с Talos Linux.

[Cluster Autoscaler для Azure](#)

Настройка автоматического масштабирования узлов в Azure с Talos Linux и VMSS.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Cluster Autoscaler для Azure

<https://cozystack.ru/docs/v1.6/operations/multi-location/autoscaling/azure/>

Настройка автоматического масштабирования узлов в Azure с Talos Linux и VMSS.

В этом руководстве описано, как настроить cluster-autoscaler для автоматического масштабирования узлов в Azure с Talos Linux.

Предварительные требования

- Подписка Azure с Service Principal, имеющим роль Contributor
- Установленный CLI az
- Существующий Kubernetes-кластер на Talos
- Настроенные [Networking Mesh](#) и [Local CCM](#)

Шаг 1. Создание инфраструктуры Azure

1.1. Вход через Service Principal

```
az login --service-principal \  
--username "<APP_ID>" \  
--password "<PASSWORD>" \  
--tenant "<TENANT_ID>"
```

1.2. Создание группы ресурсов

```
az group create \  
--name <resource-group> \  
--location <location>
```

1.3. Создание VNet и подсети

```
az network vnet create \  
--resource-group <resource-group> \  
--name cozystack-vnet \  
--address-prefix 10.2.0.0/16 \  
--subnet-name workers \
```

```
--subnet-prefix 10.2.0.0/24 \  
--location <location>
```

1.4. Создание Network Security Group

```
az network nsg create \  
--resource-group <resource-group> \  
--name cozystack-nsg \  
--location <location>
```

Разрешить WireGuard

```
az network nsg rule create \  
--resource-group <resource-group> \  
--nsg-name cozystack-nsg \  
--name AllowWireGuard \  
--priority 100 \  
--direction Inbound \  
--access Allow \  
--protocol Udp \  
--destination-port-ranges 51820
```

Разрешить Talos API

```
az network nsg rule create \  
--resource-group <resource-group> \  
--nsg-name cozystack-nsg \  
--name AllowTalosAPI \  
--priority 110 \  
--direction Inbound \  
--access Allow \  
--protocol Tcp \  
--destination-port-ranges 50000
```

Связать NSG с подсетью

```
az network vnet subnet update \  
--resource-group <resource-group> \  
--vnet-name cozystack-vnet \  
--name workers \  
--network-security-group cozystack-nsg
```

Шаг 2. Создание образа Talos

2.1. Генерация Schematic ID

Создайте schematic на factory.talos.dev с нужными расширениями:

```
curl -s -X POST https://factory.talos.dev/schematics \
-H "Content-Type: application/json" \
-d '{
  "customization": {
    "systemExtensions": {
      "officialExtensions": [
        "siderolabs/amd-ucode",
        "siderolabs/amdgpu-firmware",
        "siderolabs/bnx2-bnx2x",
        "siderolabs/drbd",
        "siderolabs/i915-ucode",
        "siderolabs/intel-ice-firmware",
        "siderolabs/intel-ucode",
        "siderolabs/qlogic-firmware",
        "siderolabs/zfs"
      ]
    }
  }
}'
```

Сохраните возвращенный `id` как `SCHEMATIC_ID`.

2.2. Создание Managed Image из VHD

Скачать образ Talos для Azure

```
curl -L -o azure-amd64.raw.xz \
"https://factory.talos.dev/image/${SCHEMATIC_ID}/<talos-version>/azure-amd64.raw.xz"
```

Распаковать

```
xz -d azure-amd64.raw.xz
```

```
# Конвертировать в VHD
qemu-img convert -f raw -o subformat=fixed,force_size -O vpc \
azure-amd64.raw azure-amd64.vhd

# Получить размер VHD
VHD_SIZE=$(stat -f%z azure-amd64.vhd) # macOS
# VHD_SIZE=$(stat -c%s azure-amd64.vhd) # Linux

# Создать managed disk для загрузки
az disk create \
--resource-group <resource-group> \
--name talos-<talos-version> \
--location <location> \
--upload-type Upload \
--upload-size-bytes $VHD_SIZE \
--sku Standard_LRS \
--os-type Linux \
--hyper-v-generation V2

# Получить SAS URL для загрузки
SAS_URL=$(az disk grant-access \
--resource-group <resource-group> \
--name talos-<talos-version> \
--access-level Write \
--duration-in-seconds 3600 \
--query accessSAS --output tsv)

# Загрузить VHD
azcopy copy azure-amd64.vhd "$SAS_URL" --blob-type PageBlob

# Отозвать доступ
az disk revoke-access \
--resource-group <resource-group> \
--name talos-<talos-version>

# Создать managed image из диска
az image create \
--resource-group <resource-group> \
--name talos-<talos-version> \
```

```
--location <location> \  
--os-type Linux \  
--hyper-v-generation V2 \  
--source $(az disk show --resource-group <resource-group> \  
--name talos-<talos-version> --query id --output tsv)
```

Шаг 3. Создание Talos machine config для Azure

В репозитории кластера сгенерируйте конфигурационный файл worker-узла:

```
talm template -t templates/worker.yaml --offline --full > nodes/azure.yaml
```

Затем отредактируйте `nodes/azure.yaml` для worker-узлов Azure:

1. Добавьте метаданные локации Azure (см. [Networking Mesh](#)):

```
machine:  
nodeAnnotations:  
kilo.squat.ai/location: azure  
kilo.squat.ai/persistent-keepalive: "20"  
nodeLabels:  
topology.kubernetes.io/zone: azure
```

2. Укажите публичный endpoint Kubernetes API: измените `cluster.controlPlane.endpoint` на публичный адрес API-сервера, например `https://<public-api-ip>:6443`. Этот адрес можно найти в `kubecfg` или опубликовать через `ingress`.

3. Удалите автоматически обнаруженные секции `installer/network`: удалите из файла секции `machine.install` и `machine.network`.

4. Укажите external cloud provider для kubelet (см. [Local CCM](#)):

```
machine:  
kubelet:  
extraArgs:  
cloud-provider: external
```

5. Настройте определение подсети IP-адресов узлов: укажите в `machine.kubelet.nodeIP.validSubnets` фактическую подсеть Azure, в которой будут работать автомасштабируемые узлы, например `192.168.102.0/23`.

6. Необязательно: добавьте registry mirrors, чтобы избежать rate limiting в Docker Hub:

```
machine:
```

registries:

mirrors:

docker.io:

endpoints:

- https://mirror.gcr.io

В результате конфигурация должна включать как минимум:

machine:

nodeAnnotations:

kilo.squat.ai/location: azure

kilo.squat.ai/persistent-keepalive: "20"

nodeLabels:

topology.kubernetes.io/zone: azure

kubelet:

nodeIP:

validSubnets:

- 192.168.102.0/23 # замените на подсеть worker-узлов Azure

extraArgs:

cloud-provider: external

registries:

mirrors:

docker.io:

endpoints:

- https://mirror.gcr.io

cluster:

controlPlane:

endpoint: https://<public-api-ip>:6443

Все остальные параметры (токены кластера, CA, расширения и т. д.) остаются такими же, как в сгенерированном шаблоне.

Шаг 4. Создание VMSS (Virtual Machine Scale Set)

```
IMAGE_ID=$(az image show \
```

```
--resource-group <resource-group> \
```

```
--name talos-<talos-version> \
```

```
--query id --output tsv)

az vmss create \
--resource-group <resource-group> \
--name workers \
--location <location> \
--orchestration-mode Uniform \
--image "$IMAGE_ID" \
--vm-sku Standard_D2s_v3 \
--instance-count 0 \
--vnet-name cozystack-vnet \
--subnet workers \
--public-ip-per-vm \
--custom-data nodes/azure.yaml \
--security-type Standard \
--admin-username talos \
--authentication-type ssh \
--generate-ssh-keys \
--upgrade-policy-mode Manual

# Включить IP forwarding на NIC в VMSS (нужно, чтобы лидер Kilo пересылал трафик)
az vmss update \
--resource-group <resource-group> \
--name workers \
--set virtualMachineProfile.networkProfile.networkInterfaceConfigurations[0].enableIPForwarding=true
```

Важно

- Обязательно используйте `--orchestration-mode Uniform`: cluster-autoscaler требует режим Uniform.
- Обязательно используйте `--public-ip-per-vm` для связности WireGuard.
- На NIC в VMSS должен быть включен IP forwarding, чтобы лидер Kilo мог пересылать трафик между WireGuard mesh и не-лидерными узлами в той же подсети.
- Проверьте квоту VM в своем регионе: `az vm list-usage --location <location>`.
- `--custom-data` передает Talos machine config новым инстансам.

Шаг 5. Развертывание Cluster Autoscaler

Создайте ресурс Package:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cluster-autoscaler-azure
spec:
variant: default
components:
cluster-autoscaler-azure:
values:
cluster-autoscaler:
azureClientID: "<APP_ID>"
azureClientSecret: "<PASSWORD>"
azureTenantID: "<TENANT_ID>"
azureSubscriptionID: "<SUBSCRIPTION_ID>"
azureResourceGroup: "<RESOURCE_GROUP>"
azureVMType: "vmss"
autoscalingGroups:
- name: workers
minSize: 0
maxSize: 10
```

Примените манифест:

```
kubectl apply -f package.yaml
```

Шаг 6. Связность Kilo WireGuard

Узлы Azure находятся за NAT, поэтому их начальный WireGuard endpoint будет приватным IP-адресом. Kilo обрабатывает это автоматически через встроенный в WireGuard NAT traversal, если настроен `persistent-keepalive` (он уже добавлен в machine config на шаге 3).

Процесс работает так:

1. Узел Azure инициирует WireGuard handshake с on-premises лидером, у которого есть публичный IP-адрес.
2. `persistent-keepalive` периодически отправляет keepalive-пакеты и поддерживает NAT mapping.
3. On-premises лидер Kilo через WireGuard определяет реальный публичный endpoint узла Azure.
4. Kilo сохраняет обнаруженный endpoint и использует его для последующих подключений.

Примечание

Ручная аннотация `force-endpoint` не нужна. Аннотации `kilo.squat.ai/persistent-keepalive: "20"` в `machine config` достаточно, чтобы Kilo автоматически обнаруживал NAT endpoints. Без этой аннотации механизм NAT traversal в Kilo отключен, и туннель не стабилизируется.

Проверка

Ручная проверка масштабирования

Увеличить размер группы

```
az vmss scale --resource-group <resource-group> --name workers --new-capacity 1
```

Проверить, что узел присоединился

```
kubecttl get nodes -o wide
```

Проверить туннель WireGuard

```
kubecttl logs -n cozy-kilo <kilo-pod-on-azure-node>
```

Уменьшить размер группы

```
az vmss scale --resource-group <resource-group> --name workers --new-capacity 0
```

Проверка autoscaler

Разверните workload, который вызовет автомасштабирование:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
name: test-azure-autoscale
```

```
spec:
```

```
replicas: 3
```

```
selector:
```

```
matchLabels:
```

```
app: test-azure
```

```
template:
```

```
metadata:
```

```
labels:
```

```
app: test-azure
```

```
spec:
nodeSelector:
  topology.kubernetes.io/zone: azure
containers:
  - name: pause
image: registry.k8s.io/pause:3.9
resources:
requests:
  cpu: "500m"
  memory: "512Mi"
```

Устранение неполадок

Подключение к удаленным worker-узлам для диагностики

Worker-узлы Azure можно диагностировать через Serial console в Azure Portal: перейдите к инстансу VMSS -> Support + troubleshooting -> Serial console. Так вы получите прямой доступ к консольному выводу узла без сетевого подключения.

Также можно использовать `talm dashboard` для подключения через control plane:

```
talm dashboard -f nodes/<control-plane>.yaml -n <worker-node-ip>
```

Здесь `<control-plane>.yaml` \- конфигурация узла control plane, а `<worker-node-ip>` \- внутренний Kubernetes IP удаленного worker-узла.

Узел застрял в maintenance mode

Если в serial console отображаются такие сообщения:

```
[talos] talosctl apply-config --insecure --nodes 10.2.0.5 --file <config.yaml>
```

```
[talos] or apply configuration using talosctl interactive installer:
```

```
[talos] talosctl apply-config --insecure --nodes 10.2.0.5 --mode=interactive
```

Это означает, что machine config не был применен или содержит ошибки. Частые причины:

- Неподдерживаемая версия Kubernetes : версия образа `kubelet` в конфигурации несовместима с текущей версией Talos.
- Некорректная конфигурация : ошибки синтаксиса YAML или недопустимые значения полей.
- `**customData` не применен **: инстанс VMSS был создан до обновления конфигурации.

Для диагностики примените конфигурацию вручную через Talos API (порт 50000 должен быть открыт в NSG):

```
talosctl apply-config --insecure --nodes <node-public-ip> --file nodes/azure.yaml
```

Если конфигурация будет отклонена, сообщение об ошибке покажет, что нужно исправить.

Чтобы обновить machine config для новых инстансов VMSS:

```
az vmss update \
--resource-group <resource-group> \
--name workers \
--custom-data @nodes/azure.yaml
```

После обновления удалите существующие инстансы, чтобы они были пересозданы с новой конфигурацией:

```
az vmss delete-instances \
--resource-group <resource-group> \
--name workers \
--instance-ids "*"
```

Предупреждение

Azure не предоставляет способ прочитать `customData` обратно из VMSS: его можно только задать. Всегда храните файл machine config (`nodes/azure.yaml`) в системе контроля версий как единственный источник истины.

Узел не присоединяется к кластеру

- Проверьте, что endpoint control plane из Talos machine config доступен из Azure.
- Убедитесь, что правила NSG разрешают исходящий трафик на порт 6443.
- Убедитесь, что правила NSG разрешают входящий трафик на порт 50000 (Talos API) для диагностики.
- Проверьте состояние provisioning у инстанса VMSS: `az vmss list-instances --resource-group <resource-group> --name workers`.

Не-лидерные узлы недоступны (`kubectl logs/exec` завершается по timeout)

Если `kubectl logs` или `kubectl exec` работает для узла-лидера Kilo, но завершается по timeout для всех остальных узлов в той же подсети Azure:

1. Проверьте, что IP forwarding включен на VMSS:

```
az vmss show --resource-group <resource-group> --name workers \
--query "virtualMachineProfile.networkProfile.networkInterfaceConfigurations[0].enableIPForwarding"
```

Если значение `false`, включите его и примените к существующим инстансам:

```
az vmss update --resource-group <resource-group> --name workers \
--set virtualMachineProfile.networkProfile.networkInterfaceConfigurations[0].enableIPForwarding=true
az vmss update-instances --resource-group <resource-group> --name workers --instance-ids "*"

```

2. Проверьте обратный маршрут с узла-лидера:

```
# Это должно работать (та же подсеть, прямое подключение)
kubectl exec -n cozy-kilo <leader-kilo-pod> -- ping -c 2 <non-leader-ip>
```

Ошибки квоты VM

- Проверьте квоту: `az vm list-usage --location <location>`.
- Запросите увеличение квоты через Azure Portal.
- Попробуйте другое семейство VM, в котором есть доступная квота.

Ошибки SkuNotAvailable

- Некоторые размеры VM могут быть недоступны в отдельных регионах из-за ограничений емкости.
- Попробуйте другой размер VM: `az vm list-skus --location <location> --size <prefix>`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Cluster Autoscaler для Hetzner Cloud

<https://cozystack.ru/docs/v1.6/operations/multi-location/autoscaling/hetzner/>

Настройка автоматического масштабирования узлов в Hetzner Cloud с Talos Linux.

В этом руководстве описано, как настроить cluster-autoscaler для автоматического масштабирования узлов в Hetzner Cloud с Talos Linux.

Предварительные требования

- Аккаунт Hetzner Cloud с API token
- Установленный CLI `hcloud`
- Существующий Kubernetes-кластер на Talos
- Настроенные [Networking Mesh](#) и [Local CCM](#)

Шаг 1. Создание образа Talos в Hetzner Cloud

Hetzner не поддерживает прямую загрузку образов, поэтому snapshot нужно создать через временный сервер.

1.1. Генерация Schematic ID

Создайте schematic на factory.talos.dev с нужными расширениями:

```
curl -s -X POST https://factory.talos.dev/schematics \
-H "Content-Type: application/json" \
-d '{
  "customization": {
    "systemExtensions": {
      "officialExtensions": [
        "siderolabs/qemu-guest-agent",
        "siderolabs/amd-ucode",
        "siderolabs/amdgpu-firmware",
        "siderolabs/bnx2-bnx2x",
        "siderolabs/drbd",
        "siderolabs/i915-ucode",
        "siderolabs/intel-ice-firmware",
        "siderolabs/intel-ucode",
```

```
"siderolabs/qlogic-firmware",
"siderolabs/zfs"
]
}
}
}'
```

Сохраните возвращенный `id` как `SCHEMATIC_ID`.

Примечание

`siderolabs/qemu-guest-agent` обязателен для Hetzner Cloud. Добавьте другие расширения (`zfs`, `drbd` и т. д.) при необходимости для ваших workload.

1.2. Настройка hcloud CLI

```
export HCLOUD_TOKEN="<your-hetzner-api-token>"
```

1.3. Создание временного сервера в rescue mode

```
# Создать сервер (без запуска)
```

```
hcloud server create \
```

```
--name talos-image-builder \
```

```
--type cpx22 \
```

```
--image ubuntu-24.04 \
```

```
--location fsn1 \
```

```
--ssh-key <your-ssh-key-name> \
```

```
--start-after-create=false
```

```
# Включить rescue mode и запустить сервер
```

```
hcloud server enable-rescue --type linux64 --ssh-key <your-ssh-key-name> talos-image-builder
```

```
hcloud server poweron talos-image-builder
```

1.4. Запись образа Talos на диск

```
# Получить IP-адрес сервера
```

```
SERVER_IP=$(hcloud server ip talos-image-builder)
```

```
# Подключиться по SSH в rescue mode и записать образ
```

```
ssh root@$SERVER_IP
```

```
# Внутри rescue mode:
wget -O- "https://factory.talos.dev/image/${SCHEMATIC_ID}/<talos-version>/hcloud-amd64.raw.xz" \
| xz -d \
| dd of=/dev/sda bs=4M status=progress
sync
exit
```

1.5. Создание snapshot и очистка

```
# Выключить сервер и создать snapshot
hcloud server poweroff talos-image-builder

hcloud server create-image --type snapshot --description "Talos <talos-version>" talos-image-builder

# Получить snapshot ID (сохраните его для дальнейших шагов)
hcloud image list --type snapshot

# Удалить временный сервер
hcloud server delete talos-image-builder
```

Шаг 2. Создание Hetzner vSwitch (необязательно, но рекомендуется)

Создайте приватную сеть для связи между узлами:

```
# Создать сеть
hcloud network create --name cozystack-vswitch --ip-range 10.100.0.0/16

# Добавить подсеть для вашего региона (eu-central покрывает FSN1, NBG1)
hcloud network add-subnet cozystack-vswitch \
--type cloud \
--network-zone eu-central \
--ip-range 10.100.0.0/24
```

Шаг 3. Создание Talos machine config

В репозитории кластера сгенерируйте конфигурационный файл worker-узла:

```
talm template -t templates/worker.yaml --offline --full > nodes/hetzner.yaml
```

Затем отредактируйте `nodes/hetzner.yaml` для worker-узлов Hetzner:

1. Добавьте метаданные локации Hetzner (см. [Networking Mesh](#)):

`machine:`

`nodeAnnotations:`

`kilo.squat.ai/location: hetzner-cloud`

`kilo.squat.ai/persistent-keepalive: "20"`

`nodeLabels:`

`topology.kubernetes.io/zone: hetzner-cloud`

2. Укажите публичный endpoint Kubernetes API: измените `cluster.controlPlane.endpoint` на публичный адрес API-сервера, например `https://<public-api-ip>:6443`. Этот адрес можно найти в `kubeconfig` или опубликовать через ingress.

3. Удалите автоматически обнаруженные секции `installer/network`: удалите из файла секции `machine.install` и `machine.network`.

4. Укажите external cloud provider для kubelet (см. [Local CCM](#)):

`machine:`

`kubelet:`

`extraArgs:`

`cloud-provider: external`

5. Настройте определение подсети IP-адресов узлов: укажите в `machine.kubelet.nodeIP.validSubnets` подсеть vSwitch, например `10.100.0.0/24`.

6. Необязательно: добавьте registry mirrors, чтобы избежать rate limiting в Docker Hub:

`machine:`

`registries:`

`mirrors:`

`docker.io:`

`endpoints:`

`- https://mirror.gcr.io`

В результате конфигурация должна включать как минимум:

`machine:`

`nodeAnnotations:`

`kilo.squat.ai/location: hetzner-cloud`

`kilo.squat.ai/persistent-keepalive: "20"`

```
nodeLabels:
topology.kubernetes.io/zone: hetzner-cloud
kubelet:
nodeIP:
validSubnets:
    - 10.100.0.0/24 # замените на свою подсеть vSwitch
extraArgs:
cloud-provider: external
registries:
mirrors:
docker.io:
endpoints:
    - https://mirror.gcr.io
cluster:
controlPlane:
endpoint: https://<public-api-ip>:6443
```

Все остальные параметры (токены кластера, CA, расширения и т. д.) остаются такими же, как в сгенерированном шаблоне.

Шаг 4. Создание Kubernetes Secrets

4.1. Создание секрета с Hetzner API token

```
kubectrl -n cozy-cluster-autoscaler-hetzner create secret generic hetzner-credentials \
--from-literal=token=<your-hetzner-api-token>
```

4.2. Создание секрета с Talos machine config

Machine config должен быть закодирован в base64:

```
# Закодировать worker.yaml (base64 одной строкой)
base64 -w 0 -i worker.yaml -o worker.b64

# Создать секрет
kubectrl -n cozy-cluster-autoscaler-hetzner create secret generic talos-config \
--from-file=cloud-init=worker.b64
```

Шаг 5. Развертывание Cluster Autoscaler

Создайте ресурс Package:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.cluster-autoscaler-hetzner
spec:
  variant: default
  components:
    cluster-autoscaler-hetzner:
  values:
    cluster-autoscaler:
      autoscalingGroups:
        - name: workers-fsn1
      minSize: 0
      maxSize: 10
      instanceType: cpx22
      region: FSN1
      extraEnv:
        HCLOUD_IMAGE: "<snapshot-id>"
        HCLOUD_SSH_KEY: "<ssh-key-name>"
        HCLOUD_NETWORK: "cozystack-vswitch"
        HCLOUD_PUBLIC_IPV4: "true"
        HCLOUD_PUBLIC_IPV6: "false"
      extraEnvSecrets:
        HCLOUD_TOKEN:
          name: hetzner-credentials
          key: token
        HCLOUD_CLOUD_INIT:
          name: talos-config
          key: cloud-init
```

Примените манифест:

```
kubectl apply -f package.yaml
```

Шаг 6. Проверка автомасштабирования

Создайте deployment с pod anti-affinity, чтобы принудительно вызвать scale-up:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-autoscaler
spec:
  replicas: 5
  selector:
    matchLabels:
      app: test-autoscaler
  template:
    metadata:
      labels:
        app: test-autoscaler
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchLabels:
                  app: test-autoscaler
              topologyKey: kubernetes.io/hostname
      containers:
        - name: nginx
          image: nginx
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
```

Если узлов меньше, чем реплик, autoscaler создаст новые серверы Hetzner.

Шаг 7. Проверка

Проверить логи autoscaler

```
kubectl -n cozy-cluster-autoscaler-hetzner logs \
deployment/cluster-autoscaler-hetzner-hetzner-cluster-autoscaler -f
```

Проверить узлы

```
kubectl get nodes -o wide
```

Проверить метки узла и internal IP

```
kubectl get node <node-name> --show-labels
```

Ожидаемый результат для автомасштабируемых узлов:

- Internal IP из диапазона vSwitch, например 10.100.0.2.
- Метка `kilo.squat.ai/location=hetzner-cloud`.

Справочник конфигурации

Переменные окружения

| Переменная | Описание | Обязательна |
|---------------------|---|---------------------------|
| HCLLOUD_TOKEN | Hetzner API token | Да |
| HCLLOUD_IMAGE | ID snapshot Talos | Да |
| HCLLOUD_CLOUD_INIT | Machine config, закодированный в base64 | Да |
| HCLLOUD_NETWORK | Имя/ID сети vSwitch | Нет |
| HCLLOUD_SSH_KEY | Имя/ID SSH-ключа | Нет |
| HCLLOUD_FIREWALL | Имя/ID Firewall | Нет |
| HCLLOUD_PUBLIC_IPV4 | Назначать публичный IPv4 | Нет (по умолчанию: true) |
| HCLLOUD_PUBLIC_IPV6 | Назначать публичный IPv6 | Нет (по умолчанию: false) |

Типы серверов Hetzner

| Тип | vCPU | RAM | Подходит для |
|-------|------|-----|--------------------|
| срх22 | 2 | 4GB | Небольших workload |
| срх32 | 4 | 8GB | Общих задач |

| | | | |
|-------|-------------|------|------------------------|
| срх42 | 8 | 16GB | Средних workload |
| срх52 | 16 | 32GB | Крупных workload |
| ccx13 | 2 dedicated | 8GB | CPU-intensive workload |
| ccx23 | 4 dedicated | 16GB | CPU-intensive workload |
| ccx33 | 8 dedicated | 32GB | CPU-intensive workload |
| сах11 | 2 ARM | 4GB | ARM workload |
| сах21 | 4 ARM | 8GB | ARM workload |

Примечание

Некоторые старые типы серверов (срх11, срх21 и т. д.) могут быть недоступны в отдельных регионах.

Регионы Hetzner

| Код | Локация |
|------|------------------------|
| FSN1 | Фалькенштайн, Германия |
| NBG1 | Нюрнберг, Германия |
| HEL1 | Хельсинки, Финляндия |
| ASH | Ашберн, США |
| HIL | Хилсборо, США |

Устранение неполадок

Подключение к удаленным worker-узлам для диагностики

Talos не позволяет открывать dashboard напрямую к worker-узлам. Используйте `talm dashboard` для подключения через control plane:

```
talm dashboard -f nodes/<control-plane>.yaml -n <worker-node-ip>
```

Здесь `<control-plane>.yaml` \- конфигурация узла control plane, а `<worker-node-ip>` \- внутренний Kubernetes IP удаленного worker-узла.

Узлы не присоединяются к кластеру

1. Проверьте VNC console через Hetzner Cloud Console или командой:

```
hcloud server request-console <server-name>
```

2. Частые ошибки:

- "unknown keys found during decoding" : проверьте формат Talos config. `nodeLabels` находится в `machine`, `nodeIP` \- в `machine.kubelet`.
- "kubelet image is not valid" : несовпадение версии Kubernetes. Используйте версию kubelet, совместимую с вашей версией Talos.
- "failed to load config" : синтаксическая ошибка в `machine config`.

У узлов неправильный Internal IP

Убедитесь, что в `machine.kubelet.nodeIP.validSubnets` указана ваша подсеть vSwitch:

machine:

kubelet:

nodeIP:

validSubnets:

- 10.100.0.0/24

Scale-up не запускается

1. Проверьте логи autoscaler на ошибки.
2. Проверьте права RBAC: требуется доступ к `leases`.
3. Проверьте, действительно ли pod находятся в состоянии Pending:

```
kubectl get pods --field-selector=status.phase=Pending
```

Rate limiting registry (ошибки 403)

Добавьте registry mirrors в Talos config:

machine:

registries:

mirrors:

docker.io:

endpoints:

- https://mirror.gcr.io

registry.k8s.io:

endpoints:

- https://registry.k8s.io

Scale-down не работает

Autoscaler кэширует информацию об узлах до 30 минут. Подождите или перезапустите autoscaler:

```
kubectl -n cozy-cluster-autoscaler-hetzner rollout restart \
deployment cluster-autoscaler-hetzner-hetzner-cluster-autoscaler
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Local Cloud Controller Manager

<https://cozystack.ru/docs/v1.6/operations/multi-location/local-ccm/>

Определение IP узлов и lifecycle management для multi-location clusters.

Package `local-ccm` предоставляет легкий cloud controller manager для self-managed clusters. Он отвечает за node IP detection и node lifecycle без необходимости во внешнем cloud provider.

Что он делает

- External IP detection : определяет external IP каждого узла через `ip route get` (target по умолчанию: `8.8.8.8`)
- Node initialization : удаляет `taint node.cloudprovider.kubernetes.io/uninitialized`, чтобы pods могли планироваться
- Node lifecycle controller (опционально): мониторит NotReady nodes через ICMP ping и удаляет их после настраиваемого timeout

Установка

```
cozypkg add cozystack.local-ccm
```

Talos machine config

На всех узлах кластера, включая control plane, должен быть задан `cloud-provider: external`, чтобы kubelet передавал node initialization в cloud controller manager:

machine:

kubelet:

extraArgs:

cloud-provider: external

Важно

Настройка `cloud-provider: external` должна присутствовать на всех узлах кластера, включая control plane nodes. Без нее cluster-autoscaler не сможет сопоставить Kubernetes nodes с экземплярами cloud provider, например Azure VMSS.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Networking Mesh

<https://cozystack.ru/docs/v1.6/operations/multi-location/networking-mesh/>

Настройка Kilo WireGuard mesh с Cilium для связности multi-location cluster.

Kilo создает WireGuard mesh между locations кластера. При работе с Cilium он использует IPIP encapsulation, маршрутизируемую через VxLAN overlay Cilium, чтобы traffic между locations работал даже тогда, когда cloud network блокирует raw IPIP packets (protocol 4).

Выберите networking variant cilium-kilo

Во время настройки platform выберите networking variant cilium-kilo. Он разворачивает Cilium и Kilo как интегрированный stack с нужной конфигурацией.

Как это работает

1. Kilo работает в режиме `--local=false` и не управляет routes внутри location (это делает Cilium)
2. Kilo создает WireGuard tunnel (`kilo0`) между location leaders
3. Non-leader nodes в каждой location достигают remote locations через IPIP encapsulation до своего location leader, маршрутизируемую через VxLAN overlay Cilium
4. Leader decapsulates IPIP и пересылает traffic через WireGuard tunnel
5. Опция Cilium `enable-ipip-termination` создает интерфейс `cilium_tunl` (переименованный kernel `tunl0`), который Kilo использует для IPIP TX/RX; без него kernel обнаруживает TX recursion на tunnel device

Talos machine config для cloud nodes

Cloud worker nodes должны содержать Kilo annotations в Talos machine config:

machine:

nodeAnnotations:

kilo.squat.ai/location: <cloud-location-name>

kilo.squat.ai/persistent-keepalive: "20"

nodeLabels:

topology.kubernetes.io/zone: <cloud-location-name>

Примечание

Kilo читает `kilo.squat.ai/location` из node annotations , а не из labels. Annotation `persistent-keepalive` критична для cloud nodes за NAT: она включает WireGuard NAT traversal, позволяя Kilo автоматически обнаружить реальный public endpoint.

Allowed location IPs

По умолчанию Kilo маршрутизирует через WireGuard mesh только pod CIDRs и individual node internal IPs. Если узлы в location используют private subnet, к которой должны обращаться другие locations (например, для kubelet communication или доступа к NodePort), добавьте annotation `kilo.squat.ai/allowed-location-ips` на узлы в этой location :

На всех on-premise nodes (через label selector) - открыть on-premise subnet для cloud nodes

```
kubectrl annotate nodes -l topology.kubernetes.io/zone=on-prem
```

```
kilo.squat.ai/allowed-location-ips=192.168.100.0/24
```

Это сообщает Kilo, что указанные CIDRs нужно включить в WireGuard allowed IPs для этой location, делая эти subnets маршрутизируемыми через tunnel из всех других locations.

Предупреждение

Добавляйте эту annotation на узлы, которым принадлежит subnet, которую нужно открыть (то есть на узлы в location, где эта сеть существует), а не на remote nodes, которым нужно до нее достучаться. Если задать ее в неправильной location, Kilo создаст route, отправляющий traffic для этого CIDR через WireGuard tunnel на всех остальных узлах, включая узлы, которые напрямую подключены к этой subnet через L2. Это ломает локальную связность между co-located nodes.

Например, если cloud nodes используют `10.2.0.0/24`, добавьте annotation на cloud nodes. Не добавляйте on-premise subnet (например, `192.168.100.0/23`) на cloud nodes: это перехватит весь локальный traffic между on-premise nodes через WireGuard tunnel.

Устранение неполадок

WireGuard tunnel не установлен

- Проверьте, что у узла есть annotation `kilo.squat.ai/persistent-keepalive: "20"`
- Проверьте, что у узла есть annotation `kilo.squat.ai/location`, а не только label
- Проверьте, что cloud firewall разрешает inbound UDP 51820
- Посмотрите logs kilo: `kubectrl logs -n cozy-kilo <kilo-pod>`
- Повторяющиеся каждые 30 секунд сообщения "WireGuard configurations are different" указывают на отсутствие annotation `persistent-keepalive`

Non-leader nodes недоступны (timeout kubectrl logs/exec)

-
- Проверьте, что IP forwarding включен на cloud network interfaces (это нужно Kilo leader для forward traffic)
 - Проверьте logs kilo pod на ошибки `cilium_tunl interface not found`; это означает, что Cilium не запущен с `enable-ipip-termination=true` (variant cilium-kilo настраивает это автоматически)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Частые вопросы и практические руководства

<https://cozystack.ru/docs/v1.6/operations/faq/>

База знаний с частыми вопросами и расширенными настройками

Диагностика

Рекомендации по диагностике доступны в [шпаргалке по troubleshooting](#).

Развертывание Cozystack

Как выделить место на системном диске под пользовательское хранилище

Развертывание Cozystack, [как установить Talos на машину с одним диском](#)

Как включить KubeSpan

Развертывание Cozystack, [как включить KubeSpan](#)

Как включить HugePages

Развертывание Cozystack, [как включить HugePages](#).

Что делать, если cloud-провайдер не поддерживает MetalLB

Большинство cloud-провайдеров не поддерживают MetalLB. Вместо него можно опубликовать основной ingress-контроллер через external IP.

Для развертывания в Hetzner используйте отдельное [руководство по установке в Hetzner](#). Для других провайдеров используйте раздел [Public IP Setup](#) в руководстве по установке Cozystack.

Развертывание Kubernetes в публичной сети

Развертывание Cozystack, [развертывание с публичными сетями](#).

Эксплуатация

Как включить доступ к dashboard через ingress-controller

Обновите приложение `ingress` и включите в нем параметр `dashboard: true`. Dashboard станет доступен по адресу: `https://dashboard.<your_domain>`

Как настраивать Cozystack с помощью FluxCD или ArgoCD

В этом [reference-репозитории](#) показано, как настраивать сервисы Cozystack через GitOps:

- <<https://github.com/aenix-io/cozystack-gitops-example>>

Как сгенерировать kubeconfig для пользователей tenant

Перенесено в раздел [Как сгенерировать kubeconfig для пользователей tenant](#).

Как использовать токены ServiceAccount для доступа к API

См. [Токены ServiceAccount для доступа к API](#).

Как ротировать Certificate Authority

Перенесено в раздел обслуживания кластера: [Как ротировать Certificate Authority](#).

Как очистить состояние etcd

Перенесено в Troubleshooting: [Как очистить состояние etcd](#).

Bundles

Как переопределять параметры отдельных компонентов

Перенесено в конфигурацию кластера: [Справочник компонентов](#).

Как отключить отдельные компоненты из bundle

Перенесено в конфигурацию кластера: [Справочник компонентов](#).

•

[Токены ServiceAccount для доступа к API](#)

Как получить и использовать токены ServiceAccount в Cozystack.

[Как сгенерировать kubeconfig для пользователей tenant](#)

Руководство по генерации kubeconfig-файла для пользователей tenant в Cozystack.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Токены ServiceAccount для доступа к API

<https://cozystack.ru/docs/v1.6/operations/faq/serviceaccount-api-access/>

Как получить и использовать токены ServiceAccount в Cozystack.

Предварительные требования

Перед началом убедитесь, что:

- Tenant уже существует в Cozystack. Если он еще не создан, см. [Создание пользовательского tenant](#).
- У вас есть доступ к namespace tenant - через OIDC-учетные данные или административный kubeconfig.
- `kubectl` установлен и настроен.
- (Опционально) установлен `jq`.

Получение токена ServiceAccount

У каждого tenant в Cozystack есть Secret с токеном ServiceAccount. Secret имеет то же имя, что и tenant, и находится в namespace этого tenant.

- Dashboard
- `kubectl`

1. Войдите в Dashboard пользователем, у которого есть доступ к tenant.
2. При необходимости переключите контекст на нужный tenant.
3. В левой боковой панели перейдите на страницу Administration -> Info и откройте вкладку Secrets.
4. Найдите secret с именем `tenant-<name>` (например, `tenant-team1`), где Key равен token.
5. Нажмите значок глаза, чтобы показать поле Value, затем нажмите на показанные данные. Текст будет автоматически скопирован в буфер обмена.

Получите токен для tenant с именем `<name>`:

| | | |
|--|--------------------------------|----------------------|
| <pre>kubectl -n tenant-<name>
get tenantsecret
tenant-<name> -o json</pre> | <pre>jq -r '.data.token'</pre> | <pre>@base64d'</pre> |
|--|--------------------------------|----------------------|

Чтобы сохранить токен в переменную для последующих команд:

| | | |
|--|-------------------------------|-----------------------|
| <pre>export TOKEN=\$(kubectl -n tenant-<name> get tenantsecret tenant-<name> -o json</pre> | <pre>jq -r '.data.token</pre> | <pre>@base64d')</pre> |
|--|-------------------------------|-----------------------|

Использование токена для доступа к API

Получив токен, вы можете [сгенерировать kubeconfig](#) для доступа через kubectl или использовать токен напрямую через curl, как показано ниже.

Безопасность токена

Токены ServiceAccount в Cozystack по умолчанию не имеют срока действия. Обращайтесь с ними так же аккуратно, как с паролями.

Проверка подключения

Сначала убедитесь, что текущий контекст kubectl указывает на нужный кластер Cozystack:

```
kubectl config current-context
```

```
kubectl cluster-info
```

Затем получите адрес API-сервера:

```
export API_SERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
```

После этого извлеките CA-сертификат из secret tenant:

| | |
|--|----------------------------------|
| <pre>kubectl -n tenant-<name> get secret tenant-<name> -o jsonpath='{.data.ca\.crt}'</pre> | <pre>base64 -d > ca.crt</pre> |
|--|----------------------------------|

Теперь проверьте подключение:

```
curl --cacert ca.crt -H "Authorization: Bearer ${TOKEN}" ${API_SERVER}/api
```

После проверки файл ca.crt можно удалить.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как сгенерировать kubeconfig для пользователей tenant

<https://cozystack.ru/docs/v1.6/operations/faq/generate-kubeconfig/>

Руководство по генерации kubeconfig-файла для пользователей tenant в Cozystack.

Чтобы сгенерировать kubeconfig для пользователей tenant, используйте следующий скрипт. В результате вы получите файл tenant-kubeconfig, который можно передать пользователю.

```
SERVER=$(kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}')
```

```
kubectl get secret tenant-root -n tenant-root -o go-template='
```

```
apiVersion: v1
```

```
kind: Config
```

```
clusters:
```

```
  - name: tenant-root
```

```
cluster:
```

```
server: "$SERVER"
```

```
certificate-authority-data: {{ index .data "ca.crt" }}
```

```
contexts:
```

```
  - name: tenant-root
```

```
context:
```

```
cluster: tenant-root
```

```
namespace: {{ index .data "namespace" | base64decode }}
```

```
user: tenant-root
```

```
current-context: tenant-root
```

```
users:
```

```
  - name: tenant-root
```

```
user:
```

```
token: {{ index .data "token" | base64decode }}
```

```
'\
```

```
> tenant-root.kubeconfig
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Руководство по устранению неполадок Cozystack

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/>

Начальные шаги для проверки состояния кластера и поиска проблем.

В этом руководстве приведены начальные шаги для проверки состояния кластера и поиска проблем. Внизу страницы находятся ссылки на руководства по устранению неполадок для разных компонентов Cozystack и аспектов эксплуатации кластера.

Чеклист диагностики

Используйте следующие команды, чтобы проверить состояние кластера.

=== Flux CD ===

не должно быть сломанных HelmRelease

```
kubectl get hr -A
```

```
grep -v True
```

=== Kubernetes ===

не должно быть узлов не в состоянии Ready

```
kubectl get node
```

=== LINSTOR ===

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -ti -- linstor'
```

узлы LINSTOR находятся online

```
linstor node list
```

storage-pool LINSTOR находятся в состоянии Ok

```
linstor storage-pool list
```

нет сломанных ресурсов

```
linstor resource list --faulty
```

=== Kube-OVN ===

```
alias ovn-appctl='kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl'
```

Проверить Northbound database

```
ovn-appctl -t /var/run/ovn/ovnnb_dbctl cluster/status OVN_Northbound
```

Проверить Southbound database

```
ovn-appctl -t /var/run/ovn/ovnsb_db.ctl cluster/status OVN_Southbound
```

```
# убедитесь, что:
```

```
# 1. Количество Servers совпадает с количеством control-plane узлов
```

```
# 2. IP-адреса корректны
```

```
# 3. Нет дубликатов (например, двух servers с одним IP)
```

```
# вывести список control-plane узлов
```

```
kubectl get node -o wide -l node-role.kubernetes.io/control-plane=
```

```
# Проверить, что вы не упираетесь в namespace quotas и есть ресурсы для обновления
```

```
kubectl get resourcequota --all-namespaces
```

Дополнительно можно проверить, есть ли в кластере pod не в состоянии Running:

| | | |
|---------------------------------|--------------------------------|-------------------------|
| <code>kubectl get pod -A</code> | <code>grep -v 'Running\</code> | <code>Completed'</code> |
|---------------------------------|--------------------------------|-------------------------|

Получение базовой информации

Логи оператора Cozystack можно посмотреть командой:

```
kubectl logs -n cozy-system deploy/cozystack-operator -f
```

Все компоненты платформы устанавливаются через Flux CD HelmRelease.

Получить все установленные HelmRelease можно так:

```
# kubectl get hr -A
```

```
NAMESPACE NAME AGE READY STATUS
```

```
cozy-cert-manager cert-manager 4m1s True Release reconciliation succeeded
```

```
cozy-cert-manager cert-manager-issuers 4m1s True Release reconciliation succeeded
```

```
cozy-cilium cilium 4m1s True Release reconciliation succeeded
```

```
cozy-cluster-api capi-operator 4m1s True Release reconciliation succeeded
```

```
cozy-cluster-api capi-providers 4m1s True Release reconciliation succeeded
```

```
cozy-dashboard dashboard 4m1s True Release reconciliation succeeded
```

```
cozy-fluxcd cozy-fluxcd 4m1s True Release reconciliation succeeded
```

```
cozy-grafana-operator grafana-operator 4m1s True Release reconciliation succeeded
```

```
cozy-kamaji kamaji 4m1s True Release reconciliation succeeded
```

```
cozy-kubeovn kubeovn 4m1s True Release reconciliation succeeded
```

```
cozy-kubevirt-cdi kubevirt-cdi 4m1s True Release reconciliation succeeded
```

```
cozy-kubevirt-cdi kubevirt-cdi-operator 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt 4m1s True Release reconciliation succeeded
cozy-kubevirt kubevirt-operator 4m1s True Release reconciliation succeeded
cozy-linstor linstor 4m1s True Release reconciliation succeeded
cozy-linstor piraeus-operator 4m1s True Release reconciliation succeeded
cozy-mariadb-operator mariadb-operator 4m1s True Release reconciliation succeeded
cozy-metallb metallb 4m1s True Release reconciliation succeeded
cozy-monitoring monitoring 4m1s True Release reconciliation succeeded
cozy-postgres-operator postgres-operator 4m1s True Release reconciliation succeeded
cozy-rabbitmq-operator rabbitmq-operator 4m1s True Release reconciliation succeeded
cozy-redis-operator redis-operator 4m1s True Release reconciliation succeeded
cozy-telepresence telepresence 4m1s True Release reconciliation succeeded
cozy-victoria-metrics-operator victoria-metrics-operator 4m1s True Release reconciliation succeeded
tenant-root tenant-root 4m1s True Release reconciliation succeeded
```

В нормальном состоянии все они должны быть `Ready` и иметь статус `Release reconciliation succeeded`.

Packages застряли в `DependenciesNotReady`

Если некоторые `packages` показывают статус `DependenciesNotReady`:

```
$ kubectl get pkg -A | grep -v True
```

```
NAME VARIANT READY STATUS
```

```
cozystack.cozystack-basics default False One or more dependencies are not ready
```

```
cozystack.tenant-application default False One or more dependencies are not ready
```

```
cozystack.monitoring-application default False One or more dependencies are not ready
```

Обычно это означает, что `package` в цепочке зависимостей отсутствует или отключен. Для диагностики:

1. Найдите первопричину : проверьте логи оператора на сообщения `"dependency not found"`:

```
kubectl logs -n cozy-system
deploy/cozystack-operator
```

```
grep "dependency not found"
```

Команда покажет, какой зависимости не хватает, например:

```
dependency not found, marking as not ready package=cozystack.monitoring-application
dependency=cozystack.postgres-operator
```

2. Проверьте, не отключили ли вы обязательный package : некоторые packages зависят от других packages. Если отключить package, например `cozystack.postgres-operator`, от которого зависят другие packages, будет заблокирована вся цепочка зависимостей.

3. Исправьте проблему : либо снова включите отключенный package, либо, если вы намеренно хотите оставить его отключенным, добавьте его в `ignoreDependencies` у затронутого package:

```
kubectrl edit pkg cozystack.monitoring-application
```

спес:

`ignoreDependencies:`

- `cozystack.postgres-operator`

Специализированные руководства

Bootstrap кластера

См. [устранение неполадок установки Kubernetes](#).

Обслуживание кластера

Удаление отказавшего узла из кластера

См. [Обслуживание кластера](#) & [Масштабирование кластера](#).

Flux CD

[Устранение неполадок Flux CD](#).

Kube-OVN

[Устранение неполадок Kube-OVN](#).

-

[Устранение неполадок etcd](#)

Как устранять проблемы и ошибки etcd.

[Устранение неполадок Flux CD](#)

Как устранять ошибки Flux CD.

[Устранение неполадок мониторинга](#)

Руководство по диагностике и устранению проблем с компонентами мониторинга в Cozystack

Устранение неполадок Kube-OVN

Как устранять сбои Kube-OVN, вызванные поврежденной базой данных OVN.

Устранение crash loop у LINSTOR controller

Как устранять проблемы LINSTOR controller.

Устранение LINSTOR CrashLoopBackOff из-за поврежденной базы данных

Как устранять LINSTOR CrashLoopBackOff, связанный с поврежденной базой данных.

Устранение неполадок Piraeus custom resources

Как устранять проблемы с зависшими Piraeus custom resources.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок etcd

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/etcd/>

Как устранять проблемы и ошибки etcd.

Как очистить состояние etcd

Чтобы удалить state etcd с узла, используйте `talm` или `talosctl` со следующими командами:

- `Talm`
- `talosctl`

Замените `nodeN` именем отказавшего узла, например `node0.yaml`:

```
talm reset -f nodes/nodeN.yaml --system-labels-to-wipe=EPHEMERAL --graceful=false --reboot
```

```
talosctl reset --system-labels-to-wipe=EPHEMERAL --graceful=false --reboot
```

[!] Эта команда удалит state с указанного узла. Используйте ее с осторожностью.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок Flux CD

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/flux-cd/>

Как устранять ошибки Flux CD.

Диагностика ошибки `install retries exhausted`

Иногда можно столкнуться с такой ошибкой:

```
# kubectl get hr -A -n cozy-dashboard dashboard
NAMESPACE NAME AGE READY STATUS
cozy-dashboard dashboard 15m False install retries exhausted
```

Попробуйте разобраться через events:

```
kubectl describe hr -n cozy-dashboard dashboard
```

Если указано `Events: <none>`, приостановите и возобновите release:

```
kubectl patch hr -n cozy-dashboard dashboard -p '{"spec": {"suspend": true}}' --type=merge
kubectl patch hr -n cozy-dashboard dashboard -p '{"spec": {"suspend": null}}' --type=merge
```

Затем снова проверьте events:

```
kubectl describe hr -n cozy-dashboard dashboard
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок мониторинга

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/monitoring-troubleshooting/>

Руководство по диагностике и устранению проблем с компонентами мониторинга в Cozystack

В этом руководстве приведены шаги диагностики типичных проблем с компонентами мониторинга в Cozystack, включая сбор метрик, alerting, визуализацию и сбор логов.

Диагностика отсутствующих метрик

Если метрики не появляются в Grafana или VictoriaMetrics, выполните следующие шаги:

Проверьте состояние VMAgent

Убедитесь, что VMAgent запущен и собирает метрики:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=vmagent
```

```
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=vmagent --tail=50
```

Проверьте targets

Проверьте, может ли VMAgent выполнять scrape targets:

```
kubectl exec -n cozy-monitoring -c vmagent  
deploy/vmagent -- curl -s  
http://localhost:8429/targets | jq .
```

Найдите targets с health: "up". Если targets недоступны, проверьте сетевую связность и права RBAC.

Лимиты ресурсов

Если VMAgent ограничен ресурсами, увеличьте лимиты в конфигурации monitoring:

vmagent:

resources:

limits:

cpu: 500m

memory: 1Gi

requests:

cpu: 100m

memory: 256Mi

Требования безопасности

Убедитесь, что TLS включен для безопасного сбора метрик:

- Проверьте сертификаты в конфигурации VMAgent.
- Проверьте, что роли RBAC позволяют VMAgent обращаться к нужным endpoints.

Подробнее см. [Настройка мониторинга](#).

Alerts не приходят

Если alerts не приходят, проверьте Alertmanager и Alerta.

Проверьте Alertmanager

Проверьте, что Alertmanager обрабатывает alerts:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=alertmanager
```

```
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=alertmanager --tail=50
```

Проверьте alert rules:

```
kubectl get prometheusrules -n cozy-monitoring
```

Проверьте конфигурацию Alerta

Убедитесь, что Alerta настроена корректно:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=alerta
```

```
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=alerta --tail=50
```

Проверьте конфигурацию routing в spec monitoring:

alerta:

alerts:

telegram:

token: "your-token"

chatID: "your-chat-id"

Проблемы масштабируемости

Если alerts задерживаются из-за большого объема, настройте лимиты ресурсов:

alerta:

resources:

limits:

cpu: 2

memory: 2Gi

Безопасность

- Используйте RBAC для ограничения доступа к alerts.
- Включите TLS для alert endpoints.

Подробности конфигурации см. в [Alerting мониторинга](#).

Проблемы Grafana

Диагностика проблем доступа и источников данных в Grafana.

Проблемы доступа

Если Grafana недоступна:

- Проверьте service и ingress:

```
kubectrl get svc,ingress -n <tenant-namespace> -l app.kubernetes.io/name=grafana
```

- Проверьте права RBAC для вашего пользователя.

Конфигурация источников данных

Убедитесь, что источники данных подключены:

1. Войдите в Grafana.
2. Перейдите в Configuration > Data Sources.
3. Проверьте, что источник данных VictoriaMetrics находится в healthy-состоянии.

Если это не так, обновите URL и credentials.

Лимиты ресурсов

При проблемах производительности увеличьте ресурсы Grafana:

grafana:

resources:

limits:

cpu: 1

memory: 1Gi

Безопасность

- Включите authentication и authorization.
- Используйте TLS для доступа к Grafana.

Настройка dashboard описана в [Dashboard мониторинга](#).

Проблемы сбора логов

Диагностика проблем Fluent Bit и VLogs.

Проверьте Fluent Bit

Проверьте, что Fluent Bit собирает логи:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=fluent-bit
```

```
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=fluent-bit --tail=50
```

Проверьте VLogs

Убедитесь, что VLogs сохраняет логи:

```
kubectl get pods -n cozy-monitoring -l app.kubernetes.io/name=vlogs
```

```
kubectl logs -n cozy-monitoring -l app.kubernetes.io/name=vlogs --tail=50
```

Проверьте прием логов:

```
kubectl exec -n cozy-monitoring -c vlogs deploy/vlogs -- curl -s http://localhost:9428/health
```

Масштабируемость

Если логи не собираются из-за нагрузки, настройте ресурсы:

logsStorages:

- name: default

storage: 50Gi # Увеличить хранилище

Безопасность

- Используйте RBAC для доступа к логам.
- Включите TLS для передачи логов.

Дополнительная информация приведена в [Логам мониторинга](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок Kube-OVN

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/kube-ovn/>

Как устранять сбои Kube-OVN, вызванные поврежденной базой данных OVN.

Получение информации о состоянии базы данных Kube-OVN

Northbound DB

```
kubectrl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl \
-t /var/run/ovn/ovnnb_dbctl cluster/status OVN_Northbound
```

Southbound DB

```
kubectrl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl \
-t /var/run/ovn/ovnsb_dbctl cluster/status OVN_Southbound
```

Пример вывода:

Name: OVN_Northbound

Cluster ID: abf6 (abf66f15-9382-4b2b-b14c-355d64ae1bda)

Server ID: 8d8a (8d8a2985-c444-43bb-99f6-21c82f05b58d)

Address: ssl:[10.200.1.22]:6643

Status: cluster member

Role: leader

Term: 3

Leader: self

Vote: self

Last Election started 146211 ms ago, reason: leadership_transfer

Last Election won: 146202 ms ago

Election timer: 5000

Log: [2, 1569]

Entries not yet committed: 0

Entries not yet applied: 0

Connections: ->2a66 ->c23f <-2a66 <-c23f

Disconnections: 30

Servers:

2a66 (2a66 at ssl:[10.200.1.18]:6643) next_index=1569 match_index=1568 last msg 17 ms ago

```
8d8a (8d8a at ssl:[10.200.1.22]:6643) (self) next_index=1471 match_index=1568
c23f (c23f at ssl:[10.200.1.1]:6643) next_index=1569 match_index=1568 last msg 18 ms ago
```

Чтобы исключить узел из кластера, например если он недоступен и его нужно удалить из кластера, используйте:

```
# Northbound DB
```

```
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl -t /var/run/ovn/ovnnb_dbctl
cluster/kick OVN_Northbound <server-id>
```

```
# Southbound DB
```

```
kubectl -n cozy-kubeovn exec deploy/ovn-central -c ovn-central -- ovn-appctl -t /var/run/ovn/ovnsb_dbctl
cluster/kick OVN_Southbound <server-id>
```

Устранение падения pod Kube-OVN

В сложных случаях pod из DaemonSet Kube-OVN могут падать или не запускаться корректно. Обычно это указывает на поврежденную базу данных OVN. Подтвердить это можно по логам pod Kube-OVN CNI.

Получите список pod в namespace `cozy-kubeovn`:

```
# kubectl get pod -n cozy-kubeovn
NAME READY STATUS RESTARTS AGE
kube-ovn-cni-5rsvz 0/1 Running 5 (35s ago) 4m37s
kube-ovn-cni-jq2zz 0/1 Running 5 (33s ago) 4m39s
kube-ovn-cni-p4gz2 0/1 Running 3 (23s ago) 4m38s
```

Прочитайте логи pod по его имени (в этом примере `kube-ovn-cni-jq2zz`):

```
# kubectl logs -n cozy-kubeovn kube-ovn-cni-jq2zz
W0725 08:21:12.479452 87678 ovs.go:35] 100.64.0.4 network not ready after 3 ping to gateway
100.64.0.1
W0725 08:21:15.479600 87678 ovs.go:35] 100.64.0.4 network not ready after 6 ping to gateway
100.64.0.1
W0725 08:21:18.479628 87678 ovs.go:35] 100.64.0.4 network not ready after 9 ping to gateway
100.64.0.1
W0725 08:21:21.479355 87678 ovs.go:35] 100.64.0.4 network not ready after 12 ping to gateway
100.64.0.1
W0725 08:21:24.479322 87678 ovs.go:35] 100.64.0.4 network not ready after 15 ping to gateway
100.64.0.1
```

```
W0725 08:21:27.479664 87678 ovs.go:35] 100.64.0.4 network not ready after 18 ping to gateway
100.64.0.1
W0725 08:21:30.478907 87678 ovs.go:35] 100.64.0.4 network not ready after 21 ping to gateway
100.64.0.1
W0725 08:21:33.479738 87678 ovs.go:35] 100.64.0.4 network not ready after 24 ping to gateway
100.64.0.1
W0725 08:21:36.479607 87678 ovs.go:35] 100.64.0.4 network not ready after 27 ping to gateway
100.64.0.1
W0725 08:21:39.479753 87678 ovs.go:35] 100.64.0.4 network not ready after 30 ping to gateway
100.64.0.1
W0725 08:21:42.479480 87678 ovs.go:35] 100.64.0.4 network not ready after 33 ping to gateway
100.64.0.1
W0725 08:21:45.478754 87678 ovs.go:35] 100.64.0.4 network not ready after 36 ping to gateway
100.64.0.1
W0725 08:21:48.479396 87678 ovs.go:35] 100.64.0.4 network not ready after 39 ping to gateway
100.64.0.1
```

Чтобы устранить проблему, можно очистить базу данных OVN. Для этого запускается DaemonSet, который удаляет файлы конфигурации OVN с каждого узла. Такая очистка безопасна: DaemonSet Kube-OVN автоматически пересоздаст нужные файлы из Kubernetes API.

Примените следующий YAML, чтобы развернуть cleanup DaemonSet:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ovn-cleanup
  namespace: cozy-kubeovn
spec:
  selector:
    matchLabels:
      app: ovn-cleanup
  template:
    metadata:
      labels:
        app: ovn-cleanup
    spec:
      component: network
      type: infra
      spec:
```

```
affinity:
podAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
  - labelSelector:
matchLabels:
app: ovn-central
topologyKey: "kubernetes.io/hostname"
containers:
  - name: cleanup
image: busybox
command: ["/bin/sh", "-xc", "rm -rf /host-config-ovn/; rm -rf /host-config-ovn/.; exec sleep infinity"]
volumeMounts:
  - name: host-config-ovn
mountPath: /host-config-ovn
nodeSelector:
kubernetes.io/os: linux
node-role.kubernetes.io/control-plane: ""
tolerations:
  - operator: "Exists"
volumes:
  - name: host-config-ovn
hostPath:
path: /var/lib/ovn
type: ""
hostNetwork: true
restartPolicy: Always
terminationGracePeriodSeconds: 1
```

Проверьте, что DaemonSet запущен:

```
# kubectl get pod -n cozy-kubeovn
ovn-cleanup-hjzxb 1/1 Running 0 6s
ovn-cleanup-wmzdv 1/1 Running 0 6s
ovn-cleanup-ztm86 1/1 Running 0 6s
```

После завершения очистки удалите DaemonSet `ovn-cleanup` и перезапустите pod Kube-OVN CNI, чтобы применить новую конфигурацию:

Удалить cleanup DaemonSet

kubectl -n cozy-kubeovn delete ds ovn-cleanup

Перезапустить pod Kube-OVN через удаление

kubectl -n cozy-kubeovn delete pod -l app!=ovs

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение crash loop у LINSTOR controller

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/linstor-controller/>

Как устранять проблемы LINSTOR controller.

Перезапуск controller

Если controller запущен, но выглядит простаивающим или не отвечает, попробуйте перезапустить его. Эта операция идемпотентна и безопасна: ожидающая или незавершенная работа продолжится после перезапуска.

Crash loop LINSTOR controller

Если linstor-controller не может запуститься, а в логах нет полезной информации, можно повысить уровень логирования (максимальный уровень - TRACE).

Пример LINSTORCluster CR с повышенным уровнем логирования:

```
apiVersion: piraeus.io/v1
kind: LINSTORCluster
spec:
  controller:
  podTemplate:
  spec:
  containers:
    - name: linstor-controller
  env:
    # обе настройки используются linstor-controller
    - name: LS_LOG_LEVEL
  value: TRACE
    - name: LS_LOG_LEVEL_LINSTOR
  value: TRACE
```

Примечание: если linstor-controller не находится в crash loop, но нужно повысить уровень логирования, это можно сделать временно во время выполнения следующей командой:

```
linstor controller set-log-level --global TRACE
```

Эта настройка вернется к исходному значению после перезапуска controller.

LINSTOR перестает отвечать после истечения срока сертификатов

Если в LINSTOR настроена внутренняя TLS-коммуникация, сертификаты создаются и ротируются автоматически. Но есть открытая проблема [piraeusdatastore/piraeus-operator#701](#): компоненты не подхватывают новые сертификаты после ротации. Обходной путь - вручную перезапустить все компоненты LINSTOR.

Выполните шаги по порядку:

1. Перезапустите `linstor-controller`.
2. Перезапустите каждый `satellite` по одному. Не перезапускайте их все одновременно. После перезапуска каждого `satellite` проверьте его логи на ошибки, прежде чем переходить к следующему.
3. Снова перезапустите `linstor-controller`. Это нужно, потому что `controller` также иницирует подключения к `satellites` и может не переподключиться автоматически к `satellite`, который был перезапущен.
4. Перезапустите все оставшиеся компоненты LINSTOR.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение LINSTOR CrashLoopBackOff из-за поврежденной базы данных

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/linstor-database/>

Как устранять LINSTOR CrashLoopBackOff, связанный с поврежденной базой данных.

[!] Только для опытных пользователей

Это руководство предназначено для опытных пользователей, которым комфортны низкоуровневая диагностика и операции восстановления данных. Повреждение баз данных LINSTOR встречается редко и обычно указывает на серьезную базовую проблему.

Если вы столкнулись с такой ситуацией в production-окружении, настоятельно рекомендуем обратиться в квалифицированную поддержку, а не пытаться исправить проблему самостоятельно. Неверные действия могут привести к безвозвратной потере данных.

Введение

При запуске вне Kubernetes LINSTOR controller использует SQL database того или иного типа. В Kubernetes `linstor-controller` не требует persistent volume или внешней базы данных. Вместо этого он хранит всю информацию как custom resources (CR) прямо в Kubernetes control plane. При запуске LINSTOR controller читает все CR и создает базу данных в памяти.

Эти CR находятся в API group `internal.linstor.linbit.com`:

```
kubectl get crds
```

```
grep internal.linstor.linbit.com
```

Пример вывода:

```
ebsremotes.internal.linstor.linbit.com 2024-12-28T00:39:50Z
files.internal.linstor.linbit.com 2024-12-28T00:39:24Z
keyvaluestore.internal.linstor.linbit.com 2024-12-28T00:39:24Z
layerbcachevolumes.internal.linstor.linbit.com 2024-12-28T00:39:24Z
layercachevolumes.internal.linstor.linbit.com 2024-12-28T00:39:25Z
layerdrbdresourcedefinitions.internal.linstor.linbit.com 2024-12-28T00:39:24Z
...
```

Если CR каким-то образом повреждаются, `linstor-controller` завершается с ошибкой и переходит в `CrashLoopBackOff`. Пока `pod controller` падает, остальные компоненты продолжают работать. Даже если падают `satellites`, `drbd` на узлах также продолжает работать. Но создание и удаление `volumes` становится невозможным.

Эти CR плохо читаются человеком, но по ним можно понять, что отсутствует или повреждено. Можно установить уровень логирования `TRACE`, чтобы увидеть процесс загрузки ресурсов в логах и убедиться, что проблема связана с CR.

CR database может повредиться, если `linstor-controller` был перезапущен во время очень долгой операции создания или удаления. "Очень долгая" также может означать "зависшая". Если видно, что что-то не может корректно удалиться, лучше разобраться и помочь операции завершиться, а не перезапускать controller.

Пример логов

LINSTOR controller находится в crash loop:

```
# kubectl get pod -n cozy-linstor
```

```
linstor-controller-6574668cf9-kjtq5 0/1 CrashLoopBackOff 2 (25s ago) 15d
```

Просмотр логов:

```
# kubectl logs -n cozy-linstor linstor-controller-6574668cf9-kjtq5
```

```
...
```

```
2025-10-31 13:08:36.016 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Initializing the k8s crd database connector
```

```
2025-10-31 13:08:36.016 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Kubernetes-CRD connection URL is "k8s"
```

```
2025-10-31 13:08:37.674 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Starting service instance 'K8sCrdDatabaseService' of type K8sCrdDatabaseService
```

```
2025-10-31 13:08:37.690 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Security objects load from database is in progress
```

```
2025-10-31 13:08:37.960 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Security objects load from database completed
```

```
2025-10-31 13:08:37.961 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load from database is in progress
```

```
2025-10-31 13:08:38.446 [Main] ERROR LINSTOR/Controller/ffffff SYSTEM - Unknown error during loading data from DB [Report number 6904B4D1-000000-000000]
```

```
...
```

```
time="2025-10-31T13:08:38Z" level=fatal msg="failed to run" err="exit status 199"
```

Это указывает на поврежденную CRD database.

Получение error report

LINSTOR controller создает внутри контейнера файл в каталоге `/var/log/linstor-controller/` с подробным stack trace. К сожалению, его сложно увидеть, потому что он сразу удаляется при перезапуске контейнера. Чтобы обойти это, нужно остановить Piraeus operator controller и изменить entrypoint контейнера linstor-controller.

Чтобы увидеть error report:

1. Остановите Piraeus operator controller (уменьшите его deployment до нуля).
2. Остановите deployment linstor-controller (уменьшите его до нуля).
3. Измените entrypoint deployment linstor-controller, чтобы он запускал `sleep infinity`.
4. Удалите probes, чтобы контейнер не перезапускался.
5. Дождитесь запуска контейнера, подключитесь к нему через `exec` и запустите controller вручную.
6. После падения controller прочитайте error report.

```
kubectl scale deployment -n cozy-linstor piraeus-operator-controller-manager --replicas 0
```

```
kubectl scale deployment -n cozy-linstor linstor-controller --replicas 0
```

```
kubectl edit deploy -n cozy-linstor linstor-controller
```

Откроется редактор. Удалите целиком секции `livenessProbe`, `readinessProbe` и `startupProbe`.

Найдите секцию контейнера `linstor-controller` (основной контейнер). В ней замените секцию `args`: на:

command:

- sleep
- infinity

После сохранения и выхода увеличьте deployment до одной реплики и дождитесь запуска pod:

```
kubectl scale deployment -n cozy-linstor linstor-controller --replicas 1
```

Запустите controller изнутри контейнера:

```
kubectl exec -ti -n cozy-linstor deploy/linstor-controller -- bash
```

```
root@linstor-controller-85fd6d6496-cdhbc:/# piraeus-entry.sh startController
```

Вы увидите error report с номером report:

```
2025-10-31 13:19:46.765 [Main] ERROR LINSTOR/Controller/ffffff SYSTEM - Unknown error during loading data from DB [Report number 6904B76C-00000-000000]
```

Теперь прочитайте error report:

```
root@linstor-controller-85fd6d6496-cdhbc:/# ls -l /var/log/linstor-controller/
```

```
root@linstor-controller-85fd6d6496-cdhbc:/# cat /var/log/linstor-controller/ErrorMessage-6904B76C-000000-000000.log
```

Пример error report:

ERROR REPORT 6904B76C-000000-000000

=====

Application: LINBIT? LINSTOR

Module: Controller

Version: 1.31.3

Build ID: 3734cb10b55e71a97b4c3004877e43641e820f9e

Build time: 2025-07-10T06:00:07+00:00

Error time: 2025-10-31 13:19:46

Node: linstor-controller-85fd6d6496-cdhbc

Thread: Main

=====

Reported error:

=====

Category: Error

Class name: ImplementationError

Class canonical name: com.linbit.ImplementationError

Generated at: Method 'loadCoreObjects', Source file 'DatabaseLoader.java', Line #680

Error message: Unknown error during loading data from DB

Call backtrace:

Method Native Class:Line number

loadCoreObjects N com.linbit.linstor.dbdrivers.DatabaseLoader:680

loadCoreObjects N com.linbit.linstor.core.DbDataInitializer:169

initialize N com.linbit.linstor.core.DbDataInitializer:101

startSystemServices N com.linbit.linstor.core.ApplicationLifecycleManager:91

start N com.linbit.linstor.core.Controller:379

main N com.linbit.linstor.core.Controller:635

Caused by:

=====

Category: LinStorException

Class name: DatabaseException

Class canonical name: com.linbit.linstor.dbdrivers.DatabaseException

Generated at: Method 'getInstance', Source file 'ObjectProtectionFactory.java', Line #89

Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not found!

ErrorContext:

Call backtrace:

Method Native Class:Line number

getInstance N com.linbit.linstor.security.ObjectProtectionFactory:89

getObjectProtection N com.linbit.linstor.dbdrivers.AbsDatabaseDriver:288

load N com.linbit.linstor.core.objects.ResourceDbDriver:173

load N com.linbit.linstor.core.objects.ResourceDbDriver:47

loadAll N com.linbit.linstor.dbdrivers.k8s.crd.K8sCrdEngine:237

loadAll N com.linbit.linstor.dbdrivers.AbsDatabaseDriver:180

loadCoreObjects N com.linbit.linstor.dbdrivers.DatabaseLoader:444

loadCoreObjects N com.linbit.linstor.core.DbDataInitializer:169

initialize N com.linbit.linstor.core.DbDataInitializer:101

startSystemServices N com.linbit.linstor.core.ApplicationLifecycleManager:91

start N com.linbit.linstor.core.Controller:379

main N com.linbit.linstor.core.Controller:635

END OF ERROR REPORT.

Ищите сообщения вроде этого:

Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not found!

or

Error message: Database entry of table LAYER_DRBD_VOLUMES could not be restored.

ErrorContext: Details: Primary key: LAYER_RESOURCE_ID = '4804', VLM_NR = '0'

Они показывают, какой ресурс отсутствует или поврежден.

Резервная копия и анализ

Перед любыми изменениями сохраните все CR для анализа и восстановления:

```
kubect1 get crds | grep -o ".*.internal.linstor.linbit.com" | \
xargs kubect1 get crds -ojson > crds.json
```

```
kubect1 get crds | grep -o ".*.internal.linstor.linbit.com" | \
xargs -I{} sh -xc "kubect1 get {} -ojson > {}.json"
```

```
tar czvf backup-$(date +%d.%m.%Y).tgz *.json
```

У CR трудночитаемые имена, поэтому удобно скачать их все в JSON и изучать удобными инструментами на рабочей машине.

Исправление базы данных

[!] ДЕСТРУКТИВНОЕ ДЕЙСТВИЕ!

Если поврежденные CR нельзя исправить иначе, кроме удаления, можно удалить проблемные ресурсы обычным `kubect1 delete`. Учитывайте, что при удалении CR физические volumes останутся на storage nodes как orphan volumes. Они больше не будут автоматически управляться LINSTOR. При необходимости их нужно будет очистить вручную позже.

Метод 1. Использование вспомогательного скрипта

Вспомогательный скрипт `linstor-find-db.sh` можно использовать для поиска ресурсов по разным критериям:

```
#!/usr/bin/env bash
set -euo pipefail

layer_resource_id=""
vlm_nr=""
resource_name=""
node_name=""

for kv in "$@"; do
k="${kv%%=*}"
v="${kv#*=}"
case "$k" in
layer_resource_id) layer_resource_id="$v" ;;
vlm_nr) vlm_nr="$v" ;;
resource_name) resource_name="$v" ;;
```

```

node_name) node_name="$v" ;;
*) echo "Unknown key: $k" >&2; exit 2 ;;
esac
done

shopt -s nullglob
files=(*.internal.linstor.linbit.com.json)
if (($#files[@]==0)); then
echo "No *.internal.linstor.linbit.com.json files found" >&2
exit 1
fi

if [[ -n "$layer_resource_id" && -n "$vlm_nr" ]]; then
cat "${files[@]}" \
| jq -r --arg lrid "$layer_resource_id" --arg vlnr "$vlm_nr" '
.items[]?
| select(.spec.layer_resource_id == ($lrid | tonumber) and .spec.vlm_nr == ($vlnr | tonumber))
| "\(.kind).\(.apiVersion | split("/") [0])/\(.metadata.name)"
'
exit 0
fi

if [[ -n "$resource_name" && -n "$node_name" ]]; then
cat "${files[@]}" \
| jq -r --arg res "$resource_name" --arg node "$node_name" '
.items[]?
| select((.spec.resource_name | tostring | ascii_upcase) == ($res | tostring | ascii_upcase)
and (.spec.node_name | tostring | ascii_upcase) == ($node | tostring | ascii_upcase))
| "\(.kind).\(.apiVersion | split("/") [0])/\(.metadata.name)"
'
exit 0
fi

echo "Usage:
$(basename "$0") layer_resource_id=NUM vlm_nr=NUM
$(basename "$0") resource_name=NAME node_name=NAME" >&2
exit 2

```

Сохраните его как `linstor-find-db.sh`, сделайте исполняемым и используйте для поиска проблемных ресурсов.

Пример 1. Отсутствует ObjProt

Если ошибка говорит об отсутствующем ObjProt для конкретного ресурса:

Error message: ObjProt (/resources/GLD-CSXHK-006/PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D) not found!

Найдите все ресурсы для этого ресурса и узла:

```
chmod +x linstor-find-db.sh
./linstor-find-db.sh resource_name=PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D
node_name=GLD-CSXHK-006
```

Пример вывода:

```
LayerResourceIds.internal.linstor.linbit.com/b3e34e9fe5a79d4e8753d0ad4107d0af969d8faaefb88fbd683
16950fa2a9242
```

```
LayerResourceIds.internal.linstor.linbit.com/dcbff8f66de95d7c6148b3fbb3a9934d226ffb6dfd405c8394ae
5454dc87d348
```

```
Resources.internal.linstor.linbit.com/43319bec4ca2bbc21324663a9b716c3e4a7ba2607f0fa513dcc59172a
1b37270
```

```
Volumes.internal.linstor.linbit.com/4ef559b8fe14b58647c99a76a2d3a11f9bbf2b413a448eaf3771777f673
c0b4c
```

Удалите их все за один раз:

```
kubectl delete $(./linstor-find-db.sh resource_name=PVC-91C1486F-CFE9-41E2-80E1-86477B187F2D
node_name=GLD-CSXHK-006)
```

Пример 2. Отсутствует LayerDRBD volume

Если ошибка говорит об отсутствующем LayerDRBD volume:

Error message: Database entry of table LAYER_DRBD_VOLUMES could not be restored.

ErrorContext: Details: Primary key: LAYER_RESOURCE_ID = '4804', VLM_NR = '0'

Найдите ресурсы по `layer_resource_id` и `vlm_nr`:

```
./linstor-find-db.sh layer_resource_id=4804 vlm_nr=0
```

Пример вывода:

```
LayerStorageVolumes.internal.linstor.linbit.com/5b597f878f6bb586ddd7d7dc3bbddacbdabedf511861a411712440f66cc61a52
```

Удалите его:

```
kubectrl delete $(./linstor-find-db.sh layer_resource_id=4804 vlm_nr=0)
```

Итеративное исправление

После удаления проблемных ресурсов снова попробуйте запустить controller изнутри контейнера:

```
piraeus-entry.sh startController
```

Если он снова падает, вы получите новый error report с другим ресурсом. Повторите процесс:

1. Прочитайте error report и определите проблемный ресурс.
2. Найдите и удалите его.
3. Попробуйте снова запустить controller.

Продолжайте, пока controller не запустится успешно:

```
2025-10-31 13:42:17.483 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load from database is in progress
```

```
2025-10-31 13:42:19.190 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Core objects load from database completed
```

```
...
```

```
2025-10-31 13:42:27.309 [Main] INFO LINSTOR/Controller/ffffff SYSTEM - Controller initialized
```

Перезапуск controller

После исправления CR-based database нужно вернуть deployment linstor-controller в исходное состояние. Выйдите из контейнера и удалите измененный deployment, чтобы Piraeus operator пересоздал его:

```
kubectrl delete deployment -n cozy-linstor linstor-controller
```

Верните piraeus operator controller:

```
kubectrl scale deployment -n cozy-linstor piraeus-operator-controller-manager --replicas 1
```

Он выполнит reconcile deployment linstor-controller и запустит его с исходным entrypoint.

Восстановление исходного состояния

Если что-то пошло не так и ситуация стала непонятной, можно восстановить как минимум состояние, которое было до исправления.

получить сохраненные файлы в другом каталоге

```
mkdir restore; cd restore
```

```
tar xzf ../backup-*.tgz
```

удалить BCE CR по именам CRD, используя json-описания (при удалении CRD удаляются все CR)

```
kubectrl delete -f crds.json
```

восстановить definitions (обратите внимание на `create` вместо обычного `apply`)

```
kubectrl create -f crds.json
```

восстановить все ресурсы

| | | |
|--------------------------------|--|---|
| <code>kubectrl get crds</code> | <code>grep -o ".*.internal.linstor.lin
bit.com"</code> | <code>xargs -I{} sh -xc "kubectrl
create -f {}.json"</code> |
|--------------------------------|--|---|

Теперь можно начать снова.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Устранение неполадок Piraeus custom resources

<https://cozystack.ru/docs/v1.6/operations/troubleshooting/piraeus-custom-resources/>

Как устранять проблемы с зависшими Piraeus custom resources.

Введение

К Piraeus custom resources относятся:

- `LinstorCluster`
- `LinstorSatellite`
- `LinstorSatelliteConfiguration`
- `LinstorNodeConnection`

Piraeus controller защищает эти ресурсы от непреднамеренных изменений и удаления. При удалении ресурса controller сначала убеждается, что все нижестоящие ресурсы удалены.

Но по умолчанию webhook отклоняет любые изменения, если что-то идет не так с самим webhook. В нестабильной системе webhook может часто падать без полезных stacktrace.

Отключение webhook

Если вы исправляете CR, но они “не редактируются”, нужно отключить webhook, а возможно и сам operator.

Несколько полезных способов отключить webhook:

- Если установкой piraeus управляет GitOps-система, можно остановить ее для этого release и просто удалить конфигурацию webhook. Когда вы снимете GitOps-систему с паузы, она пересоздаст конфигурацию webhook.
- Если вы не хотите удалять конфигурацию webhook, потому что она была установлена вручную, можно “сломать” ее selector.

Пример:

```
kubectrl edit validatingwebhookconfigurations/piraeus-operator-validating-webhook-configuration
```

В редакторе замените все `- linstor*` на `- xlinstor`. Когда закончите, верните конфигурацию тем же способом. Примеры команд Vim: `%s/- linstor/- xlinstor/g` и `%s/- xlinstor/- linstor/g`.

Примечание: не отключайте webhook навсегда. Он нужен для защиты ресурсов.

Отключение controller

Piraeus controller не только отслеживает собственные CR, но и обновляет их при необходимости. В большинстве случаев его тоже следует отключить. В частности, он постоянно пересоздает `LinstorSatellite` из `LinstorSatelliteConfiguration`.

Чтобы отключить controller, уменьшите его deployment до нуля реплик:

```
kubectl -n storage scale deployment piraeus-operator-controller-manager --replicas=0
```

После завершения обслуживания верните одну реплику:

```
kubectl -n storage scale deployment piraeus-operator-controller-manager --replicas=1
```

Удаление finalizers

У каждого Piraeus CR есть finalizer, который все равно не работает после отключения controller. Если вы собираетесь удалить CR, удалите секцию finalizers обычным способом.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Scheduling Classes

<https://cozystack.ru/docs/v1.6/operations/scheduling-classes/>

Ограничение tenant workload конкретными узлами или failure domain с помощью ресурсов SchedulingClass и планировщика Cozystack.

SchedulingClass - это cluster-scoped custom resource, который позволяет администраторам задавать политики размещения для tenant workload. Когда tenant назначен scheduling class, все его pod автоматически направляются в custom scheduler Cozystack, который объединяет ограничения, заданные классом, с ограничениями, уже указанными в pod.

Это позволяет операторам платформы закреплять tenant за конкретными дата-центрами, availability zone или группами узлов без изменения отдельных application charts.

Как это работает

Функция состоит из двух компонентов:

1. Lineage-controller webhook (часть `cozystack`): mutating admission webhook, который перехватывает создание pod в tenant namespace. Если у namespace есть метка `scheduler.cozystack.io/scheduling-class`, webhook задает `schedulerName: cozystack-scheduler` и добавляет аннотацию `scheduler.cozystack.io/scheduling-class` на каждый pod. Если указанный SchedulingClass CR не существует, например scheduler не установлен, pod остаются без изменений и планируются обычным способом.
2. Cozystack scheduler (пакет `cozystack-scheduler`): custom Kubernetes scheduler, который работает рядом со стандартным scheduler. Во время планирования он находит SchedulingClass, указанный в аннотации pod, и объединяет ограничения CR (node affinity, pod affinity/anti-affinity, topology spread) с собственным spec pod полностью в памяти, не изменяя pod в API server.

Предварительные требования

- Cozystack v1.2+
- The `cozystack-scheduler` system package (v0.2.0+)

Установка scheduler

```
cozypkg add cozystack.cozystack-scheduler
```

Создание SchedulingClass

SchedulingClass CR повторяет привычные примитивы планирования Kubernetes. Все поля необязательны: указывайте только те ограничения, которые вам нужны.

Пример: закрепление workload за дата-центром

```
apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
metadata:
  name: dc-west
spec:
  nodeSelector:
    topology.kubernetes.io/region: us-west-2
```

Pod, назначенные этому классу, будут планироваться только на узлы с меткой `topology.kubernetes.io/region=us-west-2`.

Пример: распределение по availability zone

```
apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
metadata:
  name: zone-spread
spec:
  topologySpreadConstraints:
    - maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
```

Примечание

Если у `topologySpreadConstraint` или у условия `pod affinity/anti-affinity` значение `labelSelector` равно `nil`, scheduler автоматически заполняет его селектором, который соответствует `identity labels` приложения Cozystack (`apps.cozystack.io/application.group`, `.kind`, `.name`). Это позволяет задавать универсальные политики распределения или `anti-affinity` без жестко прописанных значений меток для каждого приложения.

Пример: требование выделенных узлов с anti-affinity

```
apiVersion: cozystack.io/v1alpha1
kind: SchedulingClass
```

```
metadata:
name: dedicated-nodes
spec:
nodeAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
nodeSelectorTerms:
  - matchExpressions:
  - key: node-pool
operator: In
values:
  - dedicated
podAntiAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
  - topologyKey: kubernetes.io/hostname
```

Это закрепляет workload за узлами из пула `dedicated` и распределяет pod по хостам. `labelSelector` для anti-affinity автоматически заполняется для каждого приложения, поэтому pod из разных приложений одного tenant все еще могут попадать на один узел.

Полный справочник spec SchedulingClass

| Поле | Тип | Описание |
|-----------------------------------|--|--|
| <code>spec.nodeSelector</code> | <code>map[string]string</code> | Простые key-value метки узлов, которым должны соответствовать все подходящие узлы. |
| <code>spec.nodeAffinity</code> | <code>[NodeAffinity](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#nodeaffinity-v1-core)</code> | Обязательные и предпочтительные правила node affinity. |
| <code>spec.podAffinity</code> | <code>[PodAffinity](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#podaffinity-v1-core)</code> | Обязательные и предпочтительные правила совместного размещения pod. |
| <code>spec.podAntiAffinity</code> | <code>[PodAntiAffinity](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#podantiaffinity-v1-core)</code> | Обязательные и предпочтительные правила разнесения pod. |

| | | |
|---|--|---|
| <code>spec.topologySpreadConstraints</code> | <code>[[]TopologySpreadConstraint](https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#topologyspreadconstraint-v1-core)</code> | Ограничения topology spread для равномерного распределения по failure domain. |
|---|--|---|

Назначение SchedulingClass tenant

При создании или редактировании tenant задайте параметр `schedulingClass` равным имени существующего SchedulingClass CR:

Через dashboard:

Выберите scheduling class в выпадающем списке формы создания tenant.

****Через Helm values (`values.yaml`):****

`schedulingClass: dc-west`

Через tenant secret (наследование дочерними tenant):

Если родительскому tenant назначен scheduling class, все дочерние tenant наследуют его автоматически. Дочерний tenant не может переопределить scheduling class родителя: он может задать свой class только если у родителя его нет.

Назначение записывает метку `scheduler.cozystack.io/scheduling-class` в namespace tenant. Webhook читает эту метку (или определяет ее из owning Application CR), чтобы вставить имя scheduler и аннотацию в pod.

Автоматически заполняемые label selectors

Scheduler (v0.2.0+) автоматически заполняет поля `labelSelector` со значением `nil` в условиях pod affinity, pod anti-affinity и topology spread constraint. Он использует identity labels приложения Cozystack у pod:

- `apps.cozystack.io/application.group`
- `apps.cozystack.io/application.kind`
- `apps.cozystack.io/application.name`

Это означает, что универсальный SchedulingClass, например:

список:

podAntiAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

-
- topologyKey: kubernetes.io/hostname

автоматически ограничит anti-affinity pod одного приложения: каждое приложение получит собственное поведение anti-affinity без отдельного SchedulingClass для каждого приложения.

Ключи меток по умолчанию можно переопределить в Helm values scheduler:

defaultLabelSelectorKeys:

- apps.cozystack.io/application.group
- apps.cozystack.io/application.kind
- apps.cozystack.io/application.name

Если у условия уже есть явный labelSelector, он сохраняется без изменений.

Операторы без встроенной поддержки schedulerName

Некоторые операторы, используемые Cozystack, не предоставляют schedulerName в своих CRD. Подход на основе webhook обрабатывает такие случаи прозрачно, потому что изменяет pod напрямую во время admission, независимо от того, какой оператор их создал:

- etcd-operator
- redis-operator (spotahome)
- mariadb-operator
- clickhouse-operator (altinity)

Для workload, которыми управляют эти операторы, специальная конфигурация не требуется.

Проверка настройки

1. Убедитесь, что scheduler запущен:

```
kubectl get pods -n cozy-system -l app.kubernetes.io/name=cozystack-scheduler
```

2. Убедитесь, что SchedulingClass существует:

```
kubectl get schedulingclasses
```

3. Проверьте, что namespace tenant содержит метку:

```
kubectl get ns tenant-example -o jsonpath='{.metadata.labels.scheduler\.cozystack\.io/scheduling-class}'
```

4. Проверьте, что pod в namespace tenant используют custom scheduler:

```
kubectl get pods -n tenant-example -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.schedulerName}{"\n"}{end}'
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Running Containerized GPU Workloads

<https://cozystack.ru/docs/v1.6/operations/gpu-container-workloads/>

Run CUDA pods and other containerized GPU workloads on Cozystack management nodes that ship the NVIDIA driver and container toolkit via the distro package manager.

This page covers running GPU workloads in regular Kubernetes pods (CUDA, ML training, inference) on Cozystack management cluster nodes. It targets the typical Linux GPU node shape — `apt`-installed NVIDIA driver plus `nvidia-container-toolkit` on Ubuntu/Debian — and uses the `container` variant of the `cozystack.gpu-operator` package. Other distros with an equivalent driver + toolkit package layout should work the same way but are not regularly tested.

If instead you want to pass whole GPUs to KubeVirt VMs, see [GPU Passthrough](#) and [GPU Sharing with HAMi](#) (HAMi provides fractional sharing in tenant Kubernetes clusters; stacking it directly on the `container` variant on the management cluster is not a supported combination yet — see Fractional GPU sharing below).

Примечание: The `container` variant is validated by `helm template` and unit tests upstream, but has not yet been exercised end-to-end on physical NVIDIA hardware. Treat the CUDA-pod flow below as provisional and verify it against your own GPU node before relying on it in production.

When to pick this variant

The `cozystack.gpu-operator` package exposes three architectural variants. Pick `container` when all of the following are true:

- The host already runs the NVIDIA driver, installed via the distro package manager (`apt install nvidia-driver-*` on Ubuntu/Debian; other distros with an equivalent driver package should work the same way but are not regularly tested). The operator must not load its own kernel module.
- The host already has `nvidia-container-toolkit` installed (`apt install nvidia-container-toolkit`) and registered with `containerd`. The operator must not deploy its own toolkit DaemonSet — that would overwrite the `/etc/containerd/config.toml` the host configured (via `nvidia-ctk runtime configure`), breaking the host runtime wiring.
- You want GPUs exposed to containers as `nvidia.com/gpu`, not passed through to KubeVirt VMs.

The other two variants exist for the opposite host shape: `default` (passthrough) unbinds the host driver and binds `vfio-pci` for VM passthrough, and `vgpu` requires the proprietary NVIDIA vGPU host driver plus a license server. Neither path produces a working setup on a host that already ships the driver and container toolkit through `apt` — the operator and the host install fight each other.

Prerequisites

- A Cozystack management cluster with at least one GPU-enabled node.

-
- The GPU node runs Ubuntu or Debian with the NVIDIA driver installed via the distro package manager (other distros with an equivalent driver + toolkit package layout should work the same way but are not regularly tested). Verify with `nvidia-smi` over SSH or `kubectl debug node/<node-name>` — it must enumerate the physical GPUs and report a working driver version.
 - The GPU node must not carry a `nvidia.com/gpu.workload.config` label left over from the passthrough setup (`kubectl label node <node-name> nvidia.com/gpu.workload.config-` to remove). The `container` variant relies on the upstream default `container` workload for unlabeled nodes; a leftover `vm-passthrough` label overrides that per-node and the device plugin will not serve the GPU. Remove it before (or together with) the containerd registration step below when migrating a node off the passthrough setup.
 - `nvidia-container-toolkit` installed on the same node and registered with containerd. `apt install nvidia-container-toolkit` lays down binaries only — it does not configure containerd. Register the runtime explicitly:

```
sudo nvidia-ctk runtime configure --runtime=containerd
```

```
sudo systemctl restart containerd
```

```
grep nvidia /etc/containerd/config.toml # must show the runtime entry
```

- `kubectl` configured against the management cluster.

With `driver.enabled=false` the operator uses the pre-installed host driver at its standard location, so on a stock Ubuntu/Debian install no `hostPaths.driverInstallDir` override is needed. Talos installs the driver under a non-standard prefix, so the operator does not find it at the default location and requires a different starting point — see `packages/system/gpu-operator/examples/values-native-talos.yaml` in the [cozystack repo](#) for a working reference with the compat DaemonSet and the matching `driverInstallDir` override.

1\ Install the GPU Operator (container variant)

Do not add `cozystack.gpu-operator` to `bundles.enabledPackages` for this variant. The `iaas` bundle renders the GPU operator from `bundles.iaas.gpuOperatorVariant`, which only accepts `default` or `vgpu` — any other value, `container` included, makes the platform chart fail the Helm render (`packages/core/platform/templates/bundles/iaas.yaml`). Apply the Package CR directly instead; the platform controller installs it without a bundle entry and without the variant restriction.

Apply a Package CR with `variant: container`:

```
apiVersion: cozystack.io/v1alpha1
```

```
kind: Package
```

```
metadata:
```

```
name: cozystack.gpu-operator
```

```
spec:
```

```
variant: container
```

```
kubectl apply -f gpu-operator-container.yaml
```

The platform controller resolves the variant against the `PackageSource` (`packages/core/platform/sources/gpu-operator.yaml`), pulls `values.yaml` \+ `values-container.yaml` from the OCI repository, and installs the chart into `cozy-gpu-operator`.

2\.. Verify the operator is healthy

All pods in the `cozy-gpu-operator` namespace should reach `Running`:

```
kubectl get pods --namespace cozy-gpu-operator
```

Example output (pod names will vary):

```
NAME READY STATUS RESTARTS AGE
gpu-feature-discovery-7jpvz 1/1 Running 0 2m
gpu-operator-7976b5b8fb-xqg2z 1/1 Running 0 3m
nvidia-cuda-validator-tjkfh 0/1 Completed 0 2m
nvidia-dcgm-exporter-rmpfg 1/1 Running 0 2m
nvidia-device-plugin-daemonset-cqj9w 1/1 Running 0 2m
nvidia-operator-validator-q5n4k 1/1 Running 0 3m
```

The `container` variant does not spawn `nvidia-driver-daemonset`, `nvidia-container-toolkit-daemonset`, or `nvidia-vfio-manager` — all three are pinned off by design.

The node should advertise `nvidia.com/gpu` as an allocatable resource:

```
kubectl describe node <node-name>
```

```
...
```

```
Capacity:
```

```
...
```

```
nvidia.com/gpu: 2
```

```
...
```

```
Allocatable:
```

```
...
```

```
nvidia.com/gpu: 2
```

```
...
```

3\ Run a sample CUDA pod

Create a pod that requests one GPU and runs `nvidia-smi`:

```
apiVersion: v1
kind: Pod
metadata:
  name: cuda-smoke
spec:
  restartPolicy: OnFailure
  containers:
    - name: cuda
      image: nvcr.io/nvidia/cuda:12.4.1-base-ubuntu22.04
      command: ["nvidia-smi"]
  resources:
    limits:
      nvidia.com/gpu: 1
```

```
kubectl apply -f cuda-smoke.yaml
```

```
kubectl wait --for=jsonpath='{.status.phase}'=Succeeded pod/cuda-smoke --timeout=5m
```

```
kubectl logs cuda-smoke
```

The output should enumerate the GPU(s) visible to the pod and report the driver version that the host runs.

Fractional GPU sharing

The `container` variant exposes whole GPUs through the upstream NVIDIA device plugin. For fractional sharing (per-pod memory and compute quotas), see [GPU Sharing with HAMi](#) — currently documented for tenant Kubernetes clusters, where enabling HAMi automatically disables the GPU Operator’s built-in device plugin to avoid resource-registration conflicts. Stacking the `cozystack.hami` package directly on top of the `container` variant on the management cluster is not a supported combination yet: this variant pins the NVIDIA device plugin on, and HAMi ships its own device plugin, so the two would both register `nvidia.com/gpu`. The `cozystack.hami` PackageSource only declares `dependsOn: cozystack.gpu-operator` for install ordering — it does not disable the operator’s device plugin the way the tenant `kubernetes` app chart does.

Variant comparison

| Workload shape | Variant | Host driver | Host container toolkit | Notes |
|----------------------------|------------------------|---|------------------------|---|
| Containers (CUDA pods, ML) | <code>container</code> | required | required | This page |
| Whole GPU to one VM | <code>default</code> | must NOT be loaded — operator binds <code>vfio-pci</code> | not used | GPU Passthrough |
| Sliced GPU to multiple VMs | <code>vgpu</code> | proprietary NVIDIA vGPU host driver | not used | Requires NVIDIA vGPU license + a Delegated License Service endpoint |

Last modified 2026-07-22: [\[\[docs\] Update managed apps reference for v1.6.0 \(96e013f\)\]\(https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f\)](#)

Managed Kubernetes

Управляемый (tenant) Kubernetes

<https://cozystack.ru/docs/v1.6/kubernetes/>

Как развернуть и использовать изолированные управляемые Kubernetes-кластеры в Cozystack.

Управляемый Kubernetes в Cozystack

Если вам нужно развернуть собственное контейнеризованное приложение в Cozystack, лучше всего размещать его в управляемом Kubernetes-кластере.

Cozystack разворачивает Kubernetes-as-a-Service и управляет им как самостоятельным приложением внутри изолированной среды каждого тенанта. В Cozystack такие кластеры называются тенант-кластерами Kubernetes, а базовый кластер Cozystack — management-кластером или root-кластером. Тенант-кластеры полностью отделены от management-кластера и предназначены для приложений конкретного tenant или приложений, разработанных заказчиком.

Внутри тенант-кластера пользователи могут использовать сервисы LoadBalancer и быстро заказывать persistent volumes по мере необходимости. Control plane работает в контейнерах, а worker nodes разворачиваются как виртуальные машины; всем этим управляет приложение.

Kubernetes version in tenant clusters is independent of Kubernetes in the management cluster. Users can select the supported patch versions from 1.31 to 1.35.

Зачем использовать управляемый Kubernetes-кластер

Kubernetes стал отраслевым стандартом: он предоставляет единый и понятный API, а для конфигурации в основном использует YAML. Благодаря этому команды проще понимают Kubernetes и работают с ним, а управление инфраструктурой становится более предсказуемым.

Kubernetes использует устойчивые архитектурные паттерны и обеспечивает непрерывное восстановление системы в любых сценариях за счёт механизма reconciliation. Кроме того, он позволяет прозрачно масштабироваться на множество серверов и снимает проблемы, характерные для сложных и устаревших API традиционных платформ виртуализации. Управляемый сервис убирает необходимость разрабатывать собственные решения или изменять исходный код, экономя время и усилия.

Управляемый сервис Kubernetes в Cozystack дает удобный способ эффективно управлять серверными рабочими нагрузками.

Начало работы

Когда тенант-кластер Kubernetes готов, получите kubeconfig для работы с ним. Это можно сделать через UI или запросом `kubectl`:

- Откройте дашборд Cozystack, переключитесь в свой тенант, найдите и откройте страницу приложения. Скопируйте один из config-файлов из раздела Secrets.
- Выполните следующую команду, используя kubeconfig management-кластера:

| | |
|--|--|
| <pre>kubectl get secret -n tenant-<name>
kubernetes-<clusterName>-admin-kubeconfig
-o go-template='{{ printf "%s\n" (index .data "
admin.conf"</pre> | <pre>base64decode) }}' > admin.conf</pre> |
|--|--|

Доступно несколько вариантов kubeconfig:

- `admin.conf` — стандартный kubeconfig для доступа к новому кластеру. С его помощью можно создавать дополнительных пользователей Kubernetes.
- `admin.svc` — тот же token, что и в `admin.conf`, но адрес API server указывает на внутреннее имя service. Используйте его для приложений, которые работают внутри кластера и которым нужен доступ к API.
- `super-admin.conf` — похож на `admin.conf`, но с расширенными административными правами. Предназначен для диагностики неисправностей и задач обслуживания кластера.
- `super-admin.svc` — то же, что `super-admin.conf`, но с указанием на внутренний адрес API server.

Детали реализации

Тенант-кластер Kubernetes в Cozystack по сути является Kubernetes-in-Kubernetes. При его развертывании используются следующие компоненты:

- Kamaji Control Plane : [Kamaji](#) — open-source проект, который упрощает развертывание Kubernetes control planes в виде pod внутри root-кластера. Каждый control plane pod включает базовые компоненты вроде `kube-apiserver`, `controller-manager` и `scheduler`, что помогает эффективно использовать multi-tenancy и ресурсы.
- Etcd Cluster : A dedicated etcd cluster is deployed using the cozystack [etcd-operator](#) (`etcd-operator.cozystack.io/v1alpha2`). It provides reliable and scalable key-value storage for the Kubernetes control plane.
- Worker Nodes : виртуальные машины создаются через KubeVirt и используются как worker nodes. Эти ноды настроены на присоединение к тенант-кластеру Kubernetes, что позволяет разворачивать workload и управлять ими. Диски worker нод автоматически определяют размер блока базового тома и подстраиваются под него (`blockSize.matchVolume`), чтобы обеспечить совместимость с backend-системами хранения данных, которые используют секторы не по 512 байт, например LINSTOR DRBD с 4Kn-дисками.

-
- Cluster API : Cozystack использует [Kubernetes Cluster API](#) для подготовки компонентов кластера.

Такая архитектура обеспечивает изолированные, масштабируемые и эффективные тенант-среды Kubernetes.

Справочные материалы по компонентам, которые используются в этом сервисе:

- [Kamaji Control Plane](#)
- [Kamaji — Cluster API](#)
- github.com/clastix/kamaji
- [KubeVirt](#)
- github.com/kubevirt/kubevirt
- github.com/cozystack/etcd-operator
- [Kubernetes Cluster API](#)
- github.com/kubernetes-sigs/cluster-api-provider-kubevirt
- github.com/kubevirt/csi-driver

Breaking Changes

- ****ephemeralStorage renamed to diskSize**** (v1.4): The `nodeGroups[name].ephemeralStorage` field has been renamed to `nodeGroups[name].diskSize`. Existing clusters are migrated transparently by platform migration 41 during the pre-upgrade hook — no manual action is required. Newly written values should use `diskSize`. Existing VMs are rolling-updated via CAPI when the new values are applied. With the Talos worker rollover in this release the field now sizes the single system disk (Talos OS image streamed from `factory.talos.dev`, kubelet state, containerd image cache, local-path PVCs) — the pre-Talos `disk-kubelet` PVC layout was removed. State on that single disk persists across same-VM reboots (virt-launcher restart, guest reboot, node failure); VM replacement by CAPI (e.g. `nodeGroup` field change, `MachineHealthCheck` remediation) provisions a fresh disk.
- Air-gapped tenant workers (Phase 1 Talos rollover) : tenant worker bootstrap moves from Ubuntu containerDisk + kubeadm to a Talos OS disk image fetched over HTTP by CDI and a `./installer/...` installer reference. Both sources are now overridable — set `talos.imageFactoryURL` (default `https://factory.talos.dev`) to a self-hosted Image Factory, a caching mirror, or an internal HTTP file server, and `talos.installerRepository` (default `factory.talos.dev/installer`) to a mirrored registry. The Helm-rendered `*-patch-containerd` Secret that previously plumbed the platform-wide `registries.mirrors` config to tenant workers (via the kubeadm template's containerd certs.d mount) has no consumer in the Talos machineconfig and was removed in this release; in-guest container-image pulls (pods pulling from registries inside the tenant cluster) still honour no per-tenant `registries.mirrors` override until Phase 2 restores it via `machine.registries.mirrors` knobs. So a strict-egress tenant can now mirror the Talos OS image and installer, but rate-limited in-guest registry pulls remain a Phase 2 follow-up — file an issue if you depend on this and it is not yet landed.

- Worker MachineHealthCheck remediation is now enabled by default (Phase 1 Talos rollover): the worker MHC `maxUnhealthy` moved from a hard-coded `0` (remediation effectively off) to a configurable `nodeHealthCheck.maxUnhealthy` defaulting to `"50%"`. CAPI now auto-remediates (deletes and replaces) unhealthy worker Machines while at least 50% of a group stays healthy. `"50%"` deliberately leaves headroom for transient unhealthy nodes during the `kubeadm->Talos` rollover and slow first boots from `factory.talos.dev`; set `nodeHealthCheck.maxUnhealthy: "0%"` to keep the previous remediation-off behaviour until your fleet is stable on Talos workers.
- ****GitOps-managed tenantKubernetes CRs must bump `spec.version` in Git**** (Phase 1 Talos rollover): platform migration 46 patches live `kuberneteses.apps.cozystack.io` objects from `v1.30` to `v1.31` (`v1.30` left the Talos<->Kubernetes support matrix). When the tenant Kubernetes CR is reconciled from Git (Flux/Argo), the next source reconcile re-applies `version: v1.30` over migration 46's patch, which then trips the chart's `_versions.tpl` guard and the HelmRelease fails. Update `spec.version` to `v1.31` (or newer) in your Git source before/with the platform upgrade. Tenants managed via the API or dashboard need no action — migration 46 handles them.
- Remote-accessible LINSTOR StorageClasses are auto-propagated to tenants (v1.5): infra-cluster LINSTOR StorageClasses whose `linstor.csi.linbit.com/allowRemoteVolumeAccess` is not `"false"` are created inside each tenant under the same name (provisioned by `csi.kubevirt.io`). Upgrade caveat: delete any `_manually created_` tenant StorageClass whose name collides with a propagated infra class (e.g. a hand-made `replicated`) before upgrading, or the propagated class cannot be created and the tenant CSI release will not converge. Infra classes that must stay node-local need `allowRemoteVolumeAccess: "false"` set explicitly — an absent annotation is treated as remote-accessible. Propagation is evaluated at HelmRelease render time, so classes added or removed on the infra cluster after a tenant exists propagate only on that tenant's next reconcile.

Поле верхнего уровня `storageClass` помечено в схеме чарта как неизменяемое (`immutable`) — описание контракта и того, какие потребители его применяют, см. в [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#). Поле уровня группы нод `nodeGroups[name].storageClass` намеренно не помечено неизменяемым: оно необязательное и не имеет значения по умолчанию, поэтому строгое правило `self == oldSelf` заблокировало бы любую будущую попытку задать его для уже существующей группы нод.

Parameters

Общие параметры

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

| | | | |
|--------------|--|--------|------------|
| storageClass | Default StorageClass inside the tenant cluster. Remote-accessible LINSTOR classes are auto-propagated to the tenant under the same name. | string | replicated |
|--------------|--|--------|------------|

Application-specific Parameters

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

| | | | |
|-------------------------------|---|-------------------|-----------|
| nodeGroups | <p>Worker nodes configuration map. When left empty, a default node group <code>md0</code> is emitted with <code>minReplicas: 0</code> and <code>roles: [ingress-nginx]</code> — the <code>MachineDeployment</code> renders but provisions no workers until an unschedulable Pod triggers the cluster-autoscaler (or an operator scales the group manually). Enabling <code>addons.ingressNginx.enabled: true</code> on a CR with default <code>nodeGroups: {}</code> therefore requires either supplying a nodeGroup with <code>roles: [ingress-nginx]</code> and <code>minReplicas >= 1</code>, or waiting for the autoscaler to bring up the default <code>md0</code> in response to the <code>ingress-nginx</code> controller Pods becoming Pending. Provide your own groups to take full control — they are not merged with the default, so you may name and omit groups freely.</p> | map[string]object | {} |
| nodeGroups[name].minReplicas | Minimum number of replicas. | int | 0 |
| nodeGroups[name].maxReplicas | Maximum number of replicas. | int | 10 |
| nodeGroups[name].instanceType | Virtual machine instance type. | string | u1.medium |

| | | | |
|---------------------------|---|----------|------|
| nodeGroups[name].diskSize | System disk size for the worker VM. Carries the Talos OS image (factory.talos.dev raw artifact streamed in by CDI), kubelet state, containerd image cache, and any local-path PVCs. Pre-Talos installs used a separate disk-kubelet PVC for kubelet/containerd state; on Talos this is consolidated onto the single system disk imaged from the factory artifact. | quantity | 20Gi |
|---------------------------|---|----------|------|

| | | | |
|--|--|-----------------------|-----------------|
| <code>nodeGroups[name].storageClass</code> | StorageClass for worker node persistent disks. When empty, falls back to the application-level storageClass. Worker VMs live-migrate, so their disks need ReadWriteMany — the RWX access mode is supplied by the chosen StorageClass's CDI StorageProfile, not set on the DataVolume here — and linstor-csi rejects RWX volumes that are not on a DRBD-backed StorageClass, so the fallback targets the replicated/DRBD application storageClass rather than a possibly non-DRBD cluster default. NOTE: deliberately not marked immutable — the field is optional and undefaulted, so a strict <code>self == oldSelf</code> rule would block any future attempt to set it on an existing node group. | <code>string</code> | <code>""</code> |
| <code>nodeGroups[name].roles</code> | List of node roles. | <code>[]string</code> | <code>[]</code> |

| | | | |
|--|--|-----------------------|------------------|
| <code>nodeGroups[name].resources</code> | Explicit CPU and memory for each worker node, as an alternative to <code>instanceType</code> sizing. Optional: when omitted, the node is sized by <code>instanceType</code> . When both <code>cpu</code> and <code>memory</code> are set, they take precedence and <code>instanceType</code> is ignored for that node group (the <code>instancetype</code> is omitted from the VM, since KubeVirt cannot override an <code>instancetype</code> 's CPU/memory). Set both <code>cpu</code> and <code>memory</code> together or neither; setting only one is rejected at render time. | object | <code>{}</code> |
| <code>nodeGroups[name].resources.cpu</code> | CPU available. | quantity | <code>" "</code> |
| <code>nodeGroups[name].resources.memory</code> | Memory (RAM) available. | quantity | <code>" "</code> |
| <code>nodeGroups[name].gpus</code> | List of GPUs to attach (NVIDIA driver requires at least 4 GiB RAM). | <code>[]object</code> | <code>[]</code> |
| <code>nodeGroups[name].gpus[i].name</code> | Name of GPU, such as "nvidia.com/AD102GL_L40S". | string | <code>" "</code> |
| <code>nodeGroups[name].kubenet</code> | Kubelet resource reservations for this node group. | object | <code>{}</code> |
| <code>nodeGroups[name].kubenet.systemReservedMemory</code> | Memory reserved for host OS. Auto-computed from <code>instanceType</code> if empty. | string | <code>" "</code> |

| | | | |
|--|---|--------|-------|
| <code>nodeGroups[name].kubelet.kubeReservedMemory</code> | Memory reserved for kubelet and container runtime. Auto-computed from instanceType if empty. | string | "" |
| <code>nodeGroups[name].kubelet.systemReservedCpu</code> | CPU reserved for host OS. Auto-computed from instanceType if empty. | string | "" |
| <code>nodeGroups[name].kubelet.kubeReservedCpu</code> | CPU reserved for kubelet and container runtime. Auto-computed from instanceType if empty. | string | "" |
| <code>nodeGroups[name].kubelet.evictionHardMemory</code> | Hard eviction threshold for memory (absolute like 200Mi or percentage like 7%). | string | 7% |
| <code>nodeGroups[name].kubelet.evictionSoftMemory</code> | Soft eviction threshold for memory (absolute like 1Gi or percentage like 10%). | string | 10% |
| <code>nodeGroups[name].maxUnhealthy</code> | Per-group override for <code>nodeHealthCheck.maxUnhealthy</code> . When unset, the cluster-wide <code>nodeHealthCheck.maxUnhealthy</code> applies. Accepts a bare integer ("0", "1", ...) or an integer percentage ("0%", "50%"). | string | "" |
| <code>nodeGroups[name].nodeStartupTimeout</code> | Per-group override for <code>nodeHealthCheck.nodeStartupTimeout</code> . When unset, the cluster-wide <code>nodeHealthCheck.nodeStartupTimeout</code> applies. | string | "" |
| <code>version</code> | Kubernetes major.minor version to deploy | string | v1.35 |

| | | | |
|------|---|--------|----|
| host | External hostname for Kubernetes cluster.
Defaults to <cluster-name>.<tenant-host> if empty. | string | "" |
|------|---|--------|----|

Cluster Addons

| Name | Description | Type | Value |
|-----------------------------------|--|--------|---------|
| addons | Конфигурация расширений кластера. | object | {} |
| addons.certManager | Расширение cert-manager. | object | {} |
| addons.certManager.enabled | Включить cert-manager. | bool | false |
| addons.certManager.valuesOverride | Пользовательские переопределения значений Helm. | object | {} |
| addons.cilium | Cilium CNI плагин. | object | {} |
| addons.cilium.valuesOverride | Пользовательские переопределения значений Helm. | object | {} |
| addons.gatewayAPI | Расширение Gateway API. | object | {} |
| addons.gatewayAPI.enabled | Включить Gateway API. | bool | false |
| addons.ingressNginx | Расширение Controller Ingress-NGINX. | object | {} |
| addons.ingressNginx.enabled | Включить controller (требуется ноды с label ingress-nginx). | bool | false |
| addons.ingressNginx.exposeMethod | Метод публикации controller. Допустимые значения: Proxied, LoadBalancer. | string | Proxied |

| | | | |
|--|--|-----------------------|--------------------|
| <code>addons.ingressNginx.hosts</code> | Домены, которые направляются в этот тенант-кластер, когда <code>exposeMethod</code> равен <code>Proxied</code> . | <code>[]string</code> | <code>[]</code> |
| <code>addons.ingressNginx.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.gpuOperator</code> | NVIDIA GPU Operator. | <code>object</code> | <code>{}</code> |
| <code>addons.gpuOperator.enabled</code> | Включить GPU Operator. | <code>bool</code> | <code>false</code> |
| <code>addons.gpuOperator.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.hami</code> | HAMi GPU virtualization middleware. | <code>object</code> | <code>{}</code> |
| <code>addons.hami.enabled</code> | Включить HAMi (требуется GPU Operator). | <code>bool</code> | <code>false</code> |
| <code>addons.hami.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.fluxcd</code> | FluxCD GitOps operator. | <code>object</code> | <code>{}</code> |
| <code>addons.fluxcd.enabled</code> | Включить FluxCD. | <code>bool</code> | <code>false</code> |
| <code>addons.fluxcd.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.monitoringAgents</code> | агенты системы мониторинга. | <code>object</code> | <code>{}</code> |
| <code>addons.monitoringAgents.enabled</code> | Включить агенты системы мониторинга. | <code>bool</code> | <code>false</code> |
| <code>addons.monitoringAgents.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.verticalPodAutoscaler</code> | Vertical Pod Autoscaler. | <code>object</code> | <code>{}</code> |
| <code>addons.verticalPodAutoscaler.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |

| | | | |
|--|--|---------------------|--------------------|
| <code>addons.velero</code> | Расширение Velero для backup/restore. | <code>object</code> | <code>{}</code> |
| <code>addons.velero.enabled</code> | Включить Velero. | <code>bool</code> | <code>false</code> |
| <code>addons.velero.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.coredns</code> | Расширение CoreDNS. | <code>object</code> | <code>{}</code> |
| <code>addons.coredns.valuesOverride</code> | Пользовательские переопределения значений Helm. | <code>object</code> | <code>{}</code> |
| <code>addons.ouroboros</code> | Исправление Hairpin-NAT для ingress-nginx с PROXY-protocol. | <code>object</code> | <code>{}</code> |
| <code>addons.ouroboros.enabled</code> | Включить ouroboros. Требуется <code>addons.ingressNginx.enabled</code> , иначе chart-render завершится ошибкой. Полезно только когда PROXY-protocol подключен в тенант ingress-nginx через <code>valuesOverride</code> . | <code>bool</code> | <code>false</code> |
| <code>addons.ouroboros.valuesOverride</code> | Пользовательские переопределения значений Helm. Operator-key имеет приоритет над defaults Cozystack. | <code>object</code> | <code>{}</code> |

Конфигурация Kubernetes Control Plane

| Name | Description | Type | Value |
|---|--|---------------------|-----------------|
| <code>controlPlane</code> | Kubernetes control-plane configuration. | <code>object</code> | <code>{}</code> |
| <code>controlPlane.replicas</code> | Number of control-plane replicas. | <code>int</code> | 2 |
| <code>controlPlane.apiServer</code> | API Server configuration. | <code>object</code> | <code>{}</code> |
| <code>controlPlane.apiServer.resources</code> | CPU and memory resources for API Server. | <code>object</code> | <code>{}</code> |

| | | | |
|--|--|----------|-----------|
| <code>controlPlane.apiServer.resources.cpu</code> | CPU available. | quantity | "" |
| <code>controlPlane.apiServer.resources.memory</code> | Memory (RAM) available. | quantity | "" |
| <code>controlPlane.apiServer.resourcesPreset</code> | Preset if resources omitted. | string | cl.medium |
| <code>controlPlane.apiServer.extraArgs</code> | Extra command-line flags appended to the tenant kube-apiserver, passed through to KamajiControlPlane <code>spec.apiServer.extraArgs</code> . For OIDC use <code>spec.oidc.mode</code> — this passthrough is the escape hatch for other apiserver flags (<code>--requestheader-uid-headers=X-Remote-Uid</code> , feature gates, etc). Do NOT add legacy <code>--oidc-*</code> flags here when <code>spec.oidc.mode</code> is not <code>None</code> ; the chart injects <code>--authentication-config</code> and the apiserver refuses to boot with both. Empty by default (no change to current behavior). | []string | [] |

| | | | |
|---|--|-----------------------|-----------------|
| <code>controlPlane.apiServer.extraVolumes</code> | Extra volumes added to the control-plane Deployment, passed through to KamajiControlPlane <code>spec.deployment.extraVolumes</code> . Use to mount a ConfigMap or Secret holding an AuthenticationConfiguration file referenced by <code>extraArgs</code> . Each item is a core/v1 Volume, but because the control-plane pod runs on the management cluster only <code>configMap</code> and <code>secret</code> sources are allowed (host-reaching sources like <code>hostPath/csi</code> and token-projection via <code>projected</code> are rejected); each volume must have a unique, non-empty name and exactly one source. The names <code>talos-ca</code> and <code>talos-tls-cert</code> are reserved by the chart. Empty by default. | <code>[]object</code> | <code>[]</code> |
| <code>controlPlane.apiServer.extraVolumeMounts</code> | Extra volume mounts added to the tenant kube-apiserver container, passed through to KamajiControlPlane <code>spec.apiServer.extraVolumeMounts</code> . Each name must reference a volume declared in <code>extraVolumes</code> ; the chart-managed talos secret volumes cannot be mounted. Each item is a core/v1 VolumeMount. Empty by default. | <code>[]object</code> | <code>[]</code> |

| | | | |
|--|--|----------|-----------------------|
| <code>controlPlane.controllerManager</code> | Controller Manager configuration. | object | <code>{}</code> |
| <code>controlPlane.controllerManager.resources</code> | CPU and memory resources for Controller Manager. | object | <code>{}</code> |
| <code>controlPlane.controllerManager.resources.cpu</code> | CPU available. | quantity | <code>""</code> |
| <code>controlPlane.controllerManager.resources.memory</code> | Memory (RAM) available. | quantity | <code>""</code> |
| <code>controlPlane.controllerManager.resourcesPreset</code> | Preset if resources omitted. | string | <code>t1.micro</code> |
| <code>controlPlane.scheduler</code> | Scheduler configuration. | object | <code>{}</code> |
| <code>controlPlane.scheduler.resources</code> | CPU and memory resources for Scheduler. | object | <code>{}</code> |
| <code>controlPlane.scheduler.resources.cpu</code> | CPU available. | quantity | <code>""</code> |
| <code>controlPlane.scheduler.resources.memory</code> | Memory (RAM) available. | quantity | <code>""</code> |
| <code>controlPlane.scheduler.resourcesPreset</code> | Preset if resources omitted. | string | <code>t1.micro</code> |
| <code>controlPlane.konnectivity</code> | Konnectivity configuration. | object | <code>{}</code> |
| <code>controlPlane.konnectivity.server</code> | Konnectivity Server configuration. | object | <code>{}</code> |
| <code>controlPlane.konnectivity.server.resources</code> | CPU and memory resources for Konnectivity. | object | <code>{}</code> |
| <code>controlPlane.konnectivity.server.resources.cpu</code> | CPU available. | quantity | <code>""</code> |
| <code>controlPlane.konnectivity.server.resources.memory</code> | Memory (RAM) available. | quantity | <code>""</code> |
| <code>controlPlane.konnectivity.server.resourcesPreset</code> | Preset if resources omitted. | string | <code>t1.micro</code> |

| | | | |
|---------------------------------------|--|---------------------|-----------------|
| <code>images</code> | Optional image overrides for air-gapped or rate-limited registries. | <code>object</code> | <code>{}</code> |
| <code>images.waitForKubeconfig</code> | Image used by the wait-for-kubeconfig init container. Empty falls back to images/busybox.tag. | <code>string</code> | <code>""</code> |
| <code>images.kubectld</code> | Image used by the bootstrap-token tenant Job (kubectld). Empty falls back to images/kubectld.tag. | <code>string</code> | <code>""</code> |
| <code>images.talosCsrSigner</code> | Image used by the talos-csr-signer sidecar in the Kamaji control plane. Empty falls back to images/talos-csr-signer.tag. | <code>string</code> | <code>""</code> |

Talos Worker Image

| Name | Description | Type | Value |
|--------------------------------|---|---------------------|---|
| <code>talos</code> | Talos worker image configuration. | <code>object</code> | <code>{}</code> |
| <code>talos.version</code> | Talos release used for worker OS image and installer. Must satisfy the chart's Talos<->Kubernetes support matrix against the chosen version . | <code>string</code> | <code>v1.13.6</code> |
| <code>talos.schematicID</code> | Talos image-factory schematic ID. Defaults to the cozystack-tested vanilla schematic. Operators using custom schematics (system extensions, kernel args) override here. | <code>string</code> | <code>ce4c980550dd2ab1b17bbf2b08801c7eb59418eafe8f279833297925d67c7515</code> |

| | | | |
|--|--|--------|--|
| <code>talos.imageFactoryURL</code> | Base URL of the Talos Image Factory that serves the worker OS disk image (the <code>openstack-amd64.raw.xz</code> raw artifact streamed in by CDI over HTTP). Defaults to the public factory. Point at a self-hosted Image Factory, a caching mirror, or an internal HTTP file server for air-gapped, rate-limited, or flaky-egress environments. No trailing slash. | string | <code>https://factory.talos.dev</code> |
| <code>talos.installerRepository</code> | OCI repository prefix for the Talos installer image used by the in-guest <code>talos-reconcile</code> upgrade Job. Resolved as <code><installerRepository>/<schematicID>:<version></code> . Defaults to the public factory's installer path. Override for air-gapped or mirrored registries. No trailing slash. | string | <code>factory.talos.dev/installer</code> |

Node Health Check

| Name | Description | Type | Value |
|------------------------------|---|--------|-----------------|
| <code>nodeHealthCheck</code> | MachineHealthCheck tuning for worker node groups. | object | <code>{}</code> |

| | | | |
|---|--|--------|-----|
| <code>nodeHealthCheck.maxUnhealthy</code> | Maximum number of unhealthy nodes tolerated per node group before remediation is paused. The MHC admission webhook accepts either a bare integer ("0", "1", ...) or a percentage ("0%", "50%"); bare numeric strings are rejected, so the safer default is to express the value as a percentage. Default "50%" leaves headroom for transient unhealthy nodes during the kubeadm-to-Talos rollover and slow first boots from factory.talos.dev. Drop to "0%" once the fleet is stable on Talos workers. | string | 50% |
| <code>nodeHealthCheck.nodeStartupTimeout</code> | Maximum time a Machine is allowed to spend reaching the Ready condition before it is remediated. Raise for slow first boots (Talos image fetch from factory.talos.dev or a busy storage class on the kubevirt-csi PVC populator). | string | 10m |

OIDC

| Name | Description | Type | Value |
|-------------------|--|--------|-------|
| <code>oidc</code> | OIDC authentication and per-user RBAC for the tenant kube-apiserver. See docs/oidc-tenant.md for the operator guide. | object | {} |

| | | | |
|---|---|--------|------------------|
| <code>oidc.mode</code> | Identity mode. <code>None</code> : no OIDC, only the static admin kubeconfig works. <code>System</code> : trust the platform <code>cozy</code> realm via a per-cluster public client with audience binding; zero-config default. <code>CustomConfig</code> : trust a tenant-supplied issuer directly (BYO); <code>cozy</code> is not in the path. | string | None |
| <code>oidc.customConfig</code> | Tenant-supplied AuthenticationConfiguration; consumed only when <code>mode: CustomConfig</code> . | object | <code>{}</code> |
| <code>oidc.customConfig.config</code> | Inline AuthenticationConfiguration YAML (<code>apiserver.config.k8s.io/v1beta1</code>). The chart writes the value verbatim into a Secret mounted at <code>/etc/kubernetes/authentication-config/config.yaml</code> on the kube-apiserver. Mutually exclusive with <code>secretRef.name</code> . | string | <code>" "</code> |
| <code>oidc.customConfig.secretRef</code> | Reference to an existing Secret in the tenant namespace carrying the AuthenticationConfiguration. | object | <code>{}</code> |
| <code>oidc.customConfig.secretRef.name</code> | Name of an existing Secret in the tenant (release) namespace whose <code>config.yaml</code> key holds an <code>apiserver.config.k8s.io/v1beta1 AuthenticationConfiguration</code> . Mutually exclusive with <code>customConfig.config</code> . | string | <code>" "</code> |

| | | | |
|----------------------------------|--|-----------------------|-----------------|
| <code>oidc.users</code> | Users granted access to the tenant cluster; each entry produces one <code>ClusterRoleBinding</code> inside the tenant cluster. Works for both <code>System</code> and <code>CustomConfig</code> modes. | <code>[]object</code> | <code>[]</code> |
| <code>oidc.users[i].email</code> | Email address matched against the <code>email</code> claim from the issuer. Used verbatim as the <code>User:</code> subject in the <code>ClusterRoleBinding</code> inside the tenant cluster. The <code>email</code> claim is a built-in OIDC scope requested explicitly by the chart-generated <code>kubectrl oidc-login kubeconfig</code> ; every conformant OIDC provider (cozy realm included) emits it. | <code>string</code> | <code>""</code> |
| <code>oidc.users[i].role</code> | Role to bind: <code>admin</code> maps to <code>ClusterRole/cluster-admin</code> , <code>view</code> maps to <code>ClusterRole/view</code> . | <code>string</code> | <code>{}</code> |

Parameter examples and reference

resources and resourcesPreset

`resources` задает явную конфигурацию CPU и memory для каждой реплики. Если значение пустое, применяется preset из `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задает именованную конфигурацию CPU и memory для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

| Preset name | CPU | memory |
|-------------|------|--------|
| nano | 250m | 128Mi |
| micro | 500m | 256Mi |
| small | 1 | 512Mi |
| medium | 1 | 1Gi |
| large | 2 | 2Gi |
| xlarge | 4 | 4Gi |
| 2xlarge | 8 | 8Gi |

Ресурсы типа инстанса

В Cozystack доступны следующие ресурсы для типов инстансов:

| Имя | vCPUs | Память |
|----------------|-------|--------|
| cx1.2xlarge | 8 | 16Gi |
| cx1.2xlarge1gi | 8 | 16Gi |
| cx1.4xlarge | 16 | 32Gi |
| cx1.4xlarge1gi | 16 | 32Gi |
| cx1.8xlarge | 32 | 64Gi |
| cx1.8xlarge1gi | 32 | 64Gi |
| cx1.large | 2 | 4Gi |
| cx1.large1gi | 2 | 4Gi |
| cx1.medium | 1 | 2Gi |
| cx1.medium1gi | 1 | 2Gi |
| cx1.xlarge | 4 | 8Gi |
| cx1.xlarge1gi | 4 | 8Gi |
| d1.2xlarge | 8 | 32Gi |
| d1.2xmedium | 2 | 4Gi |
| d1.4xlarge | 16 | 64Gi |
| d1.8xlarge | 32 | 128Gi |
| d1.large | 2 | 8Gi |
| d1.medium | 1 | 4Gi |

| | | |
|----------------|----|-------|
| d1.micro | 1 | 1Gi |
| d1.nano | 1 | 512Mi |
| d1.small | 1 | 2Gi |
| d1.xlarge | 4 | 16Gi |
| gn1.2xlarge | 8 | 32Gi |
| gn1.4xlarge | 16 | 64Gi |
| gn1.8xlarge | 32 | 128Gi |
| gn1.xlarge | 4 | 16Gi |
| m1.2xlarge | 8 | 64Gi |
| m1.2xlargeelgi | 8 | 64Gi |
| m1.4xlarge | 16 | 128Gi |
| m1.4xlargeelgi | 16 | 128Gi |
| m1.8xlarge | 32 | 256Gi |
| m1.8xlargeelgi | 32 | 256Gi |
| m1.large | 2 | 16Gi |
| m1.largeelgi | 2 | 16Gi |
| m1.xlarge | 4 | 32Gi |
| m1.xlargeelgi | 4 | 32Gi |
| n1.2xlarge | 16 | 32Gi |
| n1.4xlarge | 32 | 64Gi |
| n1.8xlarge | 64 | 128Gi |
| n1.large | 4 | 8Gi |
| n1.medium | 4 | 4Gi |
| n1.xlarge | 8 | 16Gi |
| o1.2xlarge | 8 | 32Gi |
| o1.4xlarge | 16 | 64Gi |
| o1.8xlarge | 32 | 128Gi |
| o1.large | 2 | 8Gi |
| o1.medium | 1 | 4Gi |
| o1.micro | 1 | 1Gi |

| | | |
|-------------|----|-------|
| o1.nano | 1 | 512Mi |
| o1.small | 1 | 2Gi |
| o1.xlarge | 4 | 16Gi |
| rt1.2xlarge | 8 | 32Gi |
| rt1.4xlarge | 16 | 64Gi |
| rt1.8xlarge | 32 | 128Gi |
| rt1.large | 2 | 8Gi |
| rt1.medium | 1 | 4Gi |
| rt1.micro | 1 | 1Gi |
| rt1.small | 1 | 2Gi |
| rt1.xlarge | 4 | 16Gi |
| u1.2xlarge | 8 | 32Gi |
| u1.2xmedium | 2 | 4Gi |
| u1.4xlarge | 16 | 64Gi |
| u1.8xlarge | 32 | 128Gi |
| u1.large | 2 | 8Gi |
| u1.medium | 1 | 4Gi |
| u1.micro | 1 | 1Gi |
| u1.nano | 1 | 512Mi |
| u1.small | 1 | 2Gi |
| u1.xlarge | 4 | 16Gi |

Резервирование ресурсов Kubelet

Каждая группа нод поддерживает объект `kubelet`, который задает, сколько памяти и CPU kubelet резервирует для host OS и Kubernetes/system компонентов, работающих на worker ноде.

Если `systemReservedMemory` или `kubeReservedMemory` оставлены пустыми, они вычисляются автоматически по следующей формуле:

1. Определяется эффективный объем памяти ноды:

- Если `resources.memory` задан явно, используется это значение.
- Иначе ищется `instanceType` и используется его значение `memory.guest`.
- Если недоступно ни то ни другое, используется минимальная резервация (256Mi).

2. Вычисляется 5% от эффективного объема памяти (в MiB, с округлением вниз).

3. Результат ограничивается диапазоном [256Mi, 1Gi] :

- Ноды с 5 GiB или меньше получают минимальный резерв 256Mi.
- Ноды с 20 GiB или больше получают максимальный резерв 1Gi.

По умолчанию `systemReservedMemory` и `kubeReservedMemory` получают одинаковое автоматически вычисленное значение.

CPU резервирование (`systemReservedCpu`, `kubeReservedCpu`) работают по той же схеме: 5% от effective CPU с ограничением диапазоном [50m, 500m]. Оба значения вычисляются автоматически, если оставлены пустыми.

Значения kubelet по умолчанию

| Параметр | По умолчанию | Описание |
|--------------------------------------|---------------------|---|
| <code>systemReservedMemory</code> | auto-computed | Память, зарезервированная для host OS |
| <code>kubeReservedMemory</code> | auto-computed | Память, зарезервированная для kubelet и container runtime |
| <code>systemReservedCpu</code> | auto-computed | CPU, зарезервированный для host OS |
| <code>kubeReservedCpu</code> | auto-computed | CPU, зарезервированный для kubelet и container runtime |
| <code>evictionHardMemory</code> | 7% | Жёсткий порог вытеснения по памяти |
| <code>evictionSoftMemory</code> | 10% | Мягкий порог вытеснения по памяти |
| <code>evictionSoftGracePeriod</code> | 1m30s _(hardcoded)_ | Время, в течение которого мягкий порог вытеснения по памяти должен быть нарушен перед запуском вытеснения |
| <code>evictionMinimumReclaim</code> | 256Mi _(hardcoded)_ | Минимальный объем памяти, освобождаемый за одно действие вытеснения |

Порог вытеснения можно задавать в процентах (например, 7%) или абсолютных значениях (например, 200Mi). Оба порога должны использовать один тип единиц. Жёсткий порог вытеснения должен быть строго меньше мягкого порога.

Параметры `evictionSoftGracePeriod` и `evictionMinimumReclaim` сейчас жёстко заданы в шаблоне и не могут быть переопределены через values.

Capacity Аннотации

Когда настроены резервации ресурсов kubelet, аннотации `capacity.cluster-autoscaler.kubernetes.io/memory` и `capacity.cluster-autoscaler.kubernetes.io/cpu` на `MachineDeployments` показывают распределяемые значения вместо полных ресурсов ноды. Доступная для подов память вычисляется как общий объём памяти за вычетом `system-reserved`, `kube-reserved` и `eviction-hard`. Доступное для подов CPU вычисляется как общее количество за вычетом `system-reserved` и `kube-reserved`. Так аннотации соответствуют значениям, которые cluster autoscaler использует в расчетах масштабирования.

При обновлении с версии без этой возможности autoscaler после обновления аннотации может увидеть уменьшенную емкость для каждой ноды, что способно запустить дополнительные операции `scale-up`. Обычно никаких действий не требуется — новые значения отражают фактическую память, доступную для `scheduling workload`.

Note: Если не задан ни `resources.memory`, ни `instanceType`, порог вытеснения (по умолчанию 7% hard / 10% soft) все равно применяются kubelet во время выполнения, но `capacity` аннотации не рендерится. Без этой аннотации cluster-autoscaler не учитывает эти резервации и может запланировать на ноду слишком много подов, что приведёт к срабатыванию вытеснения.

Пример: переопределение резерваций kubelet

```
nodeGroups:
md0:
instanceType: "u1.large"
kubelet:
systemReservedMemory: "256Mi"
kubeReservedMemory: "256Mi"
evictionHardMemory: "500Mi"
evictionSoftMemory: "1Gi"
```

Серия U: Universal

Серия U имеет сбалансированный профиль и предоставляет ресурсы для приложений общего назначения.

`_U_` — сокращение от “Universal”, то есть серия рассчитана на универсальные рабочие нагрузки.

VM этих типов инстансов делят физические CPU-ядра с другими VM по принципу разделения процессорного времени.

Характеристики серии U

Особенности этой серии:

- `_Burstable CPU performance_` \- рабочая нагрузка имеет базовый уровень вычислительной производительности, но может кратковременно превышать его при наличии свободных вычислительных ресурсов.

-
- `_vCPU-To-Memory Ratio (1:4)_` \- соотношение vCPU к memory 1:4, чтобы снизить уровень шума на каждой ноде.

Серия O: Overcommitted

Серия O основана на серии U; единственное отличие — избыточное выделение памяти.

`_O_` — сокращение от “Overcommitted”.

Характеристики Серии O

Особенности этой серии:

- `_Burstable CPU performance_` \- рабочая нагрузка имеет базовый уровень вычислительной производительности, но может кратковременно превышать его при наличии свободных ресурсов хоста.
- `_Overcommitted Memory_` \- память используется с избытком, чтобы получить более высокую плотность размещения рабочей нагрузки.
- `_vCPU-To-Memory Ratio (1:4)_` \- соотношение vCPU к memory 1:4, чтобы снизить уровень шума каждой ноды.

Серия CX: Compute Exclusive

Серия CX предоставляет выделенные вычислительные ресурсы для приложений с высокой нагрузкой на CPU.

`_CX_` — сокращение от “Compute Exclusive”.

Выделенные ресурсы предоставляются вычислительным потокам виртуальной машины. Чтобы это обеспечить, запрашиваются дополнительные ядра (в зависимости от количества дисков и NIC), которые разгружают IO потоки с ядрами, выделенных под рабочие нагрузки. Кроме того, в этой серии топологии NUMA используемых ядер передается в VM.

Характеристики серии CX

Особенности этой серии:

- `_Hugepages_` \- hugepages используются для повышения производительности памяти.
- `_Dedicated CPU_` \- физические ядра эксклюзивно назначаются каждому vCPU, чтобы дать рабочей нагрузке фиксированные и высокие гарантии вычислительной мощности.
- `_Isolated emulator threads_` \- потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.
- `_vNUMA_` \- физическая топология NUMA отражается внутри гостевой VM, чтобы оптимизировать использование кеша на стороне гостевой VM.
- `_vCPU-To-Memory Ratio (1:2)_` \- соотношение vCPU к memory 1:2.

Серия M: Memory

Серия M предоставляет ресурсы для приложений с высокой нагрузкой на память.

M — сокращение от “Memory”.

Характеристики серии M

Особенности этой серии:

- _Hugepages_ \- hugepages используются для повышения производительности памяти.
- _Burstable CPU performance_ \- рабочая нагрузка имеет базовый уровень вычислительной производительности, но может кратковременно превышать его при наличии свободных ресурсов хоста.
- _vCPU-To-Memory Ratio (1:8)_ \- соотношение vCPU к memory 1:8, чтобы заметно снизить уровень шума каждой ноды.

Серия RT: RealTime

Серия RT предоставляет ресурсы для приложений реального времени, например Oslat.

RT — сокращение от “realtime”.

Эта серия типов инстансов требует нод, способных запускать приложения реального времени.

Характеристики серии RT

Особенности этой серии:

- _Hugepages_ \- hugepages используются для повышения производительности памяти.
- _Dedicated CPU_ \- физические ядра эксклюзивно назначаются каждому vCPU, чтобы дать рабочей нагрузке фиксированные и высокие гарантии вычислительной мощности.
- _Isolated emulator threads_ \- потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.
- _vCPU-To-Memory Ratio (1:4)_ \- соотношение vCPU к memory 1:4 начиная с размера medium.

•

GPU Sharing с HAMi

Как включить дробное разделение GPU в tenant-кластерах Kubernetes с помощью HAMi.

Как перенести реплики etcd в tenant-кластерах

Как перенести реплики tenant-кластеров etcd, которые используются tenant-кластерами Kubernetes.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

GPU Sharing с HAMi

<https://cozystack.ru/docs/v1.6/kubernetes/gpu-sharing/>

Как включить дробное разделение GPU в tenant-кластерах Kubernetes с помощью HAMi.

HAMi (Heterogeneous AI Computing Virtualization Middleware) — проект CNCF Sandbox, который включает дробное разделение GPU в Kubernetes. Вместо выделения целого GPU под одну рабочую нагрузку HAMi позволяет контейнерам запрашивать конкретный объем GPU-памяти и вычислительных ядер.

Это руководство описывает GPU шаринг для контейнеров в tenant-кластерах Kubernetes. О пробросе GPU в виртуальные машины на управляемом кластере см. [GPU Passthrough](#).

Как это работает

HAMi работает между планировщиком Kubernetes и драйвером NVIDIA GPU:

- Scheduler Extender добавляет GPU-aware решения при планировании (filtering и binding), чтобы pod попадали на ноды с достаточной GPU-емкостью.
- Device Plugin регистрирует виртуальные GPU-ресурсы (nvidia.com/gpu, nvidia.com/gpumem, nvidia.com/gpucore) в kubelet.
- MutatingWebhook автоматически направляет GPU pod в планировщик HAMi.
- HAMi-core (libvgpu.so) внедряется в контейнеры рабочей нагрузки через LD_PRELOAD, чтобы обеспечить изоляцию памяти и вычислений на уровне CUDA API.

Когда HAMi включен, встроенный плагин устройств GPU Operator автоматически отключается, чтобы избежать конфликтов при регистрации ресурсов.

Требования

- Tenant-кластер Kubernetes с worker нодами, для которых включены GPU (node groups с настроенными GPU).
- Расширение GPU Operator включен в tenant-кластере.

Включение HAMi

Включите GPU Operator и HAMi в конфигурации tenant-кластера Kubernetes:

apiVersion: apps.cozystack.io/v1alpha1

kind: Kubernetes

metadata:

```
name: my-cluster
namespace: tenant-example
spec:
  nodeGroups:
  gpu-workers:
  minReplicas: 1
  maxReplicas: 3
  instanceType: u1.xlarge
  gpus:
    - name: nvidia.com/GA102GL_A10
  addons:
  gpuOperator:
  enabled: true
  hami:
  enabled: true
```

Примените эту конфигурацию:

```
kubectl apply -f my-cluster.yaml
```

Запрос дробных GPU-ресурсов

После запуска HAMi рабочие нагрузки могут запрашивать дробные GPU-ресурсы:

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-workload
spec:
  containers:
    - name: cuda-app
  image: nvcr.io/nvidia/cuda:11.8.0-base-ubuntu20.04
  resources:
    limits:
      nvidia.com/gpu: 1
      nvidia.com/gpumem: 3000
      nvidia.com/gpucore: 30
```

В примере выше используется абсолютный объем памяти (`gpumem`). Используйте `gpumem-percentage`, если нужна переносимость между моделями GPU с разным объемом памяти.

| Ресурс | Описание |
|---|--|
| <code>nvidia.com/gpu</code> | Запрошенное количество виртуальных GPU |
| <code>nvidia.com/gpumem</code> | Лимит GPU-памяти в MiB |
| <code>nvidia.com/gpucore</code> s | Доля вычислительных ядер GPU в процентах (1–100) |
| <code>nvidia.com/gpumem-percentage</code> | Лимит GPU-памяти в процентах (1–100) |

Используйте `nvidia.com/gpumem-percentage` вместо `nvidia.com/gpumem`, если нужен переносимый лимит, который работает на разных моделях GPU без знания точного объема памяти.

Если `gpumem` и `gpucore`s не указаны, контейнер получает доступ ко всей памяти и вычислительной емкости GPU. При этом слой виртуализации HAMi остается активным — это не то же самое, что проброс GPU на bare-metal-уровне.

Пользовательская конфигурация

Поведение HAMi можно настроить через `valuesOverride` в конфигурации расширения:

```
addons:
hami:
enabled: true
valuesOverride:
hami:
devicePlugin:
deviceSplitCount: 10
deviceMemoryScaling: 1
scheduler:
defaultSchedulerPolicy:
nodeSchedulerPolicy: binpack
gpuSchedulerPolicy: spread
```

Все параметры ниже указываются относительно ключа `valuesOverride.hami`, показанного в примере выше.

| Параметр | Описание | По умолчанию |
|----------|----------|--------------|
|----------|----------|--------------|

| | | |
|---|---|----------------------|
| <code>devicePlugin.deviceSplitCount</code> | Максимальное количество виртуальных GPU на один физический GPU | 10 |
| <code>devicePlugin.deviceMemoryScaling</code> | Коэффициент избыточности для памяти (>1.0 включает избыточность) | 1 |
| <code>scheduler.defaultSchedulerPolicy.nodeSchedulerPolicy</code> | Стратегия размещения по нодам: <code>binpack</code> или <code>spread</code> | <code>binpack</code> |
| <code>scheduler.defaultSchedulerPolicy.gpuSchedulerPolicy</code> | Стратегия размещения по GPU: <code>binpack</code> или <code>spread</code> | <code>spread</code> |

Известные ограничения

Совместимость с glibc

HAMi-core использует приватный символ `glibc (_dl_sym)`, который был удален в `glibc 2.34`. Это влияет только для образов контейнеров рабочей нагрузки — собственные компоненты HAMi и host OS не затрагиваются.

| Base image | glibc | Изоляция |
|---------------|-------|---|
| Ubuntu 20.04 | 2.31 | Полная (memory + compute) |
| Ubuntu 22.04 | 2.35 | Только изоляция памяти (Изоляция вычислительных ресурсов завершается неявной ошибкой) |
| Ubuntu 24.04 | 2.39 | Изоляции нет (HAMi-core тихо не загружается) |
| Alpine (musl) | N/A | Несовместим |

Когда HAMi-core не загружается, рабочие нагрузки продолжают работать, но без каких-либо лимитов GPU-ресурсов. Это может привести к ошибкам GPU out-of-memory у рабочих нагрузок, размещенных на том же GPU.

Разница в поведении Ubuntu 22.04 и 24.04 основана на upstream-тестировании — подробности см. в [HAMi-core #174](#).

Большинство актуальных образов CUDA 12.x и PyTorch 2.x используют Ubuntu 22.04+, поэтому изоляция вычислительных ресурсов с ними работать не будет. Для полной изоляции используйте образа на базе Ubuntu 20.04 или старше, пока не будет принят [upstream fix](#).

Alpine / musl libc

HAMi-core несовместим с musl libc. Поддерживаются только образа контейнеров на базе glibc (Debian, Ubuntu, RHEL).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Как перенести реплики etcd в tenant-кластерах

<https://cozystack.ru/docs/v1.6/kubernetes/relocate-etcd/>

Как перенести реплики tenant-кластеров etcd, которые используются tenant-кластерами Kubernetes.

Tenant-кластеры Kubernetes используют собственные кластеры etcd, а не etcd management-кластера. Такие кластеры etcd разворачиваются внутри tenant и доступны управляемым Kubernetes-кластерам, развернутым в этом tenant и его sub-tenant.

Реплики tenant-кластера etcd можно переносить между нодами, например для обслуживания. Сейчас операции управления tenant-кластерами etcd не автоматизированы, но такую задачу можно выполнить вручную.

Сначала установите плагин `kubectl-etcd` для `kubectl`:

```
go install github.com/aenix-io/etcd-operator/cmd/kubectl-etcd@latest
```

Теперь можно управлять репликами etcd. Скрипт ниже удаляет реплику `etcd-2` из кластера etcd, а затем добавляет ее обратно.

```
# tenant, которому принадлежит кластер etcd
```

```
NAMESPACE=tenant-demo
```

```
# реплика etcd
```

```
RM=etcd-2
```

```
POD=$(kubectl get pod -n "$NAMESPACE" -l app.kubernetes.io/name=etcd --no-headers | awk '$2 == "1/1" && $1 != "$RM" {print $1; exit;}')
```

| | |
|---|--|
| RMID=\$(kubectl etcd -n \$NAMESPACE -p \$POD members | awk '\$2 == "\$RM" {print \$1}' |
|---|--|

```
# удалить реплику
```

```
kubectl delete -n $NAMESPACE pvc/data-$RM pod/$RM
```

```
if [ -n $RMID ]; then
```

```
kubectl etcd -n $NAMESPACE -p $POD remove-member "$RMID"
```

```
fi
```

```
# добавить реплику обратно
```

```
kubectl etcd -n $NAMESPACE -p $POD add-member "https://$RM.etcd-headless.$NAMESPACE.svc:2380"
```

```
kubectl wait --for=condition=ready pod -n $NAMESPACE $RM --timeout=2m
```

```
kubectl etcd -n $NAMESPACE -p $RM members
```

Подробнее о вложенности tenant и общих сервисах см. в [руководстве по tenants](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Managed Applications

Managed Applications: Guides and Reference

<https://cozystack.ru/docs/v1.6/applications/>

Learn how to deploy, configure, access, and backup managed applications in Cozystack.

Available Application Versions

Cozystack deploys applications in two complementary ways:

- Operator-managed applications – Cozystack bundles a specific version of a Kubernetes Operator that installs and continuously reconciles the application. As a rule, the operator chooses one of the most recent stable versions of the application by default.
- Chart-managed applications – When no mature operator exists, Cozystack packages an upstream (or in-house) Helm chart. The chart's `appVersion` pin tracks the latest stable upstream release, keeping deployments secure and up-to-date.

•

[Application Backup and Recovery](#)

Back up and restore managed databases (Postgres, MariaDB, ClickHouse, Etcd) with BackupJob, Plan, and RestoreJob.

[Adding External Applications to Cozystack Catalog](#)

Learn how to add managed applications from external sources

[Managed ClickHouse Service](#)

[FoundationDB](#)

[Managed Harbor Container Registry](#)

[Managed Kafka Service](#)

[Managed MariaDB Service](#)

[Управляемый сервис MongoDB](#)

[Управляемый сервис NATS](#)

[Управляемый сервис OpenBAO](#)

[Управляемый сервис PostgreSQL](#)

[Управляемый сервис Qdrant](#)

[Управляемый сервис RabbitMQ](#)

[Управляемый сервис Redis](#)

[Справочник по приложению Tenant](#)

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0 (96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

Application Backup and Recovery

<https://cozystack.ru/docs/v1.6/applications/backup-and-recovery/>

Back up and restore managed databases (Postgres, MariaDB, ClickHouse, Etc) with BackupJob, Plan, and RestoreJob.

This guide covers backing up and restoring Cozystack-managed databases — Postgres, MariaDB, ClickHouse, and Etc — as a tenant user: running one-off and scheduled backups, checking status, and restoring from a backup either in place or into a separate target instance.

****Storage and theBackupClass are platform-provisioned.**** Cozystack ships a single cluster-scoped BackupClass named `cozy-default` that covers Postgres, MariaDB, ClickHouse, Etc, VMInstance, and VMDisk via a per-Kind `strategies[]` array. You reference it by name from BackupJob / Plan / RestoreJob — there is no per-application BackupClass, and you do not create or supply S3 credentials, endpoints, or paths. The platform projects a credentials Secret (`cozy-backups-creds`) into your tenant namespace automatically right before each BackupJob or RestoreJob runs.

If your administrator has created additional sibling BackupClass resources (different retention, different storage, etc.), ask for the name and substitute it for `cozy-default` in the examples below. BackupClass is cluster-scoped, so you cannot list it under a tenant kubeconfig — your administrator will tell you which names are valid.

Admins follow the [Backup Classes](#) guide.

These backups are data-only. Each strategy snapshots the database contents through the operator's native mechanism (CloudNativePG barman, mariadb-operator dumps, Altinity `clickhouse-backup`). They do not capture the `apps.cozystack.io/*` CR, its HelmRelease, chart values, or operator-managed Secrets.

To restore you must either:

- keep the source application alive and restore in place (each driver re-bootstraps data into the existing operator-managed cluster), or
- pre-provision an empty target application of the same Kind, then restore into it.

For backups that include the application's Helm release, CRs, and PVC snapshots (used for VMInstance / VMDisk), see [Backup and Recovery \(VMs\)](#).

Prerequisites

- A BackupClass name. On a default install this is `cozy-default`, which covers Postgres, MariaDB, ClickHouse, and Etc. If your administrator has created a sibling class, substitute that name everywhere below.
- An existing managed-DB application (Postgres, MariaDB, ClickHouse, or Etc) in your tenant namespace.
- `kubectl` and a tenant kubeconfig with the `tenant-<ns>-admin` role.

The examples below assume `tenant-user` for the tenant namespace; substitute your own.

Run a backup

One-off backup

Use a `BackupJob` for an ad-hoc backup (for example, before a risky change):

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: BackupJob
```

```
metadata:
```

```
name: my-postgres-adhoc
```

```
namespace: tenant-user
```

```
spec:
```

```
applicationRef:
```

```
apiGroup: apps.cozystack.io
```

```
kind: Postgres
```

```
name: my-postgres
```

```
backupClassName: cozy-default
```

```
kubectl apply -f backupjob.yaml
```

```
kubectl -n tenant-user get backupjobs
```

```
kubectl -n tenant-user describe backupjob my-postgres-adhoc
```

When the `BackupJob` reaches `phase: Succeeded`, the driver creates a `Backup` object with the same name. That name is what you reference when restoring.

Replace `Postgres` with `MariaDB`, `ClickHouse`, or `EtcD` for the other drivers — the `BackupClass` (`cozy-default`) is the same; the platform-shipped class binds a strategy for every supported `Kind`.

Scheduled backup

Use a `Plan` for cron-driven recurring backups:

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: Plan
```

```
metadata:
```

```
name: my-postgres-daily
```

```
namespace: tenant-user
```

```
spec:
  applicationRef:
  apiGroup: apps.cozystack.io
  kind: Postgres
  name: my-postgres
  backupClassName: cozy-default
  schedule:
  type: cron
  cron: "0 /6 " # every 6 hours
```

Each scheduled run creates a [BackupJob](#) (and, on success, a [Backup](#)) named after the [Plan](#) with a timestamp suffix.

```
kubectl apply -f plan.yaml
kubectl -n tenant-user get plans
kubectl -n tenant-user get backupjobs -l backups.cozystack.io/plan=my-postgres-daily
```

Check backup status

List [BackupJob](#) and [Backup](#) resources in the namespace:

```
kubectl -n tenant-user get backupjobs
kubectl -n tenant-user get backups
```

Inspect a failed run:

```
kubectl -n tenant-user get backupjob my-postgres-adhoc -o jsonpath='{.status.message}'
kubectl -n tenant-user describe backupjob my-postgres-adhoc
kubectl -n tenant-user get events --field-selector involvedObject.name=my-postgres-adhoc
```

If `status.message` does not pinpoint the failure, hand the [BackupJob](#) name to your administrator and they will inspect the operator-native CR the driver created (see [Backup Classes](#) in the admin guide).

Restore in place

An in-place restore replays the backup into the same application. Use this to roll back accidental deletion or corruption on a live database you intend to keep using under the same name.

In-place restore is destructive. Each driver wipes or replaces existing data on the source application; any writes since the backup point are lost. If you cannot afford to lose recent writes, use Restore to a copy instead.

```
apiVersion: backups.cozystack.io/v1alpha1
kind: RestoreJob
metadata:
  name: my-postgres-restore-inplace
  namespace: tenant-user
spec:
  backupRef:
    name: my-postgres-adhoc
  # targetApplicationRef omitted: driver restores into Backup.spec.applicationRef.
  # options:
  # recoveryTime: "2026-05-01T12:00:00Z" # Postgres only; RFC3339 PITR

kubectl apply -f restorejob.yaml
kubectl -n tenant-user get restorejobs
kubectl -n tenant-user describe restorejob my-postgres-restore-inplace
```

Per-driver caveats

- Postgres (CNPG) — the driver deletes the live `cnpg.io/Cluster` and its PVCs, then re-bootstraps from the Barman archive. Connections drop for the duration. `spec.options.recoveryTime` (RFC3339) is supported for point-in-time recovery; omit it to restore to the latest WAL.
- MariaDB — the operator replays the logical dump into the live MariaDB via `mariadb-import`. Pre-existing tables will collide; pre-truncate the relevant schemas if your dump does not include `DROP TABLE`.
- ClickHouse — the Altinity strategy does not pass `clickhouse-backup --rm`. You are responsible for dropping conflicting tables on the source before submitting the `RestoreJob`; otherwise the operation fails with a duplicate-table error.

Restore to a copy

A to-copy restore replays the backup into a different , freshly-provisioned application of the same Kind. Use this for disaster-recovery drills, side-by-side validation, branch databases, or migrating to a new version of the upstream operator.

First, provision an empty target application with the same Kind. For example, an empty `Postgres`:

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: Postgres
metadata:
name: my-postgres-restored
namespace: tenant-user
spec:
# ...same shape as the source, no bootstrap data required...
```

Wait for the target to become Ready, then submit a `RestoreJob` that points at it:

```
apiVersion: backups.cozystack.io/v1alpha1
kind: RestoreJob
metadata:
name: my-postgres-restore-to-copy
namespace: tenant-user
spec:
backupRef:
name: my-postgres-adhoc
targetApplicationRef:
apiGroup: apps.cozystack.io
kind: Postgres
name: my-postgres-restored
```

The source application stays untouched. Cross-namespace restores are not supported — `targetApplicationRef` is a local reference; the target must live in the same namespace as the `RestoreJob`.

Limitations and lifecycle

- Data-only scope. Application CRs, HelmReleases, chart values, and operator-managed Secrets (e.g. `cnpg.io` superuser secret, `clickhouse-installation` users) are not captured. Pre-provision the target application before a to-copy restore.
- Archive retention is driver-owned. Deleting a Cozystack `Backup` CR removes the artefact reference but leaves the actual S3 object intact. Each driver enforces its own retention:
- CNPG: `retentionPolicy` on the strategy (admin-owned; default `30d` in the admin example).
- MariaDB: `cleanupStrategy` on the operator-side `Backup` CR or rotation at the bucket level (admin-owned).
- ClickHouse: governed by the in-pod sidecar's retention configuration. To purge an archive ahead of schedule, ask your administrator — the call goes against the `clickhouse-backup` HTTP API on the sidecar.
- ClickHouse depends on the in-chart sidecar. The Altinity strategy is a thin HTTP client; the backup itself runs inside each `chi-*` Pod via `clickhouse-backup`. Disabling `backup.enabled` on the

application also disables the BackupClass flow.

Troubleshooting

If a BackupJob or RestoreJob ends in phase: `Failed`, start with what you can see in your namespace:

```
kubectl -n tenant-user get backupjob my-postgres-adhoc -o jsonpath='{.status.message}'
```

```
kubectl -n tenant-user get restorejob my-postgres-restore-inplace -o jsonpath='{.status.message}'
```

```
kubectl -n tenant-user describe backupjob my-postgres-adhoc
```

```
kubectl -n tenant-user get events --field-selector involvedObject.name=my-postgres-adhoc
```

If those do not explain the failure, the next layer of diagnostics lives on the operator-native CR the driver created (cnpg.io/Backup, k8s.mariadb.com/Backup, etcd-operator.cozystack.io/EtcdSnapshot, or the ClickHouse strategy Pod logs). These resources are not reachable under the tenant kubeconfig — hand the BackupJob name to your administrator and they will follow [Backup Classes](#).

See also

- [Backup Classes](#) — how administrators define strategies and BackupClass resources for databases and VMs.
- [Backup and Recovery \(VMs\)](#) — the parallel guide for VMInstance / VMDisk backups (HelmRelease + CRs + PVC snapshots).

Last modified 2026-07-22: [\[\[docs\] Update managed apps reference for v1.6.0 \(96e013f\)\]\(https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f\)](#)

Adding External Applications to Cozystack Catalog

<https://cozystack.ru/docs/v1.6/applications/external/>

Learn how to add managed applications from external sources

Cozystack administrators can add applications from external sources in addition to the standard application catalog. These applications appear in the same catalog and behave like regular managed applications for platform users.

This guide explains the structure of an external application package and how to add it to a Cozystack cluster.

For a complete working example, see github.com/cozystack/external-apps-example.

Just like standard Cozystack applications, this external application package uses Helm and FluxCD. To learn more about developing application packages, read the Cozystack [Developer Guide](#).

Repository Structure

An external application repository has the following layout:

init.yaml # Bootstrap manifest (GitRepository + HelmRelease)

scripts/

package.mk # Shared Makefile targets for app charts

update-appdef.sh # Syncs generated chart schemas into the ApplicationDefinitions

packages/

core/platform/ # Platform chart: namespaces, operators, HelmCharts, ApplicationDefinitions

apps/<app-name>/ # Helm chart for each user-installable application

- `packages/core/platform` — a Helm chart deployed by FluxCD. It registers all applications via `ApplicationDefinition` CRDs, creates required namespaces, deploys operators, and defines `HelmChart` resources that point to the app charts in the same Git repository.
- `packages/apps/<app-name>` — standard Helm charts that template the actual Kubernetes resources (CRDs, ConfigMaps, Secrets, etc.).

Platform Chart

The platform chart (`packages/core/platform/`) is the central piece. It contains templates for:

Namespaces

Create namespaces for operators and system components:

```
apiVersion: v1
kind: Namespace
metadata:
labels:
cozystack.io/system: "true"
name: external-<operator-name>
```

HelmCharts

Define `HelmChart` resources that tell FluxCD where to find each app chart within the Git repository:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmChart
metadata:
name: external-apps-<app-name>
namespace: cozy-public
spec:
interval: 5m
chart: ./packages/apps/<app-name>
sourceRef:
kind: GitRepository
name: external-apps
reconcileStrategy: Revision
```

Use `reconcileStrategy: Revision` so that charts with a static version: `0.0.0` are re-reconciled whenever the Git content changes.

Operator Deployment

If your application requires an operator, deploy it via a `HelmRepository` and `HelmRelease`:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
name: <operator-name>
namespace: external-<operator-name>
spec:
```

type: oci

interval: 5m

url: oci://ghcr.io/<org>/charts

apiVersion: helm.toolkit.fluxcd.io/v2

kind: HelmRelease

metadata:

name: <operator-name>

namespace: external-<operator-name>

spec:

interval: 5m

releaseName: <operator-name>

targetNamespace: external-<operator-name>

chart:

spec:

chart: <operator-chart-name>

sourceRef:

kind: HelmRepository

name: <operator-name>

version: '1.2.3' # pin an exact operator version to keep installs reproducible

ApplicationDefinitions

Register each application in the Cozystack dashboard with an `ApplicationDefinition`:

apiVersion: cozystack.io/v1alpha1

kind: ApplicationDefinition

metadata:

name: <app-name>

spec:

application:

kind: <AppKind>

singular: <appkind>

plural: <appkinds>

openAPISchema: '{"title":"Chart Values","type":"object","properties":{"...}}'

release:

chartRef:

```
kind: HelmChart
name: external-apps-<app-name>
namespace: cozy-public
labels:
sharding.fluxcd.io/key: tenants
prefix: <app-name>-
dashboard:
category: <Category>
singular: <Human-readable Name>
plural: <Human-readable Names>
description: <Short description.>
tags:
  - <tag>
icon: <base64-encoded SVG>
keysOrder:
  - - apiVersion
  - - appVersion
  - - kind
  - - metadata
  - - metadata
  - - name
  - - spec
  - <field>
```

Follow these naming conventions (matching the main Cozystack repository):

| Field | Convention | Example for <code>my-app</code> |
|-----------------------------------|-------------------------------------|---------------------------------|
| <code>metadata.name</code> | lowercase, hyphens allowed | <code>my-app</code> |
| <code>application.kind</code> | PascalCase, no hyphens | <code>MyApp</code> |
| <code>application.singular</code> | lowercase, no hyphens | <code>myapp</code> |
| <code>application.plural</code> | lowercase, no hyphens | <code>myapps</code> |
| <code>release.prefix</code> | <code><metadata.name>-</code> | <code>my-app-</code> |
| <code>openAPISchema title</code> | always "Chart Values" | — |

The `openAPISchema` field contains a single-line JSON string with the schema for the application values — the minified content of the chart's generated `values.schema.json`. Keep the two in sync: in the example repository, `make generate` embeds the schema with `scripts/update-appdef.sh`. Note that Kubernetes

`apiextensions/v1 JSONSchemaProps` does not support `if/then/else`, so schemas must not rely on conditional validation.

Application Charts

Each application chart in `packages/apps/<app-name>/` is a standard Helm chart:

```
packages/apps/<app-name>/
Chart.yaml
Makefile
values.yaml
values.schema.json
templates/
<resource>.yaml
```

Chart.yaml

```
apiVersion: v2
name: <app-name>
description: <Short description>
type: application
version: 0.0.0
appVersion: "1.0.0"
```

Use `version: 0.0.0` — the actual version is derived from the Git revision by FluxCD.

Makefile

```
export NAME=<app-name>
export NAMESPACE=external-<operator-name>

include ../../scripts/package.mk

.PHONY: generate
generate:
cozyvalues-gen -v values.yaml -s values.schema.json -r README.md
../../scripts/update-appdef.sh $(NAME)
```

values.schema.json

Generate the JSON schema from annotations in `values.yaml` with `[cozyvalues-gen](https://github.com/cozystack/cozyvalues-gen)` — the same tool and annotation dialect used for packages in the main Cozystack repository. Helm validates user-supplied values against this schema at install time. Run `make generate` after changing `values.yaml` and commit the regenerated files.

Bootstrap Manifest

The `init.yaml` file creates two FluxCD resources that bootstrap the entire catalog:

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: external-apps
  namespace: cozy-public
spec:
  interval: 1m0s
  ref:
    branch: main
  timeout: 60s
  url: https://github.com/<org>/<repo>.git
```

```
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: external-apps
  namespace: cozy-system
spec:
  interval: 5m
  targetNamespace: cozy-system
  chart:
    spec:
      chart: ./packages/core/platform
      sourceRef:
        kind: GitRepository
        name: external-apps
        namespace: cozy-public
      reconcileStrategy: Revision
```

Apply it to your Cozystack cluster:

```
kubectl apply -f init.yaml
```

After FluxCD reconciles, the applications will appear in the Cozystack dashboard.

FluxCD Reference

These FluxCD documents will help you understand the resources used in this guide:

- [GitRepository](#)
- [HelmRelease](#)
- [HelmChart](#)

Last modified 2026-07-22: [\[\[docs\] Update managed apps reference for v1.6.0 \(96e013f\)\]\(https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f\)](#)

Managed ClickHouse Service

<https://cozystack.ru/docs/v1.6/applications/clickhouse/>

ClickHouse is an open source high-performance and column-oriented SQL database management system (DBMS). It is used for online analytical processing (OLAP).

Backup orchestration

Two backup paths coexist on the chart; pick one per release.

Recommended: BackupClass + Plan via the Altinity strategy

The cluster-scoped Altinity strategy (`strategy.backups.cozystack.io/v1alpha1`) wraps [Altinity's `clickhouse-backup`](<https://github.com/Altinity/clickhouse-backup>) in a one-shot `batch/v1.Job` per `BackupJob` / `RestoreJob`. It is engine-aware (FREEZE + upload), supports both in-place and `targetApplicationRef` (to-copy) restore, and does not require any in-chart `CronJob`.

Wiring per release:

1. Set `backup.enabled: true` plus the `s3*` (or `s3CredentialsSecret.name`) fields on the chart values. The chart materialises a Secret named `<release>-backup-s3` carrying bucket coordinates and credentials. That Secret is consumed by the chart-emitted `clickhouse-backup` sidecar (rendered into the ClickHouseInstallation Pod by `templates/clickhouse.yaml`); the strategy Pod itself is a curl/jq client that reaches the sidecar's HTTP API on port 7171 and never reads the Secret directly.
2. Cluster admin one-time installs an Altinity strategy and a BackupClass that maps `apps.cozystack.io/ClickHouse` to it (see `examples/backups/clickhouse/01-create-strategy.sh` and `02-create-backupclass.sh`).
3. Tenant creates a Plan (cron schedule) or submits an ad-hoc BackupJob referencing the BackupClass. Restoring is a RestoreJob referencing the resulting Backup; omit `targetApplicationRef` for in-place, set it to a second ClickHouse instance for to-copy.

Bringing your own S3 credentials Secret

Setting `backup.s3CredentialsSecret.name` makes the chart skip the materialisation of `<release>-backup-s3` and points the sidecar at the referenced Secret instead. The Secret must hold the five S3 fields as separate string keys, not a JSON blob; the per-key field names default to `bucketName` / `endpoint` / `region` / `accessKey` / `secretKey` and can be remapped via `backup.s3CredentialsSecret.{bucketKey,endpointKey,...}`.

Example:

`apiVersion: v1`

`kind: Secret`

```
metadata:
name: my-s3-creds
namespace: tenant-test
type: Opaque
stringData:
bucketName: my-clickhouse-archive
endpoint: https://s3.example.org
region: us-east-1
accessKey: AKIAIOSFODNN7EXAMPLE
secretKey: wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY
```

```
# In ClickHouse values.yaml:
```

```
backup:
enabled: true
s3CredentialsSecret:
name: my-s3-creds
```

The Cozystack `Bucket` app's `BucketInfo` Secret (`bucket-<name>-<user>`, single `BucketInfo` key holding a JSON blob) is not directly consumable as `s3CredentialsSecret`. Either let the chart read the raw values via `backup.s3*` (as `examples/backups/clickhouse/03-create-bucket.sh` does — extract coordinates from `BucketInfo` and pass them to the chart values), or materialise an intermediate Secret with the five string keys above.

Legacy: chart-emitted Restic CronJob

The chart still ships a CronJob that streams per-table `SHOW CREATE TABLE`

```
- SELECT * FORMAT TabSeparated
```

into a Restic repository. This path is kept for backward compatibility; new installations should prefer the `BackupClass` flow above.

To opt in:

1. Set `backup.enabled: true`, `backup.schedule: "0 2 * * *"` (or another non-empty cron), `backup.s3*` and `backup.resticPassword`.

2. To restore manually:

```
restic -r s3:s3.example.org/clickhouse-backups/table_name snapshots
```

```
restic -r s3:s3.example.org/clickhouse-backups/table_name restore latest --target /tmp/
```

For more details, read [Restic: Effective Backup from Stdin](#).

The legacy `backup.schedule`, `backup.cleanupStrategy`, and `backup.resticPassword` values are deprecated and will be removed once the Altinity strategy is the default in all reference deployments. The chart-emitted CronJob renders only when `backup.schedule` is non-empty.

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

Parameters

Common parameters

| Name | Description | Type | Value |
|-------------------------------|---|-----------------------|-----------------------|
| <code>replicas</code> | Number of ClickHouse replicas. | <code>int</code> | 2 |
| <code>shards</code> | Number of ClickHouse shards. | <code>int</code> | 1 |
| <code>resources</code> | Explicit CPU and memory configuration for each ClickHouse replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU available to each replica. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Memory (RAM) available to each replica. | <code>quantity</code> | <code>" "</code> |
| <code>resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | <code>string</code> | <code>t1.small</code> |
| <code>size</code> | Persistent Volume Claim size available for application data. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass used to store the data. | <code>string</code> | <code>" "</code> |

Application-specific parameters

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

| | | | |
|-----------------------------------|--|--------------------------------|--------------------|
| <code>logStorageSize</code> | Size of Persistent Volume for logs. | <code>quantity</code> | 2Gi |
| <code>logTTL</code> | TTL (expiration time) for <code>query_log</code> and <code>query_thread_log</code> . | <code>int</code> | 15 |
| <code>users</code> | Users configuration map. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Password for the user. | <code>string</code> | <code>" "</code> |
| <code>users[name].readonly</code> | User is readonly (default: false). | <code>bool</code> | <code>false</code> |

Backup parameters

| Name | Description | Type | Value |
|-----------------------------|---|---------------------|--------------------|
| <code>backup</code> | Backup configuration. | <code>object</code> | <code>{}</code> |
| <code>backup.enabled</code> | Enable backup integration. Materialises the chart-emitted <code><release>-backup-s3</code> Secret consumed by the Altinity backup strategy and, when <code>schedule</code> is non-empty, also renders the legacy chart-managed CronJob. | <code>bool</code> | <code>false</code> |

| | | | |
|-------------------------------------|---|---------------------|--|
| <code>backup.useSystemBucket</code> | Opt-in: when true, the chart-emitted <code><release>-backup-s3</code> Secret is skipped and the <code>clickhouse-backup</code> sidecar reads bucket coordinates + S3 credentials from the platform-projected <code>cozy-backups-creds</code> Secret. <code>S3_PATH</code> is set to <code><namespace>/<release></code> for cross-tenant isolation. Use together with the platform <code>cozy-default</code> BackupClass — tenants do not need to fill any <code>s3*</code> field below. | <code>bool</code> | <code>false</code> |
| <code>backup.s3Region</code> | DEPRECATED. Per-tenant S3 configuration is being phased out in favour of the platform-managed <code>cozy-default</code> BackupClass. Optional now so tenants on <code>useSystemBucket: true</code> can omit it. | <code>string</code> | <code>us-east-1</code> |
| <code>backup.s3Bucket</code> | DEPRECATED. Optional; see <code>s3Region</code> . | <code>string</code> | <code>s3.example.org/clickhouse-backups</code> |
| <code>backup.endpoint</code> | DEPRECATED. S3 endpoint URL for the legacy chart-emitted sidecar. | <code>string</code> | <code>" "</code> |
| <code>backup.s3PathOverride</code> | DEPRECATED. Object-key prefix inside the legacy <code>s3Bucket</code> ; the new platform flow scopes by namespace automatically. | <code>string</code> | <code>" "</code> |

| | | | |
|---|--|--------|--|
| <code>backup.schedule</code> | Legacy. Cron schedule for the chart-emitted CronJob that runs the dump+restic backup. Empty (default) skips the legacy CronJob; recommended when a BackupClass \+ Plan from <code>backups.cozystack.io</code> already drives backup orchestration via the Altinity strategy. | string | "" |
| <code>backup.cleanupStrategy</code> | Legacy. Restic retention policy passed to the legacy CronJob (<code>restic forget ...</code>). Unused by the Altinity strategy. | string | --keep-last=3
--keep-daily=3
--keep-within-weekly=1m |
| <code>backup.s3AccessKey</code> | DEPRECATED. Tenants no longer supply S3 keys; the platform-managed Bucket Secret is the source of truth. Optional now so <code>useSystemBucket: true</code> tenants can omit it. | string | <your-access-key> |
| <code>backup.s3SecretKey</code> | DEPRECATED. Optional; see <code>s3AccessKey</code> . | string | <your-secret-key> |
| <code>backup.resticPassword</code> | Legacy. Password for Restic backup encryption used by the legacy CronJob. Unused by the Altinity strategy. | string | <password> |
| <code>backup.s3CredentialsSecret</code> | DEPRECATED. Pre-existing Secret reference for the legacy chart-emitted sidecar. The platform flow projects <code>cozy-backups-creds</code> instead. | object | {} |

| | | | |
|--|---|--------|----|
| <code>backup.s3CredentialsSecret.name</code> | Name of the Secret in the application namespace. Empty means the chart materialises <code><release>-backup-s3</code> from the legacy <code>s3*</code> fields. | string | "" |
| <code>backup.s3CredentialsSecret.bucketKey</code> | Key in the Secret holding the bucket name. Defaults to <code>bucketName</code> . | string | "" |
| <code>backup.s3CredentialsSecret.endpointKey</code> | Key in the Secret holding the S3 endpoint URL. Defaults to <code>endpoint</code> . | string | "" |
| <code>backup.s3CredentialsSecret.regionKey</code> | Key in the Secret holding the S3 region. Defaults to <code>region</code> . | string | "" |
| <code>backup.s3CredentialsSecret.accessKeyIDKey</code> | Key in the Secret holding the access key ID. Defaults to <code>accessKey</code> . | string | "" |
| <code>backup.s3CredentialsSecret.secretAccessKeyKey</code> | Key in the Secret holding the secret access key. Defaults to <code>secretKey</code> . | string | "" |

ClickHouse Keeper parameters

| Name | Description | Type | Value |
|---|--|----------|-----------------------|
| <code>clickhouseKeeper</code> | ClickHouse Keeper configuration. | object | <code>{}</code> |
| <code>clickhouseKeeper.enabled</code> | Deploy ClickHouse Keeper for cluster coordination. | bool | <code>true</code> |
| <code>clickhouseKeeper.size</code> | Persistent Volume Claim size available for application data. | quantity | <code>1Gi</code> |
| <code>clickhouseKeeper.resourcesPreset</code> | Default sizing preset. | string | <code>t1.micro</code> |

| | | | |
|--|----------------------------|------------------|---|
| <code>clickhouseKeeper.replicas</code> | Number of Keeper replicas. | <code>int</code> | 3 |
|--|----------------------------|------------------|---|

Parameter examples and reference

resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

| Preset name | CPU | memory |
|-------------|------|--------|
| nano | 250m | 128Mi |
| micro | 500m | 256Mi |
| small | 1 | 512Mi |
| medium | 1 | 1Gi |
| large | 2 | 2Gi |
| xlarge | 4 | 4Gi |
| 2xlarge | 8 | 8Gi |

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0 (96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

FoundationDB

<https://cozystack.ru/docs/v1.6/applications/foundationdb/>

A managed FoundationDB service for Cozystack.

Overview

FoundationDB is a distributed database designed to handle large volumes of structured data across clusters of commodity servers. It organizes data as an ordered key-value store and employs ACID transactions for all operations.

This package provides a managed FoundationDB cluster deployment using the FoundationDB Kubernetes Operator.

Features

- High Availability : Multi-instance deployment with automatic failover
- ACID Transactions : Full ACID transaction support across the cluster
- Scalable : Easily scale storage and compute resources
- Backup Integration : Optional S3-compatible backup storage
- Monitoring : Built-in monitoring and alerting through WorkloadMonitor
- Flexible Configuration : Support for custom FoundationDB parameters

Configuration

Basic Configuration

Cluster process configuration

cluster:

version: "7.3.63"

processCounts:

storage: 3 # Number of storage processes (determines cluster size)

stateless: -1 # Automatically calculated

cluster_controller: 1

faultDomain:

key: "kubernetes.io/hostname"

valueFrom: "spec.nodeName"

Storage

storage:

size: "16Gi" # Storage size per instance

storageClass: "" # Storage class (optional)

Resources

Use preset sizing

resourcesPreset: "medium" # small, medium, large, xlarge, 2xlarge

Or custom resource configuration

resources:

cpu: "2000m"

memory: "4Gi"

Backups

The recommended backup flow uses the Cozystack backups framework: a cluster-scoped `BackupClass` binds `apps.cozystack.io/FoundationDB` to a `strategy.backups.cozystack.io/FoundationDB` strategy, and tenants drive runs via `BackupJob` / `RestoreJob`. See [\[examples/backups/foundationdb/\]\(../../examples/backups/foundationdb/\)](#) for an end-to-end walkthrough (admin setup, backup, in-place restore, to-copy restore).

The legacy in-chart `backup.*` values (`backup.enabled`, `backup.s3`, `backup.retentionPolicy`) are DEPRECATED but still render the original `FoundationDBBackup` CR unchanged when `backup.enabled=true`, for backward compatibility. New deployments should leave `backup.enabled=false` (the default) and use the `BackupClass` flow above. Mixing the two on the same cluster is unsupported — the `FoundationDB` operator only permits one running backup directory per cluster, and the backup driver fails fast with `Ready=False/ConflictingInChartBackup` when it sees a chart-rendered Running backup against the same target.

Advanced Configuration

Custom FoundationDB parameters

customParameters:

- "knob_disable_posix_kernel_aio=1"

Image type (split is the default; matches existing clusters' effective value)

imageType: "split"

Enable automatic pod replacements

automaticReplacements: true

Security context configuration

securityContext:

runAsUser: 4059

runAsGroup: 4059

Prerequisites

- FoundationDB Operator must be installed in the cluster
- Sufficient storage and compute resources
- For backups: S3-compatible storage credentials

Deployment

1. Install the FoundationDB operator (system package)
2. Deploy this application package with your desired configuration
3. The cluster will be automatically provisioned and configured

Monitoring

This package includes WorkloadMonitor integration for cluster health monitoring and resource tracking. Monitoring can be disabled by setting:

monitoring:

enabled: false

Security

- All containers run with restricted security contexts
- No privilege escalation allowed
- Read-only root filesystem where possible
- Custom security context configurations supported

Fault Tolerance

FoundationDB is designed for high availability:

-
- Automatic failure detection and recovery
 - Data replication across instances
 - Configurable fault domains for rack/zone awareness
 - Transaction log redundancy

The included `WorkloadMonitor` is automatically configured based on the `cluster.redundancyMode` value. It sets the `minReplicas` property on the `WorkloadMonitor` resource to ensure the cluster's health status accurately reflects its fault tolerance level. The number of tolerated failures is as follows:

- `single`: 0 failures
- `double`: 1 failure
- `triple` and datacenter-aware modes: 2 failures

For example, with the default configuration (`redundancyMode: double` and 3 storage pods), `minReplicas` will be set to 2.

Performance Considerations

- Use SSD storage for better performance
- Consider dedicating nodes for storage processes
- Monitor cluster metrics for optimization opportunities
- Scale storage and stateless processes based on workload

Support

For issues related to FoundationDB itself, refer to the [FoundationDB documentation](#).

For Cozystack-specific issues, consult the Cozystack documentation or support channels.

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

Parameters

Common parameters

| Name | Description | Type | Value |
|------------------------------------|-------------------------------------|--------|-----------------|
| <code>cluster</code> | Cluster configuration. | object | <code>{}</code> |
| <code>cluster.processCounts</code> | Process counts for different roles. | object | <code>{}</code> |

| | | | |
|---|--|----------|------------------------|
| <code>cluster.processCounts.stateless</code> | Number of stateless processes (-1 for automatic). | int | -1 |
| <code>cluster.processCounts.storage</code> | Number of storage processes (determines cluster size). | int | 3 |
| <code>cluster.processCounts.cluster_controller</code> | Number of cluster controller processes. | int | 1 |
| <code>cluster.version</code> | Version of FoundationDB to use. | string | 7.3.63 |
| <code>cluster.redundancyMode</code> | Database redundancy mode (single, double, triple, three_datacenter, three_datacenter_fallback). | string | double |
| <code>cluster.storageEngine</code> | Storage engine (ssd-2, ssd-redwood-v1, ssd-rocksdb-v1, memory). | string | ssd-2 |
| <code>cluster.faultDomain</code> | Fault domain configuration. | object | {} |
| <code>cluster.faultDomain.key</code> | Fault domain key. | string | kubernetes.io/hostname |
| <code>cluster.faultDomain.valueFrom</code> | Fault domain value source. | string | spec.nodeName |
| <code>storage</code> | Storage configuration. | object | {} |
| <code>storage.size</code> | Size of persistent volumes for each instance. | quantity | 16Gi |
| <code>storage.storageClass</code> | Storage class (if not set, uses cluster default). | string | "" |
| <code>resources</code> | Explicit CPU and memory configuration for each FoundationDB instance. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | {} |

| | | | |
|--|---|-----------------------|------------------------|
| <code>resources.cpu</code> | CPU available to each instance. | <code>quantity</code> | <code>""</code> |
| <code>resources.memory</code> | Memory (RAM) available to each instance. | <code>quantity</code> | <code>""</code> |
| <code>resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | <code>string</code> | <code>cl.small</code> |
| <code>backup</code> | DEPRECATED Backup configuration (use the Cozystack backups framework: BackupClass + FoundationDB strategy). | <code>object</code> | <code>{}</code> |
| <code>backup.enabled</code> | DEPRECATED Enable in-chart backups (superseded by BackupClass + FoundationDB strategy). | <code>bool</code> | <code>false</code> |
| <code>backup.s3</code> | DEPRECATED S3 configuration for backups. | <code>object</code> | <code>{}</code> |
| <code>backup.s3.bucket</code> | DEPRECATED S3 bucket name. | <code>string</code> | <code>""</code> |
| <code>backup.s3.endpoint</code> | DEPRECATED S3 endpoint URL. | <code>string</code> | <code>""</code> |
| <code>backup.s3.region</code> | DEPRECATED S3 region. | <code>string</code> | <code>us-east-1</code> |
| <code>backup.s3.credentials</code> | DEPRECATED S3 credentials. | <code>object</code> | <code>{}</code> |
| <code>backup.s3.credentials.accessKeyId</code> | DEPRECATED S3 access key ID. | <code>string</code> | <code>""</code> |
| <code>backup.s3.credentials.secretAccessKey</code> | DEPRECATED S3 secret access key. | <code>string</code> | <code>""</code> |
| <code>backup.retentionPolicy</code> | DEPRECATED Retention policy for backups. | <code>string</code> | <code>7d</code> |
| <code>monitoring</code> | Monitoring configuration. | <code>object</code> | <code>{}</code> |
| <code>monitoring.enabled</code> | Enable WorkloadMonitor integration. | <code>bool</code> | <code>true</code> |

FoundationDB configuration

| Name | Description | Type | Value |
|----------------------------|--|----------|-------|
| customParameters | Custom parameters to pass to FoundationDB. | []string | [] |
| imageType | Container image deployment type. | string | split |
| securityContext | Security context for containers. | object | {} |
| securityContext.runAsUser | User ID to run the container. | int | 4059 |
| securityContext.runAsGroup | Group ID to run the container. | int | 4059 |
| automaticReplacements | Enable automatic pod replacements. | bool | true |

Last modified 2026-07-22: [\[\[docs\] Update managed apps reference for v1.6.0 \(96e013f\)\]\(https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f\)](https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f)

Managed Harbor Container Registry

<https://cozystack.ru/docs/v1.6/applications/harbor/>

Harbor is an open-source trusted cloud-native registry project that stores, signs, and scans content.

Prerequisites

The Cozystack Harbor app stores its registry data exclusively in S3-compatible object storage: the chart pins the registry backend to S3 and exposes no filesystem option. That bucket is provisioned through COSI (`objectstorage.k8s.io`) from a SeaweedFS deployment, so before deploying Harbor the tenant must have SeaweedFS available — enabled on the same tenant or inherited from a parent tenant (the resolved class is propagated down the tenant tree, surfaced as the `namespace.cozystack.io/seaweedfs` namespace label).

Enable it by setting `seaweedfs: true` on the tenant (or a parent tenant):

```
seaweedfs: true
```

Without object storage in the tenant chain, Harbor cannot provision its registry bucket: the `<release>-registry BucketClaim/BucketAccess` never produces the `<release>-registry-bucket` credentials secret, so the Harbor `HelmsRelease` stays unreconciled, waiting on `BucketInfo`.

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

Parameters

Common parameters

| Name | Description | Type | Value |
|--------------|--|--------|-------|
| host | Hostname for external access to Harbor (defaults to 'harbor' subdomain for the tenant host). | string | "" |
| storageClass | StorageClass used to store the data. | string | "" |

Component configuration

| Name | Description | Type | Value |
|---------------------------|---|----------|----------|
| core | Core API server configuration. | object | {} |
| core.resources | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | {} |
| core.resources.cpu | Number of CPU cores allocated. | quantity | "" |
| core.resources.memory | Amount of memory allocated. | quantity | "" |
| core.resourcesPreset | Default sizing preset used when <code>resources</code> is omitted. | string | t1.small |
| registry | Container image registry configuration. | object | {} |
| registry.resources | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | {} |
| registry.resources.cpu | Number of CPU cores allocated. | quantity | "" |
| registry.resources.memory | Amount of memory allocated. | quantity | "" |
| registry.resourcesPreset | Default sizing preset used when <code>resources</code> is omitted. | string | t1.small |
| jobservice | Background job service configuration. | object | {} |
| jobservice.resources | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | {} |

| | | | |
|--|---|----------|----------------------|
| <code>jobservice.resources.cpu</code> | Number of CPU cores allocated. | quantity | "" |
| <code>jobservice.resources.memory</code> | Amount of memory allocated. | quantity | "" |
| <code>jobservice.resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | string | <code>t1.nano</code> |
| <code>trivy</code> | Trivy vulnerability scanner configuration. | object | <code>{}</code> |
| <code>trivy.enabled</code> | Enable or disable the vulnerability scanner. | bool | <code>true</code> |
| <code>trivy.size</code> | Persistent Volume size for vulnerability database cache. | quantity | <code>5Gi</code> |
| <code>trivy.resources</code> | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | <code>{}</code> |
| <code>trivy.resources.cpu</code> | Number of CPU cores allocated. | quantity | "" |
| <code>trivy.resources.memory</code> | Amount of memory allocated. | quantity | "" |
| <code>trivy.resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | string | <code>t1.nano</code> |
| <code>database</code> | PostgreSQL database configuration. | object | <code>{}</code> |
| <code>database.size</code> | Persistent Volume size for database storage. | quantity | <code>5Gi</code> |
| <code>database.replicas</code> | Number of database instances. | int | <code>2</code> |
| <code>redis</code> | Redis cache configuration. | object | <code>{}</code> |
| <code>redis.size</code> | Persistent Volume size for cache storage. | quantity | <code>1Gi</code> |

| | | | |
|-----------------------------|---------------------------|------------------|---|
| <code>redis.replicas</code> | Number of Redis replicas. | <code>int</code> | 2 |
|-----------------------------|---------------------------|------------------|---|

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0 (96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

Managed Kafka Service

<https://cozystack.ru/docs/v1.6/applications/kafka/>

Both `kafka.storageClass` and `zookeeper.storageClass` are annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

Parameters

Common parameters

| Name | Description | Type | Value |
|-----------------------|---|---------------------|--------------------|
| <code>external</code> | Enable external access from outside the cluster. | <code>bool</code> | <code>false</code> |
| <code>tls</code> | TLS configuration. Strimzi manages the cluster PKI automatically (no cert-manager is involved for this chart): the operator auto-creates <code><release>-cluster-ca-cert</code> and <code><release>-clients-ca-cert</code> secrets, both exposed for client trust setup. The internal TLS listener on 9093 is always on; this toggle only controls the external listener on 9094. | <code>object</code> | <code>{}</code> |

| | | | |
|--------------------------|--|--------------------|-------------------|
| <code>tls.enabled</code> | Enable TLS on the external listener. When unset, inherits the value of <code>external</code> (TLS is on when external access is enabled). Warning: setting this to false while external is true exposes Kafka over plaintext on a public IP via LoadBalancer. Strimzi does not provide authentication on this listener unless SCRAM, mTLS, or OAuth is separately configured. Use only in controlled networks. | <code>*bool</code> | <code>null</code> |
|--------------------------|--|--------------------|-------------------|

Application-specific parameters

| Name | Description | Type | Value |
|-----------------------------------|-----------------------|-----------------------|-----------------|
| <code>topics</code> | Topics configuration. | <code>[]object</code> | <code>[]</code> |
| <code>topics[i].name</code> | Topic name. | <code>string</code> | <code>""</code> |
| <code>topics[i].partitions</code> | Number of partitions. | <code>int</code> | <code>0</code> |
| <code>topics[i].replicas</code> | Number of replicas. | <code>int</code> | <code>0</code> |
| <code>topics[i].config</code> | Topic configuration. | <code>object</code> | <code>{}</code> |

Kafka configuration

| Name | Description | Type | Value |
|------------------------------|---|---------------------|-----------------|
| <code>kafka</code> | Kafka configuration. | <code>object</code> | <code>{}</code> |
| <code>kafka.replicas</code> | Number of Kafka replicas. | <code>int</code> | <code>3</code> |
| <code>kafka.resources</code> | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code> | <code>{}</code> |

| | | | |
|-------------------------------------|--|----------|-----------------------|
| <code>kafka.resources.cpu</code> | CPU available to each replica. | quantity | "" |
| <code>kafka.resources.memory</code> | Memory (RAM) available to each replica. | quantity | "" |
| <code>kafka.resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | string | <code>cl.small</code> |
| <code>kafka.size</code> | Persistent Volume size for Kafka. | quantity | <code>10Gi</code> |
| <code>kafka.storageClass</code> | StorageClass used to store the Kafka data. | string | "" |

ZooKeeper configuration

| Name | Description | Type | Value |
|---|---|----------|-----------------------|
| <code>zookeeper</code> | ZooKeeper configuration. | object | <code>{}</code> |
| <code>zookeeper.replicas</code> | Number of ZooKeeper replicas. | int | <code>3</code> |
| <code>zookeeper.resources</code> | Explicit CPU and memory configuration. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | object | <code>{}</code> |
| <code>zookeeper.resources.cpu</code> | CPU available to each replica. | quantity | "" |
| <code>zookeeper.resources.memory</code> | Memory (RAM) available to each replica. | quantity | "" |
| <code>zookeeper.resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | string | <code>cl.small</code> |
| <code>zookeeper.size</code> | Persistent Volume size for ZooKeeper. | quantity | <code>5Gi</code> |
| <code>zookeeper.storageClass</code> | StorageClass used to store the ZooKeeper data. | string | "" |

Parameter examples and reference

resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#) for the full size matrix and the legacy-to-instance-type mapping.

Authentication

This chart does not configure listener authentication. When TLS is enabled on the external listener, clients can connect without credentials. To require authentication, use Strimzi's `KafkaUser` resource with an appropriate `authentication` type (`tls`, `scram-sha-512`, or `oauth`) outside this chart. See the [Strimzi documentation on KafkaUser](#) for details.

topics

`topics:`

`- name: Results`

`partitions: 1`

`replicas: 3`

`config:`

`min.insync.replicas: 2`

`- name: Orders`

`config:`

`cleanup.policy: compact`

`segment.ms: 3600000`

`max.compaction.lag.ms: 5400000`

min.insync.replicas: 2

partitions: 1

replicas: 3

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0

(96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

Managed MariaDB Service

<https://cozystack.ru/docs/v1.6/applications/mariadb/>

The Managed MariaDB Service offers a powerful and widely used relational database solution. This service allows you to create and manage a replicated MariaDB cluster seamlessly.

Deployment Details

This managed service is controlled by mariadb-operator, ensuring efficient management and seamless operation.

- Docs: <<https://mariadb.com/kb/en/documentation/>>
- GitHub: <<https://github.com/mariadb-operator/mariadb-operator>>

`storageClass` is annotated as immutable in the chart schema — see [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#) for the contract and which consumers enforce it.

HowTos

How to switch master/slave replica

```
kubectl edit mariadb <instance>
```

update:

spec:

replication:

primary:

podIndex: 1

check status:

```
NAME READY STATUS PRIMARY POD AGE
```

```
<instance> True Running app-db1-1 41d
```

How to back up and restore a MariaDB application

The recommended path is the Cozystack [BackupClass / Plan / RestoreJob](#) flow with the operator-native `strategy.backups.cozystack.io/MariaDB` strategy. See [examples/backups/mariadb](#) for a numbered,

end-to-end walkthrough (bucket, source, strategy, BackupClass, Plan, ad-hoc BackupJob, and both in-place + to-copy RestoreJob fixtures).

The chart's `backup.*` block (mariadb-dump + restic CronJob) is deprecated and remains supported for backward compatibility only. Existing tenants can keep using it unchanged; new deployments should use the operator-native flow above.

How to restore from a deprecated restic-based backup

find snapshot:

```
restic -r s3:s3.example.org/mariadb-backups/database_name snapshots
```

restore:

```
restic -r s3:s3.example.org/mariadb-backups/database_name restore latest --target /tmp/
```

more details:

- <https://blog.aenix.io/restic-effective-backup-from-stdin-4bc1e8f083c1>

Known issues

- Replication can't be finished with various errors
- **Replication can't be finished in case if `binlog` purged**

Until `mariadbbackup` is not used to bootstrap a node by mariadb-operator (this feature is not implemented yet), follow these manual steps to fix it:

<https://github.com/mariadb-operator/mariadb-operator/issues/141#issuecomment-1804760231>

- Corrupted indices Sometimes some indices can be corrupted on master replica, you can recover them from slave:

```
mysqldump -h <slave> -P 3306 -u<user> -p<password> --column-statistics=0 <database> <table>  
~/tmp/fix-table.sql
```

```
mysql -h <master> -P 3306 -u<user> -p<password> <database> < ~/tmp/fix-table.sql
```

Parameters

Common parameters

| Name | Description | Type | Value |
|----------|-----------------------------|------|-------|
| replicas | Number of MariaDB replicas. | int | 2 |

| | | | |
|-------------------------------|--|-----------------------|----------------------|
| <code>resources</code> | Explicit CPU and memory configuration for each MariaDB replica. When omitted, the preset defined in <code>resourcesPreset</code> is applied. | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU available to each replica. | <code>quantity</code> | <code>""</code> |
| <code>resources.memory</code> | Memory (RAM) available to each replica. | <code>quantity</code> | <code>""</code> |
| <code>resourcesPreset</code> | Default sizing preset used when <code>resources</code> is omitted. | <code>string</code> | <code>t1.nano</code> |
| <code>size</code> | Persistent Volume Claim size available for application data. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass used to store the data. | <code>string</code> | <code>""</code> |
| <code>external</code> | Enable external access from outside the cluster. | <code>bool</code> | <code>false</code> |
| <code>version</code> | MariaDB major.minor version to deploy | <code>string</code> | <code>v11.8</code> |

Application-specific parameters

| Name | Description | Type | Value |
|---|--------------------------------------|--------------------------------|-----------------|
| <code>users</code> | Users configuration map. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Password for the user. | <code>string</code> | <code>""</code> |
| <code>users[name].maxUserConnections</code> | Maximum number of connections. | <code>int</code> | <code>0</code> |
| <code>databases</code> | Databases configuration map. | <code>map[string]object</code> | <code>{}</code> |
| <code>databases[name].roles</code> | Roles assigned to users. | <code>object</code> | <code>{}</code> |
| <code>databases[name].roles.admin</code> | List of users with admin privileges. | <code>[]string</code> | <code>[]</code> |

| | | | |
|---|--|-----------------------|-----------------|
| <code>databases[name].roles.readonly</code> | List of users with read-only privileges. | <code>[]string</code> | <code>[]</code> |
|---|--|-----------------------|-----------------|

Backup parameters (DEPRECATED)

| Name | Description | Type | Value |
|-------------------------------------|---|---------------------|---|
| <code>backup</code> | DEPRECATED: Backup configuration. Prefer the BackupClass / Plan flow under examples/backups/mariadb/. | <code>object</code> | <code>{}</code> |
| <code>backup.enabled</code> | DEPRECATED: Enable regular backups (default: false). | <code>bool</code> | <code>false</code> |
| <code>backup.s3Region</code> | DEPRECATED: AWS S3 region where backups are stored. | <code>string</code> | <code>us-east-1</code> |
| <code>backup.s3Bucket</code> | DEPRECATED: S3 bucket used for storing backups. | <code>string</code> | <code>s3.example.org/mariadb-backups</code> |
| <code>backup.schedule</code> | DEPRECATED: Cron schedule for automated backups. | <code>string</code> | <code>0 2 * * *</code> |
| <code>backup.cleanupStrategy</code> | DEPRECATED: Retention strategy for cleaning up old backups. | <code>string</code> | <code>--keep-last=3
--keep-daily=3
--keep-within-weekly=1m</code> |
| <code>backup.s3AccessKey</code> | DEPRECATED: Access key for S3 authentication. | <code>string</code> | <code><your-access-key></code> |
| <code>backup.s3SecretKey</code> | DEPRECATED: Secret key for S3 authentication. | <code>string</code> | <code><your-secret-key></code> |
| <code>backup.resticPassword</code> | DEPRECATED: Password for Restic backup encryption. | <code>string</code> | <code><password></code> |

Parameter examples and reference

resources and resourcesPreset

`resources` sets explicit CPU and memory configurations for each replica. When left empty, the preset defined in `resourcesPreset` is applied.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` sets named CPU and memory configurations for each replica. This setting is ignored if the corresponding `resources` value is set.

Presets follow a cloud-style `<series>.<size>` naming convention. Five series cover the full CPU-to-memory ratio range (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`) and each series ships eight sizes (`nano` through `4xlarge`). The legacy flat names (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) remain accepted as deprecated aliases of their 1:1 instance-type equivalents.

See [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#) for the full size matrix and the legacy-to-instance-type mapping.

users

`users:`

`user1:`

`maxUserConnections: 1000`

`password: hackme`

`user2:`

`maxUserConnections: 1000`

`password: hackme`

databases

`databases:`

`myapp1:`

`roles:`

`admin:`

`- user1`

`readonly:`

`- user2`

Last modified 2026-07-22: [\[\[docs\] Update managed apps reference for v1.6.0 \(96e013f\)\]\(https://github.com/katamarina-ru/cozystack-website/commit/96e013fad266e0cdc43772a869dffbdea6c20c9f\)](https://github.com/katamarina-ru/cozystack-website/commit/96e013fad266e0cdc43772a869dffbdea6c20c9f)

Управляемый сервис MongoDB

<https://cozystack.ru/docs/v1.6/applications/mongodb/>

MongoDB — популярная документоориентированная NoSQL-база данных, известная своей гибкостью и масштабируемостью. Управляемый сервис MongoDB предоставляет самовосстанавливающийся реплицируемый кластер под управлением Percona Operator for MongoDB.

Детали развёртывания

Этот управляемый сервис контролируется Percona Operator for MongoDB, обеспечивающим эффективное управление и бесперебойную работу.

- Документация: <<https://docs.percona.com/percona-operator-for-mongodb/>>
- Github: <<https://github.com/percona/percona-server-mongodb-operator>>

Режимы развёртывания

Режим набора реплик (Replica Set, по умолчанию)

По умолчанию MongoDB разворачивается как набор реплик (replica set) с указанным числом реплик. Этот режим подходит для большинства сценариев, требующих высокой доступности.

Режим шардированного кластера

Включите `sharding: true` для горизонтального масштабирования по нескольким шардам. Каждый шард представляет собой набор реплик, а маршрутизаторы mongos выполняют маршрутизацию запросов.

Примечания

Внешний доступ

Когда включён `external: true`:

- Режим набора реплик : Трафик балансируется между всеми участниками набора реплик. Это хорошо работает для операций чтения, но для операций записи требуется подключение к первичному узлу (primary). Драйверы MongoDB автоматически определяют первичный узел, используя строку подключения к набору реплик.
- Шардированный режим : Трафик направляется через маршрутизаторы mongos, которые корректно обрабатывают и чтение, и запись.

Учётные данные

При первой установке Secret с учётными данными будет пустым, пока оператор Percona не инициализирует кластер. После готовности MongoDB выполните `helm upgrade`, чтобы заполнить Secret с учётными данными фактическим паролем.

Жизненный цикл данных

При удалении релиза MongoDB финализаторы оператора высвобождают ресурсы, относящиеся к релизу:

****Высвобождаются финализатором `percona.com/delete-psmdb-pvc`:****

- Все PVC, обеспечивающие хранилище набора реплик. Будут ли фактически удалены базовый PersistentVolume и данные на диске, зависит от `reclaimPolicy` у StorageClass (`Delete` удаляет их, `Retain` оставляет для ручной очистки).
- Управляемые оператором Secret'ы:
 - `<release>-percona-server-mongodb-users` — учётные данные пользователей оператора
 - `internal-<release>` — внутреннее состояние оператора
 - `internal-<release>-users` — внутренние данные пользователей оператора
 - `<release>-mongodb-encryption-key` — ключ шифрования данных в состоянии покоя (at-rest)

****Высвобождаются командой `helm uninstall`:****

- `<release>-credentials` — строка подключения для кода приложения
- `<release>-user-<username>` — пароли отдельных пользователей
- `<release>-s3-creds` — учётные данные места назначения резервных копий (если резервное копирование настроено)

Не высвобождаются автоматически:

- TLS-секреты `<release>-ssl` и `<release>-ssl-internal` (выпущенные `cert-manager`) остаются в пространстве имён после удаления. Удалите их вручную, если они больше не нужны.

Восстановление при зависшем удалении:

Если `psmdb-operator` удалён до удаления ресурсов MongoDB CR, финализаторы не могут отработать, и ресурс `PerconaServerMongoDB` зависает в состоянии `Terminating`. Для восстановления очистите финализаторы вручную:

```
kubectl --namespace <namespace> patch psmdb <release> --type merge --patch '{
"metadata":{"finalizers":[]}'
```

Учтите, что это пропускает очистку, выполняемую оператором — PVC и управляемые оператором Secret'ы останутся неиспользуемыми (orphaned) и должны быть удалены вручную.

Если нужно сохранить данные, создайте резервную копию перед удалением. См. [документацию Percona Operator for MongoDB](#) по процессам резервного копирования и восстановления.

Обновление с более ранних версий

Более ранние версии этого чарта ссылались на общий для пространства имён Secret системных пользователей (`percona-server-mongodb-users`). Обновление до релиза, в котором этот Secret привязан к конкретному CR (`<release>-percona-server-mongodb-users`), запускает ротацию паролей управляемых оператором системных пользователей. Ротация выполняется оператором Percona на месте через `db.changeUserPassword()` для работающего `mongod` (в логе оператора: `Secret data changed. Updating users...`); поды не перезапускаются, и кластер остаётся доступным.

Ротируются автоматически при обновлении:

- Пять управляемых оператором системных учётных записей: `databaseAdmin`, `userAdmin`, `backup`, `clusterAdmin`, `clusterMonitor`.
- Secret `<release>-percona-server-mongodb-users` (создаётся заново, для каждого CR) и `internal-<release>-users` получают новые значения.
- Secret `<release>-credentials` пересоздаётся; его ключи `password` и `uri` отражают новый пароль `databaseAdmin`.

Не затрагиваются:

- Пользовательские учётные записи, определённые в `users`: в `values` чарта. Их Secret'ы `<release>-user-<name>` не изменяются.
- Ключ шифрования данных в покое (`<release>-mongodb-encryption-key`) и `keyfile` набора реплик (`<release>-mongodb-keyfile`) не изменяются, поэтому данные на диске остаются читаемыми.

Требуется действие после обновления:

Рабочие нагрузки, монтирующие `<release>-credentials`, продолжают использовать зашифрованный старый пароль, пока не перечитают Secret. Перезапустите эти поды или используйте контроллер вроде [Reloader](#) для их автоматического перезапуска (Roll). Без этого подключения приложений завершатся с ошибками аутентификации, как только истекут их текущие сессии.

Осиротевший устаревший Secret:

Прежний общий для пространства имён Secret `percona-server-mongodb-users` после обновления больше не используется ни одним MongoDB CR, но оператор не удаляет его автоматически. Если несколько релизов MongoDB в одном пространстве имён ранее использовали его совместно, все они переходят на собственные Secret'ы для каждого CR — пароли больше не являются общими между CR в пространстве имён, что и является ожидаемым результатом. Убедитесь, что на него не ссылаются другие потребители, затем удалите его вручную:

```
kubectl --namespace <namespace> delete secret percona-server-mongodb-users
```


`storageClass` помечен как неизменяемый (immutable) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|--|-----------------------|-----------------------|
| <code>replicas</code> | Количество реплик MongoDB в наборе реплик. | <code>int</code> | 3 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики MongoDB. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.small</code> |
| <code>size</code> | Размер Persistent Volume Claim, доступный для данных приложения. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>" "</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |
| <code>version</code> | Мажорная версия MongoDB для развёртывания. | <code>string</code> | <code>v8</code> |

Конфигурация шардирования

| Имя | Описание | Тип | Значение |
|-----------------------------------|--|----------|----------|
| sharding | Включить режим шардированного кластера. Когда отключено, разворачивается набор реплик. | bool | false |
| shardingConfig | Конфигурация режима шардированного кластера. | object | {} |
| shardingConfig.configServers | Количество реплик серверов конфигурации (config server). | int | 3 |
| shardingConfig.configServerSize | Размер PVC для серверов конфигурации. | quantity | 3Gi |
| shardingConfig.mongos | Количество реплик маршрутизаторов mongos. | int | 2 |
| shardingConfig.shards | Список конфигураций шардов. | []object | [...] |
| shardingConfig.shards[i].name | Имя шарда. | string | "" |
| shardingConfig.shards[i].replicas | Количество реплик в этом шарде. | int | 0 |
| shardingConfig.shards[i].size | Размер PVC для этого шарда. | quantity | "" |

Конфигурация пользователей

| Имя | Описание | Тип | Значение |
|----------------------|---|-------------------|----------|
| users | Карта конфигурации пользователей. | map[string]object | {} |
| users[name].password | Пароль пользователя (генерируется автоматически, если не указан). | string | "" |

Конфигурация баз данных

| Имя | Описание | Тип | Значение |
|--------------------------------|--|-------------------|----------|
| databases | Карта конфигурации баз данных. | map[string]object | {} |
| databases[name].roles | Роли, назначенные пользователям. | object | {} |
| databases[name].roles.admin | Список пользователей с правами администратора (readWrite + dbAdmin). | []string | [] |
| databases[name].roles.readonly | Список пользователей с правами только на чтение. | []string | [] |

Параметры резервного копирования

| Имя | Описание | Тип | Значение |
|------------------------|--|--------|-----------------------------------|
| backup | Конфигурация резервного копирования. | object | {} |
| backup.enabled | Включить регулярное резервное копирование. | bool | false |
| backup.schedule | Расписание cron для автоматического резервного копирования. | string | 0 2 * * * |
| backup.retentionPolicy | Политика хранения (например, "30d"). | string | 30d |
| backup.destinationPath | Путь назначения для резервных копий (например, s3://bucket/path/). | string | s3://bucket/path/to/folder/ |
| backup.endpointURL | URL эндпоинта S3 для загрузки. | string | http://minio-gateway-service:9000 |
| backup.s3AccessKey | Ключ доступа для аутентификации в S3. | string | " " |
| backup.s3SecretKey | Секретный ключ для аутентификации в S3. | string | " " |

Параметры начальной загрузки (восстановления)

| Имя | Описание | Тип | Значение |
|-------------------------------------|---|---------------------|--------------------|
| <code>bootstrap</code> | Конфигурация начальной загрузки. | <code>object</code> | <code>{}</code> |
| <code>bootstrap.enabled</code> | Восстанавливать ли из резервной копии. | <code>bool</code> | <code>false</code> |
| <code>bootstrap.recoveryTime</code> | Метка времени для восстановления на момент времени; пусто означает последнюю. | <code>string</code> | <code>""</code> |
| <code>bootstrap.backupName</code> | Имя резервной копии, из которой восстанавливать. | <code>string</code> | <code>""</code> |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис NATS

<https://cozystack.ru/docs/v1.6/applications/nats/>

NATS — это простая, безопасная и высокопроизводительная система обмена сообщениями с открытым исходным кодом. Она предоставляет уровень данных для облачных (cloud native) приложений, обмена сообщениями IoT и микросервисных архитектур.

`storageClass` помечен как неизменяемый (immutable) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|---|-----------------------|----------------------|
| <code>replicas</code> | Количество реплик. | <code>int</code> | 2 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики NATS. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>storageClass</code> | <code>StorageClass</code> , используемый для хранения данных. | <code>string</code> | <code>" "</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

| | | | |
|--------------------------|---|---------------------|-------------------|
| <code>tls</code> | Конфигурация TLS. Если не задано, TLS следует значению флага <code>external</code> . | <code>object</code> | <code>{}</code> |
| <code>tls.enabled</code> | Включить TLS. Если не задано, TLS включается автоматически, когда <code>external</code> равен <code>true</code> . | <code>*bool</code> | <code>null</code> |

Параметры, специфичные для приложения

| Имя | Описание | Тип | Значение |
|-----------------------------------|---|--------------------------------|-------------------|
| <code>users</code> | Карта конфигурации пользователей. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Пароль пользователя. | <code>string</code> | <code>""</code> |
| <code>jetstream</code> | Конфигурация Jetstream. | <code>object</code> | <code>{}</code> |
| <code>jetstream.enabled</code> | Включить или отключить Jetstream для постоянного обмена сообщениями в NATS. | <code>bool</code> | <code>true</code> |
| <code>jetstream.size</code> | Размер постоянного хранилища Jetstream. | <code>quantity</code> | <code>10Gi</code> |
| <code>config</code> | Конфигурация NATS. | <code>object</code> | <code>{}</code> |
| <code>config.merge</code> | Дополнительная конфигурация для слияния с конфигурацией NATS. | <code>*object</code> | <code>{}</code> |
| <code>config.resolver</code> | Дополнительная конфигурация resolver для слияния с конфигурацией NATS. | <code>*object</code> | <code>{}</code> |

Примеры и справочник по параметрам

`resources` и `resourcesPreset`

`resources` задаёт явные конфигурации CPU и памяти для каждой реплики. Если оставить пустым, применяется пресет, указанный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованные конфигурации CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют облачной схеме именования `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия поставляется с восемью размерами (от `nano` до `4xlarge`). Устаревшие плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов инстансов с соотношением 1:1.

См. [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#), где приведены полная матрица размеров и сопоставление устаревших имён с типами инстансов.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис OpenBAO

<https://cozystack.ru/docs/v1.6/applications/openbao/>

OpenBAO — это решение для управления секретами с открытым исходным кодом, ответвлённое (форк) от HashiCorp Vault. Оно обеспечивает управление секретами на основе идентичности и шифрованием для облачной инфраструктуры.

`storageClass` помечен как неизменяемый (`immutable`) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|--|-----------------------|------------------|
| <code>replicas</code> | Количество реплик OpenBAO. Режим высокой доступности (HA) с Raft включается автоматически при <code>replicas > 1</code> . Переключение между режимами <code>standalone</code> (файловое хранилище) и HA (хранилище Raft) требует миграции данных. | <code>int</code> | <code>1</code> |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики OpenBAO. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |

| | | | |
|-----------------|--|----------|----------|
| resourcesPreset | Пресет размера по умолчанию, используемый, когда resources не задан. | string | t1.small |
| size | Размер Persistent Volume Claim для хранения данных. | quantity | 10Gi |
| storageClass | StorageClass, используемый для хранения данных. | string | "" |
| external | Включить внешний доступ извне кластера. | bool | false |

Параметры, специфичные для приложения

| Имя | Описание | Тип | Значение |
|-----|---------------------------------|------|----------|
| ui | Включить веб-интерфейс OpenBAO. | bool | true |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис PostgreSQL

<https://cozystack.ru/docs/v1.6/applications/postgres/>

PostgreSQL в настоящее время является лидирующим выбором среди реляционных баз данных и известен своей надёжной функциональностью и производительностью. Управляемый сервис PostgreSQL использует реализацию на стороне платформы для предоставления самовосстанавливающегося реплицируемого кластера. Этот кластер эффективно управляется с помощью широко признанного оператора CloudNativePG, получившего популярность в сообществе.

Детали развёртывания

Этот управляемый сервис контролируется оператором CloudNativePG, обеспечивающим эффективное управление и бесперебойную работу.

- Документация: <<https://cloudnative-pg.io/docs/>>
- Github: <<https://github.com/cloudnative-pg/cloudnative-pg>>

Операции

Резервное копирование PostgreSQL имеет два уровня, и в рекомендуемой конфигурации используются оба уровня вместе, а не один из них:

| Layer | What it does | Configured via |
|-------|--------------|----------------|
|-------|--------------|----------------|

| | | |
|----------------------------------|---|---|
| Archive plumbing (chart, legacy) | <p>Renders <code>spec.plugins</code> (referencing a <code>barman-cloud ObjectStore</code>) on the <code>cnpg.io</code> Cluster from <code>helm-install</code> onwards. CNPG runs the <code>barman-cloud</code> plugin as a WAL-archiver sidecar, every WAL switch ships to object storage, and any backup driver gets a complete WAL chain to start replay from. Renders only when <code>backup.enabled=true</code>, <code>useSystemBucket=false</code>, <code>destinationPath</code> is non-empty, AND inline-or-external creds are supplied. **Skipped in the <code>platformuseSystemBucket=true</code> mode** — the CNPG backup driver SSA-applies the <code>ObjectStore</code> and patches <code>spec.plugins</code> onto the live Cluster at first <code>BackupJob</code> time; until that first <code>BackupJob</code> fires, plugin WAL archiving is not active and WAL accumulates on the PVC. Fire an ad-hoc <code>BackupJob</code> immediately after enabling the flag on existing releases.</p> | <p>Legacy: <code>backup.enabled=true</code> plus <code>backup.destinationPath</code>, <code>backup.endpointURL</code>, and either <code>backup.s3AccessKey+backup.s3SecretKey</code> or <code>backup.s3CredentialsSecret</code>. Platform: <code>backup.enabled=true</code> plus <code>backup.useSystemBucket=true</code> — no S3 fields.</p> |
|----------------------------------|---|---|

| | | |
|---------------------------------------|--|---|
| Backup orchestration (recommended) | Drives ad-hoc and scheduled backups, retention, and restores from <code>backups.cozystack.io</code> resources that can span multiple Postgres apps in a tenant. The driver SSA-applies its own barman-cloud <code>ObjectStore</code> and patches <code>spec.plugins</code> with <code>ForceOwnership</code> (so compression / target settings flow from the strategy template, while the live Cluster's <code>serverName</code> — its WAL-archive S3 prefix — is always preserved); chart-managed values cover the same destination so there is no tug-of-war on the live Cluster. | <code>strategy.backups.cozystack.io/CNPG \+ BackupClass \+ Plan</code> (recurring) or <code>BackupJob</code> (ad-hoc), restores via <code>RestoreJob</code> |
| Legacy chart-emitted scheduled backup | The chart can also emit a <code>cnpg.io/ScheduledBackup</code> directly. Superseded by the <code>BackupClass + Plan</code> path, kept around for clusters that did not migrate. Off by default \- rendered only when both <code>backup.enabled</code> and <code>backup.schedule</code> are non-empty. With <code>backup.schedule=""</code> plugin WAL archiving still runs; only the chart-emitted scheduled backup is silent. | <code>backup.enabled=true</code> plus <code>backup.schedule</code> (CNPG 6-field cron, e.g. <code>"0 2 * * * *"</code>) |

Канонический вариант настройки зависит от того, поставляется ли в кластере платформенный `BackupClass cozy-default`.

Platform-managed flow (recommended for new clusters) — opt in via `backup.useSystemBucket: true` and reference `cozy-default` from `BackupJob/Plan/RestoreJob`. The chart leaves S3 coordinates blank; the CNPG driver SSA-applies a barman-cloud `ObjectStore` and patches `spec.plugins` from the platform-managed bucket coordinates at first `BackupJob` time.

`spec:`

`backup:`

`enabled: true`

`useSystemBucket: true` # platform projects `cozy-backups-creds` + driver SSA-applies `ObjectStore` + patches `spec.plugins`

Legacy chart-managed flow — for clusters that pre-date the platform BackupClass or use a tuned non-default bucket. Provide all S3 coordinates inline (or via `s3CredentialsSecret.name`); `spec.plugins` and the barman-cloud `ObjectStore` render from helm-install onwards.

spec:

backup:

enabled: true # архивировать WAL в объектное хранилище

destinationPath: s3://my-bucket/pg-src/

endpointURL: https://seaweedfs-s3.tenant-foo:8333

s3CredentialsSecret:

name: pg-src-cnpg-backup-creds

endpointCA:

name: pg-src-cnpg-backup-ca

backup.schedule намеренно оставлен пустым — BackupClass / Plan

ниже управляют повторяющимся резервным копированием; никакой

формируемый чартом ScheduledBackup не должен конкурировать с этим расписанием.

paired with a `strategy.backups.cozystack.io/CNPG \+ BackupClass \+ Plan` in the same tenant. The end-to-end e2e fixture under `[examples/backups/postgres/](../../examples/backups/postgres/)` is the canonical reference (`05-postgres-src.yaml` shows the chart side, `10-cnpg-strategy.yaml` and `15-backupclass.yaml` show the orchestration side, `25-backupjob-adhoc.yaml` and `40-restorejob-to-copy.yaml` show ad-hoc backup and restore).

Why both layers and not one? WAL archiving is handled by the barman-cloud plugin's WAL-archiver sidecar - CNPG can attach or detach the plugin at runtime, but any WAL that closed before the plugin began archiving is gone for good. A backup taken from such a cluster is missing the WAL its `begin_wal` points at, and recovery later fails with `WAL not found`. Letting the chart wire the plugin from helm-install removes the race.

How to enable backups (preferred: BackupClass + Plan)

End-to-end manifests live under `[examples/backups/postgres/](../../examples/backups/postgres/)`. Briefly, the moving parts are:

1. A `strategy.backups.cozystack.io/CNPG` describing the destination bucket and templating the barman-cloud `ObjectStore` (including a Secret reference to S3 credentials - the credentials never appear on the Postgres CR .spec; see Security note below).
2. A `backups.cozystack.io/BackupClass` that names the strategy and is selected by an `applicationRef` matching the Postgres app's Kind/Name.
3. A `backups.cozystack.io/Plan` (recurring) or `BackupJob` (ad-hoc) that references the BackupClass. The controller materialises a Backup artifact when the cnpg.io Backup completes; restores then reference that Backup via `RestoreJob`.

Both in-place restores (overwrite the source app's data) and to-copy restores (restore into a separate target Postgres app in the same namespace) are supported via the `RestoreJob.spec.targetApplicationRef` field.

Security: With the BackupClass path, S3 credentials live in a tenant-readable Secret referenced from the strategy template. The CNPG driver forwards that Secret reference into the Postgres app's `spec.backup.s3CredentialsSecret` on restore, so access keys never land in the Postgres CR `.spec`, etcd object store, or `kubectl get -o yaml` output. Prefer this over the chart-managed path whenever possible.

How to enable chart-managed scheduled backups (legacy)

The chart can also emit a `cnpg.io/ScheduledBackup` directly, without a BackupClass. Superseded by the BackupClass + Plan path above and kept around for clusters that did not migrate. It does not run by default - `backup.schedule` defaults to an empty string, which gates the chart's ScheduledBackup template off. To turn it on, fill in a CNPG 6-field cron expression:

@param backup.enabled Enable plugin WAL archiving + render the chart-managed ScheduledBackup

@param backup.schedule Cron schedule (CNPG 6-field). Empty means no chart-managed ScheduledBackup

@param backup.retentionPolicy Retention policy

@param backup.destinationPath Path to store the backup (i.e. `s3://bucket/path/to/folder`)

@param backup.endpointURL S3 Endpoint used to upload data to the cloud

@param backup.s3AccessKey Access key for S3, used for authentication

@param backup.s3SecretKey Secret key for S3, used for authentication

backup:

enabled: true

retentionPolicy: 30d

destinationPath: `s3://bucket/path/to/folder/`

endpointURL: `http://minio-gateway-service:9000`

schedule: "0 2 * * *" # включение — пусто (по умолчанию) означает отсутствие управляемого чартом расписания

s3AccessKey: oobaiRus9pah8PhohL1ThaeTa4UVa7gu

s3SecretKey: ju3eum4dekeich9ahM1te8waeGai0oog

Как восстановить из резервной копии (предпочтительно: RestoreJob)

Для резервных копий, управляемых через BackupClass, создайте `backups.cozystack.io/RestoreJob`, ссылающийся на нужный Backup. См. `[examples/backups/postgres/35-restorejob-in-place.yaml](../examples/backups/postgres/35-restorejob-in-place.yaml)` и `[examples/backups/postgres/40-restorejob-to-copy.yaml](../examples/backups/postgres/40-restorejob-to-copy.yaml)`

b-to-copy.yaml). On a to-copy restore the controller replaces the target app's `spec.databases` and `spec.users` with the source-spec snapshot persisted in `Backup.status.underlyingResources`, so the chart's post-install init-job does not drop the recovered roles or databases.

e2e coverage: the same-namespace to-copy restore (Steps 0-7) leaves the source running and is the deterministic end-to-end signal; it stays a manual / dev-cluster reference flow, not part of the automated e2e suite. The cross-tenant variant (target Postgres in a different tenant from the source's seaweedfs) stays a manual / dev-cluster exercise — reachability is blocked by the per-tenant Cilium egress policy. The in-place restore code path is shipped and is covered at the unit level by `TestClusterHasRecoveryBootstrap_TerminatingCluster`, `TestCNPGBackupWALArchived`, `TestCNPGPurgeNeeded`, and the rest of the `internal/backupcontroller/cnpgstrategy_controller_test.go` suite.

How to recover a backup (chart-managed bootstrap)

CloudNativePG supports point-in-time-recovery. Recovering a backup is done by creating a new database instance and restoring the data in it.

Create a new PostgreSQL application with a different name, but identical configuration. Set `bootstrap.enabled` to `true` and fill in the name of the database instance to recover from and the recovery time:

```
## @param bootstrap.enabled Восстановить кластер базы данных из резервной копии
## @param bootstrap.recoveryTime Метка времени (PITR), до которой будет выполнено
восстановление, в формате RFC 3339. Если оставить пустым, будет восстановлена последняя
## @param bootstrap.oldName Имя кластера базы данных до удаления
##

bootstrap:
enabled: false

recoveryTime: "" # оставьте пустым для последней или укажите точную метку времени; пример:
2020-11-26 15:22:00.000000+00

oldName: "<previous-postgres-instance>"
```

Как переключить первичную/вторичную реплику

См.:

- https://cloudnative-pg.io/documentation/1.15/rolling_update/#manual-updates-supervised

`storageClass` помечен как неизменяемый (immutable) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

TLS для серверных подключений

CNPG управляет цепочкой сертификатов сквозным образом. Оператор автоматически генерирует самоподписанный CA, подписывает им конечные (leaf) сертификаты сервера, клиента и репликации и ротирует их по мере необходимости. Чарт не формирует никаких объектов `Issuer/Certificate cert-manager` — этот путь взаимоисключающий с управляемой оператором цепочкой на admission webhook CNPG.

Что добавляет чарт: когда TLS включён и `external: true`, чарт устанавливает `spec.certificates.serverAltDNSNames` в ресурсе CNPG Cluster CR, чтобы внедрить внешнее имя хоста `<release>.<_namespace>.host` в список SAN автоматически сгенерированного серверного сертификата. Стандартное покрытие SAN в CNPG уже включает три встроенных сервиса ClusterIP (`-rw, -r, -ro`) во всех четырёх формах DNS (`<svc>`, `<svc>.<ns>`, `<svc>.<ns>.svc`, `<svc>.<ns>.svc.<cluster-domain>`); требуется добавить только внешнее имя хоста.

Трёхпозиционный параметр `tls.enabled` определяет, внедряет ли чарт `serverAltDNSNames`:

- `tls.enabled: null` (по умолчанию) — режим TLS наследуется от `external`. При `external: true` чарт внедряет внешнее имя хоста в управляемый оператором сертификат.
- `tls.enabled: true` при `external: true` — эффект тот же, что и по умолчанию.
- `tls.enabled: true` при `external: false` — внедрение `serverAltDNSNames` не требуется (внешнего имени хоста для добавления нет); автоматически сгенерированный сертификат CNPG покрывает внутренние сервисы.
- `tls.enabled: false` — чарт пропускает внедрение `serverAltDNSNames`. Примечание: CNPG сохраняет встроенный TLS на уровне соединения независимо от этого флага; данный переключатель управляет лишь тем, добавляется ли внешнее имя хоста в сертификат. Чтобы полностью отключить TLS в PostgreSQL, потребовалось бы установить `postgresql.parameters.ssl = "off"` на уровне CNPG, что выходит за рамки этого флага.

Получение CA-бандла для проверки на стороне клиента:

CNPG включает сертификат CA в каждый создаваемый им Secret с учётными данными пользователя под ключом `ca.crt`. Получите его из Secret `<release>-credentials`, который уже доступен тенантам через RBAC дашборда:

```
kubectrl --context <ctx> --namespace <tenant> \  
get secret <release>-credentials \
```

```
--output jsonpath='{.data.ca\.crt}'
```

```
base64 --decode
```

Подключение с полной проверкой (пример psql):

```
psql "host=<host> port=5432 dbname=app user=app \  
sslmode=verify-full sslrootcert=ca.crt"
```

Чтобы `sslmode=verify-full` работал, полученный выше CA-бандл должен быть сохранён в `ca.crt`. Без него используйте `sslmode=require` (шифрует, но не проверяет серверный сертификат).

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|---|-----------------------|-----------------------|
| <code>replicas</code> | Количество реплик Postgres. | <code>int</code> | 2 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики PostgreSQL. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.micro</code> |
| <code>size</code> | Размер Persistent Volume Claim, доступный для данных приложения. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>" "</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |
| <code>version</code> | Мажорная версия PostgreSQL для развёртывания | <code>string</code> | <code>v18</code> |

Конфигурация TLS

| Имя | Описание | Тип | Значение |
|-----|----------|-----|----------|
|-----|----------|-----|----------|

| | | | |
|--------------------------|---|---------------------|-------------------|
| <code>tls</code> | Конфигурация TLS для серверных подключений. | <code>object</code> | <code>{}</code> |
| <code>tls.enabled</code> | Трёхпозиционный переключатель, определяющий, внедряет ли чарт внешнее имя хоста в управляемый оператором сертификат CNPG через <code>spec.certificates.serverAltDNSNames</code> . Если не задан, чарт внедряет SAN при <code>external: true</code> и пропускает в противном случае. Установите явно в <code>true</code> , чтобы внедрять независимо от <code>external</code> (не имеет эффекта при <code>external: false</code> , так как внешнего имени хоста для добавления нет). Установите в <code>false</code> , чтобы пропустить внедрение. Учтите, что CNPG сохраняет встроенный TLS на уровне соединения независимо от этого флага — данный переключатель управляет лишь внедрением SAN на стороне чарта; чтобы полностью отключить TLS в PostgreSQL, установите <code>postgresql.parameters.ssl = "off"</code> на уровне CNPG. | <code>*bool</code> | <code>null</code> |

Параметры, специфичные для приложения

| Имя | Описание | Тип | Значение |
|-----------------------|---|------------------------|----------|
| postgresql | Конфигурация сервера PostgreSQL. | object | {} |
| postgresql.parameters | Параметры сервера PostgreSQL. Значения могут быть строками или целыми числами; целые числа приводятся к строкам шаблоном (например, принимаются и <code>max_connections: 100</code> , и <code>max_connections: "100"</code>). ЗАБЛОКИРОВАНЫ (позволяют выполнение произвольного кода): <code>archive_command</code> , <code>restore_command</code> , <code>ssl_passphrase_command</code> , <code>archive_cleanup_command</code> , <code>recovery_end_command</code> , <code>dynamic_library_path</code> , <code>local_preload_libraries</code> , <code>session_preload_libraries</code> , <code>shared_preload_libraries</code> . НЕ переопределяйте параметры, управляемые CloudNativePG: <code>archive_mode</code> , <code>primary_conninfo</code> , <code>wal_level</code> , <code>max_replication_slots</code> . | map[string]intOrString | {} |

Синхронная репликация на основе кворума

| Имя | Описание | Тип | Значение |
|--------|---|--------|----------|
| quorum | Конфигурация кворума для синхронной репликации. | object | {} |

| | | | |
|-------------------------------------|--|------------------|----------------|
| <code>quorum.minSyncReplicas</code> | Минимальное число синхронных реплик, необходимое для фиксации (commit). | <code>int</code> | <code>0</code> |
| <code>quorum.maxSyncReplicas</code> | Максимально допустимое число синхронных реплик (должно быть меньше общего числа реплик). | <code>int</code> | <code>0</code> |

Конфигурация пользователей

| Имя | Описание | Тип | Значение |
|--------------------------------------|--|--------------------------------|--------------------|
| <code>users</code> | Карта конфигурации пользователей. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Пароль пользователя. | <code>string</code> | <code>""</code> |
| <code>users[name].replication</code> | Имеет ли пользователь привилегии репликации. | <code>bool</code> | <code>false</code> |

Конфигурация баз данных

| Имя | Описание | Тип | Значение |
|---|--|--------------------------------|-----------------|
| <code>databases</code> | Карта конфигурации баз данных. | <code>map[string]object</code> | <code>{}</code> |
| <code>databases[name].roles</code> | Роли, назначенные пользователям. | <code>object</code> | <code>{}</code> |
| <code>databases[name].roles.admin</code> | Список пользователей с правами администратора. | <code>[]string</code> | <code>[]</code> |
| <code>databases[name].roles.readonly</code> | Список пользователей с правами только на чтение. | <code>[]string</code> | <code>[]</code> |
| <code>databases[name].extensions</code> | Список включённых расширений PostgreSQL. | <code>[]string</code> | <code>[]</code> |

Параметры резервного копирования

| Name | Description | Type | Value |
|------|-------------|------|-------|
|------|-------------|------|-------|

| | | | |
|------------------------|--|--------|-------|
| backup | Backup configuration. | object | {} |
| backup.enabled | Enable regular backups. | bool | false |
| backup.useSystemBucket | <p>Opt-in: when true, the chart-emitted <code><release>-s3-creds</code> Secret is skipped AND <code>spec.plugins</code> (plus the <code>barman-cloud</code> ObjectStore) is left UNSET in the chart-rendered Cluster — the <code>cozy-default</code> BackupClass driver SSA-applies an ObjectStore (carrying <code>destinationPath/endpointURL/credentials</code>) and patches <code>spec.plugins</code> on the live Cluster when the first BackupJob runs. Consequence: plugin WAL archiving is NOT active until that first BackupJob fires; WAL accumulates on the PVC in the meantime, so fire an ad-hoc BackupJob immediately after enabling the flag on existing releases. Use together with the platform <code>cozy-default</code> BackupClass — tenants do not need to fill <code>s3AccessKey/s3SecretKey</code> or <code>destinationPath/endpointURL</code>. The destination path automatically scopes to <code>s3://cozy-backups/<namespace>/<release>/</code>.</p> | bool | false |

| | | | |
|-------------------------------------|--|---------------------|------------------|
| <code>backup.schedule</code> | <p>Legacy. Cron schedule (CNPg 6-field format) for the chart-emitted <code>ScheduledBackup</code>. Empty means no chart-managed schedule, which is the recommended setup when a <code>BackupClass</code> from <code>backups.cozystack.io</code> already drives backup orchestration. In the legacy chart-managed flow <code>spec.plugins</code> plus the <code>barman-cloud</code> <code>ObjectStore</code> is rendered when <code>backup.enabled=true</code> AND <code>useSystemBucket=false</code> AND <code>destinationPath</code> is non-empty AND inline-or-external creds are supplied; in the platform <code>useSystemBucket=true</code> flow the chart skips emitting <code>spec.plugins</code> and the CNPg driver SSA-applies the <code>ObjectStore</code> and patches <code>spec.plugins</code> onto the live Cluster at first <code>BackupJob</code> time.</p> | <code>string</code> | <code>""</code> |
| <code>backup.retentionPolicy</code> | Retention policy (e.g. "30d"). | <code>string</code> | <code>30d</code> |

| | | | |
|-------------------------------------|---|--------|---|
| <code>backup.destinationPath</code> | DEPRECATED. Per-tenant S3 configuration is superseded by the platform-managed <code>cozy-default BackupClass</code> and the <code>cozy-backups</code> system bucket. Leave empty for new installations; the BackupClass driver picks up the system-managed coordinates. Kept for in-place upgrade compatibility. | string | <code>s3://bucket/path/to/fo
lder/</code> |
| <code>backup.endpointURL</code> | DEPRECATED. See <code>destinationPath</code> . | string | <code>http://minio-gateway-s
ervice:9000</code> |
| <code>backup.s3AccessKey</code> | DEPRECATED. Tenants no longer supply S3 keys; the system Bucket Secret is projected into the tenant namespace by the backup controller. Ignored when <code>s3CredentialsSecret.name</code> is set or <code>useSystemBucket</code> is true. The chart skips materialising <code><release>-s3-creds</code> whenever this field is empty so a default install does not leak placeholder credentials into the tenant namespace. | string | <code>" "</code> |
| <code>backup.s3SecretKey</code> | DEPRECATED. See <code>s3AccessKey</code> . | string | <code>" "</code> |

| | | | |
|--|--|--------|-----------------|
| <code>backup.s3CredentialsSecret</code> | DEPRECATED. Pre-existing Secret with S3 credentials. Use the platform-managed <code>cozy-default BackupClass</code> instead. When set, the chart references this Secret directly (legacy chart-managed flow). The CNPG backup driver writes this field on restore so credentials never land in the CR <code>.spec</code> . | object | <code>{}</code> |
| <code>backup.s3CredentialsSecret.name</code> | Name of the Secret in the application namespace. Empty means the chart materialises <code><release>-s3-creds</code> from <code>s3AccessKey/s3SecretKey</code> . | string | <code>""</code> |
| <code>backup.s3CredentialsSecret.accessKeyIDKey</code> | Key in the Secret holding the access key ID. Defaults to <code>AWS_ACCESS_KEY_ID</code> . | string | <code>""</code> |
| <code>backup.s3CredentialsSecret.secretAccessKeyKey</code> | Key in the Secret holding the secret access key. Defaults to <code>AWS_SECRET_ACCESS_KEY</code> . | string | <code>""</code> |
| <code>backup.endpointCA</code> | DEPRECATED. Pre-existing Secret with the CA bundle the barman-cloud plugin should trust when reaching a self-signed S3 endpoint. Used for both backup and bootstrap recovery in the legacy chart-managed flow. | object | <code>{}</code> |

| | | | |
|-------------------------------------|---|--------|----|
| <code>backup.endpointCA.name</code> | Name of the Secret in the application namespace. Empty means no endpointCA is emitted (the plugin uses the system trust store). | string | "" |
| <code>backup.endpointCA.key</code> | Key within the Secret containing the CA bundle. Defaults to <code>ca.crt</code> . | string | "" |

Параметры начальной загрузки (восстановления)

| Name | Description | Type | Value |
|-------------------------------------|--|--------|--------------------|
| <code>bootstrap</code> | Bootstrap configuration. | object | <code>{}</code> |
| <code>bootstrap.enabled</code> | Whether to restore from a backup. | bool | <code>false</code> |
| <code>bootstrap.recoveryTime</code> | Timestamp (RFC3339) for point-in-time recovery; empty means latest. | string | "" |
| <code>bootstrap.oldName</code> | Previous cluster name before deletion. | string | "" |
| <code>bootstrap.serverName</code> | Server name (S3 path prefix) used by the original cluster when writing backups; passed to the barman-cloud plugin via <code>externalClusters[].plugin.parameters.serverName</code> . Defaults to <code>bootstrap.oldName</code> . Set this only when the original cluster wrote backups under an explicit server name that differed from its Kubernetes resource name. | string | "" |

Примеры и справочник по параметрам

resources и resourcesPreset

`resources` задаёт явные конфигурации CPU и памяти для каждой реплики. Если оставить пустым, применяется пресет, указанный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованные конфигурации CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют облачной схеме именования `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия поставляется с восемью размерами (от `nano` до `4xlarge`). Устаревшие плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов инстансов с соотношением 1:1.

См. [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#), где приведены полная матрица размеров и сопоставление устаревших имён с типами инстансов.

users

`users:`

`user1:`

`password: strongpassword`

`user2:`

`password: hackme`

`airflow:`

`password: qwerty123`

`debezium:`

`replication: true`

databases

`databases:`

`myapp:`

`roles:`

`admin:`

-
- user1
 - debezium

readonly:

- user2

airflow:

roles:

admin:

- airflow

extensions:

- hstore

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис Qdrant

<https://cozystack.ru/docs/v1.6/applications/qdrant/>

Qdrant — это высокопроизводительная векторная база данных и движок поиска по сходству, разработанный для приложений искусственного интеллекта и машинного обучения. Он обеспечивает эффективное хранение и извлечение многомерных векторов с продвинутыми возможностями фильтрации, что делает его идеальным для рекомендательных систем, семантического поиска и приложений RAG (Retrieval-Augmented Generation, генерация с дополненной выборкой).

Детали развёртывания

Сервис разворачивает Qdrant как StatefulSet с автоматическим режимом кластера, когда настроено несколько реплик.

- Документация: <<https://qdrant.tech/documentation/>>
- GitHub: <<https://github.com/qdrant/qdrant>>

`storageClass` помечен как неизменяемый (`immutable`) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|------------------------|---|---------------------|-----------------|
| <code>replicas</code> | Количество реплик Qdrant. Режим кластера включается автоматически при <code>replicas > 1</code> . | <code>int</code> | 1 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики Qdrant. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |

| | | | |
|-------------------------------|---|-----------------------|-----------------------|
| <code>resources.cpu</code> | ЦПУ, доступный каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>tl.small</code> |
| <code>size</code> | Размер Persistent Volume Claim, доступный для хранения векторных данных. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>""</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

Параметры TLS

| Имя | Описание | Тип | Значение |
|--------------------------|--|---------------------|-------------------|
| <code>tls</code> | Конфигурация TLS. Если не задано, фактическое состояние TLS следует значению <code>external</code> . | <code>object</code> | <code>{}</code> |
| <code>tls.enabled</code> | Включить TLS. Если не задано, наследует значение <code>external</code> (TLS включён, когда включён внешний доступ). Установите явно в <code>true</code> или <code>false</code> , чтобы переопределить. | <code>*bool</code> | <code>null</code> |

Примеры и справочник по параметрам

`resources` и `resourcesPreset`

`resources` задаёт явные конфигурации CPU и памяти для каждой реплики. Если оставить пустым, применяется пресет, указанный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованные конфигурации CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют облачной схеме именования `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия поставляется с восемью размерами (от `nano` до `4xlarge`). Устаревшие плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов инстансов с соотношением 1:1.

См. [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#), где приведены полная матрица размеров и сопоставление устаревших имён с типами инстансов.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис RabbitMQ

<https://cozystack.ru/docs/v1.6/applications/rabbitmq/>

RabbitMQ — это надёжный брокер сообщений, играющий ключевую роль в современных распределённых системах. Наш управляемый сервис RabbitMQ упрощает развёртывание и управление кластерами RabbitMQ, обеспечивая надёжность и масштабируемость для ваших задач обмена сообщениями.

Детали развёртывания

Сервис использует официальный оператор RabbitMQ. Это обеспечивает надёжность и бесперебойную работу ваших экземпляров RabbitMQ.

- Github: <<https://github.com/rabbitmq/cluster-operator/>>
- Документация: <<https://www.rabbitmq.com/kubernetes/operator/operator-overview.html>>

`storageClass` помечен как неизменяемый (immutable) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|---|-----------------------|------------------|
| <code>replicas</code> | Количество реплик RabbitMQ. | <code>int</code> | 3 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики RabbitMQ. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |

| | | | |
|------------------------------|---|-----------------------|----------------------|
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>size</code> | Размер Persistent Volume Claim, доступный для данных приложения. | <code>quantity</code> | <code>10Gi</code> |
| <code>storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>" "</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |
| <code>version</code> | Версия RabbitMQ (мажорная.минорная) для развёртывания | <code>string</code> | <code>v4.2</code> |

Параметры, специфичные для приложения

| Имя | Описание | Тип | Значение |
|--|---|--------------------------------|------------------|
| <code>users</code> | Карта конфигурации пользователей. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Пароль пользователя. | <code>string</code> | <code>" "</code> |
| <code>vhosts</code> | Карта конфигурации виртуальных хостов. | <code>map[string]object</code> | <code>{}</code> |
| <code>vhosts[name].roles</code> | Список ролей виртуального хоста. | <code>object</code> | <code>{}</code> |
| <code>vhosts[name].roles.admin</code> | Список пользователей-администраторов. | <code>[]string</code> | <code>[]</code> |
| <code>vhosts[name].roles.readonly</code> | Список пользователей только для чтения. | <code>[]string</code> | <code>[]</code> |

Примеры и справочник по параметрам

`resources` и `resourcesPreset`

`resources` задаёт явные конфигурации CPU и памяти для каждой реплики. Если оставить пустым, применяется пресет, указанный в `resourcesPreset`.

resources:

cpu: 4000m

memory: 4Gi

`resourcesPreset` задаёт именованные конфигурации CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

| Имя пресета | CPU | память |
|-------------|------|--------|
| nano | 100m | 128Mi |
| micro | 250m | 256Mi |
| small | 500m | 512Mi |
| medium | 500m | 1Gi |
| large | 1 | 2Gi |
| xlarge | 2 | 4Gi |
| 2xlarge | 4 | 8Gi |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис Redis

<https://cozystack.ru/docs/v1.6/applications/redis/>

Redis — это исключительно универсальное и очень быстрое хранилище данных и кэш в оперативной памяти, способное значительно повысить производительность ваших приложений. Управляемый сервис Redis предлагает бесперебойное решение для развёртывания кластеров Redis и управления ими, гарантируя, что ваши данные всегда доступны и отзывчивы.

Детали развёртывания

Сервис использует Spotahome Redis Operator для эффективного управления и оркестрации кластеров Redis.

- Документация: <https://redis.io/docs/>
- GitHub: <https://github.com/spotahome/redis-operator>

`storageClass` помечен как неизменяемый (immutable) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-------------------------------|--|-----------------------|------------------|
| <code>replicas</code> | Количество реплик Redis. | <code>int</code> | 2 |
| <code>resources</code> | Явная конфигурация CPU и памяти для каждой реплики Redis. Если не задано, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |

| | | | |
|------------------------------|---|-----------------------|----------------------|
| <code>resourcesPreset</code> | Пресет размера по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>size</code> | Размер Persistent Volume Claim, доступный для данных приложения. | <code>quantity</code> | <code>1Gi</code> |
| <code>storageClass</code> | StorageClass, используемый для хранения данных. | <code>string</code> | <code>" "</code> |
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |
| <code>version</code> | Мажорная версия Redis для развёртывания | <code>string</code> | <code>v8</code> |

Параметры, специфичные для приложения

| Имя | Описание | Тип | Значение |
|--------------------------|----------------------------|-------------------|-------------------|
| <code>authEnabled</code> | Включить генерацию пароля. | <code>bool</code> | <code>true</code> |

Примеры и справочник по параметрам

`resources` и `resourcesPreset`

`resources` задаёт явные конфигурации CPU и памяти для каждой реплики. Если оставить пустым, применяется пресет, указанный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованные конфигурации CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют облачной схеме именования `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия поставляется с восемью размерами (от `nano` до `4xlarge`). Устаревшие плоские имена (`nano`, `micro`,

`small, medium, large, xlarge, 2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов инстансов с соотношением 1:1.

См. [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#), где приведены полная матрица размеров и сопоставление устаревших имён с типами инстансов.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Справочник по приложению Tenant

<https://cozystack.ru/docs/v1.6/applications/tenant/>

Тенант — это основная единица безопасности на платформе. Ближайшая аналогия — пространства имён ядра Linux.

Тенанты могут создаваться рекурсивно и подчиняются следующим правилам:

Именованние тенантов

Имена тенантов должны быть буквенно-цифровыми:

- Только строчные буквы (a-z) и цифры (0-9)
- Должны начинаться со строчной буквы
- Дефисы (-) не допускаются, в отличие от других сервисов
- Максимальная длина зависит от конфигурации кластера (префикс Helm-релиза и корневой домен)

Это ограничение существует для поддержания согласованного именования в тенантах, вложенных тенантах и развёрнутых в них сервисах. Тенант нельзя назвать `foo-bar`, потому что разбор внутренних имён ресурсов вроде `tenant-foo-bar` был бы неоднозначным.

Например:

- Корневой тенант называется `root`, но внутри он обозначается как `tenant-root`.
- Вложенный тенант может называться `foo`, что даст `tenant-foo` в именах сервисов и URL.

Уникальные домены

Каждый тенант имеет собственный домен. По умолчанию (если не указано иное) он наследует домен своего родителя с префиксом из своего имени. Например, если у родителя был домен `example.org`, то `tenant-foo` по умолчанию получит домен `foo.example.org`.

Кластеры Kubernetes, созданные в пространстве имён этого тенанта, получают домены вида:
`kubernetes-cluster.foo.example.org`

Пример:

`tenant-root (example.org)`

`+-- tenant-foo (foo.example.org)`

`+-- kubernetes-cluster1 (kubernetes-cluster1.foo.example.org)`

Вложенность тенантов и переиспользование родительских сервисов

Тенанты могут быть вложенными. Администратор тенанта может создавать вложенные тенанты с помощью приложения «Tenant» из каталога.

Тенанты более высокого уровня могут просматривать приложения всех своих дочерних тенантов и управлять ими. Если тенант не запускает собственные кластерные сервисы, он может использовать сервисы своего родителя.

Например, вы создаёте:

- Тенант `tenant-u1` с набором сервисов, таких как `etcd`, `ingress`, `monitoring`.
- Тенант `tenant-u2`, вложенный в `tenant-u1`.

Посмотрим, что произойдёт, когда вы запустите Kubernetes и Postgres в пространстве имён `tenant-u2`.

Поскольку у `tenant-u2` нет собственных кластерных сервисов, таких как `etcd`, `ingress` и `monitoring`, приложения, работающие в `tenant-u2`, будут использовать кластерные сервисы родительского тенанта.

Это, в свою очередь, означает:

- Данные кластера Kubernetes будут храниться в `etcd` тенанта `tenant-u1`.
- Доступ к кластеру будет осуществляться через общий `ingress` тенанта `tenant-u1`.
- По сути, все метрики будут собираться в `monitoring` тенанта `tenant-u1`, и только этот тенант будет иметь к ним доступ.

Пример:

```
tenant-u1
+-- etcd
+-- ingress
+-- monitoring
+-- tenant-u2
+-- kubernetes-cluster1
+-- postgres-db1
```

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-----|----------|-----|----------|
|-----|----------|-----|----------|

| | | | |
|------------|--|--------|-------|
| host | Имя хоста, используемое для доступа к сервисам тенанта (по умолчанию используется имя тенанта как поддомен хоста родительского тенанта). | string | "" |
| etcd | Развернуть собственный кластер Etcd. | bool | false |
| monitoring | Развернуть собственный стек мониторинга. | bool | false |
| ingress | Развернуть собственный Ingress-контроллер. | bool | false |

| | | | |
|-----------------|---|---------------------|-------|
| gateway | <p>Развернуть собственный шлюз Gateway API (на основе контроллера Cilium Gateway API).</p> <p>Если не задано (по умолчанию), чарт автоматически включает Gateway для tenants, чей apex-домен наследуется от родителя (т.е. <code>host</code> пуст), и оставляет его выключенным для tenants с пользовательским ненаследуемым apex-доменом.</p> <p>Установите явно <code>true</code> или <code>false</code>, чтобы переопределить это автоматическое поведение. Примечание: оставьте ключ отсутствующим (не пишите <code>gateway: null</code>) — чарт различает «не задано» по отсутствию ключа, а не по значению <code>null</code>, чтобы соответствовать JSON-схеме, сгенерированной из этого комментария.</p> | bool | false |
| seaweedfs | Развернуть собственный SeaweedFS. | bool | false |
| schedulingClass | Имя ресурса Scheduling Class CR для применения ограничений планирования к рабочим нагрузкам этого tenant. | string | " " |
| resourceQuotas | Определить квоты ресурсов для tenant. | map[string]quantity | {} |

Конфигурация

Квоты ресурсов

Параметр `resourceQuotas` позволяет ограничить ресурсы, доступные тенанту. Поддерживаемые ключи:

Вычислительные ресурсы (преобразуются в `requests.X` и `limits.X`):

- `cpu` — Суммарное число ядер CPU (например, "4" или "500m")
- `memory` — Суммарный объём памяти (например, "4Gi" или "512Mi")
- `ephemeral-storage` — Лимит эфемерного хранилища (например, "10Gi")
- `storage` — Суммарный объём постоянного хранилища (например, "100Gi")

Квоты на количество объектов (передаются как есть):

- `Pods` — Максимальное число подов
- `services` — Максимальное число сервисов
- `services.loadbalancers` — Максимальное число сервисов типа LoadBalancer
- `services.nodeports` — Максимальное число сервисов типа NodePort
- `configmaps` — Максимальное число ConfigMap
- `secrets` — Максимальное число Secret
- `persistentvolumeclaims` — Максимальное число PVC

Пример:

```
resourceQuotas:
```

```
cpu: 4
```

```
memory: 4Gi
```

```
storage: 10Gi
```

```
services.loadbalancers: "3"
```

```
pods: "50"
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Виртуализация

Возможности виртуализации в Cozystack

<https://cozystack.ru/docs/v1.6/virtualization/>

Всё о развёртывании, настройке и использовании виртуальных машин в Cozystack.

В этом руководстве объясняется, как работает виртуализация в Cozystack.

Пакеты виртуализации

Каталог Cozystack включает два пакета, связанных с виртуализацией:

- `vm-disk` \- диск виртуальной машины
- `vm-instance` \- экземпляр виртуальной машины

Диск виртуальной машины

Прежде чем создать экземпляр виртуальной машины, необходимо создать диск, с которого будет загружаться VM.

Этот пакет определяет диск виртуальной машины, используемый для хранения данных. Вы можете использовать подготовленный образ (также известный как `golden image`), загрузить образ на диск по HTTP или загрузить его из локального образа. Также можно создать пустой образ.

1. Golden Image :

`## @param source` The source image location used to create a disk

`source:`

`image:`

`name: ubuntu`

2. HTTP:

`source:`

`http:`

`url: "https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img"`

3. Upload:

source:

upload: {}

После создания диска будет сгенерирована команда для загрузки с помощью инструмента virtctl.

Примечание:

Если вы хотите, чтобы virtctl знал о правильной конечной точке для загрузки образов, необходимо настроить кластер, указав для него конечную точку:

1. Пропатчите Platform Package, чтобы открыть доступ к `cdi-uploadproxy` вместе с панелью управления:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p ' [{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "
cdi-uploadproxy"} ]'
```

2. Укажите действительную конечную точку CDI uploadproxy, пропатчив Package `kubevirt-cdi`:

```
kubectl patch packages.cozystack.io cozystack.kubevirt-cdi --type=merge -p '{
"spec": {
"components": {
"kubevirt-cdi": {
"values": {
"uploadProxyURL": "https://cdi-uploadproxy.example.org"
}
}
}
}'
```

4. Empty:

source: {}

При желании можно указать, что диск является оптическим CD-ROM:

optical: true

Созданные диски можно подключать к экземпляру виртуальной машины.

См. справочник по приложению: `[vm-disk]`(<https://cozystack.ru/docs/v1.6/virtualization/vm-disk/>).

Экземпляр виртуальной машины

Этот пакет определяет экземпляр виртуальной машины, для которого требуется указать ранее созданный `vm-disk`. Первый диск всегда является загрузочным, и VM будет пытаться загрузиться с него.

disks:

- name: example-system
- name: example-data

Остальные параметры аналогичны Virtual Machine (simple).

См. справочник по приложению: `[vm-instance]`(<https://cozystack.ru/docs/v1.6/virtualization/vm-instance/>).

Доступ к виртуальным машинам

Получить доступ к виртуальной машине можно с помощью инструмента `virtctl`:

- [KubeVirt User Guide - Virtctl Client Tool](#)

Для доступа к последовательной консоли:

```
virtctl console <vm>
```

Для доступа к VM через VNC:

```
virtctl vnc <vm>
```

Для подключения к VM по SSH:

```
virtctl ssh <user>@<vm>
```

•

[Виртуальная машина](#)

[Диск виртуальной машины](#)

[Создание и использование именованных образов VM](#)

Руководство по созданию, управлению и использованию золотых (именованных) образов VM в Cozystack для ускорения развёртывания виртуальных машин.

[Клонируемые виртуальные машины](#)

Создание клонируемых виртуальных машин

[Запуск VM с пробросом GPU \(passthrough\)](#)

Запуск VM с пробросом GPU (passthrough)

[Резервное копирование и восстановление](#)

Как создавать резервные копии ресурсов VMInstance и VMDisk и управлять ими с помощью BackupJob и Plan.

[Запуск виртуальных машин Windows в Cozystack](#)

Запуск виртуальных машин Windows в Cozystack

[Запуск MikroTik RouterOS в Cozystack](#)

Развёртывание MikroTik RouterOS (CHR) как виртуального устройства в Cozystack

[Миграция виртуальных машин из Proxmox](#)

Пошаговое руководство по миграции виртуальных машин из Proxmox VE в Cozystack

[Ресурсы виртуальных машин](#)

Справочник по типам инстансов и профилям инстансов VM

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Виртуальная машина

<https://cozystack.ru/docs/v1.6/virtualization/vm-instance/>

Виртуальная машина (ВМ) имитирует аппаратное обеспечение компьютера, позволяя запускать различные операционные системы и приложения в изолированной среде.

Детали развёртывания

Виртуальная машина управляется и размещается с помощью KubeVirt, что позволяет использовать преимущества виртуализации в рамках вашей экосистемы Kubernetes.

- Документация: [KubeVirt User Guide](#)
- GitHub: [KubeVirt Repository](#)

Доступ к виртуальной машине

Вы можете получить доступ к виртуальной машине с помощью инструмента virtctl:

- [KubeVirt User Guide - Virtctl Client Tool](#)

Для доступа к последовательной консоли:

```
virtctl console <vm>
```

Для доступа к ВМ по VNC:

```
virtctl vnc <vm>
```

Для подключения к ВМ по SSH:

```
virtctl ssh <user>@<vm>
```

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-----------------------|---|-------------------|--------------------|
| <code>external</code> | Включить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

| | | | |
|-------------------|--|----------|-----------|
| externalMethod | Метод передачи трафика к VM. | string | PortList |
| externalPorts | Порты для проброса извне кластера. | []int | [22] |
| externalAllowICMP | Принимать ли ICMP-трафик к VM в режиме PortList (сохраняет ping и обнаружение PMTU). Не действует в режиме WholeIP. По умолчанию true, чтобы ping работал так, как ожидают пользователи, даже когда действует фильтрация портов. | bool | true |
| runStrategy | Запрошенное состояние работы VirtualMachineInstance | string | Always |
| instanceType | Тип инстанса виртуальной машины. | string | u1.medium |
| instanceProfile | Профиль предпочтений виртуальной машины. | string | ubuntu |
| disks | Список дисков для подключения. | []object | [] |
| disks[i].name | Имя диска. | string | "" |
| disks[i].bus | Тип шины диска (например, "sata"). | string | "" |
| networks | Сети, к которым подключается VM. | []object | [] |
| networks[i].name | Имя сетевого подключения (network attachment). | string | "" |
| subnets | Устарело: используйте networks вместо этого. | []object | [] |
| subnets[i].name | Имя сетевого подключения (network attachment). | string | "" |

| | | | |
|--------------------------------|--|-----------------------|-----------------|
| <code>gpus</code> | Список GPU для подключения (драйвер NVIDIA требует не менее 4 Гиб RAM). | <code>[]object</code> | <code>[]</code> |
| <code>gpus[i].name</code> | Имя ресурса GPU для подключения. | <code>string</code> | <code>""</code> |
| <code>cpuModel</code> | Model задаёт модель CPU внутри VMI. Список доступных моделей < https://github.com/libvirt/libvirt/tree/master/src/cpu_map > | <code>string</code> | <code>""</code> |
| <code>resources</code> | Конфигурация ресурсов для виртуальной машины. | <code>object</code> | <code>{}</code> |
| <code>resources.cpu</code> | Количество выделенных ядер CPU. | <code>quantity</code> | <code>""</code> |
| <code>resources.memory</code> | Объём выделенной памяти. | <code>quantity</code> | <code>""</code> |
| <code>resources.sockets</code> | Количество сокетов CPU (топология vCPU). | <code>quantity</code> | <code>""</code> |
| <code>sshKeys</code> | Список открытых SSH-ключей для аутентификации. | <code>[]string</code> | <code>[]</code> |
| <code>cloudInit</code> | Пользовательские данные cloud-init. | <code>string</code> | <code>""</code> |
| <code>cloudInitSeed</code> | Seed-строка для генерации SMBIOS UUID для VM. | <code>string</code> | <code>""</code> |

Серия U

Серия U довольно нейтральна и предоставляет ресурсы для приложений общего назначения.

U — это сокращение от «Universal», что намекает на универсальное отношение к рабочим нагрузкам.

ВМ этих типов инстансов делят физические ядра CPU на основе квантования времени с другими ВМ.

Характеристики серии U

Специфические характеристики этой серии:

- _Пакетная (burstable) производительность CPU_ — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
- _Соотношение vCPU к памяти (1:4)_ — соотношение vCPU к памяти 1:4, для меньшего «шума» на узел.

Серия O

Серия O основана на серии U, с единственным отличием, заключающимся в том, что память переподписана (overcommitted).

O — это сокращение от «Overcommitted».

Характеристики серии UO

Специфические характеристики этой серии:

- _Пакетная (burstable) производительность CPU_ — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
- _Переподписанная память_ — память переподписана для достижения более высокой плотности рабочих нагрузок.
- _Соотношение vCPU к памяти (1:4)_ — соотношение vCPU к памяти 1:4, для меньшего «шума» на узел.

Серия CX

Серия CX предоставляет эксклюзивные вычислительные ресурсы для вычислительно интенсивных приложений.

CX — это сокращение от «Compute Exclusive».

Эксклюзивные ресурсы предоставляются вычислительным потокам VM. Чтобы обеспечить это, будут запрошены некоторые дополнительные ядра (в зависимости от количества дисков и NIC), чтобы вынести потоки ввода-вывода с ядер, выделенных для рабочей нагрузки. Кроме того, в этой серии топология NUMA используемых ядер предоставляется VM.

Характеристики серии CX

Специфические характеристики этой серии:

-
- `_Hugepages_` — Hugepages используются для улучшения производительности памяти.
 - `_Выделенный CPU_` — физические ядра эксклюзивно назначаются каждому vCPU, чтобы предоставить фиксированные и высокие гарантии вычислений для рабочей нагрузки.
 - `_Изолированные потоки эмулятора_` — потоки эмулятора гипервизора изолированы от vCPU, чтобы уменьшить влияние, связанное с эмуляцией, на рабочую нагрузку.
 - `_vNUMA_` — физическая топология NUMA отражается в гостевой системе, чтобы оптимизировать использование кэша на стороне гостя.
 - `_Соотношение vCPU к памяти (1:2)_` — соотношение vCPU к памяти 1:2.

Серия M

Серия M предоставляет ресурсы для приложений, интенсивно использующих память.

`_M_` — это сокращение от «Memory».

Характеристики серии M

Специфические характеристики этой серии:

- `_Hugepages_` — Hugepages используются для улучшения производительности памяти.
- `_Пакетная (burstable) производительность CPU_` — рабочая нагрузка имеет базовый уровень вычислительной производительности, но может выходить за пределы этого базового уровня, если доступны избыточные вычислительные ресурсы.
- `_Соотношение vCPU к памяти (1:8)_` — соотношение vCPU к памяти 1:8, для гораздо меньшего «шума» на узел.

Серия RT

Серия RT предоставляет ресурсы для приложений реального времени, таких как Oslat.

`_RT_` — это сокращение от «realtime».

Эта серия типов инстансов требует узлов, способных запускать приложения реального времени.

Характеристики серии RT

Специфические характеристики этой серии:

- `_Hugepages_` — Hugepages используются для улучшения производительности памяти.
- `_Выделенный CPU_` — физические ядра эксклюзивно назначаются каждому vCPU, чтобы предоставить фиксированные и высокие гарантии вычислений для рабочей нагрузки.
- `_Изолированные потоки эмулятора_` — потоки эмулятора гипервизора изолированы от vCPU, чтобы уменьшить влияние, связанное с эмуляцией, на рабочую нагрузку.

- `_Соотношение vCPU к памяти (1:4)_` — соотношение vCPU к памяти 1:4, начиная с размера medium.

Разработка

Чтобы начать настройку или создание собственных `instancetype` и `preferences`, см. [DEVELOPMENT.md](#).

Ресурсы

Следующие ресурсы `instancetype` предоставляются Cozystack:

| Name | vCPUs | Memory |
|----------------|-------|--------|
| cx1.2xlarge | 8 | 16Gi |
| cx1.2xlarge1gi | 8 | 16Gi |
| cx1.4xlarge | 16 | 32Gi |
| cx1.4xlarge1gi | 16 | 32Gi |
| cx1.8xlarge | 32 | 64Gi |
| cx1.8xlarge1gi | 32 | 64Gi |
| cx1.large | 2 | 4Gi |
| cx1.large1gi | 2 | 4Gi |
| cx1.medium | 1 | 2Gi |
| cx1.medium1gi | 1 | 2Gi |
| cx1.xlarge | 4 | 8Gi |
| cx1.xlarge1gi | 4 | 8Gi |
| d1.2xlarge | 8 | 32Gi |
| d1.2xmedium | 2 | 4Gi |
| d1.4xlarge | 16 | 64Gi |
| d1.8xlarge | 32 | 128Gi |
| d1.large | 2 | 8Gi |
| d1.medium | 1 | 4Gi |
| d1.micro | 1 | 1Gi |
| d1.nano | 1 | 512Mi |

| | | |
|---------------|----|-------|
| d1.small | 1 | 2Gi |
| d1.xlarge | 4 | 16Gi |
| gn1.2xlarge | 8 | 32Gi |
| gn1.4xlarge | 16 | 64Gi |
| gn1.8xlarge | 32 | 128Gi |
| gn1.xlarge | 4 | 16Gi |
| m1.2xlarge | 8 | 64Gi |
| m1.2xlarge1gi | 8 | 64Gi |
| m1.4xlarge | 16 | 128Gi |
| m1.4xlarge1gi | 16 | 128Gi |
| m1.8xlarge | 32 | 256Gi |
| m1.8xlarge1gi | 32 | 256Gi |
| m1.large | 2 | 16Gi |
| m1.large1gi | 2 | 16Gi |
| m1.xlarge | 4 | 32Gi |
| m1.xlarge1gi | 4 | 32Gi |
| n1.2xlarge | 16 | 32Gi |
| n1.4xlarge | 32 | 64Gi |
| n1.8xlarge | 64 | 128Gi |
| n1.large | 4 | 8Gi |
| n1.medium | 4 | 4Gi |
| n1.xlarge | 8 | 16Gi |
| o1.2xlarge | 8 | 32Gi |
| o1.4xlarge | 16 | 64Gi |
| o1.8xlarge | 32 | 128Gi |
| o1.large | 2 | 8Gi |
| o1.medium | 1 | 4Gi |
| o1.micro | 1 | 1Gi |
| o1.nano | 1 | 512Mi |
| o1.small | 1 | 2Gi |

| | | |
|-------------|----|-------|
| o1.xlarge | 4 | 16Gi |
| rt1.2xlarge | 8 | 32Gi |
| rt1.4xlarge | 16 | 64Gi |
| rt1.8xlarge | 32 | 128Gi |
| rt1.large | 2 | 8Gi |
| rt1.medium | 1 | 4Gi |
| rt1.micro | 1 | 1Gi |
| rt1.small | 1 | 2Gi |
| rt1.xlarge | 4 | 16Gi |
| u1.2xlarge | 8 | 32Gi |
| u1.2xmedium | 2 | 4Gi |
| u1.4xlarge | 16 | 64Gi |
| u1.8xlarge | 32 | 128Gi |
| u1.large | 2 | 8Gi |
| u1.medium | 1 | 4Gi |
| u1.micro | 1 | 1Gi |
| u1.nano | 1 | 512Mi |
| u1.small | 1 | 2Gi |
| u1.xlarge | 4 | 16Gi |

Следующие ресурсы preference предоставляются Cozystack:

| Name | Guest OS |
|-------------------------|------------------|
| alpine | Alpine |
| centos.stream10 | CentOS Stream 10 |
| centos.stream10.desktop | CentOS Stream 10 |
| centos.stream9 | CentOS Stream 9 |
| centos.stream9.desktop | CentOS Stream 9 |
| centos.stream9.dpdk | CentOS Stream 9 |
| cirros | Cirros |
| debian | Debian |
| fedora | Fedora (amd64) |

| | |
|--------------------------|---|
| fedora.arm64 | Fedora (arm64) |
| fedora.s390x | Fedora (s390x) |
| legacy | Legacy Guest |
| linux | Linux Guest |
| linux.efi | Linux EFI Guest |
| linux.virtiotransitional | Linux Virtio Transitional Guest |
| opensuse.leap | OpenSUSE Leap |
| opensuse.tumbleweed | OpenSUSE Tumbleweed |
| oraclelinux | Oracle Linux |
| rhel.10 | Red Hat Enterprise Linux 10 (amd64) |
| rhel.10.arm64 | Red Hat Enterprise Linux 10 (arm64) |
| rhel.10.s390x | Red Hat Enterprise Linux 10 (s390x) |
| rhel.7 | Red Hat Enterprise Linux 7 |
| rhel.7.desktop | Red Hat Enterprise Linux 7 |
| rhel.8 | Red Hat Enterprise Linux 8 |
| rhel.8.desktop | Red Hat Enterprise Linux 8 |
| rhel.8.dpkg | Red Hat Enterprise Linux 8 |
| rhel.9 | Red Hat Enterprise Linux 9 (amd64) |
| rhel.9.arm64 | Red Hat Enterprise Linux 9 (arm64) |
| rhel.9.desktop | Red Hat Enterprise Linux 9 Desktop (amd64) |
| rhel.9.dpkg | Red Hat Enterprise Linux 9 DPKG (amd64) |
| rhel.9.realtime | Red Hat Enterprise Linux 9 Realtime (amd64) |
| rhel.9.s390x | Red Hat Enterprise Linux 9 (s390x) |
| sles | SUSE Linux Enterprise Server |
| ubuntu | Ubuntu |
| windows.10 | Microsoft Windows 10 |
| windows.10.virtio | Microsoft Windows 10 (virtio) |
| windows.11 | Microsoft Windows 11 |
| windows.11.virtio | Microsoft Windows 11 (virtio) |
| windows.2k12 | Microsoft Windows Server 2012/2012 R2 |

| | |
|---------------------|--|
| windows.2k12.virtio | Microsoft Windows Server 2012/2012 R2 (virtio) |
| windows.2k16 | Microsoft Windows Server 2016 |
| windows.2k16.virtio | Microsoft Windows Server 2016 (virtio) |
| windows.2k19 | Microsoft Windows Server 2019 |
| windows.2k19.virtio | Microsoft Windows Server 2019 (virtio) |
| windows.2k22 | Microsoft Windows Server 2022 |
| windows.2k22.virtio | Microsoft Windows Server 2022 (virtio) |
| windows.2k25 | Microsoft Windows Server 2025 |
| windows.2k25.virtio | Microsoft Windows Server 2025 (virtio) |
| windows.2k3 | Microsoft Windows Server 2003 |
| windows.2k8 | Microsoft Windows Server 2008/2008 R2 |
| windows.2k8.virtio | Microsoft Windows Server 2008/2008 R2 (virtio) |
| windows.7 | Microsoft Windows 7 |
| windows.7.virtio | Microsoft Windows 7 (virtio) |
| windows.xp | Microsoft Windows XP |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Диск виртуальной машины

<https://cozystack.ru/docs/v1.6/virtualization/vm-disk/>

Диск виртуальной машины

`storageClass` помечен как неизменяемый (`immutable`) в схеме чарта — см. [\[docs/storage-immutability.md\]\(../docs/storage-immutability.md\)](#), где описан этот контракт и какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|--------------------------------|--|----------------------|--------------------|
| <code>source</code> | Расположение исходного образа, используемого для создания диска. | <code>object</code> | <code>{}</code> |
| <code>source.image</code> | Использовать образ по имени из коллекции по умолчанию. | <code>*object</code> | <code>null</code> |
| <code>source.image.name</code> | Имя используемого образа. | <code>string</code> | <code>""</code> |
| <code>source.upload</code> | Загрузить локальный образ. | <code>*object</code> | <code>null</code> |
| <code>source.http</code> | Скачать образ из источника по HTTP. | <code>*object</code> | <code>null</code> |
| <code>source.http.url</code> | URL для скачивания образа. | <code>string</code> | <code>""</code> |
| <code>source.disk</code> | Клонировать существующий <code>vm-disk</code> . | <code>*object</code> | <code>null</code> |
| <code>source.disk.name</code> | Имя клонируемого <code>vm-disk</code> . | <code>string</code> | <code>""</code> |
| <code>optical</code> | Определяет, следует ли считать диск оптическим. | <code>bool</code> | <code>false</code> |

| | | | |
|--------------|---|----------|------------|
| storage | Размер диска, выделенного для виртуальной машины. | quantity | 5Gi |
| storageClass | StorageClass, используемый для хранения данных. | string | replicated |

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Создание и использование именованных образов VM

<https://cozystack.ru/docs/v1.6/virtualization/vm-image/>

Руководство по созданию, управлению и использованию золотых (именованных) образов VM в Cozystack для ускорения развёртывания виртуальных машин.

Золотые образы (golden images) в Cozystack позволяют администраторам подготовить именованные образы операционных систем, которые пользователи смогут впоследствии повторно использовать при создании виртуальных машин.

В этом руководстве объясняются преимущества золотых образов, способы их создания и использования при развёртывании VM.

По умолчанию каждый раз, когда пользователь создаёт виртуальную машину, Cozystack загружает требуемый образ из его источника по URL.

Это может стать узким местом, когда несколько VM создаются в быстрой последовательности.

Золотые образы решают эту проблему, кэшируя образ локально, что устраняет повторные загрузки и ускоряет развёртывание.

Соглашения об именовании (важно)

Cozystack автоматически добавляет префиксы к внутренним ресурсам Kubernetes:

| Имя, видимое пользователю | Resource Kind | Фактическое имя ресурса |
|---------------------------|--|---------------------------|
| <image> | DataVolume в cozy-public (золотой образ) | vm-default-images-<image> |
| <disk> | DataVolume, созданный из VMDisk | vm-disk-<disk> |
| <vm> | VirtualMachine, созданная из VMInstance | vm-instance-<vm> |

Это означает, что если вы создадите VMInstance с именем `ubuntu`, VirtualMachine в Kubernetes будет называться `vm-instance-ubuntu`.

Коллекция образов по умолчанию (подключаемый пакет)

Cozystack поставляется с необязательным пакетом `vm-default-images`, который предоставляет тщательно подобранную коллекцию готовых золотых образов (Ubuntu, Rocky Linux, AlmaLinux,

Debian, CentOS Stream, openSUSE, Alpine) в пространстве имён `cozy-public`. Пакет отключён по умолчанию и должен быть включён явно.

Storage requirements

Набор образов по умолчанию запрашивает примерно 320Gi хранилища (16 образов × 20Gi каждый). Перед включением сократите список образов или уменьшите размеры хранилища для каждого образа, чтобы они соответствовали ёмкости вашего кластера.

Коллекция по умолчанию включает:

| Имя образа | Описание |
|---------------------------------|------------------------------------|
| <code>ubuntu-20.04</code> | Ubuntu 20.04 LTS (Focal Fossa) |
| <code>ubuntu-22.04</code> | Ubuntu 22.04 LTS (Jammy Jellyfish) |
| <code>ubuntu-24.04</code> | Ubuntu 24.04 LTS (Noble Numbat) |
| <code>debian-12</code> | Debian 12 (Bookworm) |
| <code>debian-13</code> | Debian 13 (Trixie) |
| <code>rocky-8</code> | Rocky Linux 8 |
| <code>rocky-9</code> | Rocky Linux 9 |
| <code>rocky-10</code> | Rocky Linux 10 |
| <code>almalinux-8</code> | AlmaLinux 8 |
| <code>almalinux-9</code> | AlmaLinux 9 |
| <code>almalinux-10</code> | AlmaLinux 10 |
| <code>centos-stream-9</code> | CentOS Stream 9 |
| <code>centos-stream-10</code> | CentOS Stream 10 |
| <code>opensuse-leap-15.6</code> | openSUSE Leap 15.6 |
| <code>opensuse-leap-16.0</code> | openSUSE Leap 16.0 |
| <code>alpine-3.21</code> | Alpine Linux 3.21 |

Вы можете вывести список всех доступных образов с помощью:

```
kubectl -n cozy-public get dv -l app.kubernetes.io/managed-by=cozystack
```

Включение пакета

Добавьте `cozystack.vm-default-images` в `bundles.enabledPackages` в [Platform Package](#):

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
```

```
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozystack.vm-default-images"}]'
```

Подождите минуту, пока чарт платформы согласует состояние, затем проверьте HelmRelease и DataVolumes:

```
kubectl get helmrelease -n cozy-system vm-default-images
```

```
kubectl -n cozy-public get dv
```

DataVolumes, предоставляемые пакетом, именуются `vm-default-images-<image>` и доступны тенантам как золотые образы с именем `<image>` (например, `ubuntu-24.04`, `debian-12`).

Настройка списка образов

Переопределите список по умолчанию, отредактировав Package `cozystack.vm-default-images` и задав значения в `spec.components.vm-default-images.values`. Схема определена в файле [values.yaml](#) чарта:

- `storageClass` — StorageClass по умолчанию для всех образов; при пустом значении используется StorageClass кластера по умолчанию.
- `images[]` — список записей золотых образов. Каждая запись содержит:
 - `name` — имя образа в том виде, в котором оно доступно пользователям (например, `ubuntu-24.04`).
 - `url` — HTTP(S)-URL источника образа.
 - `storage` — выделяемый размер хранилища (например, `20Gi`).
 - `storageClass` — переопределение глобального StorageClass для конкретного образа.
 - `os.family`, `os.name`, `os.version`, `architecture`, `description` — необязательные метаданные, отображаемые в UI.

Пример: сократить список по умолчанию до двух образов и зафиксировать StorageClass:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.vm-default-images
spec:
variant: default
components:
vm-default-images:
values:
storageClass: replicated
images:
```

```
- name: ubuntu-24.04
url: https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
storage: 20Gi
os:
family: Linux
name: Ubuntu
version: "24.04"
architecture: amd64
description: "Ubuntu 24.04 LTS (Noble Numbat) cloud image"

- name: debian-12
url: https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-generic-amd64.qcow2
storage: 20Gi
os:
family: Linux
name: Debian
version: "12"
architecture: amd64
description: "Debian 12 (Bookworm) generic cloud image"
```

Чтобы удалить образ после установки пакета, уберите его из `images[]` и удалите осиротевший `DataVolume`:

```
kubect! -n cozy-public delete dv vm-default-images-<name>
```

Добавление пользовательских золотых образов

Для создания дополнительных именованных образов ВМ требуется учётная запись администратора в Cozystack.

Простейший способ добавить пользовательский образ — использовать CLI-скрипт.

Скрипт `[cdi_golden_image_create.sh](https://github.com/cozystack/cozystack/blob/v1.6.0/hack/cdi_golden_image_create.sh)` можно скачать из тега релиза Cozystack v1.6.0:

```
wget https://raw.githubusercontent.com/cozystack/cozystack/v1.6.0/hack/cdi_golden_image_create.sh
chmod +x cdi_golden_image_create.sh
```

Этот скрипт использует вашу конфигурацию `kubect!l`.

Перед его запуском убедитесь, что ваша конфигурация указывает на целевой кластер Cozystack.

Чтобы добавить пользовательский образ, запустите скрипт с именем образа и его URL:

```
cdi_golden_image_create.sh '<name>' 'https://<image-url>'
```

Например, чтобы добавить образ Talos:

```
cdi_golden_image_create.sh 'talos' 'https://github.com/siderolabs/talos/releases/download/v1.7.6/nocloud-amd64.raw.xz'
```

Внутри скрипт создаёт ресурс Kubernetes `kind: DataVolume` в пространстве имён `cozy-public`.

Имя ресурса — это имя образа с префиксом `vm-default-images-`.

Например, ресурс `vm-default-images-talos` создаёт образ, доступный как `talos`.

Вы можете отслеживать ход выполнения с помощью:

```
kubectl -n cozy-public get dv
```

```
kubectl -n cozy-public describe dv vm-default-images-talos
```

Использование золотых образов

Создание VMDisk из золотого образа

Чтобы использовать золотой образ в качестве источника для диска VM, создайте VMDisk с `source.image.name`, ссылающимся на имя образа:

```
kubectl -n tenant-root create -f- <<EOF
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
  name: ubuntu
spec:
  source:
    image:
      name: ubuntu-24.04
      storage: 20Gi
EOF
```

Вы можете отслеживать процесс с помощью следующих команд:

```
kubectl -n tenant-root get vmdisk
```

```
kubectl -n tenant-root get dv
```

```
kubectl -n tenant-root describe dv vm-disk-ubuntu
```

Подключение диска к ВМ

Далее создайте VMInstance, использующий этот диск:

```
kubectl -n tenant-root create -f- <<EOF
```

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: VMInstance
```

```
metadata:
```

```
name: ubuntu
```

```
spec:
```

```
disks:
```

```
  - name: ubuntu
```

```
EOF
```

Вы можете проверить статус VirtualMachine с помощью:

```
kubectl get vm -n tenant-root
```

Чтобы подключиться к ВМ, выполните:

```
virtctl console vm-instance-ubuntu -n tenant-root
```

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Клонируемые виртуальные машины

<https://cozystack.ru/docs/v1.6/virtualization/cloneable-vms/>

Создание клонируемых виртуальных машин

Чтобы создать клонируемую VM, вам потребуется создать `VMDisk` и `VMInstance`. В этом руководстве в качестве примера используется базовый образ `ubuntu`.

1. Создайте `VMDisk`

`apiVersion: apps.cozystack.io/v1alpha1`

`kind: VMDisk`

`metadata:`

`name: ubuntu-source`

`namespace: tenant-root`

`spec:`

`optical: false`

`source:`

`http:`

`url: https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img`

`storage: 20Gi`

`storageClass: replicated`

Поскольку расширение диска может быть сложной задачей, мы рекомендуем создавать его с запасом места для будущего роста.

2. Создайте `VMInstance`

Поскольку пользовательский ресурс `VirtualMachine` не предоставляет удобного способа работы с несколькими дисками, используйте вместо него `VMInstance`.

Создайте `VMInstance`, используя следующий шаблон:

`apiVersion: apps.cozystack.io/v1alpha1`

`kind: VMInstance`

`metadata:`

`name: sourcevm`

`namespace: tenant-root`

`spec:`

`externalMethod: PortList`

disks:

- name: ubuntu-source

externalPorts:

- 22

instanceProfile: ubuntu

instanceType: ""

running: true

sshKeys:

- <paste your ssh public key here>

external: true

resources:

cpu: "2"

memory: 4Gi

После создания VM она получит внешний IP-адрес балансировщика нагрузки. Вы можете получить его с помощью:

```
kubectl get svc -n tenant-root vm-instance-sourcevm
```

3. Подключитесь к VM по SSH

Теперь вы можете подключиться к VM по SSH, используя внешний IP-адрес. Пользователь по умолчанию для базового образа `ubuntu` — `ubuntu`.

```
ssh ubuntu@<external IP>
```

Настройте виртуальную машину перед клонированием.

4. Удалите VMInstance

Данные на диске будут сохранены.

```
kubectl delete vminstance -n tenant-root sourcevm
```

5. Создайте образ диска

apiVersion: cdi.kubevirt.io/v1beta1

kind: DataVolume

metadata:

name: "vm-image-sourcevm" # prefix vm-image is necessary

namespace: cozy-public # do not change

annotations:

cdi.kubevirt.io/storage.bind.immediate.requested: "true"

```
spec:
source:
pvc:
name: vm-disk-ubuntu-source
namespace: tenant-root
storage:
resources:
requests:
storage: 20Gi
storageClassName: replicated
```

Это займёт некоторое время. Подождите, прежде чем продолжить. Вы можете проверить ход выполнения с помощью:

```
kubectrl get datavolume -n cozy-public vm-image-sourcevm
```

Пример вывода, когда всё готово:

```
NAME PHASE PROGRESS RESTARTS AGE
vm-image-sourcevm Succeeded 100.0% 7m32s
```

6. Создайте VMDisk из клонированного образа

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
name: ubuntu-cloned-1
namespace: tenant-root
spec:
optical: false
source:
image:
name: sourcevm # image name without prefix
storage: 20Gi # size greater or equal to the disk image size
storageClass: replicated
```

7. Создайте VMInstance из клонированного диска

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
```

```
metadata:
name: cloned-vm
namespace: tenant-root
spec:
external: true
externalMethod: PortList
cloudInit: "hostname: my-cloned-vm"
cloudInitSeed: "1"
disks:
  - name: ubuntu-cloned-1
externalPorts:
  - 22
instanceProfile: ubuntu
running: true
```

Чтобы сетевые функции клонированной VM работали корректно, вы должны переопределить её `hostname` через `.spec.cloudInit` и указать уникальный `.spec.cloudInitSeed`, чтобы избежать конфликтов с исходной VM.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Запуск ВМ с пробросом GPU (passthrough)

<https://cozystack.ru/docs/v1.6/virtualization/gpu/>

Запуск ВМ с пробросом GPU (passthrough)

В этом разделе показано, как развёртывать виртуальные машины (ВМ) с пробросом GPU (passthrough) с помощью Cozystack. Сначала мы развернём GPU Operator, чтобы настроить рабочий узел для проброса GPU (passthrough). Затем мы развернём ВМ KubeVirt, запрашивающую GPU.

По умолчанию для обеспечения проброса GPU (passthrough) GPU Operator развёртывает следующие компоненты:

- VFIO Manager для привязки драйвера `vfio-pci` ко всем GPU на узле.
- Sandbox Device Plugin для обнаружения проброшенных GPU и их анонсирования kubelet.
- Sandbox Validator для проверки остальных операндов.

Предварительные требования

- Кластер Cozystack с хотя бы одним узлом с поддержкой GPU.
- Установленный `kubectl` и настроенные учётные данные для доступа к кластеру.

1\ Установка GPU Operator

Выполните следующие шаги:

1. Явно пометьте рабочий узел меткой для рабочих нагрузок с пробросом GPU (passthrough):

```
kubectl label node <node-name> --overwrite nvidia.com/gpu.workload.config=vm-passthrough
```

2. Включите GPU Operator в ваш Platform Package, добавив его в список включённых пакетов:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozystack.gpu-operator"}]'
```

Это развернёт компоненты (операнды).

3. Убедитесь, что все поды находятся в состоянии `running` и все проверки компонента `sandbox-validator` проходят успешно:

```
kubectl get pods -n cozy-gpu-operator
```

Пример вывода (имена ваших подов могут отличаться):

```
NAME READY STATUS RESTARTS AGE
```

```
...
```

```
nvidia-sandbox-device-plugin-daemonset-4mxsc 1/1 Running 0 40s
```

```
nvidia-sandbox-validator-vxj7t 1/1 Running 0 40s
```

```
nvidia-vfio-manager-thfwf 1/1 Running 0 78s
```

Чтобы проверить привязку GPU, получите доступ к узлу с помощью `kubectl node-shell -n cozy-system -x` или `kubectl debug node` и выполните:

```
lspci -nnk -d 10de:
```

Под `vfio-manager` привяжет все GPU на узле к драйверу `vfio-pci`. Пример вывода:

```
3b:00.0 3D controller [0302]: NVIDIA Corporation Device [10de:2236] (rev a1)
```

```
Subsystem: NVIDIA Corporation Device [10de:1482]
```

```
Kernel driver in use: vfio-pci
```

```
86:00.0 3D controller [0302]: NVIDIA Corporation Device [10de:2236] (rev a1)
```

```
Subsystem: NVIDIA Corporation Device [10de:1482]
```

```
Kernel driver in use: vfio-pci
```

`sandbox-device-plugin` обнаружит эти ресурсы и анонсирует их `kubelet`. В этом примере узел показывает два GPU A10 как доступные ресурсы:

```
kubectl describe node <node-name>
```

Пример вывода:

```
...
```

```
Capacity:
```

```
...
```

```
nvidia.com/GA102GL_A10: 2
```

```
...
```

```
Allocatable:
```

```
...
```

```
nvidia.com/GA102GL_A10: 2
```

```
...
```

Примечание: Имена ресурсов формируются путём объединения столбцов `device` и `device_name` из базы данных [PCI IDs](#). Например, запись в базе данных для A10 выглядит как `2236 GA102GL [A10]`, что приводит к имени ресурса `nvidia.com/GA102GL_A10`.

2\ KubeVirt is wired automatically

When `cozystack.gpu-operator` is in `bundles.enabledPackages`, Cozystack mirrors the chosen GPU variant into the KubeVirt Custom Resource for you. There is no `kubectl edit kubevirt` step.

Specifically, the platform injects:

- `HostDevices` into `spec.configuration.developerConfiguration.featureGates` (current KubeVirt splits this from the GPU gate; the admission webhook rejects `domain.devices.hostDevices` without it).
- A starter `spec.configuration.permittedHostDevices.pciHostDevices` table (rendered in the default `gpuOperatorVariant: default` — `vfio-pci passthrough`) covering common NVIDIA datacenter GPUs — Hopper (H100, H200), Ada Lovelace (L4, L40, L40S), Ampere (A100 PCIe/SXM, A40, A30, A10), Turing (T4), Volta (V100, V100S). PCI vendor:device pairs are stable; each `resourceName` slug is whatever `nvidia-sandbox-device-plugin` derives mechanically from the card's PCI-IDs database name — uppercase the name, turn `/`, `.` and whitespace into `_`, then strip the surrounding `[ / ]`. The slug therefore carries every token the PCI-IDs string holds (the `GL` die suffix, the `Tesla` brand on Turing/Volta, the form factor, the memory size), not a tidy `<arch>_<model>`: `TU104GL [Tesla T4]` becomes `nvidia.com/TU104GL_TESLA_T4`, `GA100GL [A30 PCIe]` becomes `nvidia.com/GA100GL_A30_PCIE`, and the H200 SXM becomes `nvidia.com/GH100_H200_SXM_141GB`. Confirm the exact strings your nodes advertise with `kubectl describe node <node> | grep nvidia.com/`. `externalResourceProvider: true` is set on every entry because the resources are advertised by the sandbox plugin, not by KubeVirt's in-tree device plugin.

Verify the resulting CR:

```
kubectl -n cozy-kubevirt get kubevirt kubevirt -o json \
```

| | | |
|--|--------------------------------------|---|
| | <code>jq '.spec.configuration</code> | <code>{featureGates: .developerCon
figuration.featureGates,
permittedHostDevices: .
permittedHostDevices}'</code> |
|--|--------------------------------------|---|

****My GPU isn't in the default table — where's the `oldkubectl edit kubevirt` step?**** It is gone on purpose. `permittedHostDevices` is now owned by the chart template and reconciled from platform values, so any hand edit to the live CR is reverted on the next Flux/Helm reconcile. Add your card through `.gpu.permittedHostDevices` instead — see [Extending or replacing the NVIDIA defaults](#) below. If you are upgrading from a release where you hand-edited the CR, follow [Upgrading from a hand-edited KubeVirt CR](#) first.

Extending or replacing the NVIDIA defaults

If your cluster ships a GPU not in the default table, or your `nvidia-sandbox-device-plugin` version emits a different `resourceName` (check with ``kubectl describe node <node>`

`grep nvidia.com/`), extend the defaults via platform values:`

Platform Package values

gpu:

Append (default) — your entries land alongside the NVIDIA table.

Set to true to drop the NVIDIA table entirely (useful for non-NVIDIA-only

clusters or strict allowlists). With `replaceDefaults: true` and an empty

list below, the rendered CR carries no `permittedHostDevices` block at all

and the admission webhook rejects every GPU VM — supply your own list.

`replaceDefaults: false`

`permittedHostDevices:`

`pciHostDevices:`

`- pciVendorSelector: "10DE:2236"`

`resourceName: nvidia.com/GA102GL_A10`

`externalResourceProvider: true`

To re-point a card already in the NVIDIA table (for example to give `10DE:1EB8` a different `resourceName`), do not append a second entry for the same `pciVendorSelector` — both entries are rendered and KubeVirt resolves the duplicated selector non-deterministically. Set `replaceDefaults: true` and supply the full list you want instead.

Upgrading from a hand-edited KubeVirt CR

Earlier Cozystack releases left `spec.configuration.permittedHostDevices` for operators to hand-edit (`kubectl edit kubevirt`). The bundle now owns that field: the first reconcile after the upgrade replaces your manual entries with the rendered NVIDIA default table.

Before upgrading:

1. Dump your current entries:

`kubectl -n cozy-kubevirt get kubevirt kubevirt -o json \`

`jq '.spec.configuration.permittedHostDevices'`

2. Move any custom entries into the Platform Package values under `.gpu.permittedHostDevices` (set `.gpu.replaceDefaults: true` if you want only your own list instead of appending to the NVIDIA defaults).

3. Verify every `resourceName` against what your nodes actually advertise. The default table carries the slug `nvidia-sandbox-device-plugin` generates from each card's PCI-IDs name (uppercased, e.g.

`nvidia.com/TU104GL_TESLA_T4` for a Tesla T4), but a different plugin build or PCI-IDs snapshot can emit a different string:

```
kubectl describe node <node>
```

```
grep nvidia.com/
```

A `resourceName` mismatch is silent until a GPU VM restarts or migrates, at which point the admission webhook rejects it.

Manual Package-CR override path

If you opt out of bundle management and hand-craft a `cozystack.gpu-operator` Package CR directly (to apply overrides the bundle does not expose — driver settings, custom node selectors, validator / `dcgmExporter` tweaks), the platform does NOT auto-wire `HostDevices` or `permittedHostDevices` into the KubeVirt CR. In that flow, mirror the bundle behaviour by also creating a `cozystack.kubevirt` Package CR that carries `extraFeatureGates` and the matching `permittedHostDevices` block under `spec.components.kubevirt.values` (a `cozystack` Package always nests component values under `spec.components.<name>.values`, never a top-level `spec.values`):

```
apiVersion: cozystack.io/v1alpha1
```

```
kind: Package
```

```
metadata:
```

```
name: cozystack.kubevirt
```

```
spec:
```

```
variant: default
```

```
components:
```

```
kubevirt:
```

```
values:
```

```
extraFeatureGates:
```

```
  - HostDevices
```

```
permittedHostDevices:
```

```
pciHostDevices:
```

```
  - pciVendorSelector: "10DE:2236"
```

```
resourceName: nvidia.com/GA102GL_A10
```

```
externalResourceProvider: true
```

The manual Package-CR override path takes precedence over the bundle render whenever both exist.

3\. Создание виртуальной машины

Теперь мы готовы создать ВМ.

1. Создайте тестовую виртуальную машину с помощью следующей спецификации VMI, которая запрашивает ресурс nvidia.com/GA102GL_A10.

vmi-gpu.yaml :

```
apiVersion: apps.cozystack.io/v1alpha1
appVersion: '*'
kind: VirtualMachine
metadata:
name: gpu
namespace: tenant-example
spec:
running: true
instanceProfile: ubuntu
instanceType: u1.medium
systemDisk:
image: ubuntu
storage: 5Gi
storageClass: replicated
gpus:
  - name: nvidia.com/GA102GL_A10
cloudInit: |
#cloud-config
password: ubuntu
chpasswd: { expire: False }

kubectl apply -f vmi-gpu.yaml
```

Пример вывода:

```
virtualmachines.apps.cozystack.io/gpu created
```

2. Проверьте статус VM:

```
kubectl get vmi
```

```
NAME AGE PHASE IP NODENAME READY
```

```
virtual-machine-gpu 73m Running 10.244.3.191 luc-csxhk-002 True
```

3. Войдите в VM и убедитесь, что у неё есть доступ к GPU:

```
virtctl console virtual-machine-gpu
```

Пример вывода:

```
Successfully connected to vmi-gpu console. The escape sequence is ^]
```

```
vmi-gpu login: ubuntu
```

```
Password:
```

```
ubuntu@virtual-machine-gpu:~$ lspci -nnk -d 10de:
```

```
08:00.0 3D controller [0302]: NVIDIA Corporation GA102GL [A10] [10de:26b9] (rev a1)
```

```
Subsystem: NVIDIA Corporation GA102GL [A10] [10de:1851]
```

```
Kernel driver in use: nvidia
```

```
Kernel modules: nvidiafb, nvidia_drm, nvidia
```

Совместное использование GPU виртуальными машинами

Проброс GPU (passthrough) назначает весь физический GPU одной ВМ. Чтобы разделить один GPU между несколькими ВМ, требуется NVIDIA vGPU.

vGPU (виртуальный GPU)

NVIDIA vGPU использует опосредованные устройства (mdev) для создания виртуальных GPU, назначаемых ВМ. Это единственное готовое к промышленной эксплуатации решение для совместного использования GPU между ВМ.

Требования:

- Лицензия NVIDIA vGPU (коммерческая, приобретается у NVIDIA)
- NVIDIA vGPU Manager, установленный на узлах-хостах

Почему не MIG? MIG (Multi-Instance GPU) разделяет GPU на изолированные экземпляры, но это логические разделы внутри одного устройства PCIe. VFIO не может пробросить их в ВМ — MIG работает только с контейнерами. Чтобы использовать MIG с ВМ, нужен vGPU поверх разделов MIG (всё равно требуется лицензия).

Open-Source vGPU (экспериментально)

NVIDIA разрабатывает поддержку vGPU с открытым исходным кодом для ядра Linux. После её включения это может обеспечить совместное использование GPU без лицензии.

- Статус: стадия RFC, не включено в основную ветку ядра
- Поддерживает Ada Lovelace и новее (L4, L40 и т. д.)

- Ссылки: анонс [Phoronix](#), патчи ядра

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Резервное копирование и восстановление

<https://cozystack.ru/docs/v1.6/virtualization/backup-and-recovery/>

Как создавать резервные копии ресурсов VMInstance и VMDisk и управлять ими с помощью BackupJob и Plan.

Стратегии резервного копирования кластера и BackupClass настраиваются администраторами кластера. Если для вашего тенанта ещё нет BackupClass, попросите администратора выполнить руководство [Backup Classes](#), чтобы настроить хранилище, стратегии и BackupClass.

В этом руководстве описывается резервное копирование и восстановление ресурсов VMInstance и VMDisk от лица пользователя-тенанта: запуск разовых и плановых резервных копий, проверка их статуса и восстановление из резервной копии с помощью RestoreJob.

Cozystack использует [Velero](#) под капотом для хранения резервных копий и снимков (snapshot) томов.

В этом руководстве рассматривается резервное копирование BM (HelmRelease + CR + снимки PVC, упакованные Velero). Резервное копирование только данных управляемых баз данных (Postgres, MariaDB, ClickHouse, FoundationDB) описано в разделе [Application Backup and Recovery](#).

Предварительные требования

- Дополнение Velero включено для вашего кластера (администратором).
- Для пространства имён вашего тенанта доступен как минимум один BackupClass (предоставлен администратором).
- `kubectl` и `kubeconfig` для кластера, резервную копию которого вы создаёте.

Список доступных BackupClass

BackupClass определяют, где и как хранятся резервные копии. Вы можете использовать только те, что созданы администраторами.

```
kubectl get backupclasses
```

Пример вывода:

```
NAME AGE
cozy-default 14m
```

Используйте имя BackupClass при создании BackupJob или Plan.

`cozy-default` is the platform-shipped `BackupClass`; its `strategies[]` array binds the Velero driver for both `VMInstance` and `VMDisk`. Use this name when creating a `BackupJob` or `Plan`, or substitute a sibling class name if your administrator has created one.

Fresh-cluster bootstrap window. On a fresh-cluster install, the Velero `BackupStorageLocation` `cozy-default` reports `Unavailable` for tens of seconds after `helm install` returns, until the platform's credentials projector lands `cozy-backups-creds` into `cozy-velero`. Velero rejects new `Backup` and `Restore` requests against `storageLocation: cozy-default` during that window. If a `BackupJob` you submit fails immediately with a Velero error referencing storage, wait and retry, or ask your administrator to check that `kubectl -n cozy-velero get bsl cozy-default -o jsonpath='{.status.phase}'` returns `Available`. See the [Backup Classes admin guide](#) for details.

Резервное копирование VMInstance

Резервная копия `VMInstance` захватывает конфигурацию ВМ и все подключённые тома `VMDisk`.

Разовая резервная копия

Используйте `BackupJob`, когда нужно выполнить резервное копирование один раз — например, перед рискованным изменением.

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: BackupJob
```

```
metadata:
```

```
name: my-vm-backup
```

```
namespace: tenant-user
```

```
spec:
```

```
applicationRef:
```

```
apiGroup: apps.cozystack.io
```

```
kind: VMInstance
```

```
name: my-vm
```

```
backupClassName: cozy-default
```

Примените его и следите за статусом:

```
kubectl apply -f backupjob.yaml
```

```
kubectl get backupjobs -n tenant-user
```

```
kubectl describe backupjob my-vm-backup -n tenant-user
```

Когда `BackupJob` завершается успешно, он создаёт объект `Backup` с тем же именем (`my-vm-backup`). Это имя вы будете использовать при восстановлении.

Плановая резервная копия

Используйте Plan , чтобы выполнять резервное копирование по расписанию.

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: Plan
```

```
metadata:
```

```
name: my-vm-daily
```

```
namespace: tenant-user
```

```
spec:
```

```
applicationRef:
```

```
apiGroup: apps.cozystack.io
```

```
kind: VMInstance
```

```
name: my-vm
```

```
backupClassName: cozy-default
```

```
schedule:
```

```
cron: "0 2 *" # Каждый день в 02:00
```

Примените его и проверьте:

```
kubectl apply -f plan.yaml
```

```
kubectl get plans -n tenant-user
```

```
kubectl describe plan my-vm-daily -n tenant-user
```

Каждый плановый запуск создаёт BackupJob (а при успехе — объект Backup), названный по имени Plan с суффиксом-временной меткой.

Резервное копирование VMDisk

Вы можете создать резервную копию VMDisk независимо — например, чтобы захватить конкретный диск без конфигурации VM.

BackupClass должен включать стратегию для VMDisk. Попросите администратора добавить её, если она отсутствует (см. [Backup Classes](#)).

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: BackupJob
```

```
metadata:
```

```
name: my-disk-backup
```

```
namespace: tenant-user
```

```
spec:
applicationRef:
apiGroup: apps.cozystack.io
kind: VMDisk
name: my-disk
backupClassName: cozy-default
```

Примените и проверьте статус:

```
kubectl apply -f backupjob-disk.yaml
kubectl get backupjobs -n tenant-user
kubectl describe backupjob my-disk-backup -n tenant-user
```

Проверка статуса резервного копирования

Список всех BackupJob в пространстве имён:

```
kubectl get backupjobs -n tenant-user
```

Опишите конкретный BackupJob, чтобы увидеть фазу и любые ошибки:

```
kubectl describe backupjob my-vm-backup -n tenant-user
```

Список созданных объектов Backup (по одному на каждый завершённый BackupJob):

```
kubectl get backups -n tenant-user
```

Список BackupJob, созданных некоторым Plan:

```
kubectl get backupjobs -n tenant-user -l backups.cozystack.io/plan=my-vm-daily
```

Восстановление VMInstance

Вы можете восстановить VMInstance как на месте (откат работающей VM), так и с нуля (после того как VM и её диски были удалены). Резервная копия VMInstance включает все подключённые тома VMDisk и их данные.

Сначала найдите объект Backup, из которого хотите восстановиться:

```
kubectl get backups -n tenant-user
```

Пример вывода:

NAME AGE

my-vm-backup 2h

Создайте RestoreJob со ссылкой на этот Backup:

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: RestoreJob
```

```
metadata:
```

```
name: restore-my-vm
```

```
namespace: tenant-user
```

```
spec:
```

```
backupRef:
```

```
name: my-vm-backup
```

Примените его и проверьте ход выполнения:

```
kubectl apply -f restorejob.yaml
```

```
kubectl get restorejobs -n tenant-user
```

```
kubectl describe restorejob restore-my-vm -n tenant-user
```

Контроллер восстановления автоматически подготавливает окружение пошагово:

- Приостанавливает HelmRelease для VMInstance и её VMDisk, чтобы Flux не вмешивался.
- Останавливает VM (устанавливает `runStrategy=Halted`) и ждёт корректного завершения работы VMI.
- Переименовывает существующие PVC в `<name>-orig-<hash>`, чтобы Velero мог создать новые из резервной копии. Старые PVC сохраняются на случай, если вам нужно будет их изучить.
- Удаляет DataVolume, чтобы CDI не пересоздавал PVC после переименования.

RestoreJob проходит через `Pending` -> `Running` -> `Succeeded` (или `Failed`). При успехе VMInstance и её VMDisk восстанавливаются до состояния, захваченного в резервной копии.

Проверка после восстановления

После успешного завершения RestoreJob убедитесь, что VM действительно работает:

Проверьте, что VMInstance и VMDisk находятся в состоянии Ready

```
kubectl get vminstances,vmdisks -n tenant-user
```

Проверьте, что VirtualMachineInstance запущен (а не только CR)

```
kubectl get vmi -n tenant-user
```

Проверьте доступ по SSH

```
virtctl ssh -i ~/.ssh/my-key -l ubuntu vmi/vm-instance-my-vm -n tenant-user -c "ip a"
```

После того как вы убедились, что восстановление прошло успешно и VM работает корректно, вы можете удалить PVC `*-orig-*`, содержащие исходные данные, чтобы освободить хранилище:

Не удаляйте PVC `*-orig-*`, пока не убедитесь, что восстановленная VM полностью работоспособна. Они — ваш последний резерв для ручного восстановления, если во время восстановления что-то пошло не так.

Список PVC -orig

```
kubectrl get pvc -n tenant-user
```

```
grep -- '-orig-'
```

Удалите их, когда они больше не нужны (замените на фактические имена PVC из списка выше)

```
kubectrl delete pvc -n tenant-user <name>-orig-<hash>
```

Восстановление VMDisk на месте

Чтобы восстановить только VMDisk, не затрагивая конфигурацию VM.

Velero пропускает существующий DataVolume во время восстановления. Чтобы восстановить фактическое содержимое диска из резервной копии, сначала удалите DataVolume:

```
kubectrl delete datavolume vm-disk-my-disk -n tenant-user
```

Затем RestoreJob пересоздаст его и загрузит данные диска из хранилища резервных копий.

```
kubectrl get backups -n tenant-user
```

```
apiVersion: backups.cozystack.io/v1alpha1
```

```
kind: RestoreJob
```

```
metadata:
```

```
name: restore-my-disk
```

```
namespace: tenant-user
```

```
spec:
```

```
backupRef:
```

```
name: my-disk-backup
```

Примените и проверьте:

```
kubectrl apply -f restorejob-disk.yaml
```

```
kubectrl get restorejobs -n tenant-user
```

```
kubectl describe restorejob restore-my-disk -n tenant-user
```

Устранение неполадок

Если BackupJob или RestoreJob завершается в фазе `Failed`, проверьте поле `message` в его статусе:

```
kubectl get backupjob my-vm-backup -n tenant-user -o jsonpath='{.status.message}'
```

```
kubectl get restorejob restore-my-vm -n tenant-user -o jsonpath='{.status.message}'
```

Для получения деталей более низкого уровня проверьте логи Velero в управляющем кластере:

```
kubectl logs -n cozy-velero -l app.kubernetes.io/name=velero --tail=100
```

Исправление доступа по SSH (несоответствие MAC-адреса cloud-init после восстановления)

Контроллер восстановления автоматически пытается сохранить OVN IP и MAC-адреса ВМ на момент резервного копирования, патча восстановленный VirtualMachine исходными аннотациями. Если это не удаётся, вы можете вручную получить исходные IP/MAC из `Backup . status.underlyingResources` и назначить их.

После восстановления VMInstance гостевая ОС может потерять сетевое соединение. Известно, что это происходит на Ubuntu Server , где cloud-init генерирует конфигурацию netplan, привязанную к MAC-адресу старой ВМ. После восстановления ВМ получает новый виртуальный NIC с другим MAC-адресом, но в гостевой ОС всё ещё остаётся старая конфигурация netplan, привязанная к предыдущему MAC — поэтому сетевой интерфейс так и не настраивается. Другие операционные системы, которые не привязывают сетевую конфигурацию к конкретному MAC-адресу, могут не быть подвержены этой проблеме.

Чтобы это исправить, обновите поле `cloudInitSeed` в спецификации VMInstance и перезапустите ВМ. Изменение seed генерирует новый SMBIOS UUID, из-за чего cloud-init воспринимает ВМ как новый экземпляр и заново выполняет сетевую настройку с правильным MAC-адресом.

Задайте новое значение cloudInitSeed (любая строка, отличная от текущей)

```
kubectl patch vminstance my-vm -n tenant-user --type merge \
-p '{"spec":{"cloudInitSeed":"reseed1"}}'
```

Дождитесь согласования (reconcile) VMInstance

```
kubectl wait vminstance/my-vm -n tenant-user --for=condition=Ready --timeout=180s
```

Перезапустите ВМ, чтобы новый seed вступил в силу

```
virtctl restart vm-instance-my-vm -n tenant-user
```

После перезапуска убедитесь, что у ВМ есть сетевое соединение:

Проверьте, что VMI запущен

```
kubectl get vmi -n tenant-user
```

Проверьте доступ по SSH

```
virtctl ssh -i ~/.ssh/my-key -l ubuntu vmi/vm-instance-my-vm -n tenant-user -c "ip a"
```

Если в будущем вам понадобится снова изменить seed (например, после очередного восстановления), используйте каждый раз другое значение (например, [reseed2](#), [reseed3](#) и т. д.).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Запуск виртуальных машин Windows в Cozystack

<https://cozystack.ru/docs/v1.6/virtualization/windows/>

Запуск виртуальных машин Windows в Cozystack

Cozystack может запускать виртуальные машины Windows. В этом руководстве описаны предварительные требования и шаги, необходимые для загрузки виртуальной машины с ОС Windows.

Предварительные требования

- ISO-образ установки Windows.
- ISO-образ драйверов Virtio.
- Клиент KubeVirt `virtctl` установленный в вашем локальном окружении и настроенный для пространства имён вашего тенанта.
- Cozystack версии v0.34.2 или новее.

Установка

Создание виртуальной машины с ОС Windows начинается с создания объектов `VMDisk` и продолжается созданием `VMInstance`.

1\ Создание объектов `VMDisk`

Вам понадобится три диска :

1. Установочный ISO – оптический.
2. ISO с драйверами Virtio – оптический.
3. Системный диск – не оптический.

В следующем примере используются минимально рекомендованные тома хранилища.

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: VMDisk
```

```
metadata:
```

```
name: win2k25-iso
```

```
spec:
```

```
source:
```

http:

url: https://software-static.download.prss.microsoft.com/dbazure/888969d5-f34g-4e03-ac9d-1f9786c66749/26100.1742.240906-0331.ge_release_svc_refresh_SERVER_EVAL_x64FRE_en-us.iso

optical: true

storage: 6Gi

storageClass: replicated

apiVersion: apps.cozystack.io/v1alpha1

kind: VMDisk

metadata:

name: virtio-drivers

spec:

source:

http:

url: <https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso>

optical: true

storage: 1Gi

storageClass: replicated

apiVersion: apps.cozystack.io/v1alpha1

kind: VMDisk

metadata:

name: win2k25-system

spec:

optical: false

storage: 50Gi

storageClass: replicated

2\ Создание VMInstance

Выберите профиль инстанса с поддержкой Virtio и подключите пустой системный диск. Выберите из доступных профилей Virtio:

windows.10.virtio

windows.11.virtio

windows.2k16.virtio

windows.2k19.virtio

windows.2k22.virtio

windows.2k25.virtio

Создайте объект `VMInstance`, как показано в этом примере:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
  name: win2k25-demo
spec:
  running: true
  instanceType: "u1.xlarge"
  instanceProfile: windows.2k25.virtio # выбран из списка выше
  disks:
    - name: win2k25-system
    - name: win2k25-iso
    - name: virtio-drivers
```

3\. Установка Windows

1. Откройте консоль с помощью клиента `virtctl`:

```
virtctl vnc vm-instance-win2k25-demo
```

2. Продолжите стандартную установку Windows.

3. Когда появится запрос “Where do you want to install Windows?” (Куда вы хотите установить Windows?), выберите Load driver , затем перейдите к CD-ROM с Virtio, например `E:\viosstor\amd64\`.

4. После появления виртуального диска продолжите установку и дайте Windows перезагрузиться.

5. После первой перезагрузки отключите установочный диск Windows (`win2k25-iso`) и драйверы Virtio (`virtio-drivers`) от `VMInstance`.

Преобразование существующего образа Windows

Если у вас уже есть диск Windows, созданный на VMware, Hyper-V или в другом облаке, вы можете воспользоваться этим путём, чтобы подготовить его к работе с Virtio в Cozystack.

1\. Создание фиктивного VMDisk для драйвера Virtio

Создайте фиктивный VMDisk, который будет использоваться для установки драйвера Virtio:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMDisk
metadata:
name: dummy-disk-for-virtio
spec:
optical: false
storage: "1Gi"
storageClass: "replicated"
```

2\ Запуск с диском и не-Virtio шиной

При создании `VMInstance` подключите системный диск с шиной `sata`, а фиктивный диск — с неуказанной шиной — тогда для диска по умолчанию будет использована шина Virtio SCSI. Вы также можете одновременно смонтировать Virtio ISO, чтобы упростить установку драйвера.

```
spec:
instanceProfile: windows.2k25.virtio
disks:
  - name: win2k19-system
bus: sata
  - name: dummy-disk-for-virtio
  - name: virtio-drivers
bus: sata
```

3\ Установка драйверов хранилища Virtio

Выполните следующие шаги для установки драйверов Virtio:

1. Смонтируйте `virtio-win.iso` внутри гостевой системы.
2. Запустите мастер установки для установки драйверов
3. Убедитесь, что установка драйвера прошла успешно:
 1. Откройте Диспетчер устройств (Device Manager)
 2. Вы должны увидеть устройство SCSI в списке с иконкой восклицательного знака рядом с ним.
 3. Если драйвер не установлен, щёлкните правой кнопкой мыши по устройству и выберите Update Driver.
 4. Выберите Install the hardware that I manually select from a list , затем Show All Devices , затем Next.
 5. Нажмите Have Disk... , перейдите к файлу `.inf` и завершите работу мастера.

В качестве альтернативы щёлкните правой кнопкой мыши по файлу `.inf` в Проводнике и выберите Install.

4\ Переключение на шину Virtio

После установки драйверов вам нужно переключиться на шину Virtio. Выполните следующие шаги:

1. Выключите VM.

2. Отредактируйте `VMInstance` и удалите строку `bus: sata` из системного диска, а также фиктивный диск:

spec:

disks:

- name: win2k19-system

bus: sata

#- name: dummy-disk-for-virtio

3. Примените манифест и снова включите VM. Windows должна нормально загрузиться с использованием Virtio.

4. Теперь фиктивный диск можно удалить:

```
kubectl delete vmdisk dummy-disk-for-virtio
```

Соображения по поводу MTU сети

Cozystack устанавливает размер MTU равным 1400 на каждом vNIC. Windows корректно определяет это только при использовании VirtioNet. С устаревшими сетевыми драйверами вы можете столкнуться с потерей пакетов.

Чтобы заставить Windows учитывать MTU 1400, выполните следующие команды в PowerShell:

```
# List interfaces
```

```
Get-NetIPInterface
```

```
# Set MTU permanently
```

```
Set-NetIPInterface -InterfaceAlias "Ethernet Instance 0" -NIMtuBytes 1400
```

Настоятельно рекомендуется использовать профиль Virtio.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Запуск MikroTik RouterOS в Cozystack

<https://cozystack.ru/docs/v1.6/virtualization/mikrotik/>

Развёртывание MikroTik RouterOS (CHR) как виртуального устройства в Cozystack

Предварительные требования

- ISO-образ MikroTik RouterOS (установочный образ CHR или NPK), например, `mikrotik-7.19.3.iso`.
- Свободный статический IP-адрес или DHCP в подключённой сети тенанта.
- Клиент KubeVirt `virtctl`, установленный в вашей локальной среде и настроенный для пространства имён вашего тенанта.
- Cozystack версии v0.34.2 или новее.

Установка

1\ Подготовка дисков

Вам нужно два диска :

1. Установочный ISO – оптический.
2. Системный диск – неоптический.

apiVersion: apps.cozystack.io/v1alpha1

kind: VMDisk

metadata:

name: mikrotik-iso

spec:

source:

http:

url: <https://download.mikrotik.com/routeros/7.19.3/mikrotik-7.19.3.iso>

optical: true

storage: 1Gi

storageClass: replicated

apiVersion: apps.cozystack.io/v1alpha1

kind: VMDisk

```
metadata:
name: mikrotik-system
spec:
optical: false
storage: 1Gi
storageClass: replicated
```

2\ Создание VMInstance

RouterOS не требует специального профиля экземпляра. Используйте лёгкий профиль Linux, такой как `ubuntu`, с небольшим типом экземпляра, например `u1.medium`:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
name: mikrotik-demo
spec:
running: true
instanceType: "u1.medium"
instanceProfile: ubuntu
disks:
  - name: mikrotik-system
    bus: sata
  - name: mikrotik-iso
    bus: sata
```

3\ Установка RouterOS

1. Запустите консоль:

```
virtctl vnc mikrotik-demo -n tenant-test
```

2. При запросе выбора пакетов выберите нужный набор (обычно `_system_`, `_routing_`, `_security_`). Подтвердите форматирование системного диска.

3. После завершения установки извлеките установочный ISO.

4\ Настройка MTU (необязательно)

Виртуальные сетевые интерфейсы Cozystack по умолчанию используют MTU 1400. RouterOS учитывает это автоматически на адаптерах Virtio?Net, но вы можете проверить или изменить значение:

```
/interface ethernet print detail
```

```
/interface ethernet set [find default-name~"ether1"] mtu=1400
```

Избегайте устаревших драйверов `e1000/vmxnet`, поскольку они игнорируют значения MTU, отличные от 1500, и могут отбрасывать большие пакеты.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Миграция виртуальных машин из Proxmox

<https://cozystack.ru/docs/v1.6/virtualization/proxmox-migration/>

Пошаговое руководство по миграции виртуальных машин из Proxmox VE в Cozystack

В этом руководстве описывается процесс миграции виртуальных машин из Proxmox VE в Cozystack путём экспорта образов дисков VM и их загрузки в целевую среду.

Примечание: Миграция выполняется путём экспорта дисков VM в файлы и их загрузки в Cozystack. Состояние VM и снимки (snapshot) при миграции не сохраняются.

Предварительные требования

Перед началом миграции убедитесь, что у вас есть:

1. **Клиент KubeVirt** `virtctl`, установленный на вашей локальной машине:

- Руководство по установке: [KubeVirt User Guide - Virtctl Client Tool](#)

2. Настроенный доступ к прокси загрузки в вашем кластере Cozystack:

- Примените патч к Platform Package, чтобы открыть `cdi-uploadproxy`:

```
kubectrl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "
cdi-uploadproxy"}]'
```

- Настройте конечную точку прокси загрузки CDI, применив патч к Package `kubevirt-cdi`:

```
kubectrl patch packages.cozystack.io cozystack.kubevirt-cdi --type=merge -p '{
"spec": {
"components": {
"kubevirt-cdi": {
"values": {
"uploadProxyURL": "https://cdi-uploadproxy.example.org"
}
}
}
}'
```

3. Настройка DNS или файла `hosts` для доступа к прокси загрузки:

- При необходимости добавьте запись в `/etc/hosts` на вашей локальной машине:

<UPLOAD_PROXY_IP> cdi-uploadproxy.example.org

Шаг 1: Экспорт дисков VM из Proxmox

Перед экспортом убедитесь, что виртуальные машины остановлены в Proxmox.

Экспортируйте диск VM в файл в формате qcow2 (или другом формате, поддерживаемом KubeVirt):

Пример: Экспорт диска VM из хранилища Proxmox

```
qm disk export <vmid> <disk> /tmp/vm-disk.qcow2
```

Результатом должен стать файл образа диска (например, `vm-disk.qcow2`), готовый к загрузке.

Примечание: Конкретные команды для экспорта дисков могут различаться в зависимости от бэкенда хранилища Proxmox и его конфигурации. Подробнее см. в [документации Proxmox VE](#).

Шаг 2: Создание VMDisk для загрузки

Создайте ресурс `VMDisk` в Cozystack с `source.upload`, чтобы подготовиться к загрузке образа:

```
apiVersion: apps.cozystack.io/v1alpha1
```

```
kind: VMDisk
```

```
metadata:
```

```
name: proxmox-vm-disk
```

```
namespace: tenant-root
```

```
spec:
```

```
source:
```

```
upload: {}
```

```
storage: 10Gi
```

```
storageClass: replicated
```

Примените манифест:

```
kubectl apply -f vmdisk-upload.yaml
```

Отслеживайте статус создания диска:

```
kubectl get vmdisk -n tenant-root
```

```
kubectl describe vmdisk proxmox-vm-disk -n tenant-root
```

Шаг 3: Загрузка образа диска

Как только VMDisk создан и готов к загрузке, используйте `virtctl` для загрузки образа диска:

```
virtctl image-upload dv vm-disk-proxmox-vm-disk \
-n tenant-root \
--image-path=./vm-disk.qcow2 \
--uploadproxy-url https://cdi-uploadproxy.example.org \
--insecure
```

Примечание: Имя DataVolume следует шаблону `vm-disk-<vmdisk-name>`. Если ваш VMDisk называется `proxmox-vm-disk`, то DataVolume будет `vm-disk-proxmox-vm-disk`.

Дождитесь завершения загрузки. Вы можете отслеживать прогресс:

```
kubectl get dv -n tenant-root
kubectl describe dv vm-disk-proxmox-vm-disk -n tenant-root
```

Загрузка завершена, когда статус показывает `Succeeded`.

Шаг 4: Создание VMInstance

После завершения загрузки диска создайте VMInstance для загрузки с загруженного диска:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VMInstance
metadata:
  name: migrated-vm
  namespace: tenant-root
spec:
  running: true
  instanceType: u1.medium
  disks:
    - name: proxmox-vm-disk
# Optional: configure network, cloud-init, etc.
```

Примените манифест:

```
kubectl apply -f vminstance.yaml
```

Убедитесь, что VM запущена:

```
kubectl get vm -n tenant-root
```

```
kubectl get vmi -n tenant-root
```

Шаг 5: Доступ к мигрированной VM

Получите доступ к консоли VM с помощью `virtctl`:

```
# Serial console
```

```
virtctl console vm-instance-migrated-vm -n tenant-root
```

```
# VNC access
```

```
virtctl vnc vm-instance-migrated-vm -n tenant-root
```

```
# SSH (if configured)
```

```
virtctl ssh user@vm-instance-migrated-vm -n tenant-root
```

Контрольный список миграции

Используйте этот контрольный список для отслеживания прогресса миграции:

- Экспортировать диски VM из Proxmox (в формате qcow2 или совместимом)
- Установить `virtctl` на вашей локальной машине
- Настроить доступ к прокси загрузки в Cozystack
- Добавить запись DNS/hosts для прокси загрузки (при необходимости)
- Создать VMDisk с `source.upload` в Cozystack
- Загрузить образ диска с помощью `virtctl image-upload`
- Дождаться завершения загрузки (статус: Succeeded)
- Создать VMInstance с загруженным диском
- Убедиться, что VM успешно загружается
- Проверить сетевую связность и работоспособность VM

Устранение неполадок

Загрузка завершается с ошибкой подключения

Проблема: `virtctl image-upload` завершается с ошибкой `connection refused` или тайм-аутом.

Решение:

-
- Убедитесь, что прокси загрузки доступен: `curl -k https://cdi-uploadproxy.example.org`
 - Проверьте, что запись в `/etc/hosts` соответствует IP-адресу прокси загрузки
 - Убедитесь, что Platform Package включает `cdi-uploadproxy` в `publishing.exposedServices`

Загрузка застряла на 0%

Проблема: Загрузка начинается, но не продвигается.

Решение:

- Проверьте статус DataVolume: `kubectl describe dv vm-disk-<name> -n tenant-root`
- Убедитесь, что у класса хранилища есть доступная ёмкость
- Проверьте логи пода CDI: `kubectl logs -n cozy-system -l app=cdi-uploadproxy`

ВМ не загружается после миграции

Проблема: ВМ загружается, но не запускается корректно.

Решение:

- Проверьте, что диск ВМ подключён как первый диск в `spec.VMInstance`
- Убедитесь, что формат диска совместим (`qcow2`, `raw`)
- Просмотрите логи ВМ: `virtctl console vm-instance-<name> -n tenant-root`
- Убедитесь, что драйверы ВМ совместимы с KubeVirt (рекомендуется `VirtIO`)

Дальнейшие шаги

После успешной миграции:

- Настройте [cloud-init](#) для автоматизированной настройки ВМ
- Изучите [типы инстансов и профили](#) для оптимального распределения ресурсов
- Рассмотрите возможность создания [золотых образов](#) для будущих развёртываний ВМ

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Ресурсы виртуальных машин

<https://cozystack.ru/docs/v1.6/virtualization/resources/>

Справочник по типам инстансов и профилям инстансов VM

У каждой виртуальной машины есть две следующие настройки конфигурации:

- `instanceType` определяет ресурсы, предоставляемые виртуальной машине.
- `instanceProfile` определяет набор предпочтений для виртуальных машин в соответствии с используемой ОС.

Ресурсы типа инстанса

Справочная таблица

Следующие ресурсы `instancetype` предоставляются Cozystack:

| Имя | vCPUs | память |
|-------------|-------|--------|
| cx1.2xlarge | 8 | 16Gi |
| cx1.4xlarge | 16 | 32Gi |
| cx1.8xlarge | 32 | 64Gi |
| cx1.large | 2 | 4Gi |
| cx1.medium | 1 | 2Gi |
| cx1.xlarge | 4 | 8Gi |
| gn1.2xlarge | 8 | 32Gi |
| gn1.4xlarge | 16 | 64Gi |
| gn1.8xlarge | 32 | 128Gi |
| gn1.xlarge | 4 | 16Gi |
| m1.2xlarge | 8 | 64Gi |
| m1.4xlarge | 16 | 128Gi |
| m1.8xlarge | 32 | 256Gi |
| m1.large | 2 | 16Gi |
| m1.xlarge | 4 | 32Gi |
| n1.2xlarge | 16 | 32Gi |

| | | |
|-------------|----|-------|
| n1.4xlarge | 32 | 64Gi |
| n1.8xlarge | 64 | 128Gi |
| n1.large | 4 | 8Gi |
| n1.medium | 4 | 4Gi |
| n1.xlarge | 8 | 16Gi |
| o1.2xlarge | 8 | 32Gi |
| o1.4xlarge | 16 | 64Gi |
| o1.8xlarge | 32 | 128Gi |
| o1.large | 2 | 8Gi |
| o1.medium | 1 | 4Gi |
| o1.micro | 1 | 1Gi |
| o1.nano | 1 | 512Mi |
| o1.small | 1 | 2Gi |
| o1.xlarge | 4 | 16Gi |
| rt1.2xlarge | 8 | 32Gi |
| rt1.4xlarge | 16 | 64Gi |
| rt1.8xlarge | 32 | 128Gi |
| rt1.large | 2 | 8Gi |
| rt1.medium | 1 | 4Gi |
| rt1.micro | 1 | 1Gi |
| rt1.small | 1 | 2Gi |
| rt1.xlarge | 4 | 16Gi |
| u1.2xlarge | 8 | 32Gi |
| u1.2xmedium | 2 | 4Gi |
| u1.4xlarge | 16 | 64Gi |
| u1.8xlarge | 32 | 128Gi |
| u1.large | 2 | 8Gi |
| u1.medium | 1 | 4Gi |
| u1.micro | 1 | 1Gi |
| u1.nano | 1 | 512Mi |

| | | |
|-----------|---|------|
| u1.small | 1 | 2Gi |
| u1.xlarge | 4 | 16Gi |

Серия U

Серия U достаточно нейтральна и предоставляет ресурсы для приложений общего назначения.

U — это сокращение от «Universal» (универсальный), намекающее на универсальное отношение к рабочим нагрузкам.

ВМ этих типов инстансов будут совместно использовать физические ядра CPU с другими ВМ на основе разделения по времени.

Особые характеристики этой серии:

- _Burstable-производительность CPU_ — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- _Соотношение vCPU к памяти (1:4)_ — соотношение vCPU к памяти 1:4 для меньшего уровня помех на узел.

Серия O

Серия O основана на серии U, единственное отличие заключается в том, что память переподписана (overcommitted).

O — это сокращение от «Overcommitted» (переподписанный).

Особые характеристики этой серии:

- _Burstable-производительность CPU_ — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- _Переподписанная память_ — память переподписана для достижения более высокой плотности рабочих нагрузок.
- _Соотношение vCPU к памяти (1:4)_ — соотношение vCPU к памяти 1:4 для меньшего уровня помех на узел.

Серия CX

Серия CX предоставляет эксклюзивные вычислительные ресурсы для вычислительно-интенсивных приложений.

CX — это сокращение от «Compute Exclusive» (эксклюзивные вычисления).

Эксклюзивные ресурсы предоставляются вычислительным потокам VM. Чтобы это обеспечить, будут запрошены некоторые дополнительные ядра (в зависимости от количества дисков и NIC) для разгрузки потоков ввода-вывода с ядер, выделенных под рабочую нагрузку. Кроме того, в этой серии топология NUMA используемых ядер предоставляется VM.

Особые характеристики этой серии:

- `_Hugepages_` — Hugepages используются для повышения производительности памяти.
- `_Выделенный CPU_` — физические ядра эксклюзивно назначаются каждому vCPU, чтобы обеспечить фиксированные и высокие вычислительные гарантии для рабочей нагрузки.
- `_Изолированные потоки эмулятора_` — потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.
- `_vNUMA_` — физическая топология NUMA отражается в гостевой системе для оптимизации использования кэша на стороне гостя.
- `_Соотношение vCPU к памяти (1:2)_` — соотношение vCPU к памяти 1:2.

Серия M

Серия M предоставляет ресурсы для приложений, интенсивно использующих память.

`_M_` — это сокращение от «Memory» (память).

Особые характеристики этой серии:

- `_Hugepages_` — Hugepages используются для повышения производительности памяти.
- `_Burstable-производительность CPU_` — у рабочей нагрузки есть базовая вычислительная производительность, но ей разрешено превышать этот базовый уровень, если доступны избыточные вычислительные ресурсы.
- `_Соотношение vCPU к памяти (1:8)_` — соотношение vCPU к памяти 1:8 для гораздо меньшего уровня помех на узел.

Серия RT

Серия RT предоставляет ресурсы для приложений реального времени, таких как Oslat.

`_RT_` — это сокращение от «realtime» (реальное время).

Этой серии типов инстансов требуются узлы, способные выполнять приложения реального времени.

Особые характеристики этой серии:

- `_Hugepages_` — Hugepages используются для повышения производительности памяти.
- `_Выделенный CPU_` — физические ядра эксклюзивно назначаются каждому vCPU, чтобы обеспечить фиксированные и высокие вычислительные гарантии для рабочей нагрузки.
- `_Изолированные потоки эмулятора_` — потоки эмулятора гипервизора изолируются от vCPU, чтобы снизить влияние эмуляции на рабочую нагрузку.

- `_Соотношение vCPU к памяти (1:4)_` — соотношение vCPU к памяти 1:4, начиная с размера medium.

Ресурсы профиля инстанса

Следующие ресурсы предпочтений предоставляются Cozystack:

| Имя | Гостевая ОС |
|-------------------------|--|
| alpine | Alpine |
| centos.7 | CentOS 7 |
| centos.7.desktop | CentOS 7 |
| centos.stream10 | CentOS Stream 10 |
| centos.stream10.desktop | CentOS Stream 10 |
| centos.stream8 | CentOS Stream 8 |
| centos.stream8.desktop | CentOS Stream 8 |
| centos.stream8.dpdk | CentOS Stream 8 |
| centos.stream9 | CentOS Stream 9 |
| centos.stream9.desktop | CentOS Stream 9 |
| centos.stream9.dpdk | CentOS Stream 9 |
| cirros | Cirros |
| fedora | Fedora (amd64) |
| fedora.arm64 | Fedora (arm64) |
| opensuse.leap | OpenSUSE Leap |
| opensuse.tumbleweed | OpenSUSE Tumbleweed |
| rhel.10 | Red Hat Enterprise Linux 10 Beta (amd64) |
| rhel.10.arm64 | Red Hat Enterprise Linux 10 Beta (arm64) |
| rhel.7 | Red Hat Enterprise Linux 7 |
| rhel.7.desktop | Red Hat Enterprise Linux 7 |
| rhel.8 | Red Hat Enterprise Linux 8 |
| rhel.8.desktop | Red Hat Enterprise Linux 8 |
| rhel.8.dpdk | Red Hat Enterprise Linux 8 |
| rhel.9 | Red Hat Enterprise Linux 9 (amd64) |

| | |
|---------------------|---|
| rhel.9.arm64 | Red Hat Enterprise Linux 9 (arm64) |
| rhel.9.desktop | Red Hat Enterprise Linux 9 Desktop (amd64) |
| rhel.9.dpdk | Red Hat Enterprise Linux 9 DPDK (amd64) |
| rhel.9.realtime | Red Hat Enterprise Linux 9 Realtime (amd64) |
| sles | SUSE Linux Enterprise Server |
| ubuntu | Ubuntu |
| windows.10 | Microsoft Windows 10 |
| windows.10.virtio | Microsoft Windows 10 (virtio) |
| windows.11 | Microsoft Windows 11 |
| windows.11.virtio | Microsoft Windows 11 (virtio) |
| windows.2k16 | Microsoft Windows Server 2016 |
| windows.2k16.virtio | Microsoft Windows Server 2016 (virtio) |
| windows.2k19 | Microsoft Windows Server 2019 |
| windows.2k19.virtio | Microsoft Windows Server 2019 (virtio) |
| windows.2k22 | Microsoft Windows Server 2022 |
| windows.2k22.virtio | Microsoft Windows Server 2022 (virtio) |
| windows.2k25 | Microsoft Windows Server 2025 |
| windows.2k25.virtio | Microsoft Windows Server 2025 (virtio) |

Разработка типов и профилей инстансов

Чтобы начать настройку или создание собственных instancetypes и предпочтений, см. [Руководство разработчика](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Хранилище

Руководства по подсистеме хранения

<https://cozystack.ru/docs/v1.6/storage/>

Практические руководства по подсистеме хранения

Эти руководства покажут, как выполнять типовые задачи, связанные с системой хранения LINSTOR в Cozystack.

-

Подготовка дисков для пулов хранения LINSTOR

Как очистить метаданные дисков и подготовить физическое хранилище для LINSTOR

Настройка выделенной сети для LINSTOR

Перенаправление трафика репликации LINSTOR на выделенный сетевой интерфейс для повышения надёжности и производительности.

Настройка контроллера ресинхронизации DRBD в LINSTOR

Узнайте, как настроить параметры контроллера ресинхронизации DRBD в LINSTOR для ускорения синхронизации

Использование NFS-ресурсов в Cozystack

Настройка дополнительного модуля `nfs-driver` для заказа томов из NFS-ресурсов в Cozystack

LINSTOR GUI

Включение и доступ к дополнительной веб-консоли LINSTOR для управления узлами хранения, ресурсами и томами.

Создание зашифрованного хранилища на LINSTOR

Узнайте, как настроить и использовать шифрование постоянных томов at-rest в LINSTOR

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Подготовка дисков для пулов хранения LINSTOR

<https://cozystack.ru/docs/v1.6/storage/disk-preparation/>

Как очистить метаданные дисков и подготовить физическое хранилище для LINSTOR

В этом руководстве описано, как подготовить физические диски к использованию с LINSTOR, если на них остались старые метаданные, мешающие автоматическому обнаружению.

Описание проблемы

При настройке хранилища на новых или повторно используемых узлах на физических дисках могут оставаться следы предыдущих установок:

- суперблоки RAID
- таблицы разделов
- сигнатуры LVM
- метаданные файловых систем

Эти старые метаданные не позволяют LINSTOR определить диски как доступное хранилище.

Симптомы

1. `linstor physical-storage list` выводит пустой список или в нём отсутствуют диски
2. Диски отображаются с неожиданными типами файловых систем (например, `linux_raid_member`)
3. В пулах хранения виден только `DfltDisklessStorPool` без фактического хранилища

Диагностика

Настройка псевдонима для LINSTOR

Для удобного доступа к командам LINSTOR настройте псевдоним:

```
alias linstor='kubectl exec -n cozy-linstor deploy/linstor-controller -- linstor'
```

Проверка узлов LINSTOR

Выведите список узлов и проверьте их готовность:

```
linstor node list
```

В ожидаемом выводе все узлы должны находиться в состоянии `Online`.

Проверка пулов хранения

Проверьте текущие пулы хранения:

```
linstor storage-pool list
```

Проверка доступного физического хранилища

Проверьте, какие физические диски видит LINSTOR:

```
linstor physical-storage list
```

Если команда выводит пустой список или в нём нет ожидаемых дисков, скорее всего, на дисках остались старые метаданные и их нужно очистить.

Проверка состояния дисков на узле

Проверьте состояние дисков на конкретном узле через под `satellite`:

```
# Вывести список подов LINSTOR satellite
```

```
kubectl get pod -n cozy-linstor -l app.kubernetes.io/component=linstor-satellite
```

```
# Проверить состояние дисков
```

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \
```

```
lsblk -f
```

В ожидаемом выводе у чистых дисков поле `FSTYPE` должно быть пустым:

```
NAME FSTYPE LABEL UUID MOUNTPOINT
```

```
nvme0n1
```

```
nvme1n1
```

Если вы видите `linux_raid_member`, `LVM2_member` или другие типы файловых систем, диски нужно очистить.

Решение: очистка метаданных дисков

ВНИМАНИЕ : очистка дисков уничтожает все данные на указанных устройствах. Очищайте только те диски, которые НЕ используются для операционной системы или установки Talos.

Шаг 1: определите системные диски

Перед очисткой определите, на каком диске установлен Talos. Проверьте конфигурацию Talos в файле `nodes/<node-name>.yaml`:

```
# In nodes/<node-name>.yaml
machine:
install:
disk: /dev/sda # This disk should NOT be wiped
```

Как правило, системный диск - это `/dev/sda`, `/dev/vda` или похожее устройство.

Шаг 2: найдите конфигурацию узла

Если для развёртывания кластера вы использовали [Talm](#), конфигурации узлов хранятся в файлах `nodes/*.yaml` в каталоге конфигурации кластера.

Каждый файл соответствует конкретному узлу (например, `nodes/node1.yaml`, `nodes/node2.yaml`).

Шаг 3: очистите диски

Выведите список всех дисков на узле:

```
talm -f nodes/<node-name>.yaml get disks
```

Очистите все несистемные диски:

```
talm -f nodes/<node-name>.yaml wipe disk nvme0n1 nvme1n1 nvme2n1 ...
```

Примечание: Перечислите все диски, которые нужно очистить, в одной команде. НЕ указывайте системный диск (например, `sda`, если на нём установлен Talos).

Шаг 4: убедитесь, что диски чистые

После очистки убедитесь, что диски стали видны LINSTOR:

```
linstor physical-storage list
```

Теперь в ожидаемом выводе должны появиться ваши диски:

```
+-----+
| Device | Size | Rotational |
|=====|
| /dev/nvme0n1 | 3.49 TiB | False |
```

```
| /dev/nvme1n1 | 3.49 TiB | False |  
| ... | ... | ... |  
+-----+
```

Также можно проверить непосредственно на узле:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \  
lsblk -f
```

У чистых дисков поле `FSTYPE` должно быть пустым.

Создание пулов хранения

Когда диски очищены, создайте пул хранения LINSTOR.

Для пулов хранения ZFS из нескольких дисков:

```
linstor physical-storage create-device-pool zfs <node-name> \  
/dev/nvme0n1 /dev/nvme1n1 /dev/nvme2n1 ... \  
--pool-name data \  
--storage-pool data
```

Примечание: Укажите все диски в одной команде, чтобы создать единый пул ZFS. Повторный запуск команды с тем же именем пула завершится ошибкой.

Убедитесь, что пул хранения создан:

```
linstor storage-pool list
```

Ожидаемый вывод:

```
+-----+  
| StoragePool | Node | Driver | PoolName | FreeCapacity | TotalCapacity | State |  
|=====|  
| data | node1 | ZFS | data | 47.34 TiB | 47.62 TiB | Ok |  
| data | node2 | ZFS | data | 47.34 TiB | 47.62 TiB | Ok |  
+-----+
```

Устранение неполадок

После очистки на дисках по-прежнему видны старые метаданные

Попробуйте очистку методом ZEROES для более тщательной обработки:

```
taln -f nodes/<node-name>.yaml wipe disk --method ZEROES nvme0n1
```

Этот метод записывает на диск нули: он занимает больше времени, но гарантирует полное удаление метаданных.

Ошибка «Zpool name already used»

Если нужно пересоздать пул хранения:

1. Удалите его из LINSTOR:

```
linstor storage-pool delete <node-name> <pool-name>
```

2. Уничтожьте пул ZFS на узле:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- \
zpool destroy <pool-name>
```

3. Пересоздайте пул, указав все диски в одной команде.

Отказ в доступе на рабочих узлах

Рабочие узлы могут не разрешать прямой доступ к API Talos. Используйте под satellite для проверки состояния дисков:

```
kubectl exec -n cozy-linstor <satellite-pod-name> -c linstor-satellite -- lsblk -f
```

Если требуется очистить диски на рабочих узлах, убедитесь, что конфигурация узла разрешает доступ, или обратитесь к администратору кластера.

Краткая справка

| Команда | Описание |
|---|---|
| <code>linstor sp l</code> | Вывести список пулов хранения |
| <code>linstor ps l</code> | Вывести список доступного физического хранилища |
| <code>linstor ps cdp zfs <node> <disks> --pool-name <name> --storage-pool <name></code> | Создать пул хранения ZFS |
| <code>taln -f nodes/<node>.yaml wipe disk <disks></code> | Очистить метаданные дисков |

| | |
|--|-------------------------------|
| <code>talm -f nodes/<node>.yaml get disks</code> | Вывести список дисков на узле |
|--|-------------------------------|

Связанная документация

- Развёртывание Cozystack с помощью Talm
- Настройка выделенной сети для LINSTOR
- Настройка контроллера ресинхронизации DRBD
- Устранение неполадок LINSTOR

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Настройка выделенной сети для LINSTOR

<https://cozystack.ru/docs/v1.6/storage/dedicated-network/>

Перенаправление трафика репликации LINSTOR на выделенный сетевой интерфейс для повышения надёжности и производительности.

В этом руководстве описано, как повысить надёжность и производительность хранилища, перенаправив трафик репликации LINSTOR на выделенный сетевой интерфейс.

Введение

Платформа Cozystack создана для поддержки высокодоступных (HA) рабочих нагрузок, а значит, система должна продолжать работать, даже если один или несколько узлов выйдут из строя. Kubernetes хорошо справляется с этим для нагрузок без сохранения состояния (stateless). Однако нагрузкам с сохранением состояния (stateful), таким как файловые хранилища или диски виртуальных машин, требуется надёжная система хранения.

Когда вы выбираете класс хранения `replicated` для PersistentVolumeClaim (PVC) или DataVolume, LINSTOR создаёт несколько синхронных реплик данных на разных узлах. Благодаря этому при отказе узла (из-за отключения питания, проблем с оборудованием и т.п.) данные мгновенно остаются доступными на другом узле.

За такой уровень надёжности приходится платить: репликация хранилища может создавать значительный сетевой трафик в зависимости от интенсивности рабочей нагрузки.

Если у узлов несколько сетевых интерфейсов, производительность можно повысить, выделив один из них под трафик хранилища. Это изолирует трафик репликации от остальных нагрузок и предотвращает возможные узкие места в сети.

Когда нужна выделенная сеть для хранилища?

Не всегда необходимо настраивать выделенную сеть для трафика хранилища с самого начала. В небольших кластерах и кластерах для проверки концепции (PoC) стандартной конфигурации сети, как правило, достаточно. Преждевременная оптимизация может привести к неоправданному усложнению.

Однако в более крупных кластерах трафик репликации хранилища часто становится узким местом производительности. Он может исчерпать доступную пропускную способность и мешать другим нагрузкам, использующим ту же сеть.

Если у узлов только один сетевой интерфейс и вы не уверены, нужен ли выделенный интерфейс, начните с наблюдения за фактическим трафиком хранилища под реалистичной нагрузкой. Один из практических подходов - выделить VLAN на существующем интерфейсе под трафик репликации

хранилища. Это позволит отслеживать использование пропускной способности без добавления оборудования.

Если уровень трафика остаётся стабильно высоким или влияет на производительность приложений, переход на выделенную сеть становится оправданной оптимизацией.

Термины и определения

Прежде чем продолжить, полезно уточнить терминологию, используемую для сетевых интерфейсов в конфигурации на основе LINSTOR:

- Маршрут по умолчанию узла Kubernetes

Это маршрут по умолчанию, используемый для исходящего трафика узла. Его можно посмотреть командой `ip route show default`.

- IP-адрес источника по умолчанию узла Kubernetes

Это IP-адрес, связанный с маршрутом по умолчанию. Обычно это адрес источника для всего исходящего трафика; его можно проверить той же командой.

- Внутренний IP-адрес узла Kubernetes

Это адрес, используемый Kubernetes для внутреннего взаимодействия. Часто он совпадает с адресом источника по умолчанию, но не всегда. Его можно проверить командой `kubectl get nodes -o wide`.

- Интерфейс по умолчанию LINSTOR satellite

Здесь конфигурация становится менее очевидной. Когда оператор Piraeus запускает LINSTOR Satellite, он задаёт один или два интерфейса по умолчанию: `default-ipv4` и `default-ipv6`. Как правило, доступен только IPv4. Интерфейс по умолчанию устанавливается в IP-адрес источника по умолчанию узла при запуске и не меняется до перезапуска Satellite. Этот интерфейс можно изменить во время работы Satellite, но оператор Piraeus сбросит его при следующем перезапуске.

- Дополнительные интерфейсы LINSTOR satellite

К LINSTOR Satellite можно вручную добавить дополнительные интерфейсы. Они хранятся в базе данных контроллера LINSTOR и сохраняются между перезапусками. Оператор Piraeus не изменяет и не удаляет их. На момент написания оператор Piraeus не поддерживает объявление дополнительных интерфейсов через пользовательские ресурсы Kubernetes.

- Активное подключение LINSTOR satellite

Контроллер LINSTOR использует ровно один интерфейс для взаимодействия с каждым Satellite. По умолчанию это интерфейс `default-ipv4`. Хотя CLI LINSTOR позволяет его изменить, оператор Piraeus вернёт его обратно при следующем перезапуске Satellite.

- Соединение узлов LINSTOR (путь)

Оно определяет, как LINSTOR Satellite взаимодействуют друг с другом при синхронной репликации. Именно эту настройку мы будем выполнять в данном руководстве. Соединения узлов можно определять либо через CRD Piraeus, либо командами CLI. Оператор Piraeus не управляет ими и не переопределяет их. При этом приоритет имеет конфигурация, применённая последней.

Как назначаются IP-адреса по умолчанию

Вот типичный вывод `node list` в LINSTOR:

```
LINSTOR ==> node list

+-----+
: Node : NodeType : Addresses : State :
+=====+
: node01 : SATELLITE : 10.78.22.146:3367 (SSL) : Online :
: node02 : SATELLITE : 10.78.22.37:3367 (SSL) : Online :
: node03 : SATELLITE : 10.78.22.53:3367 (SSL) : Online :
: node04 : SATELLITE : 10.78.22.230:3367 (SSL) : Online :
: node05 : SATELLITE : 10.78.22.150:3367 (SSL) : Online :
+-----+
```

Эти IP-адреса обычно совпадают с внутренними IP-адресами узлов Kubernetes и адресами маршрута по умолчанию. По умолчанию именно они используются для трафика репликации хранилища.

Может показаться, что достаточно изменить их, чтобы перенаправить трафик хранилища на другой интерфейс. Однако это не работает.

Оператор Piraeus получает текущий IP-адрес пода Satellite при каждом перезапуске. Затем он использует этот IP - как правило, внутренний адрес узла Kubernetes - для настройки интерфейса LINSTOR по умолчанию.

Даже если попытаться изменить внутренний IP-адрес Kubernetes, трафик репликации хранилища не перейдёт на другой интерфейс. Более того, такое изменение затронет весь трафик узла, а не только репликацию LINSTOR.

Чтобы корректно изолировать трафик хранилища от остальных нагрузок, используйте соединения узлов , как описано в следующем разделе.

Интерфейсы Satellite

По умолчанию LINSTOR настраивает только один интерфейс на каждый satellite: `default-ipv4`, полученный из IP-адреса по умолчанию узла Kubernetes. Вот пример того, как выглядит список интерфейсов по умолчанию LINSTOR satellite:

```
LINSTOR ==> node interface list node01

+-----+
```

```
: node01 : NetInterface : IP : Port : EncryptionType :
```

```
+=====+
```

```
: + StltCon : default-ipv4 : 10.78.22.146 : 3367 : SSL :
```

```
+-----+
```

Этот IP берётся с узла Kubernetes при запуске Satellite. Если на узле есть дополнительные интерфейсы, автоматически они не добавляются.

Чтобы разрешить трафик хранилища через другую сеть, можно вручную добавить второй интерфейс:

```
LINSTOR ==> node interface create node01 optic-san 10.78.24.201
```

```
SUCCESS:
```

```
Description:
```

```
New netInterface 'optic-san' on node 'node01' registered.
```

```
Details:
```

```
NetInterface 'optic-san' on node 'node01' UUID is: aa28f583-ca91-4cac-b749-67fe54e72c10
```

```
LINSTOR ==> node interface list node01
```

```
+-----+
```

```
: node01 : NetInterface : IP : Port : EncryptionType :
```

```
+=====+
```

```
: + StltCon : default-ipv4 : 10.78.22.146 : 3367 : SSL :
```

```
: + : optic-san : 10.78.24.201 : : :
```

```
+-----+
```

Знать имя базового сетевого интерфейса Linux не обязательно. Можно использовать любое имя - LINSTOR не проверяет, действительно ли этот IP назначен узлу.

Флагом `StltCon` помечен только интерфейс `default-ipv4`. Это означает, что именно его контроллер LINSTOR использует для взаимодействия с Satellite.

Хотя активное подключение Satellite можно изменить вручную через CLI LINSTOR, оператор Piraeus сбросит его при следующем перезапуске. Это поведение относится только к установкам вне Kubernetes.

Чтобы использовать новый интерфейс для репликации хранилища, нужно определить соединения узлов. Но сначала повторите создание интерфейса на остальных узлах:

```
LINSTOR ==> node interface create node02 optic-san 10.78.24.202
```

```
LINSTOR ==> node interface create node03 optic-san 10.78.24.203
```

```
LINSTOR ==> node interface create node04 optic-san 10.78.24.204
```

LINSTOR ==> node interface create node05 optic-san 10.78.24.205

Соединения узлов

Соединения узлов LINSTOR определяют, как satellite взаимодействуют друг с другом при синхронной репликации данных. Для каждой пары satellite должен быть определён путь соединения.

В отличие от интерфейсов satellite, соединения узлов можно определять как вручную через CLI, так и декларативно через пользовательские ресурсы Kubernetes. Для небольших кластеров создание соединений вручную обычно не вызывает трудностей. Однако по мере роста кластера эффективнее использовать соглашение об именовании и описать все пути соединений в одном определении ресурса.

В следующих разделах рассматриваются оба подхода.

Ручной способ

Соединения узлов можно создать вручную с помощью CLI LINSTOR. Вот как посмотреть синтаксис команды:

```
LINSTOR ==> node-connection path create -h
```

```
usage: linstor node-connection path create [-h]
```

```
node_a node_b path_name
```

```
netinterface_a netinterface_b
```

Creates a new node connection path.

positional arguments:

node_a 1. Node of the connection

node_b 2. Node of the connection

path_name Name of the created path

netinterface_a Netinterface name to use for 1. node

netinterface_b Netinterface name to use for the 2. node

Пример: создание соединения между `node01` и `node02` с использованием интерфейса `optic-san` с обеих сторон:

```
LINSTOR ==> node-connection path create node01 node02 node01-02 optic-san optic-san
```

```
SUCCESS:
```

```
Successfully set property key(s): Paths/node01-02/node01,Paths/node01-02/node02
```

```
SUCCESS:
```

```
Description:
```

Node connection between nodes 'node01' and 'node02' modified.

Details:

Node connection between nodes 'node01' and 'node02' UUID is: c1f4ee6a-776e-46ba-9e74-99afce38d90f

SUCCESS:

(node02) Node changes applied.

SUCCESS:

(node02) Resource 'pvc-6f535d3a-82c1-46ab-80fe-5a59ee8bff44' [DRBD] adjusted.

....

После создания соединения LINSTOR сразу обновляет все затронутые ресурсы DRBD, чтобы они использовали новый путь.

Путь создаётся один раз для каждой пары узлов. Определять отдельный обратный путь не нужно.

Проверить конфигурацию можно так:

```
LINSTOR ==> node-connection path list node01 node02
```

```
+-----+
```

```
: Key : Value :
```

```
+=====+
```

```
: Paths/node01-02/node01 : optic-san :
```

```
: Paths/node01-02/node02 : optic-san :
```

```
+-----+
```

```
LINSTOR ==> node-connection path list node02 node01
```

```
+-----+
```

```
: Key : Value :
```

```
+=====+
```

```
: Paths/node01-02/node01 : optic-san :
```

```
: Paths/node01-02/node02 : optic-san :
```

```
+-----+
```

```
LINSTOR ==> node-connection list node01 node02
```

```
+-----+
```

```
: Node A : Node B : Properties :
```

```
+=====+
```

```
: node01 : node02 : node01=optic-san,node02=optic-... :
```

```
+-----+
```

```
LINSTOR ==> node-connection list node02 node01
```

```
+-----+
: Node A : Node B : Properties :
+=====+
: node01 : node02 : node01=optic-san,node02=optic-... :
+-----+
```

Способ с CRD

Рассмотрим способ с использованием Kubernetes CRD (Custom Resource Definition).

На момент написания у оператора Piraeus есть известная проблема с обработкой отсутствующих интерфейсов. Между попытками согласования нет задержки (backoff). Если отсутствует хотя бы один интерфейс, контроллер может потреблять 100% своего лимита CPU.

Всегда проверяйте, что все необходимые интерфейсы существуют, прежде чем применять CRD. Следите за подом контроллера, чтобы убедиться, что после изменения он не перегружен.

С увеличением числа узлов количество необходимых соединений быстро растёт. Для 3 узлов нужно 3 соединения. Для 5 узлов - 10. Для 10 узлов - 45, и так далее.

Вместо создания каждого соединения вручную можно определить все пути сразу с помощью одного пользовательского ресурса `LinstorNodeConnection`.

Для использования этого способа должны выполняться следующие условия:

- На всех участвующих узлах интерфейс должен называться одинаково.
- Интерфейс должен быть создан на всех узлах до применения CRD.

Пример пользовательского ресурса `LinstorNodeConnection`:

```
apiVersion: piraeus.io/v1
kind: LinstorNodeConnection
metadata:
name: dedicated
spec:
paths:
  - name: dedicated
interface: optic-san
```

Примените CRD командой:

```
kubectl apply -f linstor-node-connections.yaml
```

После применения можно проверить соединения:

LINSTOR ==> node-connection list

```
+-----+
: Node A : Node B : Properties :
+=====+
: node01 : node02 : last-applied=["Paths/dedicated... :
: node01 : node03 : last-applied=["Paths/dedicated... :
: node01 : node04 : last-applied=["Paths/dedicated... :
: node01 : node05 : last-applied=["Paths/dedicated... :
: node02 : node03 : last-applied=["Paths/dedicated... :
: node02 : node04 : last-applied=["Paths/dedicated... :
: node02 : node05 : last-applied=["Paths/dedicated... :
: node03 : node04 : last-applied=["Paths/dedicated... :
: node03 : node05 : last-applied=["Paths/dedicated... :
: node04 : node05 : last-applied=["Paths/dedicated... :
+-----+
```

Старые пути, созданные вручную, могут по-прежнему отображаться. Приоритет имеет соединение, применённое последним.

В этом примере видны старые пути для `node01` и `node02`:

LINSTOR ==> node-connection path list node01 node02

```
+-----+
: Key : Value :
+=====+
: Paths/dedicated/node01 : optic-san :
: Paths/dedicated/node02 : optic-san :
: Paths/node01-02/node01 : optic-san :
: Paths/node01-02/node02 : optic-san :
+-----+
```

LINSTOR ==> node-connection path list node01 node03

```
+-----+
: Key : Value :
+=====+
: Paths/dedicated/node01 : optic-san :
: Paths/dedicated/node03 : optic-san :
```

+-----+

Чтобы конфигурация оставалась аккуратной, старый ручной путь можно удалить:

LINSTOR ==> `linstor node-connection path delete node01 node02 node01-02`

SUCCESS:

Successfully deleted property key(s): Paths/node01-02/node02,Paths/node01-02/node01

....

После удаления:

LINSTOR ==> `node-connection path list node01 node02`

+-----+

: Key : Value :

+=====+

: Paths/dedicated/node01 : optic-san :

: Paths/dedicated/node02 : optic-san :

+-----+

Расширенный способ с CRD

См. пример в руководстве [Выделенная сеть хранилища для нескольких дата-центров](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Настройка контроллера ресинхронизации DRBD в LINSTOR

<https://cozystack.ru/docs/v1.6/storage/drbd-tuning/>

Узнайте, как настроить параметры контроллера ресинхронизации DRBD в LINSTOR для ускорения синхронизации

Администраторы Cozystack могут регулировать производительность синхронизации DRBD, задавая параметры настройки для контроллера LINSTOR.

Это позволяет оптимизировать скорость ресинхронизации, не перегружая сеть репликации и систему хранения.

Подробное описание всех доступных параметров и рекомендации по настройке приведены в официальном руководстве LINBIT: [Tuning the DRBD Resync Controller](#).

Для конфигурации с несколькими дата-центрами также прочитайте руководство [Настройка DRBD для нескольких дата-центров](#).

Рекомендуемые настройки для сетей 10G

Мы считаем следующие значения оптимальными для кластеров, соединённых 10-гигабитной сетью:

```
linstor controller set-property DrbdOptions/Net/max-buffers 36864
linstor controller set-property DrbdOptions/Net/rcvbuf-size 10485760
linstor controller set-property DrbdOptions/Net/sndbuf-size 10485760
linstor controller set-property DrbdOptions/PeerDevice/c-fill-target 2048
linstor controller set-property DrbdOptions/PeerDevice/c-max-rate 737280
linstor controller set-property DrbdOptions/PeerDevice/c-min-rate 245760
linstor controller set-property DrbdOptions/PeerDevice/resync-rate 245760
linstor controller set-property DrbdOptions/PeerDevice/c-plan-ahead 10
```

- `c-max-rate` указывается в КиБ/с и должен соответствовать максимальной устойчивой пропускной способности дисков или сети (в зависимости от того, что меньше). Значение `737280` в примере соответствует 720 МиБ/с.
- `c-min-rate` и `resync-rate` также указываются в КиБ/с; их следует устанавливать примерно в одну треть от `c-max-rate`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Использование NFS-ресурсов в Cozystack

<https://cozystack.ru/docs/v1.6/storage/nfs/>

Настройка дополнительного модуля `nfs-driver` для заказа томов из NFS-ресурсов в Cozystack

Включение драйвера NFS

Добавьте `cozystack.nfs-driver` в `bundles.enabledPackages` в [Platform Package](#):

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozystack.nfs-driver"}]'
```

Подождите около минуты, пока чарт платформы выполнит согласование, затем убедитесь, что `HelmRelease` создан:

```
kubectl get helmrelease --namespace cozy-nfs-driver nfs-driver
```

Экспорт общего ресурса

```
apt install nfs-server
mkdir /data
chmod 777 /data
echo '/data *(rw,sync,no_subtree_check)' >> /etc/exports
exportfs -a
```

Настройка подключения

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: nfs
provisioner: nfs.csi.k8s.io
parameters:
server: 10.244.57.210
share: /data
```

reclaimPolicy: Delete
volumeBindingMode: Immediate
allowVolumeExpansion: true
mountOptions:
 - nfsvers=4.1

Заказ тома

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: task-pv-claim
spec:
 storageClassName: nfs
 accessModes:
 - ReadWriteMany
resources:
 requests:
 storage: 3Gi

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

LINSTOR GUI

<https://cozystack.ru/docs/v1.6/storage/linstor-gui/>

Включение и доступ к дополнительной веб-консоли LINSTOR для управления узлами хранения, ресурсами и томами.

Пакет `linstor-gui` развёртывает **LINSTOR GUI от LINBIT** \- веб-консоль для просмотра и управления узлами LINSTOR, определениями ресурсов, томами, пулами хранения и снимками. Интерфейс проксирует REST API контроллера LINSTOR внутри кластера с использованием mTLS, поэтому учётные данные никогда не попадают в браузер.

Пакет является опциональным и включается по требованию. Рабочий процесс с CLI не меняется - включение GUI никак не влияет на поведение LINSTOR.

Включение пакета

Добавьте `cozystack.linstor-gui` в `bundles.enabledPackages` в **Platform Package**:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/bundles/enabledPackages/-", "value": "cozystack.linstor-gui"}]'
```

Подождите около минуты, пока чарт платформы выполнит согласование, затем убедитесь, что HelmRelease создан:

```
kubectl get helmrelease --namespace cozy-linstor linstor-gui
```

Доступ к интерфейсу

Вариант 1 - Ingress, защищённый Keycloak (рекомендуется)

Если включена **аутентификация OIDC**, интерфейс можно опубликовать по адресу `https://linstor-gui.<root-host>` за realm Keycloak кластера. Добавьте `linstor-gui` в `publishing.exposedServices` в Platform Package:

```
kubectl patch packages.cozystack.io cozystack.cozystack-platform --type=json \
-p '[{"op": "add", "path": "/spec/components/platform/values/publishing/exposedServices/-", "value": "linstor-gui"}]'
```

Ingress создаётся только при выполнении обоих условий: `linstor-gui` указан в `publishing.exposedServices` и включён OIDC (`authentication.oidc.enabled: true`). Без Keycloak перед прокси REST API LINSTOR нет слоя аутентификации, поэтому чарт намеренно не создаёт Ingress.

Доступ ограничен участниками группы Keycloak `cozystack-cluster-admin` \- той же группы, которая выдаёт права `cluster-admin` RBAC в управляющем кластере. После включения откройте `https://linstor-gui.<root-host>` в браузере и войдите, используя учётные данные Keycloak.

Вариант 2 - Проброс порта

Для разового доступа без Keycloak пробросьте порт сервиса `ClusterIP`:

```
kubectl -n cozy-linstor port-forward svc/linstor-gui 3373:80
```

Затем откройте `<http://localhost:3373>`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Создание зашифрованного хранилища на LINSTOR

<https://cozystack.ru/docs/v1.6/storage/disk-encryption/>

Узнайте, как настроить и использовать шифрование постоянных томов at-rest в LINSTOR

Администраторы Cozystack могут включить зашифрованное хранилище, создав собственный StorageClass. В этом руководстве описано, как задать парольную фразу шифрования, создать зашифрованный класс хранения и использовать его в приложениях.

LINSTOR обеспечивает шифрование постоянных томов at-rest с помощью [LUKS](#). Это гарантирует, что данные на диске хранятся в зашифрованном виде и доступны только тогда, когда том смонтирован и разблокирован.

Настройка шифрования в LINSTOR

Чтобы начать использовать шифрование, задайте парольную фразу шифрования в LINSTOR.

```
kubectl exec -i -t -n cozy-linstor deploy/linstor-controller -- linstor encryption create-passphrase
```

[!] Сохраните парольную фразу в надёжном месте.

Если вы потеряете парольную фразу шифрования, все зашифрованные данные будут безвозвратно утеряны.

Парольную фразу нужно вводить каждый раз после перезапуска контроллера LINSTOR. Для ввода парольной фразы используйте следующую команду:

```
kubectl exec -i -t -n cozy-linstor deploy/linstor-controller -- linstor encryption enter-passphrase
```

Создание зашифрованного класса хранения

Создайте [StorageClass](#) для зашифрованного хранилища:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-encrypted
provisioner: linstor.csi.linbit.com
parameters:
  linstor.csi.linbit.com/storagePool: "data"
```

linstor.csi.linbit.com/layerList: "luks storage"

linstor.csi.linbit.com/encryption: "true"

linstor.csi.linbit.com/allowRemoteVolumeAccess: "false"

volumeBindingMode: WaitForFirstConsumer

allowVolumeExpansion: true

apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:

name: replicated-encrypted

provisioner: linstor.csi.linbit.com

parameters:

linstor.csi.linbit.com/storagePool: "data"

linstor.csi.linbit.com/autoPlace: "3"

linstor.csi.linbit.com/layerList: "drbd luks storage"

linstor.csi.linbit.com/encryption: "true"

linstor.csi.linbit.com/allowRemoteVolumeAccess: "true"

property.linstor.csi.linbit.com/DrbdOptions/auto-quorum: suspend-io

property.linstor.csi.linbit.com/DrbdOptions/Resource/on-no-data-accessible: suspend-io

property.linstor.csi.linbit.com/DrbdOptions/Resource/on-suspended-primary-outdated: force-secondary

property.linstor.csi.linbit.com/DrbdOptions/Net/rr-conflict: retry-connect

volumeBindingMode: Immediate

allowVolumeExpansion: true

Теперь вы можете использовать этот `StorageClass` для создания `PersistentVolumeClaims` (PVC) под шифрованное хранилище.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Сеть

Сетевые возможности

<https://cozystack.ru/docs/v1.6/networking/>

Настройка сети, виртуальные маршрутизаторы, балансировщики нагрузки и другие сетевые возможности Cozystack.

Этот раздел документации описывает настройку сети, виртуальные маршрутизаторы, балансировщики нагрузки и другие сетевые возможности Cozystack.

-

Сетевая архитектура

Обзор сетевой архитектуры кластера Cozystack: балансировка нагрузки MetalLB, сеть Cilium на eBPF и изоляция тенантов с Kube-OVN.

VPC

Выделенные подсети

Управляемый сервис VPN

Управляемый сервис VPN упрощает развёртывание VPN-сервера и управление им, позволяя легко устанавливать защищённые соединения.

Gateway API (Cilium)

Ingress на основе Gateway API для отдельных тенантов с использованием Cilium - CRD TenantGateway, наследование через метки пространств имён, интеграция с cert-manager, терминатия и сквозная передача TLS, двухуровневая модель безопасности.

Управляемый сервис HTTP-кеширования на основе Nginx

Сервис HTTP-кеширования на основе Nginx предназначен для оптимизации веб-трафика и повышения производительности веб-приложений.

PROXY-protocol и исправление hairpin-NAT

Включение PROXY-protocol на ingress-nginx в Cozystack и исправление возникающей проблемы hairpin-NAT с помощью входящего в состав платформы компонента ouroboros.

Публикация конечной точки Kubernetes API через external-dns

Преобразование EndpointSlice default/kubernetes в аннотированные headless-сервисы, чтобы external-dns мог публиковать конечную точку Kubernetes API в DNS с помощью входящего в состав платформы компонента kuberture.

[Управляемый сервис балансировки TCP-нагрузки](#)

Управляемый сервис балансировки TCP-нагрузки упрощает развёртывание балансировщиков нагрузки и управление ими.

[Виртуальные маршрутизаторы](#)

Развёртывание виртуального маршрутизатора в VM

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Сетевая архитектура

<https://cozystack.ru/docs/v1.6/networking/architecture/>

Обзор сетевой архитектуры кластера Cozystack: балансировка нагрузки MetalLB, сеть Cilium на eBPF и изоляция тенантов с Kube-OVN.

Обзор

Cozystack использует многослойный сетевой стек, разработанный для bare-metal-кластеров Kubernetes. Архитектура объединяет несколько компонентов, каждый из которых отвечает за свой уровень сети:

| Уровень | Компонент | Назначение |
|--------------------------------|----------------------|--|
| Внешняя балансировка нагрузки | MetalLB | Публикация сервисов во внешние сети |
| Балансировка нагрузки сервисов | Cilium eBPF | Замена kube-proxy, DNAT внутри ядра |
| Сетевые политики | Cilium eBPF | Изоляция тенантов и обеспечение безопасности |
| Сеть подов (CNI) | Kube-OVN | Централизованный IPAM, оверлейная сеть |
| Проброс IP в VM | cozy-proxy | Проброс внешних IP-адресов внутрь виртуальных машин |
| Вторичные интерфейсы VM | Multus CNI | Подключение вторичных L2-интерфейсов к виртуальным машинам |
| Наблюдаемость | Hubble (опционально) | Видимость сетевого трафика (по умолчанию отключено) |

flowchart TD

EXT["External Clients"]

RTR["Upstream Router / Gateway"]

MLB["MetalLB
(L2 ARP / BGP)"]

CIL["Cilium eBPF
(Service Load Balancing + Network Policies)"]

OVN["Kube-OVN
(Pod Networking + IPAM)"]

PODS["Pods"]

EXT --> RTR

RTR --> MLB
MLB --> CIL
CIL --> OVN
OVN --> PODS

Сетевая конфигурация кластера

| Параметр | Значение по умолчанию |
|-------------------|-----------------------|
| Pod CIDR | 10.244.0.0/16 |
| Service CIDR | 10.96.0.0/16 |
| Join CIDR | 100.64.0.0/16 |
| Домен кластера | cozy.local |
| Тип оверлея | GENEVE |
| CNI | Kube-OVN |
| Замена kube-proxy | Cilium eBPF |

Варианты сетевого стека

Cozystack поддерживает несколько вариантов сетевого стека для разных типов кластеров. Вариант выбирается через `bundles.system.variant` в конфигурации платформы.

| Вариант | Компоненты | Целевая платформа |
|-------------------------------------|-----------------------------------|--------------------|
| <code>kubeovn-cilium</code> | Kube-OVN + Cilium (по умолчанию) | Talos Linux |
| <code>kubeovn-cilium-generic</code> | Kube-OVN + Cilium | kubeadm, k3s, RKE2 |
| <code>cilium</code> | Только Cilium | Talos Linux |
| <code>cilium-generic</code> | Только Cilium | kubeadm, k3s, RKE2 |
| <code>cilium-kilo</code> | Cilium + Kilo | Talos Linux |
| <code>noop</code> | Нет (используйте собственный CNI) | Любая |

В вариантах с Kube-OVN Cilium работает как цепочечный CNI (режим `generic-veth`): Kube-OVN отвечает за сеть подов и IPAM, а Cilium обеспечивает балансировку нагрузки сервисов, применение сетевых политик и опциональную наблюдаемость через Hubble.

В вариантах только с Cilium он выступает одновременно и как CNI, и как балансировщик нагрузки сервисов.

Далее в этом документе описывается вариант по умолчанию `kubeovn-cilium`.

Выделение Pod CIDR (Kube-OVN)

Kube-OVN использует модель общего Pod CIDR :

- Все поды получают адреса из единого общего пула IP-адресов (10.244.0.0/16)
- IP-адреса выделяются централизованно через IPAM Kube-OVN
- Нет разбиения CIDR по узлам (в отличие от Calico или Flannel)
- Поскольку IP-адреса не привязаны к CIDR-блокам конкретных узлов, поды можно переносить на другие узлы с сохранением адресов
- Взаимодействие подов между узлами использует туннели GENEVE (Join CIDR: 100.64.0.0/16)

Приём внешнего трафика через MetalLB

MetalLB - реализация балансировщика нагрузки для bare-metal-кластеров Kubernetes. Он назначает внешние IP-адреса сервисам типа `LoadBalancer`, позволяя внешнему трафику достигать кластера.

flowchart TD

CLIENT["External Client"]

RTR["Upstream Router"]

subgraph CLUSTER["Kubernetes Cluster"]

S1["Node 1
MetalLB Speaker"]

S2["Node 2
MetalLB Speaker"]

S3["Node 3
MetalLB Speaker"]

CIL["Cilium (eBPF)
Service Load Balancing
DNAT to Pod IP"]

POD["Target Pod
(Pod CIDR)"]

end

CLIENT --> |"Traffic to external IP
(e.g. 10.x.x.20)"| RTR

RTR --> |"L2 (ARP) or BGP"| S1

RTR --> |"L2 (ARP) or BGP"| S2

RTR --> |"L2 (ARP) or BGP"| S3

S1 --> CIL

S2 --> CIL

S3 --> CIL

CIL --> POD

Режим Layer 2 (ARP)

В режиме L2 MetalLB отвечает на ARP-запросы для внешнего IP-адреса сервиса. Один узел становится «лидером» для этого IP и принимает весь трафик.

Как это работает:

1. Спикер MetalLB на одном из узлов забирает внешний IP себе
2. Спикер отвечает на ARP-запросы: «IP X находится по MAC-адресу aa:bb:cc:dd:ee:ff»
3. Весь трафик для этого IP идёт на узел-лидер
4. Cilium на узле выполняет DNAT к нужному поду

sequenceDiagram

participant C as Client

participant L as Node (MetalLB Leader)

participant CIL as Cilium (eBPF)

participant P as Pod

C->>L: ARP: Who has 10.x.x.20?

L-->>C: ARP Reply: 10.x.x.20 is at aa:bb:cc:dd:ee:ff

C->>L: Send traffic to 10.x.x.20

L->>CIL: Packet enters kernel

CIL->>P: DNAT -> Pod 10.244.x.x:8080

В режиме L2 трафик для конкретного IP сервиса обрабатывает только один узел. При отказе узла-лидера происходит переключение, но настоящей балансировки нагрузки между узлами для одного сервиса нет.

Режим BGP

В режиме BGP MetalLB устанавливает BGP-сессии с вышестоящими маршрутизаторами и анонсирует маршруты /32 для IP-адресов сервисов. Это обеспечивает настоящую балансировку нагрузки ECMP между узлами.

Как это работает:

1. Спикеры MetalLB устанавливают BGP-сессии с вышестоящими маршрутизаторами
2. Каждый спикер анонсирует IP сервиса как маршрут /32
3. У маршрутизатора появляется несколько next-hop для одного префикса
4. ECMP распределяет трафик между узлами
5. Cilium на принимающем узле выполняет DNAT к нужному поду

```
sequenceDiagram
```

```
participant S1 as Node 1 (Speaker)
```

```
participant S2 as Node 2 (Speaker)
```

```
participant S3 as Node 3 (Speaker)
```

```
participant R as Upstream Router
```

```
participant CIL as Cilium (eBPF)
```

```
participant P as Pod
```

```
S1->>R: BGP UPDATE: 10.x.x.20/32 via Node 1
```

```
S2->>R: BGP UPDATE: 10.x.x.20/32 via Node 2
```

```
S3->>R: BGP UPDATE: 10.x.x.20/32 via Node 3
```

```
Note over R: ECMP: 3 next-hops for 10.x.x.20/32
```

```
R->>S1: Traffic (1/3)
```

```
R->>S2: Traffic (1/3)
```

```
R->>S3: Traffic (1/3)
```

```
S1->>CIL: Packet enters kernel
```

```
CIL->>P: DNAT -> Pod
```

Интеграция VLAN для внешнего трафика

Внешний трафик может доставляться в кластер через дополнительные VLAN (клиентские VLAN, DMZ, публичные сети и т.п.), откуда он направляется к сервисам через MetalLB и Cilium.

```
flowchart TD
```

```
EXT["External Traffic"]
```

```
subgraph VLANs["Additional VLANs<br/>(Client, DMZ, Public, etc.)"]
```

```
V1["VLAN A"]
```

```
V2["VLAN B"]
```

```
end
```

```
subgraph LB["MetalLB"]
```

```
L2["L2 Mode -> Service -> Pod"]
```

```
BGP["BGP Mode -> Service -> Pod"]
```

```
end
```

```
EXT --> VLANs
```

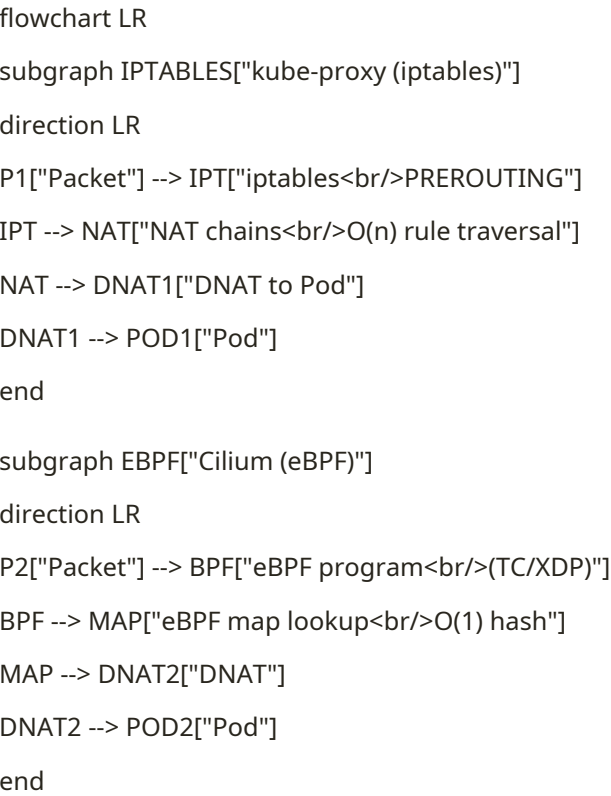
```
V1 --> L2
```

```
V2 --> BGP
```

Cilium как замена kube-proxy

Cilium заменяет kube-проxy, подключая программы eBPF непосредственно в ядре Linux. Это обеспечивает более эффективную обработку пакетов и расширенные возможности.

Традиционный kube-proxy (iptables) против Cilium eBPF



Ключевые отличия:

| Аспект | kube-proxy (iptables) | Cilium (eBPF) |
|------------------------|---|-------------------------------|
| Сложность поиска | Обход правил за O(n) | Поиск по хешу за O(1) |
| Контекст выполнения | Накладные расходы в пользовательском пространстве | Нативно в ядре |
| Переключения контекста | Требуются | Отсутствуют |
| Масштабируемость | Деградирует с ростом числа сервисов | Постоянная производительность |

Архитектура eBPF



```

subgraph BPF["eBPF Programs"]
  TC["TC<br/>(ingress/egress)"]
  XDP["XDP<br/>(fastest path)"]
  SOCK["Socket-level<br/>(connect, sendmsg)"]
end

subgraph MAPS["eBPF Maps"]
  SVC["Service Tables"]
  EP["Endpoint Maps"]
  POL["Policy Maps"]
end

TC --> MAPS
XDP --> MAPS
SOCK --> MAPS
end

```

Изоляция тенантов с Kube-OVN и Cilium

В мультитенантном кластере Cozystack все тенанты используют общий Pod CIDR. Это безопасно, потому что изоляция обеспечивается политиками Cilium eBPF на уровне ядра, а не сегментацией сети. Тенанты не могут взаимодействовать друг с другом, хотя используют общий пул IP-адресов. Kube-OVN выделяет IP-адреса из этого общего пула централизованно, без разбиения CIDR по узлам.

Архитектура CNI

```

flowchart TD
  subgraph KO["Kube-OVN"]
    IPAM["Centralized IPAM — Shared pool 10.244.0.0/16"]
    OVN["OVN/OVS Overlay Network (GENEVE)"]
    SUBNET["Subnet management per namespace/tenant"]
  end

  subgraph CIL["Cilium"]
    POLICY["eBPF Network Policies"]
    SVCBAL["Service Load Balancing (kube-proxy replacement)"]
    IDENT["Identity-based Security"]
    HUB["Observability via Hubble"]
  end

```

end

KO --> CIL

Kube-OVN является основным CNI-плагином для сети подов и IPAM. Собственный механизм сетевых политик Kube-OVN отключён (`ENABLE_NP: false`), и всё применение политик делегировано Cilium. Cilium работает как цепочечный CNI-компонент (режим `generic-veth`), который применяет сетевые политики через eBPF и заменяет kube-proxy для балансировки нагрузки сервисов.

Модель изоляции tenants

flowchart TD

TA["Tenant A — Namespace app-a
Pods: 10.244.0.10, 10.244.0.11"]

TB["Tenant B — Namespace app-b
Pods: 10.244.1.20, 10.244.1.21"]

TC["Tenant C — Namespace app-c
Pods: 10.244.2.30, 10.244.2.31"]

ENGINE{"Cilium eBPF Policy Engine"}

TA --> ENGINE

TB --> ENGINE

TC --> ENGINE

ENGINE -->|"A <-> A — ALLOWED"| ALLOW["Same-tenant traffic passes"]

ENGINE -->|"A <-> B — DENIED"| DENY["Cross-tenant traffic dropped"]

Безопасность на основе идентичностей

Cilium присваивает каждой конечной точке (поду) идентичность безопасности на основе её меток. Политики применяются с использованием этих идентичностей, а не IP-адресов.

flowchart LR

POD["Pod: frontend-abc123
Labels: app=frontend,
tenant=acme, env=prod"]

AGENT["Cilium Agent
Hash(labels) -> Identity: 12345"]

BPFMAP["eBPF Map
10.244.0.10 -> Identity 12345"]

POD --> AGENT

AGENT --> BPFMAP

Применение политик в ядре

Когда пакет передаётся между подами, Cilium применяет политики полностью в пространстве ядра:

flowchart TD

```
PKT["Packet: 10.244.0.10 -> 10.244.1.20"]
STEP1["1. Lookup source identity:<br/>10.244.0.10 -> ID 12345 (tenant-a)"]
STEP2["2. Lookup destination identity:<br/>10.244.1.20 -> ID 67890 (tenant-b)"]
STEP3["3. Check policy map:<br/>(12345, 67890, TCP, 80) -> DENY"]
DROP["4. DROP packet"]

PKT --> STEP1 --> STEP2 --> STEP3 --> DROP
```

Всё это происходит в пространстве ядра примерно за 100 наносекунд.

Почему применение политик через eBPF безопасно

| Свойство | Описание |
|--|---|
| Верификатор | Программы eBPF проверяются перед загрузкой - без сбоев и бесконечных циклов |
| Изоляция | Программы выполняются в ограниченном контексте ядра |
| Нет обхода из пользовательского пространства | Весь сетевой трафик обязан проходить через хуки eBPF |
| Атомарные обновления | Изменения политик атомарны - без состояний гонки |
| Внутри ядра | Не нужны переключения контекста, быстрее, чем в пользовательском пространстве |

Применение на уровне ядра

```
flowchart TD
    subgraph US["User Space"]
        PODA["Pod A<br/>(Tenant A)"]
        PODB["Pod B<br/>(Tenant B)"]
        NOTE["Cannot bypass policy —<br/>traffic MUST go through kernel"]
    end

    subgraph KS["Kernel Space"]
        EBPF["eBPF Programs<br/>• Attached to network interfaces<br/>• Run in privileged kernel context<br/>• Verified by kernel<br/>• Cannot be bypassed by userspace<br/>• Atomic policy updates"]
    end

    US --> |"all traffic"| KS
```

Запрет по умолчанию и изоляция пространств имён

По умолчанию Kubernetes разрешает весь трафик между подами. Cozystack автоматически применяет ресурсы `CiliumNetworkPolicy` и `CiliumClusterwideNetworkPolicy` при создании тенанта. Эти политики обеспечивают изоляцию на уровне пространств имён и ограничивают доступ к системным портам (etcd, kubelet, контроллеры).

Для изоляции Cozystack использует иерархические метки тенантов. Политики сопоставляются по меткам пространств имён `tenant.cozystack.io/*`, что позволяет родительским тенантам включать пространства имён дочерних тенантов. Пример:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: allow-internal-communication
  namespace: tenant-example
spec:
  endpointSelector: {}
  ingress:
    - fromEndpoints:
    - matchLabels:
        k8s:io.cilium.k8s.namespace.labels.tenant.cozystack.io/tenant-example: ""
  egress:
    - toEndpoints:
    - matchLabels:
        k8s:io.cilium.k8s.namespace.labels.tenant.cozystack.io/tenant-example: ""
    - toEntities:
      - kube-apiserver
      - cluster
```

Host Firewall and Node IPv6

Cozystack enables Cilium's host firewall (`hostFirewall.enabled: true`) to enforce the system-port restrictions described above on the nodes themselves. The Cilium IPv6 datapath stays disabled (`ipv6.enabled: false`), since pod networking is provided by Kube-OVN.

In upstream Cilium this combination drops all IPv6 traffic on the node's network devices before any policy evaluation. This breaks IPv6 Neighbor Discovery and, with it, all node-level IPv6 connectivity — for example, BGP unnumbered peering over link-local addresses on L3 fabrics. A `CiliumClusterwideNetworkPolicy` cannot allow this traffic back, because the drop happens before policy enforcement.

The Cilium image shipped with Cozystack carries a BPF patch that passes IPv6 to the kernel stack instead, matching the behavior when the host firewall is disabled. The patch is carried until an equivalent fix is available upstream. See [cozystack/cozystack#2871](#) for the implementation. Practical consequences:

- Node IPv6 (Neighbor Discovery, BGP over link-local addresses, and any other node-level IPv6 traffic) keeps working with the host firewall enabled.
- Cilium host policies apply to IPv4 only. Node IPv6 is not filtered by Cilium; if nodes exposed over IPv6 need filtering, it must be done by other means.

To opt out of the host firewall entirely, set `cilium.hostFirewall.enabled: false` in the values of the `cozystack.networking` Package. Note that this also disables all IPv4 host policies, including the system-port restrictions.

Наблюдаемость с Hubble

Hubble обеспечивает видимость сетевого трафика для плоскости данных Cilium. Он входит в сетевой стек Cozystack, но по умолчанию отключён, чтобы минимизировать потребление ресурсов.

Во включённом состоянии Hubble предоставляет:

- Журналы потоков в реальном времени для всего трафика между подами и внешнего трафика
- Видимость DNS-запросов
- Метрики уровня запросов HTTP/gRPC
- Интеграцию с метриками Prometheus
- Веб-интерфейс для визуализации трафика

Чтобы включить Hubble, задайте следующее в конфигурации Cilium:

```
cilium:  
  hubble:  
    enabled: true  
  relay:  
    enabled: true  
  ui:  
    enabled: true
```

Полные сведения о настройке см. в разделе [Enabling Hubble](#).

Сводка потоков трафика

Внешний доступ

flowchart LR

C["Client"] --> R["Router"]

R --> M["MetalLB
(L2/BGP)"]

M --> N["Node"]

N --> E["Cilium eBPF"]

E --> P["Pod"]

Изоляция tenants

flowchart LR

A["Pod A"] --> CHECK{"eBPF
Policy Check"}

CHECK --> |"Cross-tenant"| DENY["DENY"]

CHECK --> |"Same tenant"| ALLOW["ALLOW -> Pod A"]

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

VPC

<https://cozystack.ru/docs/v1.6/networking/vpc/>

Выделенные подсети

VPC предоставляет набор выделенных подсетей вместе со связанными с ними сетевыми сервисами. По мере развития сервиса будет появляться больше способов изоляции рабочих нагрузок.

Детали сервиса

Для работы сервису требуются kube-ovn и multus CNI, поэтому по умолчанию он работает только с бандлом `paas-full`. Kube-ovn предоставляет ресурсы VPC и Subnet, а также обеспечивает изоляцию и обслуживание сети, например DHCP. Внутри он использует виртуальные маршрутизаторы и виртуальные коммутаторы ovn. Multus добавляет поддержку нескольких сетевых интерфейсов, поэтому под или VM могут иметь два и более сетевых интерфейса.

Сейчас каждая рабочая нагрузка подключается к управляющей сети по умолчанию, в которой также находится шлюз по умолчанию, и большая часть трафика идёт через неё. Подсети VPC пока являются дополнительными выделенными сетевыми пространствами.

Замечания по развёртыванию

Имя VPC должно быть уникальным в пределах тенанта. Имя подсети и диапазон IP-адресов должны быть уникальными в пределах VPC. Адресное пространство подсети не должно пересекаться с диапазоном IP-адресов управляющей сети по умолчанию; рекомендуется использовать подмножества 172.16.0.0/12. Сейчас защитных проверок нет, но они запланированы на будущее.

Разные VPC могут содержать подсети с пересекающимися диапазонами IP-адресов.

VM или под могут быть подключены к нескольким вторичным подсетям одновременно. Каждое вторичное подключение будет представлено дополнительным сетевым интерфейсом.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|----------------------|-------------|--------------------------------|--------------------|
| <code>subnets</code> | Подсети VPC | <code>map[string]object</code> | <code>{...}</code> |

| | | | |
|---------------------------------|--|-------------------|-----------------|
| <code>subnets[name].cidr</code> | CIDR подсети,
например 192.168.0.0/24 | <code>cidr</code> | <code>{}</code> |
|---------------------------------|--|-------------------|-----------------|

Примеры

```
apiVersion: apps.cozystack.io/v1alpha1
kind: VirtualPrivateCloud
metadata:
name: vpc00
spec:
subnets:
sub00:
cidr: 172.16.0.0/24
sub01:
cidr: 172.16.1.0/24
sub02:
cidr: 172.16.2.0/24

Last modified 2026-07-23: translate 1.6 \(c21c9f4\)
```

Управляемый сервис VPN

<https://cozystack.ru/docs/v1.6/networking/vpn/>

Управляемый сервис VPN упрощает развёртывание VPN-сервера и управление им, позволяя легко устанавливать защищённые соединения.

Виртуальная частная сеть (VPN) - важнейший инструмент для обеспечения безопасного и приватного обмена данными через интернет. Управляемый сервис VPN упрощает развёртывание VPN-сервера и управление им, позволяя легко устанавливать защищённые соединения.

- Клиентские приложения VPN: <<https://shadowsocks5.github.io/en/download/clients.html>>

Детали развёртывания

Сервис VPN работает на основе Outline Server - продвинутого и удобного VPN-решения. Его внутреннее название - «Shadowbox»; он упрощает процесс настройки и совместного использования серверов Shadowsocks. Он работает, запуская экземпляры Shadowsocks по требованию. Кроме того, Shadowbox совместим со стандартными клиентами Shadowsocks, что обеспечивает гибкость и удобство использования VPN.

- Документация: <<https://shadowsocks.org/>>

- Документация: <<https://github.com/Jigsaw-Code/outline-server/tree/master/src/shadowbox>>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|---------------|--|----------|----------|
| replicas | Количество реплик VPN-сервера. | int | 2 |
| resources | Явная конфигурация CPU и памяти для каждой реплики VPN-сервера. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | object | {} |
| resources.cpu | CPU, доступный каждой реплике. | quantity | "" |

| | | | |
|-------------------------------|--|-----------------------|----------------------|
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>external</code> | Разрешить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

Параметры приложения

| Имя | Описание | Тип | Значение |
|-----------------------------------|--|--------------------------------|-----------------|
| <code>host</code> | Хост, подставляемый в генерируемые URL. | <code>string</code> | <code>""</code> |
| <code>users</code> | Карта конфигурации пользователей. | <code>map[string]object</code> | <code>{}</code> |
| <code>users[name].password</code> | Пароль пользователя (генерируется автоматически, если не указан). | <code>string</code> | <code>""</code> |
| <code>externalIPs</code> | Список внешних IP-адресов (<code>externalIPs</code>) для сервиса. Необязательный. Если не указан, по умолчанию используется сервис LoadBalancer. | <code>[]string</code> | <code>[]</code> |

Примеры параметров и справка

`resources` и `resourcesPreset`

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют принятому в облаках соглашению об именовании <серия>.<размер>. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия включает восемь размеров (от `nano` до `4xlarge`). Старые плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов с соотношением 1:1.

Полная матрица размеров и соответствие старых имён типам приведены в [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#).

users

users:

user1:

password: hackme

user2: {} # autogenerated password

externalIPs

externalIPs:

- "11.22.33.44"

- "11.22.33.45"

- "11.22.33.46"

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Gateway API (Cilium)

<https://cozystack.ru/docs/v1.6/networking/gateway-api/>

Ingress на основе Gateway API для отдельных tenants с использованием Cilium - CRD TenantGateway, наследование через метки пространств имён, интеграция с cert-manager, терминатор и сквозная передача TLS, двухуровневая модель безопасности.

Обзор

Cozystack предоставляет поддержку Gateway API как включаемую по выбору альтернативу ingress-nginx. При включении tenant, который явно включил опцию через `tenant.spec.gateway: true`, получает собственный Gateway (собственный сервис LoadBalancer, собственный IP балансировщика, а в режимах сертификатов ACME - собственные Issuer и Certificate этого tenant), материализованный в его собственном пространстве имён. Все остальные tenants дерева публикуются через Gateway ближайшего предка, который им владеет, - та же схема, что и у существующего наследования `_namespace.ingress`.

Чарт не рендерит ресурсы Gateway, Issuer или Certificate напрямую. Вместо этого он рендерит один CR `gateway.cozystack.io/v1alpha1 TenantGateway` на каждый включивший опцию tenant, а `cozystack-controller` согласует из него все нижестоящие объекты Gateway API и cert-manager. Это устраняет гонку между Helm и контроллером за `Gateway.spec.listeners`, которую иначе вызывала бы динамическая материализация слушателей на основе маршрутов.

На этой странице описаны архитектура, модель наследования, выбор режима сертификатов (HTTP-01 по умолчанию, wildcard DNS-01 по выбору или предоставленный оператором wildcard-Secret), двухуровневая модель безопасности и порядок миграции с ingress-nginx.

Gateway API и ingress-nginx сосуществуют в одном кластере - режимы выбираются для каждого сервиса / tenant, а не глобально. Существующие кластеры обновляются с `gateway.enabled=false` и не видят изменений в поведении.

Поверхность API для tenants

Tenants в Cozystack взаимодействуют с платформой исключительно через ресурсы `apps.cozystack.io/*`, обслуживаемые `cozystack-api`; RBAC tenant не даёт права записи в `gateway.networking.k8s.io/*`, базовые Namespaces или `cozystack.io/Package`. В разделе Безопасность объясняется, как уровни допуска (admission) выстроены с учётом этого ограничения.

Архитектура

Поток согласования

flowchart TD

CHART["extra/gateway chart
(invoked by apps/tenant when
tenant.spec.gateway=true)"]

CR["TenantGateway CR
(gateway.cozystack.io/v1alpha1)"]

CTRL["cozystack-controller
(TenantGatewayReconciler)"]

GW["Gateway
(per-tenant, dynamic listeners)"]

ISS["Issuer
(per-tenant ACME account,
ACME modes only)"]

CERT["Certificate(s)
HTTP-01: per-listener
DNS-01: single wildcard + per-child
SANs
existingSecret: none minted"]

REDIR["HTTPRoute
(http->https redirect, owned)"]

NSLBL["Namespace label patching
(namespace.cozystack.io/gateway on attachedNamespaces)"]

HTR["HTTPRoute / TLSRoute
(app-owned, watched)"]

CHART --> |renders| CR

CR --> CTRL

CTRL --> |materialises| GW

CTRL --> |materialises| ISS

CTRL --> |materialises| CERT

CTRL --> |materialises| REDIR

CTRL --> |patches| NSLBL

HTR --> |hostnames feed listener set| CTRL

Контроллер:

- Материализует `Gateway`, `HTTPRoute` перенаправления, а в режимах сертификатов ACME - `Issuer` тенанта и `Certificate(s)` из `TenantGateway.spec`. В режиме `existingSecret` он не выпускает ни то, ни другое, а вместо этого указывает слушателям на `Secret`, предоставленный оператором.
- Наблюдает за ресурсами `HTTPRoute` и `TLSRoute` в масштабе кластера. Для каждого маршрута, прикреплённого к его `Gateway`, он подхватывает имена хостов и (в режиме HTTP-01) добавляет HTTPS-слушатель и `Certificate` для каждого приложения.
- В режиме DNS-01 расширяет wildcard-`Certificate` SAN-записями `<child-apex> \+ *.<child-apex>` для каждого тенанта, наследующего через этот `Gateway` (они обнаруживаются перечислением пространств имён с `namespace.cozystack.io/gateway = <owner>` и чтением их `namespace.cozystack.io/host`), и добавляет по одному HTTPS-слушателю `*.<child-apex>` на каждого наследующего потомка.
- Проставляет `namespace.cozystack.io/gateway = <owner>` на каждое пространство имён из `TenantGateway.spec.attachedNamespaces` (системные пространства имён `cozu-*`, публикуемые через `Gateway`). Патч сопровождается аннотацией `cozystack.io/gateway-attached-by`, чтобы контроллер знал, какие метки записал он сам, а какие принадлежат чарту `apps/tenant`, - метки, записанные чартом, никогда не затрагиваются. Метки, записанные контроллером, удаляются при удалении пространства имён из `attachedNamespaces`.

- Разрешает межпространственные конфликты имён хостов: пространства имён `cozy-*` (платформенные сервисы под управлением администратора кластера) выигрывают у пространств имён tenants; проигравший получает условие `HostnameConflict` под именем контроллера в `Status.Parents`.
- Отказывается молча присваивать уже существующие объекты `Gateway`, `Issuer`, `Certificate` или `HTTPRoute` перенаправления, которые носят выведенное контроллером имя, но не несут `OwnerReference` на `TenantGateway`. Операторы видят явное условие `Ready=False/ReconcileError` вместо перезаписи их вручную закреплённой конфигурации.

Путь трафика

flowchart LR

CLIENT["External client"]

LB["Cluster LB allocator
(MetalLB / Cilium LB-IPAM /
robotlb / externalIPs)"]

ENV["cilium-envoy DaemonSet
(L7 termination / L4 passthrough)"]

GW["Gateway 'cozystack'
(owning tenant namespace)"]

HTR["HTTPRoute
dashboard, keycloak, harbor, bucket, ..."]

TLR["TLSRoute
kubernetes-api, vm-exportproxy,
cdi-uploadproxy"]

CM["cert-manager
(per-tenant Issuer + Certificate(s),
ACME modes only)"]

SVC["Service
(backend)"]

CLIENT --> |DNS --> LB IP| LB

LB --> ENV

ENV --> GW

GW --> HTR

GW --> TLR

HTR --> SVC

TLR --> SVC

CM -. -> |"issues Certificate(s)"| GW

- **GatewayClass** задаётся для каждого `TenantGateway` через настраиваемое оператором поле `gatewayClassName` в чарте (по умолчанию `cilium`). У tenants нет RBAC на запись CR `TenantGateway`, поэтому они не могут выбрать класс самостоятельно.
- **Один Gateway на тенанта-владельца** в пространстве имён этого тенанта. `HTTPRoute` / `TLSRoute` каждого наследующего потомка прикрепляются к тому же `Gateway` через межпространственный `ParentRef`; слияния между `Gateway` нет.
- Envoy работает как `DaemonSet Cilium` (`cilium.envoy.enabled=true`) и обеспечивает и терминацию TLS (HTTPS-слушатели), и сквозную передачу TLS (выделенные слушатели на каждый сервис для kubeapiserver и прокси экспорта VM KubeVirt / загрузки CDI). `envoy.enabled=true` \- значение по умолчанию для новых установок Cozystack; операторам, обновляющим существующий кластер, где значения `Cilium` были заданы явно, следует

проверить, что флаг включён, прежде чем переключать `gateway.enabled`.

- IP LoadBalancer выделяется тем механизмом балансировки, который администратор кластера настроил на уровне платформы, - та же схема, что и у `ingress-nginx` сегодня.

Cozystack поставляется с установленным MetalLB, но не рендерит из чарта тенанта ни `IPAddressPool`, ни `L2Advertisement`, ни `BGPAdvertisement`, ни `CiliumLoadBalancerIPPool`.

Администраторы подключают тот аллокатор, который подходит их окружению (пул MetalLB с L2 / BGP, Cilium LB-IPAM с анонсером, [robotlb](#) поверх парка Hetzner Robot либо `Service.spec.externalIPs` как механизм ручного закрепления). API тенанта остаётся независимым от механизма - поля `gatewayIP` в CR Tenant нет. Чтобы закрепить конкретный адрес, оператор заранее создаёт сервис LoadBalancer с заданным `loadBalancerIP` либо передаёт тенанту ссылку на именованный пул под управлением администратора.

- ****externalTrafficPolicy**** : сервис LoadBalancer, стоящий за Gateway, создаётся Cilium и использует значение Kubernetes по умолчанию (`Cluster`). Поэтому исходные IP внешних клиентов транслируются (NAT) в адрес принимающего узла. Прежний путь через `ingress-nginx` ведёт себя так же всякий раз, когда задан `publishing.externalIPs` \- обычная установка на голом железе, - потому что хостовой сервис ingress в этом случае представляет собой `ClusterIP` с `spec.externalIPs` и `externalTrafficPolicy: Cluster`. Исходные IP сохраняются только тогда, когда `publishing.externalIPs` оставлен пустым: сервис становится LoadBalancer с `externalTrafficPolicy: Local`, что ограничивает IP балансировщика узлами, где размещены поды ingress. Операторы, которым нужно сохранение исходных IP для трафика Gateway API, должны пропатчить сервис самостоятельно либо поставить перед ним вышестоящий балансировщик с поддержкой протокола PROXY.

Раскладка слушателей на Gateway тенанта

Gateway тенанта всегда материализует HTTP-слушатель:

| # | Имя | Протокол | Порт | Имя хоста | Назначение |
|---|------|----------|------|----------------|---|
| 1 | http | HTTP | 80 | нет (wildcard) | ACME /.well-known/acme-challenge/* \+ HTTPRoute перенаправления HTTP->HTTPS - перенаправление HTTP->HTTPS рендерится в любом режиме сертификатов; путь проверки ACME используется только в режимах ACME |

Плюс HTTPS-слушатели, зависящие от режима сертификатов:

- Режим HTTP-01 (по умолчанию): по одному HTTPS-слушателю на каждое имя хоста прикреплённого HTTPRoute, с именем `https-<first-label>-<8-hex>`. Шестнадцатеричный суффикс - первые 32 бита `sha256(hostname)`, поэтому два разных имени хоста с одинаковой первой меткой (`harbor.foo.example.com` и `harbor.alice.example.com`) получают разные имена слушателей. `tls.certificateRefs` каждого слушателя указывает на `Certificate` этого слушателя с именем `<tgw>-<first-label>-<8-hex>-tls`, также выпускаемый автоматически.
- Режим DNS-01 (по выбору): слушатели `https (*.<owner apex>)` и `https-apex (<owner apex>)`, использующие единственный wildcard-Certificate, плюс по одному слушателю `https-child-<first-label>-<8-hex>` на apex каждого наследующего потомка (со ссылкой на тот же wildcard-сертификат, чьи `dnsNames` расширены SAN-записями `<child-apex> \+ *.<child-apex>`).
- Режим `existingSecret` (wildcard, предоставленный оператором): тот же набор слушателей, что и в DNS-01, - `https (*.<owner apex>)`, `https-apex (<owner apex>)` и по одному `https-child-<first-label>-<8-hex>` на apex каждого наследующего потомка, - за исключением того, что каждый `tls.certificateRefs` указывает на предоставленный оператором `Secret` с именем из `publishing.certificates.wildcardSecretName`, и ни для одного из них `Certificate` не выпускается.

Плюс по одному дополнительному слушателю на каждый сервис со сквозной передачей TLS (см. `TLSRoute` (TLS passthrough)).

`allowedRoutes.namespaces` слушателей использует два разных селектора в зависимости от роли слушателя:

- HTTPS-слушатели и слушатели сквозного TLS сопоставляются по метке `namespace.cozystack.io/gateway` и допускают маршруты из любого пространства имён, чья метка равна имени пространства имён тенанта-владельца (например, `tenant-root`, `tenant-alice` \- имя пространства имён, а не «голое» имя тенанта). Это точка опоры наследования - пространство имён каждого наследующего потомка несёт то же значение метки (записанное чартом `apps/tenant`), а системные пространства имён `cozy-*` из `attachedNamespaces` получают ту же метку от контроллера.
- Простой HTTP-слушатель (порт 80) использует строго более узкий белый список по встроенной метке `kubernetes.io/metadata.name` \- только само пространство имён тенанта-владельца (где живёт принадлежащий контроллеру HTTPRoute перенаправления) и `cozy-cert-manager` (HTTPRoute для проверок ACME HTTP-01). Поэтому HTTPRoute приложений, прикрепляющиеся к Gateway по имени хоста, не могут привязаться к порту 80 и отдавать незашифрованный трафик.

Port-443 listeners pin `allowedRoutes.kinds` to `HTTPRoute` and `TLSRoute` (the pair is listed on each of them so that Cilium keeps the listeners apart), preventing `GRPCRoute` / `TCPRoute` / `UDPRoute` from attaching outside the route-hostname VAP's coverage.

Включение Gateway API

Gateway API включается на двух уровнях. Оба значения по умолчанию остаются `false`; обновления не переключают tenants незаметно.

1\ Флаг на уровне платформы

Установите `gateway.enabled: true` в Package `cozystack.cozystack-platform`. Полные таблицы значений `gateway.*` и `publishing.certificates.dns01.*` см. в [справочнике Platform Package](#).

`apiVersion: cozystack.io/v1alpha1`

`kind: Package`

`metadata:`

`name: cozystack.cozystack-platform`

`spec:`

`variant: isp-full`

`components:`

`platform:`

`values:`

`publishing:`

`host: example.org`

`gateway:`

`enabled: true`

`attachedNamespaces:`

- `cozy-cert-manager`
- `cozy-dashboard`
- `cozy-keycloak`
- `cozy-system`
- `cozy-harbor`
- `cozy-bucket`
- `cozy-kubevirt`
- `cozy-kubevirt-cdi`
- `cozy-monitoring`
- `cozy-linstor-gui`
- `default`

Пространство имён `default` включено потому, что `TLSRoute` Kubernetes API (поставляемый пакетом `cozystack-api`) живёт рядом с сервисом `kubernetes`, на который он указывает, а тот всегда находится в `default`.

Включение `gateway.enabled=true` задействует три вещи:

-
- `ClusterIssuer.spec.acme.solvers` в `cert-manager` переключается с `http01.ingress.ingressClassName` на `http01.gatewayHTTPRoute`, прикрепляющийся к Gateway публикующего тенанта.
 - Шаблоны публикуемых сервисов (`dashboard`, `keycloak`, `grafana`, `alerta`) перестают рендерить свой `Ingress` и начинают рендерить свой `HTTPRoute`.
 - Сервисы со сквозной передачей TLS (`cozystack-api`, `vm-exportproxy`, `cdi-uploadproxy`) перестают рендерить свой `Ingress` и начинают рендерить `TLSRoute`, прикрепленный к выделенному слушателю `Passthrough`.

Список `attachedNamespaces` перечисляет системные пространства имён `cozy-*`, маршруты которых должны публиковаться через Gateway тенанта-владельца. Контроллер проставляет `namespace.cozystack.io/gateway = <owner>` на каждую запись, чтобы её маршруты проходили селектор `allowedRoutes` слушателя. Пространства имён тенантов (`tenant-*`) тоже могут быть в списке - они просто получают ту же метку наряду с записями `cozy-*`. Статический список не является вектором межтенантного перехвата; эту роль закрывают уровни 1, 2, 4 и 5 в разделе Безопасность.

2\ Gateway для отдельного тенанта

Тенант получает собственный CR `TenantGateway` (а через контроллер - собственные Gateway, сервис `LoadBalancer`, а в режимах ACME - собственные `Issuer` и `Certificate(s)`) только тогда, когда явно запрашивает это через `tenant.spec.gateway: true`. Все остальные тенанты дерева публикуются через Gateway ближайшего предка, который им владеет, - та же схема, что и у наследования `_namespace.ingress` сегодня. По умолчанию `gateway` не задан, что разрешается в `false` (наследовать).

Отдельный Gateway имеет смысл, когда:

- тенанту нужен собственный IP балансировщика (DNS уже закреплён за конкретным адресом, правило межсетевого экрана на этот адрес);
- apex тенанта не выводится из родительского (оператор задал собственный `tenant.spec.host`, например `customer1.example`, а не поддомен - wildcard-сертификат / `Issuer` предка не может его покрыть);
- тенант хочет собственную учётную запись ACME / `Issuer` (отдельный бюджет ограничений частоты, отдельный удостоверяющий центр).

В остальных случаях оставьте `gateway` незадаанным и наследуйте.

Тенант 'alice' под `tenant-root`: apex выводится как `alice.<parent apex>`,

наследует Gateway родителя. Отдельного IP балансировщика и `Issuer` нет.

`apiVersion: apps.cozystack.io/v1alpha1`

`kind: Tenant`

`metadata:`

`name: alice`

`namespace: tenant-root`

spec: {}

```
# Тенант 'acme' с полностью независимым apex: должен включить опцию, чтобы
# владеть Gateway, ведь сертификат/Issuer родителя не покрывает customer1.example.
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
  name: acme
  namespace: tenant-root
spec:
  host: customer1.example
  gateway: true
```

```
# Тенант 'bob' под tenant-root: выведенный apex, но хочет собственные
# IP балансировщика и учётную запись ACME (DNS закреплён за конкретным адресом).
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
  name: bob
  namespace: tenant-root
spec:
  gateway: true
```

Установка собственного значения `tenant.spec.host` зарезервирована за администраторами кластера и сервисными учётными записями `cozystack/Flux` (обеспечивается во время выполнения политикой `cozystack-tenant-host-policy`, см. Безопасность).

Наследование

Чарт `apps/tenant` записывает `namespace.cozystack.io/gateway: <owner-namespace>` на каждое пространство имён тенанта, где значение - либо имя собственного пространства имён этого тенанта (когда `gatewayEffective` разрешается в `true`), либо имя пространства имён наследуемого предка (при наследовании). То же значение попадает в `_namespace.gateway` внутри секрета `cozystack-values` тенанта, поэтому вендоренные приложения (`harbor`, `bucket`, ...) рендерят свои `HTTPRoute` с `parentRefs.namespace`, указывающим на пространство имён владельца.

Чтобы проверить, через какой Gateway сейчас наследуется данное пространство имён тенанта:

```
kubectl get namespace <tenant-ns> \
-o jsonpath='{.metadata.labels.namespace\.cozystack\.io/gateway}'"
```

Пустое значение означает, что ни у одного предка в цепочке нет `tenant.spec.gateway: true`, и маршруты в этом пространстве имён не прикрепятся ни к какому Gateway.

Селектор `allowedRoutes.namespaces.selector` слушателя Gateway-владельца сопоставляется ровно с этой меткой, поэтому один и тот же селектор допускает маршруты из каждого пространства имён дерева владельца - как потомков, так и записей `cozy-*` из `attachedNamespaces`. Отдельного `ReferenceGrant` на каждого потомка нет: селектор по метке и есть межпространственный шлюз.

В режиме DNS-01 контроллер расширяет wildcard-Certificate Gateway-владельца SAN-записями `<child-apex> \+ *.<child-apex>` для каждого наследующего потомка (они обнаруживаются перечислением пространств имён с тем же значением `namespace.cozystack.io/gateway` и чтением их `namespace.cozystack.io/host`) и добавляет HTTPS-слушатель `*.<child-apex>` на каждый apex потомка. Без этого расширения одноуровневый wildcard родителя не может покрыть имя хоста маршрута потомка (`harbor.alice.example.org` на две метки глубже родительского `*.example.org`).

Проверка ACME DNS-01 должна пройти для каждой SAN-записи, а значит, настроенная учётная запись DNS-провайдера должна уметь записывать TXT-записи под каждым уровнем apex, который обслуживает родитель. Для глубоко вложенных наследующих потомков это требует либо делегирования зоны, либо учётных данных провайдера с правами на все уровни apex. Режим HTTP-01 не затронут - каждая проверка на отдельный слушатель выполняется для конкретного имени хоста.

Тенант, включивший опцию собственного Gateway, становится отдельной границей: отдельный Gateway, а в режимах сертификатов ACME - отдельные `Issuer`, учётная запись ACME и `Certificate(s)`, собственное подмножество наследующих потомков. Дочерние тенанты под ним не разделяют состояние проверок HTTP-01 с прародителем.

Режим сертификатов: HTTP-01 (по умолчанию), DNS-01 (по выбору) или существующий Secret

`publishing.certificates.solver` управляет тем, откуда `Issuer` тенанта берёт сертификаты TLS, - но только на двух путях ACME. Установка `publishing.certificates.wildcardSecretName` выбирает третий режим, `existingSecret`, в котором тенант отдаёт предоставленный оператором wildcard-Secret, а контроллер вообще не выпускает `Issuer`; настройки солвера, провайдера DNS-01 и издателя при этом пропускаются. См. Сертификаты ниже и [справочник Platform Package](#) для полного набора ключей провайдеров `publishing.certificates.dns01.*`.

HTTP-01 (по умолчанию)

Работает из коробки, дополнительная настройка не требуется. Контроллер:

- Рендерит ACME `Issuer` в пространстве имён тенанта с солвером `http01.gatewayHTTPRoute`, указывающим на собственный Gateway тенанта / слушатель `http`.
- Наблюдает за `HTTPRoute` / `TLSRoute`, прикрепленными к Gateway (с `parentRefs` на него). Для каждого нового имени хоста он добавляет HTTPS-слушатель и `Certificate` этого приложения (`dnsNames` содержит ровно это имя хоста).

- Именованние слушателей приложений: `https-<first-label>-<8-hex>` (например, `https-harbor-deadbeef`).
- Именованние сертификатов приложений: `<tgw>-<first-label>-<8-hex>-tls`.

Добавление приложения тенанта - будь то под тенантом-владельцем или под любым наследующим потомком - сводится к развёртыванию его HTTPRoute. Правки платформенного Package не нужны. Платформенные сервисы cozy-* (dashboard, keycloak, grafana, alerta, cozystack-api, vm-exportproxy, cdi-uploadproxy) по-прежнему управляются `publishing.exposedServices`, как и в схеме с ingress, - при `gateway.enabled=true` свой HTTPRoute / TLSRoute рендерят только сервисы из этого списка.

DNS-01 (по выбору)

Установите `publishing.certificates.solver: dns01` и выберите провайдера:

| <code>publishing.certificates.dns01.provider</code> | Чарт проверяет заранее | Оператор должен предоставить |
|---|--|---|
| cloudflare (по умолчанию) | (ничего - чарт никогда не падает) | Secret с именем из <code>cloudflare.secretName</code> (по умолчанию <code>cloudflare-api-token-secret</code>), содержащий API-токен Cloudflare под ключом из <code>cloudflare.secretKey</code> (по умолчанию <code>api-token</code>) |
| route53 | <code>route53.region</code> (чарт падает на рендере, если пусто) | Либо IRSA / instance profile, либо <code>route53.secretName</code> , указывающий на Secret с секретным ключом доступа IAM под <code>route53.secretKey</code> (по умолчанию <code>secret-access-key</code>); опционально <code>route53.accessKeyID</code> |
| digitalocean | (ничего) | Secret с именем из <code>digitalocean.secretName</code> (по умолчанию <code>digitalocean-api-token-secret</code>), содержащий API-токен DigitalOcean под <code>digitalocean.secretKey</code> (по умолчанию <code>access-token</code>) |

| | | |
|---------|---|---|
| rfc2136 | rfc2136.nameserver (чарт падает на рендере, если пусто) | rfc2136.tsigKeyName и rfc2136.secretName; Secret содержит материал ключа TSIG под rfc2136.secretKey (по умолчанию tsig-secret-key); rfc2136.tsigAlgorithm по умолчанию HMACSHA256 |
|---------|---|---|

Чарт вызывает `fail()` на этапе рендера только для ключей из второго столбца; остальное проверяется `cert-manager` во время проверки ACME, поэтому неправильно настроенный провайдер даёт `Challenge`, застрявший в `Pending`, а не ошибку рендера чарта.

Режим DNS-01 рендерит один `wildcard-Certificate`, покрывающий `<owner apex>` и `*.<owner apex>`, плюс соответствующие слушатели `https (*.<owner apex>)` и `https-apex (<owner apex>)`. Новые приложения, публикуемые под `apex`, используют существующий `wildcard-сертификат` без выпуска отдельного сертификата на слушатель. Для каждого наследующего дочернего тенанта контроллер расширяет `dnsNames wildcard-сертификата` SAN-записями `<child-apex> \+ *.<child-apex>` и добавляет слушатель `*.<child-apex>`.

Платформенный чарт записывает конфигурацию провайдера в ключи `_cluster.dns01-*`, которые используются и чартом `gateway` отдельного тенанта (рендерящим `CR TenantGateway`), и общекластерными `ClusterIssuer letsencrypt-prod / letsencrypt-stage`, применяемыми в прежней схеме с `ingress`. Оба пути согласованы в том, какой провайдер активен.

Выбирайте DNS-01, когда вам нужен именно `wildcard-сертификат` - долгоживущий кластер со множеством приложений под одним `apex`, глубокие деревья наследования или жёсткие ограничения частоты Let's Encrypt. Gateway API ограничивает `Gateway.spec.listeners` 64 записями; HTTP-01 добавляет по одному HTTPS-слушателю на каждое публикуемое имя хоста (плюс обязательный слушатель `http` и слушатели сквозного TLS), поэтому развёртывание с одним тенантом, приближающееся к 60+ опубликованным приложениям на HTTP-01, упрётся в лимит, и отрендеренный `Gateway` не пройдёт допуск. DNS-01 сворачивает все имена хостов под `apex` в небольшое фиксированное число слушателей.

existingSecret (wildcard, предоставленный оператором)

Установите `[publishing.certificates.wildcardSecretName]` (<https://cozystack.ru/docs/v1.6/operations/configuration/platform-package/>) \- и тенант полностью уходит от ACME: `TenantGateway` указывает своим слушателям на этот заранее созданный `Secret`, а контроллер не выпускает ни `Issuer`, ни `Certificate`. Настройки солвера, провайдера DNS-01 и издателя на этом пути игнорируются. Раскладка слушателей совпадает с DNS-01, поэтому лимит в 64 слушателя обходится тем же способом.

Выбирайте его, когда сертификаты выпускаются вне кластера, - корпоративный УЦ, уже имеющийся `wildcard` или терминирующий балансировщик, который уже им располагает. Прочитайте Сертификаты перед включением: имя `Secret` доходит до каждого тенанта, а на пути `ingress` по умолчанию дочерний тенант с собственным контроллером `ingress` остаётся с самоподписанным сертификатом ([cozystack/cozystack#3296](https://cozystack.ru/docs/v1.6/operations/configuration/platform-package/#3296)).

Маршрутизация по сервисам

При `gateway.enabled=true` следующие сервисы переключаются с Ingress на ресурсы Gateway API. Столбец Условие рендера отличает сервисы, которые всегда рендерят свой маршрут при включённом флаге платформы, от тех, которым дополнительно нужна запись в `publishing.exposedServices` (и от приложений тенантов, зависящих от заполненности `_namespace.gateway`).

HTTPRoute (терминация TLS на Gateway)

| Сервис | Пространство имён | Имя HTTPRoute | Бэкенд | Слушатель | Условие рендера |
|-----------|---------------------------|------------------|-----------------------------|---|---|
| dashboard | cozy-dashboard | dashboard | incloud-web-gatekeeper:8000 | свой https-dashboard-... (HTTP-01) или https (DNS-01) | <code>gateway.enabled</code> И <code>dashboard</code> в <code>publishing.exposedServices</code> |
| keycloak | cozy-keycloak | keycloak | keycloak-http:80 | так же | <code>gateway.enabled</code> |
| grafana | cozy-monitoring | grafana | grafana-service:3000 | так же | <code>gateway.enabled</code> |
| alerta | cozy-monitoring | alerta | alerta:80 | так же | <code>gateway.enabled</code> |
| harbor | пространство имён тенанта | <release-name> | <release-name>:80 | Gateway тенанта-владельца | задан <code>_namespace.gateway</code> (любой предок включил опцию) |
| bucket | пространство имён тенанта | <bucket-name>-ui | <bucket-name>-ui:8080 | Gateway тенанта-владельца | задан <code>_namespace.gateway</code> |

Солвер HTTP-01 cert-manager размещает свой короткоживущий HTTPRoute на слушателе http того же Gateway с сопоставлением по пути `/.well-known/acme-challenge/`. Более специфичное сопоставление пути выигрывает у общего HTTPRoute перенаправления HTTP->HTTPS.

TLSRoute (TLS passthrough)

Сервисы, которым нужна сквозная передача на основе SNI (клиенты предъявляют сертификаты, TLS терминируется на бэкенде), используют TLSRoute на выделенном слушателе Passthrough. По одному слушателю на сервис, имя хоста ограничено FQDN этого сервиса:

| Сервис | Пространство имён | Имя TLSRoute | Бэкенд | Слушатель | Условие рендера |
|--------|-------------------|--------------|--------|-----------|-----------------|
|--------|-------------------|--------------|--------|-----------|-----------------|

| | | | | | |
|-----------------------|-------------------|-----------------|---------------------|---------------------|--|
| Kubernetes API | default | kubernetes-api | kubernetes:443 | tls-api | gateway.enabled И api В publishing.exposedServices |
| Экспорт ВМ KubeVirt | cozy-kubevirt | vm-exportproxy | vm-exportproxy:443 | tls-vm-exportproxy | gateway.enabled И vm-exportproxy В publishing.exposedServices |
| Загрузка CDI KubeVirt | cozy-kubevirt-cdi | cdi-uploadproxy | cdi-uploadproxy:443 | tls-cdi-uploadproxy | gateway.enabled И cdi-uploadproxy В publishing.exposedServices |

Все три слушателя Passthrough (tls-api, tls-vm-exportproxy, tls-cdi-uploadproxy) рендерятся на Gateway всегда - контроллер материализует по одному на каждую запись в значении чарта `tlsPassthroughServices` (по умолчанию: [api, vm-exportproxy, cdi-uploadproxy]). `publishing.exposedServices` на самом деле управляет соответствующим шаблоном `TLSTRoute` в каждом вышестоящем чарте: если сервис удалён из `publishing.exposedServices`, его слушатель по-прежнему существует, но к нему ничего не прикрепляется, и трафик не допускается.

`TLSTRoute` is shipped from the Gateway API experimental channel (CRD `gateway.networking.k8s.io/v1alpha2`) in v1.5.x. It graduates to v1 upstream; Cozystack will follow the rename when it lands.

Безопасность

Тенанты в Cozystack взаимодействуют с платформой исключительно через ресурсы `apps.cozystack.io/*` (Tenant, Bucket, Kubernetes, ...), обслуживаемые `cozystack-api`. RBAC тенанта (`cozy:tenant:*`, агрегируемый в `RoleBinding` в собственном пространстве имён тенанта) не даёт права записи в `gateway.networking.k8s.io/*`, базовые `Namespaces` или `cozystack.io/Package`. Описанные ниже защиты делятся на две группы по тому, от кого они защищают: большинство из пяти уровней не защищают от ввода пользователя-тенанта (соответствующий RBAC ему изначально не выдан); они защищают от ошибок в `cozystack-controller` / Flux, компрометации цепочки поставки чарта приложения и ошибок типа «сбитый с толку помощник» со стороны администратора кластера. Все проверки на этапе допуска работают по принципу fail-closed (`failurePolicy: Fail, validationActions: [Deny]`).

Шлюз для ввода пользователя-тенанта \- уровень 3 (`cozystack-tenant-host-policy`). `Tenant.spec.host` \- то самое задаваемое пользователем поле, которое проявляется как граница безопасности на уровне имён хостов; оно проверяется при каждом Create / Update через цепочку допуска `cozystack-api`.

Эшелонированная защита \- уровни 1, 2, 4, 5. Современная модель угроз - ошибки чартов, ошибки контроллера, компрометация цепочки поставки чарта приложения и ошибки администратора кластера; у tenants нет RBAC для прямой записи Gateway или HTTPRoute. Если этот RBAC когда-нибудь расширится (будущий RoleBinding, агрегированная через CRD роль, включающая `gateway.networking.k8s.io/*`, чарт приложения, выдающий своей ServiceAccount права записи маршрутов), эти уровни продолжают применять те же ограничения имён хостов к новому вызывающему - они не обходятся вводом tenanta, просто сейчас им не задействуются.

Записи `tenant-*` в `gateway.attachedNamespaces` разрешены намеренно: вектор межtenantного перехвата - это селектор меток слушателя (закрытый уровнями 1, 2, 4 и 5), а не статический список прикрепления, поэтому пространства имён `tenant-*` в списке просто получают метку прикрепления к gateway наряду с записями `cozy-*`.

flowchart TD

USER["Tenant user
(apps.cozystack.io/* via cozystack-api)"]

CHART["App chart bug /
supply-chain compromise"]

L3["L3 VAP: Tenant spec.host writes
restricted to trusted callers"]

L1["L1: Listener allowedRoutes selector
(HTTPS: namespace.cozystack.io/gateway;
HTTP-80: kubernetes.io/metadata.name)"]

L2["L2 VAP: Gateway listener hostname
matches namespace.cozystack.io/host"]

L4["L4 VAP: namespace.cozystack.io/host label
writes restricted to trusted callers"]

L5["L5 VAP: HTTPRoute/TLSRoute hostnames
match namespace label (tenant-* only)"]

GW["Cross-tenant hostname hijack
BLOCKED"]

USER --> |Tenant spec.host| L3

L3 --> GW

CHART --> |emits Gateway/HTTPRoute| L1

CHART --> L2

CHART --> L4

CHART --> L5

L1 --> GW

L2 --> GW

L4 --> GW

L5 --> GW

Уровень 1 - селектор пространств имён `allowedRoutes` слушателя

Каждый слушатель на Gateway tenanta закрепляет `allowedRoutes.namespaces.from: Selector`. Механика селектора различается по роли слушателя:

-
- HTTPS-слушатели и слушатели сквозного TLS используют `matchLabels: { namespace.cozystack.io/gateway: <owner-namespace> }`. Значение метки - пространство имён TenantGateway: для `tenant-root` это `tenant-root`, для `tenant-alice` \- `tenant-alice` (то есть имя пространства имён, а не «голое» имя тенанта). Метку записывает чарт `apps/tenant` на каждое пространство имён тенанта (имя собственного пространства имён при владении Gateway, имя пространства имён наследуемого предка в остальных случаях) и предоставляет контроллер на каждое пространство имён из `attachedNamespaces`. Пространства имён без совпадающего значения не могут прикрепить к этим слушателям ни один `HTTPRoute` / `TLSRoute`.
 - Простой HTTP-слушатель (порт 80) использует строго более узкий белый список `matchExpressions` по встроенной метке `kubernetes.io/metadata.name` \- только собственное пространство имён тенанта-владельца (где живёт принадлежащий контроллеру `HTTPRoute` перенаправления) и `cozy-cert-manager` (`HTTPRoute` проверок ACME HTTP-01). Поэтому `HTTPRoute` приложений, прикрепляющиеся по имени хоста, не могут привязаться к порту 80 и незаметно отдавать незашифрованный трафик.

Port-443 listeners pin `allowedRoutes.kinds` to `HTTPRoute` and `TLSRoute` — the pair is listed on each so Cilium keeps them apart — preventing `GRPCRoute` / `TCPRoute` / `UDPRoute` from attaching outside the Layer 5 VAP's coverage.

Уровень 2 - cozystack-gateway-hostname-policy

`ValidatingAdmissionPolicy`, ограниченная операциями `CREATE/UPDATE` Gateway группы `gateway.networking.k8s.io` в версиях `v1` и `v1beta1` (так что кластер, всё ещё обслуживающий Gateway `v1beta1`, тоже покрыт). CEL читает `namespaceObject.metadata.labels["namespace.cozystack.io/host"]` и отклоняет любой слушатель, чьё имя хоста не равно этому значению и не является его поддоменом. `matchConditions` ограничивают VAP только пространствами имён `tenant-*` \- Gateway в посторонних пространствах имён (например, `kube-system`) не затрагиваются.

Поскольку VAP читает метку пространства имён (а не общекластерный `ConfigMap`), тенант с полностью независимым арех-доменом (например, `customer1.example`, не поддомен арех платформы) валидируется корректно - VAP не предполагает иерархию поддоменов.

Уровень 3 - cozystack-tenant-host-policy

`ValidatingAdmissionPolicy`, ограниченная операциями `CREATE/UPDATE` `apps.cozystack.io/v1alpha1` Tenant. Отклоняет установку или изменение `spec.host`, если вызывающий не входит в группу `system:masters` и не является одним из `system:serviceaccounts:cozy-system`, `system:serviceaccounts:cozy-cert-manager`, `system:serviceaccounts:cozy-fluxcd`, `system:serviceaccounts:kube-system`. Тенанты по-прежнему могут создавать тенантов с пустым `spec.host` (обычный поток наследования).

Это закрывает путь, при котором пользователь-тенант создаёт Tenant с `spec.host=dashboard.example.org`, чтобы чарт тенанта записал перехваченную метку в его пространство имён.

`cozystack-api` \- это агрегированный APIServer с собственной реализацией. REST-обработчик CR Tenant в `pkg/registry/apps/application/rest.go` явно вызывает колбэки `createValidation / updateValidation / deleteValidation` в `Create / Update / Delete` - в отличие от `genericregistry.Store`, пользовательские REST-обработчики должны подключать эти хуки сами. С подключёнными хуками каждая `ValidatingAdmissionPolicy` и `ValidatingWebhook`, ограниченная `apps.cozystack.io/*`, срабатывает на всех трёх операциях, как того требует контракт `apiserver`.

Уровень 4 - `cozystack-namespace-host-label-policy`

`ValidatingAdmissionPolicy`, ограниченная операциями CREATE/UPDATE базового `v1 Namespace`. Считает `namespace.cozystack.io/host` фактически неизменяемой: отклоняет любое изменение значения (включая переход от пустого к заданному, что покрывает первые записи и на CREATE, и на UPDATE), если вызывающий не входит в тот же белый список доверенных вызывающих, что и на уровне 3. Идемпотентные повторные применения того же значения разрешены любому вызывающему - фактическое сообщение об ошибке CEL («namespace label namespace.cozystack.io/host is immutable once set») это отражает. Устанавливать или менять значение метки могут только сервисные учётные записи `cozystack/Flux` (которые применяют чарт тенанта).

В сочетании с уровнем 3 пользователь-тенант не может установить или изменить свой хост ни через CR Tenant, ни через метку пространства имён.

Уровень 5 - `cozystack-route-hostname-policy (HTTPRoute)` и `cozystack-route-hostname-policy-tls (TLSRoute)`

Пара объектов `ValidatingAdmissionPolicy` с одинаковым CEL-выражением. `cozystack-route-hostname-policy` нацелена на CREATE/UPDATE `HTTPRoute` группы `gateway.networking.k8s.io (v1 и v1beta1)`; `cozystack-route-hostname-policy-tls` \- на `TLSRoute` в `v1alpha2`. Обе ограничены пространствами имён `tenant-*` (`cozy-*` управляются администратором кластера, и им доверено публиковаться под любым apex) и отклоняют любую запись `spec.hostnames`, которая не равна метке `namespace.cozystack.io/host` пространства имён и не является её поддоменом. Fail-closed при отсутствии метки \- пространство имён `tenant-*` без `namespace.cozystack.io/host` отклоняется, а не молча пропускается. Операторы, выполняющие `kubectl get validatingadmissionpolicy`, увидят оба объекта.

Это эшелонированная защита от ошибки чарта приложения или компрометации цепочки поставки, генерирующей ресурсы Gateway API вне apex тенанта, - тенанты в Cozystack по замыслу не имеют RBAC `gateway.networking.k8s.io/*`, так что это не защита от пользователя-тенанта. Внутриапекс-ный межпространственный случай (чарт тенанта, претендующий на имя хоста, принадлежащее приложению `cozy-*`) обрабатывается контроллером на этапе согласования - см. Разрешение `HostnameConflict` ниже.

Допустимый суффикс хоста - всегда значение метки `namespace.cozystack.io/host` самого пространства имён: у уровня 5 нет особого случая для `tenant-root` и нет жёстко заданного правила вывода. Что бы чарт `apps/tenant` ни записал в эту метку (выведенное `<name>.<parent apex>` для наследующих потомков, `publishing.host` кластера для `tenant-root`, заданный оператором `tenant.spec.host` для тенантов с собственным apex), - именно этим должен оканчиваться каждый

маршрут в этом пространстве имён. Тенант с независимым apex (`customer1.example` вместо поддомена) обрабатывается корректно, потому что VAP читает метку, а не предполагает иерархию поддоменов.

Разрешение HostnameConflict

Когда два маршрута из разных пространств имён претендуют на одно имя хоста, контроллер выбирает победителя детерминированно:

- Маршрут из пространства имён `cozy-*` (платформенный сервис под управлением администратора кластера) выигрывает у маршрута из любого другого пространства имён.
- В пределах одного приоритетного уровня выигрывает маршрут с лексикографически наименьшей парой `<namespace>/<name>`.

Проигравший маршрут получает `Accepted=False` с `Reason=HostnameConflict` в `Status.Parents` под именем контроллера (`gateway.cozystack.io/tenantgateway-controller`). Записи статуса других контроллеров (Cilium и т.д.) не затрагиваются.

Защита от захвата чужих объектов

Шесть путей согласования отказываются молча перезаписывать или присваивать уже существующее состояние, которое носит выведенное контроллером имя / аннотацию, но не происходит от этого `TenantGateway`:

- `Gateway` (названный по имени `TenantGateway`)
- `HTTPRoute` перенаправления (`<tgw>-http-redirect`)
- `Issuer` тенанта (`<tgw>-gateway`, только режимы сертификатов ACME)
- `wildcard-Certificate` (`<tgw>-gateway-tls`, режим DNS-01)
- `Certificate` на слушатель (`<tgw>-<first-label>-<8-hex>-tls`, режим HTTP-01)
- Метка пространства имён `namespace.cozystack.io/gateway` - контроллер записывает или снимает эту метку только на тех пространствах имён, которым он проставляет аннотацию `cozystack.io/gateway-attached-by`. Метки, записанные чартом `apps/tenant` (без аннотации), никогда не затрагиваются, поэтому наследование для пространств имён тенантов переживает любое согласование.

Для путей с именованными объектами оператор, вручную закрепивший `Certificate` или `Issuer` под выведенным контроллером именем (частный УЦ, ручное закрепление сертификата, внутренний ACME), получает явное условие `Ready=False/ReconcileError` на `TenantGateway` вместо молчаливого уничтожения его конфигурации и перевыпуска ресурса из другой учётной записи ACME. Сообщение об ошибке указывает на конфликтующий объект, чтобы оператор мог либо удалить его (передав владение контроллеру), либо переименовать.

От чего это НЕ защищает

Эти остаточные риски - осознанные проектные решения, а не пробелы в реализации:

-
- Учётные данные администратора кластера. Любой, кто входит в `system:masters` или располагает подходящей `ServiceAccount` `cozystack/Flux`, может задать любой хост. Изоляция Gateway API - не самое слабое звено на этом уровне доверия.
 - Контроль над DNS. Тенант, чьё разрешённое VAP имя хоста не резолвится в IP балансировщика кластера, не сможет пройти ACME HTTP-01. Certificate не выпускается; перехвата не происходит, даже если допуск каким-то образом пропустил Gateway. Основанное на DNS доказательство владения в ACME - последний рубеж. В режиме `existingSecret` этого уровня нет: ничего не выпускается, а значит, ничто не доказывает контроль над доменом; Secret оператора принимается на доверии в том виде, в каком предоставлен.
 - Общий аллокатор балансировщика. Несколько тенантов-владельцев, берущих адреса из одного пула под управлением администратора (MetalLB, Cilium LB-IPAM и т.д.), конкурируют за адреса по правилам этого аллокатора. Уникальность IP на сервис - ответственность аллокатора, как и для любого другого сервиса LoadBalancer в кластере.

Сертификаты

В двух режимах ACME каждый тенант с `spec.gateway: true` получает собственный `Issuer` `cert-manager` (ограниченный пространством имён, а не `ClusterIssuer`) с именем `<tgw>-gateway`. `Issuer` несёт собственную учётную запись ACME через `privateKeySecretRef: <tgw>-acme-account`. Certificate ссылаются на `issuerRef.kind: Issuer, name: <tgw>-gateway`.

В режиме HTTP-01 \- по одному Certificate на имя хоста опубликованного приложения (с именем `<tgw>-<first-label>-<8-hex>-tls`). В режиме DNS-01 один wildcard-Certificate (с именем `<tgw>-gateway-tls`) покрывает `<owner apex>` и `*.<owner apex>`, плюс SAN-записи на `apex` каждого потомка (`<child-apex>` и `*.<child-apex>`) для каждого наследующего тенанта.

Both of the above are ACME modes. Setting `[publishing.certificates.wildcardSecretName]`(<https://cozystack.ru/docs/v1.6/operations/configuration/platform-package/>) selects a third mode, `existingSecret`: the `TenantGateway` references the operator-provided Secret directly, and the controller mints no per-tenant `Issuer` and no `Certificate` — the solver, DNS-01 provider, and issuer settings are skipped on this path, and any `Issuer` or `Certificate` it previously owned is garbage-collected. The ACME account private key (`<tgw>-acme-account`) is NOT removed on a mode switch — only the `Issuer` referencing it. It is cleaned up by a post-delete hook when the tenant's gateway release is removed, not when the mode changes. The cluster-wide `letsencrypt-prod` / `letsencrypt-stage` `ClusterIssuers` are unaffected: they are rendered from `publishing.certificates.solver` regardless, and still validate their DNS-01 provider settings at render time. The listener shape matches DNS-01: one `*.<apex>` listener, one `<apex>` listener, and one `*.<child-apex>` listener per inheriting child, all pointing at that one Secret.

Покрывание сертификатами для потомков далее распадается на три случая с разными режимами отказа и разными способами устранения. Первые два относятся к пути Gateway; третий относится к пути ingress-nginx по умолчанию и является открытым багом, а не ограничением, которое можно обойти настройками:

- Наследующие потомки (по умолчанию, без `spec.gateway`) не имеют собственного Gateway. Их слушатель `*.<child-apex>` рендерится на Gateway владельца и привязывается к Secret в

пространстве имён `_владелец_`, поэтому покрытие целиком зависит от списка SAN этого Secret: голый `*.<apex>` не соответствует `*.<child-apex>`, и клиентам поддомена потомка отдаётся сертификат владельца, а они видят несовпадение имени хоста. Реплицирование Secret в пространство имён потомка здесь ничего не исправит, потому что его никакой слушатель не читает; SAN-записи должны покрывать apex каждого потомка.

- Потомки, владеющие Gateway (`spec.gateway: true`), рендерят собственный TenantGateway, наследуют имя Secret через канал значений кластера и разрешают его в своём собственном пространстве имён. Gateway API действительно позволяет слушателю ссылаться на Secret в другом пространстве имён через `ReferenceGrant`, но контроллер этим путём не пользуется: он рендерит `certificateRefs` без пространства имён и не выпускает `ReferenceGrant`, поэтому ссылка всегда локальна. Здесь Secret действительно необходимо реплицировать, иначе арендатор останется без сертификата.

- Потомки на пути ingress-nginx (`gateway.enabled=false`, по умолчанию) страдают сильнее всего. Имя Secret доходит до каждого арендатора, и шаблоны ingress приложений и системных сервисов убирают свою аннотацию ACME на каждый хост, как только она непуста, - но чарт ingress передаёт `default-ssl-certificate` только `_публикующему_` контроллеру. Поэтому потомок, запускающий собственный контроллер ingress (`ingress: true`), остаётся без того и без другого: у его приложений нет собственного сертификата, а у его контроллера нет сертификата по умолчанию, так что ingress-nginx отдаёт встроенный самоподписанный сертификат для каждого хоста этого арендатора. Реплицирование Secret не помогает - флаг привязан к пространству имён, а не к Secret. Это отслеживается как [cozystack/cozystack#3296](https://github.com/cozystack/cozystack/issues/3296); пока баг не исправлен, не устанавливайте `wildcardSecretName` в кластере, дочерние арендаторы которого запускают собственные контроллеры ingress.

Именно поэтому режим `_поддерживается_` только для корневого арендатора. Ничто не обеспечивает эту границу: имя Secret доходит до каждого арендатора по тому же каналу значений, поэтому включение режима в кластере, где уже есть потомки, владеющие Gateway, переводит на `existingSecret` и их.

Из коробки поддерживаются два сервера ACME:

```
- publishing.certificates.issuerName: letsencrypt-prod ->  
https://acme-v02.api.letsencrypt.org/directory  
- publishing.certificates.issuerName: letsencrypt-stage ->  
https://acme-staging-v02.api.letsencrypt.org/directory
```

Любое другое значение приводит к ошибке рендера чарта. Чтобы поддержать нового провайдера ACME: добавьте константу `letsencrypt*Server` (или аналогичную) рядом с существующими в `internal/controller/tenantgateway/reconciler.go`, затем добавьте ветку в `acmeServerForIssuer` в `internal/controller/tenantgateway/renderers.go`, сопоставляющую имя издателя с этой константой.

Ограничения частоты

Let's Encrypt применяет квоты на учётную запись и на зарегистрированный домен:

- 50 новых сертификатов на зарегистрированный домен в неделю
- 5 дублирующих сертификатов в неделю для одного и того же набора имён хостов

- 300 новых заказов на учётную запись за 3 часа

Установка `publishing.certificates.wildcardSecretName` полностью обходит квоты - этот режим вообще не выпускает сертификаты ACME - ценой оговорок из раздела Сертификаты. Кластер, где множество tenants делят один apex-домен, может исчерпать их быстро, особенно в режиме HTTP-01, где каждое опубликованное приложение добавляет по одному сертификату. Меры смягчения:

- `publishing.certificates.issuerName: letsencrypt-stage` для непроекционных кластеров (квоты staging не влияют на prod).
- `tenant.spec.resourceQuotas.count/certificates.cert-manager.io`, чтобы ограничить создание сертификатов на tenant.
- Переход на DNS-01, чтобы объединить приложения каждого tenant под одним wildcard-сертификатом (сокращает количество сертификатов с N приложений до 1 на tenant-владельца; наследующие потомки сворачиваются в wildcard родителя через расширение SAN).
- Для изолированных развёртываний используйте поставляемый `selfsigned-cluster-issuer` или внутренний сервер ACME.

Рекомендуемая квота на уровне tenant, сдерживающая tenant, ведущего себя некорректно:

`apiVersion: apps.cozystack.io/v1alpha1`

`kind: Tenant`

`spec:`

`gateway: true`

`resourceQuotas:`

`count/certificates.cert-manager.io: "10"`

Миграция с ingress-nginx

Два режима сосуществуют. Переключение выполняется на уровне кластера (`gateway.enabled`) и на уровне tenant (`tenant.spec.gateway`), а не глобально.

Для нового кластера

Установите `gateway.enabled: true` при установке. Ingress-nginx можно полностью отключить, когда каждый tenant-владелец получит `spec.gateway: true` и каждое опубликованное приложение под этими tenants мигрирует.

Tenants- владельцы затем объявляют `spec.gateway: true` при создании. Их потомки наследуют через метку пространства имён без явного включения.

Для существующего кластера

Порядок важен - переключение `gateway.enabled: true` до того, как появился хоть один Gateway, вызывает реальный простой публикуемых сервисов под управлением платформы. HTTPRoute для `cozu-*` начинают рендериться, а соответствующие Ingress удаляются, но Gateway, на который они ссылаются через ParentRef, ещё не существует, поэтому внешний трафик к dashboard / keycloak / Kubernetes API / экспорту VM / загрузке CDI отбрасывается до тех пор, пока Gateway не станет `Programmed`, а в режимах сертификатов ACME - пока его Certificate не станут `Ready`. Сначала выполните включение опции на уровне тенантов, потом переключайте платформу.

Каждый TenantGateway тенанта, отрендеренный из него Gateway, а также (в режимах сертификатов ACME) его Issuer и - в режиме DNS-01 - его wildcard-Certificate выводятся из фиксированного имени: чарт жёстко задаёт TenantGateway как `cozystack`, а контроллер выводит из него `cozystack-gateway` (Issuer), `cozystack-gateway-tls` (wildcard-сертификат DNS-01) и `cozystack-http-redirect` (маршрут перенаправления HTTP->HTTPS). Все команды kubectl ниже используют эти буквенные имена независимо от того, какой тенант владеет Gateway.

1. Для каждого тенанта, который должен владеть Gateway (обычно как минимум `tenant-root`), установите `tenant.spec.gateway: true`. Чарт тенанта материализует CR TenantGateway, а контроллер согласует Gateway и, в режимах сертификатов ACME, Issuer и Certificate(s). Потомки тенанта-владельца подхватывают Gateway родителя автоматически через метку пространства имён.

2. Дождитесь Gateway, а в режимах сертификатов ACME - и Certificate. Gateway всегда носит имя `cozystack`:

```
kubectl -n <owner-tenant-ns> wait gateway/cozystack --for=condition=Programmed --timeout=5m
```

`Programmed` доказывает меньше, чем кажется. Cilium выставляет его, как только Gateway назначен адрес и принят хотя бы один слушатель, - это не проверка на каждый слушатель. Слушатель, который не может разрешить свой сертификат, помечается `ResolvedRefs=False`, выпадает из набора принятых, а Gateway остаётся `Programmed`, пока остаётся хоть один слушатель, - а простой слушатель `http` на порту 80 остаётся всегда. Поэтому `Programmed=True` говорит вам, что балансировщик выдал Gateway адрес; о том, нашёл ли хоть один HTTPS-слушатель свой Secret, он не говорит ничего.

Собственное условие `Ready` у TenantGateway \- агрегирующее: оно вычисляется по всем слушателям и сообщает `ListenersNotReady`, когда цель `certificateRefs` отсутствует.

```
kubectl -n <owner-tenant-ns> get tenantgateway cozystack -o  
jsonpath='{.status.conditions[?(@.type=="Ready")]}'
```

Чтобы увидеть, какой слушатель неисправен, прочитайте собственный статус TenantGateway по каждому слушателю. Контроллер выводит его из условий `Accepted` и `Programmed` каждого слушателя и записывает причину, поэтому это единственное место, где ответ на вопрос уже есть:

```
kubectl -n <owner-tenant-ns> get tenantgateway cozystack \n  
-o jsonpath='{range .status.listeners[*]}{.name}{"\t"}{.ready}{"\t"}{.reason}{"\n"}}{end}'
```

Do not go by `ResolvedRefs` on the raw `Gateway`. On this platform `cozystack-controller` lists both `HTTPRoute` and `TLSRoute` in `allowedRoutes.kinds` on every port-443 listener — a workaround that stops Cilium collapsing them together — and Cilium answers with a permanent `ResolvedRefs=False/InvalidRouteKinds` on each of them. It is cosmetic and affects neither traffic nor readiness, but it means `ResolvedRefs` reads `False` on a perfectly healthy cluster.

В режиме `existingSecret` никакого `Certificate` нет вовсе, поэтому пропустите ожидание сертификата ниже: `kubectl wait` не блокируется на отсутствующем объекте, а сразу падает с `NotFound`, и это довольно запутанный способ узнать, что в этом режиме сертификат никогда не создаётся. Вместо этого проверьте `Secret`, предоставленный оператором. Cilium не смотрит на `type` у `Secret`; он требует лишь, чтобы `tls.crt` и `tls.key` разбирались как PEM, поэтому проверяйте содержимое, а не метку:

```
kubectl -n <owner-tenant-ns> get secret <wildcardSecretName> \
```

| | | |
|---|------------------------|--|
| <code>-o jsonpath='{.data.tls\.crt}'</code> | <code>base64 -d</code> | <code>openssl x509 -noout -subject -dates</code> |
|---|------------------------|--|

В режиме `DNS-01` есть один wildcard-сертификат с именем `cozystack-gateway-tls`:

```
kubectl -n <owner-tenant-ns> wait certificate/cozystack-gateway-tls --for=condition=Ready --timeout=10m
```

В режиме `HTTP-01` каждый `Certificate` на слушатель несёт метку `cozystack.io/per-listener-cert=true` (проставляется контроллером), поэтому их можно ждать как набор:

```
kubectl -n <owner-tenant-ns> wait certificate \
--selector cozystack.io/per-listener-cert=true \
--for=condition=Ready --timeout=10m
```

3. Включите `gateway.enabled: true` в `Package` платформы. Это перерендеривает `ClusterIssuer` `cert-manager` и шаблоны публикуемых сервисов. Существующие объекты `Ingress` для `dashboard` / `keycloak` / `grafana` / `alerta` / `cozystack-api` (Kubernetes API) / `vm-exportproxy` / `cdi-uploadproxy` удаляются Flux по мере замены на `HTTPRoute` / `TLSRoute`, которые теперь прикрепляются к уже готовому (`Programmed`) `Gateway` без окна простоя.

4. Когда все деревья tenants мигрируют, `ingress-nginx` больше не нужен для сервисов, встроенных в `Cozystack`. Механизм отключения источника пакета `cozystack.ingress-application` на уровне платформы отслеживается отдельно; сегодня бандл всё ещё рендерит его, и контроллер может оставаться установленным вхолостую для tenants, которые ещё не мигрировали.

Приложения из вендоренных вышестоящих чартов (`harbor`, `bucket`) прикрепляются к `Gateway` своего tenant-владельца через `_namespace.gateway`, который чарт tenant заполняет автоматически, как только владелец устанавливает `spec.gateway: true` (и распространяет на наследующих потомков).

Откат

Чтобы откатиться в течение окна миграции, верните `gateway.enabled` в `false` в Package платформы. HTTPRoute / TLSRoute для `cozy-*` перестают рендериться, и Flux удаляет их; исходные объекты Ingress для `dashboard / keycloak / grafana / alerta / cozystack-api / vm-exportproxy / cdi-uploadproxy` заново рендерятся теми же чартами при следующем согласовании, и ingress-nginx их подхватывает. Окно простоя такое же, как и на прямом пути для сервисов `cozy-*`, - HTTPRoute исчезает раньше, чем возвращается Ingress, - так что ожидайте короткого перерыва плюс времени, которое Flux потратит на согласование. TenantGateway и Gateway тенанта, оставшиеся после `tenant.spec.gateway: true`, - а в режимах сертификатов ACME также Issuer и Certificate - не мешают пути ingress-nginx и могут быть оставлены на месте; чтобы полностью откатить тенанта, установите ещё и `tenant.spec.gateway: false`, и чарт удалит HelmRelease gateway (контроллер уберёт Gateway, а также Issuer там, где он им владеет).

Известные ограничения

- ****existingSecret cert mode is root-tenant only, and nothing enforces it.**** The Secret name rides the cluster values channel to every tenant, so enabling it also switches gateway-owning children onto the mode (they need the Secret replicated into their own namespace) and, on the default ingress path, leaves a child running its own ingress controller with no certificate at all — ingress-nginx serves its built-in self-signed one. That last case is an open bug, [cozystack/cozystack#3296](#).
- Multi-tenant shared LB IP. Multiple owning tenants cannot share a single LB IP on current Cilium: each owning tenant Gateway claims 443/TCP and `lbipam.cilium.io/sharing-key` is inactive on port collision ([cilium#21270](#), [cilium#42756](#)). Each owning Gateway therefore needs its own LB IP from the admin-managed allocator until Cilium ships ListenerSet. Within a single Gateway, inheritance (parent + all inheriting children sharing one IP) works today.
- TLSRoute v1alpha2. Gateway API v1.5 ships TLSRoute at `v1alpha2`. It graduates to `v1` upstream; Cozystack will follow the rename when it lands.
- DNS-01 wildcards require DNS provider access for every apex level. When a deeply nested tenant tree (e.g. `tenant-root -> alice -> alice-bob`) inherits DNS-01 mode through the root, the parent's `*.alice.example.org` SAN requires the parent's ACME challenge to write a TXT record under `_acme-challenge.alice.example.org`. If the operator hasn't delegated that subzone to the parent's DNS provider account, cert issuance for the grandchild apex stalls. HTTP-01 mode is unaffected.
- Supported ACME issuers. `publishing.certificates.issuerName` must be `letsencrypt-prod` or `letsencrypt-stage` (the controller maps those to ACME server URLs). To support another ACME provider, extend the controller's renderer with an additional branch.
- ****tenant.spec.host enforcement.**** A tenant cannot set their own host (runtime-blocked), but a cluster-admin who misconfigures it produces a tenant publishing a hostname they do not own. On the ACME cert modes ACME will fail (no DNS control), so no cert is issued and no hijack materialises — though the diagnostics stop at "Certificate stuck in Pending". In `existingSecret` mode that safety net is absent: nothing is issued, so nothing proves domain control, the Gateway goes `Programmed`, and the operator Secret is served with whatever SANs it happens to carry.
- Upstream application features. Some chart-level features in harbor / bucket still rely on ingress-nginx annotations upstream. Cozystack tracks those as upstream PRs; they remain the reason some ops teams will keep ingress-nginx alongside Gateway API for a while.

- cert-manager namespace is hardcoded for ACME HTTP-01. The port-80 listener's `allowedRoutes` whitelist names `cozy-cert-manager` explicitly. Operators running cert-manager in a non-default namespace cannot use HTTP-01 with Gateway API today — the ACME challenge HTTPRoute will be rejected with no obvious diagnostic. DNS-01 mode is unaffected (no in-cluster challenge HTTPRoute is involved).

Устранение неполадок

Начните отсюда: выведите список TenantGateway

```
kubectl get tenantgateway --all-namespaces
```

или, используя короткое имя

```
kubectl get tgw --all-namespaces
```

TenantGateway тенанта-владельца появляется здесь после согласования `tenant.spec.gateway: true`. Отсутствие строки означает, что чарт `apps/tenant` ещё не отрендерил CR (проверьте `tenant.spec.gateway` и что HelmRelease gateway в пространстве имён тенанта в состоянии Ready); существующая строка с `Ready=False` \- отправная точка для рецептов ниже.

TenantGateway застрял в `Ready=False` с `ReconcileError`

```
kubectl -n <tenant-ns> describe tenantgateway cozystack
```

Сообщение условия статуса называет неудавшийся шаг. Типичные случаи:

- `gateway <ns>/cozystack exists but is not owned by TenantGateway ...` \- найден уже существующий Gateway с выведенным контроллером именем, и он был отвергнут. Переименуйте или удалите чужой Gateway либо вручную установите его `OwnerReference` на TenantGateway, если намерены передать владение.
- `issuer <ns>/<tgw>-gateway exists but is not owned ...` \- то же самое для чужого Issuer.
- `certificate <ns>/... exists but is not owned ...` \- то же для чужого Certificate.

Gateway застрял в `Programmed=False`

```
kubectl -n cozy-cilium logs  
deploy/cilium-operator --tail=100
```

```
grep -i gateway
```

Отсутствующий Secret сертификата сюда не приводит - Gateway сохраняет свой адрес и слушатель `http`, поэтому остаётся `Programmed`. Для этого отказа читайте статус TenantGateway по каждому слушателю (`.status.listeners[*].ready / .reason`) либо его агрегирующее условие `Ready`. Настоящие причины `Programmed=False`: у сервиса LoadBalancer ещё нет адреса (`AddressNotAssigned` \-

безусловно самая частая), опечатка в `gatewayClassName` (должно быть ровно `cilium`), слушатель, конфликтующий с другим (тот же порт + протокол + имя хоста), или отказ всех слушателей разом.

HTTPS не работает или отдаёт не тот сертификат в режиме `existingSecret`

В этом режиме объекта `Certificate` не существует, поэтому и `describe` делать нечего. Типичные отказы:

- Secret отсутствует или назван неверно. Каждый HTTPS-слушатель не проходит валидацию (`InvalidCertificateRef`) и перестаёт быть `Accepted`. Порт остаётся открытым - Cilium транслирует конфигурацию Envoy из `_спецификации_ Gateway`, а не из принятого подмножества, и сертификат отдаётся через ссылку SDS, - поэтому клиенты по-прежнему подключаются на 443, а затем не проходят рукопожатие TLS (сброс или пустой ответ), вместо того чтобы получить отказ в соединении. Сам Gateway остаётся `Programmed; TenantGateway` сообщает `ListenersNotReady`.
- Secret есть, содержимое непригодно. Secret без `tls.crt` или `tls.key` либо содержащий под ними что-то, что не является PEM, не проходит валидацию: `PEM format error in TLS Certificate` для сертификата, `PEM format error in TLS Key` для ключа. Обратите внимание, что Cilium проверяет содержимое, а не метку: он никогда не читает `type` у Secret. Всё равно создавайте Secret с типом `kubernetes.io/tls` \- именно этого ожидает платформа, - но при диагностике не смотрите на `type`, потому что никто в цепочке его не читает.
- Secret в порядке, SAN-записи слишком узкие. TLS завершается успешно, а клиент видит несовпадение имени хоста. Голый `*.<apex>` не покрывает `*.<child-apex>`.

`TenantGateway` сообщает, КАКОЙ слушатель не готов (`reason: NotAccepted`); сообщения Cilium он не несёт. За приведёнными выше строками причин обращайтесь к условиям слушателей на самом Gateway (`.status.listeners[*].conditions[*].message`) либо к логу `cilium-operator`.

```
kubectl -n <owner-tenant-ns> get tenantgateway cozystack \
-o jsonpath='{range .status.listeners[*]}.{name}"\t"}.{ready}"\t"}.{reason}"\n"}{end}'
kubectl -n <owner-tenant-ns> get secret <wildcardSecretName> \
```

| | | | |
|--------------------------------------|------------------------|----------------------------------|--------------------------------|
| <code>-o jsonpath='{.data.tls</code> | <code>base64 -d</code> | <code>openssl x509 -noout</code> | <code>grep -A1 'Subject</code> |
| <code>\.crt}'</code> | | <code>-text</code> | <code>Alternative Name'</code> |

Certificate застрял в `Ready=False`

```
kubectl -n <tenant-ns> describe certificate <cert-name>
```

```
kubectl -n <tenant-ns> describe challenge
```

Если у `HTTPRoute` проверки `Accepted=False`, белый список `allowedRoutes` HTTP-слушателя не включает пространство имён проверки - ожидается `cozy-cert-manager`, всегда неявно входящий в список. Если проверка сообщает об ошибках сервера ACME, проверьте DNS: `<host>` (HTTP-01) либо `<apex>`, `*.<apex>` и SAN-записи потомков (DNS-01) должны резолвиться в IP балансировщика Gateway / обслуживаться настроенным провайдером DNS-01.

HTTPRoute отклонён с HostnameConflict

```
kubectl -n <tenant-ns> describe httproute <route-name>
```

Найдите в `Status.Parents` запись с `controllerName: gateway.cozystack.io/tenantgateway-controller` и `Reason: HostnameConflict`. Сообщение называет конфликтующие имена хостов и маршрут, который ими владеет. Конфликты внутри одного арех разрешаются с приоритетом `cozy-*`; проигравший должен использовать другое имя хоста.

Отказ допуска: «Gateway listener hostname must equal...»

Уровень 2 (`cozystack-gateway-hostname-policy`) отклонил Gateway, потому что имя хоста слушателя не соответствует `namespace.cozystack.io/host` пространства имён Gateway. Исправьте имя хоста слушателя или (если метка пространства имён неверна) обновите `spec.host` тенанта через доверенного вызывающего.

Отказ допуска: «HTTPRoute hostnames must equal...»

Уровень 5 (`cozystack-route-hostname-policy`) отклонил HTTPRoute или TLSRoute, потому что имя хоста выходит за пределы арех метки `namespace.cozystack.io/host` пространства имён. Либо измените имя хоста так, чтобы оно находилось под арех, либо перенесите маршрут в пространство имён, чья метка покрывает нужное имя хоста.

Отказ допуска: «tenant.spec.host can only be set...»

Недоверенный вызывающий попытался задать `tenant.spec.host`. Используйте пустой `spec.host` (наследование от родителя) или попросите администратора кластера применить Tenant.

HTTPRoute наследующего потомка не принят на Gateway родителя

```
kubectl get namespace <child-tenant-ns> -o jsonpath='{.metadata.labels.namespace\cozystack\io/gateway}'
```

```
kubectl -n <owner-tenant-ns> describe gateway cozystack
```

Если метка `namespace.cozystack.io/gateway` пространства имён потомка пуста или не совпадает с именем пространства имён тенанта-владельца (например, `tenant-root`, `tenant-alice` \-пространство имён, а не «голое» имя тенанта), селектор `allowedRoutes` слушателя не допустит маршрут. Метку записывает чарт `apps/tenant`; убедитесь, что ресурс тенанта существует, чарт согласован и у родителя (или любого предка выше) задан `spec.gateway: true`.

У сервиса Gateway IP LoadBalancer в состоянии <pending>

Автоматически создаваемый сервис LoadBalancer Gateway тенанта получает IP из того аллокатора, который настроил администратор кластера. Состояние `<pending>` означает, что аллокатор не назначил адрес. Типичные причины:

- В кластере не подключён аллокатор (нет пула MetalLB, нет пула Cilium LB-IPAM, нет предварительного закрепления `externalIP`). Cozystack не рендерит манифесты аллокатора автоматически - ожидается, что оператор настроит его на уровне платформы.
- Аллокатор подключён, но исчерпан (все адреса пула уже заняты).
- Сервис запросил конкретный адрес (`loadBalancerIP`, заданный оператором), которым аллокатор не управляет.

`kubectl -n <tenant-ns> describe svc cilium-gateway-cozystack` показывает события; используемый аллокатор записывает туда свои решения.

См. также

- Спецификация Gateway API: gateway-api.sigs.k8s.io
- Документация Cilium по Gateway API: docs.cilium.io/.../gateway-api
- Ключи совместного использования Cilium LB-IPAM: docs.cilium.io/.../lb-ipam
- KEP-5707 (устаревание `Service.spec.externalIPs`): kubernetes/enhancements#5707
- Ограничения частоты Let's Encrypt: letsencrypt.org/docs/rate-limits

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис HTTP-кеширования на основе Nginx

<https://cozystack.ru/docs/v1.6/networking/http-cache/>

Сервис HTTP-кеширования на основе Nginx предназначен для оптимизации веб-трафика и повышения производительности веб-приложений.

Сервис HTTP-кеширования на основе Nginx предназначен для оптимизации веб-трафика и повышения производительности веб-приложений. Сервис объединяет специально собранные экземпляры Nginx и HAProxy для эффективного кеширования и балансировки нагрузки.

Информация о развёртывании

Экземпляры Nginx включают следующие модули и возможности:

- модуль VTS для статистики
- интеграция с ip2location
- интеграция с ip2proxy
- поддержка 51Degrees
- функция очистки кеша

HAProxy играет ключевую роль в этой схеме: он направляет входящий трафик на конкретные экземпляры Nginx на основе консистентного хеша, вычисляемого от URL. Каждый экземпляр Nginx использует Persistent Volume Claim (PVC) для хранения кешированного содержимого, обеспечивая быстрый и надёжный доступ к часто используемым ресурсам.

Детали развёртывания

Архитектура развёртывания показана на схеме ниже:

```
+-----+
| metallb | arp announce
+----+----+
|
|
+-----v-----+
| kubernetes service | node
| (externalTrafficPolicy: Local) | level
+-----+-----+
```

```

|
|
+---v---+ +-----+
| haproxy | | haproxy | loadbalancer
| (active)| | (backup)| layer
+---+---+ +-----+
|
| balance uri whole
| hash-type consistent
+---+---+ +-----+
+---v---+ +---v---+ +---v---+ caching
| nginx | | nginx | | nginx | layer
+---+---+ +---+---+ +---+---+
| | |
+---+---+ +---+---+ +---+---+ +---+
| | | |
+---v---+ +---v---+ +---v---+ +---v---+
| origin | | origin | | origin | | origin |
+---+---+ +---+---+ +---+---+ +---+---+

```

Известные проблемы

- Модуль VTS показывает неверное время ответа апстрима,
github.com/vozlt/nginx-module-vts#198

`storageClass` помечен как неизменяемый (immutable) в схеме чарта - см.
[\[docs/storage-immutability.md\]\(../../docs/storage-immutability.md\)](#), где описан этот контракт и
 какие потребители его обеспечивают.

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|-----|----------|-----|----------|
|-----|----------|-----|----------|

| | | | |
|--------------|--|----------|-------|
| size | Размер Persistent Volume Claim, доступный для данных приложения. | quantity | 10Gi |
| storageClass | StorageClass, используемый для хранения данных. | string | "" |
| external | Разрешить внешний доступ извне кластера. | bool | false |

Параметры приложения

| Имя | Описание | Тип | Значение |
|-----------|--|----------|----------|
| endpoints | Конфигурация конечных точек в виде списка <code>ip:port</code> . | []string | [] |

Параметры HAProxy

| Имя | Описание | Тип | Значение |
|------------------------------|---|----------|----------|
| haproxy | Конфигурация HAProxy. | object | {} |
| haproxy.replicas | Количество реплик HAProxy. | int | 2 |
| haproxy.resources | Явная конфигурация CPU и памяти. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | object | {} |
| haproxy.resources.cpu | CPU, доступный каждой реплике. | quantity | "" |
| haproxy.resources.мемо
гу | Память (RAM), доступная каждой реплике. | quantity | "" |
| haproxy.resourcesPreset | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | string | t1.nano |

Параметры Nginx

| Имя | Описание | Тип | Значение |
|-------------------------------------|---|-----------------------|----------------------|
| <code>nginx</code> | Конфигурация Nginx. | <code>object</code> | <code>{}</code> |
| <code>nginx.replicas</code> | Количество реплик Nginx. | <code>int</code> | 2 |
| <code>nginx.resources</code> | Явная конфигурация CPU и памяти. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | <code>object</code> | <code>{}</code> |
| <code>nginx.resources.cpu</code> | CPU, доступный каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>nginx.resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>" "</code> |
| <code>nginx.resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |

Примеры параметров и справка

resources и resourcesPreset

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

`resources:`

`cpu: 4000m`

`memory: 4Gi`

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

| Имя пресета | CPU | Память |
|---------------------|------|--------|
| <code>nano</code> | 250m | 128Mi |
| <code>micro</code> | 500m | 256Mi |
| <code>small</code> | 1 | 512Mi |
| <code>medium</code> | 1 | 1Gi |

| | | |
|---------|---|-----|
| large | 2 | 2Gi |
| xlarge | 4 | 4Gi |
| 2xlarge | 8 | 8Gi |

endpoints

`endpoints` \- плоский список IP-адресов:

`endpoints`:

- 10.100.3.1:80
- 10.100.3.11:80
- 10.100.3.2:80
- 10.100.3.12:80
- 10.100.3.3:80
- 10.100.3.13:80

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

PROXY-protocol и исправление hairpin-NAT

<https://cozystack.ru/docs/v1.6/networking/hairpin-proxy-protocol/>

Включение PROXY-protocol на ingress-nginx в Cozystack и исправление возникающей проблемы hairpin-NAT с помощью входящего в состав платформы компонента ouroboros.

О чём эта страница

PROXY-protocol на ingress-nginx управляющего кластера, порождаемая им проблема hairpin-NAT, входящее в состав Cozystack исправление (ouroboros, реализация [compumike/hairpin-proxy](https://github.com/compumike/hairpin-proxy) на Go - см. [lexfrei/ouroboros](https://github.com/lexfrei/ouroboros)), единый флаг платформы, связывающий обе части, аддон для того же исправления внутри кластеров тенантов и несимметричные пути отключения на каждом уровне.

Почему PROXY-protocol ломает трафик внутри кластера

Когда вышестоящий балансировщик L4 (облачный LB, hcloud-CCM, F5, haproxy за MetalLB и т.д.) добавляет **заголовок PROXY-protocol v1** к каждому соединению, приходящему на ingress-nginx, ingress-nginx настраивается с `use-proxy-protocol: "true"` и принимает только соединения с заголовком. Внешний трафик работает.

Трафик внутри кластера - нет. Под, который резолвит публичное имя самого кластера (self-check и HTTP-01 в cert-manager, внутренние вызовы `https://` по публичному DNS, обращения ArgoCD к самому себе и т.д.), обращается к CoreDNS и получает в ответ IP LoadBalancer. kube-proxy / Cilium-KPR видит этот IP LB и срезает путь напрямую к локальному сервису, минуя вышестоящий балансировщик. Соединение приходит на ingress-nginx без заголовка PROXY, который тот теперь требует. Ingress-nginx закрывает соединение. Hairpin молча ломается.

KEP-1860 (`status.loadBalancer.ingress[].ipMode: Proxy`) - это апстрим-решение для кластеров, стоящих за внешним прокси-балансировщиком под управлением CCM. Оно не подходит для настроек Cozystack по умолчанию: при `kubeProxyReplacement: true` и IP-адресах LB, анонсируемых Cilium, Cilium полностью убирает фронтенд LB, когда `ipMode != VIP`, ломая сервис и для внутрикластерного, и для внешнего трафика. Вместо этого Cozystack использует ouroboros.

Как это исправляет ouroboros

ouroboros - небольшой контроллер внутри кластера, который отслеживает имена хостов в Ingress (и опционально Gateway / HTTPRoute) и переписывает внутрикластерный DNS так, чтобы эти имена резолвились в небольшой внутрикластерный прокси, добавляющий заголовок PROXY-protocol перед пересылкой на ingress-nginx. Он состоит из двух взаимодействующих частей:

- контроллер , который отслеживает ресурсы Ingress / Gateway и добавляет в CoreDNS строки `rewrite name <host> ouroboros-proxy....svc.cluster.local.;`
- TCP-прокси , который слушает порты 8080/8443, добавляет заголовок PROXY-protocol v1 и пересылает трафик на настоящий сервис ingress-nginx.

Cozystack использует два режима работы в зависимости от уровня:

| Уровень | Режим | Что записывает ouroboros |
|-----------------------------|-----------------------------|---|
| Управляющий кластер | <code>coredns</code> | Рабочий Corefile kube-system/coredns, между маркерами <code># === BEGIN ouroboros (do not edit by hand) ===</code> . CoreDNS управляющего кластера управляется Talos и не содержит директивы <code>import</code> , поэтому альтернативный режим там молча ничего бы не делал. |
| Кластеры Kubernetes tenants | <code>coredns-import</code> | Только строки <code>rewrite name</code> в ключ данных <code>ouroboros.override</code> ConfigMap kube-system/coredns-custom \- отдельного ConfigMap, который поставляется чартом <code>cozystack-coredns</code> и подключается в Corefile через <code>import /etc/coredns/custom/*.override</code> . <code>ouroboros</code> никогда не трогает Corefile, отрендеренный чартом, поэтому не конфликтует с чартом при каждом согласовании Flux. |

Включение в управляющем кластере

Один флаг платформы включает PROXY-protocol на ingress-nginx управляющего кластера и автоматически развёртывает ouroboros на стороне управляющего кластера:

publishing:

proxyProtocol: true

По умолчанию `false` \- кластеры, не использующие PROXY-protocol, не получают ни новых ресурсов, ни изменений RBAC, ни изменений поведения.

Предусловие: вышестоящий балансировщик

Флаг не настраивает за вас балансировщик L4 перед ingress-nginx - тот находится вне кластера (облачный LB, F5, MetalLB+haproxy, hcloud-cloud-controller-manager и т.д.). Вышестоящий балансировщик обязан уже добавлять заголовки PROXY-protocol v1 до переключения флага.

Кадры PROXY-protocol передаются от вышестоящего балансировщика к ingress-nginx (балансировщик открывает новое TCP-соединение к бэкенду и добавляет кадр в это соединение), а не обратно к исходному клиенту - nc с ноутбука их никогда не увидит. Есть два практических способа проверки, выберите один:

- Запустите `tcpdump --interface any port 80 or port 443 -A` на узле с ingress-nginx до переключения флага и посмотрите, есть ли ведущий кадр PROXY TCP4 ...r\n во входящем TCP-трафике от балансировщика. Если кадр есть, балансировщик выполняет вставку и флаг можно безопасно переключать.
- Выполните любой внешний HTTP-запрос после изменения на стороне балансировщика, но до переключения `publishing.proxyProtocol`, и посмотрите в логах ingress-nginx (`kubectl --namespace cozy-ingress-nginx logs deploy/ingress-nginx-controller`) сообщение `client sent invalid proxy protocol header \` - эта ошибка означает, что балансировщик начал вставлять кадры PROXY, а ingress-nginx ещё не настроен их принимать; это ровно то переходное состояние, которое флаг и призван устранить. Противоположный лог (`broken header` после переключения флага, когда балансировщик не вставляет кадры) сигнализирует, что предусловие на самом деле не выполнялось, и из этого состояния нужно откатываться.

Без выполнения предусловия каждый внешний запрос к ingress-nginx ломается в момент включения флага. Если ни один из способов проверки не показывает кадры PROXY, сначала исправьте вышестоящий балансировщик и только потом трогайте `publishing.proxyProtocol`.

Окно нестабильности при перерендеринге Talos

CoreDNS управляющего кластера - это Deployment под управлением Talos; ouroboros изменяет его рабочий Corefile на месте между маркерами BEGIN/END. Применение machine-config Talos или обновление перерендеривает Corefile и затирает маркеры - возникает окно нестабильности длиной в один цикл согласования ouroboros (значение чарта по умолчанию `controller.resync: 10m`, сверху ограниченное ближайшим событием Ingress/Gateway), пока контроллер не применит блок заново. Любое событие Ingress или Gateway в течение окна запускает согласование раньше и укорачивает его. Внутрикластерный DNS для hairpin-имён в это окно ненадолго возвращает IP вышестоящего балансировщика вместо ClusterIP прокси, что ломается так же, как ломается внутрикластерный hairpin вовсе без ouroboros (короткое замыкание kube-proxy снова вступает в силу; запросы без PROXY попадают на ingress-nginx и отклоняются). Это приемлемо, но об этом стоит помнить во время обновления Talos.

Обёртка cozystack-coredns

Cozystack предоставляет собственную тонкую обёртку над апстрим-пакетом `coredns/helm-charts`: в Corefile добавляется директива `import /etc/coredns/custom/*.override` в серверный блок `.:53`, а

Deployment CoreDNS монтирует пустой ConfigMap `coredns-custom` в `/etc/coredns/custom/` (с `optional: true`). Шаблон обёртки рендерит этот ConfigMap ****вообще без поля `data`:** ****** \- поэтому трёхстороннему слиянию Helm не с чем сравнивать со стороны шаблона при каждом согласовании, и записи `data.ouroboros.override`, которые `ouroboros` добавляет на стороне `apiserver` во время работы, переживают любое обновление чарта нетронутыми. Аннотация `helm.sh/resource-policy: keep` на том же ConfigMap - отдельная, более слабая мера: она лишь не даёт `helm uninstall` удалить ConfigMap, но не защищает от затирания при обновлении чарта. Инвариант закреплён проверкой `notExists: data` в `packages/system/coredns/tests/coredns_custom_test.yaml` \- будущее изменение чарта, добавляющее хотя бы `data: {}` «для явности», уронит эту проверку до того, как регрессия попадёт в релиз. Сегодня эта схема используется Deployment'ами CoreDNS tenants (где `ouroboros` работает в режиме `coredns-import`); CoreDNS управляющего кластера управляется Talos и никогда не использует этот чарт, поэтому установка в управляющем кластере работает в режиме `coredns` с рабочим Corefile.

Домен кластера DNS у tenants

Чарт `ouroboros` начиная с версии 0.7.0 поддерживает два пути определения `clusterDomain`: явное закрепление через `controller.clusterDomain` или автоопределение во время работы из `/etc/resolv.conf`. Cozystack закрепляет в обёртке аддона `controller.clusterDomain: cluster.local`, потому что автоопределение возвращает неверное значение на tenants Cozystack. Kubelet `tenant` подставляет `clusterDomain` платформы управляющего кластера (обычно `cozy.local`) в `resolv.conf` пода как поисковый домен, тогда как управляемый Kamaji CoreDNS `tenant` обслуживает собственный домен кластера `tenant` (`cluster.local` согласно `TenantControlPlane.networkProfile.clusterDomain`). Автоопределение собрал бы `<service>.<namespace>.svc.cozy.local.`, который CoreDNS `tenant` не обслуживает, и `/readyz` прокси вечно получал бы NXDOMAIN. Закреплённое значение совпадает с тем, что CoreDNS `tenant` действительно обслуживает, и контроллер выдаёт `--proxy-fqdn=<service>.<namespace>.svc.cluster.local.`

Операторы tenants с нестандартным доменом кластера `tenant` (федерации, пользовательский `TenantControlPlane.networkProfile.clusterDomain` в Kamaji и т.п.) могут переопределить его через `addons.ouroboros.valuesOverride.ouroboros.controller.clusterDomain` в CR Kubernetes. Чарт учитывает переопределение и выдаёт `--proxy-fqdn=<service>.<namespace>.svc.<cluster-domain>.` Указывать `_cluster.cluster-domain` (соглашение платформы Cozystack на стороне управляющего кластера, часто `cozy.local`) здесь было бы неправильно: это значение отражает `clusterDomain` платформы управляющего кластера, а не то, что обслуживает CoreDNS `tenant`.

Включение для отдельного tenant

Tenants, которые запускают собственный `ingress-nginx` с `PROXY-protocol`, нужен `ouroboros` внутри кластера `tenant`. Параметр `addons.ouroboros` в CR Kubernetes конкретного `tenant` включает там этот чарт:

`apiVersion: apps.cozystack.io/v1alpha1`

`kind: Kubernetes`

```
metadata:
name: production
namespace: tenant-acme
spec:
addons:
ingressNginx:
enabled: true
hosts: [acme.example.org]
valuesOverride:
ingress-nginx:
controller:
config:
use-proxy-protocol: "true"
use-forwarded-headers: "true"
compute-full-forwarded-for: "true"
real-ip-header: proxy_protocol
enable-real-ip: "true"
# SECURITY: when use-forwarded-headers is on, ALWAYS pair it with
# proxy-real-ip-cidr scoped to the upstream LB's CIDR. Without
# that pairing, any client can spoof X-Forwarded-For and
# bypass IP-based controls. The host platform deliberately
# omits use-forwarded-headers / compute-full-forwarded-for
# for this reason — turn them on at the tenant only when you
# have a paired CIDR.
# proxy-real-ip-cidr: "10.0.0.0/24"
ouroboros:
enabled: true
```

Прежде чем переключать `enabled` обратно в `false`, прочитайте раздел Отключение ниже - блок `rewrite`, который `ouroboros` записал в CoreDNS тенанта, не вычищается автоматически.

`addons.ouroboros.enabled: true` требует `addons.ingressNginx.enabled: true`. Иначе рендер чарта тенанта завершается понятной ошибкой - без `ingress-nginx` исправлению `hairpin` нечего исправлять, а аддон-пустышка, который молча ничего не делает, отлаживать сложнее, чем явную ошибку на этапе рендера. Если вам нужен именно `ouroboros`, сначала включите `ingress-nginx`. В отличие от флага управляющего кластера, аддон тенанта не включает автоматически `PROXY-protocol` на `ingress-nginx` тенанта - конфигурация `use-proxy-protocol` / `use-forwarded-headers` / `real-ip-header` задаётся вручную через `valuesOverride`, потому что у тенантов часто разные

конфигурации вышестоящих балансировщиков (одни получают кадры PROXY от балансировщика управляющего кластера Cozystack, другие - от собственной граничной инфраструктуры тенанта). Без этой конфигурации ouroboros ничего не делает, поэтому, когда PROXY-protocol выключен, отключённый аддон экономит ресурсы.

Отключение

Отключение устроено по-разному на двух уровнях, и это сделано намеренно. Путь управляющего кластера имеет шлюз подтверждения на этапе рендера (`publishing.proxyProtocolAcknowledgeUnclean`), потому что `helm.sh/resource-policy: keep` на CR Package платформы создаёт асимметрию «перестали генерировать, но осталось установленным», которую опасно обнаружить слишком поздно. Путь тенанта шлюза не имеет - `addons.ouroboros.enabled: false` напрямую запускает `helm uninstall`, pre-delete-хук чарта выполняет очистку при удалении, а шаблон тенанта Cozystack намеренно не содержит рендер-защиты на основе `lookup` (проверка на остаточный HelmRelease заблокировала бы обычное отключение: рендер родителя выполняется до того, как helm-controller применит diff с отсутствующим дочерним ресурсом, поэтому lookup всегда находил бы остаток в момент переключения флага).

Уровень управляющего кластера. Переключение `publishing.proxyProtocol` из `true` в `false` делает две вещи:

- Со стороны ingress-nginx: следующее согласование регенерирует `cozystack.ingress-application` без `use-proxy-protocol / real-ip-header / enable-real-ip` (Cozystack намеренно НЕ устанавливает `use-forwarded-headers` и `compute-full-forwarded-for` в управляющем кластере: эти ключи позволили бы любому вышестоящему прокси подделывать X-Forwarded-For без парного `proxy-real-ip-cidr`). ingress-nginx перестаёт принимать заголовки PROXY-protocol. Если вышестоящий балансировщик L4 в этот момент всё ещё добавляет кадры PROXY, каждый внешний запрос к ingress-nginx ломается, пока балансировщик тоже не будет перенастроен. Сначала выключите PROXY на балансировщике, затем переключайте этот флаг.
- Со стороны ouroboros: платформа перестаёт генерировать CR Package `cozystack.ouroboros`, но каждый CR Package несёт `helm.sh/resource-policy: keep`. Helm оставляет существующий Package в кластере, ouroboros остаётся установленным, а блок `rewrite` в рабочем Corefile продолжает указывать на всё ещё работающий сервис `ouroboros-proxy`. Само по себе переключение флага не удаляет ouroboros.

Платформа отказывается рендерить простое переключение флага, когда CR Package `cozystack.ouroboros` уже есть в кластере - `helm template / helm upgrade` сразу завершается ошибкой, указывающей на `kubectl delete package.cozystack.io cozystack.ouroboros` (что запускает `helm uninstall` и pre-delete-хук очистки чарта) и на поле подтверждения. Вендоренный чарт содержит pre-delete-хук (`charts/ouroboros/templates/coredns-cleanup-hook.yaml`), который останавливает контроллер и с помощью `sed` вырезает блок `# === BEGIN ouroboros ... END ouroboros ===` из `kube-system/coredns` автоматически, когда helm действительно удаляет чарт, - операторам не нужно выполнять ручной рецепт с `sed` при обычном отключении. Полная последовательность отключения в управляющем кластере:

-
1. Выключите вставку PROXY-protocol на вышестоящем балансировщике (предусловие для внешнего трафика).
 2. Удалите CR Package командой `kubectl delete package.cozystack.io cozystack.ouroboros` (или добавьте его в `bundles.disabledPackages`). Это запускает `helm uninstall`, который выполняет `pre-delete`-хук чарта и автоматически патчит `kube-system/coredns`.
 3. Установите `publishing.proxyProtocol: false` в значениях платформы. Рендер-защита теперь проходит (lookup для `cozystack.ouroboros` возвращает `nil` после шага 2), поэтому `publishing.proxyProtocolAcknowledgeUnclean` остаётся в значении по умолчанию `false`.

Если у оператора есть причина переключить `publishing.proxyProtocol: false` ДО удаления CR Package (строгий GitOps, где `kubectl delete` не входит в основной процесс, параллельные откаты и т.п.), установите `publishing.proxyProtocolAcknowledgeUnclean: true` вместе с переключением флага в том же коммите, а удаление Package выполните отдельно. Верните `proxyProtocolAcknowledgeUnclean` в `false`, когда кластер пробудет чистым один цикл согласования. Это аварийный клапан, а не рекомендуемый путь - поэтапная последовательность выше оставляет флаг подтверждения в значении по умолчанию и избавляет от лишнего цикла.

Рецепт очистки управляющего кластера ниже - ручной запасной вариант для редкого случая, когда `pre-delete`-хук чарта не сработал (под контроллера застрял в `CrashLoop` и заблокировал шаг остановки, дрейф RBAC на `ConfigMap`, `helm uninstall` прерван до выполнения хука). Прибегайте к нему только тогда, когда автоматический путь не сработал и блок BEGIN/END всё ещё находится в `kube-system/coredns` после удаления пакета.

Известная дыра : оператор, который удаляет CR Package до переключения флага, полностью обходит защиту (lookup возвращает `nil`, рендер платформы проходит). Этот путь в основном безопасен - `kubectl delete package` запускает `helm uninstall`, и `pre-delete`-хук чарта патчит `Corefile` при удалении. Дыра - это редкий случай, когда сам хук не сработал (застывший контроллер, дрейф RBAC, тайм-аут хука), а оператор этого не заметил. Шлюз подтверждения - это эшелонированная защита, а не герметичный замок.

Уровень тенанта. Переключение `addons.ouroboros.enabled` из `true` в `false` прекращает рендер `HelmRelease`, и Flux удаляет чарт при следующем согласовании на стороне тенанта. Аннотация `helm.sh/resource-policy: keep`, упомянутая выше для управляющего кластера, находится на CR Package платформы - на стороне тенанта она не действует, поэтому отключение там действительно удаляет рабочую нагрузку, а не просто прекращает её генерацию. `helm uninstall` выполняет тот же `pre-delete`-хук чарта на тенанте: он автоматически обнуляет ключ `ouroboros.override` в `kube-system/coredns-custom` до удаления пода контроллера. Полная последовательность отключения на тенанте состоит из одного шага:

1. Установите `addons.ouroboros.enabled: false` в CR Kubernetes тенанта. Flux выполнит `helm uninstall`, и `pre-delete`-хук чарта сам патчит `kube-system/coredns-custom`.

Если `pre-delete`-хук чарта не сработал (под контроллера застрял в `CrashLoop`, дрейф RBAC на `ConfigMap`, тайм-аут Job, ручной `kubectl delete hr` в обход `helm uninstall`), симптомом будет устаревшая запись `rewrite` в `ConfigMap kube-system/coredns-custom` тенанта, указывающая на уже

несуществующий сервис. Для восстановления выполните рецепт очистки тенанта ниже с использованием `admin-kubeconfig` тенанта.

Ingress-nginx тенанта не затрагивается переключением одного лишь `addons.ouroboros` \-PROXY-protocol на ingress тенанта настраивается вручную через `valuesOverride` и остаётся таким, каким его задал оператор.

Рецепт очистки управляющего кластера (запасной вариант)

Этот рецепт - ручной запасной вариант для редкого случая, когда `pre-delete`-хук чарта не сработал (штатный путь, автоматически выполняющий хук, описан в последовательности отключения выше). Требования: `kubectl` с `kubeconfig` управляющего кластера, а также `jq` и `sed` на рабочей станции оператора. Блок ограничен строками `# === BEGIN ouroboros (do not edit by hand) ===` и `# === END ouroboros ===` (пояснение `(do not edit by hand)` есть только в строке `BEGIN`). Диапазон `sed` ниже использует сопоставление по префиксу с обеих сторон, поэтому маркеры находятся независимо от их хвостовой части.

```
# === BEGIN cleanup-recipe ===
existing=$(kubectl --namespace kube-system get configmap coredns \
--output jsonpath='{.data.Corefile}')
cleaned=$(printf '%s\n' "$existing" \
| sed '/# === BEGIN ouroboros/,/# === END ouroboros/d')
kubectl --namespace kube-system patch configmap coredns \
--type merge \
--patch "$(jq --null-input --arg c "$cleaned" \
'{data: {Corefile: $c}}')"
```

```
kubectl --namespace kube-system rollout restart deployment/coredns
# === END cleanup-recipe ===
```

Рецепт использует JSON `merge-patch`, поэтому метки, аннотации и другие ключи данных `ConfigMap coredns` (включая любые метаданные, управляемые Talos) сохраняются.

Рецепт очистки тенанта (запасной вариант)

Этот рецепт - ручной запасной вариант для редкого случая, когда `pre-delete`-хук чарта не смог самостоятельно обнулить ключ `ouroboros.override` (штатный путь на тенанте оставляет очистку чарту и вообще не требует ручного рецепта). Требования: `kubectl` с `admin-kubeconfig` тенанта (`jq` / `sed` не нужны - рецепт для тенанта состоит из одного разового патча). Выполните его после неудавшегося отключения, чтобы вернуть `ConfigMap` в чистое состояние.

```
kubectl --kubeconfig <tenant-admin-kubeconfig> --namespace kube-system patch \
configmap coredns-custom --type merge \
--patch '{"data":{"ouroboros.override":null}}'
```

`ouroboros.override` \- единственный ключ, которым `ouroboros` владеет внутри `coredns-custom`; его обнуление не затрагивает остальные ключи (принадлежащие оператору фрагменты `*.override`, будущие дополнения `Cozystack`).

Операторы изолированных сред

Чарт и образ загружаются напрямую из апстрим-реестра `lexfrei/ouroboros` \- они не зеркалируются в `ghcr.io/cozystack/*`. Операторам изолированных (`air-gapped`) сред нужно отзеркалировать два дополнительных источника:

- `oci://ghcr.io/lexfrei/charts/ouroboros:<version>` (чарт, дайджест зафиксирован в `packages/system/ouroboros/Makefile` как `OUROBOROS_CHART_DIGEST=sha256:...` \- точные значения `OUROBOROS_CHART_VERSION` и `OUROBOROS_CHART_DIGEST` берите из `Makefile` того релиза `Cozystack`, который вы зеркалируете);
- `ghcr.io/lexfrei/ouroboros:<version>@sha256:...` (образ, дайджест зафиксирован в `packages/system/ouroboros/values.yaml` в `image.tag` \- передайте именно эту ссылку в `regsync / crane copy / skopeo copy`).

Ссылка на образ в `Cozystack` включает дайджест `@sha256:...`, указанный выше. Инструменты зеркалирования должны либо сохранить этот дайджест на всём пути (стандартное поведение `regsync`, `crane copy`, `skopeo copy`), либо убрать закрепление `@sha256:...` из `values.yaml` после зеркалирования - иначе `kubelet` при загрузке будет резолвить апстрим-дайджест, не найдёт его среди тегов зеркала и попадёт в `ErrImagePull`.

Почему в настройках Cozystack по умолчанию не используется KEP-1860

Значение по умолчанию `kubeProxyReplacement: true` в `Cozystack` означает, что `Cilium` полностью убирает фронтенд LB, когда `ipMode != VIP`, ломая приём трафика сервисов для IP-адресов, анонсируемых по L2/BGP. Режим `Proxy` из KEP-1860 предназначен для кластеров за внешним прокси-балансировщиком под управлением CCM, который сам обрабатывает соединение; это не тот контракт, который предоставляет `Cozystack`. Для топологий с анонсами L2/BGP, которые `Cozystack` предоставляет по умолчанию, правильное решение - `ouroboros`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Публикация конечной точки Kubernetes API через external-dns

<https://cozystack.ru/docs/v1.6/networking/kuberture/>

Преобразование EndpointSlice default/kubernetes в аннотированные headless-сервисы, чтобы external-dns мог публиковать конечную точку Kubernetes API в DNS с помощью входящего в состав платформы компонента kuberture.

О чём эта страница

Как опубликовать конечную точку Kubernetes API самого кластера в виде DNS-записи с помощью external-dns, почему очевидный подход (направить external-dns напрямую на EndpointSlice default/kubernetes) не работает, что представляет собой входящий в состав Cozystack мост (kuberture, см. [lexfrei/kuberture](https://github.com/lexfrei/kuberture)), как одна установка kuberture может одновременно обслуживать несколько экземпляров external-dns, а также как всё это отключить.

Почему external-dns не может читать EndpointSlice напрямую

external-dns считывает значимое для DNS состояние из источников Kubernetes - service, ingress, gateway-httproute и ряда других. Источник EndpointSlice он не поддерживает. EndpointSlice управляющего слоя, который Kubernetes поддерживает в default/kubernetes, - это канонический, всегда актуальный список адресов узлов, обслуживающих API; кроме того, это единственный полноценный объект, который несёт эту информацию без меток или селекторов, задаваемых оператором. Операторы, которые хотят публиковать конечную точку API своего кластера в публичный DNS через external-dns (проверки dns01 в cert-manager, внутренняя автоматизация, обращающаяся к API по FQDN, обнаружение сервисов между кластерами и т.д.), сразу упираются в этот пробел.

Обычно предлагаемые обходные пути - поддерживаемый вручную Service типа ExternalName, поддерживаемый вручную headless-Service с вручную закреплёнными Endpoints, внешний скрипт, опрашивающий API и обновляющий запись, - имеют один и тот же недостаток: они перестают соответствовать действительности, как только узел управляющего слоя добавляется, удаляется или меняет адрес. EndpointSlice уже корректно отслеживает это состояние. kuberture соединяет одно с другим.

Как это решает kuberture

kuberture - небольшой контроллер внутри кластера, который следит за EndpointSlice и создаёт аннотированные headless-объекты Service. Поток данных таков:

EndpointSlice (kubernetes) -> kuberture -> headless Service(s) with annotations -> external-dns -> DNS

Для каждого настроенного оператором выхода (output) kuberture создаёт headless-сервис (spec.clusterIP: None) в собственном пространстве имён (cozy-kuberture) и проставляет на нём три аннотации. Каждый ключ состоит из заданного оператором annotationPrefix и фиксированного суффикса:

| Полный ключ аннотации | Источник |
|----------------------------|--|
| <annotationPrefix>hostname | заданные оператором имена хостов для этого выхода |
| <annotationPrefix>target | IP-адреса через запятую, полученные из EndpointSlice или из узлов, на которые он указывает |
| <annotationPrefix>ttl | заданный оператором recordTTL; если он не указан, используется значение по умолчанию контроллера |

С префиксом по умолчанию это даёт external-dns.alpha.kubernetes.io/hostname, external-dns.alpha.kubernetes.io/target, external-dns.alpha.kubernetes.io/ttl \- ключи, которые платформенный external-dns читает из коробки. external-dns читает сервис, видит аннотации и создаёт DNS-запись. У сервиса нет селектора и вручную управляемых Endpoints - он существует исключительно как носитель аннотаций для external-dns.

Контроллер не создаёт объекты EndpointSlice или Endpoints (у него есть только право чтения EndpointSlice) и не трогает исходный EndpointSlice default/kubernetes. Он строго аддитивен: отдельный сервис на каждый выход, в собственном пространстве имён, без побочных эффектов для существующего состояния кластера.

Включение в управляющем кластере

kuberture \- опциональный системный пакет, включаемый через bundles.enabledPackages. Сам пакет не содержит пригодных значений по умолчанию, кроме включения ServiceMonitor для платформенного Prometheus, - развёртывание сразу завершается ошибкой, если config.outputs пуст. Оператор обязан объявить хотя бы один выход, потому что единственное, чего kuberture не может определить сам, - это DNS-имя, которое оператор действительно хочет опубликовать.

Один выход, потребляемый платформенным external-dns:

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.cozystack-platform
spec:
```

```
variant: isp-full
components:
platform:
values:
bundles:
enabledPackages:
  - cozystack.external-dns
  - cozystack.kuberture
```

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
name: cozystack.kuberture
spec:
variant: default
components:
kuberture:
values:
kuberture:
config:
outputs:
  - name: api
hostname:
  - api.k8s.example.com
serviceName: kuberture-api
addressSource: endpointslice
recordTTL: 60
```

`annotationPrefix` здесь не указан, поэтому контроллер использует значение по умолчанию `external-dns.alpha.kubernetes.io/` \- префикс, за которым платформенный `external-dns` следит из коробки. Когда включены оба пакета, `HelmRelease external-dns` подхватывает сервис `kuberture-api` в `cozy-kuberture` и создаёт запись `api.k8s.example.com` у настроенного провайдера.

Пакет `cozystack.external-dns` \- это общекластерный системный экземпляр (он работает с `namespaced: false` и наблюдает за сервисами по всему кластеру). `cozystack.external-dns-application` \- вариант, ограниченный пространством имён тенанта, и он не подхватит сервисы за пределами своего тенанта. В управляющем кластере используйте `kuberture` в паре с `cozystack.external-dns`, а не с `cozystack.external-dns-application`.

Маршрутизация на несколько экземпляров external-dns

Одна установка `kuberture` может обслуживать любое количество экземпляров `external-dns`, если задавать разный `annotationPrefix` для каждого выхода. Каждый экземпляр `external-dns` запускается с флагом `--annotation-prefix=<your-prefix>/`, из-за чего он строит все ключи аннотаций `hostname/target/ttl` под этим префиксом и игнорирует всё остальное; `kuberture` проставляет соответствующий префикс на каждом сервисе. Это описанный в апстриме паттерн [Split Horizon DNS](#) \- в отличие от `--annotation-filter`, который является селектором меток Kubernetes и фильтрует, _какие сервисы_ рассматривает экземпляр, а не из какого префикса он читает данные.

```
apiVersion: cozystack.io/v1alpha1
kind: Package
metadata:
  name: cozystack.kuberture
spec:
  variant: default
  components:
    kuberture:
      values:
        kuberture:
          config:
            outputs:
              - name: public
            hostname:
              - api.k8s.example.com
            serviceName: kuberture-public
            addressSource: endpointslice
              - name: internal
            hostname:
              - api-internal.k8s.example.internal
            annotationPrefix: internal-dns.example.com/
            serviceName: kuberture-internal
            addressSource: endpointslice
            recordTTL: 300
```

В результате в `cozy-kuberture` появляются два headless-сервиса:

- `kuberture-public` несёт стандартные аннотации `external-dns.alpha.kubernetes.io/*` и потребляется платформенным `external-dns`.

- `kuberture-internal` несёт аннотации `internal-dns.example.com/*` и невидим для платформенного `external-dns` (который читает префикс по умолчанию). Его потребляет второй экземпляр `external-dns`, запущенный с `--annotation-prefix=internal-dns.example.com/`: этот флаг указывает экземпляру искать `hostname/target/ttl` под `internal-dns.example.com/*`, поэтому он видит `kuberture-internal` и не видит `kuberture-public`.

Каждый сервис несёт только аннотации своего префикса - перекрёстного смешивания между выходами нет.

`annotationPrefix` допускает две формы: не указывать поле, чтобы унаследовать значение по умолчанию контроллера `external-dns.alpha.kubernetes.io/`, либо задать непустую строку, оканчивающуюся на `/`. Пустая строка `"` отклоняется схемой значений чарта; единственный способ вернуться к префиксу по умолчанию - не указывать поле.

Стратегии определения адресов

`addressSource` определяет, откуда каждый выход берёт целевые IP-адреса:

| <code>addressSource</code> | Что публикует <code>kuberture</code> |
|---|---|
| <code>endpointslice</code> (по умолчанию) | Адреса из <code>EndpointSlice default/kubernetes</code> как есть. Используйте, когда IP-адреса из <code>EndpointSlice</code> - это именно те адреса, которые должны попасть в DNS. |
| <code>node-internal</code> | <code>InternalIP</code> каждого узла, на котором находится конечная точка <code>EndpointSlice</code> . Используйте, когда <code>EndpointSlice</code> содержит IP-адреса сети подов, а <code>external-dns</code> должен публиковать внутренние адреса узлов. |
| <code>node-external</code> | <code>ExternalIP</code> каждого узла. Используйте для облачных кластеров, где у узлов публичные адреса на другом интерфейсе, нежели тот, на котором слушает API. |
| <code>node-public</code> | Публичный IP узла, полученный через провайдер-зависимый механизм обнаружения публичных IP. |

`addressType` (IPv4 / IPv6) дополнительно фильтрует полученный набор; по умолчанию `IPv4`.

Отключение

Удаление `cozystack.kuberture` из `bundles.enabledPackages` прекращает генерацию `CR Package` платформенным Helm-чартом. Как и у любого опционального системного пакета, существующий `CR Package` не удаляется автоматически: платформенный помощник проставляет

`helm.sh/resource-policy: keep` на каждый сгенерированный Package, поэтому Helm/Flux оставляет его на месте, когда тот перестаёт рендериться. Чтобы полностью удалить `kuberture`:

```
kubectrl delete package.cozystack.io cozystack.kuberture
```

Каскадное удаление убирает HelmRelease в `cozy-kuberture`, что удаляет Deployment, созданные по конфигурации оператора выходные сервисы и (со временем) само пространство имён. Записи `external-dns`, ранее опубликованные из этих сервисов, подчиняются собственной политике удаления `external-dns` (`policy: upsert-only` \- значение по умолчанию в Cozystack, при котором записи не отзываются при удалении сервиса; заранее переключитесь на `policy: sync`, если записи должны вычищаться автоматически).

Замечания о цепочке поставки

Чарт загружается из `oci://ghcr.io/lexfrei/kuberture/charts/kuberture`, а образ контроллера - из `ghcr.io/lexfrei/kuberture`. Оба находятся в личном пространстве имён мейнтейнера и намеренно не зеркалируются в `ghcr.io/cozystack/*`. Операторы изолированных (air-gapped) сред должны отзеркалировать и чарт, и образ контроллера в свой внутренний реестр и переопределить `kuberture.image.repository` в значениях Package. Версия чарта и дайджест OCI-манифеста зафиксированы в Makefile пакета cozystack; тег и дайджест образа контроллера зафиксированы в `values.yaml` пакета. Фиксации обновляются синхронно с каждым релизом апстрима.

Полный перечень значений, процедура обновления и причина отключённого по умолчанию `NetworkPolicy` описаны в README пакета: `[packages/system/kuberture/](https://github.com/cozystack/cozystack/tree/main/packages/system/kuberture)`.

См. также

- [Включение и отключение компонентов](#) \- используемый здесь механизм `bundles.enabledPackages / bundles.disabledPackages`.
- [Справочник Platform Package](#) \- справка по полю `bundles.enabledPackages`.
- `[lexfrei/kuberture](https://github.com/lexfrei/kuberture)` \- исходный код контроллера и чарт, справочник по конфигурации и планы развития.
- `[packages/system/kuberture/](https://github.com/cozystack/cozystack/tree/main/packages/system/kuberture)` \- пакет cozystack: прослойка значений, фикстуры helm-unittest, сквозной smoke-тест.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Управляемый сервис балансировки TCP-нагрузки

<https://cozystack.ru/docs/v1.6/networking/tcp-balancer/>

Управляемый сервис балансировки TCP-нагрузки упрощает развёртывание балансировщиков нагрузки и управление ими.

Управляемый сервис балансировки TCP-нагрузки упрощает развёртывание балансировщиков нагрузки и управление ими. Он эффективно распределяет входящий TCP-трафик между несколькими бэкенд-серверами, обеспечивая высокую доступность и оптимальное использование ресурсов.

Детали развёртывания

Управляемый сервис балансировки TCP-нагрузки использует для балансировки HAProxy. HAProxy - проверенное и надёжное решение для распределения входящего TCP-трафика между несколькими бэкенд-серверами, обеспечивающее высокую доступность и эффективное использование ресурсов. Такой выбор гарантирует бесперебойную и надёжную работу вашей инфраструктуры балансировки нагрузки.

- Документация: <<https://www.haproxy.com/documentation/>>

Параметры

Общие параметры

| Имя | Описание | Тип | Значение |
|---------------|---|----------|----------|
| replicas | Количество реплик HAProxy. | int | 2 |
| resources | Явная конфигурация CPU и памяти для каждой реплики балансировщика TCP. Если не задана, применяется пресет, указанный в <code>resourcesPreset</code> . | object | {} |
| resources.cpu | CPU, доступный каждой реплике. | quantity | "" |

| | | | |
|-------------------------------|--|-----------------------|----------------------|
| <code>resources.memory</code> | Память (RAM), доступная каждой реплике. | <code>quantity</code> | <code>""</code> |
| <code>resourcesPreset</code> | Пресет размеров по умолчанию, используемый, когда <code>resources</code> не задан. | <code>string</code> | <code>t1.nano</code> |
| <code>external</code> | Разрешить внешний доступ извне кластера. | <code>bool</code> | <code>false</code> |

Параметры приложения

| Имя | Описание | Тип | Значение |
|---|--|-----------------------|--------------------|
| <code>httpAndHttps</code> | Конфигурация HTTP и HTTPS. | <code>object</code> | <code>{}</code> |
| <code>httpAndHttps.mode</code> | Режим работы балансировщика. | <code>string</code> | <code>tcp</code> |
| <code>httpAndHttps.targetPorts</code> | Конфигурация целевых портов. | <code>object</code> | <code>{}</code> |
| <code>httpAndHttps.targetPorts.http</code> | Номер порта HTTP. | <code>int</code> | <code>80</code> |
| <code>httpAndHttps.targetPorts.https</code> | Номер порта HTTPS. | <code>int</code> | <code>443</code> |
| <code>httpAndHttps.endpoints</code> | Список адресов конечных точек. | <code>[]string</code> | <code>[]</code> |
| <code>whitelistHTTP</code> | Защитить HTTP с помощью списка разрешённых клиентских сетей (по умолчанию: <code>false</code>). | <code>bool</code> | <code>false</code> |
| <code>whitelist</code> | Список разрешённых клиентских сетей. | <code>[]string</code> | <code>[]</code> |

Примеры параметров и справка

`resources` и `resourcesPreset`

`resources` задаёт явную конфигурацию CPU и памяти для каждой реплики. Если значение не указано, применяется пресет, заданный в `resourcesPreset`.

resources:

cpu: 4000m

memory: 4Gi

`resourcesPreset` задаёт именованную конфигурацию CPU и памяти для каждой реплики. Эта настройка игнорируется, если задано соответствующее значение `resources`.

Пресеты следуют принятому в облаках соглашению об именовании `<серия>.<размер>`. Пять серий покрывают весь диапазон соотношений CPU к памяти (`t1 1:0.5`, `c1 1:1`, `s1 1:2`, `u1 1:4`, `m1 1:8`), и каждая серия включает восемь размеров (от `nano` до `4xlarge`). Старые плоские имена (`nano`, `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`) по-прежнему принимаются как устаревшие псевдонимы соответствующих типов с соотношением 1:1.

Полная матрица размеров и соответствие старых имён типам приведены в [\[docs/operations/resource-presets.md\]\(../../docs/operations/resource-presets.md\)](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Виртуальные маршрутизаторы

<https://cozystack.ru/docs/v1.6/networking/virtual-router/>

Развёртывание виртуального маршрутизатора в ВМ

Начиная с версии **v0.27.0** Cozystack может развёртывать виртуальные маршрутизаторы (также известные как «router appliances» или «middlebox appliances»). Эта возможность позволяет создать виртуальный маршрутизатор на основе экземпляра виртуальной машины. Виртуальный маршрутизатор может маршрутизировать трафик между разными сетями.

Создание виртуального маршрутизатора

Для создания виртуального маршрутизатора требуется учётная запись администратора Cozystack.

1. Создайте экземпляр ВМ

Используйте стандартные пакеты `vm-instance` и `virtual-machine`, чтобы создать экземпляр виртуальной машины.

2. Отключите защиту от подмены адресов (anti-spoofing)

Чтобы экземпляр ВМ мог работать как виртуальный маршрутизатор, у него должна быть отключена защита от подмены адресов:

```
kubectrl patch virtualmachines.kubevirt.io virtual-machine-example --type=merge \
-p '{"spec":{"template":{"metadata":{"annotations":{"ovn.kubernetes.io/port_security": "false"}}}}}'
```

3. Перезапустите виртуальную машину

```
virtctl stop virtual-machine-example
virtctl start virtual-machine-example
```

4. Получите IP-адрес ВМ

```
kubectrl get vmi
```

В выводе будет строка с IP-адресом новой ВМ:

```
NAME AGE PHASE IP NODENAME READY
virtual-machine-example 3d4h Running 10.244.8.56 gld-csxhk-003 True
```

5. Настройте пользовательские маршруты для тенанта

Отредактируйте пространство имён тенанта:

```
kubectrl edit namespace tenant-example
```

Добавьте следующую аннотацию, указав полученный ранее IP-адрес маршрутизатора в `gw` и маску подсети, которую должен обслуживать маршрутизатор, в `dst`:

```
ovn.kubernetes.io/routes: |
[{
  "gw": "10.244.8.56",
  "dst": "10.10.13.0/24"
}]
```

Теперь эти пользовательские маршруты будут применяться ко всем подам в пространстве имён `tenant`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Cozystack API

Cozystack API

<https://cozystack.ru/docs/v1.6/cozystack-api/>

Cozystack API для управления сервисами и ресурсами

Cozystack API

Cozystack предоставляет мощный API, который позволяет развёртывать сервисы с помощью различных инструментов. Вы можете управлять ресурсами через `kubectl`, Terraform или программно с помощью Go.

Оптимальный способ изучения Cozystack API:

1. Используйте дашборд для развёртывания приложения.
2. Изучите развёрнутый ресурс в Cozystack API и используйте его как пример.
3. Параметризируйте и воспроизведите пример ресурса, чтобы создавать собственные ресурсы через API.

Обнаружение ресурсов

Вы можете получить список всех доступных ресурсов с помощью `kubectl`:

```
# kubectl api-resources | grep apps.cozystack
buckets apps.cozystack.io/v1alpha1 true Bucket
clickhouses apps.cozystack.io/v1alpha1 true ClickHouse
etcds apps.cozystack.io/v1alpha1 true Etcd
foundationdbs apps.cozystack.io/v1alpha1 true FoundationDB
harbors apps.cozystack.io/v1alpha1 true Harbor
httpcaches apps.cozystack.io/v1alpha1 true HTTPCache
infos apps.cozystack.io/v1alpha1 true Info
ingresses apps.cozystack.io/v1alpha1 true Ingress
kafkas apps.cozystack.io/v1alpha1 true Kafka
kuberneteses apps.cozystack.io/v1alpha1 true Kubernetes
```

```
mariadbs apps.cozystack.io/v1alpha1 true MariaDB
mongodbs apps.cozystack.io/v1alpha1 true MongoDB
monitorings apps.cozystack.io/v1alpha1 true Monitoring
natses apps.cozystack.io/v1alpha1 true NATS
openbaos apps.cozystack.io/v1alpha1 true OpenBAO
postgreses apps.cozystack.io/v1alpha1 true Postgres
qdrants apps.cozystack.io/v1alpha1 true Qdrant
rabbitmq apps.cozystack.io/v1alpha1 true RabbitMQ
redises apps.cozystack.io/v1alpha1 true Redis
seaweedfses apps.cozystack.io/v1alpha1 true SeaweedFS
tcpbalancers apps.cozystack.io/v1alpha1 true TCPBalancer
tenants apps.cozystack.io/v1alpha1 true Tenant
virtualprivate apps.cozystack.io/v1alpha1 true VirtualPrivateCloud
vmdisks apps.cozystack.io/v1alpha1 true VMDisk
vminstances apps.cozystack.io/v1alpha1 true VMInstance
vpns apps.cozystack.io/v1alpha1 true VPN
```

Использование kubectl

Запросите конкретный тип ресурса в пространстве имён вашего тенанта:

```
# kubectl get postgreses -n tenant-test
NAME READY AGE VERSION
test True 46s 0.7.1
```

Просмотрите вывод в формате YAML:

```
# kubectl get postgreses -n tenant-test test -o yaml
apiVersion: apps.cozystack.io/v1alpha1
appVersion: 0.7.1
kind: Postgres
metadata:
name: test
namespace: tenant-test
spec:
databases: {}
replicas: 2
```

```
size: 10Gi
storageClass: ""
users: {}
status:
conditions:
  - lastTransitionTime: "2024-12-10T09:53:32Z"
message: Helm install succeeded for release tenant-test/postgres-test.v1 with chart postgres@0.7.1
reason: InstallSucceeded
status: "True"
type: Ready
  - lastTransitionTime: "2024-12-10T09:53:32Z"
message: Helm install succeeded for release tenant-test/postgres-test.v1 with chart postgres@0.7.1
reason: InstallSucceeded
status: "True"
type: Released
version: 0.7.1
```

Вы можете использовать этот ресурс как пример для создания аналогичного сервиса через API. Просто сохраните вывод в файл, обновите `name` и нужные параметры, затем используйте `kubectl` для создания нового экземпляра Postgres:

```
kubectl apply -f postgres.yaml
```

Использование Terraform

Cozystack интегрируется с Terraform. Для создания ресурсов в Cozystack API можно использовать стандартный провайдер `kubernetes`.

Пример:

```
provider "kubernetes" {
  config_path = "~/kube/config"
}

resource "kubernetes_manifest" "vm_disk_iso" {
  manifest = {
    "apiVersion" = "apps.cozystack.io/v1alpha1"
    "appVersion" = "0.7.1"
    "kind" = "Postgres"
  }
}
```

```
"metadata" = {  
  "name" = "test2"  
  "namespace" = "tenant-test"  
}  
"spec" = {  
  "replicas" = 2  
  "size" = "10Gi"  
}  
}  
}
```

Затем выполните:

```
terraform plan  
terraform apply
```

Ваш новый кластер Postgres будет развёрнут.

Использование кода на Go

Cozystack публикует пользовательские типы ресурсов Kubernetes в виде модуля Go, что позволяет управлять ресурсами Cozystack из любого кода на Go. Подробности и примеры см. на странице [Go Types](#).

-

[Go Types](#)

Программное управление ресурсами Cozystack с помощью типов Go

[REST API Reference](#)

Cozystack REST API Reference

[Справочник ApplicationDefinition](#)

Как ресурсы ApplicationDefinition описывают типы приложений и как искать их из клиентского кода

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Go Types

<https://cozystack.ru/docs/v1.6/cozystack-api/go-types/>

Программное управление ресурсами Cozystack с помощью типов Go

Типы Go

Cozystack публикует свои типы ресурсов Kubernetes в виде Go-модуля, что позволяет управлять ресурсами Cozystack из любого Go-кода. Типы доступны по адресу pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1.

Установка

Добавьте зависимость в ваш Go-модуль:

```
go get github.com/cozystack/cozystack/api/apps/v1alpha1@v1.6.0
```

Сценарии использования

Типы Go полезны для:

- Создания инструментов автоматизации — разработки скриптов или приложений, программно развёртывающих ресурсы Cozystack и управляющих ими
- Интеграции с внешними системами — подключения Cozystack к собственным CI/CD-конвейерам, системам мониторинга или оркестрации
- Валидации конфигураций — проверки спецификаций ресурсов перед их применением к кластеру
- Генерации документации — разбора и анализа существующих ресурсов Cozystack
- Создания дашбордов — разработки пользовательских интерфейсов для управления Cozystack

Доступные пакеты

Модуль содержит пакеты для каждого типа ресурса; вы можете изучить их для вашей конкретной версии по адресу pkg.go.dev/github.com/cozystack/cozystack/api/apps/v1alpha1

Простой пример

Для базового использования достаточно импортировать нужный пакет:

```
package main

import (
    "fmt"

    "github.com/cozystack/cozystack/api/apps/v1alpha1/vmdisk"
)

func main() {
    // Создание источника VMDisk из именованного образа
    image := vmdisk.SourceImage{Name: "ubuntu"}
    fmt.Printf("Source: %+v\n", image)
}
```

Сложный пример

Этот пример демонстрирует создание и маршалинг нескольких типов ресурсов Cozystack:

```
package main

import (
    "encoding/json"
    "fmt"

    "github.com/cozystack/cozystack/api/apps/v1alpha1/postgresql"
    "github.com/cozystack/cozystack/api/apps/v1alpha1/vminstance"
    "github.com/cozystack/cozystack/api/apps/v1alpha1/redis"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/api/resource"
)

func main() {
    // Создание конфигурации PostgreSQL с пользователями и базами данных
    pgConfig := postgresql.Config{
        TypeMeta: metav1.TypeMeta{
            APIVersion: "apps.cozystack.io/v1alpha1",
            Kind: "Postgres",
        },
        ObjectMeta: metav1.ObjectMeta{
            Name: "my-app-db",
        },
    }
```

```
Namespace: "tenant-myapp",
},
Spec: postgresql.ConfigSpec{
Replicas: 3,
Size: resource.MustParse("50Gi"),
Version: postgresql.Version("v18"),
Users: map[string]postgresql.User{
"appuser": {
Password: "secretpassword",
Replication: false,
},
"readonly": {
Password: "readonlypass",
},
},
Databases: map[string]postgresql.Database{
"myapp": {
Extensions: []string{"pg_trgm", "uuid-oss"},
Roles: postgresql.DatabaseRoles{
Admin: []string{"appuser"},
Readonly: []string{"readonly"},
},
},
},
Backup: postgresql.Backup{
Enabled: true,
DestinationPath: "s3://mybackups/postgres/",
EndpointURL: "http://minio:9000",
RetentionPolicy: "30d",
S3AccessKey: "myaccesskey",
S3SecretKey: "mysecretkey",
Schedule: "0 2 * * *",
},
Quorum: postgresql.Quorum{
MinSyncReplicas: 1,
```

```
MaxSyncReplicas: 1,
},
Postgresql: postgresql.PostgreSQL{
Parameters: postgresql.PostgreSQLParameters{
MaxConnections: 200,
},
},
},
}

// Маршалинг в JSON для kubectl apply
pgJSON, _ := json.MarshalIndent(pgConfig, "", " ")
fmt.Println(string(pgJSON))

// Создание конфигурации Redis
redisConfig := redis.Config{
TypeMeta: metav1.TypeMeta{
APIVersion: "apps.cozystack.io/v1alpha1",
Kind: "Redis",
},
ObjectMeta: metav1.ObjectMeta{
Name: "cache",
Namespace: "tenant-myapp",
},
Spec: redis.ConfigSpec{
Replicas: 2,
Size: resource.MustParse("5Gi"),
Version: redis.Version("v8"),
AuthEnabled: true,
ResourcesPreset: redis.ResourcesPreset("medium"),
},
}

// Создание VMInstance с дисками
vmConfig := vminstance.Config{
TypeMeta: metav1.TypeMeta{
APIVersion: "apps.cozystack.io/v1alpha1",
```

```
Kind: "VMInstance",
},
ObjectMeta: metav1.ObjectMeta{
Name: "my-vm",
Namespace: "tenant-myapp",
},
Spec: vminstance.ConfigSpec{
InstanceType: "u1.medium",
InstanceProfile: "ubuntu",
RunStrategy: vminstance.RunStrategy("Always"),
External: true,
ExternalMethod: vminstance.ExternalMethod("PortList"),
ExternalPorts: []int{22, 80, 443},
Resources: vminstance.Resources{
Cpu: resource.MustParse("2"),
Memory: resource.MustParse("4Gi"),
Sockets: resource.MustParse("1"),
},
Disks: []vminstance.Disk{
{Bus: "sata", Name: "rootdisk"},
{Bus: "sata", Name: "datadisk"},
},
Subnets: []vminstance.Subnet{
{Name: "default"},
},
SshKeys: []string{
"ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ...",
},
CloudInit: `#cloud-config
packages:
  - nginx`,
},
}
```

Развёртывание ресурсов

После создания конфигураций ресурсов их можно развернуть с помощью:

1. `kubectl` — маршалинг в YAML и применение:

```
yamlData, _ := json.Marshal(yourConfig)
```

```
// Используйте библиотеку маршалинга YAML для конвертации в YAML
```

2. Прямой клиент Kubernetes — использование `client-go`:

```
import (
```

```
"k8s.io/client-go/kubernetes"
```

```
"k8s.io/apimachinery/pkg/runtime"
```

```
)
```

```
scheme := runtime.NewScheme()
```

```
// Зарегистрируйте ваши типы в схеме
```

Дополнительные ресурсы

- [Документация Go-пакета](#)
- [Репозиторий Cozystack на GitHub](#)
- [Справочник Kubernetes API](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

REST API Reference

<https://cozystack.ru/docs/v1.6/cozystack-api/rest/>

Cozystack REST API Reference

Last modified 2026-07-22: [[docs] Update managed apps reference for v1.6.0 (96e013f)](<https://github.com/katamarina-ru/cozystack-website//commit/96e013fad266e0cdc43772a869dffbdea6c20c9f>)

Справочник ApplicationDefinition

<https://cozystack.ru/docs/v1.6/cozystack-api/application-definitions/>

Как ресурсы ApplicationDefinition описывают типы приложений и как искать их из клиентского кода

Обзор

ApplicationDefinition (`applicationdefinitions.cozystack.io/v1alpha1`) — это CRD кластерного масштаба, описывающий каждый тип приложения, предоставляемый платформой. Каждое определение объявляет Kind Kubernetes, который используют tenants в агрегированном API (`spec.application.kind`), схему OpenAPI для отображения формы на дашборде и валидации пользовательского ввода (`spec.application.openAPISchema`), а также метаданные дашборда: категорию, иконку и отображаемые имена (`spec.dashboard`).

Агрегированный API-сервер (`cozystack-api`) перечисляет все ApplicationDefinition один раз при запуске и регистрирует соответствующий ресурс под `apps.cozystack.io/v1alpha1`. Набор tenant-ориентированных Kind не меняется пока работает API-сервер — добавление, удаление или переименование ApplicationDefinition вступает в силу только после перезапуска `cozystack-api`.

Специальный контроллер (`applicationdefinition-controller`, поставляемый вместе с Cozystack) следит за ApplicationDefinition и автоматически инициирует этот перезапуск: при любом изменении набора он вычисляет контрольную сумму SHA-256 по отсортированным определениям и записывает её в аннотацию `cozystack.io/config-hash` шаблона пода в Deployment `cozy-system/cozystack-api`, после чего Kubernetes выполняет плавный перезапуск. События дебаунсятся в течение короткого окна, и если контрольная сумма не изменилась — перезапуск пропускается. Операторам не нужно вручную выполнять `kubectl rollout restart`.

Когда пользователь создаёт CR Postgres через дашборд, `kubectl` или Go-клиент, агрегированный слой транслирует его в Flux `HelmRelease`, использующий чарт, указанный в определении.

Соглашение об именовании

ApplicationDefinition использует два независимых стиля именования. Каждое определение задаёт их явно, и связь между ними не может быть выведена никаким строковым преобразованием :

| Поле | Стиль | Пример (HTTP-кэш) | Пример (диск VM) | Пример (TCP-балансировщик) |
|----------------------------|---------------------|-------------------------|----------------------|----------------------------|
| <code>metadata.name</code> | строчные-с-дефисами | <code>http-cache</code> | <code>vm-disk</code> | <code>tcp-balancer</code> |

| | | | | |
|--|---------------------------------|------------|---------|--------------|
| <code>spec.application.kind</code> | CamelCase, акронимы сохраняются | HTTPCache | VMDisk | TCPBalancer |
| <code>spec.application.singular</code> | строчные, без дефисов | httpcache | vmdisk | tcpbalancer |
| <code>spec.application.plural</code> | строчные, без дефисов | httpcaches | vmdisks | tcpbalancers |

Обратите внимание, что `metadata.name` не является функцией от `spec.application.kind`. Расположение дефисов (`tcp-balancer`, `vm-disk`, `http-cache`) и их отсутствие в `singular/plural` (`tcpbalancer`, `vmdisk`, `httpcache`) — это соглашения, выбранные для каждого приложения отдельно, а не результат работы общего алгоритма. `strings.ToLower(kind)` даёт `httpcache`, что совпадает с `spec.application.singular`, но не с `metadata.name`. Прямой поиск по Kind в нижнем регистре поэтому завершится ошибкой:

Ресурс агрегированного API использует plural в нижнем регистре:

```
$ kubectl get httpcaches --namespace tenant-demo
```

```
NAME READY AGE VERSION
```

```
frontend True 2m 1.2.0
```

Но ApplicationDefinition, лежащий в его основе, хранится под другим именем:

```
$ kubectl get applicationdefinition httpcache
```

```
Error from server (NotFound): applicationdefinitions.cozystack.io "httpcache" not found
```

```
$ kubectl get applicationdefinition http-cache
```

```
NAME AGE
```

```
http-cache 14d
```

Акронимы делают это особенно наглядным: `TCPBalancer`, `HTTPCache` и `VMDisk` — все теряют заглавные буквы в имени агрегированного ресурса (`tcpbalancers`, `httpcaches`, `vmdisks`), но сохраняют дефисы в имени CRD (`tcp-balancer`, `http-cache`, `vm-disk`).

Рекомендуемый шаблон поиска

Клиентский код, которому нужно разрешить Kind Cozystack — например дашборд, получающий `HTTPCache` из метки `HelmRelease` и желающий отрендерить соответствующую форму — должен ****получить список всех `ApplicationDefinition` и фильтровать по `spec.application.kind`**** вместо попытки прямого `Get` по Kind в нижнем регистре. Набор определений невелик (порядка десятков элементов) и изменяется редко, поэтому такой подход дешёв и надёжен. Возвращайте весь найденный объект, чтобы вызывающий код мог читать `spec.application.openAPISchema`, `spec.dashboard` или любое другое поле без второго обращения к API.

Перед использованием названий группы и ресурса ниже убедитесь в их актуальности для вашего кластера:

```
$ kubectl api-resources | grep applicationdefinition
applicationdefinitions cozystack.io/v1alpha1 false ApplicationDefinition
```

Строка должна содержать `applicationdefinitions` в столбце `NAME`, `cozystack.io/v1alpha1` в столбце `APIVERSION`, `false` в столбце `NAMESPACED` (ресурс имеет кластерный масштаб) и `ApplicationDefinition` в столбце `KIND`. Если группа отличается в вашем кластере, скорректируйте `GroupVersionResource` в примере соответственно.

```
import (
    "context"
    "fmt"

    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
    "k8s.io/apimachinery/pkg/apis/meta/v1/unstructured"
    "k8s.io/apimachinery/pkg/runtime/schema"
    "k8s.io/client-go/dynamic"
)

// findByKind возвращает ApplicationDefinition, чей spec.application.kind
// совпадает с запрошенным kind, или ошибку, если совпадение не найдено. Вызывающий
// получает полный объект, поэтому поля вроде spec.application.openAPISchema
// доступны без второго обращения к API.
func findByKind(ctx context.Context, client dynamic.Interface, kind string) (*unstructured.Unstructured,
error) {
    if kind == "" {
        return nil, fmt.Errorf("kind must not be empty")
    }

    gvr := schema.GroupVersionResource{
        Group: "cozystack.io",
        Version: "v1alpha1",
        Resource: "applicationdefinitions",
    }

    // Набор ApplicationDefinition на кластере Cozystack невелик
    // (порядка десятков), поэтому одного непагинированного List достаточно.
    // Если вы адаптируете этот хелпер для большего каталога, задайте ListOptions.Limit
    // и переходите по continue-токену, чтобы избежать неявного усечения.
```

```

list, err := client.Resource(gvr).List(ctx, metav1.ListOptions{})
if err != nil {
return nil, fmt.Errorf("list %s/%s/%s: %w",
gvr.Group, gvr.Version, gvr.Resource, err)
}
for i := range list.Items {
specKind, found, err := unstructured.NestedString(
list.Items[i].Object, "spec", "application", "kind")
if err != nil || !found {
// Пропускаем определения с отсутствующим или нестроковым kind, чтобы
// итерация не совпала с некорректной записью.
continue
}
if specKind == kind {
return &list.Items[i], nil
}
}
// Включаем GVR в ошибку, чтобы неверная группа (например после переименования
// CRD) отличалась от настоящего "такого kind нет".
return nil, fmt.Errorf("no ApplicationDefinition with spec.application.kind %q found under %s/%s/%s",
kind, gvr.Group, gvr.Version, gvr.Resource)
}

```

Набор `ApplicationDefinition`, предоставляемых через агрегированный API, заморожен на момент запуска `cozystack-api` (см. Обзор), однако базовые CRD можно редактировать в режиме реального времени: администратор может изменить `spec.application.openAPISchema` или `spec.dashboard` в существующем определении или добавить новый Kind — `applicationdefinition-controller` затем инициирует плавный перезапуск `cozystack-api`, чтобы изменение стало доступным через агрегированный API без ручного вмешательства. Насколько агрессивно клиент должен кэшировать — зависит от его собственного времени жизни:

- Короткоживущие процессы (CLI-инструменты, одноразовые скрипты, `serverless`-функции) могут безопасно кэшировать результат `findByKind` на всё время жизни процесса.
- Долгоживущие процессы (дашборды, контроллеры, операторы) должны повторно получать список `ApplicationDefinition` с периодичностью, соответствующей тому, как часто их операторы редактируют схемы — обычно раз в несколько минут достаточно. Определения меняются редко, поэтому `watch` не оправдывает сложности. Новый `ApplicationDefinition` станет доступен через агрегированный API вскоре после создания, как только инициированный контроллером плавный перезапуск `cozystack-api` завершится.

Plural в нижнем регистре (`httpcaches`, `vmdisks`) является корректным именем для
тенант-ориентированных ресурсов под `apps.cozystack.io/v1alpha1`. Только CRD
`applicationdefinitions.cozystack.io` использует форму с дефисами.

См. также

- [Обзор Cozystack API](#) — использование `kubectl`, `Terraform` и Go-клиента для
тенант-ориентированных ресурсов.
- [Go Types](#) — типизированные Go-клиенты для ресурсов `apps.cozystack.io/v1alpha1`.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)



Прочее

Cozystack Руководство по разработке

<https://cozystack.ru/docs/v1.6/development/>

Cozystack Руководство по разработке

Как это работает

Cozystack — платформа на основе операторов. Начальная загрузка и текущее управление осуществляются набором контроллеров, работающих внутри кластера. Высокоуровневый поток выглядит так:

1. Инсталлятор чарта (`packages/core/installer`) применяется через `helm install`. Он разворачивает Deployment `cozystack-operator` в пространстве имён `cozy-system`.

2. `cozystack-operator` запускается и выполняет однократную начальную загрузку:

- Устанавливает CRD Cozystack (`Package`, `PackageSource`) из встроенных манифестов (`internal/crdinstall`).
- Устанавливает компоненты Flux (`source-controller`, `helm-controller`, `source-watcher`) из встроенных манифестов (`internal/fluxinstall`).
- Создаёт начальный OCIRepository (`cozystack-platform`) из значений `platformSourceUrl` и `platformSourceRef`, заданных в инсталляторе.
- Создаёт `PackageSource`, ссылающийся на начальный OCIRepository.

3. Цикл согласования берёт управление. Оператор следит за CRD `PackageSource` и `Package` и преобразует их в объекты Flux `HelmRelease`. Flux затем устанавливает реальные Helm-чарты и управляет ими.

4. Platform chart (`packages/core/platform`) разворачивается как обычный `Package`. Он читает конфигурацию кластера из ресурса `cozystack.cozystack-platform Package` и создаёт шаблоны манифестов пакетов, определяющих, какие системные компоненты должны быть установлены.

Platform chart также создаёт вторичный OCIRepository (`cozystack-packages`), копируя спецификацию из начального OCIRepository. Все `PackageSource` ссылаются на этот вторичный репозиторий. При обновлениях platform chart выполняет миграции как `pre-upgrade` хуки перед созданием или обновлением `HelmRelease` компонентов.

5. FluxCD — движок выполнения, он согласует объекты `HelmRelease`, созданные оператором, загружает артефакты чартов из ресурсов `ExternalArtifact` и применяет их к кластеру.

Полную цепочку согласования (PackageSource -> ArtifactGenerator -> ExternalArtifact -> Package -> HelmRelease -> Pods), разрешение зависимостей, потоки обновления и отката, а также CLI cozyrpg см. в разделе [Ключевые концепции](#).

OCIRepository и поток миграции

Cozystack использует два ресурса OCIRepository для управления обновлениями платформы:

| OCIRepository | Создаётся | Ссылается на |
|--------------------|----------------------------------|--|
| cozystack-platform | cozystack-operator | Настраивается через значения инсталлятора (platformSourceUrl, platformSourceRef) |
| cozystack-packages | Platform chart (repository.yaml) | Копирует спецификацию из cozystack-platform |

Все PackageSource в packages/core/platform/sources/ ссылаются на cozystack-packages.

Выполнение миграций

Миграции выполняются как Helm pre-upgrade хуки в platform chart:

```
# packages/core/platform/templates/migration-hook.yaml
```

```
metadata:
```

```
name: cozystack-migration-hook
```

```
annotations:
```

```
helm.sh/hook: pre-upgrade,pre-install
```

```
helm.sh/hook-weight: "1"
```

Контейнер миграции считывает текущую версию из ConfigMap cozystack-version и выполняет скрипты миграции последовательно от CURRENT_VERSION до TARGET_VERSION - 1. Каждая миграция обновляет ConfigMap при успехе, обеспечивая идемпотентность миграций и возможность возобновления после сбоев.

Зачем два репозитория?

Разделение гарантирует, что:

1. Начальный OCIRepository управляется оператором (через значения инсталлятора).
2. Все PackageSource имеют согласованную ссылку (cozystack-packages), а не указывают напрямую на источник, управляемый оператором.
3. Platform chart может выполнять миграции перед созданием вторичного OCIRepository, гарантируя выполнение миграций до обновления компонентов.

Ключевые бинарные файлы

| Бинарный файл | Источник | Роль |
|----------------------|--------------------------|---|
| cozystack-operator | cmd/cozystack-operator | Начальная загрузка (CRD, Flux, источник платформы), согласование <code>PackageSource</code> и <code>Package</code> , репликация секрета <code>cozystack-values</code> . |
| cozystack-controller | cmd/cozystack-controller | Согласование рабочих нагрузок и <code>ApplicationDefinition</code> , управление дашбордами. |
| cozystack-api | cmd/cozystack-api | Уровень агрегации API Kubernetes для групп API <code>apps.cozystack.io</code> и <code>core.cozystack.io</code> . |
| cozypkg | cmd/cozypkg | CLI-инструмент для управления пакетами — визуализация зависимостей, интерактивная установка и удаление. |

Структура репозитория

Основная структура репозитория [cozystack](#):

```
.
+-- api # Go-типы для CRD Cozystack (Package, PackageSource и т.д.)
+-- cmd # Точки входа для всех бинарных файлов
| +-- cozystack-operator # Основной оператор платформы
| +-- cozystack-controller # Контроллеры рабочих нагрузок и приложений
| +-- cozystack-api # Агрегированный API-сервер
| +-- cozypkg # CLI для управления пакетами
+-- internal # Реализации контроллеров и согласователей
| +-- operator # Согласователи PackageSource и Package
| +-- controller # Контроллеры рабочих нагрузок и ApplicationDefinition
| +-- fluxinstall # Встроенные манифесты Flux и инсталлятор
| +-- crdinstall # Встроенные манифесты CRD и инсталлятор
| +-- cozyvaluesreplicator # Логика репликации секретов
+-- packages # Helm-чарты, организованные по слоям
```

- | +-- core # Начальная загрузка и конфигурация платформы
- | +-- system # Инфраструктурные операторы и upstream-чарты
- | +-- apps # Пользовательские чарты приложений
- | +-- extra # Чарты приложений для конкретных тенантов
- +-- pkg # Общие Go-библиотеки
- +-- dashboards # Дашборды Grafana
- +-- hack # Вспомогательные скрипты для локальной разработки
- +-- docs # Журналы изменений и примечания к релизам

Разработку можно вести локально, изменяя и обновляя файлы в этом репозитории.

Пакеты

core

Core-пакеты отвечают за начальную загрузку и конфигурацию на уровне платформы.

installer

Helm-чарт, разворачивающий Deployment `cozystack-operator`. Он создаёт пространство имён `cozy-system`, ServiceAccount с правами `cluster-admin` и Deployment оператора с флагами, иницилирующими установку CRD и Flux при запуске. Образ оператора и URL источника платформы задаются во время сборки.

platform

Helm-чарт, разворачиваемый как обычный `Package` (не применяется напрямую). Он читает конфигурацию кластера из ресурса `cozystack.cozystack-platform Package` и создаёт шаблоны манифестов согласно указанному [варианту](#) и настройкам компонентов, определяя, какие системные компоненты должны быть установлены.

flux-aio

Компоненты Flux, упакованные для развёртывания оператором.

talos

Конфигурационные ресурсы Talos OS.

Core-пакеты не используют Helm для применения манифестов; они предназначены для использования только как `helm template . | kubectl apply -f -.`

system

System-пакеты настраивают систему для управления и развёртывания пользовательских приложений. Необходимые системные компоненты указываются в конфигурации пакета.

System-пакеты включают два вида компонентов:

- Операторы (например, `postgres-operator`, `kafka-operator`, `redis-operator`): Контроллеры, умеющие управлять полным жизненным циклом конкретного приложения, включая операции второго дня.
- Upstream Helm-чарты для приложений без выделенного оператора (например, `nats`, `ingress-nginx`): Эти чарты размещаются в `system`, чтобы пакеты `apps` и `extra` могли разворачивать их через Flux `HelmRelease` CR, фактически используя FluxCD в качестве оператора.

System-пакеты используют Helm для установки и управляются FluxCD.

apps

These user-facing applications appear in the dashboard and include manifests to be applied to the cluster.

Apps charts serve as a high-level API for users. They define only the parameters that should be exposed and validated through `values.schema.json`, keeping the interface minimal and secure. Apps charts should not contain business logic for deploying the application itself — instead they delegate to an operator or to FluxCD.

В зависимости от наличия выделенного оператора приложения `apps` следуют одному из двух паттернов:

Паттерн на основе оператора

Когда для приложения есть выделенный оператор (например, PostgreSQL, MongoDB, Redis, Kafka), чарт `app` создаёт экземпляры CRD, которыми управляет оператор:

```
packages/system/postgres-operator/ # Helm-чарт оператора
```

```
packages/apps/postgres/ # App-чарт создаёт postgresql.cnpg.io/v1.Cluster CR
```

Оператор обрабатывает все детали развёртывания и операции второго дня (масштабирование, резервное копирование, переключение при сбое). App-чарт просто создаёт соответствующий CRD со значениями, полученными из пользовательского ввода.

Паттерн на основе HelmRelease

Когда для приложения нет выделенного оператора и стандартным методом развёртывания является Helm-чарт, upstream-чарт размещается в `system/`, а app-чарт создаёт `**FluxHelmRelease CR**`, указывающий на него:

```
packages/system/nats/ # Upstream Helm-чарт NATS
```

packages/apps/nats/ # App-чарт создаёт helm.toolkit.fluxcd.io/v2.HelmRelease

В этом случае FluxCD выступает оператором, управляя жизненным циклом Helm-релиза. App-чарт контролирует, какие upstream-значения открываются пользователю, обеспечивая дополнительный уровень безопасности — пользователи не могут обойти валидацию для развёртывания чарта с произвольными значениями.

Другие примеры этого паттерна: `extra/ingress`, `extra/seaweedfs`, `extra/monitoring`.

extra

Similar to `apps` but not shown in the application catalog. They can only be installed as part of a tenant. They are allowed to use by bottom tenants installed in current tenant namespace.

Подробнее о [Системе тенантов](#) читайте на странице основных концепций.

В одном пространстве имён тенанта можно использовать только один тип приложения.

Extra-пакеты следуют тем же двум архитектурным паттернам, что и `apps` (на основе оператора или `HelmRelease`).

Пакеты `apps` и `extra` используют Helm для установки приложения и управляются FluxCD через дашборд.

Package Structure

Every package is a typical Helm chart containing all necessary images and manifests for the platform. We follow an umbrella chart logic to keep upstream charts in the `./charts` directory and override `values.yaml` in the application's root. This structure simplifies upstream chart updates.

```
.
+-- Chart.yaml # Определение Helm-чарта и описание параметров
+-- Makefile # Общие цели для упрощения локальной разработки
+-- charts # Директория для upstream-чартов
+-- images # Директория для Docker-образов
+-- patches # Опциональная директория для патчей upstream-чартов
+-- templates # Дополнительные манифесты для upstream Helm-чарта
+-- templates/dashboard-resourcemap.yaml # Роль для отображения ресурсов k8s в дашборде
+-- values.yaml # Значения переопределения для upstream Helm-чарта
+-- values.schema.json # JSON-схема для валидации входных значений и отрисовки элементов UI в дашборде
```

Используйте `[cozyvalues-gen](https://github.com/cozystack/cozyvalues-gen)` для генерации таблицы параметров в `README.md` и схемы `values.schema.json` на основе аннотаций в `values.yaml`.

Установите бинарный файл `cozyvalues-gen` со [страницы релизов](#) и выполните `make generate` в директории пакета. Помимо схемы и README, эта команда также обновляет `ApplicationDefinition` пакета (`spec.application.openAPISchema` и `spec.dashboard.keysOrder`) через `hack/update-crd.sh`.

Принципы разработки Helm-чартов

Структура пакетов и рабочий процесс разработки в Cozystack основаны на следующих принципах:

Простое обновление upstream-чартов

Оригинальный upstream-чарт должен быть легко обновляемым, переопределяемым и изменяемым. Мы используем паттерн `umbrella`-чарта — upstream-чарты находятся в директории `/charts` и хранятся в оригинальном виде. Кастомизации вносятся через переопределения `values.yaml` и дополнительные `templates/`, а структурные изменения в upstream-чарте применяются через `patches/`. Такое разделение гарантирует простое обновление до новой upstream-версии: выполните `make update`, просмотрите `diff` и при необходимости повторно примените патчи.

Локальные артефакты

Патчи и образы контейнеров хранятся локально и являются частью пакета. Директория `patches/` содержит любые изменения upstream-чарта, а директория `images/` — `Dockerfile` для сборки всех необходимых образов. Это обеспечивает полную воспроизводимость — всё необходимое для сборки и развёртывания пакета самодостаточно внутри репозитория.

В настоящее время не все пакеты собирают свои образы локально — некоторые всё ещё ссылаются на образы, собранные внешне. Мы активно работаем над переходом к полностью локальной сборке образов для достижения полной самодостаточности и воспроизводимости.

Рабочий процесс локальной разработки и тестирования

Каждый пакет должен быть легко обновляемым и тестируемым локально на реальном кластере без использования CI. Стандартные цели `make` (`make image`, `make diff`, `make apply`) обеспечивают быструю обратную связь: сборка образов, сравнение отрендеренных манифестов с живым кластером и применение изменений — всё с рабочей станции разработчика.

Без внешних зависимостей

Пакеты не должны зависеть от внешних ресурсов во время выполнения. Все чарты, образы и патчи включены в репозиторий. Это гарантирует детерминированность сборок и развёртываний, а также отсутствие сбоев из-за недоступности upstream-реестров, удалённых тегов или проблем с сетью.

Как отмечалось выше, полная самодостаточность образов находится в процессе реализации. Некоторые пакеты всё ещё загружают образы из внешних реестров — это известный пробел, который мы планируем устранить по мере возможности.

Разработка

Настройка Buildx

Для сборки образов необходимо установить и настроить плагин `[docker buildx]`(<https://github.com/docker/buildx>).

Вместо встроенного сборщика можно [настроить дополнительные](#), которые могут быть удалёнными или поддерживать несколько архитектур. В этом примере показано, как создать сборщик с драйвером `kubernetes`, позволяющим собирать образы непосредственно в кластере Kubernetes:

```
docker buildx create \
--bootstrap \
--name=buildkit \
--driver=kubernetes \
--driver-opt=namespace=tenant-kvaps,replicas=2 \
--platform=linux/amd64 \
--platform=linux/arm64 \
--use
```

Либо опустите параметры `-driver*`, чтобы настроить среду сборки в локальном Docker-окружении.

Управление пакетами

Каждое приложение включает Makefile для упрощения процесса разработки. Для каждого пакета мы следуем такой логике:

```
make update # Обновить Helm-чарт и версии из upstream-источника
make image # Собрать Docker-образы, используемые в пакете
make show # Показать вывод отрендеренных шаблонов
make diff # Сравнить Helm-релиз с объектами в кластере Kubernetes
make apply # Применить Helm-релиз к кластеру Kubernetes
```

Например, для обновления `cilium`:

```
cd packages/system/cilium # Перейти в директорию приложения
```

```
make update # Загрузить новую версию из upstream
make image # Собрать образ cilium
git diff . # Показать diff с изменёнными манифестами
make diff # Показать diff с применёнными манифестами кластера
make apply # Применить изменённые манифесты к кластеру
kubectl get pod -n cozy-cilium # Проверить корректность работы
git commit -m "Update cilium" # Зафиксировать изменения в ветке
```

Для сборки контейнера cozystack с обновлённым чартом:

```
cd packages/core/installer # Перейти в директорию пакета cozystack
make image-packages # Собрать образ пакетов
make apply # Применить к кластеру
kubectl get pod -n cozy-system # Проверить корректность работы
kubectl get hr -A # Проверить объекты HelmRelease
```

При пересборке образов указывайте переменную окружения `REGISTRY`, указывающую на ваш Docker-реестр.

Не стесняйтесь заглядывать в каждый Makefile, чтобы лучше понять логику.

Тестирование

Платформа включает скрипт `[e2e.sh]`(<https://github.com/cozystack/cozystack/blob/main/hack/e2e.sh>), выполняющий следующие задачи:

- Запускает три виртуальные машины QEMU
- Настраивает Talos Linux
- Устанавливает Cozystack
- Ожидает установки всех HelmRelease
- Выполняет дополнительные проверки работоспособности компонентов

Скрипт `e2e.sh` можно запускать как локально, так и непосредственно в контейнере Kubernetes.

Для запуска тестов в кластере Kubernetes перейдите в директорию `packages/core/testing` и выполните следующие команды:

```
make apply # Создать тестовую песочницу в кластере Kubernetes
make test # Запустить end-to-end тесты в существующей песочнице
make delete # Удалить тестовую песочницу из кластера Kubernetes
```

[!] Для запуска e2e-тестов в кластере Kubernetes узлы должны иметь достаточно свободных ресурсов для создания 3 VM и хранения данных развёртываемых приложений.

Рекомендуется использовать bare-metal узлы родительского кластера Cozystack.

Динамическая среда разработки

Если вы предпочитаете разрабатывать Cozystack в виртуальных машинах вместо изменения существующего кластера, можно использовать ту же песочницу из тестовой среды. Makefile в `packages/core/testing` включает дополнительные опции:

`make exec` # Открывает интерактивную оболочку в контейнере песочницы.

`make login` # Загружает kubecfg во временную директорию и запускает оболочку с окружением песочницы; требуется установленный mirrord.

`make proxy` # Включает SOCKS5 прокси-сервер; требуются mirrord и gost.

Прокси Socks5 можно настроить в браузере для доступа к сервисам кластера, работающего в песочнице. В Firefox есть удобное расширение для переключения прокси:

- [Proxy Toggle](#)

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Cozystack Рoadmap

<https://cozystack.ru/docs/v1.6/roadmap/>

Cozystack Рoadmap

Вы можете ознакомиться с нашей дорожной картой по адресу: [Ссылка](#).

Если у вас есть вопросы и комментарии по поводу плана развития или вам необходима определенная функциональность, пожалуйста, сообщите нам об этом по адресу info@katamarina.ru

Присоединяйтесь к нашим [community-встречам](#).

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)

Контрибуторы

<https://cozystack.ru/docs/v1.6/contributors/>

Люди, которые помогают развивать документацию и проект Cozystack.

Cozystack развивается благодаря людям, которые не только пишут код, но и помогают делать проект понятнее, доступнее и удобнее для новых пользователей.

Артем Старик

Артем помог с русской версией документации Cozystack.

Эти материалы важны для первого знакомства с проектом: они проводят пользователя от подготовки инфраструктуры до установки Talos Linux, Kubernetes, Cozystack и первых tenant'ов. Хорошая документация здесь снижает порог входа и помогает быстрее перейти от интереса к работающему кластеру.

Спасибо за вклад в документацию и за внимание к тому, чтобы Cozystack был понятен русскоязычному сообществу.

Леонид Филиппов

Леонид перевёл на русский язык разделы документации Cozystack о подсистеме хранения (storage) и сетевых возможностях (networking) для версий v1.4 и v1.5.

Эти разделы охватывают ключевые темы эксплуатации платформы: настройку LINSTOR и DRBD, подготовку дисков, шифрование хранилища, сетевую архитектуру на основе Cilium и Kube-OVN, Gateway API, балансировку нагрузки и публикацию сервисов. Их перевод делает эксплуатацию Cozystack доступнее для русскоязычных инженеров.

Фуад Алекперов

Фуад перевёл на русский язык разделы документации Cozystack об управляемых приложениях (managed applications) и виртуализации (virtualization) для версии v1.5.

В разделе приложений переведены страницы PostgreSQL, MongoDB, Redis, RabbitMQ, NATS, Qdrant, OpenBAO и Tenant, а в разделе виртуализации — работа с виртуальными машинами, дисками и образами, ресурсы, проброс GPU, резервное копирование, миграция из Proxmox, запуск Windows и MikroTik RouterOS. Их перевод делает эксплуатацию Cozystack доступнее для русскоязычных инженеров.

Команда Katamarina

Сотрудники компании содействовали как переводу документации Cozystack на русский, так и обеспечению работы сайта <<https://cozystack.ru>> и развитию экосистемы проекта в целом.

Компания Katamarina первой добилась включения в реестр российского ПО проекта на основе Cozystack и работает над сертификацией решения по требованиям ФСТЭК.

Благодарим компанию Katamarina за вклад в развитие проекта Cozystack на территории РФ и за ее пределами.

Last modified 2026-07-23: [translate 1.6 \(c21c9f4\)](#)
